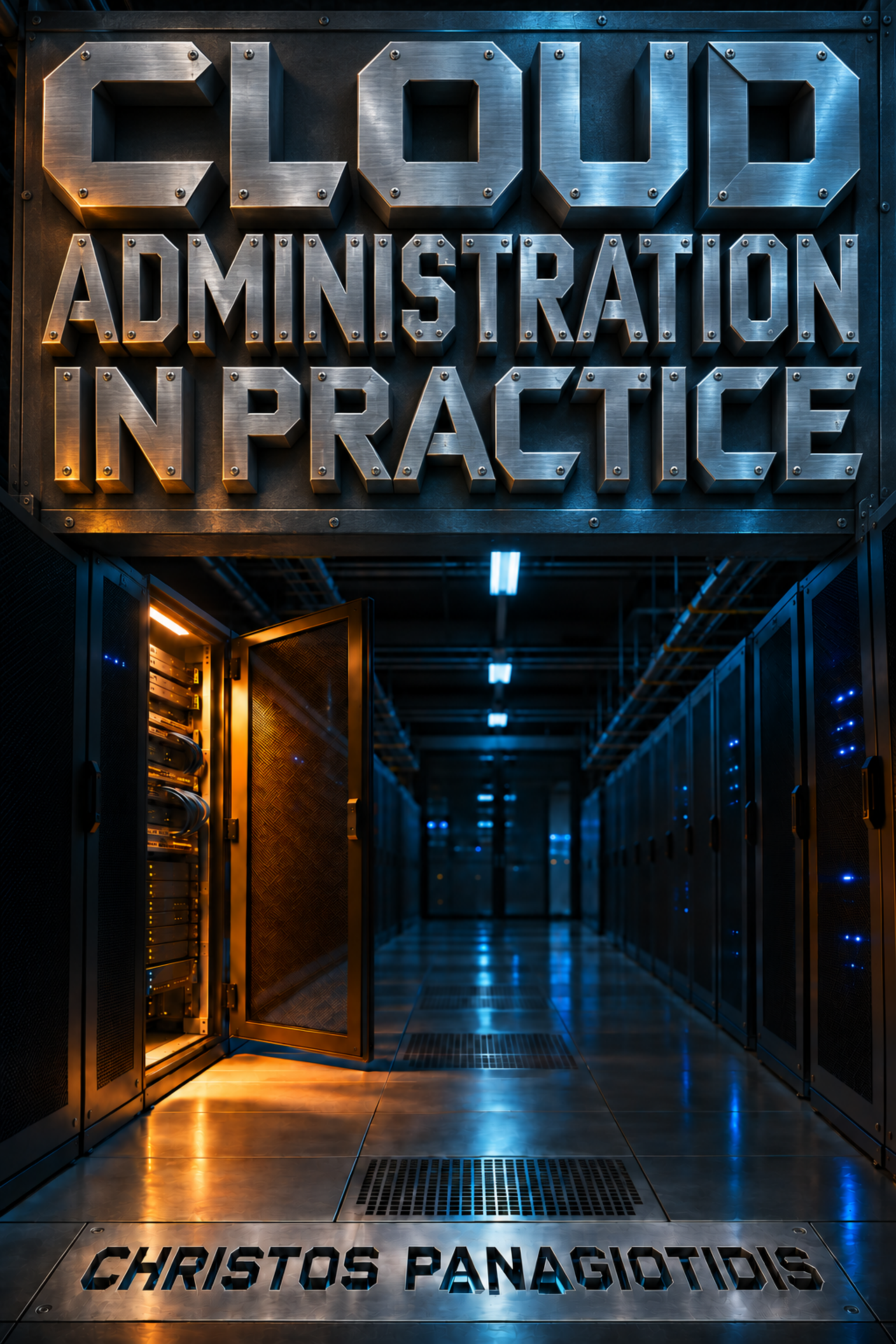
CLOUD ADMINISTRATION IN PRACTICE



CHRISTOS PANAGIOTIDIS

About the Author

Christos "Chris" Panagiotidis

Christos "Chris" Panagiotidis is a Germany-based Azure Cloud Engineer, Microsoft Certified Trainer, cloud coach, mentor, and hands-on lab builder. His work focuses on Azure infrastructure, networking, automation, and practical education.

Chris writes and builds learning environments that help people develop repeatable cloud systems. His areas of interest include Azure infrastructure and networking, Terraform and infrastructure as code, cloud coaching, and lab development. He emphasizes systems that can be inspected, tested, documented, improved, and rebuilt.

His approach to learning is grounded in practice. A useful lab connects an architectural idea to its configuration, commands, verification steps, and cleanup. Understanding why a component exists matters alongside knowing how to deploy it.

That approach shapes this book's treatment of Azure administration. The explanations connect identity, resource organization, network boundaries, storage, compute, monitoring, and recovery. ARM and Bicep examples make deployment intent visible, while guided exercises help readers examine the results of their decisions.

Chris encourages learners to keep examples concrete and their operating procedures understandable. Naming, permissions, dependencies, outputs, and cleanup belong to the same learning process. The aim is to build confidence through work that can be explained and checked.

Author links

Biography and website

GitHub profile

LinkedIn

Table of Contents

About the Author 2
Introduction: A Clear Direction Through the Cloud 10
Chapter 1: Entering the Azure Administrator's World 15
1.1 The Work Behind the Cloud 15
1.2 Regions, Zones, and Resilience 18
1.3 Resources and the Azure Management Hierarchy 21
1.4 Finding a Way Around the Portal and Cloud Shell 25
1.5 Preparing a Practice Environment with CLI and PowerShell 28
Chapter 2: Building the Identity Foundation 35
2.1 Understanding Microsoft Entra Tenants 35
2.2 Creating and Maintaining User Accounts 39
2.3 Organizing Groups and Membership 42
2.4 Managing User Licenses 45
2.5 Working with External Users 48
Chapter 3: Giving Access with Purpose 54
3.1 Authentication and Authorization 54
3.2 Enabling Self-Service Password Reset 57
3.3 Choosing Built-In Azure Roles 60
3.4 Assigning Permissions Through Scope and Inheritance 63
3.5 Explaining and Troubleshooting Effective Access 66
Chapter 4: Giving Resources a Clear Home 73
4.1 Organizing Management Groups 73
4.2 Administering Subscriptions 77
4.3 Managing Resource Groups and Their Lifecycle 80
4.4 Making Naming and Tags Useful 83
4.5 Protecting Resources with Locks 86
Chapter 5: Governing Change and Controlling Spending 93
5.1 Defining Standards with Azure Policy 93
5.2 Combining Policies and Reading Compliance Results 96
5.3 Handling Exemptions and Remediation 100
5.4 Understanding Costs, Budgets, and Alerts 103
5.5 Turning Advisor Recommendations into Decisions 106
Chapter 6: Building the Paths Between Resources 113
6.1 Understanding Address Spaces and CIDR 113
6.2 Creating Subnets and Managing IP Addresses 116

6.3 Connecting Virtual Networks Through Peering 119
6.4 Directing Traffic with Routes 122
6.5 Tracing Connectivity Problems 125
Chapter 7: Controlling What Can Connect 132
7.1 Using Network and Application Security Groups 132
7.2 Interpreting Effective Security Rules 135
7.3 Providing Administrative Access with Bastion 138
7.4 Connecting Services Through Service Endpoints 141
7.5 Using Private Endpoints 144
Chapter 8: Making Services Reachable and Reliable 152
8.1 Publishing Names with Azure DNS 152
8.2 Resolving Names Privately 155
8.3 Designing Internal and Public Load Balancing 158
8.4 Configuring Probes and Traffic Rules 161
8.5 Diagnosing DNS and Load-Balancing Failures 164
Chapter 9: Choosing a Home for Cloud Data 171
9.1 Selecting and Creating Storage Accounts 171
9.2 Choosing Redundancy 174
9.3 Understanding Object Replication 178
9.4 Protecting Data Through Encryption 181
9.5 Moving Data with Storage Explorer and AzCopy 184
Chapter 10: Managing the Life of Blob Data 191
10.1 Organizing Containers and Blobs 191
10.2 Selecting Access Tiers 194
10.3 Automating Lifecycle Decisions 197
10.4 Recovering Earlier Blob Versions 200
10.5 Using Soft Delete for Blobs and Containers 204
Chapter 11: Sharing Files and Governing Storage Access 211
11.1 Creating and Operating File Shares 211
11.2 Applying Identity-Based File Access 214
11.3 Protecting Shares with Snapshots and Soft Delete 217
11.4 Managing Keys, SAS, and Stored Access Policies 220
11.5 Restricting Storage Network Access 223
Chapter 12: Bringing Virtual Machines into Service 231
12.1 Selecting and Deploying a Virtual Machine 231
12.2 Managing Disks 234
12.3 Choosing and Changing VM Sizes 237

12.4 Configuring Encryption at Host 241
12.5 Moving VMs Between Resource Groups, Subscriptions, and Regions 244
Chapter 13: Keeping Compute Available 252
13.1 Understanding Availability Sets 252
13.2 Distributing Workloads Across Zones 255
13.3 Operating Virtual Machine Scale Sets 258
13.4 Matching Capacity to Demand 262
13.5 Diagnosing Availability and Scaling Problems 266
Chapter 14: Making Deployments Repeatable 273
14.1 Reading ARM Templates 273
14.2 Understanding Bicep Files 276
14.3 Changing Parameters and Resource Definitions 279
14.4 Validating and Running Deployments 283
14.5 Exporting Templates and Converting ARM to Bicep 285
Chapter 15: Running Containers with Confidence 292
15.1 Understanding Images, Containers, and Hosting Choices 292
15.2 Managing Azure Container Registry 295
15.3 Operating Container Instances 298
15.4 Deploying Container Apps 301
15.5 Managing Container Resources and Scaling 304
Chapter 16: Hosting Applications with App Service 311
16.1 Creating Plans and Applications 311
16.2 Scaling Application Capacity 314
16.3 Configuring Domains, Certificates, and TLS 317
16.4 Managing Application Networking 320
16.5 Using Deployment Slots and Backups 323
Chapter 17: Seeing What the Environment Is Doing 330
17.1 Interpreting Azure Monitor Metrics 330
17.2 Collecting Logs and Asking Questions with KQL 333
17.3 Building Alerts, Action Groups, and Processing Rules 336
17.4 Using Insights for Compute, Storage, and Networks 339
17.5 Investigating with Network Watcher and Connection Monitor 342
Chapter 18: Recovering When Something Fails 349
18.1 Choosing Recovery Services Vaults and Backup Vaults 349
18.2 Creating Backup Policies 352
18.3 Performing and Verifying Restores 355
18.4 Planning Site Recovery and Regional Failover 358

18.5 Interpreting Recovery Reports and Alerts 361
Chapter 19: Operating a Complete Azure Environment 368
19.1 Turning Requirements into a Deployment Plan 368
19.2 Establishing Identity and Governance 371
19.3 Connecting Storage and Compute 373
19.4 Investigating Deliberately Introduced Faults 376
19.5 Demonstrating Recovery and Completing Cleanup 378
Chapter 20: Turning Understanding into Exam Readiness 381
20.1 Reviewing Coverage Across the Five Domains 381
20.2 Reading Technical Questions Carefully 382
20.3 Completing an Original Practice Assessment 384
20.4 Learning from Answer Explanations 396
20.5 Building a Focused Revision Plan 411
Conclusion: Beyond the First Deployment 414
Appendix A: AZ-104 Objective Coverage Map 417
Appendix B: Azure CLI and PowerShell Reference 426
Appendix C: ARM and Bicep Reading Reference 429
Appendix D: KQL and Troubleshooting Reference 431
Appendix E: Lab Preparation and Cleanup Reference 433
Glossary 435
Appendix F: Chapter Review Answers 441
Subject Index 460
Additional Questions and Answers 462

November 2026

Edition information

This independent educational book follows the Microsoft Azure Administrator learning objectives effective April 17, 2026. Azure services, interfaces, licensing arrangements, and certification requirements change. The source references and objective map identify the baseline used for this edition.

Microsoft, Azure, Microsoft Entra, and other product names belong to their respective owners. This publication is not an official Microsoft publication. Its assessment questions are original teaching material.

Commands and lab procedures are supplied for an isolated practice subscription. The accompanying verification record distinguishes documentation review and static checks from execution in Azure. A reviewed command is not evidence that a particular subscription, permission assignment, region, or service tier supports the operation.

Preface

An administrator's work begins with a requirement and ends with evidence. A resource may exist, but that does not prove that the intended person can use it. A backup job may finish, but that does not show that an application can be restored. A deployment may return a success message while its network path remains unusable. Learning to ask what a result proves is part of learning Azure.

This book develops that habit alongside the technical subjects. It introduces the services required for Azure administration, explains how they relate, and shows how to examine their behavior. Identity, network connectivity, storage, compute, monitoring, and recovery appear as parts of one operating environment. Each remains a distinct discipline with its own controls and evidence.

The intended reader already understands the basic purpose of an operating system, a server, and a network. Previous Azure experience is not required. A reader who has administered local servers will recognize many familiar concerns, including access, availability, configuration, and recovery. Azure changes where those concerns are addressed and how responsibility is divided. Familiar language occasionally describes an unfamiliar boundary, so important terms receive explicit definitions.

Preparation for AZ-104 gives the book a clear scope. The objective map connects that scope to the teaching sections, exercises, and review questions. The explanation does not stop at memorizing product names. A useful answer must fit the permissions, location, service capability, and operational requirement described in a problem. Those same constraints determine whether an administrative change succeeds outside an exam.

The chapters follow a learning sequence. Identity and governance establish the environment's authority and organization. Networking establishes the paths used by later services. Storage and compute make those paths useful. Monitoring and recovery show how the environment behaves and how it can be restored. An integrated project then requires several decisions to be made together.

Examples use small practice environments so that the relationship between an action and its result stays visible. Their resource names are teaching conventions, not a universal naming standard. Their scope is intentionally limited. An organization may need additional controls, approvals, integrations, and service designs. The principles in the examples help explain those additional requirements rather than replacing them.

The manuscript grew from the BookVol2 writing framework. Its connected progression, careful revision, and accessible language remain. Technical teaching also requires exact syntax, quantities, diagrams, comparisons, and repeatable procedures. Those elements are integrated where they make a decision or operation easier to understand.

How to use this book

Read for relationships

On a first pass, read the chapters in order. Later chapters assume that terms such as tenant, subscription, scope, subnet, and identity have already been explained. The objective map also supports targeted revision, but it is most useful after those relationships are familiar. A reader returning to one service should check the section's prerequisites before beginning its lab.

Each chapter begins with learning objectives. These describe what the reader should be able to explain or perform after study. Five teaching sections then develop the subject through definitions, administrative decisions, worked examples, and failure analysis. A diagram highlights a relationship that can otherwise disappear inside a long paragraph. Comparisons organize choices that share a useful set of attributes.

The guided lab applies the chapter's central idea. Its verification steps matter as much as its deployment steps. An expected result describes what the operation should demonstrate; it is not a fabricated transcript of a test run. When actual output differs, compare the resource context, permissions, current configuration, and command version before repeating the action.

Review questions follow the chapter. Answer them before reading their explanations in Appendix F. Record the reasoning, including any assumption that makes one choice appropriate. A correct letter without a sound explanation is weak evidence of understanding. An incorrect answer with a clearly identified misunderstanding is useful evidence for revision.

Work in a controlled practice environment

Use an Azure subscription intended for learning. A company production subscription may contain policies, network restrictions, and access boundaries that are appropriate for the organization but unsuitable for unrestricted experimentation. The labs identify required capabilities without assuming that a learner has authority over every part of a tenant.

Check the active subscription and tenant before changing resources. Name practice resource groups consistently and record their purpose. Where a globally unique name is required, supply a suitable suffix rather than copying an example name unchanged. A resource group provides a useful cleanup boundary, although directory objects, role assignments at other scopes, and certain retained data require separate attention.

Before each lab, read its cost and cleanup notes. Free credits and free service tiers have conditions. A stopped virtual machine can leave billable disks or other resources behind. A budget sends information and alerts; it does not automatically enforce a spending ceiling. Practice designs should be small, short-lived, and easy to inventory.

The steps assume Azure public cloud. Select a region that supports the required services and is available to the subscription. Region names in examples are illustrative parameters. Service availability, subscription restrictions, capacity, and policy can affect a deployment even when its syntax is valid.

Recognize command conventions

Code blocks state the intended language. Bash examples use Azure CLI, while PowerShell examples use the relevant PowerShell modules. The continuation character and variable syntax differ between these shells. Copying a command into the wrong shell can fail before Azure receives a request.

Values enclosed in angle brackets describe information to replace, such as a subscription identifier. They are not literal Azure identifiers. Examples use descriptive variables so that the target of an operation can be checked before execution. A command that reads configuration is separated from one that changes it.

Resource identifiers and output properties deserve careful reading. A display name can be familiar while an identifier points to a different object. Many commands return structured data. Select the fields that answer the current question rather than treating a large JSON response as confirmation of everything about the resource.

Keep a learning record

A short practice record should include the requirement, the action, the expected result, and the observed result. Add the resource scope and any important prerequisite. When a task fails, retain the meaningful error category and a correlation identifier when one is available. Remove credentials, tokens, personal information, and other sensitive values before sharing a record.

The record becomes more useful when it explains a correction. Writing “fixed permissions” conceals the distinction between a missing role, a role assigned at the wrong scope, a different signed-in identity, and a data-plane operation requiring another permission. Name the cause that the evidence supports.

Chapters 19 and 20 use this record in different ways. The integrated project tests whether several services can be operated together. The assessment tests whether a requirement can be interpreted and connected to an appropriate choice. Both expose gaps that rereading alone can hide.

How the labs were checked

Both Bicep templates compiled locally. The VM’s Blob probe passed eight offline tests. The Azure labs still need live validation in a practice subscription.

Source references appear with each chapter and in the supporting editorial record. For changeable settings, consult the linked Microsoft documentation when performing the task. Learn the reason behind a control as well as its current location in an interface. A menu can move while the access boundary it represents remains important.

Introduction: A Clear Direction Through the Cloud

An Azure environment can contain many resources and still be difficult to administer. A virtual machine has a size, an operating system, disks, and network interfaces. Those interfaces belong to networks whose rules determine which traffic can pass. The machine's administrators receive permissions through identities and role assignments. Its useful operation depends on services and boundaries that are not visible on a single overview page.

Understanding those relationships makes the environment easier to reason about. When a person cannot reach an application, the cause may lie in name resolution, routing, network filtering, application configuration, or identity. Each possibility suggests different evidence. An administrator who follows the relationship can narrow the problem. An administrator who changes settings at random can accidentally remove a necessary control without addressing the cause.

This book follows the first approach. It develops an organized understanding of Azure and applies that understanding through small, observable tasks. The aim is confidence grounded in evidence. A successful exercise should leave the reader able to explain what changed, why that change was appropriate, and how its result was verified.

The work behind an administrative decision

Consider a requirement to give a team access to files. The request sounds simple, but several questions shape the solution. The team might need a shared file system or access to objects through an application. Members might belong to the same directory or include external collaborators. Access might be required only from an office network, from managed devices, or from an application identity. The data might need recovery after accidental deletion.

These questions do not make the request unnecessarily complicated. They reveal different requirements hidden inside the word “access.” The service choice, identity mechanism, network exposure, and recovery method must fit together. Configuring one of them correctly does not make the others correct automatically.

The same pattern appears in other tasks. Increasing compute capacity requires an understanding of both demand and the workload's scaling behavior. Restricting a resource requires identifying the relevant management or data operation. Creating an alert requires deciding which condition deserves attention and what response should follow. Administrative work combines configuration with interpretation.

A useful discipline begins by rewriting a request as an observable outcome. Instead of “improve storage security,” state which identity should perform which operation, through which network path, and under which conditions. Instead of “enable backup,” state what data should be recoverable, how much loss is acceptable, and how recovery will be demonstrated. A measurable outcome provides a basis for testing the change.

The first result is not always the final answer. A newly assigned permission can take time to become effective. A client can continue using an old token. A DNS cache can retain an earlier answer. These

possibilities do not justify waiting indefinitely. They explain why an administrator checks timing and state while keeping the original requirement in view.

A service catalog is only the beginning

Product names provide a useful vocabulary, but their value depends on the distinctions they express. A resource group is an organizational and management container. A virtual network defines a network address space and connectivity context. A Microsoft Entra tenant provides an identity directory. These are different kinds of boundary, even when one deployment uses all of them.

Treating every boundary as interchangeable creates predictable errors. Moving resources into another resource group does not move them physically to another region. Giving a person a directory role does not necessarily grant the required Azure resource permission. Allowing a network path does not establish an authenticated identity. Later chapters examine these distinctions in detail.

Service features also have boundaries. A replication mechanism can preserve availability while replicating an unwanted change. A lock can protect a management operation without preventing every data operation. A private endpoint can provide private connectivity while the service's public network settings still require deliberate configuration. The useful question is what a feature actually controls.

An administrator should therefore be able to explain a feature in an operational sentence. The sentence identifies an actor, an action, a resource, and a condition. “The role allows this identity to read this type of resource at this scope” is more useful than “the role provides access.” Precision makes the next verification step easier to choose.

This approach does not require memorizing every product limitation. It requires recognizing when a limitation matters and knowing which documentation or observation can settle it. A region-specific deployment issue calls for checking regional availability and subscription capacity. A permission issue calls for inspecting the effective identity and the operation being attempted. Different uncertainties require different evidence.

Foundations that carry through the book

Several ideas recur throughout the chapters. Scope determines where a setting applies. Identity establishes who or what is acting. A dependency explains why one component needs another. State describes the current configuration or condition. Evidence shows whether the expected outcome occurred.

Scope deserves particular attention because Azure contains nested management boundaries. A setting applied high in a hierarchy can affect resources below it, depending on that setting's inheritance behavior. A setting applied to one resource may leave a neighboring resource unchanged. Reading the scope is often the quickest way to understand an unexpected result.

Identity also includes more than people. Applications and Azure resources can act through identities. A credential proves an identity in a particular authentication flow, while an authorization decision determines what the authenticated identity may do. An administrator who combines these concepts into one vague idea of “login” can miss the reason an authenticated request is denied.

Dependencies explain why a deployment's parts cannot be treated as a flat list. A virtual machine needs supporting resources. An application may need a reachable storage endpoint and the ability to resolve its name. A diagnostic rule requires an appropriate destination. The dependency is meaningful even when a deployment tool creates the components automatically.

State changes over time. A resource can be configured, provisioning, healthy, degraded, or absent. A policy assignment can exist while existing resources still require evaluation or remediation. A backup policy can be assigned before a usable recovery point exists. Reading a control's presence is different from observing its completed effect.

Evidence connects these ideas to an outcome. A resource identifier confirms which object was inspected. A role assignment records a permission relationship. A successful data request demonstrates a specific access path. A restoration test demonstrates a recovery result under the tested conditions. No single item proves every property of an environment.

Learning through a deliberate sequence

The book begins with the operating context. The reader learns to identify the active subscription, locate resources, understand the management hierarchy, and work with the portal and command-line tools. These habits reduce confusion in every later exercise.

Identity and governance establish how people and resources receive authority and how an environment stays organized. The discussion connects access decisions with resource scope, policy, and spending. A well-organized environment makes its purpose and ownership easier to discover. Organization also helps an administrator identify the likely effect of a change before making it.

Networking follows because storage and compute depend on communication. Address spaces, routes, name resolution, and filtering describe different parts of that communication. Learning those parts before deploying a workload gives later errors a place in the reader's understanding. A failed connection becomes a path to investigate.

The treatment of data and workloads then applies those foundations. Storage decisions affect durability, access, and recovery. Compute decisions affect capacity, availability, and responsibility for an operating system or application platform. Repeatable deployment makes the chosen configuration explicit enough to inspect and reproduce.

Monitoring and recovery complete the operating cycle. Telemetry helps determine whether the environment behaves as intended. Recovery techniques address failures and unwanted changes. Both depend on preparation: logs that were never collected cannot explain an earlier event, and an untested recovery design can conceal missing dependencies.

The final project brings these subjects together through a VM identity, private storage path, monitoring, and content recovery. Its purpose is to expose the joins between familiar components. The final assessment then asks the reader to recognize those relationships in written requirements. The learning sequence moves from identifying boundaries to making decisions across them.

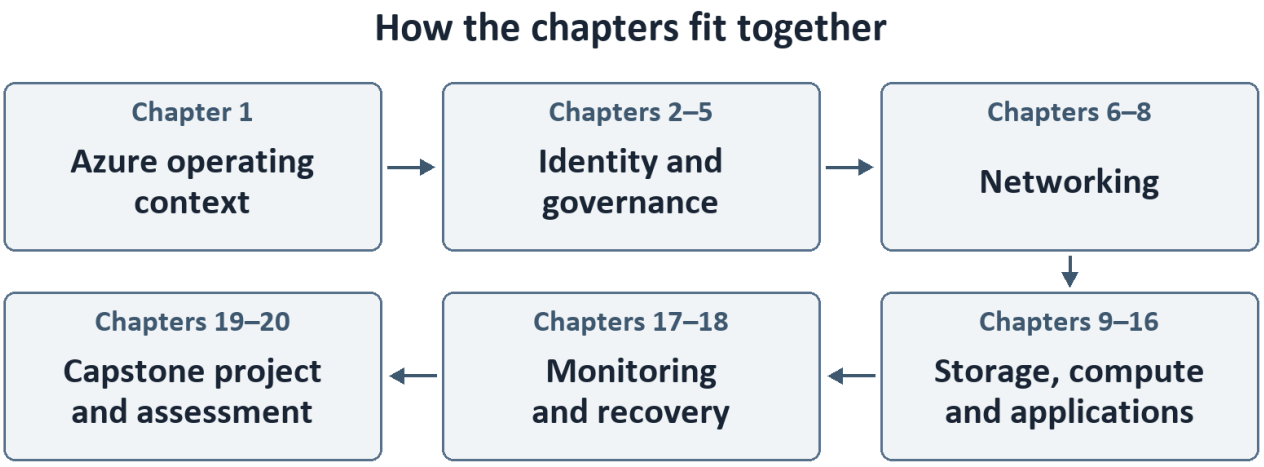


Figure I.1 A roadmap to Chapters 1–20

Practice that produces usable evidence

A lab begins before the first creation command. The reader checks the required permissions, subscription context, service availability, and expected resource costs. These checks are part of the task because they determine whether its instructions apply to the current environment.

During the exercise, keep changes small enough to interpret. If a client cannot connect, inspect its destination and the relevant path before changing several controls. A successful connection after five unrelated changes offers weak evidence about the original cause. A controlled correction provides a better explanation and a more useful practice record.

The verification step should match the stated objective. A storage account appearing in the portal demonstrates that an account exists. It does not demonstrate that the intended identity can read a blob through the intended network route. That requires a suitable request using the appropriate identity and path.

Troubleshooting variations make the distinction visible. A lab may remove a permitted path, use a deliberately insufficient role, or point a test at an incorrect name. The purpose is to recognize a category of failure and restore the intended condition. The variation must remain within the disposable practice environment.

Cleanup is an administrative operation with its own verification. Deleting a resource group can remove its contained resources, but it does not necessarily remove tenant-level users, external assignments, or retained backup information. The lab lists the objects it creates and explains how to confirm their removal. An exercise is complete when the intended temporary state has been removed or consciously retained.

Practice records should remain concise. Record enough information to repeat the reasoning and distinguish the environment from another one. A copied screen of an entire portal page can contain less useful information than the exact scope, relevant property, and observed outcome. Avoid collecting credentials or personal data merely to make a record appear thorough.

Preparing for AZ-104 with understanding

AZ-104 provides a defined set of administration skills. This book maps those skills to the chapters so that study can be checked for coverage. The map is a navigation aid. It does not make every paragraph an exam prediction, and it does not imply that a single practice assessment can reproduce the real exam.

Technical questions often turn on one condition. The required scope may be a resource group rather than a subscription. The requirement may concern data access rather than resource creation. The workload may need private name resolution rather than simply a private address. Reading these conditions carefully is part of applying the underlying knowledge.

When reviewing an answer, explain why the alternatives do not meet the requirement. This exercise exposes incomplete distinctions. An alternative might solve a different problem, require an unstated dependency, grant unnecessary access, or operate at the wrong scope. Understanding those limitations strengthens future decisions.

Use mistakes to guide a focused revision cycle. Return to the relevant explanation, reproduce a small example, and answer a different question on the same concept. Repeating a memorized question can demonstrate recall of its wording while leaving the original misunderstanding intact.

The closing assessment offers a snapshot of understanding across the planned domains. Practical confidence also requires operating the services and explaining their behavior. Keep both kinds of evidence. They answer different questions about readiness.

Start each exercise by confirming its identity, scope, resource, and intended outcome before interpreting the result.

Chapter 1: Entering the Azure Administrator's World

An Azure administrator needs a reliable starting point for every task. That starting point includes the acting identity, the subscription, the resource scope, and the intended result. The tools become easier to use once these elements are understood. This chapter establishes them before introducing larger deployments.

Learning objectives

- Describe the administrator's responsibilities across infrastructure and managed services.
- Distinguish regions, availability zones, and recovery across regions.
- Explain how tenants, subscriptions, resource groups, and resources relate.
- Navigate the portal and use Cloud Shell without confusing interface state with resource state.
- Verify CLI and PowerShell context and complete a small resource-group lifecycle.

1.1 The Work Behind the Cloud

Begin with an outcome

Administration is the work of keeping a technical environment suitable for its purpose. In Azure, that purpose might be running an internal application, storing documents, supporting a public website, or providing a development platform. Resources exist to serve those requirements. A collection of successfully deployed resources is useful only when their combined behavior meets the requirement.

The administrator translates an outcome into configuration and then checks the result. A request to make an application available to employees involves more than creating its hosting resource. Employees need an appropriate identity, a permitted network path, a resolvable name, and a functioning application. The administrator may own some of these components and coordinate with other teams for the rest.

A clear task statement separates the desired result from a suggested implementation. “The support team must read diagnostic logs for this application” states an outcome. “Give the support team Owner” proposes an implementation that may grant substantially more authority than the outcome requires. Examining the outcome first makes it possible to choose a narrower, more suitable permission.

This distinction also improves troubleshooting. If the outcome is “an authorized employee can download a particular file,” the final test must involve that employee's permitted access to that file. A successful login to the portal or an administrator's successful download proves a different condition. Good evidence is specific to the requirement.

Understand the responsibility boundary

Cloud services change the division of operational work. With a virtual machine, an organization normally remains responsible for the guest operating system and the application installed on it. The platform supplies the underlying cloud infrastructure. With a managed application platform, more of the

hosting environment is operated by the provider, while application behavior, data, identities, and configuration still require customer decisions.

The service model is a starting point for understanding this division. Infrastructure as a service provides infrastructure building blocks with substantial customer control. Platform as a service provides a managed environment for an application or data workload. Software as a service provides an application that users consume. Real environments often combine these models, so responsibility must be examined per component.

The practical question is which team can change which layer. An administrator cannot repair a provider's physical switch, but can investigate the configuration of a virtual network. A managed web application removes the need to patch its underlying host operating system directly, yet its deployment settings and access restrictions still matter. Responsibility does not disappear; its location changes.

The following comparison identifies common operating questions rather than a complete service contract.

Operating question	Virtual machine workload	Managed application workload
Who maintains the guest operating system?	The customer controls and maintains it	The platform manages the hosting operating system
Who manages application configuration?	The customer	The customer
Who decides who may access the data?	The customer	The customer
What needs verification?	Guest, application, network, and supporting services	Application, service configuration, network integration, and supporting services

An organization's internal ownership is another boundary. A cloud administrator might be able to create a network without being authorized to alter an application's release process. A security team might define policy while an application team implements a compliant configuration. Technical ability and organizational responsibility should be recorded separately.

Distinguish management from use

Creating a resource and using the data inside it are different operations. The management plane handles resource configuration and lifecycle. The data plane handles service-specific operations such as reading a blob or accessing application data. Azure Resource Manager provides the principal management layer for Azure resources, while services expose their own data interfaces.

This distinction explains a common beginner surprise. A person may see a storage account and inspect its properties without being allowed to read its blobs through an identity-based data request. Conversely, an application may read data while having no permission to resize or delete the storage account. Permission should match the plane and operation involved.

The separation also affects troubleshooting evidence. A successful management request shows that the identity can perform that management action at the specified scope. It does not establish that a client can reach the service's data endpoint. A data request may encounter network restrictions, a different credential, or a different authorization mechanism.

When describing a failure, name the attempted action. “Storage access failed” leaves too many possibilities open. “The signed-in user can inspect the account but receives an authorization error when listing blobs with an Entra token” narrows the investigation. The description identifies both the successful and unsuccessful operations without assuming their cause.

Work through a change cycle

A useful change cycle begins with inspection. Identify the existing configuration and the resource that owns it. Determine the smallest change that can satisfy the requirement. Check permissions, dependencies, policy constraints, and the potential effect on other users. Only then apply the change and test the intended outcome.

The cycle does not require a formal document for every small lab action. It requires enough thought to distinguish an experiment from an uncontrolled alteration. In a production environment, the same reasoning supports change records, approvals, maintenance windows, and rollback planning. In a practice environment, it supports learning.

Reversibility deserves explicit attention. A configuration value may be easy to restore, while a data deletion may require a recovery feature that had to be enabled earlier. A resource move may affect resource identifiers or dependent configuration. Before acting, identify which part of the previous state can be restored and which part cannot.

Verification should include more than the absence of an error. A command can complete successfully while addressing the wrong subscription or resource. Check the returned identifier and the relevant property. Then perform the operation that the change was intended to enable. These checks catch both targeting errors and mistaken assumptions about behavior.

Interpret failures as evidence

Failures can occur at different layers. A shell may reject a command before authentication. An authentication flow may fail before Azure evaluates a resource permission. Authorization may deny an otherwise valid request. A policy may reject a permitted deployment. A service may reject an unsupported combination of configuration values.

The first investigation is to locate the layer. Repeatedly granting permissions does not correct malformed shell syntax. Reinstalling a client tool does not correct a policy restriction. Reading an error carefully prevents an administrator from making changes in the wrong part of the system.

The error message may contain an operation name, a resource identifier, a correlation identifier, or a service-specific code. Preserve the details needed to investigate, while removing secrets and personal information. The useful record explains what was attempted, which context was active, and what Azure reported.

An error is not a complete diagnosis. A permission-related response can result from an unexpected identity or scope as well as a missing role. A name-resolution failure can arise from the client configuration or the DNS records. Treat the message as a starting point for a testable explanation.

Make a result understandable to another administrator

A good handover contains the requirement, current configuration, ownership, dependencies, and evidence of operation. It should also identify unresolved limitations. A statement such as “the application is working” is difficult to evaluate without knowing which path was tested and when.

For example, a useful access record states that a designated test identity successfully read a designated blob through an approved endpoint. It names the scope of the relevant permission and the network condition used for the test. It does not claim that every user, operation, and network has been validated.

This precision supports collaboration. Another administrator can reproduce the test or identify the boundary that was not examined. It also prevents a narrow success from being mistaken for complete operational readiness. The goal is a result that can be understood and checked.

The rest of the book builds the technical knowledge needed to produce such results. Its first practical habit is consistent across every service: identify the actor, target, operation, and expected outcome. A tool is most useful when those elements are clear before it is used.

1.2 Regions, Zones, and Resilience

Understand where a resource operates

An Azure region is a geographic deployment area supported by datacenters and network infrastructure. Many resources require a region when they are created. That choice affects service availability, latency, resilience options, and other operating characteristics. It is a design input rather than a convenient label.

The closest region is not automatically the correct region. A workload may have requirements concerning data location, organizational policy, availability features, or a particular service capability. A subscription may also have restrictions or quota conditions. The administrator should check the requirements together rather than selecting a location solely from a map.

Latency is the delay experienced by communication between components. Keeping dependent resources near each other can reduce delay, but the precise result depends on the application and its traffic pattern. A service making many sequential network requests may be more sensitive to latency than one transferring a large batch asynchronously.

Data location also needs careful interpretation. A resource's selected region does not by itself explain every location involved in backup, replication, support operations, or service metadata. For a workload with explicit residency obligations, consult the service's documented behavior and the organization's approved design. A general regional label is not a substitute for that review.

Separate placement from organization

A resource group has a location used for its metadata. The resources inside it can have their own locations. A resource group's name might contain a region abbreviation for readability, but the name

does not force its members into that region. Policy can enforce permitted locations where the relevant resource type supports that control.

This distinction matters when inspecting an existing environment. Seeing a resource group named for one location does not prove where each member runs. Read the resource's actual location property. Some services are global or use location concepts that differ from those of a virtual machine, so interpret the property according to the service.

A resource move between groups is also different from a regional relocation. The former changes a management relationship when the resource type supports it. The latter changes where the workload operates and may require a service-specific process or a new deployment. The words “move the resource” need enough context to distinguish these operations.

In a practice exercise, use a region variable and record the selection. The variable makes the example reusable, but it does not validate that every service or SKU exists there. A SKU identifies a service's particular offering or configuration tier. Availability checks must match the chosen service, region, and subscription.

Understand availability zones

An availability zone represents a physically separated failure boundary within a supported region. Zone infrastructure is designed to separate major dependencies such as power and cooling. Services use zones in different ways. Some resources are placed in one chosen zone; some services distribute their operation across zones through a supported configuration.

A zonal resource remains associated with a particular zone. A zone-redundant service is designed to operate across zones according to its documented implementation. These terms describe different arrangements. Selecting a zone for one virtual machine does not create a second machine or an application failover mechanism.

The workload must be examined end to end. Multiple application instances provide limited protection if they all depend on one unavailable data component. Similarly, distributing compute does not help if clients have no functioning path to the surviving instance. Resilience depends on the dependencies needed to provide the service.

Logical zone numbers should not be treated as a universal map of physical datacenters across unrelated subscriptions. Where cross-subscription zone alignment matters, use supported zone mapping information rather than assuming that a label alone establishes physical alignment. Most introductory labs do not require this coordination, but the distinction prevents misleading diagrams.

Distinguish availability from recovery

Availability concerns whether a service can provide its function when needed. Recovery concerns restoring service or data after disruption. These concerns overlap, but they lead to different controls. A workload might continue operating through the loss of an instance and still need a backup to recover from an accidental data deletion.

Replication copies information between locations or instances according to a service's behavior. It can improve resilience, but it can also copy an unwanted change. A backup or version history can preserve

earlier states under defined retention rules. The administrator must connect the recovery method to the failure it is intended to address.

Two useful planning terms are recovery point objective and recovery time objective. The recovery point objective describes the acceptable amount of data loss measured in time. The recovery time objective describes the acceptable time to restore the required operation. These are requirements to plan and test against, not properties guaranteed by merely enabling a feature.

For example, a requirement to recover yesterday's deleted configuration differs from a requirement to continue serving requests during a zone failure. The first requires access to an earlier valid state. The second requires a functioning alternative while the failure is occurring. A design may need both capabilities.

Consider regional failure separately

A region can contain multiple failure boundaries, but a design confined to that region still has regional dependencies. A regional recovery design examines what is needed to operate elsewhere. It may include data replication, alternate compute capacity, network configuration, identity access, name resolution, and a procedure for directing clients to the recovery environment.

Some Azure regions have a region pair, while others do not. A pairing relationship should not be interpreted as automatic protection for every resource. Service-specific replication and recovery behavior must be configured and understood. An application does not become recoverable in another region simply because its primary location has a paired region.

Regional recovery also creates operational choices. A secondary environment may run continuously, remain partly provisioned, or be created during recovery. These choices affect cost and recovery time. An administrator needs an explicit design rather than an assumption that spare capacity and configuration will be available when required.

Testing reveals missing dependencies. A recovered server can start while its application cannot resolve a required name or obtain a necessary secret. A data copy can exist while the recovery identity lacks access. A useful test exercises the business operation, not only the creation of its individual resources.

Make a proportionate choice

The appropriate resilience design depends on the requirement. A short-lived training resource does not need the same arrangement as a critical production application. Overbuilding every exercise can obscure the concept being taught and increase cost. Underbuilding a production workload can conceal a failure risk until the service is needed most.

Use a small decision record. State the required availability and recovery outcomes, the selected region, the relevant service capabilities, and the remaining single points of failure. Record why a more complex arrangement was unnecessary or which later stage will address it. This makes the decision reviewable without pretending that every risk has been removed.

Requirement	Relevant question	Evidence to seek

Requirement	Relevant question	Evidence to seek
Survive an instance failure	Is another functioning instance available?	A controlled instance-failure test
Survive a zone failure	Are required components distributed appropriately?	Service configuration and a suitable resilience test
Recover deleted information	Is a valid earlier state retained?	A successful restoration of selected data
Recover in another region	Are data, dependencies, and procedures ready there?	An end-to-end regional recovery exercise

Read availability claims carefully

A service-level agreement describes specified service commitments and conditions. It is not a complete statement of an application's availability. The application combines multiple services, configurations, and customer-managed components. Its behavior depends on that combination.

For an exam question or a real design decision, identify the precise requirement before interpreting an availability feature. A reference to a zone does not automatically answer a regional recovery question. A successful replication configuration does not automatically answer a point-in-time recovery question. Read the failure boundary and the required outcome together.

The administrator's immediate task is to make placement deliberate. Record where resources run, understand the failure boundary of each important dependency, and identify how the workload recovers. Later chapters apply these ideas to storage redundancy, virtual-machine availability, backups, and Site Recovery. The foundational distinction remains the same: a location choice establishes a boundary, and a resilience design determines how the workload behaves when that boundary is affected.

1.3 Resources and the Azure Management Hierarchy

Name the boundaries

An Azure resource is a manageable item exposed by a service. A virtual machine, a storage account, and a virtual network are common examples. Each has a resource type and configuration properties. The resource type identifies the kind of item, while the resource identifier locates a particular item in the management structure.

A resource group gathers resources that can be managed together. It belongs to a subscription. A subscription provides a management and billing boundary for its resources and is associated with a Microsoft Entra tenant. Management groups organize subscriptions so that supported governance and access settings can be applied across them.

A tenant is an identity boundary rather than a folder that contains an application's Azure resources. It holds directory objects such as users and groups. Azure subscriptions trust a tenant for identity. A

person can have access to more than one subscription and can participate in more than one tenant through appropriate identity arrangements.

The relationship is easier to understand by following a task. A person authenticates through an identity context. A management request targets a resource in a particular subscription. Azure evaluates the operation against the applicable permissions and controls. The directory identifies the actor; the resource hierarchy establishes the target and relevant scope.

Understand scope and inheritance

Scope is the boundary at which an assignment or operation applies. Common Azure RBAC scopes include a management group, subscription, resource group, and individual resource. An assignment at a broader scope can affect child resources according to the permission system's inheritance rules. A narrower scope reduces the set of resources covered by that assignment.

The hierarchy does not mean that every property inherits in the same way. Azure RBAC, Azure Policy, resource locks, and tags have different behaviors. A tag on a resource group does not automatically appear as the same tag on every contained resource merely because they share a parent. Later governance chapters distinguish these mechanisms.

This is a general reading habit: identify both the scope and the type of setting. “The subscription has the setting” is incomplete if the question concerns a property that must be configured separately on resources. Conversely, inspecting only a resource's direct assignments may miss an applicable inherited role or policy.

In a permission investigation, begin with the operation and its target. Then examine assignments at the relevant scopes. A role on one resource group does not grant the same permission in an unrelated group. A familiar role name is therefore insufficient evidence without its assignment scope and principal.

Read a resource identifier

Resource identifiers make hierarchy concrete. A typical resource-group-scoped identifier contains a subscription segment, a resource-group segment, a provider namespace, a resource type, and a resource name. Child resources add further type and name segments. The exact structure follows the service's resource model.

```
/subscriptions/<subscription-id>/resourceGroups/rg-compass-ch01
/providers/Microsoft.Network/virtualNetworks/vnet-compass-ch01
```

The example is wrapped across two lines for reading; a real identifier is one continuous string. Its subscription segment answers which subscription contains the resource. Its resource-group segment identifies the management grouping. The provider and type explain what service resource is being addressed.

An identifier is more reliable than a display name when a task must distinguish otherwise similar resources. Two subscriptions can contain groups with the same name. A person inspecting a familiar name in the wrong subscription can make a technically valid change to the wrong environment. The full identifier reveals the difference.

Identifiers should be treated as output from Azure or constructed from verified components. Do not guess one from a screenshot if the object can be queried. Copying an identifier also requires checking that it belongs to the intended tenant and subscription context. Precision in the string does not correct an incorrect target choice.

Choose resource-group boundaries deliberately

A useful resource-group boundary often follows a shared lifecycle. Resources created, changed, and removed together can be easier to manage in one group. This arrangement supports clear ownership and cleanup. It is not a requirement that every component of an application belong to one group.

Shared components may need a different lifecycle from the workloads that use them. For example, a shared network can outlive several short-lived practice deployments. Deleting an application group should not unexpectedly remove infrastructure needed by another application. Grouping decisions should therefore follow relationships and ownership rather than screen convenience.

Resource groups also provide useful scopes for permissions and governance. Granting a team a suitable role at its workload's group can be more appropriate than granting the role across the subscription. A lock at a group can affect management operations on its members. Those benefits depend on understanding the resulting scope.

The group's location stores management metadata and is separate from the location of each member resource. A resource group can contain resources located in multiple regions. Whether that is operationally appropriate depends on the workload. The grouping itself does not provide cross-region replication or resilience.

Follow a management request

The portal, Azure CLI, Azure PowerShell, and deployment tools are interfaces through which an administrator can request resource operations. Azure Resource Manager provides the central management layer. The relevant resource provider handles the service's supported resource types and operations.

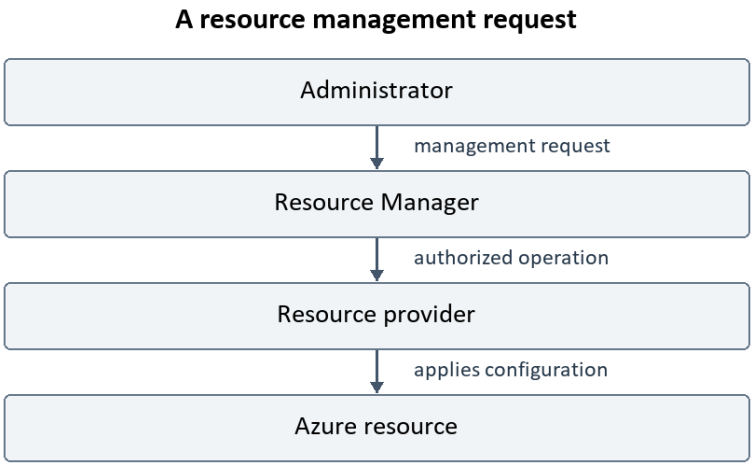


Figure 1.1 A resource management request

The management request reaches the appropriate resource provider after applicable checks. Service data operations use service-specific interfaces and can require different permissions.

A resource provider is the Azure service component that exposes a resource namespace and its operations. Microsoft.Network and Microsoft.Storage are examples of provider namespaces. Some deployments require the relevant provider to be registered in the subscription. Registration is a subscription-level condition, distinct from whether a resource name is valid. It also does not grant a user permission to deploy that provider's resources.

A request can pass one check and fail another. The person may authenticate successfully but lack the required action permission. The action may be authorized but denied by a policy. The request may satisfy policy yet specify a size unavailable in the chosen location. Understanding the sequence helps interpret the resulting error.

Not every administrative tool exposes every new feature at the same time. A particular interface may lag behind a service API or use different terminology. When an option appears unavailable, consult current documentation rather than assuming that the underlying service cannot perform it. Choose a supported interface whose behavior is understood.

Distinguish configuration from relationships

A resource contains properties, but it also participates in relationships. A network interface references a subnet. A virtual machine references disks and interfaces. A role assignment connects a principal, role definition, and scope. These references help determine whether a change is possible and what it affects.

Dependencies should be inspected before deletion or movement. A resource might be used by an object in another group. The management grouping does not necessarily describe every dependency. A successful deletion of one component can leave another component unable to perform its function.

A deployment description makes many dependencies explicit, but operational dependencies can also exist outside that description. An application might contain a service endpoint in configuration, or a

client might rely on a DNS name created by a different team. Documentation should capture the relationships necessary for operation, including those not enforced by a deployment engine.

For a small learning environment, an inventory table can record resource identifier, purpose, owner, and important dependencies. Its value comes from keeping these facts clear. Adding many decorative fields makes the inventory harder to maintain without necessarily improving a decision.

Diagnose a missing resource

Suppose a learner expects to find a virtual network but the portal resource list is empty. An empty list does not prove that the resource was deleted. The selected directory or subscription might differ from the one used during creation. A filter might hide the group or type. The signed-in identity might lack access to the resource.

Begin by checking the current directory and subscription. Remove unintended filters. Query the expected resource group by its known name or identifier. Review deployment history or the activity record if creation may have failed. Each check narrows a specific explanation without making a change to the environment.

Only after establishing the context should a replacement resource be created. Creating another network immediately can conceal the original issue and leave duplicate resources in different subscriptions. A resource that seems missing is an opportunity to practice scope awareness.

The hierarchy gives the administrator a way to navigate both resources and authority. It does not replace service-specific knowledge, but it provides the frame in which that knowledge applies. Before moving into a service's details, identify the tenant relationship, subscription, group, and exact resource. Those facts make the next operation both clearer and easier to verify.

1.4 Finding a Way Around the Portal and Cloud Shell

Use the portal as a view of the environment

The Azure portal presents resources and management functions through a browser interface. Its search, navigation, resource lists, and service pages help an administrator discover configuration. The visible page is a view of the environment under the current identity and selected context. It is not the environment itself.

This distinction matters when a page is stale or a filter is active. Refreshing a page can update its display without changing a resource. Changing a directory or subscription filter can reveal resources that were present all along. A page that omits an option may reflect permissions, service state, or interface design rather than the absence of an underlying capability.

Begin by locating the signed-in account and directory information. Then inspect which subscriptions the current portal view includes. An identity associated with several directories can otherwise lead to a familiar but incorrect environment. Record the intended subscription identifier before starting a lab.

The search bar can locate services and resources, but similar names require careful reading. A service page may list all resources of a type, while a resource page refers to one object. The breadcrumb or

resource header helps establish which level is currently open. The resource's properties provide stronger identification than its position in a recent-items list.

Read resource pages systematically

A resource overview summarizes important properties and status. Other pages expose configuration, access control, monitoring, deployment history, and related operations. The available pages vary by resource type. Learn to identify the purpose of a page rather than memorizing a menu's exact screen position.

When investigating a setting, read its current value before opening an edit action. Some portal workflows apply changes immediately; others collect several choices before a final save or creation action. The difference affects how a learner can inspect configuration without modifying it.

An overview's status field is useful but limited. “Running” for a virtual machine describes a particular state; it does not prove that the guest application responds correctly. A deployment status describes provisioning, while application health may require a separate check. Interpret the label according to the operation it represents.

Portal notifications can show that an asynchronous operation has begun or completed. If a task continues in the background, wait for an appropriate completion signal before testing a dependent operation. Record an error's meaningful details rather than repeatedly clicking the same action without understanding the result.

Keep context visible

Resource names often encode purpose, but they are not a security boundary. Two names may differ by only an environment suffix. Inspect the subscription and group before modifying a similarly named object. Use the resource identifier when the distinction is important.

Favorites and dashboards help navigate frequently used resources, but they can outlive the resource or permission arrangement that made them useful. A shortcut may point to a deleted object or a resource no longer accessible to the current identity. Treat shortcuts as navigation aids rather than proof of current state.

For a practice session, keep a small note containing the selected subscription, lab resource group, and resource names. This reduces reliance on memory when moving between portal pages and command-line tools. It also makes the cleanup inventory more reliable.

Portal screenshots can illustrate a concept, but a screenshot is tied to an interface version and a particular state. This book emphasizes properties, scopes, and verification steps that remain meaningful when a menu moves. When a screen differs, search for the function described and consult the current service documentation.

Understand Cloud Shell's role

Cloud Shell provides an authenticated browser-accessible command environment with a choice of Bash or PowerShell. It includes common Azure administration tools. This can reduce workstation setup for a

learner, but it does not remove the need to understand identity, subscription context, or command syntax.

The shell environment runs separately from the learner's local computer. A file path on a Windows desktop is not automatically available inside Cloud Shell. Uploading, downloading, or using a repository are separate ways to move required files. Commands refer to paths in the environment where they execute.

Session storage behavior depends on the Cloud Shell mode and configuration in use. Persistent storage can involve Azure Storage resources and charges. A temporary session should not be treated as a reliable location for the only copy of a script. Save important work in a deliberate destination and understand what will remain after the session ends.

Cloud Shell is useful for interactive administration. It is not a substitute for a planned unattended automation environment. A long-running production process should have a suitable execution platform, identity, logging, and failure behavior. A browser terminal's convenience does not provide those properties automatically.

Separate shell language from Azure commands

Bash and PowerShell parse input differently. Variables, quoting, pipelines, line continuations, and error handling follow the selected shell's rules. Azure CLI commands can run in more than one shell, but the surrounding syntax still needs to match the shell.

For example, Bash commonly assigns a variable with a name followed directly by an equals sign. PowerShell uses a dollar-prefixed variable and its own expression rules. A multiline command copied with the wrong continuation character can become several unintended commands. Short examples are therefore easier to inspect when learning.

The tool's output format also matters. Azure CLI commonly returns JSON and can format selected output as a table or text. PowerShell cmdlets return objects that can be inspected and selected before formatting. A formatted display is designed for reading; a structured object or JSON value is usually better for subsequent automation.

Do not paste an entire transcript containing prompts or explanatory output into the shell. Copy only the command block intended for execution. A displayed error or returned JSON document is evidence to read, not a command to rerun. Keeping these roles separate prevents confusing secondary failures.

Compare interfaces with a small read operation

A useful first comparison is to inspect the current subscription in the portal and then query it in a shell. The result should identify the same intended subscription, although the display fields differ. If the contexts differ, correct the tool context before creating anything.

The next comparison can inspect one resource group. Read its name, identifier, location, and tags in both interfaces. This demonstrates that the interfaces are different views of the same resource configuration. It also shows which details a compact portal view omits.

An interface difference is not necessarily a contradiction. A portal page may show a friendly label while an API returns a machine-readable value. A value may need a refresh after an operation. Compare the property and target rather than expecting every display string to match exactly.

Handle errors without losing the original question

If Cloud Shell fails to start, separate environment initialization from the Azure task intended afterward. The failure might concern shell service availability, session setup, storage configuration, or organizational restrictions. It does not demonstrate that a target virtual machine or storage account is unavailable.

If the shell starts but a command fails, inspect which component produced the message. A command-not-found error concerns the command environment. An authentication prompt concerns credentials or session state. An Azure authorization response concerns the authenticated operation and scope. These are different investigations.

The portal can provide a second view during troubleshooting, but switching interfaces should answer a question. If the portal can read a resource while a CLI request cannot, compare identities and contexts before concluding that the CLI is defective. An interface is only one part of the request.

The practical goal is fluency across views. The portal supports exploration and visual inspection. Cloud Shell provides a convenient place to issue explicit commands. Both become reliable when the administrator keeps context visible and verifies the returned resource. The following section applies those habits to creating, inspecting, and removing a resource group, with evidence collected at each stage of its lifecycle.

1.5 Preparing a Practice Environment with CLI and PowerShell

Establish a small operating boundary

A practice environment should make its purpose and limits obvious. The first exercise needs a subscription in which the learner is permitted to create and delete a resource group. It does not require a virtual machine, public endpoint, or stored application data. Keeping the first operation small makes context and lifecycle visible without introducing unrelated dependencies.

Use a subscription designated for learning or a scope explicitly made available for the exercise. The ability to sign in does not establish permission to create resources. If resource-group creation is unavailable, an administrator can provide a suitable existing practice group and the permissions needed for later tasks. The learner should not attempt to bypass an organizational restriction.

Choose a naming convention that distinguishes the exercise from other work. This book uses names such as rg-compass-ch01. The name makes the purpose recognizable, while the actual subscription and resource identifier establish the target. A descriptive name reduces confusion but does not replace context verification.

Before creation, check whether the intended group already exists. Reusing an existing group without inspecting it can make cleanup unsafe. If it contains unrelated resources, choose a different practice

name or use the group's documented ownership process. A disposable group should be dedicated to the current exercise.

Inspect Azure CLI context

Azure CLI uses commands beginning with az and organizes operations into command groups. The account command group helps inspect and select subscription context. In an already authenticated Cloud Shell session, begin with a read operation. On a local workstation, sign in using the organization's supported interactive authentication method before inspecting the context.

```
az account show --query "{name:name,id:id,tenantId:tenantId}" --output table
```

The query selects fields relevant to the task. It does not change the subscription. Read the returned identifier and compare it with the intended practice subscription. A familiar display name is useful, but the identifier removes ambiguity where names are similar.

If the current subscription is wrong, select the intended subscription and inspect the result again. The selection changes the default context for subsequent applicable CLI commands. A command can also accept an explicit subscription argument, which is helpful in scripts that should not depend on an inherited default.

```
az account set --subscription "<practice-subscription-id>"
az account show --query "{name:name,id:id,tenantId:tenantId}" --output table
```

Replace the placeholder with an actual accessible subscription identifier. Do not copy the angle brackets as part of the value. If the subscription cannot be selected, confirm access and tenant context. Inventing a different identifier or creating resources elsewhere does not solve the original requirement.

Inspect PowerShell context

Azure PowerShell uses modules that provide cmdlets for Azure operations. The Az modules include commands for resource management and context selection. Identity administration may require different modules, such as Microsoft Graph PowerShell, which later chapters introduce. A command's module determines which service and authentication context it uses.

```
Get-AzContext
Set-AzContext -Subscription "<practice-subscription-id>"
Get-AzContext
```

The context includes the selected subscription, account, tenant, and environment. Inspect these values before a change. If a local session has not authenticated, use the supported Connect-AzAccount flow first. Avoid embedding a personal password in a command or script.

PowerShell objects can be selected and passed through pipelines without first converting them to text. Formatting commands are intended for display near the end of a pipeline. A script that needs a property should obtain the property from the object rather than scrape a formatted table. This distinction becomes useful as inventories grow.

CLI context and Az PowerShell context should not be assumed to be identical merely because both tools run in the same terminal environment. Inspect the context of the tool that will execute the operation. A separate Graph connection is another context to check when directory operations are introduced.

Create only what the exercise needs

The first creation action is a resource group. Specify its name, location, and simple purpose tags. The group's location describes its management metadata, as explained earlier. No application resources are required inside it for this exercise.

```
lab_group="rg-compass-ch01"
lab_region="westeurope"
az group create --name "$lab_group" --location "$lab_region" --tags Purpose=Training Book=Compass
az group show --name "$lab_group" --query "{id:id,location:location,tags:tags}" --output json
```

The region value is an example. Confirm that the chosen location and group creation are allowed by the subscription's policy. If the command is denied, read whether the response concerns authorization, policy, or another condition. A successful creation should be followed by the explicit read operation.

The PowerShell equivalent uses parameters and an object representing tags. Run one creation path during the lab, then use the other tool for inspection if both are available. There is no need to create two groups merely to compare interfaces.

```
$labGroup = "rg-compass-ch01"
$labRegion = "westeurope"
New-AzResourceGroup -Name $labGroup -Location $labRegion -Tag @{Purpose="Training"; Book="Compass"}
Get-AzResourceGroup -Name $labGroup
```

Read the returned evidence

The returned identifier should contain the intended subscription and resource-group name. The location should match the requested allowed location. Tags should contain the selected values. These observations demonstrate the specific properties inspected; they do not establish permissions for every future resource type.

Check the group in the portal as a second view. If it does not appear immediately, verify filters and refresh the resource list. Compare its identifier with the command output. A matching name in another subscription is a different resource group and must not be treated as confirmation.

Now list the resources inside the group. The expected result is an empty collection because the lab created only the group. If resources appear, stop before cleanup and identify their ownership. The safe assumption is not that every object inside a familiar group was created by the current exercise.

```
az resource list --resource-group "$lab_group" --output table
```

The distinction between a group and its members becomes concrete here. The group is a manageable object, yet the resource-list command concerns its contained resources. An empty member list is consistent with a successfully created group.

Practice a controlled mistake

A useful troubleshooting variation is a read request for a deliberately nonexistent group name. This does not change Azure. It demonstrates how a missing target is reported and how that differs from a syntax error or an authorization response.

Before interpreting the error as evidence of absence, keep the context visible. The requested name and current subscription together define the target. The same name might exist elsewhere. A missing resource response is therefore scoped evidence, not a global search result.

Do not turn the variation into a series of guessed creation commands. Correct the target name and repeat the read. The purpose is to connect an error to a specific input and show that the intended group can still be inspected.

Finish the lifecycle

Cleanup begins with another inventory. Verify the subscription, exact group name, and contained resources. The lab group should still be empty. Delete only that dedicated group and wait for the operation to finish before checking its existence.

```
az group delete --name "$lab_group" --yes
az group exists --name "$lab_group"
```

The expected final value is false after successful removal. A pending deletion requires appropriate waiting and rechecking. An error requires investigation rather than a claim that cleanup succeeded. The absence check provides evidence for the specific group in the current subscription.

The PowerShell cleanup path uses Remove-AzResourceGroup with the verified name. Its confirmation behavior provides an opportunity to inspect the target. Avoid suppressing confirmation casually in interactive learning. Automated cleanup needs an equally explicit target check and a documented ownership boundary.

This first lifecycle is deliberately modest. It exercises authentication context, resource targeting, creation, inspection, troubleshooting, and removal. Those same stages recur when the resources become more complex. A learner who can explain every result in this exercise has established a practical foundation for the identity and governance work that follows.

Guided lab

Verify context and manage a dedicated resource group

Objective: Demonstrate that a resource-group operation targets the intended subscription, inspect the result through two interfaces, and verify cleanup.

Prerequisites and permissions: An Azure public-cloud practice subscription, permission to create and delete a resource group, and an authenticated CLI or Az PowerShell session. A suitable Contributor assignment at subscription scope permits the necessary group operations, subject to policy and other controls. A group-scoped assignment on another group does not authorize creating a new subscription-level group.

Cost: An empty resource group does not create billable compute or storage by itself. Cloud Shell storage, if provisioned for the session, is separate and may incur charges. No VM or storage account is required.

1. Record the intended subscription identifier and confirm it in the portal. Inspect the current command-tool context using the read commands in Section 1.5.

2. Set the tool context if necessary and inspect it again. Record the tenant identifier as well as the subscription.
3. Check whether rg-compass-ch01 already exists. If it contains any unrelated resources, choose a new dedicated name and use that name consistently.
4. Select an allowed region. Create the group with Purpose=Training and Book=Compass tags using one of the command examples.
5. Read the group's identifier, location, and tags. Compare the identifier with the portal's resource-group properties.
6. List the group's contents. The expected collection is empty. Explain why the group can exist while its contained-resource list is empty.
7. Query a deliberately nonexistent group name. Record the error category and correct the name. Confirm that the original group remains readable.
8. Inspect the group's contents again, delete the dedicated empty group, and verify its absence. Do not remove a group containing resources whose ownership is uncertain.

Expected evidence: A context record, the created group's identifier and properties, an empty member inventory, the controlled read error, and an absence result after deletion. These are expected outcomes to collect during execution, not claims that this manuscript's examples have been run.

If the lab fails: Separate shell syntax, authentication, authorization, policy, and target-name issues. Check the command's active subscription before making a replacement group. If Cloud Shell initialization fails, use an approved local CLI or PowerShell installation or resolve the session issue first.

Chapter synthesis

Azure administration begins with an identifiable actor, an explicit target, and a result that can be checked. Regions and zones describe placement and failure boundaries. The management hierarchy organizes resources and the scope of controls. The portal and command-line tools expose those concepts through different interfaces. The completed resource-group lifecycle demonstrates the habits that later deployments will require: inspect context, make a deliberate change, verify the intended property, and account for cleanup.

Review questions

01-Q1. A learner can inspect a storage account's properties but cannot read a blob using an Entra token. What should be examined first?

- **A.** The identity's data-access authorization and the data request's path
- **B.** The storage account's display color
- **C.** Whether the resource group is in the same region as the account
- **D.** Whether the learner has created a dashboard

01-Q2. A single VM is placed in an availability zone. Which conclusion is justified?

- **A.** Its application automatically runs in every zone
- **B.** Its deployment has a selected zonal placement
- **C.** It automatically fails over to a paired region
- **D.** Its data is protected from accidental deletion

01-Q3. A resource group is located in one region. What does that establish about a VM inside it?

- **A.** The VM must run in that region
- **B.** The VM is automatically replicated there
- **C.** The VM's actual location must be inspected separately
- **D.** The VM has no regional property

01-Q4. Two subscriptions contain a resource group named rg-training. What best distinguishes the exact target of a change?

- **A.** The resource group's name alone
- **B.** The browser tab position
- **C.** The most recently opened dashboard
- **D.** The full resource identifier including subscription

01-Q5. The portal shows an empty resource list immediately after a successful deployment in another session. What is the best first action?

- **A.** Create the resources again with different names
- **B.** Check directory, subscription, and resource filters
- **C.** Delete the subscription's locks
- **D.** Grant all users Owner

01-Q6. A Bash example fails in PowerShell before an Azure request is sent. What should be checked first?

- **A.** The shell's syntax and variable conventions
- **B.** Azure Policy compliance
- **C.** Storage replication state
- **D.** The application's health probe

01-Q7. An empty practice resource group was created successfully. Which observation verifies that the intended subscription was targeted?

- **A.** The creation command used a short resource name
- **B.** The portal page contains an Azure logo
- **C.** The returned resource identifier contains the intended subscription ID
- **D.** The shell window remains open

01-Q8. Before deleting a practice resource group, its inventory reveals an unfamiliar VM. What is the appropriate action?

- **A.** Delete the group because its name contains training
- **B.** Delete only the VM without investigating
- **C.** Rename the VM and delete the group
- **D.** Establish ownership before deleting the group

Further reading

- Azure Resource Manager overview
- Shared responsibility in Azure
- Azure regions
- Availability zones
- Azure Cloud Shell overview
- Manage subscriptions with Azure CLI
- Manage subscriptions with Azure PowerShell

Chapter 2: Building the Identity Foundation

Every useful Azure environment begins with a reliable answer to a simple question: which identity is making this request? Resources, subscriptions, and networks depend on that answer, even when identity is not visible on the screen. Building the Identity Foundation develops a practical understanding of directories, accounts, groups, licenses, and external collaboration. The aim is to make identity records dependable enough that later access decisions have a sound basis.

Learning objectives

After studying this chapter, the reader should be able to distinguish a tenant from a subscription, maintain user properties through an account lifecycle, choose appropriate group membership, investigate license assignment problems, and explain how an external identity is represented in a resource tenant. The reader should also recognize the difference between recording an identity and granting that identity permission.

2.1 Understanding Microsoft Entra Tenants

A directory establishes an identity boundary

A Microsoft Entra tenant is an organization's instance of the Microsoft Entra identity service. Its directory holds objects such as users, groups, applications, and devices. Those objects describe identities and their relationships. An Azure subscription, by comparison, organizes Azure resource consumption and management. Confusing these two containers leads to errors because the administrator begins looking for identity settings under a billing or resource boundary.

A tenant can support several subscriptions. For example, an organization might separate development and production into different subscriptions while using the same employee identities in both. That arrangement does not give every employee access to both subscriptions. Identity provides the objects to which permissions can be assigned; authorization determines the actual access. A subscription associated with a directory does not become an open resource catalog for all directory members.

The directory also serves services beyond Azure infrastructure. An employee might use the same organizational account to sign in to Microsoft 365 and to an Azure administrative tool. The products still have separate permissions, licenses, and service settings. A successful Microsoft 365 sign-in therefore does not prove that the account can manage a virtual machine. It establishes only that the identity can authenticate in the relevant context.

Names identify; identifiers distinguish

Every tenant has a tenant ID, an identifier that is more dependable for automation than a friendly organization name. A display name helps a person recognize the organization but does not establish uniqueness. When an administrator supports several directories with similar names, recording the tenant ID in the change request prevents work from drifting into the wrong environment. Resource

subscriptions likewise have subscription IDs, and both identifiers may be needed to describe the intended target.

A new directory includes an initial domain ending in `onmicrosoft.com`. Organizations can add verified custom domains to support familiar sign-in names. Verifying a domain demonstrates control of that domain; it does not automatically create users, migrate mail, or transfer Azure resources. The verification task and the later use of the domain are separate changes. Keeping them separate makes rollback and troubleshooting more understandable.

A user principal name, or UPN, is a sign-in name such as `analyst@example.org`. It resembles an email address, but the resemblance should not be treated as a guarantee that a mailbox exists. An account can have a sign-in name without a licensed mail service. Applications may also store email properties separately. During an account rename, the administrator therefore checks the sign-in identity, mail behavior, and application dependencies individually.

The object ID identifies a particular directory object. Two user objects with similar display names remain different identities. Deleting an account and creating another with the same visible name does not recreate the old object's identity. References that point to the former object can become stale. This matters when investigating a user who appears correctly named but does not receive the access previously associated with that name.

Administrative context is part of every task

An administrator can have access to more than one tenant. Browser sessions and command-line tools can retain a previously selected directory or subscription. A portal screen that looks familiar is weak evidence of the current context. The useful evidence is the signed-in account, tenant ID, and selected subscription. Those values belong in the pre-change check, especially before account deletion, license removal, or role assignment.

The following read-only Azure CLI inspection displays subscription context:

```
az account show --query '{subscription:id,tenant:tenantId,name:name}' -o table
```

The expected output is a small table describing the current CLI account context. It does not show every Entra permission held by the signed-in user, and it does not prove that a Microsoft Graph session uses the same tenant. Different tools can maintain different authentication contexts. A reliable runbook states which tool is being checked and which identifier is expected before the tool performs a change.

For identity administration through PowerShell, Microsoft Graph tools are the relevant interface for directory objects. Azure PowerShell's Az modules serve Azure resource administration. Installing both does not join their sessions. An administrator who signs in with `Connect-AzAccount` has not thereby established every Graph connection required for user management. The permission model and session context must match the API being used.

In an established Graph PowerShell session, `Get-MgContext` exposes the tenant and granted scopes for inspection. A listed scope does not replace the directory role required by a delegated operation. Both the application's delegated permission and the signed-in user's applicable authority can matter. The administrator verifies the operation's requirements instead of assuming that a successful connection grants unrestricted directory control.

Administrative authority has boundaries

Microsoft Entra roles control directory administration. Azure roles control Azure resource access. A User Administrator can perform supported user-management tasks within Entra, while an Azure resource Contributor can manage allowed Azure resources at the assigned scope. Neither role should be inferred from the other. The distinction is particularly important when a person can create a virtual machine but cannot create a directory user for a lab.

The tenant is an administrative boundary, but it is not a substitute for careful delegation inside that boundary. Large organizations separate account operations, application administration, and privileged access management. Smaller organizations may give one person several responsibilities, yet the responsibilities remain technically distinct. Recording the reason for each role makes later reviews easier and reduces the temptation to solve every failure with the broadest available role.

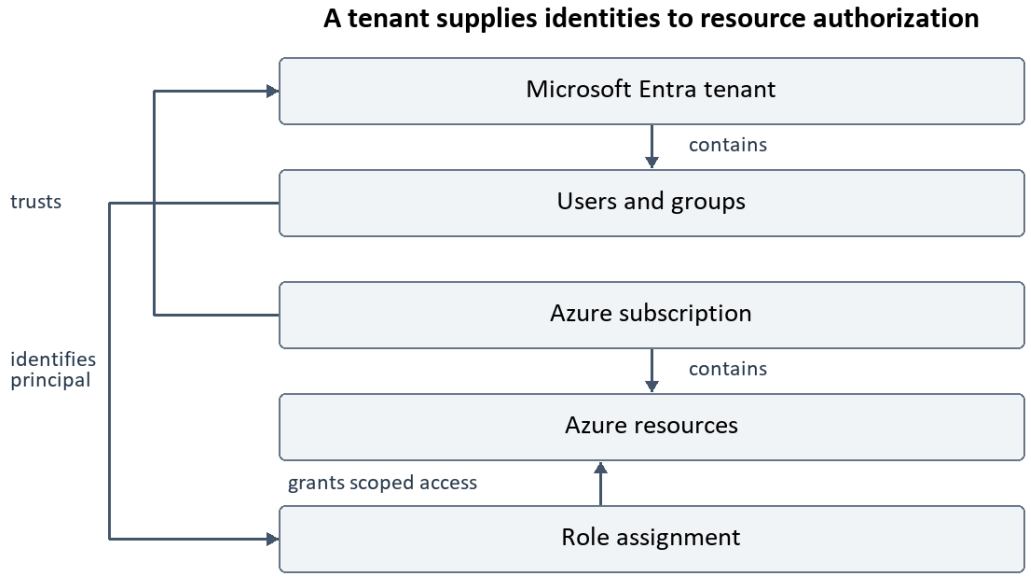


Figure 2.1 Identity and subscription relationships

A subscription trusts a tenant for identities. Directory membership alone grants no Azure resource role; an authorization assignment connects the principal, role, and scope.

The figure connects a tenant to directory objects and to subscriptions that trust that tenant. It deliberately separates the user object from a resource permission. Membership in the directory creates a candidate identity for authorization. An additional authorization decision is still required before the identity can operate an Azure resource. This separation becomes the basis for understanding Azure role assignments in the next chapter.

A small design example

Consider an organization with a production subscription and a training subscription. Both use one workforce tenant. The production operations group receives narrowly selected permissions in production. A separate training group receives access to disposable training resources. A trainee can be a member of the same tenant as a production administrator without receiving production permissions.

Separate subscriptions help express the resource boundary; group membership helps express the identity relationship.

The alternative of creating a new tenant for every subscription introduces other work. Accounts, application registrations, external collaboration, and administration need coordination across those directories. Additional tenants can be justified by specific isolation requirements, but they should not be created merely because a new subscription exists. The decision should describe the boundary that must be isolated and the operational responsibilities introduced by that boundary.

Question	Identity context	Resource context
Which account exists?	Tenant and directory object	Not answered by a resource group
Which resources belong together?	Not determined by group membership alone	Subscription and resource group
Who can reset a supported account?	Appropriate Entra role	Azure Contributor is insufficient
Who can change a virtual network?	A recognized principal is required	Appropriate Azure role and scope

This comparison is useful because Azure exposes both identity and resource tasks through related interfaces. The same browser can display a user blade and a virtual network blade, but similar navigation does not imply a shared authorization model. The administrator should identify the service receiving the request before deciding which permission needs to change.

Diagnosing a directory mismatch

A common failure begins with a valid user who cannot be found in an assignment picker. The first investigation is not whether the account needs a stronger role. It is whether the picker is searching the intended tenant and whether the expected object exists there. An external collaborator may have an account in another organization without yet having a representation in the resource tenant. A local user may exist only in a separate test directory.

The evidence sequence is straightforward: record the current tenant, locate the object in that tenant, compare its object ID with the intended principal, and check the operation's permissions. If the account exists elsewhere, the administrator chooses the supported collaboration or tenancy arrangement instead of creating an unrelated duplicate. Duplicate accounts often make the immediate screen look correct while producing a less understandable identity estate.

Identity work begins with context. A tenant name, a person's name, and a subscription name help navigation, while identifiers establish the exact target. Authority determines which operations are permitted there. Checking those facts before a change gives account maintenance a dependable starting point.

2.2 Creating and Maintaining User Accounts

An account is a lifecycle record

Creating a user establishes a directory object, but useful administration continues through changes and eventual departure. A well-maintained account records who the identity represents, how it signs in, which source owns its properties, and who approves its access. The creation form is only the first moment in that lifecycle. Later changes can be more consequential because applications and permissions already depend on the object.

Before creation, the administrator determines whether the identity should be created locally, synchronized from another directory, or invited through external collaboration. These are not interchangeable methods. A synchronized identity has an upstream source of authority for certain properties. An external collaborator usually retains a home identity outside the resource organization. Creating a local account simply because it is easy can create an unnecessary password and an unclear responsibility for future maintenance.

The account request should contain a business owner, intended start and end dates where applicable, required applications, and the authoritative personal details. The administrator should not infer privileged access from a job title. A person working in an IT department may need only ordinary application access. Conversely, a temporary contractor may require a carefully scoped technical permission. Identity attributes describe the person; access decisions require separate justification.

Create properties deliberately

Display names help people recognize an account in lists and reports. They are not a dependable unique key. A naming convention can make duplicate names less confusing, but object IDs remain important when scripts or investigations must identify exactly one object. Reports should include enough information to distinguish similar names without unnecessarily distributing personal data. The goal is a useful operational record, not an ever-expanding profile.

The UPN should use a verified domain supported by the tenant. A sign-in suffix that looks plausible is not enough. If a custom domain has not been verified, the administrator must complete the domain task or use the initial tenant domain for the account. Domain ownership and account creation are separate operations, and failures in one should not be hidden by improvising a different identity without documenting the decision.

Usage location matters when licenses are assigned because service availability can depend on location. Department and other business properties can matter when dynamic group rules use them. A property that appears descriptive may therefore influence access or service entitlement indirectly. Organizations should identify which attributes drive automation and apply stronger validation to those attributes than to purely descriptive fields.

Initial credentials deserve their own handling process. A temporary password is a secret even if the account must change it at first sign-in. It should not be stored in a shared spreadsheet, copied into source control, or printed into command output that becomes a build log. The account owner should receive it through the organization's approved channel. A password-change requirement does not make insecure delivery harmless.

A controlled creation procedure

The following procedure describes a cloud-only training account. It is not a recommendation to create production identities without an approved request.

1. Confirm the intended tenant ID and an appropriate Entra administrative role.
2. Open the user creation experience in the Microsoft Entra admin center.
3. Enter an unused UPN in the tenant's verified domain and a clearly marked training display name.
4. Set the relevant profile properties, including usage location before any planned license assignment.
5. Create the account and record its object ID in the lab notes without recording its password.
6. Confirm that the account appears with the intended properties and no unintended role assignments.

The expected result is one new directory object. Successful creation does not prove that the account has a mailbox, a product license, an Azure subscription role, or a configured authentication method. Each of those is a separate state that can be checked independently. Keeping the checks separate produces more useful evidence than a single statement that the user is “ready.”

Changes have downstream effects

A department update can change dynamic group membership. A UPN update can affect how a user signs in and how an application recognizes that sign-in. A license change can affect service availability. Administrators should therefore classify a property update by its consumers. The important question is not only whether the directory accepts the new value, but which processes will act on that value afterward.

For example, an account moving from a finance team to an engineering team may need access removed as well as added. Updating the department alone might be sufficient for one dynamic group but irrelevant to a directly assigned Azure role. A complete transition reviews memberships, direct assignments, licenses, and application-specific permissions. The directory profile is one source of evidence, not a universal switch that moves every entitlement.

Synchronized users require special care because a cloud-side change may be unavailable or later replaced by the source directory. The administrator should inspect the identity's source and choose the authoritative system for the property. Repeatedly editing the wrong side creates apparent instability: a value changes and then seems to change back. The behavior becomes understandable once the synchronization direction and ownership are identified.

Bulk operations need the same care at greater scale. A CSV file can repeat a mistaken domain, usage location, or department across many users. Before submission, the administrator validates required columns, unique sign-in names, and the intended operation. A small approved pilot offers evidence about the input format and resulting properties. After the bulk job, the result report should be reconciled with the request rather than assuming every row succeeded.

Disable, remove, and restore are different actions

Blocking sign-in stops an account from obtaining normal new access through the blocked sign-in path, but incident response may also require session revocation and service-specific actions. A departure process should not treat one directory switch as proof that every existing session or application token has disappeared. The appropriate response depends on the risk and the services involved. Identity operations should follow the organization's established offboarding procedure.

Deletion changes the object's lifecycle state. Microsoft Entra provides a deleted-user recovery experience for a defined period, but recovery should not be treated as the routine backup strategy for careless administration. Before deletion, the organization should address ownership, data retention, licenses, and applications that reference the user. A restored object also deserves validation because the operational consequence can extend beyond its visible profile.

Recreating a deleted user with the same UPN is not equivalent to restoring the deleted object. The new object has its own identity, and old references may still point at the former object ID. This is why an access investigation should compare identifiers rather than relying on matching names. A restored account and a newly created look-alike can produce very different permission results.

Troubleshooting with a lifecycle view

Suppose a newly created user signs in successfully but cannot open a required service. Authentication is working well enough to establish identity, so repeatedly resetting the password is unlikely to address the service entitlement. The next checks are license state, application assignment, relevant group membership, and the service's authorization requirements. The observed failure should determine the investigation, rather than habitually returning to the creation form.

If creation itself fails, the administrator records the error and checks the tenant, domain, duplicate identity, and administrative permission. If an update appears to revert, the source of authority becomes a stronger candidate. If a previously working account loses access after a department change, attribute-driven membership becomes a useful lead. These hypotheses connect the symptom to the stage of the account lifecycle that recently changed.

Operational records should preserve the reason, approver, target object ID, changed properties, and verification result. They should exclude passwords and unnecessary personal detail. A clear record allows another administrator to understand why the account exists and which later changes were intentional. That understanding is often more valuable than an elaborate naming convention because it explains the relationship between the directory object and the actual work.

Account administration is complete only when the intended state is verified. The user exists in the correct tenant, the properties have the right authority, secrets have been handled properly, and access has been considered separately. This discipline makes future maintenance predictable and reduces the number of unexplained exceptions that accumulate around otherwise ordinary accounts.

2.3 Organizing Groups and Membership

Groups turn repeated decisions into manageable relationships

A group collects identities that share a purpose. Instead of assigning the same access separately to every person, an administrator can assign suitable permissions to a group and manage its membership. That reduces repetitive work, but it also makes group membership a security decision. A change that appears to add a name to a list can grant access to several resources at once.

The group should have a clear purpose before it receives members. A name such as `sg-compass-storage-readers` communicates more than a general label such as `CloudUsers`. The description can explain the resource boundary, business owner, and membership criteria. Those details help prevent the group from becoming a convenient container for unrelated permissions. A group that accumulates unrelated uses becomes difficult to review because membership no longer expresses one understandable requirement.

Security groups are commonly used for access management. Microsoft 365 groups support collaboration experiences and related services. The choice should follow the consuming service and the intended behavior. Creating a Microsoft 365 group merely to obtain an access list can introduce collaboration resources and ownership expectations that were never part of the request. Conversely, a basic security group does not provide every collaboration capability associated with a Microsoft 365 group.

Assigned and dynamic membership solve different problems

Assigned membership is maintained through explicit changes. An administrator or authorized owner adds and removes members according to a request or review. This model is easy to understand when membership reflects a small, stable project team. Its main operational weakness is that someone must remember to process departures and role changes. The group can remain technically valid while its membership becomes outdated.

Dynamic membership evaluates a rule against supported directory attributes. For example, a rule can express a department-based membership condition. The advantage is consistency when the underlying attributes are reliable. The risk is that an inaccurate attribute can systematically produce incorrect membership. A dynamic rule does not correct poor source data; it applies the rule to the data available. Licensing requirements and supported rule behavior must be checked before implementation.

The choice therefore depends on the authority for membership. If an approved project owner decides who belongs, assigned membership may fit well. If a dependable personnel attribute determines eligibility, dynamic membership can reduce manual reconciliation. The administrator should document the reason for the chosen model and the owner of the source information. Without that ownership, a failed membership can become a disagreement between teams rather than a solvable technical issue.

Membership approach	Useful when	Main verification question
Assigned	Membership follows explicit approval	Was the intended object added and later reviewed?

Membership approach	Useful when	Main verification question
Dynamic	Trusted attributes express eligibility	Does the rule match accurate current attributes?
External group relationship	Another organization owns identity data	Which system actually controls the local membership?

This table is a decision aid rather than a list of interchangeable features. The consuming service may restrict which group types or membership relationships it supports. An administrator should confirm that the chosen group works for the intended assignment before designing an elaborate membership process around it. A well-maintained group still cannot supply a capability the destination service does not accept.

Owners and members carry different responsibilities

A group owner helps maintain the group; a group member receives whatever the group's membership grants through its consumers. Ownership should not be assumed to confer every resource permission granted to members. Likewise, being a member does not automatically authorize membership changes. These relationships should be tested rather than inferred from a friendly title in the portal. The administrative boundary belongs to the group service and the resource service involved.

At least one accountable owner should understand the group's business purpose. Technical administrators can maintain the object, but the business owner usually knows whether a person still needs the access. A recurring review can compare current membership with the approved population and record exceptions. The cadence should reflect the sensitivity and rate of change of the access, rather than applying the same schedule without thought to every group.

Groups used for privileged directory roles have additional requirements and restrictions. An ordinary security group should not be assumed to support every privileged role assignment. Role-assignable groups require deliberate creation choices and appropriate permissions and licensing. For AZ-104 administration, the essential distinction is that Azure resource roles and Entra directory roles have different group-related rules. Treating every “administrator group” as equivalent obscures those boundaries.

Membership is only one part of effective access

A user's access depends on the group's assignments as well as membership. Adding a user to a correctly named group does nothing if the group has never received the intended role. Conversely, a group can hold broader permissions than its name suggests. A review should therefore inspect both sides: who belongs and what the group grants. Neither list alone provides a complete account of the risk.

Nested groups require particular caution. Different services process nested membership differently, and group-based licensing does not grant licenses through nested groups in the same way as direct membership. A structure that looks elegant in the directory may not produce the expected result in the consuming service. The administrator should use the destination service's documented behavior and test the exact membership path rather than assuming transitive access everywhere.

Changes also take time to become effective across services and sessions. A membership update appearing in the directory is evidence that the directory accepted it. It is not necessarily evidence that a previously issued token or every downstream service has refreshed its view. A test should distinguish configuration state from observed access. Repeatedly adding and removing the same member during this interval makes the evidence harder to interpret.

An example that reveals the complete relationship

An operations team needs read access to a training resource group. The administrator creates a purpose-specific security group, adds the intended users, and assigns the Azure Reader role at the training resource-group scope. The group name describes intent, the membership expresses the approved people, and the role assignment implements resource access. All three should be inspected when a new team member reports that the resource group is missing.

If one member can read the resources and another cannot, the administrator first compares the member objects and sign-in context. The failing person may be using a different account with a similar name. If no members can read the resources, the group's role assignment or scope is a stronger lead. This comparison avoids treating every access complaint as a generic group problem and uses the pattern of failure to narrow the cause.

A group that serves both licensing and resource access deserves special scrutiny. Removing a member to withdraw a product license may also remove Azure permissions. Adding a person for an application may grant unrelated infrastructure access. Separate purpose-specific groups often make those relationships easier to manage. The number of groups is less important than whether each group expresses a coherent, reviewable decision.

Practical membership validation

A useful validation record includes the group's object ID, type, membership model, owner, and intended consumer. It also records the test principal and the expected access result. For a dynamic group, the record includes the relevant source attributes and rule evaluation state. For an assigned group, it includes the membership request and the completed change. These records support later troubleshooting without preserving an unnecessary copy of the entire directory.

The cleanup process should remove unused assignments before deleting a group that applications or resources still reference. Otherwise, an assignment can be left pointing at an object that no longer has a useful visible identity. Before deletion, the administrator inventories the group's known consumers and confirms that membership is no longer required. A group is a shared dependency, even when its portal page contains only a short member list.

Good group administration makes access easier to explain. The administrator can state what the group is for, who decides membership, which attributes or approvals control it, and what the group grants. When those answers are clear, groups reduce complexity. When they are missing, groups merely move complexity out of individual accounts and into a less visible place.

2.4 Managing User Licenses

A license supplies entitlement, not universal permission

A license makes a product or service capability available under the organization's subscription terms. It does not automatically grant the user an Azure administrative role. A licensed employee can still lack permission to manage a resource group, and an authorized Azure administrator can still lack a separately licensed identity feature. License state and authorization state should be checked as different dimensions whenever a service is unavailable.

The word subscription can cause confusion here. An Azure subscription organizes Azure resources and consumption. A product subscription can provide purchased license capacity for a service such as Microsoft 365 or a Microsoft Entra feature. The administrator should identify which product and purchasing arrangement are involved before interpreting a license count. A screen showing available product seats does not describe the remaining budget in an Azure resource subscription.

A product license can contain service plans. Those plans represent capabilities within the purchased product. Assigning the product while disabling a particular service plan can produce a user who appears licensed but cannot use the expected feature. Troubleshooting should therefore inspect both the product assignment and the enabled services. The visible product name alone may be insufficient to explain the user's experience.

Direct and group-based assignment

A direct assignment applies a license to an individual user. It is easy to understand for a small exception or a controlled test. Its weakness is that large numbers of direct assignments require repeated maintenance. A role change or departure must trigger a corresponding review, or the account can retain a product that is no longer needed. Direct assignment should remain intentional rather than becoming an invisible default for every request.

Group-based licensing applies a license configuration through membership in a supported group. It connects product entitlement to an established membership process. A team group can therefore provide a consistent set of services to its direct members. However, the group is now a dependency of service availability. Removing a member can remove the associated entitlement, and an incorrect membership rule can affect several users at once.

Current Microsoft guidance places group license assignment in the Microsoft 365 admin center. Microsoft Graph PowerShell also supports license operations. Older screenshots or training materials may point to an earlier administrative location. When navigation differs from a book or remembered procedure, the administrator should verify the current documented interface rather than assuming that the feature has disappeared. Product administration surfaces change more often than the underlying distinction between direct and inherited entitlement.

This interface guidance was checked in November 2026. It identifies the administration surface, not a promise that every license product exposes identical settings. The product's supported service plans and the tenant's purchased entitlements still determine what can be assigned.

Assignment source	Main advantage	Operational question
Direct user assignment	Clear individual exception	Who reviews and removes the exception?
Group-based assignment	Consistent entitlement for a defined population	Is membership accurate and processing complete?
Both sources	Can support a controlled transition	Which source still keeps the license assigned?

Multiple assignment sources require careful interpretation. Removing a user from one licensed group does not necessarily remove the product if another assignment still provides it. The administrator should inspect the assignment path before concluding that removal failed. Conversely, removing the last valid source can interrupt service after processing completes. A clean licensing record makes these dependencies visible before a change is made.

Prerequisites are part of the license decision

Available purchased capacity is an obvious prerequisite, but it is not the only one. Usage location affects service availability, and service-plan combinations can have dependencies or conflicts. The administrator should validate the location and product requirements before applying a broad group assignment. An error affecting many members may originate in a common property or product configuration rather than in the group mechanism itself.

Group-based licensing does not treat nested groups as a general way to expand the licensed population. The documented behavior assigns licenses to users who are direct members of the licensed group. An administrator who adds an entire department group as a nested member should not assume that every user inside it receives the product. The intended population must be compared with actual resulting license assignments.

Permissions also matter. The role that administers Azure virtual machines does not automatically authorize product licensing changes. Microsoft 365 and Entra administrative roles must be appropriate for the operation, with the least privilege supported by the specific task. A disabled license control can indicate insufficient administrative authority rather than missing purchased capacity. The error and current role should be examined before changing procurement assumptions.

Processing creates an observable transition

Group license changes can be processed asynchronously. The initial submission confirms that the change was accepted, while later processing determines each user's resulting assignment. This distinction is important in a large group or during a busy service period. An administrator should not promise an immediate fixed completion time without evidence from the actual tenant and operation. Progress and per-user errors are more useful than an arbitrary waiting interval.

Consider an employee moving between two licensed groups that provide the same required product. Removing the old membership first can create a temporary gap before the new membership produces an assignment. A safer planned transition adds the destination membership, verifies the resulting

license, and then removes the obsolete source. The sequence protects continuity while still achieving the desired final membership. The verification step is what distinguishes a controlled transition from a hopeful delay.

The same logic applies when simplifying mixed direct and group assignments. First establish and verify the intended durable assignment source. Then remove the redundant direct assignment. Finally inspect the resulting state and test the relevant service. A license cleanup that only reduces the visible number of assignments can be misleading if it also disables a required service plan or removes the last effective source.

Investigate errors by category

A capacity failure requires checking available licenses and the intended licensed population. A usage-location failure requires checking the user's location and the product's availability rules. A service-plan conflict requires examining the combination of enabled services. A membership issue requires verifying direct membership and group processing. These categories help an administrator choose the next piece of evidence rather than repeatedly selecting the same license option.

Suppose several new employees belong to a licensed group, but only some receive the product. A comparison between successful and unsuccessful users can expose a shared difference. The unsuccessful accounts may lack an explicit usage location, have a conflicting assignment, or still show a processing error. The administrator should preserve the error details and fix the underlying cause before requesting reprocessing. Reprocessing an unchanged invalid configuration simply repeats the failure.

The test should include the service the user actually needs. A product marked assigned is an important control-plane observation, but the user's application may still require sign-in, provisioning, or a separate permission. For example, a license can enable access to a service while the service's own resource authorization remains restrictive. A successful license assignment should not be reported as proof that every application workflow is available.

Removal and cost control need context

Unused entitlement can waste money, but license removal can also affect service access and data handling. Before removing a production license, the organization should follow the product-specific retention and offboarding process. An administrator should not assume that all products preserve data for the same period or behave identically after removal. The operational owner and documented service behavior determine the appropriate sequence.

A useful license review distinguishes purchased capacity, assigned capacity, active use, and justified reserve. Those figures answer different questions. An unused purchased seat is not the same as a seat assigned to an inactive account. A newly assigned seat may be intentional onboarding capacity. A report becomes actionable when it connects the numbers with an owner and a decision, rather than merely ranking users by apparent inactivity.

The audit record should identify the product, service-plan configuration, assignment source, affected population, error state, and verification result. License names can change, so automation should use the identifiers and current product information appropriate to the API. Human-readable names remain

useful in reports, but a familiar label should not replace a precise target. Clear records make future product migrations and troubleshooting less dependent on memory.

License administration works best when entitlement follows a dependable identity process. Accurate accounts support accurate groups; accurate groups support consistent assignments; verified assignments support usable services. Each relationship can fail independently, so each needs evidence. Once those relationships are understood, licensing becomes a manageable service lifecycle rather than a collection of unexplained checkboxes.

2.5 Working with External Users

Collaboration spans identity boundaries

External collaboration allows a resource organization to work with people whose primary identity belongs elsewhere. Microsoft Entra business-to-business collaboration, commonly called B2B collaboration, can represent those people in a workforce tenant without requiring the resource organization to manage an ordinary local password for each partner. The arrangement separates the source of authentication from the resources being shared. That separation is the central idea behind external-user administration.

The home organization or identity provider is responsible for the external person's primary sign-in identity. The resource organization controls its own applications, resources, and local authorization decisions. An invited partner does not become an unrestricted member of the resource environment merely because authentication succeeds. The resource tenant must still recognize the external identity and grant the intended access. Trust in an authentication result is not the same as permission to every resource.

The external user object provides a local representation that can participate in supported assignments. Its object ID belongs to the resource tenant. The person's home object has a different identity in its own tenant. Recording the resource-tenant object ID is therefore important when investigating an Azure role assignment. A script that uses the home object ID without considering tenant context can target the wrong identity or fail to find a principal.

Guest and external are related but different descriptions

The `Guest` user type describes a directory relationship and associated default privileges. It does not, by itself, describe every authentication detail. Microsoft documents internal and external member and guest combinations. An administrator should inspect the actual identity source and properties rather than assuming that every guest is a person with an account in another Entra tenant. The relationship and the authentication method are separate attributes of the account.

This distinction is useful when an account's visible type has been changed. Changing a user type does not magically migrate the person's password authority or transfer their home organization. It changes a local classification that can influence default access behavior. Any broader identity transition needs a supported design and validation. A label in a user profile is too small a piece of evidence to explain the whole sign-in path.

For ordinary partner collaboration, the administrator should prefer a clear invitation and authorization process. The request identifies the partner, the internal sponsor, the resources required, and the expected end date. The sponsor provides business accountability when the external person is not represented in the organization's employee records. Without a sponsor, guest access can outlive the project that originally justified it.

Invitation and redemption establish a relationship

An invitation identifies the external address and can provide a route into the resource organization's application. Redemption associates the invited person with the supported authentication path. The administrator should distinguish an invitation that was sent from an invitation that was successfully redeemed. A pending state may explain why a collaborator has not yet reached the intended service, but it does not justify granting broader resource permissions.

Before sending an invitation, the administrator checks the organization's collaboration settings and confirms that the inviter has authority. Different policies can restrict who may invite guests or which external domains are allowed. The existence of a Guest Inviter role does not mean that every invitation will bypass those policies. Administrative authority operates within the configured controls, and a blocked invitation should be investigated at that layer.

An invitation email should be sent only as part of an approved collaboration request. For a training lab, an address controlled by the learner or an explicitly consenting colleague is appropriate. A fabricated address should not receive mail, and a real partner should not be invited merely to test the interface. Identity exercises can use existing approved guest objects for observation when no test recipient is available.

Two policy areas answer different questions

External collaboration settings govern matters such as invitation permissions and guest directory visibility. Cross-tenant access settings govern supported inbound and outbound collaboration relationships with other Entra organizations. These controls are related, but they answer different administrative questions. One asks who can establish the collaboration relationship; another asks which cross-organization access is allowed and under what trust conditions.

An inbound restriction concerns external users accessing resources in the local organization. An outbound restriction concerns local users accessing another organization. The direction must be named in a troubleshooting record. A statement that “the partner is blocked” is incomplete without identifying which tenant is enforcing the block and which direction the attempted access follows. Clear directional language prevents changes to the wrong policy.

Cross-tenant trust can include decisions about claims supplied by another organization, such as multifactor authentication claims. The AZ-104 administrator needs to recognize that these settings can influence a sign-in result, without treating advanced identity architecture as a routine shortcut. Trust should be deliberate and limited to the approved relationship. Accepting a partner's claim does not remove the need to authorize access to the actual Azure resource.

A collaboration example with separate evidence

A partner analyst needs to read a training resource group for a limited project. The resource organization invites the approved identity, verifies that the invitation is redeemed, adds the local guest representation to an appropriate security group, and grants the group the Azure Reader role at the resource-group scope. Each action serves a different purpose. Invitation creates the relationship, membership expresses the approved population, and the role assignment implements resource access.

If the analyst can sign in but cannot see the resource group, the administrator examines local authorization and directory context. If sign-in is blocked before the resource is reached, the investigation moves toward identity and cross-tenant policy. If the analyst sees a different organization's portal context, the directory selection becomes relevant. The point is to locate the failed stage before changing a permission that may already be correct.

Observation	First useful evidence
Invitation could not be sent	Inviter authority and collaboration restrictions
Invitation is pending	Intended address and redemption state
Sign-in is blocked	Sign-in details, tenant direction, applicable policy
Sign-in succeeds but resource access fails	Resource-tenant object, role, group, and scope

This comparison is intentionally based on observations rather than error-message folklore. Different applications can display different messages for similar underlying failures. The administrator should preserve the correlation details and sign-in evidence available in the relevant service. A screenshot that says access was denied is a starting point; a record identifying the principal, resource, tenant, and failed control is a useful diagnosis.

External access needs an ending

The partner's home organization might disable the original identity when the person leaves, but the resource organization still owns its local access inventory. It should review the guest relationship and remove unnecessary assignments when the project ends. Depending solely on another organization's lifecycle process leaves uncertainty about local permissions and sponsorship. Both organizations have responsibilities, and one organization's action should not be assumed to complete the other's review.

Before deleting a guest object, the administrator checks its groups, application assignments, and resource permissions. Where the same guest supports several approved projects, removing one project's access may be the correct action rather than deleting the entire relationship. The scope of cleanup should match the ended requirement. A narrow project closure should not accidentally interrupt unrelated approved collaboration.

External-user records should preserve the sponsor, purpose, relevant tenant and object identifiers, approval period, and review outcome. They should not attempt to copy the partner's entire personnel record. The resource organization needs enough information to justify and manage access, while the

home organization remains responsible for its own identity lifecycle. Good collaboration administration respects that division of responsibility and makes the resulting access understandable.

Local and external users now have identifiable objects, groups express approved relationships, and licenses provide specific service entitlements. Resource permission requires another relationship: an authorization assignment connecting a principal to allowed operations at a defined scope. That relationship is the subject of Chapter 3.

Guided lab

Objective: Create a disposable cloud-only user and security group, verify their relationship, and inspect license and guest states without purchasing products or contacting an external recipient.

Prerequisites and permissions: Use an approved training workforce tenant. The operator needs supported user and group administration authority, such as User Administrator for this limited exercise. The tenant must permit the planned account creation. No Azure resource deployment is required, and there are no charge-generating Azure resources in this lab. Do not assign paid licenses or enable trials. The account and group are directory objects, so deleting a resource group will not remove them.

1. Record the tenant ID and the signed-in administrator. Open Microsoft Entra admin center and verify that the selected directory matches the training tenant.
2. Create a cloud-only user named `Compass Chapter 02 Learner` with an unused UPN such as `compass-ch02-learner@<verified-training-domain>`. Use the approved secret-handling method for any initial password. Do not record the password in the lab notes.
3. Set a meaningful department value such as `CompassTraining` and the correct usage location for the test identity. Record the user's object ID.
4. Create an assigned-membership security group named `sg-compass-ch02-learners`. Give it a description stating that it is disposable chapter practice. Do not enable role-assignable group behavior. Record its object ID.
5. Add the test user as a direct member. Inspect the group's member list and the user's group information. The expected result is the same user object ID in both views.
6. Inspect the user's license information. Record that no license was assigned by this exercise. If the tenant automatically assigns licenses through another mechanism, stop and identify that source before continuing; do not remove an unrelated organizational assignment.
7. Inspect an existing approved guest object if one is available. Compare its user type, local object ID, identity source, and invitation state with the local test user. Do not send an invitation for this inspection exercise.
8. Introduce a controlled fault by removing the test user's direct membership from the disposable group. Verify that the member list changes. Add the same object back and verify its identity, avoiding a similarly named account.
9. Remove the test user from the disposable group, delete `sg-compass-ch02-learners`, and delete only the test account identified by its recorded object ID. Follow the tenant's approved deleted-user retention process; do not permanently delete unrelated objects.

The verification record should show the intended tenant, object IDs, property values, membership transitions, and cleanup result. The exercise has not been executed in a live tenant during authoring. Portal labels can change, so the object types and expected states are the durable reference.

Chapter synthesis

Identity administration becomes easier when its relationships are kept explicit. A tenant provides the directory boundary. A user object identifies a particular account, groups collect identities for a defined purpose, and licenses provide product entitlement. External collaboration adds a local representation of an identity whose authentication can be managed elsewhere. Reliable maintenance follows those relationships through creation, change, verification, and removal.

The strongest troubleshooting habit is to ask which relationship failed. A user can exist without a license, hold a license without an Azure role, belong to a group without the group having useful permissions, or authenticate externally without receiving resource access. Each observation narrows the next check. That habit prepares the administrator to reason about authorization without treating every problem as a missing password or a missing administrator role.

Review questions

02-Q1. Which identifier is most suitable for confirming that an automation script is targeting the intended Microsoft Entra directory?

- **A.** The organization's display name
- **B.** The tenant ID
- **C.** The current user's department
- **D.** The resource-group name

02-Q2. A user can manage virtual machines as an Azure Contributor but cannot create a cloud-only directory user. What best explains the difference?

- **A.** The tenant must have a separate subscription for every user
- **B.** A Contributor license is missing
- **C.** Azure resource roles and Microsoft Entra administrative roles govern different operations
- **D.** The virtual machines must be moved into another region

02-Q3. An administrator deletes an account and later creates a new account with the same visible name. Why should old access references be investigated?

- **A.** The new account has a different object identity
- **B.** Display names cannot be reused
- **C.** All Azure roles automatically transfer to the new account
- **D.** A user can belong to only one group

02-Q4. A synchronized user's department repeatedly returns to its former value after a cloud-side edit. What is the best next check?

- **A.** Whether the user's VM is running
- **B.** Whether the group has a public IP
- **C.** Whether every account has Global Administrator
- **D.** Which system is authoritative for the synchronized property

02-Q5. A group should automatically reflect a reliable department attribute. Which design best matches that requirement, assuming required licensing and supported rules?

- **A.** Assigned membership with no review
- **B.** A group with no members and a descriptive name
- **C.** Dynamic membership based on the trusted attribute
- **D.** A direct resource role for every tenant user

02-Q6. A licensed group contains another group as a member. Users exist only inside the nested group. What should the administrator expect from group-based licensing?

- **A.** Every nested user always receives the license immediately
- **B.** Nested membership should not be assumed to assign the license; direct user membership must be evaluated
- **C.** The license becomes an Azure role
- **D.** All nested users are converted to guests

02-Q7. An employee is moving between two groups that provide the same required product. Which sequence best avoids an avoidable entitlement gap?

- **A.** Add the destination membership, verify the license, then remove the old membership
- **B.** Delete the user, then recreate the account
- **C.** Remove both memberships and wait for an application error
- **D.** Change the user's display name before removing the license

02-Q8. An invited external user authenticates successfully but cannot read an Azure resource group. Which check most directly addresses the remaining resource-access question?

- **A.** Whether the guest's home organization uses the same display name
- **B.** Whether a new password can be set in every resource tenant
- **C.** Whether the resource group has the guest's email address as a tag
- **D.** Whether the correct resource-tenant principal has a suitable Azure role at the required scope

Further reading

- Microsoft Entra overview
- Create, invite, and delete users
- Manage groups and membership
- Manage group licenses in Microsoft 365 admin center
- Microsoft Entra B2B collaboration

Chapter 3: Giving Access with Purpose

An identity becomes useful when it can perform the work it is meant to perform. Giving Access with Purpose connects the directory foundation to deliberate authorization. The chapter explains how sign-in differs from permission, how password recovery supports availability, and how Azure roles and scopes combine into effective access. Its central question is precise: which principal needs which operation on which resource?

Learning objectives

The reader should be able to separate authentication failures from authorization failures, explain an SSPR rollout, select appropriate built-in roles, reason about inherited assignments, and investigate access without reflexively granting broader privileges. The reader should also distinguish management operations from data operations and identify evidence that a permission change has actually become effective.

3.1 Authentication and Authorization

Two questions that should never be collapsed

Authentication establishes the identity presenting a request. Authorization determines whether that identity may perform the requested operation. A successful sign-in answers the first question but leaves the second open. An account can authenticate correctly and still receive an access-denied response when it tries to modify a resource. Treating that response as a password problem wastes effort and can distract from the actual permission boundary.

The distinction also explains why an Azure resource can exist without being visible to every directory member. The directory knows who the person is, but the resource-management service still evaluates the permissions associated with that principal. Directory membership is not an Azure Reader assignment. When a portal lists no subscriptions for an authenticated account, the administrator should investigate tenant context and resource access before assuming that authentication failed.

A security principal is an identity that can receive authorization. Users and groups are familiar examples. Applications can use service principals, and Azure workloads can use managed identities where supported. The authorization question remains similar across these forms: what principal is making the request, and what permission applies to the operation? The principal's type changes how identity is established and maintained, not the need for a clear permission decision.

Follow the request through its layers

A typical interactive workflow begins when a user signs in and obtains tokens for a service. The service receives a request and evaluates the token and applicable authorization rules. The portal is a client in this process, not an independent source of permission. Using the command line instead of the portal does not legitimately bypass the resource service's authorization. Both clients must present an acceptable identity and satisfy the operation's controls.

Tokens have context, including the identity and intended service. A token acquired for one purpose should not be treated as a universal credential for every API. This is visible when administrators switch between Azure resource tools and Microsoft Graph. The tools can use different sessions and permissions, even when the same human account appears in both. Troubleshooting should identify the client, target API, tenant, and principal before changing roles.

An application's identity can differ from the identity of the person who deployed it. A deployment performed by an administrator does not mean the running application inherits that administrator's permissions. If the application uses a managed identity to read storage data, the managed identity needs the required data access. Granting the human deployer another role may leave the application's failed request completely unchanged.

Management and data operations need separate reasoning

The management plane controls Azure resource configuration. Creating a storage account, changing a virtual machine size, or adding a role assignment are management operations. The data plane concerns the service's actual data use, such as reading a blob. These planes can use different permissions and, depending on the service, different authorization mechanisms. A role that permits management is not automatically a complete statement of data access.

An Azure Reader can inspect supported resource configuration through control-plane read operations. That does not mean the reader can download every blob or read every database row. Conversely, a data role can grant access to content without broad permission to change the resource's management configuration. The distinction supports separation of duties: a data consumer need not become an infrastructure administrator merely to read the required content.

Owner is a broad resource-management role, but its name should not be interpreted as a magical data-plane bypass. Some management permissions can enable indirect access paths, such as retrieving keys where the service permits that operation. That potential must be considered in security design. It still does not make Owner a substitute for understanding the data service's authorization model. The actual request path determines what is being evaluated.

Request	Main question
Sign in to the portal	Can the identity authenticate under applicable controls?
Read a resource's configuration	Does the principal have the required management-plane read action?
Download a protected blob using Entra authentication	Does the principal have the required data-plane permission?
Add an Azure role assignment	Does the caller have authorization to manage access at that scope?

This separation prevents a common diagnostic error: using evidence from one operation to prove a different operation should work. A user who can inspect a storage account has demonstrated a management read capability. A user who can download one blob has demonstrated access along that

data path. Neither observation alone establishes the ability to create a new role assignment or read another container.

Other controls can still stop a permitted identity

Authorization is necessary, but it is not the only condition for a successful request. Network restrictions can prevent the client from reaching a service. Azure Policy can reject a management change that violates an assigned rule. Resource locks can prevent certain management operations. A service can also enforce its own prerequisites. An administrator should not respond to every failed operation by increasing the role because the role may already be sufficient.

For example, a Contributor may have permission to modify a resource while a read-only lock blocks the attempted change. Assigning Owner does not make the lock irrelevant. Similarly, a data reader may have a correct role while a storage firewall blocks the client's network path. The failure belongs to a different control layer. A useful incident record names that layer and includes the evidence supporting the conclusion.

Least privilege is a concrete design activity

Least privilege means choosing the permissions and scope needed for the task, with no unnecessary expansion. It becomes practical when the task is expressed as operations. “Help with Azure” is too broad to map well to a role. “Read the configuration of this training resource group” is precise enough to evaluate. Clear requests make it possible to select a built-in role and a scope without guessing the user's future responsibilities.

The duration of access also matters. A standing assignment can be appropriate for a continuing operational duty, while temporary work may warrant an approved time-limited access process. Where privileged access tooling is used, eligibility and activation should not be confused with an active assignment. The administrator should inspect the state relevant to the attempted request. A person listed as eligible may still need to activate access under the organization's configured process.

Service identities should receive the same care. A managed identity reduces the burden of handling application credentials, but it does not decide which resources the application should access. An overly broad role assigned to a managed identity remains overly broad. The benefit of the identity mechanism and the quality of its authorization design are separate judgments. Both need attention before a workload is considered ready.

An evidence-first access conversation

A useful access request identifies the principal's object ID, target resource ID, required operation, business reason, and expected duration. The administrator then maps the operation to a supported role and chooses the narrowest suitable scope. A separate verification confirms the intended success and an appropriate failure outside the scope. This approach tests the actual boundary rather than merely confirming that an assignment appears in a list.

Suppose an analyst needs to inspect one training resource group. Reader at that group is a plausible starting design. Reader at the subscription would expose configuration beyond the stated requirement. Contributor at the group would permit changes that were never requested. The decision becomes

straightforward once the request is specific. A broader role should require a broader justified task, not merely a desire to avoid future tickets.

Authentication and authorization become easier to troubleshoot when each observation is kept narrow. A successful sign-in proves identity establishment for that context. A successful operation proves the relevant path worked at that moment. An assignment record explains one source of permission. Together these observations form evidence; individually they should not be stretched into claims about access they never tested.

This discipline also helps communication between support teams. Identity support can report that sign-in succeeded for a particular tenant and principal, while the resource team examines the denied provider operation. Neither team needs to claim that the entire environment is healthy. A precise handoff includes the correlation details, relevant time, target service, and the boundary already verified, allowing the next team to continue the investigation without repeating unrelated checks.

3.2 Enabling Self-Service Password Reset

Recovery is part of identity availability

Self-service password reset, abbreviated SSPR, allows eligible users to recover access through approved verification methods without requiring the help desk to perform every reset. It addresses a different problem from Azure role assignment. A user who cannot authenticate needs an identity recovery path; a user who authenticates but lacks a resource permission needs authorization work. Enabling SSPR does not grant Azure access or repair an incorrectly scoped role.

Password change and password reset are related but different situations. A signed-in user changing a known password is not in the same position as a user who has forgotten the password. Recovery must establish enough confidence through registered methods before replacing the secret. The administrator should identify the exact supported scenario and licensing requirement rather than assuming that every password-related capability is included in every product edition.

SSPR also depends on where the password is managed. A cloud-only user's password is handled in the cloud identity system. A synchronized or federated identity can involve an on-premises source and a writeback arrangement. The visible presence of the user in Entra does not move password authority automatically. A recovery design that ignores the source of authority can succeed at the wrong layer or fail because the required connection does not exist.

Begin with population and prerequisites

An SSPR rollout should identify the intended user population, supported account types, license entitlement, required verification methods, and support process. The service can be enabled for a selected group before a wider rollout. A pilot group allows the organization to validate registration and recovery without making an untested assumption for every account. The pilot should include representative conditions rather than only the administrator's own privileged account.

Administrator accounts have additional reset-policy considerations. Testing only with an administrative account can therefore produce a misleading picture of the ordinary user experience. A standard, dedicated test identity is more suitable for validating the intended user flow. The administrator retains a

separate account for configuration work and avoids changing the recovery methods of the account needed to recover from an accidental configuration error.

Licensing must be checked against the actual scenario. Microsoft distinguishes cloud password operations from on-premises writeback capabilities, and product entitlements can change. The book therefore does not treat a generic “Microsoft 365 license” as a sufficient answer. The administrator should record the supported product and feature combination from current licensing guidance before enabling the service for the target population.

Registration makes recovery possible

Enabling SSPR for a group does not create usable recovery methods on every member's account. Users must register methods that meet the applicable policy. An account can be within the enabled population and still lack enough registered information to complete a reset. This is why rollout communication and registration tracking are operational prerequisites, rather than optional additions after the settings are saved.

The configured authentication methods and the required number of methods must be consistent with supported service behavior. Older training material may refer to legacy method-management surfaces. Administrators should use current Microsoft guidance for the tenant's authentication-method configuration and migration state. The durable principle is that the policy allows specific verification methods and the user must have suitable registered methods available when recovery is needed.

Registration should be completed while the user still has normal access. Waiting until a password is forgotten defeats the purpose of an independently verifiable recovery method. The support process should also explain what happens when the registered device or contact method is unavailable. SSPR reduces routine help-desk work, but it does not eliminate every recovery case. A documented exception path remains necessary.

Test the actual recovery journey

A controlled pilot uses a standard test account that is eligible, appropriately licensed, and registered for the intended methods. The test is performed in a separate browser session so the administrator's authenticated session does not obscure the user's experience. The goal is to observe the recovery process from identification through verification and successful new sign-in. A configuration screenshot alone cannot demonstrate that the workflow is usable.

The expected result includes more than the password changing. The user should receive the intended prompts, use the approved methods, and regain access through the normal sign-in path. The administrator checks relevant audit evidence and notifications where configured. If the design includes an on-premises directory, the test also confirms the expected writeback behavior in that environment. A cloud-only test cannot validate a hybrid dependency it never exercised.

A simple rollout procedure is useful:

1. Confirm licensing, account type, and the password's source of authority.
2. Select an approved pilot group and record the existing configuration.
3. Configure supported methods and registration requirements using current guidance.
4. Register a standard test user through the normal user experience.

5. Perform a reset in a separate session and verify a subsequent sign-in.
6. Review the evidence, resolve failures, and expand only through the approved rollout process.

This sequence keeps configuration, registration, and recovery separate. If the test fails, the administrator can identify which stage needs attention. Repeating the whole rollout without locating the failed stage increases uncertainty and can disrupt users whose registrations were already correct. A small, documented pilot makes the evidence easier to interpret.

Diagnose failures by prerequisite

An account outside the enabled population needs a scope check. An eligible account without enough registered methods needs a registration check. A synchronized user whose password must be written back needs the supported hybrid configuration and connectivity examined. An unsupported or unlicensed scenario needs an entitlement decision. These are distinct failure categories even when the user reports only that password reset “does not work.”

Consider a pilot where cloud-only users reset successfully but synchronized users fail. That pattern suggests a dependency specific to the hybrid identities. The administrator should inspect password authority, writeback configuration, and relevant service evidence before changing the number of required methods for everyone. The difference between the successful and failed populations is useful diagnostic information. A tenant-wide relaxation would not address a missing writeback path.

Another common pattern is that users who completed registration succeed while recently added group members fail. Here the group assignment may be correct, but the new members have not registered acceptable methods. The appropriate response is to complete the registration process and verify policy applicability. Granting those users an Azure role would have no bearing on the recovery prerequisite. The distinction between identity availability and resource authorization remains important even during support work.

Maintain the recovery service

Recovery information can become outdated when phones, devices, or working arrangements change. The organization should maintain a process for keeping registered methods usable and for handling lost methods. Administrative changes to a person's recovery information are sensitive because they can influence account access. The help desk should follow an approved identity-verification procedure rather than treating method replacement as an ordinary profile edit.

Operational monitoring should look for registration gaps, repeated reset failures, and unusual activity using the evidence available in the tenant. A successful deployment is not a permanent guarantee that every new account is prepared. Onboarding should include the required registration, and account changes should preserve an appropriate recovery path. The service becomes reliable when it is part of identity lifecycle management rather than a one-time portal configuration.

SSPR improves access availability when its prerequisites are explicit: eligible identities, correct licensing, supported methods, completed registration, and a working connection to the password authority. The administrator's task is to verify those conditions and maintain them over time. Once authentication is restored, Azure authorization still determines what the recovered identity may do.

The rollout record should include an exit condition for the pilot. For example, the organization can require successful recovery by representative standard accounts, documented handling of an unavailable method, and a support team able to recognize the main failure categories. These are practical acceptance criteria rather than a promise that every future reset will succeed. They establish that the service and its support process have been exercised together. If the pilot is withdrawn, the previous targeting and method configuration should be restored through the approved change process, without deleting unrelated user registrations merely to make the tenant resemble a fresh lab.

3.3 Choosing Built-In Azure Roles

Start with the required operation

An Azure role definition is a collection of allowed operations and related exclusions or data permissions. A role assignment applies that definition to a principal at a scope. The definition answers what can be done; the assignment answers who can do it and where. This distinction prevents an administrator from assuming that a role's existence in the catalog means any user currently holds it.

Built-in roles provide maintained permission sets for common responsibilities. They are usually the first place to look when a task needs authorization. Selecting a role begins with the actual work, not with the most impressive title. A request to view configuration differs from a request to modify virtual machines, and both differ from a request to grant other users access. The role should reflect those differences.

Reader supports management-plane visibility without resource changes. Contributor supports broad resource management within its scope but does not generally grant the authority to assign Azure roles. Owner includes broad management and access-management capability. Specialized roles can provide a narrower set of operations for particular services. These descriptions are useful starting points, but the current role definition is the source for the exact actions.

Read the definition rather than trusting the name

Role names summarize intent but cannot list every dependency. A role for managing virtual machines may include related actions needed for that responsibility without granting unrestricted control of every supporting network or storage resource. The administrator should inspect the definition when a task crosses service boundaries. A failure involving a network interface does not automatically mean the virtual machine role is broken; the required linked operation may be outside its permission set.

The role definition uses `Actions` and `NotActions` for management-plane operations, and `DataActions` and `NotDataActions` for supported data-plane operations. An exclusion removes an operation from that particular role's allowed set. It is not the same as a deny assignment that overrides another grant. If a different applicable role grants the operation, the principal may still receive it. Understanding this prevents incorrect conclusions from reading one role in isolation.

Assignable scopes describe where a role definition can be assigned. They do not themselves assign the role to anyone. A custom role available at a subscription scope still needs a role assignment to a principal. Built-in roles have their own maintained definitions and availability. This chapter focuses on selecting and interpreting built-in roles, while recognizing definition structure sufficiently to explain the result.

Distinguish resource operation from access administration

An operator who can maintain resources does not necessarily need permission to give others access. Keeping these responsibilities separate reduces the consequences of an operational account being misused. Contributor is often considered for resource management, while access administration uses roles that contain the relevant role-assignment permissions. The appropriate choice depends on the task, scope, and any configured delegation conditions.

Role Based Access Control Administrator and User Access Administrator are examples of access-management roles. Their exact capabilities and any assignment conditions should be examined before delegation. A role assignment can be constrained so that an administrator can grant only certain roles or assign only to permitted principals. The presence of an access-administration role does not justify ignoring those conditions. A reduced role picker can be evidence of intentional delegation boundaries.

Owner is broad enough that it should not be used as a diagnostic convenience. If a user needs to restart a virtual machine, the administrator should investigate a role that supports the required virtual machine operations at the correct scope. Assigning Owner to make the test pass can also allow unrelated configuration changes and access grants. The successful restart would prove little about whether the final permission design is appropriate.

Required duty	Candidate role direction	Important boundary
Inspect resource configuration	Reader	Does not generally grant service data access
Manage a broad set of resources	Contributor	Does not generally permit Azure role assignment
Manage authorization assignments	An appropriate access-administration role	Scope and delegation conditions still apply
Read blobs through Entra authentication	A suitable storage data role	Network access and target scope still matter

The table deliberately describes candidate directions. A production request still needs the current role definition checked against its exact operations. Resource types and permissions evolve, and service-specific prerequisites can matter. The role name is a useful search term, not a substitute for inspecting the permissions that the service evaluates.

Data roles express a different responsibility

A person who needs to read storage data should be evaluated for a data role that supports the intended operation. Management Reader alone is insufficient for that purpose. Similarly, a workload writing blobs needs the appropriate write-capable data permission rather than a broad subscription role chosen merely because it is familiar. Matching the permission plane to the operation makes both troubleshooting and review more precise.

Some workflows use access keys or other credentials instead of Entra data authorization. In those cases, the service evaluates that credential path rather than the user's Azure data role in the same way. An

administrator must identify the actual authentication mode before interpreting a result. A successful key-based download does not prove that a newly assigned data role works. The test must use the authorization method the design intends.

Compare roles through realistic tasks

Suppose a support engineer must inspect virtual machine status and restart selected machines in one resource group. Reader can support inspection but not the restart. A suitable compute-management role may support the operational duty, while subscription Owner would greatly exceed the target. The administrator checks the exact restart action, any related read operations, and the desired resource boundary. That analysis creates a defensible choice instead of a guess based on rank.

Now suppose a separate coordinator must grant that support role to approved team members. The coordinator's responsibility is authorization management, not machine maintenance. The two duties can be assigned to different principals and scopes. Keeping them separate makes reviews more understandable: one person operates resources, another maintains the approved access relationship. An organization can combine duties when justified, but the combination should be explicit.

Validate the selected role

A role validation should include a positive test and a boundary test. The positive test checks the required operation on the intended resource. The boundary test checks that an operation outside the approved duty remains unavailable, using a safe read or non-mutating check where possible. The purpose is to confirm the design, not to experiment destructively with production resources. A disposable lab can demonstrate the principle with Reader and Contributor behavior.

The test must use the intended principal. Running a command under the administrator who created the assignment can produce a false success because that administrator already has broader access. A separate session with the test account makes the evidence more meaningful. The record should identify the principal and scope alongside the result. Without those details, a screenshot of a successful command is difficult to interpret.

Built-in role selection is a form of operational design. The administrator translates a task into actions, selects a maintained role that fits those actions, chooses a narrow scope, and validates the result. The result should be explainable in ordinary language: this principal can perform this work on these resources. If that sentence cannot be stated clearly, the assignment is not yet ready for reliable administration.

Role definitions should also be revisited when the operational task changes. An inspection-only duty that grows into deployment work may need a different permission set, while a completed migration may make a temporary broad role unnecessary. The access review should compare current responsibilities with current permissions rather than approving the previous decision automatically. When no built-in role fits a legitimate requirement, the administrator should document the gap and follow the organization's custom-role process. Creating a custom role is a design choice with maintenance costs, not the first reaction to a single unexplained failure. Existing specialized roles and the precise failed action should be examined first.

3.4 Assigning Permissions Through Scope and Inheritance

Scope defines the reach of an assignment

An Azure role assignment combines a principal, a role definition, and a scope. Scope identifies the resource boundary within which the assignment applies. Azure provides management-group, subscription, resource-group, and resource scopes. These form a hierarchy rather than four unrelated choices. An assignment at a parent scope can apply to resources below it, making scope selection as important as role selection.

A Reader assignment on one resource group is narrower than Reader on the subscription containing that group. The role definition is identical, but the reachable resource set differs. This is why an access request should include the target boundary before the administrator opens the role picker. Choosing the role first and accepting whichever scope happens to be open can grant access far beyond the task.

Resource IDs provide the precise form of scope used by APIs and command-line tools. A resource-group ID includes the subscription ID and resource-group name. A resource ID adds the provider namespace, resource type, and resource name. The full string matters because similarly named resources can exist in different subscriptions or groups. A script should use an inspected resource ID rather than reconstructing an uncertain target from memory.

Inheritance carries grants downward

A role assigned at a subscription can affect resource groups and resources within that subscription. A role assigned at one resource group does not grant access to a sibling resource group. This downward relationship allows central administration without repeating the same grant on every resource. It also means that a local resource's assignment list may not tell the whole story unless inherited assignments are included.

Consider a user with Reader at a subscription and Contributor on one development resource group. The user can read supported configuration throughout the subscription and perform Contributor operations in the development group. The narrower Contributor assignment adds capabilities inside that boundary. It does not convert the subscription-level Reader assignment into Contributor for sibling groups. The effective result follows the applicable grants for each target resource.

Now reverse the roles: Contributor at the subscription and Reader on one resource group. The group-level Reader assignment does not reduce the broader Contributor grant. Azure RBAC grants are generally additive, subject to applicable denies and conditions. A narrow read assignment is not a mechanism for subtracting inherited write access. To reduce the access, the administrator must examine the broader grant and redesign or remove it at the appropriate source.

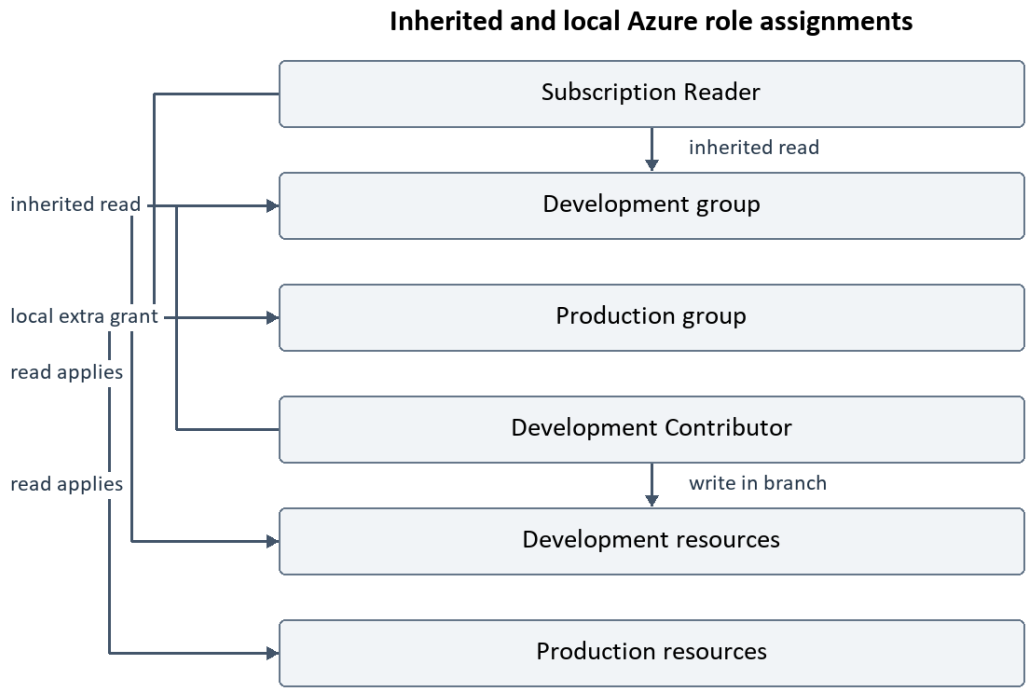


Figure 3.1 Scope inheritance and additional grants

Reader inherited from a subscription applies to both branches. A Contributor assignment at the development resource group adds operations only in that branch; a narrower Reader grant would not subtract a broader Contributor grant.

The figure shows a subscription grant flowing to child resources and a separate resource-group grant affecting only its branch. These arrows describe authorization inheritance. Two resources can communicate without sharing a resource group, and two resources in the same group can have no permitted network path. Scope organizes management permission; networking controls packet reachability.

Choose scope from operational responsibility

An administrator should ask which resources the principal is responsible for now, not every resource the principal might someday encounter. A team that supports one application can receive roles on that application's resource group if the grouping matches its lifecycle and responsibility. A central platform team may need broader scope for a documented duty. The correct scope is the narrowest one that expresses the actual responsibility without creating an unmanageable collection of exceptions.

Very narrow assignments can become difficult to maintain if a workload consists of many related resources and frequent replacements. Very broad assignments can expose unrelated systems. The design should balance those concerns using the resource hierarchy. A well-designed resource group can offer a useful middle ground: related resources share an administrative boundary while unrelated applications remain outside it. This is one reason resource organization and access design belong together.

Assignments to groups can separate personnel changes from resource changes. The role remains on the group at the chosen scope, while approved membership changes control who benefits. This reduces repeated role edits when team members join or leave. It also makes the group's membership process part of the authorization design. A group with uncontrolled membership is not made safe merely by a carefully chosen role definition.

Create and verify an assignment deliberately

A standard assignment procedure starts at the intended target scope. The administrator opens Access control, selects the appropriate role, identifies the principal by more than its display name, and reviews the final scope before submission. The resulting record should include the assignment identifier, principal object ID, role, and scope. Those values make removal and audit work more precise than a screenshot of a friendly name alone.

The following Bash example shows how to obtain a resource-group scope. It performs no assignment itself:

```
scope=$(az group show --name rg-compass-ch03 --query id -o tsv)
az role assignment list --scope "$scope" --include-inherited -o table
```

The expected output lists assignments visible to the caller at that scope, including inherited entries requested by the command. The output is an inspection aid, not proof that every possible group membership and condition has been fully evaluated for a particular user. The portal's Check access view and the actual operation test provide complementary evidence. A reliable investigation uses the tool that answers the specific question.

Assignment authority has its own scope

The person creating a role assignment must have the necessary access-management permission at the target scope or a suitable ancestor. Permission to manage one resource group does not automatically authorize assignments in a sibling group. Conditions can further limit which roles or principals the administrator may choose. A disabled Add role assignment action therefore requires checking the caller's effective authority, not simply the requested recipient's identity.

This creates two separate principals in the operation: the caller who is making the assignment and the recipient who will receive it. An error can concern either one. The caller may lack authorization, while the recipient may be absent from the expected tenant or have an incorrect object ID. An accurate troubleshooting record names both. Saying that “the user has no permission” is ambiguous when two users participate in the change.

Moving resources changes the surrounding context

Resource moves can alter which parent assignments apply. A resource moved into another group can inherit that group's authorization context. Resource-level assignments and move support need to be assessed according to the resource and supported operation; the administrator should not assume that every reference remains valid. The important planning step is to compare effective access before and after the move, including inherited grants.

The same principle applies when a subscription moves between management groups. The hierarchy change can introduce or remove inherited access and policies. A seemingly organizational change can therefore affect actual operations. The move request should include a review of both the old and new ancestry, not just the names of the containers. The administrative tree is an active control structure.

Remove the correct source of access

If a user retains access after a direct assignment is removed, another assignment may still apply through a group or ancestor. The administrator should inspect the complete relevant path before concluding that removal failed. Repeatedly removing and recreating the direct assignment does not eliminate an inherited grant. The correct action depends on the source that still authorizes the operation.

Inherited assignments are managed at their originating scope. A local administrator cannot reliably neutralize an inherited broad grant by adding a narrower role below it. If the broader assignment is inappropriate, its owner must review it. That review may produce a more specific group, a narrower scope, or a different role. The goal is to make the intended boundary explicit instead of layering contradictory-looking assignments.

Scope and inheritance make Azure authorization scalable, but they also make local views incomplete unless ancestry is considered. An administrator who can explain the principal, role, source scope, and target resource can usually explain why a grant applies. That explanation is the starting point for testing effective access and resolving the cases where a visible assignment still does not produce the expected result.

An assignment inventory is therefore most useful when it includes its origin. A flat report that lists only principal and role hides the difference between a resource-specific exception and a subscription-wide grant. Including the full scope and assignment identifier makes review decisions actionable. The reviewer can identify the owner of the source boundary and remove the exact obsolete assignment without confusing it with another legitimate grant to the same principal.

3.5 Explaining and Troubleshooting Effective Access

Effective access is a result, not a single row

Effective access is the permission outcome for a principal attempting an operation on a particular target under the applicable conditions. A role assignment is one input to that result. Group memberships, inherited scopes, additional grants, deny assignments, conditions, and activation state can also matter. A useful explanation therefore follows the actual request rather than stopping at the first familiar role name in a list.

The investigation begins by capturing the failed operation accurately. The administrator records the principal, tenant, resource ID, client, time, and error details. A report that “Azure is denying access” lacks the information needed to distinguish a wrong account from a missing action or a network block. Precise evidence reduces the number of changes required and makes the eventual correction easier to verify.

It is helpful to separate what is known from what is assumed. A role assignment may be visible, but its principal could be a similarly named account. The scope may look correct, but it could belong to another

subscription. The user's group membership may be present, but the current session may not reflect a recent change. Each assumption should be tested with an identifier or an observed operation.

Start with the principal and target

The first check is whether the request comes from the intended principal. Interactive users can have several organizational accounts or guest representations. Automated workloads can use an application identity instead of the human who deployed them. A portal's signed-in display name is insufficient when several objects have similar names. The object ID and tenant establish which identity the service is evaluating.

The second check is the target. Resource IDs reveal the subscription, group, provider, type, and name. A script using an old resource ID can fail even when a new resource with the same short name exists elsewhere. The administrator should compare the requested ID with the inspected resource, rather than searching by name and assuming the first result is equivalent. Exact targets prevent accidental permission changes on unrelated resources.

Once identity and target are confirmed, the operation can be mapped to the required permission. The error may name a provider action, which helps distinguish management-plane authorization from a service's data authorization. The administrator then compares that action with the applicable role definitions. A missing operation can be addressed with an appropriate role or task redesign; it does not automatically justify a subscription-wide assignment.

Read direct, group, and inherited grants together

The Access control page's Check access feature helps inspect a principal's assignments at a resource scope. It can reveal roles that originate higher in the hierarchy and roles obtained through membership. The administrator should distinguish the assignment's origin from the resource currently being inspected. A role displayed on a virtual machine may actually be assigned at the subscription and managed there.

Multiple grants generally combine to provide the allowed operations. If a user has Reader directly and Contributor through a group at the same target, the Reader assignment does not cancel the group's write capability. Removing only the direct Reader assignment would not remove Contributor access. A complete explanation identifies every relevant grant that supports the operation, especially when the goal is to reduce access.

Conditions can make an assignment more selective. An assignment visible in a list might apply only when its supported condition is satisfied. The administrator should inspect the condition rather than reducing the explanation to the role name alone. The same care applies to privileged eligibility and activation. A record that describes potential access is not necessarily an active authorization at the moment of the failed request.

Recognize denies and non-RBAC controls

A deny assignment can block actions even when a role grants them. In generally available scenarios, Azure services create and manage these assignments for supported protections; they should not be treated as ordinary custom deny rules that every administrator can freely create. Deployment stacks are

one context where managed deny behavior can appear. The investigation should identify the owning mechanism and supported management path.

Resource locks and Azure Policy are separate controls that can also block a management operation. An RBAC review might correctly show that the user has the required action while the operation is still prevented. The error and activity evidence should identify whether the refusal comes from authorization, a lock, policy, or another service requirement. Granting a broader role is ineffective when the blocking control does not depend on that extra grant.

Data-service failures require the same layered approach. A storage data role does not bypass a firewall or repair an incorrect DNS path. A valid network connection does not grant blob-read permission. A service may report a failure that requires both identity and network context to interpret. The administrator should preserve the actual request method, including whether the client uses Entra authorization, a key, or another supported credential.

Account for propagation without using it as an excuse

Permission and membership changes can take time to propagate, and existing sessions can retain earlier state. A fresh sign-in or token refresh may be appropriate after an assignment change. However, propagation should be considered after verifying the configuration, not used to explain every persistent failure. Waiting cannot correct an assignment made to the wrong principal or at the wrong scope.

A disciplined test records when the change was made and when each verification occurred. The administrator performs the same minimal operation with the intended identity and notes the result. If the documented propagation behavior has been considered and the failure persists, the investigation returns to the specific action, scope, condition, and control layer. Repeatedly changing several variables during the waiting period destroys the ability to identify the cause.

A worked troubleshooting example

An operator can view a storage account but cannot read its blobs using Entra authentication. The administrator confirms the operator's object ID and the storage account's resource ID. Reader appears at the resource-group scope, explaining the successful configuration view. The data operation requires a suitable storage data role, which is absent. The narrow correction is to grant the approved data access at the required storage scope and then test that same authentication method.

If the corrected assignment is present but the blob request still fails, the administrator checks the error and network controls. A storage firewall might reject the client, or a private-endpoint name might resolve incorrectly from that network. The new failure does not invalidate the role distinction; it reveals another required condition. A good diagnosis can contain more than one cause while keeping the evidence for each separate.

Evidence	Interpretation	Appropriate next check
Sign-in fails before a resource request	Identity establishment is incomplete	Account, methods, and applicable sign-in controls

Evidence	Interpretation	Appropriate next check
Required action absent from applicable roles	Authorization gap	Suitable role and scope
Correct action but lock error	Management protection applies	Lock type and authorized removal process
Data role present but network rejection	Reachability or service network restriction	Endpoint, DNS, and firewall configuration

Close the incident with a bounded explanation

The final record should explain the cause, changed control, resulting scope, and verification. It should also record temporary diagnostic permissions that were removed. A statement that the issue was fixed by “adding access” is too vague for later review. A useful statement identifies the missing operation and why the selected role and scope now satisfy the approved requirement.

The administrator should test the intended boundary again after cleanup. Temporary broad assignments can make a test pass while hiding a still-incomplete narrow configuration. Removing those assignments and repeating the minimal operation confirms that the final design stands on its own. Effective-access troubleshooting is complete when the access is both usable and explainable, with no unexplained privilege left behind.

When the investigation cannot yet establish a cause, the record should say which hypotheses remain open. An unexplained failure is preferable to an invented explanation that sends the next administrator in the wrong direction. Escalation should include the minimum reproducible operation, identifiers, time, error, and checks already completed. Those details turn an ambiguous access complaint into a technical question that another team can investigate efficiently.

Guided lab

Objective: Demonstrate that Reader at one resource-group scope permits configuration inspection but not a management change, and distinguish that result from account authentication.

Prerequisites and permissions: Use an approved Azure public-cloud training subscription and an existing dedicated standard test user in its associated tenant. The operator needs permission to create and delete the disposable resource group and to assign and remove Reader at that scope. The test user should have no broader inherited resource-management role in the subscription. Confirm that condition before the lab. No paid compute, storage, or networking resources are created; the resource group and role assignment themselves do not incur resource-consumption charges. Use a locally installed Azure CLI or an approved shell environment.

1. Sign in as the operator and set the intended subscription with `az account set --subscription '<training-subscription-id>'`. Choose an available metadata region using `location='<approved-region>'`. Run `az group create --name rg-compass-ch03 --location "$location"` in Bash. Keep this operator session open for cleanup; do not run its cleanup commands from the Reader session.

2. Obtain the group ID with `scope=$(az group show --name rg-compass-ch03 --query id -o tsv)` and confirm that it contains the intended subscription ID.
3. Set `test_object_id='<test-user-object-id>'`. Create the narrow assignment with `az role assignment create --assignee-object-id "$test_object_id" --assignee-principal-type User --role Reader --scope "$scope"`. Record the returned assignment ID for exact cleanup.
4. Open a separate CLI session authenticated as the test user. Confirm its tenant and subscription context. After allowing the documented authorization propagation behavior and refreshing the session where appropriate, run `az group show --name rg-compass-ch03`. The expected result is readable resource-group configuration.
5. As the test user, attempt `az group update --name rg-compass-ch03 --set tags.AccessTest=reader`. The expected result is an authorization failure because Reader does not grant the write operation. Do not broaden the role to force success.
6. As the operator, inspect Check access for the test user at the group. Explain the role and source scope. If the update unexpectedly succeeded, investigate broader assignments before proceeding; the intended lab isolation was not satisfied.
7. Remove the exact lab assignment using its recorded ID: `az role assignment delete --ids '<recorded-assignment-id>'`. Do not remove unrelated inherited assignments.
8. Delete the empty lab group with `az group delete --name rg-compass-ch03 --yes`. Confirm its absence using `az group exists --name rg-compass-ch03`, whose expected result is `false`.

The controlled failure in this lab is the Reader user's attempted tag update. No SSPR settings are changed because that service requires a separately approved, licensed pilot and registered test methods. All live execution remains unperformed during authoring. A real lab record must capture its own results rather than copying the expected outcomes as observations.

Chapter synthesis

Authentication identifies the principal, while authorization evaluates the principal's requested work. SSPR supports recovery of the identity's sign-in capability. Azure roles define operations, scopes define reach, and inheritance explains how grants extend through the resource hierarchy. Effective access is the result of those relationships together with applicable conditions, denies, and other service controls.

A careful administrator can explain both a success and a failure without using vague claims about being an administrator. The explanation names the principal, action, resource, scope, and evidence. That precision makes access easier to grant, review, and remove, and it provides a sound basis for the resource organization developed in Chapter 4.

Review questions

03-Q1. A user signs in successfully but a resource update returns an authorization error. Which statement is most accurate?

- **A.** The password must be wrong
- **B.** Successful authentication does not establish permission for the update
- **C.** SSPR must be disabled
- **D.** The tenant needs to be recreated

03-Q2. A workload uses its managed identity to read blobs. The human deployer has Owner but the workload is denied. Which principal's data permissions should be investigated first?

- **A.** Every directory guest
- **B.** The deployment tool's publisher
- **C.** The managed identity making the request
- **D.** The subscription's display name

03-Q3. An ordinary user belongs to an SSPR-enabled pilot group but has no acceptable registered verification methods. What is the most direct missing prerequisite?

- **A.** Completion of supported method registration
- **B.** An Azure Contributor role
- **C.** A public IP address
- **D.** A new resource group

03-Q4. Cloud-only pilot users reset passwords successfully, while identities whose passwords are managed on premises fail. Which investigation best matches the pattern?

- **A.** Increase every user's Azure scope
- **B.** Rename the pilot group
- **C.** Delete the successful test accounts
- **D.** Inspect password authority and the supported writeback configuration

03-Q5. A principal needs to inspect a resource group's configuration without changing resources. Which candidate is the narrowest appropriate choice among these?

- **A.** Owner at subscription
- **B.** Reader at the resource group
- **C.** Contributor at subscription
- **D.** User Access Administrator at subscription

03-Q6. A user has Contributor at subscription scope and Reader on one child resource group. What does the child Reader assignment do?

- **A.** It removes all inherited write access
- **B.** It converts Contributor into a data-only role
- **C.** It does not subtract the inherited Contributor grant
- **D.** It prevents all subscription access

03-Q7. An action is excluded through NotActions in one role, but another applicable role grants it. In the absence of a deny or other restriction, what is the correct interpretation?

- **A.** NotActions is not a universal deny; the other grant can allow the action
- **B.** The first role always wins alphabetically
- **C.** All roles become invalid
- **D.** The user must change passwords

03-Q8. A user has the required management action but the error identifies a read-only resource lock. What is the appropriate next focus?

- **A.** Grant Owner to bypass every lock
- **B.** Reset the user's password
- **C.** Add a storage data role regardless of resource type
- **D.** Investigate the lock and the authorized process for changing it

Further reading

- Azure RBAC overview
- Azure role definitions
- Scope and inheritance
- Enable self-service password reset
- SSPR licensing
- Check access to an Azure resource
- Troubleshoot Azure RBAC
- Deny assignments

Chapter 4: Giving Resources a Clear Home

Resource organization is an operating decision. It determines where administrators apply permissions, how changes are grouped, which policies are inherited, and what is removed when a workload ends. Giving Resources a Clear Home builds on the access model from Chapter 3 and turns the Azure hierarchy into a practical structure for daily work. The chapter develops management groups, subscriptions, resource groups, naming, tags, and locks as related tools with distinct purposes.

Learning objectives

The reader should be able to explain management-group inheritance, distinguish resource and billing administration, group resources by lifecycle, plan metadata that supports operations, and choose locks without confusing them with backups or data-plane protection. The reader should also recognize the consequences of moving resources or changing their place in the hierarchy.

4.1 Organizing Management Groups

A governance scope above subscriptions

A management group provides a scope above Azure subscriptions. It allows an organization to apply governance across several subscriptions without repeating every assignment independently. Azure Policy and Azure role assignments can inherit through this hierarchy. The management group does not host a virtual machine or replace a resource group. Its job is to organize the subscriptions and their shared governance relationships.

All subscriptions within a management-group hierarchy must trust the same Microsoft Entra tenant. The tenant supplies the identity context while the hierarchy supplies resource-governance scope. This distinction matters in organizations with several directories. A management group is not a mechanism for silently merging tenants or moving identities between them. The administrator must understand the directory boundary before assuming that subscriptions can be placed together.

A useful hierarchy reflects stable governance needs. Production and nonproduction may need different policies, operational roles, or permitted deployment patterns. A hierarchy based entirely on a frequently changing organization chart can require repeated moves whenever a department is renamed. The administrator should identify which differences actually require inherited controls. Those differences are more durable than labels chosen only to resemble a management presentation.

Structure follows common requirements

Suppose several production subscriptions require a shared set of resource-location restrictions and central audit visibility. A production management group can provide a common assignment scope. A separate training branch can use different restrictions suited to disposable learning resources. The purpose is not to create a hierarchy because a diagram looks tidy. It is to make common control requirements explicit and maintain them consistently.

The hierarchy should be no more complex than the organization's governance model requires. Every extra level introduces another place where assignments can originate. A deep tree can be difficult to explain during an incident because the effective policy and role set comes from several ancestors. A simpler tree with documented purposes often makes change impact easier to predict. Hierarchy depth is an administrative cost as well as a design option.

Management groups are also not a replacement for tags. A hierarchy gives a resource one ancestry path through its subscription, while tags can describe several independent properties such as owner, application, environment, and cost center. Trying to encode every reporting dimension into the tree produces a rigid structure. The administrator should use the hierarchy for inherited control and metadata for additional classification.

Inheritance is the main behavior to understand

A policy assignment at a parent management group can affect resources in descendant subscriptions. An Azure role assignment at that parent can likewise grant applicable access below it. This makes central administration efficient, but it also increases the impact of a mistake. A broad assignment near the root can affect more resources than the person editing one subscription expects. Reviews should therefore state the complete target population.

An inherited assignment remains associated with its originating scope. An administrator inspecting a child subscription may see its effect without having authority to change the parent assignment. The correct response is to identify the origin and its owner, not to create a contradictory-looking local setting and assume inheritance has been canceled. Policies and role assignments have their own combination rules, which must be understood separately.

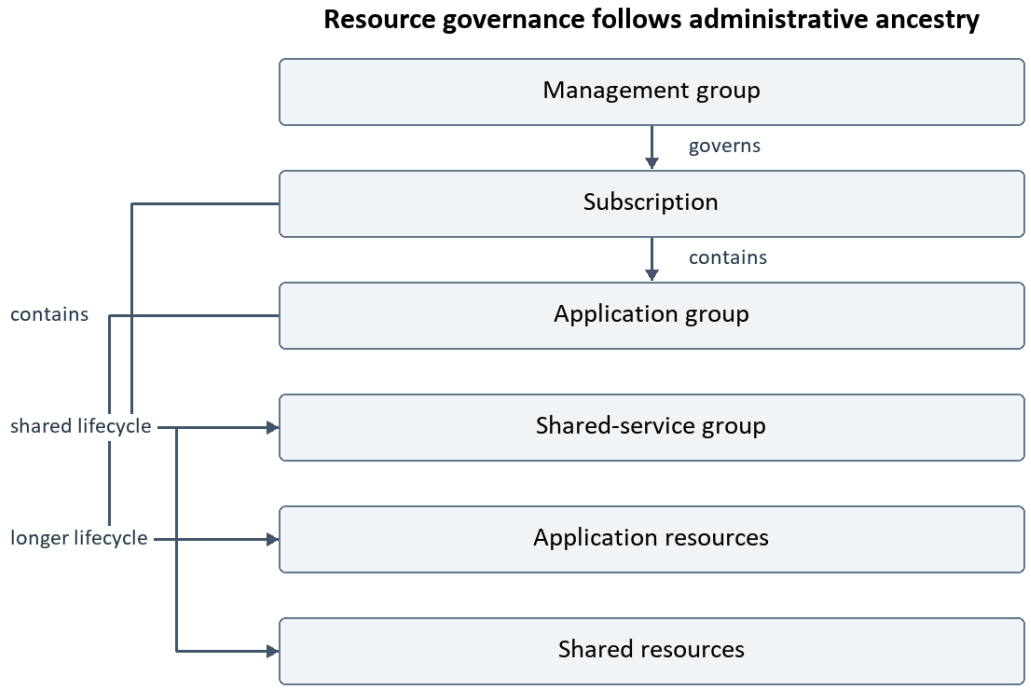


Figure 4.1 Management hierarchy and inherited controls

Management groups and subscriptions establish inherited governance. Resource groups should reflect compatible ownership and lifecycle; shared services need not be deleted with a temporary application.

The diagram emphasizes that governance flows through the subscription hierarchy. Resource groups beneath a subscription receive the relevant inherited context even if their resources are in different Azure regions. A hierarchy branch is an administrative relationship, not a geographic placement. Region choices and network topology must be designed alongside the hierarchy rather than inferred from it.

A management-group move is a control change

Moving a subscription into another branch can change the policies and roles it inherits. The resource inventory may remain the same while the permitted management behavior changes. A deployment that worked under the old ancestry can fail under the new policy context. An operator can gain or lose inherited visibility. These consequences make a hierarchy move more substantial than moving a folder in a personal document collection.

Before a move, the administrator compares the current and proposed ancestry. The comparison includes applicable policy assignments, exemptions where relevant, role assignments, and the teams responsible for those controls. The expected differences should be reviewed with workload owners. A test subscription or controlled pilot can expose incompatible assumptions before a production subscription is moved. The objective is to make the inherited change visible before it becomes operational.

The move also requires appropriate permissions under the documented management-group rules. Authority over the destination alone may not be enough to complete every move scenario. The

administrator should consult the current prerequisites and use an approved identity. A failed move should not be solved by casually assigning broad root permissions. The exact missing operation and scope should be identified first.

Keep the root deliberately quiet

Controls near the tenant root management group have a wide reach. They are suitable for requirements that truly apply across the hierarchy, but a requirement that applies only to one workload class belongs lower down. Broad placement should be justified by broad applicability. Otherwise, the organization accumulates exclusions and exceptions that make the supposedly simple global rule difficult to understand.

Root-level administration should have clear ownership and change review. This does not mean every routine subscription task needs central approval. It means the organization recognizes the difference between a local resource change and a shared control affecting many subscriptions. Delegation works best when local teams have usable scopes and the central team maintains a small, comprehensible baseline above them.

A practical design exercise

An organization operates customer-facing production workloads, internal test environments, and short-lived training exercises. The administrator first lists the controls common to all three: perhaps an approved identity model, central visibility, and required ownership metadata. Next, the administrator lists differences, such as deployment regions or restrictions on public exposure. Only the differences that need inherited enforcement should influence the branch design.

The resulting design might use a shared root baseline with separate production and nonproduction branches, while training remains within a documented nonproduction subscription pattern. The exact number of branches is less important than their stated purpose. A hierarchy description should explain why each branch exists, which subscriptions belong there, and which control differences it introduces. If those answers are missing, the tree may be decorative rather than useful.

Design question	Management-group contribution	Complementary tool
Apply a baseline across subscriptions	Common inherited scope	Azure Policy and Azure RBAC
Identify a workload owner	Provides broad administrative context	Tags and an ownership register
Group resources for deletion	Does not provide a workload deletion unit	Resource group
Separate resource consumption	Organizes subscriptions	Subscription and billing arrangement

This comparison prevents the hierarchy from being asked to solve every organization problem. A management group can establish shared governance, but it cannot replace a deployment boundary, a

cost-allocation process, or a network design. Good administration assigns each tool a clear responsibility and documents the relationships between them. The hierarchy then supports operations instead of becoming another unexplained layer.

Verify the structure as it evolves

Periodic review should compare the actual hierarchy with its documented purpose. New subscriptions need a deliberate placement decision. Old branches may remain after a project ends, retaining assignments that nobody actively maintains. The administrator should inspect both the tree and the controls attached to it. Removing an empty branch is straightforward only after confirming that no intended governance relationship depends on it.

Reports should include management-group identifiers, display names, parent relationships, subscription membership, and assignment ownership. Identifiers make the inventory precise, while descriptions make it understandable. A name change should not erase the reason a branch was created. Keeping both technical and business context in the record makes future migrations less dependent on the memory of the original designer.

Management groups make common rules easier to maintain when each branch has a documented purpose and owner. The administrator should be able to identify which controls apply to a subscription and where they originate. That inherited context accompanies the subscription throughout its resource and consumption lifecycle.

4.2 Administering Subscriptions

A subscription is a resource-management boundary

An Azure subscription provides a boundary for resource management, resource consumption, and many service limits. Resources are created within subscriptions, normally through resource groups. The subscription has a unique ID that should be used to confirm the target of an administrative operation. A display name helps navigation, but it is not a safe unique identifier for automation or change records.

A subscription trusts a Microsoft Entra tenant for identities. Users from that directory, including supported external representations, can receive Azure roles at the subscription or narrower scopes. The trust relationship does not automatically give every tenant member access. The administrator must still inspect role assignments. This keeps the distinction between identity and resource authorization intact as the book moves from directory objects to resource organization.

Organizations use separate subscriptions for several reasons: ownership, environment isolation, cost visibility, service limits, or administrative boundaries. The reason should be documented because it influences how resources are placed and how access is delegated. A subscription created for production isolation should not gradually become a general-purpose container for unrelated experiments. Its purpose is part of its operating contract.

Resource access and billing authority differ

Azure resource roles and billing roles are not identical. A person who can manage resources may not have authority to change the billing arrangement, and a billing administrator may not automatically

have every desired resource operation. The exact billing scope and available roles depend on the organization's agreement and account type. Administrators should identify that arrangement before assuming that a familiar title grants a particular financial or administrative action.

This distinction is visible when an operator can create a resource group but cannot manage an invoice or transfer billing ownership. Granting another resource role may not solve the billing task. Conversely, a person who can inspect billing information may still need an Azure role to read resource configuration. The request should name the desired operation and the service responsible for authorizing it.

A subscription's lifecycle also matters. Disabled or otherwise restricted subscription states can affect resource operations independently of the user's role. An administrator should inspect subscription status when a broad set of deployments fails unexpectedly. The role assignment may be correct while the subscription cannot accept the requested activity. The billing or subscription owner may need to resolve the underlying state through the supported process.

Select context before every resource operation

Azure CLI and PowerShell can remember a previously selected subscription. In an environment with several subscriptions, an unqualified command can therefore act in a context different from the one the operator intended. A resource name that exists in both environments makes the mistake harder to notice. Explicit context checks are a small investment compared with recovering from a change made in the wrong subscription.

The following Bash commands inspect available context and select a known subscription:

```
az account list --query '[].{name:name,id:id,tenant:tenantId}' -o table
az account set --subscription '<training-subscription-id>'
az account show --query '{name:name,id:id,tenant:tenantId}' -o table
```

The expected output identifies the selected subscription and its tenant. The list reflects what the authenticated account can see; it is not necessarily an inventory of every subscription in the organization. Missing entries can indicate access or tenant-context differences. The administrator should avoid interpreting a partial visible list as proof that a subscription does not exist.

Providers and quotas affect deployment readiness

Azure resource providers expose resource types for services. A subscription may need the appropriate provider registration before certain resource types can be deployed. Provider registration is distinct from the administrator's role and from the availability of a service in a region. A deployment error should be read carefully to determine which prerequisite is missing. Broad access cannot create regional capacity or substitute for a required registration state.

Quotas and service limits also shape what can be deployed. A requested virtual machine configuration can be valid in principle while exceeding the subscription's available regional quota. An approved quota increase is not a guarantee of immediate physical capacity for every size. The administrator should distinguish the configured limit, current consumption, regional support, and actual allocation result. These are related but separate deployment conditions.

Before a planned workload deployment, the subscription readiness review should inspect required providers, relevant quotas, policy restrictions, and permissions. It should also confirm cost ownership

and the target region. This review prevents the deployment window from becoming the first moment anyone discovers that the subscription cannot support the requested configuration. The depth of review should match the workload's significance and complexity.

Provider registration should follow the services actually needed by the workload. Some deployment experiences register providers automatically, while dependencies can still require explicit attention. Registering every available provider is unnecessary. The administrator should identify the provider named by the planned resource types or the deployment error, verify the relevant registration state, and follow the documented registration process with appropriate authority.

Separate move scenarios carefully

Moving resources between resource groups, moving supported resources between subscriptions, changing a subscription's directory association, and changing billing ownership are different operations. They have different prerequisites and consequences. An administrator should not use the word transfer without describing which boundary changes. A resource move does not necessarily change its physical region, and a billing change does not necessarily redesign its identity relationships.

For supported cross-subscription resource moves, Microsoft documents requirements including the relevant tenant relationship and provider support. Each resource type can have additional restrictions and dependencies. The administrator should validate the planned move before the maintenance window and inspect the resulting resource IDs and access context. A generic statement that Azure resources can be moved is too broad to support a safe implementation.

Changing a subscription's associated directory is especially consequential for identity-based access. Existing role assignments and service identities can require repair or recreation according to the documented transfer process. This chapter does not present that operation as routine cleanup. The important AZ-104 habit is to recognize that changing an identity boundary is not equivalent to renaming a subscription or moving an ordinary resource group.

Use subscription boundaries deliberately

Consider two application teams with different production release cycles and cost owners. Separate subscriptions can make ownership and broad administration clearer, while common management-group policies maintain a shared baseline. The teams can still need network connectivity or shared services across those subscriptions. Administrative separation does not automatically provide a complete application architecture. Connectivity and data access require their own design.

Another organization may use one subscription with several resource groups for smaller workloads. That can be workable when the control and consumption requirements are compatible. The administrator should avoid claiming that one subscription per application is universally correct. The boundary should follow the organization's operational needs, with an explicit understanding of the resulting permissions, quotas, cost reporting, and governance inheritance.

Subscription concern	Evidence to inspect
Correct target	Subscription ID and tenant ID

Subscription concern	Evidence to inspect
Resource authority	Applicable Azure role assignments
Financial authority	Agreement-specific billing role and scope
Deployment readiness	Provider state, quota, region, and policy
Lifecycle issue	Subscription status and responsible owner

The evidence should be gathered before changing permissions. A deployment denied by policy, a quota error, and a disabled subscription can all appear to a user as an inability to create resources. Their corrections are different. Reading the actual failure and checking the appropriate subscription property is more effective than applying a broad administrative role and trying again.

Maintain an operating record

A subscription register should identify its purpose, owner, billing contact, tenant, management-group placement, critical policies, and approved use. The register does not need to duplicate every resource detail. It should explain how the subscription is governed and whom to contact when a cross-cutting issue arises. That information makes onboarding, incident response, and eventual closure more reliable.

Before closing a subscription or ending a project, the organization should inventory remaining resources and obligations. Backups, retained data, public endpoints, monitoring destinations, and shared dependencies can survive the apparent end of an application. Deleting visible compute alone is not a complete closure process. The subscription boundary helps organize the review, but the administrator must still examine what the resources actually do and what must be retained.

Reliable subscription administration keeps context, authority, consumption, and lifecycle distinct while connecting them in one operating record. With that boundary established, resource groups can provide the smaller units used for deployment, access, and cleanup within the subscription.

4.3 Managing Resource Groups and Their Lifecycle

Group resources that should be managed together

A resource group is a container for related Azure resources. It provides a scope for management operations, role assignments, policy, tags, deployments, and locks. A resource belongs to one resource group at a time. The group is not a network boundary and does not require every member to be in the same region. Its main value is a coherent administrative and lifecycle boundary.

Resources that are created, changed, and removed together are often good candidates for the same group. A disposable application exercise can place its virtual network, virtual machine, disk, and related components in one group so cleanup can remove the whole exercise. A shared production network with a longer life should not be placed in a short-lived application's group merely because the application uses it. Shared dependencies deserve a lifecycle that matches their actual use.

This principle makes resource-group design more useful than grouping only by resource type. A group containing every virtual machine in the organization may combine unrelated owners and deletion schedules. A workload-oriented group can provide a clearer responsibility boundary. There are legitimate shared-service designs, but the grouping should explain who owns the resources and how they change together rather than relying on a simple category label.

Location describes group metadata

A resource group has a location used for its metadata. The resources inside it have their own location settings where applicable. Creating the group in one region does not force every contained resource into that region. An administrator planning resilience or data placement must inspect the resource properties themselves. The group location is an important setting, but it is not a substitute for the workload's regional design.

This distinction can surprise a learner who creates a group in one region and later deploys a resource elsewhere. The configuration is not automatically invalid simply because the names differ. The relevant questions are whether the resource supports the chosen region, whether policy permits it, and whether the application design supports the arrangement. The group remains the administrative container despite the resource's separate placement.

The group provides a useful scope

An Azure role assigned at the group can apply to its contained resources. A lock on the group can affect management operations on child resources. A policy assignment can evaluate applicable resources in that scope. Those behaviors make the group a practical boundary for application teams. They also mean that moving a resource into a group changes more than its place in a list.

Tags on a resource group do not automatically become tags on its resources. A policy can implement desired tag behavior where supported, but the default metadata relationship should not be assumed to provide inheritance. This difference is important for cost reporting: tagging only the group may not produce the expected resource-level cost classification. The administrator should inspect the actual resource tags and the reporting behavior being used.

Deployment and deletion are related but not identical

A deployment record describes an operation submitted through Azure Resource Manager. It can include a template, parameters, and operation results. The resource group provides a place to inspect relevant deployment history. A successful deployment record is useful evidence, but it does not guarantee that the application is healthy or that later manual changes have not altered the resources. Configuration success and workload success require different checks.

Deleting a resource group removes the resources within it through the supported management process. This makes a disposable lab easy to clean up, but the same convenience can be dangerous when unrelated resources share the group. Before deletion, the administrator inventories the contents and dependencies, checks locks, and confirms that retention requirements have been met. The group name alone is not adequate evidence that everything inside is disposable.

Some resources contain protected data or have service-specific deletion requirements. A group deletion can fail or require additional handling when those protections apply. The administrator should read the failure and follow the relevant service process rather than repeatedly issuing deletion commands. A resource group is a common management container, not a guarantee that every service has identical cleanup behavior.

Moving resources requires dependency awareness

Azure supports moving many resource types between groups and, in supported scenarios, subscriptions. Support is resource-specific. A move can require related resources to move together or satisfy particular conditions. The administrator should use the documented move-support matrix and validation process for the actual resource types. A successful move of one kind of resource does not prove that a different workload can be moved the same way.

Resource IDs include the group path, so a move can change identifiers even when the resource's short name stays the same. Scripts, dashboards, alerts, or external records that store those IDs may need updates. This is an important reason to inventory dependencies before a move. A resource can continue operating while management references elsewhere become stale. Operational validation should therefore include the tools that manage and observe it.

During a supported move, management operations on the source and target groups can be restricted while Azure performs the change. The resource's runtime behavior and the management lock are distinct concerns. The maintenance plan should describe expected restrictions and the validation needed afterward. It should not promise universal zero impact without checking the services and dependencies involved.

A lifecycle example

A team creates a temporary test environment for an application release. The test resources have one owner, one purpose, and one planned removal date, so a dedicated resource group is appropriate. The shared identity infrastructure and production monitoring workspace remain outside the disposable group because their lifecycles differ. References between the resources are documented, including any monitoring destinations and network dependencies.

At the end of testing, the administrator first exports or retains the required results. The group inventory is compared with the original deployment record to identify manually added resources. The owner confirms that no component has been repurposed for another project. Only then is the group deleted, and the administrator verifies the result. Cleanup is a controlled lifecycle action, not an assumption based on the word test in a name.

Inspect before changing

The following Bash inspection lists resources in a known group without changing them:

```
az resource list --resource-group rg-compass-ch04 \
  --query '[].{name:name,type:type,location:location,id:id}' -o table
```

The expected output gives a compact inventory. It should be compared with the intended workload rather than treated as a complete dependency graph. Resources outside the group can depend on those

inside, and some service-level objects may require separate inspection. The list is a starting point for lifecycle review, with resource IDs providing exact targets for further checks.

A useful group record includes owner, application, environment, deployment source, retention expectations, and cleanup procedure. The record should also identify shared resources that must remain after the workload ends. This information makes it possible for another administrator to perform maintenance without reconstructing the original design from portal screenshots. Clear ownership is especially valuable when several teams can deploy into the same subscription.

Troubleshoot group-level failures

If deployment fails only in one resource group, the administrator compares its inherited policies, locks, and access assignments with a working group. The resource configuration may be valid while the target scope imposes different requirements. If deletion fails, the administrator checks the specific blocking resource and lock rather than assuming the subscription is broken. Scope-specific evidence makes the investigation manageable.

If a resource seems missing after a move, the administrator searches by its new group and inspects the resulting ID. Old bookmarked URLs or scripts may still reference the previous path. The correction is to update the reference and verify the new scope, not recreate a duplicate resource immediately. A lifecycle-aware view recognizes that resource identity in management paths can change even while the underlying service continues operating.

A move-validation result should be retained with the change record. It establishes what Azure checked for the proposed move, but it does not replace application-level testing or an inventory of external references. The administrator still verifies monitoring, automation, and inherited access afterward. This separation keeps a successful platform operation from being overstated as proof that every surrounding workflow has also adapted.

Resource groups work best when their boundaries match a real operating relationship. They make deployment, delegation, and cleanup easier because the grouped resources belong to a coherent lifecycle. That relationship should remain visible in naming and metadata, which provide the everyday clues administrators use to navigate the environment.

4.4 Making Naming and Tags Useful

Metadata should answer operational questions

A useful resource name helps an administrator recognize the resource in a list, command, alert, or report. A useful tag provides structured metadata that can be queried or used by supported governance and reporting tools. Neither should be designed merely to look professional. The starting point is the question the organization needs to answer: who owns this resource, which workload uses it, which environment is it in, and what process governs its lifecycle?

Names and tags have different strengths. A name is visible in many interfaces and often forms part of the resource ID or endpoint. It may be difficult or impossible to change without replacing the resource. Tags are more flexible metadata, although support and reporting behavior vary by resource type. Stable identity belongs in the name; changeable classification often belongs in tags or an external inventory.

A naming convention should remain short enough to use and precise enough to distinguish resources. A resource-type abbreviation, workload identifier, environment marker, and sequence can be useful components. The exact convention is an organizational choice, but it must respect each service's naming restrictions. A convention that works for a virtual network may fail for a storage account because the allowed characters and uniqueness requirements differ.

Uniqueness has a scope

Some Azure resource names need to be unique within a resource group, while others have broader uniqueness requirements. Services that expose public endpoints can require globally unique names within the relevant service namespace. The administrator should check the resource-specific rule rather than assuming that a unique name in the local portal search is sufficient. A deployment can fail because another customer already uses the desired public service name.

For repeatable labs, a user-selected unique suffix makes global names practical without embedding personal information. The suffix should be stable enough to identify the lab's resources and compliant with the service's character rules. A script should validate the assembled name before deployment. Merely adding punctuation to make a name look different may violate a service's allowed format.

Names should also avoid confidential information. Resource names can appear in logs, URLs, cost reports, and support records. A customer secret, personal identifier, or sensitive project description should not be encoded simply because the name field accepts text. The organization should assume that operational metadata is more widely visible than the protected data stored inside a service. Clear names do not require confidential detail.

Define a small tag vocabulary

Tags are key-value pairs. A small required vocabulary might include `Application`, `Environment`, `Owner`, and `CostCenter`, with documented values and ownership rules. The vocabulary should be agreed with the teams that use the resulting reports. If one team records `Prod`, another `Production`, and another `Live`, a simple environment filter may produce an incomplete view. Consistent meaning matters more than the visual style of the tag.

Tag keys and values should be treated consistently even where the platform's comparison behavior differs. Resource-provider behavior and reporting tools can have their own details, so a house convention reduces ambiguity. The administrator should avoid relying on case variations to represent distinct business categories. A controlled vocabulary makes searches, policy checks, and cost allocation easier to explain.

The owner tag should identify an accountable team or approved contact mechanism, not a secret or an unmanaged personal note. Individual names can become stale when staff move roles. A team alias or inventory reference can provide a more durable route, depending on organizational practice. The important requirement is that the value leads to someone who can make a decision about the resource.

Metadata item	Good purpose	Weak use
Resource name	Stable resource identity and recognition	Frequently changing cost allocation

Metadata item	Good purpose	Weak use
Environment tag	Search and governance classification	A substitute for actual isolation
Owner tag	Route operational questions	Store credentials or sensitive personal data
Cost-center tag	Support allocation reports	Assume all historic charges are automatically relabeled

The weak uses matter because metadata can describe a control without implementing it. A resource tagged `Private` is not necessarily unreachable from the internet. A group named `Production` does not automatically apply production policies. Reports should distinguish declared classification from observed configuration. Tags support administration, but the relevant service settings and policies implement the actual technical behavior.

Tags do not automatically inherit

A tag placed on a resource group does not automatically appear on every resource in that group. The administrator must apply resource tags directly or use a supported policy approach to produce the desired behavior. This distinction should be verified in the resource's actual properties. Assuming inheritance can leave a resource inventory or cost report incomplete even though the group page looks perfectly tagged.

Azure Policy can audit, require, or modify supported tag behavior according to its definition and effect. The policy should be tested for the intended resource types and deployment paths. Existing resources may need a remediation process rather than changing instantly when the assignment is created. The next chapter examines that distinction in detail. Here the important point is that metadata propagation is a designed behavior, not a default consequence of grouping.

Tags also have service-specific support and limits. A resource inventory may include types that do not support tags in the same way. The administrator should inspect the current support guidance before enforcing a universal rule. A rule that cannot sensibly apply to a resource type should not become a stream of unexplained exceptions. Governance should reflect the platform's actual capabilities.

Update tags without losing unrelated metadata

Tag update methods can replace the entire tag set or merge specific changes, depending on the tool and operation. The administrator must know which behavior the chosen command performs. Replacing tags to add one cost center can accidentally remove ownership and environment values maintained by another process. A pre-change read and a post-change comparison make the impact visible.

For example, a resource may already contain deployment metadata and an operations contact. A finance update should change the approved allocation field while preserving unrelated keys unless the change explicitly includes cleanup. The record should describe which keys are owned by which process. Competing automation that repeatedly overwrites the same tag can create unstable reports even when every individual update succeeds.

A practical procedure is to read the current tag set, validate the intended key and value, choose a merge or replacement operation deliberately, apply the change, and inspect the result. The same procedure is useful in bulk updates. A pilot on a disposable resource can confirm command behavior before an organization-wide metadata repair. The goal is predictable state, not merely a successful command exit code.

Use tags as evidence with limitations

Cost reports can use supported resource tags to help allocate spending, but administrators should understand when tags are captured and how historical charges are represented. Adding a tag now should not be assumed to rewrite all past billing records. The reporting tool and billing data determine the actual behavior. A cost-allocation design should state its treatment of untagged and historical usage rather than hiding those gaps.

Tags can also support inventory queries and operational queues. A missing owner tag can identify a resource needing review; an expired project value can identify a cleanup candidate. The candidate still requires verification before deletion. Metadata can be stale, and a resource may have acquired a new dependency. Automated destructive action based on one unverified tag deserves a stronger design than an inventory report.

Troubleshooting metadata drift

Suppose a resource group shows the expected environment tag, but a resource-level report lists several members as unclassified. The first check is whether those resources actually have the tag. If they do not, the administrator investigates the intended propagation process. If they do, the next checks concern reporting scope, refresh timing, and tag support. This sequence separates configuration drift from reporting behavior.

Another common problem is a tag that keeps reverting. The administrator should look for deployment templates, policies, or automation that own the property. Manually correcting the value repeatedly treats the symptom while leaving the source unchanged. The durable fix is to align the authoritative configuration with the approved value. Metadata has a lifecycle and source of authority just as identity properties do.

Names and tags are effective when they make ownership and intent easy to discover without pretending to enforce behavior they only describe. A compact convention, a controlled vocabulary, and a clear update process provide more value than a long collection of optional fields. The result is an environment that another administrator can navigate and understand under ordinary operational pressure.

4.5 Protecting Resources with Locks

A lock protects management operations

An Azure resource lock adds a management restriction at a supported scope. It can help prevent accidental deletion or modification even when a caller has a role that would otherwise permit the operation. Locks apply through the management plane. They are not a general data-retention system, a backup, or a guarantee that an application's data cannot be changed through its normal service interface.

Azure exposes two principal lock levels: `CanNotDelete` and `ReadOnly`. A delete lock allows permitted management changes while preventing deletion. A read-only lock prevents management changes as well as deletion, subject to the platform's operation semantics. The choice should match the risk being addressed. Applying the stronger-looking lock without understanding its effects can interrupt legitimate administration.

The lock should have a clear owner and purpose. A note such as “protect shared production network from accidental group deletion” explains the intended boundary better than an unexplained default name. The owner should know when removal is allowed and how the lock is restored afterward. A lock that nobody understands can turn a routine maintenance task into an avoidable incident.

Inheritance extends the restriction

Locks can be placed at subscription, resource-group, or resource scope. Child resources inherit applicable parent locks, and the most restrictive applicable lock governs the relevant operation. A group-level delete lock can therefore protect contained resources from deletion even if those resources have no lock listed as locally created. The administrator should inspect inherited context when an apparently unlocked resource refuses deletion.

A lock at a broad scope has broad operational consequences. A read-only lock on a resource group can affect several different services, including management operations that appear to be ordinary maintenance. The administrator should test the intended control on representative resources and consult service-specific behavior. The fact that a lock is easy to create does not make its impact simple.

The distinction between lock origin and observed effect mirrors role inheritance. A resource administrator may see the lock's effect without being authorized to remove the lock at its source. The correct process is to identify the owning scope and approved maintainer. Moving to a different client or retrying the operation as an ordinary Contributor does not remove the inherited restriction.

Read-only can affect operations that do not look like edits

Some management operations use request methods that are treated as changes or actions even when the administrator thinks of them as inspection. Listing storage account keys is a familiar example of an operation that can be affected by a read-only lock. The administrator should not equate the everyday word read with every API operation whose result is information. The service's management-operation semantics determine the effect.

This is why a delete lock is often a more targeted choice when the actual requirement is protection from accidental removal. It preserves more routine management activity than read-only. A requirement to freeze configuration may justify read-only, but that decision should include the necessary maintenance and recovery workflows. The control should be tested against the work that must continue while the lock is present.

Requirement	Candidate lock	Remaining concern
Prevent accidental resource deletion	CanNotDelete	Permitted configuration changes can still occur

Requirement	Candidate lock	Remaining concern
Freeze supported management changes	ReadOnly	Some administrative actions can stop working
Protect blob contents from deletion	A resource lock is insufficient by itself	Use service data-protection controls
Recover from a lost workload	A lock is insufficient	Use verified backup and recovery design

The comparison makes the boundary explicit. A storage account can be protected from management deletion while its data still changes through authorized data-plane operations. A virtual machine can remain present while its guest operating system is damaged. Locks reduce a particular class of management mistakes; they do not replace the controls that protect service data or recover workload function.

RBAC and locks work together

RBAC determines whether the principal has the relevant operation. A lock can then restrict an otherwise permitted management action. Granting Owner does not make the lock irrelevant. However, a principal with the appropriate lock-management permissions can remove the lock through a supported change. The design must therefore consider who can manage locks, not merely whether a lock appears in the portal.

Contributor does not generally include the authorization-management operations needed to manage resource locks. The administrator should inspect the exact role definition and scope for the caller performing the lock change. A person who can update a virtual network may be unable to remove its lock. This separation can be useful, but it should be understood before a maintenance window begins.

Locks are also different from Azure Policy. A lock restricts categories of management operations at its scope. A policy evaluates resources or requests against a defined rule and effect. A location policy can reject a resource deployment based on its properties; a delete lock does not express that property rule. The tools can complement one another, but their error messages and correction paths differ.

Plan a maintenance path

A resource that needs occasional replacement cannot remain permanently undeletable without a maintenance process. The approved procedure should identify who removes the lock, which exact lock is affected, what work is performed, and when the protection is restored. The record should preserve the original lock level and notes. Removing all locks in a subscription to fix one failed operation is an unnecessarily broad response.

The maintenance test should include a final protection check. It is easy to remove a lock, complete the urgent change, and forget to recreate it. The work then appears successful while the original accidental-deletion risk has returned. A clear completion criterion includes restored protection where required and verification that no temporary broad permissions remain. The lock is part of the resource's intended state.

A controlled example

An administrator creates an empty training resource group and applies a delete lock. A deletion attempt should fail because of the lock, even when the caller otherwise has permission to delete the group. Removing the exact lock and repeating the deletion should then succeed, provided no other control blocks it. This exercise demonstrates management protection without risking a paid workload or deleting useful data.

If the first deletion unexpectedly succeeds, the administrator checks whether the lock was applied to the intended group and whether it existed before the deletion request. A similar name in another subscription is a plausible cause. If deletion remains blocked after the local lock is removed, the administrator checks inherited locks or other protections. The evidence follows scope and timing instead of assuming the lock feature is unreliable.

Diagnose before removing protection

A failed management operation should be examined for a lock-specific error and the relevant resource ID. The administrator then inspects local and inherited locks, identifies the owner, and determines whether the attempted operation is part of an approved change. The correct response may be to keep the lock and reject an accidental operation. Troubleshooting does not always mean removing the control that produced the failure.

For a storage account, an inability to list keys under a read-only lock should not immediately lead to removing all protection. The administrator can ask whether the workflow should use an identity-based data path that avoids retrieving account keys. If the maintenance task truly requires the blocked management action, the lock change should be scoped and documented. The failure can reveal a design improvement rather than merely an inconvenience.

A resource inventory should record locks alongside ownership and lifecycle information. That makes the protection visible before deployment automation or cleanup starts. Automation should handle a lock failure explicitly and report the blocked resource, rather than looping indefinitely or attempting unrelated privilege changes. A predictable failure with a clear owner is easier to manage than a silent partial cleanup.

Locks complete this chapter's view of resource organization by adding protection to an understandable hierarchy. Management groups establish shared governance, subscriptions define broad resource boundaries, groups collect related lifecycles, and metadata explains purpose. A lock can then protect the intended management state at the right scope, with a documented route for legitimate change.

Guided lab

Objective: Create a disposable resource group, apply useful tags, demonstrate a delete lock, and remove all lab objects.

Prerequisites and permissions: Use an approved Azure public-cloud training subscription. The operator needs resource-group creation and deletion permissions plus lock-management permissions at the lab scope. An appropriately scoped Owner can perform the exercise; Contributor alone should not be assumed to manage the lock. Use Bash with Azure CLI. No virtual machines, storage accounts, or paid

networking services are created. The empty resource group, tags, and lock have no direct resource-consumption charge. A region parameter supplies the resource-group metadata location.

1. Set the intended context with `az account set --subscription '<training-subscription-id>'` and inspect `az account show`. Set `location='<approved-region>'`.
2. Create the group: `az group create --name rg-compass-ch04 --location "$location" --tags Environment=Training Owner=CompassLab Application=Chapter04`.
3. Inspect `az group show --name rg-compass-ch04 --query '{id:id,location:location,tags:tags}'`. Record the exact ID and tag values. Confirm `az resource list --resource-group rg-compass-ch04 -o table` shows no workload resources before attempting the deletion test.
4. Create a delete lock: `az lock create --name lock-compass-ch04-delete --lock-type CanNotDelete --resource-group rg-compass-ch04 --notes 'Disposable lab: demonstrate deletion protection'`.
5. List the group's locks with `az lock list --resource-group rg-compass-ch04 -o table`. Verify the intended lock name and level.
6. Attempt `az group delete --name rg-compass-ch04 --yes`. The expected result is a lock-related failure. Confirm that the group still exists; this deliberate failure is the controlled troubleshooting variation.
7. Remove only the lab lock: `az lock delete --name lock-compass-ch04-delete --resource-group rg-compass-ch04`.
8. Delete the empty group with `az group delete --name rg-compass-ch04 --yes`. Verify cleanup with `az group exists --name rg-compass-ch04`, expecting `false`.

If deletion remains blocked, inspect inherited locks and the returned error. Do not remove subscription-wide protections for the exercise. Record the result and ask the training-subscription owner to resolve the approved lab scope. The lab was not executed during authoring, so its expected outcomes must be verified in the actual training environment.

Chapter synthesis

The Azure hierarchy becomes useful when each level has a clear purpose. Management groups apply common controls across subscriptions. Subscriptions provide resource and consumption boundaries. Resource groups gather resources with compatible ownership and lifecycles. Names and tags make those relationships discoverable, while locks protect selected management operations.

These tools do not replace one another. A tag does not enforce network privacy, a resource group does not determine every resource's region, and a lock does not back up data. Understanding those boundaries allows administrators to combine the tools deliberately. The next chapter adds explicit resource standards and financial observation through Azure Policy, budgets, and Advisor.

Review questions

04-Q1. What is the principal purpose of an Azure management group?

- **A.** Host virtual machine disks
- **B.** Provide a governance scope above subscriptions
- **C.** Replace every resource group
- **D.** Assign product licenses to users

04-Q2. A subscription is moved to another management-group branch. Which consequence must be reviewed?

- **A.** Every resource must change its physical region
- **B.** Every user automatically receives Owner
- **C.** Inherited policies and Azure roles can change
- **D.** All resource names must become globally unique

04-Q3. Which evidence most directly confirms the CLI's selected resource-management target?

- **A.** The subscription ID and tenant ID returned by account inspection
- **B.** The terminal window color
- **C.** The resource owner's surname
- **D.** The number of installed CLI extensions

04-Q4. A subscription has sufficient configured quota for a VM size. What should the administrator still recognize?

- **A.** Quota guarantees physical capacity in every region
- **B.** Quota grants Owner
- **C.** Quota removes location policies
- **D.** Quota does not reserve or guarantee actual capacity

04-Q5. Which grouping best matches a resource-group lifecycle boundary?

- **A.** All resources whose names begin with the same letter
- **B.** Resources created, maintained, and retired together for one workload
- **C.** Every storage account from every business unit
- **D.** All resources that must use identical private IP addresses

04-Q6. A resource group has an Environment tag. What is the correct expectation for contained resource tags by default?

- **A.** They automatically receive every group tag
- **B.** The tag grants network isolation
- **C.** They do not automatically inherit the tag
- **D.** The group becomes read-only

04-Q7. A team wants to prevent accidental deletion while retaining ordinary permitted configuration updates. Which lock is the closer fit?

- **A.** CanNotDelete
- **B.** ReadOnly
- **C.** A tag named Protected
- **D.** A Reader role assigned to an unrelated user

04-Q8. A storage account has a management delete lock. Which claim would be incorrect?

- **A.** The lock can be inherited from a resource group
- **B.** A management deletion can be blocked
- **C.** Authorized lock management can remove the protection through an approved change
- **D.** The lock guarantees that blob contents cannot be deleted through the data plane

Further reading

- Management groups overview
- Azure Resource Manager overview
- Manage resource groups
- Move resources between groups and subscriptions
- Resource tags
- Resource locks

Chapter 5: Governing Change and Controlling Spending

Clear resource organization creates places where standards can be applied. Governing Change and Controlling Spending explains how administrators turn those standards into observable controls without confusing compliance, authorization, and financial reporting. Azure Policy evaluates intended resource rules, budgets make spending visible, and Advisor supplies recommendations for review. Each tool supports a decision; none removes the need to understand the workload being governed.

Learning objectives

The reader should be able to distinguish policy definitions from assignments, explain effects and initiatives, investigate compliance results, plan exemptions and remediation, interpret budgets without treating them as spending caps, and evaluate Advisor recommendations against workload requirements. The chapter also develops a staged approach to governance changes with clear evidence and rollback boundaries.

5.1 Defining Standards with Azure Policy

A standard must be expressed as a testable rule

Azure Policy evaluates resources and requests against defined conditions. It can help an organization assess or enforce requirements such as approved locations, supported resource types, or required metadata. A useful policy begins with a clear requirement. “Keep Azure organized” is too vague to implement. “Applicable resources must have an approved environment tag” is specific enough to investigate, test, and explain.

The requirement should identify the resources it applies to, the properties being evaluated, and the intended response when the condition is met. Those choices prevent a broad statement from becoming a rule that blocks legitimate workloads. The administrator should also identify exceptions that are known before rollout. An exception process is easier to design when the underlying standard is precise rather than aspirational.

Policy differs from Azure RBAC. RBAC asks whether a principal may perform an operation at a scope. Policy asks whether a resource or request meets a rule. A Contributor can have permission to deploy a virtual machine while a policy denies the chosen configuration. Granting Owner does not make an ordinary policy restriction disappear. The two controls answer different questions about the same request.

Definition, assignment, and parameters

A policy definition describes the condition and effect. An assignment applies that definition at a chosen scope with specified parameter values and other settings. A definition that exists in the catalog has no automatic effect on every subscription. It must be assigned. This distinction mirrors the relationship between a role definition and a role assignment, although the policy evaluation model is different from authorization.

Parameters allow one definition to be reused with different approved values. A location policy can accept a list of locations rather than embedding one organization's choices permanently into the rule. The assignment supplies the appropriate list for its scope. Parameters improve reuse, but they also become part of the effective control. Reviewing only the definition without its assignment values can produce an incomplete explanation.

The assignment scope determines the resource population considered, subject to the policy's applicability and any exclusions or exemptions. An assignment at a management group can affect descendant subscriptions, while a resource-group assignment has a narrower reach. The administrator should choose the smallest practical pilot scope before broader enforcement. Testing an unproven rule at the top of a production hierarchy creates unnecessary uncertainty.

Effects describe different responses

Audit records noncompliance without blocking the relevant request. Deny can reject requests that match the rule. Modify can change supported properties under the definition's conditions, while deployIfNotExists can deploy related configuration when required conditions are satisfied. These effects are not interchangeable. The administrator should understand both request-time behavior and the treatment of resources that already exist.

An audit result is evidence of a rule comparison, not proof that Azure repaired the resource. A deny rule is a gate on applicable operations, not a general cleanup service for existing resources. Modify and deployIfNotExists can support remediation, but they require appropriate design and permissions. Choosing an effect based only on how strict its name sounds can produce a control that behaves differently from the stated requirement.

Effect direction	Typical purpose	Important limitation
Audit	Observe a standard before enforcement	Does not fix or block the resource by itself
Deny	Reject applicable nonconforming requests	Does not automatically repair existing state
Modify	Change supported properties	Depends on supported aliases and configuration
DeployIfNotExists	Establish related required configuration	Needs deployment permissions and careful scope

The table is a conceptual guide. Each actual definition contains conditions, supported modes, parameters, and effect-specific details that determine behavior. A built-in definition may provide several effect options, or it may be designed for one specific response. The administrator should inspect the definition rather than assuming that every effect can be substituted without changing the policy's meaning.

Applicability matters as much as the condition

A policy mode helps determine which resources or properties the definition evaluates. Resource-property aliases allow supported properties to be referenced consistently through the policy system. A rule that looks correct as general JSON may not apply to the intended resource type or API behavior. The administrator should prefer an appropriate built-in definition where one fits and verify custom definitions against current documentation.

For example, a tag policy needs to account for resource types that support the relevant tags and locations. Applying an unsuitable mode can produce unexpected results or leave resources unevaluated. The right investigation asks whether the resource is in scope, whether the definition applies to its type, and whether the referenced property is supported. An absent compliance result is not automatically proof that the resource meets the standard.

A staged rollout makes the rule reviewable

A practical rollout begins with a written requirement and a representative pilot. The administrator identifies examples that should pass and examples that should fail. An audit phase can reveal the current population and possible unintended matches. The results are reviewed with workload owners before an enforcement change. This creates evidence about the rule rather than treating the first production denial as the test plan.

The pilot should include deployment paths used by real teams. A resource created through the portal and one deployed through a template both reach Resource Manager, but the submitted properties and defaults can differ. The policy evaluates the actual request and resource state, not the administrator's intention. Testing the organization's real automation catches mismatches between the standard and deployment templates before enforcement expands.

Once the rule is understood, the change record describes the assignment scope, parameters, effect, exclusions, expected noncompliant population, and rollback method. Rollback should identify the specific assignment or effect change rather than a vague instruction to disable governance. If the rule blocks an unexpected legitimate operation, the administrator can restore the approved prior state while investigating the definition. A controlled response is more useful than a broad emergency exemption.

Diagnose a denied deployment

Suppose an administrator tries to create a resource in a region outside the approved list. The caller has sufficient RBAC permissions, yet the request is denied by policy. The error should identify the relevant policy context. The administrator compares the requested location with the assignment's parameter values and checks whether the workload should use an approved region or follow the exception process. Adding more RBAC privilege does not address the property mismatch.

Another failure might involve a required tag. The correct fix can be to update the deployment input so the intended metadata is present from creation. Manually adding the tag afterward is impossible if the creation request was denied. This shows why governance and deployment authoring must be aligned. A rule is sustainable when the normal deployment path produces compliant resources without repeated manual intervention.

Keep the policy understandable

Definitions and assignments should have useful descriptions and owners. A cryptic custom policy with no business rationale is difficult to maintain when a service changes or a workload needs an exception. The description should explain the requirement, while the technical definition expresses the test. These are complementary records. One helps a reviewer understand why the policy exists; the other determines what Azure actually evaluates.

Version and change management also matter. Updating a shared definition can affect several assignments. The administrator should identify its consumers before changing conditions or parameters, and should retain the previous definition for comparison. Built-in definitions can evolve as the platform changes, so supported version controls and current documentation deserve attention where applicable. Governance is an ongoing configuration responsibility rather than a static document.

Azure Policy is most effective when a resource owner can understand the rule and an administrator can explain its evidence. The condition, effect, scope, and parameters together define the control. Keeping those elements explicit makes it possible to combine policies coherently and interpret the compliance results they produce.

A useful negative test should fail for the reason the rule intends. If a test deployment is rejected by missing RBAC permission before its resource properties are evaluated, it has not demonstrated the policy's deny behavior. The test account needs the ordinary deployment authority, and the failure evidence should name the policy. Isolating the tested control prevents an unrelated error from being counted as successful governance validation.

5.2 Combining Policies and Reading Compliance Results

Initiatives organize related definitions

An initiative is a collection of policy definitions managed as a related set. It can simplify assignment when several rules express one governance objective. A baseline might include ownership metadata, approved locations, and required diagnostic configuration. The initiative gives the set a shared administrative identity while retaining the behavior of its component definitions. It does not turn different policy effects into one universal action.

The grouping should have a clear purpose. Combining unrelated policies solely to reduce the number of assignments can make ownership and exceptions harder to understand. A useful initiative has an audience, an accountable owner, and a reason its policies should travel together. The administrator should be able to explain which requirement each component supports and which resource populations it evaluates.

Initiative parameters can pass values to component definitions. This helps maintain a consistent environment label or approved value list across related policies. It also creates another layer to inspect during troubleshooting. The effective value may originate in the initiative assignment and then map into a specific definition parameter. A reviewer should trace that mapping instead of assuming that the displayed default is the value currently enforced.

Assignment is still the operational boundary

An initiative definition, like a single policy definition, needs an assignment before it governs a scope. The assignment supplies its scope, parameters, exclusions, and other relevant settings. Different assignments of the same initiative can have different approved values. The administrator should identify the exact assignment when explaining a compliance result. The initiative's name alone may refer to several distinct deployments of the baseline.

Overlapping assignments can also apply. A resource might be evaluated by a central baseline and a workload-specific policy. The resulting controls need to be compatible. A local assignment does not automatically replace an inherited one simply because it is closer to the resource. The administrator must inspect all relevant assignments and their effects. Governance inheritance should be reasoned about explicitly, not treated like a simple last-writer-wins configuration file.

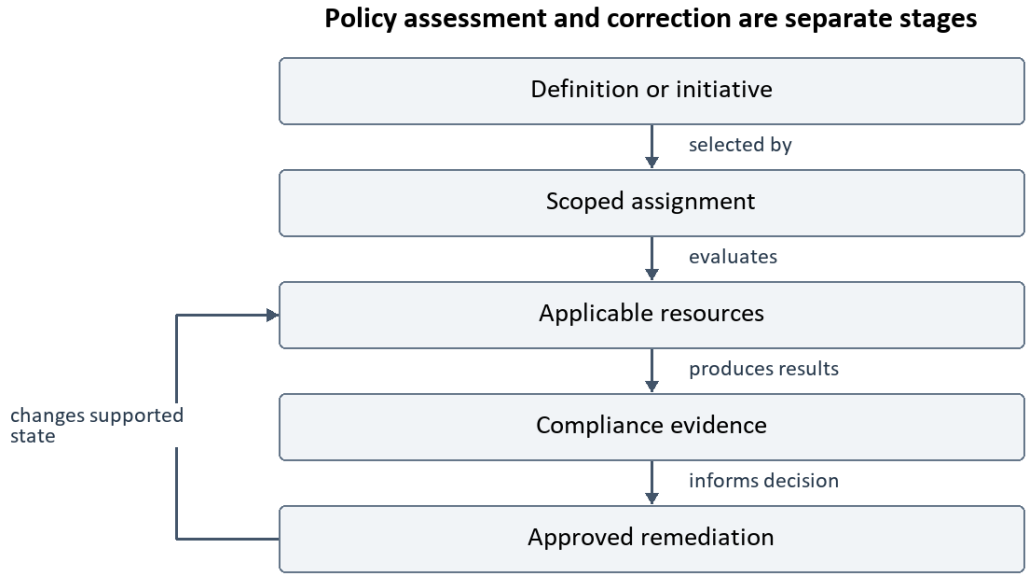


Figure 5.1 From policy definition to observed compliance

An assignment applies a reusable rule at a scope. Compliance evidence describes evaluation; supported remediation is a separate configuration action requiring permission and validation.

The figure separates the reusable definition, the assignment that applies it, the resources being evaluated, and the resulting compliance evidence. Remediation is a later action for supported effects. Keeping that sequence visible prevents the common assumption that assigning a policy means every existing resource has already been repaired. Assessment and correction are related stages with different evidence.

Compliance results are observations at a time

The compliance view summarizes policy evaluation results. It can help administrators find resources that do not meet an assigned requirement, but the result has a scope and an evaluation time. Newly assigned policies and newly changed resources may not appear instantly with their final state. The

administrator should inspect evaluation timing and trigger a supported scan when appropriate rather than treating a stale dashboard as current proof.

A result should also be read at the correct level. An initiative summary combines multiple definitions, while a resource-level detail can reveal the specific rule that failed. A single percentage is useful for overview but insufficient for diagnosis. The administrator should drill into the resource, definition, assignment, evaluated property, and reason. Those details turn a red status into a concrete change request.

Compliant, noncompliant, exempt, and not-applicable situations have different meanings. A resource that is not evaluated by a rule has not thereby demonstrated that it meets the rule's technical standard. An exemption records a governed exception, not necessarily a matching resource configuration. Reports should preserve these distinctions. Combining them into a single favorable number can obscure what has actually been tested.

Understand the population behind a percentage

Suppose one policy evaluates many ordinary resources while another applies only to a small number of critical systems. An overall compliance percentage can be dominated by the large population. The remaining failure might still represent the most consequential risk. Administrators should prioritize based on the requirement and workload impact, not only the numerical contribution to a dashboard. Aggregate metrics support triage but do not replace judgment.

Likewise, a newly created assignment can initially have incomplete evaluation coverage. A sudden change in percentage may reflect a different evaluated population rather than an abrupt configuration improvement. The administrator should compare scope, resource count, evaluation time, and assignment changes before celebrating or escalating. A meaningful trend requires comparable inputs. Otherwise, the chart can tell a story that the underlying configuration does not support.

An initiative's policy details help identify whether a failure is widespread or isolated. If every applicable resource fails the same rule, the assignment parameters or definition may be incorrect. If one resource differs from otherwise compliant peers, its configuration becomes a stronger lead. The pattern of results guides the investigation. It is often more efficient than opening resources one by one without a hypothesis.

A practical compliance investigation

An initiative requires environment metadata and a particular diagnostic configuration. The dashboard reports noncompliance for a storage resource. The administrator opens the resource's policy details and discovers that the tag rule passes while the diagnostic requirement fails. The next action concerns diagnostics, not a wholesale rewrite of tags or permissions. The initiative summary identified the target; the component result identified the actual issue.

The administrator then checks the expected related resource, the assignment parameters, and the definition's existence condition. If the policy expects diagnostics to a specific workspace, a different diagnostic destination might not satisfy the rule even though monitoring exists. The question is whether the resource meets this documented standard. A technically plausible configuration is not necessarily the configuration the assigned policy evaluates.

If the requirement is inappropriate for the workload, the response should follow the exemption or policy-design process. If the requirement is correct, the resource can be changed or remediated through the supported mechanism. The compliance result informs that decision but does not authorize an arbitrary production change. The workload owner and change process still determine how and when the correction is performed.

Evidence	Useful interpretation
Definition and effect	What behavior the rule is designed to produce
Assignment and parameters	Which version of the requirement applies here
Resource and evaluated property	What configuration was compared
Evaluation time	How recent the observation is
Exemption details	Why a governed exception applies

Preserve evidence through change

Before remediation, the administrator records the noncompliant condition and intended correction. Afterward, the resource state and the refreshed policy result are both checked. A successful remediation task is evidence that the operation completed, but it should be reconciled with the actual configuration and compliance result. This avoids reporting success when a later process has already reverted the property or a different rule remains unsatisfied.

The evidence should be concise enough to maintain. A resource ID, assignment ID, definition reference, observed value, expected value, evaluation time, and owner usually provide a more useful record than a large collection of screenshots. Screenshots can supplement the record, but text identifiers allow another administrator to find the same objects. The result should support follow-up without depending on the original operator's browser history.

Treat compliance as an operating process

New resources, changed templates, updated services, and revised business requirements continually alter the evaluated environment. A baseline should therefore have a review process, not merely an initial deployment date. Owners should know how new failures are routed and how obsolete rules are retired. A dashboard that nobody acts on provides visibility without governance. The value comes from a defined response to the evidence.

Initiatives help organize that response by grouping related standards. Detailed compliance results show which resource and rule need attention. Clear assignments explain the scope and parameter choices. Together they create a manageable path from a broad requirement to a specific decision, preparing the administrator to distinguish legitimate exceptions from resources that should be corrected.

Compliance exports should retain the assignment and definition references along with the status. A spreadsheet containing only resource names and colored cells loses the context needed to reproduce the result. The same resource can pass one rule and fail another, and its state can change between

evaluations. Including those references and the observation time allows a later reviewer to compare like evidence rather than infer meaning from a detached status label.

5.3 Handling Exemptions and Remediation

Exceptions should remain visible decisions

A policy exemption records that a resource or hierarchy is excused from an applicable assignment under a documented arrangement. It is useful when a temporary waiver or an alternative control is justified. The exemption should explain why the standard cannot be met as written and who accepts the exception. Without that explanation, an exemption becomes a way to make a dashboard look healthier while leaving the underlying requirement unresolved.

An exclusion and an exemption are not identical administrative choices. An assignment exclusion removes a scope from the assignment's evaluation population. An exemption is a separate governed object associated with the assignment and can preserve exception context. The administrator should choose the mechanism that matches the organization's policy process. The distinction matters when reporting how much of the environment actually meets a baseline and how much is outside it.

Exemption categories such as waiver and mitigated communicate different reasons. A waiver can acknowledge a temporary accepted gap. A mitigated exemption can describe how another control addresses the requirement. The category does not create the alternative control automatically. The evidence supporting that claim belongs in the review record. A label should summarize a decision that has already been justified.

Bound the exception in scope and time

The exemption should be as narrow as the legitimate need. Exempting an entire subscription because one resource cannot comply expands the exception to unrelated workloads. Where supported, an initiative exemption can identify particular component definitions rather than excusing the whole baseline. The administrator should inspect the resulting target carefully. A narrow technical exception should not silently remove unrelated standards.

An expiration date gives a temporary exception a review point. The owner should know what is expected before expiry: a resource change, a replacement workload, a renewed risk decision, or removal of the exemption. Expiration is not a substitute for follow-up. A resource can remain nonconforming after the exception ends, at which point the policy's normal behavior becomes relevant again. The operational plan should anticipate that transition.

The request should include the affected resource IDs, assignment, relevant definitions, reason, owner, approval, expiry, and compensating controls where applicable. It should also state the intended exit condition. These fields make the exception reviewable by someone who was not part of the original conversation. A vague note that a team “needs flexibility” does not provide enough information to maintain a meaningful governance boundary.

Remediation changes supported existing resources

Remediation is the process of bringing existing noncompliant resources toward the desired state through supported policy effects. Modify and deployIfNotExists are the central effects used for Azure Policy remediation tasks. An audit-only definition does not become an automatic repair operation simply because the administrator creates a task elsewhere. The definition's effect and configuration determine what correction is possible.

For deployIfNotExists, remediation can deploy related configuration described by the policy's template. For modify, it can apply supported property operations. The administrator should inspect those operations before running them against production. A remediation task is a real configuration change, not merely a compliance scan. It deserves the same understanding of impact, dependencies, and rollback as any other administrative change.

Existing resources and newly submitted requests can follow different timing paths. Assigning a policy does not mean that every historical resource changes immediately. A remediation task can be needed for the existing noncompliant population. The administrator should identify that population deliberately and verify the task's scope. Otherwise, a broad assignment can lead to a broader change than the operator expected.

The assignment identity needs permission

Supported remediation uses a managed identity associated with the policy assignment to perform resource changes. That identity needs the relevant Azure permissions. The administrator who created the assignment and the assignment's managed identity are different principals. A human administrator's broad permissions do not automatically prove that the remediation identity can deploy the required configuration. The failing operation must be investigated under the actual actor.

The definition can specify role requirements for its remediation work, but the resulting role assignments must still be established appropriately. Portal and command-line workflows can differ in how much related setup they perform. The administrator should inspect the identity and its actual assignments rather than assuming the tool completed every dependency. A visible managed identity without sufficient scope remains unable to make the intended change.

Permissions should match the remediation operations and their target scopes. A diagnostic deployment may need authority on both the monitored resource and a destination relationship, depending on the definition. A broad subscription role should not be assigned automatically if a narrower supported design meets the need. The remediation identity is an operational principal and belongs in the same access-review discipline as other workload identities.

Pilot the correction before expanding

A useful remediation pilot selects a small representative set of resources. The administrator records the initial state, inspects the proposed operation, confirms permissions, and runs the task within the approved scope. After completion, the resource configuration and policy result are checked. The pilot should include the relevant resource types and deployment patterns, because an operation that works for one service may not demonstrate every case in the broader assignment.

The rollback approach depends on what the remediation changes. Removing a newly added tag differs from undoing a deployed diagnostic configuration. Some changes can affect retained data, monitoring coverage, or other workflows. The administrator should not assume that deleting the policy assignment reverses prior remediation. The assignment controls future evaluation; already changed resources can retain their resulting configuration.

Situation	Appropriate direction
Requirement is valid and supported repair exists	Plan and validate remediation
Temporary legitimate gap	Narrow, approved exemption with an exit condition
Alternative control satisfies intent	Document evidence for a mitigated exception
Definition is incorrect for the population	Correct the policy design and reassess

This comparison prevents two opposite mistakes. One is exempting every difficult resource instead of correcting it. The other is forcing remediation when the rule itself is inappropriate. Good governance distinguishes a configuration defect from a justified exception and from a flawed standard. The compliance dashboard alone cannot make that distinction; the administrator and workload owner must interpret the evidence.

Investigate remediation failures precisely

Suppose a task reports deployment failures for several resources. The administrator examines the operation details, assignment identity, required actions, target scope, and resource-specific error. Insufficient permission is one possibility, but policy conflicts, locks, unsupported configuration, or missing dependencies can also prevent the change. Repeatedly restarting the task without addressing the cause creates more records without producing a more compliant environment.

If the task succeeds but compliance remains unchanged, the administrator checks the evaluation time and actual resulting properties. The policy may not have reevaluated yet, or another automation process may have reverted the change. The task may also have changed one component while another initiative rule still fails. Comparing the resource, definition, and time makes the next check specific rather than speculative.

The remediation identity should be reviewed after the work as well. It may remain necessary for ongoing policy behavior, so deleting it blindly can break future operations. Conversely, an obsolete assignment should not leave unnecessary privilege indefinitely. The lifecycle of the policy, its identity, its role assignments, and its deployed configuration should be documented together. Removing one object does not necessarily remove every related consequence.

Make the final state explainable

A completed governance action should leave each resource in an understood state: compliant with the rule, covered by a justified exception, or still awaiting a clearly assigned correction. The record should not hide uncertainty behind a generic success status. A reviewer should be able to see what changed, what evidence was checked, and who owns any remaining gap.

Exemptions and remediation are complementary tools. One records a controlled departure from the standard; the other changes resources to meet it. Both require precise scope, accountable ownership, and verification. When used this way, they preserve the meaning of compliance instead of turning it into a contest to remove warning symbols from a dashboard.

5.4 Understanding Costs, Budgets, and Alerts

Cost is a result of usage and purchasing context

Azure cost is influenced by the resources deployed, their configuration, their use, and the organization's purchasing arrangement. A resource's existence can generate charges even when the application appears idle. Storage, public IP resources, gateways, backup retention, and other components can continue to matter after a virtual machine stops doing useful work. Cost review should therefore inspect the whole workload rather than only its most visible compute component.

Price and cost are different concepts. A price describes a rate under particular conditions; cost reflects actual billed or allocated usage and the relevant purchasing terms. A public pricing example cannot be assumed to equal an organization's final invoice. Region, service tier, consumption, discounts, reservations, commitments, and agreement details can all affect interpretation. The administrator should use current tenant-specific cost evidence for decisions about existing resources.

For planning, an estimate should state its assumptions: region, size, expected runtime, storage volume, data movement, and any purchased commitment being considered. The estimate is not a promise. A workload that transfers more data or runs longer than expected can exceed it. Useful financial administration compares the estimate with observed usage and adjusts the operating plan when the assumptions change.

Choose the right scope and view

Cost Management can present information at supported billing and resource scopes, with access depending on the account arrangement and permissions. An administrator should confirm the selected scope before interpreting a total. A resource-group view and a subscription view answer different questions. A filter can also hide part of the cost population. The report should preserve scope, date range, currency, and relevant filters alongside the number.

Actual and amortized views can answer different questions about purchases and their allocation over time. The administrator should select the view appropriate to the review rather than comparing unlike figures. A sudden difference between two reports may reflect the chosen cost basis rather than a resource change. Consistent reporting definitions make trends meaningful and prevent teams from debating numbers that were calculated for different purposes.

Grouping by service, resource, resource group, or supported tag can help identify the source of a change. A large subscription total becomes actionable when the administrator can connect the increase to a workload owner and a usage pattern. Tags support this process, but incomplete metadata leaves an unallocated population that must be handled explicitly. A report should not silently ignore resources simply because they lack the preferred tag.

Budgets create accountability, not a hard stop

A budget compares cost against a planned amount over a defined period and scope. It can generate notifications when actual or forecast thresholds are crossed. A budget does not, by itself, stop resource consumption or cap the invoice. This distinction is essential for labs and production alike. Setting a small budget is a useful warning mechanism, but it is not a substitute for cleanup or a controlled deployment design.

Budget evaluation depends on cost data becoming available and on the service's evaluation process. It should not be treated as a real-time transaction limiter. An expensive workload can continue consuming resources before a notification is evaluated, delivered, and acted on. The administrator should consider data latency and response time when selecting thresholds. A threshold at the entire available amount leaves little room for a delayed response.

Notifications need owners who can act. An email sent to an unattended mailbox does not control spending. The operating process should say who investigates, what evidence is reviewed, and which changes are permitted. A budget alert for a production workload may require capacity review rather than immediate shutdown. The response should preserve business requirements while addressing the unexpected cost.

Define a useful budget

A budget should have a clear scope, amount, period, filter set, threshold strategy, and notification destination. The amount comes from an approved operating plan, not a random round number selected to make the dashboard look complete. Filters should match the population whose owner can respond. If several unrelated teams share a budget without a response owner, each can assume another team is responsible when the alert arrives.

Actual-cost thresholds indicate that recorded cost has crossed a defined proportion of the budget. Forecast thresholds can provide earlier warning based on projected spending where supported. Forecasts remain estimates and can be less informative for new or irregular workloads. The administrator should inspect the underlying usage trend rather than treating the forecast as an exact future invoice. Its value is to prompt investigation before the end of the period.

Tool or view	Main purpose	Does it automatically cap spending?
Cost analysis	Explain recorded cost and trends	No
Budget notification	Alert accountable owners	No
Pricing estimate	Model assumptions before deployment	No
Explicit shutdown automation	Perform a designed operational action	Only the implemented action, with its own limits

The final row requires care. An organization can connect notifications to automation in supported ways, but that is a separate design with permissions, conditions, timing, and failure modes. Shutting down

compute may leave other costs running. It can also interrupt required service. The automation should be tested and approved as an operational control rather than assumed to be an inherent budget feature.

Investigate a cost increase through evidence

Suppose a training subscription's cost rises after a lab ends. The administrator confirms the date range and groups the cost by resource and service. A virtual machine may have been deallocated while a managed disk, public IP, or another service remains. The correct cleanup follows the remaining resource and its owner. Deleting an unrelated resource merely because it is large in the portal would not be a reasoned response.

Another increase may come from data transfer, logging volume, or storage growth rather than new compute. The administrator should connect the charge category to the workload's activity and monitoring evidence. A cost chart identifies where to investigate; it does not always explain why usage changed. Application changes, diagnostic settings, retention periods, and traffic patterns can provide the missing cause.

The comparison should use a meaningful baseline. A monthly workload may have a predictable peak that looks unusual in a short daily view. A deployment may intentionally add capacity for a temporary event. The administrator should consult the workload owner before labeling every increase as waste. Cost optimization aims to remove unnecessary expenditure while preserving the service's required outcome.

Reduce cost through controlled changes

Rightsizing, schedules, cleanup, storage lifecycle, and purchasing choices can all influence cost. Each has tradeoffs. A smaller virtual machine can affect performance, shorter retention can affect recovery, and a commitment can reduce flexibility. The administrator should document the expected saving, operational effect, verification period, and rollback option. A cheaper configuration that fails the workload's requirement is not a successful optimization.

For disposable training resources, prompt cleanup is often the most direct control. A lab should name the resources that generate charges, define the expected duration, and provide exact removal steps. A budget supports this by making unexpected consumption visible. It does not replace the final inventory check. The learner should verify that the intended resources are gone and recognize that cost reporting can lag behind cleanup.

Maintain a recurring review

A useful cost review examines significant changes, unowned resources, budget alerts, and accepted optimization actions. It should record decisions and follow-up dates rather than merely distribute a chart. The owner of an action needs to know what will be changed and how success will be measured. Without that connection, the same recommendation can reappear every month without affecting the environment.

Financial evidence should be interpreted with the same precision as access evidence. The scope, period, cost basis, and filters define what the number means. The resource and usage pattern explain where it came from. The approved change and later observation demonstrate whether the response worked.

This discipline turns cost management into an ordinary operating practice, supported by budgets and alerts rather than replaced by them.

A cost alert should also be tested as a communication path where the organization's process permits it. A correctly configured threshold is of little help when its recipient address is obsolete. The response owner should confirm receipt procedures and escalation arrangements. Testing the notification route does not require deliberately creating a large charge; the supported notification and operational testing methods should be used instead.

5.5 Turning Advisor Recommendations into Decisions

A recommendation is an input to judgment

Azure Advisor analyzes supported resource configuration and usage information to provide recommendations. Its categories include cost, reliability, security, performance, and operational excellence. The service can reveal useful improvements that an administrator might otherwise miss. A recommendation is still a proposal to evaluate against the workload's requirements, dependencies, and change process. It is not a universal instruction to apply every suggested action immediately.

The recommendation's context matters. A suggestion based on observed utilization reflects the data available to the service and the assumptions of that recommendation. A workload with an infrequent critical peak can look underused during a quiet period. A resource reserved for recovery can intentionally remain idle. The administrator should determine whether the observed behavior represents the intended duty before changing capacity or removing a component.

This does not mean recommendations should be ignored whenever a team claims its workload is special. The claim should be supported by evidence such as a capacity plan, recovery requirement, scheduled peak, or measured dependency. Advisor creates a useful prompt for that conversation. The goal is a reasoned decision that can be reviewed later, whether the action is accepted, deferred, or rejected.

Read the recommendation completely

A useful review identifies the affected resources, category, rationale, suggested action, estimated impact, and any stated prerequisites. The administrator should inspect the exact resource IDs because similarly named resources can serve different environments. A recommendation for a test machine should not become a change to the production machine with a similar name. The same identity precision used in access work applies here.

Estimated savings should be read as estimates under the recommendation's assumptions. They can differ from realized savings because usage, prices, commitments, or scope change. The administrator should record the basis and compare the result afterward. A large estimated saving is a reason to investigate, not sufficient proof that the change is operationally acceptable. Reliability and performance requirements remain part of the decision.

Recommendations can also interact. A cost suggestion might reduce capacity while a reliability requirement calls for redundancy. The administrator should evaluate the workload as a system rather than optimizing each resource in isolation. The relevant question is whether the proposed final

configuration meets the service's agreed requirements at an appropriate cost. A local improvement can create a broader problem if dependencies are ignored.

Build a decision record

The decision record should identify the recommendation, resource owner, proposed change, supporting evidence, risk, test, and outcome. It should also explain a deferral or dismissal. A reason such as “needed for monthly processing peak” is useful only when linked to an actual requirement and review date. Otherwise, it can become a permanent excuse that survives long after the original workload disappears.

A temporary postponement is appropriate when a recommendation is valid but the timing is unsuitable. A documented dismissal can be appropriate when the recommendation does not fit the workload. These actions should be used according to the current Advisor interface and organizational process. The important behavior is that the recommendation remains connected to an accountable decision rather than simply disappearing from a person's screen.

Decision	Required evidence
Apply	Requirement remains satisfied and a validation plan exists
Pilot	Potential benefit is plausible but operational effect needs testing
Defer	A valid timing or dependency issue and a review date
Dismiss	A documented reason the recommendation does not fit

The table turns recommendation handling into a manageable workflow. Different workloads can reach different decisions for the same recommendation type. That is acceptable when the reasons are explicit and evidence-based. Consistency means using the same quality of reasoning, not forcing every resource into the same configuration regardless of purpose.

A rightsizing example

Advisor identifies a virtual machine as a candidate for a smaller size. The administrator reviews utilization across a period that includes the workload's meaningful peak and checks memory, disk, network, and application behavior where relevant. A low average CPU figure alone may not capture the resource's limiting factor. The proposed size must also support the necessary features and be available under the target subscription and region conditions.

The team then defines a controlled change window and rollback plan. After resizing, the administrator verifies application health and relevant performance measures, not merely the new size shown in the portal. Cost observation follows over a suitable period using comparable reporting scope and basis. If the workload no longer meets its requirement, the change is reversed or adjusted. The recommendation has led to an experiment with acceptance criteria, not a blind reduction.

If the machine is a standby component for recovery, the decision may differ. The owner should explain the required recovery capacity and whether a smaller standby configuration can still meet the objective. The idle appearance is not by itself evidence of waste. Equally, a recovery label should not prevent examination of a cheaper supported design. Advisor's value is to trigger the question and supply a candidate action.

A reliability example

A recommendation may point toward improved redundancy or recovery readiness. The administrator examines the workload's failure tolerance and the services involved. Adding a second instance helps only if traffic distribution, data consistency, and operational procedures support it. A redundant resource that receives no traffic or shares an unaddressed failure dependency may provide less benefit than the diagram suggests. The implementation must satisfy the recommendation's actual intent.

The change can also affect cost. The administrator should present the reliability benefit and the additional consumption together so the workload owner can make an informed decision. Cost control does not always mean reducing spend. Sometimes the appropriate action is to spend more to meet an agreed availability requirement. The decision should be explicit rather than hidden inside a technical implementation.

Security recommendations need the right workflow

Security-related guidance can involve other Azure services and organizational security processes. An administrator should identify the owning tool and the action's prerequisites, then coordinate with the responsible team. The recommendation should not be treated as authorization to alter production access or expose data. The same change discipline applies: understand the current state, define the intended state, assess dependencies, and verify the result.

An apparent security improvement can disrupt a legacy client if its authentication or network assumptions are not understood. That does not make the weaker configuration acceptable indefinitely. It means the remediation needs a migration plan and a documented deadline or exception. Advisor can identify the issue, while the organization determines the supported path to a safer operating state.

Prioritize by consequence and feasibility

A useful queue considers business impact, risk reduction, expected saving, effort, and dependency. An easily implemented low-risk improvement can be completed quickly, while a high-impact architecture change may need a separate project. The administrator should avoid ranking solely by the count of recommendations closed. Closing many minor items while leaving one serious exposure unresolved can create a misleading picture of progress.

The queue should distinguish an assessment awaiting refresh from unresolved work. After a resource changes, Advisor may need time to reassess it. The administrator records the completed change and checks the recommendation state later, preserving the original evidence. If the recommendation persists, the actual configuration and prerequisites should be reviewed. Refresh timing and failed implementation are different possibilities.

Verify benefits and retain learning

After an accepted change, the owner should compare the outcome with the intended benefit. Performance should remain within requirements, reliability behavior should be tested where practical, and cost should be measured on a comparable basis. A recommendation marked resolved is useful evidence, but the workload's own acceptance criteria provide the final operational test. The two observations should support one another.

The resulting decision record helps future administrators understand why the resource has its current shape. It can also improve deployment templates so new resources begin with the accepted configuration. That is the durable value of recommendation handling: useful lessons become repeatable standards. Advisor then supports ongoing refinement while Policy, budgets, and ordinary operational review maintain the environment's agreed boundaries.

Guided lab

Objective: Audit a missing tag on a disposable virtual network, correct the resource, inspect evaluation evidence, and draft a budget response.

Prerequisites and permissions: Use an approved training subscription and Bash with Azure CLI. The operator needs resource-group and virtual-network administration, permission to create a custom policy definition in the subscription, and policy-assignment permission at the lab group. Resource Policy Contributor can support policy operations but does not replace resource-deployment authority. The lab creates a basic virtual network, group, definition, and audit assignment. It creates no paid VM, public IP, gateway, peering, remediation identity, or monitoring destination. The basic unconnected virtual network has no direct service charge. Do not attach other services to it.

1. Select the training subscription and set `location='<approved-region>'`. Create the group with `az group create --name rg-compass-ch05 --location "$location"`. Create a network with `az network vnet create --resource-group rg-compass-ch05 --name vnet-compass-ch05 --location "$location" --address-prefixes 10.55.0.0/16`. No subnet or workload is needed for this metadata exercise.
2. Save the following JSON as a local lab file named `compass-ch05-rule.json`. It audits only virtual networks missing the specified tag.

```
{
  "if": {
    "allOf": [
      {"field": "type", "equals": "Microsoft.Network/virtualNetworks"},
      {"field": "tags['CompassOwner']", "exists": "false"}
    ]
  },
  "then": {"effect": "audit"}
}
```

3. Confirm that no existing definition named `compass-ch05-audit-tag` belongs to other work. Run `az policy definition create --name compass-ch05-audit-tag --display-name 'Compass Chapter 05 audit tag' --mode Indexed --rules @compass-ch05-rule.json`. Record the returned definition ID. Set `scope=$(az group show --name rg-compass-ch05 --query id -o tsv)` and `definition_id=$(az policy definition show --name compass-ch05-audit-tag --query id -o tsv)`.

4. Assign only to the lab group: `az policy assignment create --name compass-ch05-audit --scope "$scope" --policy "$definition_id"`. Record the assignment ID. Allow assignment propagation and run `az policy state trigger-scan --resource-group rg-compass-ch05` when ready.
5. Inspect the assignment in the portal's Policy compliance view. The expected evaluated result is noncompliance for the untagged virtual network. Record the resource ID, definition, assignment, and evaluation time. A pending result is not a successful or failed test until evaluation completes.
6. Correct the controlled fault with `az network vnet update --resource-group rg-compass-ch05 --name vnet-compass-ch05 --set tags.CompassOwner=Training`. Trigger a new scan and compare the actual resource tag with the refreshed compliance result. The expected final result is compliance with this tag rule. Audit did not add the tag; the explicit update did.
7. Inspect Cost Management budget fields read-only. Record a proposed amount, period, thresholds, recipient, and response owner. State that the budget does not cap spending. Inspect Advisor read-only and record an actual recommendation if one exists, or record that none is available. Do not invent a recommendation or trigger notifications to another person for this exercise.
8. Delete the assignment with `az policy assignment delete --name compass-ch05-audit --scope "$scope"`. Delete the lab definition with `az policy definition delete --name compass-ch05-audit-tag`. Delete the group with `az group delete --name rg-compass-ch05 --yes`, then verify `az group exists --name rg-compass-ch05` returns `false`. Remove the local rule file only if it is no longer needed for the learning record.

The lab was not executed during authoring. Its expected evaluation results must be observed in the training tenant. If an inherited organizational policy blocks the network, preserve that control and use an approved training scope; do not alter the organization's baseline for this lab.

Chapter synthesis

Governance becomes practical when standards, evidence, and decisions remain connected. A policy definition describes a rule, an assignment applies it, and compliance results show observed matches and gaps. Exemptions preserve justified departures, while supported remediation changes existing resources. Cost analysis and budgets make consumption visible, and Advisor supplies proposals that must be assessed against workload needs.

The tools have clear limits. Audit does not repair a resource, a budget does not stop spending, and a recommendation does not authorize a production change. An administrator uses those limits to design dependable processes: narrow pilots, accountable owners, explicit parameters, measured outcomes, and complete cleanup. That operating discipline carries into the networking chapters, where reachability depends on equally precise relationships.

Review questions

05-Q1. What causes a policy definition to govern a resource population?

- **A.** Saving its name in a tag
- **B.** Assigning it at a scope with the relevant settings
- **C.** Giving every user Reader
- **D.** Opening the definition in the portal

05-Q2. A deployment caller has sufficient RBAC permission, but a deny policy rejects the requested region. What is the correct response direction?

- **A.** Reset the caller's password
- **B.** Add an unrelated data role
- **C.** Use an approved configuration or follow the policy exception process
- **D.** Assume Owner always bypasses the policy

05-Q3. What is an initiative?

- **A.** A related collection of policy definitions managed together
- **B.** A mandatory subscription spending cap
- **C.** A replacement for every role assignment
- **D.** A virtual network routing table

05-Q4. A dashboard shows a favorable aggregate compliance percentage. What should a reviewer still inspect?

- **A.** Only the dashboard color
- **B.** Whether all resources have the same name
- **C.** Whether the portal uses dark mode
- **D.** The underlying rules, resource population, exemptions, and evaluation time

05-Q5. Which policy effects are central to supported remediation of existing resources?

- **A.** Audit alone and Disabled
- **B.** Modify and deployIfNotExists
- **C.** Reader and Contributor
- **D.** CanNotDelete and ReadOnly

05-Q6. A remediation task lacks permission to deploy its required related resource. Which identity's actual assignments must be checked?

- **A.** Every guest in the tenant
- **B.** The resource's display name
- **C.** The managed identity associated with the policy assignment
- **D.** Only the person reading the dashboard

05-Q7. What does a Cost Management budget do by itself when its threshold is crossed?

- **A.** Generate configured notifications without automatically stopping consumption
- **B.** Delete every resource in scope
- **C.** Guarantee the invoice cannot exceed the amount
- **D.** Reserve physical VM capacity

05-Q8. Advisor recommends downsizing a lightly used standby VM. Which response is best?

- **A.** Apply every recommendation immediately
- **B.** Dismiss it solely because the word standby appears in a tag
- **C.** Treat the estimated saving as a guaranteed invoice reduction
- **D.** Check recovery requirements, utilization evidence, dependencies, and a validation plan

Further reading

- Azure Policy overview
- Policy definition structure
- Initiative definitions
- Policy exemptions
- Policy remediation
- Create and manage budgets
- Azure Advisor overview

Chapter 6: Building the Paths Between Resources

Azure resources need a deliberate path before they can exchange useful traffic. Building the Paths Between Resources develops that path from address planning through subnets, peering, routes, and diagnosis. The management hierarchy established earlier controls ownership and administration; the network establishes reachability. A resource can be correctly owned, tagged, and authorized while still having no working route to the service it needs.

Learning objectives

The reader should be able to calculate simple IPv4 subnet ranges, plan nonoverlapping address spaces, distinguish private and public addressing, explain peering boundaries, select routes by destination, and investigate a failed connection layer by layer. The reader should also recognize that modern private-subnet defaults require explicit outbound connectivity when workloads need public endpoints.

6.1 Understanding Address Spaces and CIDR

Addresses describe locations in a network

An IP address identifies an interface or endpoint within a network context. IPv4 represents an address as four decimal octets, such as `10.60.1.4`. Each octet corresponds to eight bits, so an IPv4 address contains thirty-two bits. The network design divides those bits into a network portion and a host portion. Understanding that division allows an administrator to calculate which addresses belong to the same prefix.

Classless Inter-Domain Routing, abbreviated CIDR, writes a prefix as an address followed by a slash and a prefix length. In `10.60.0.0/16`, the first sixteen bits identify the network. The remaining sixteen bits describe addresses within that block. A larger prefix length leaves fewer host bits and therefore a smaller address block. This is why `/24` is more specific than `/16`, even though the number after the slash is larger.

CIDR is not merely an exam notation. Azure uses prefixes for virtual-network address spaces, subnets, route destinations, and security rules. A mistaken prefix can create overlap, direct traffic to the wrong next hop, or permit a broader population than intended. The administrator should be comfortable translating the notation into a range before applying it to a live environment.

Calculate the block before allocating it

An IPv4 prefix contains `2^(32 - prefix length)` total addresses. A `/24` contains 256 addresses; a `/26` contains 64. These are mathematical block sizes, not the number of deployable Azure endpoints. Azure reserves addresses in each subnet, and some services need additional capacity for their own operation. Planning should distinguish the total block from the usable capacity of the intended service.

For a `/26`, the last octet advances in blocks of 64. Within `10.60.1.0/24`, the aligned `/26` blocks begin at `.0`, `.64`, `.128`, and `.192`. The first block extends from `10.60.1.0` through `10.60.1.63`. The second

extends from `.64` through `.127`. These boundaries matter because a prefix must describe an aligned network block, not an arbitrary starting address and a desired count.

Prefix	Total IPv4 addresses	Example aligned range
`10.60.1.0/24`	256	`10.60.1.0–10.60.1.255`
`10.60.1.0/25`	128	`10.60.1.0–10.60.1.127`
`10.60.1.0/26`	64	`10.60.1.0–10.60.1.63`
`10.60.1.64/26`	64	`10.60.1.64–10.60.1.127`

The table shows why two adjacent smaller blocks can fit inside a larger block without overlapping. It also shows why changing the prefix length without checking alignment is unsafe. An administrator should calculate the complete range, compare it with existing allocations, and record the intended use. A spreadsheet or IP address management system can assist, but the operator still needs to understand what the tool is checking.

A virtual-network address space contains subnets

An Azure virtual network has an address space containing one or more prefixes. Subnets divide that space into portions for resources and supported services. A subnet prefix must fit within the virtual network's address space and must not overlap another subnet in that network. The administrator should allocate subnets according to functional and capacity requirements rather than simply choosing the smallest block that accepts the first deployment.

Suppose a training virtual network uses `10.60.0.0/16`. An application subnet might use `10.60.1.0/24`, while a separate test subnet uses `10.60.2.0/24`. Both fit inside the parent range and do not overlap. The large parent range leaves room for additional subnets, but unused space is not a reason to create unnecessary resources. Address planning reserves logical capacity; it does not itself deploy workloads.

A different network intended for peering might use `10.61.0.0/16`. The nonoverlapping range allows routes to distinguish the destinations. If both networks use `10.60.0.0/16`, peering cannot simply infer which identical destination range a packet means. Address overlap is a design conflict, not a problem solved by naming the networks differently. Resource names and IP prefixes serve different purposes.

Plan beyond the first virtual network

Organizations often connect Azure networks to other Azure networks or to on-premises address spaces. An address block that looks unused in one subscription may already be assigned elsewhere. The administrator should consult the organization-wide address inventory before selecting prefixes. A local deployment success does not prove that the range can later participate in the intended connected network.

Private IPv4 ranges are commonly used for Azure workload networks because they support internal addressing without requiring public address ownership. Private addressing does not automatically

make a resource secure or isolated. Routing, peering, public endpoints, and security controls determine actual reachability. A private address can be reachable from many connected networks if the design permits it. The word private describes an addressing category, not a complete security guarantee.

Growth planning should include expected instances, scale operations, private endpoints, service-specific subnet needs, and future integration. A subnet that barely fits today's resources may block a scale event or require disruptive redesign. The appropriate reserve depends on the workload rather than a universal percentage. Administrators should document the assumption behind the chosen size so later teams understand whether the spare addresses are deliberate.

Separate arithmetic from service requirements

Knowing that a `/27` contains thirty-two addresses is only the beginning. Azure reserves part of the subnet, and a service may require a particular minimum size or dedicated subnet. The administrator must consult that service's requirements before deployment. A mathematically valid subnet can still be operationally unsuitable. Service-specific constraints are discussed where those services appear in later chapters.

IPv6 introduces a different address format and allocation model. An AZ-104 administrator should recognize that IPv4 arithmetic cannot simply be copied into IPv6 planning. This chapter uses IPv4 examples to build the route and subnet reasoning needed for the labs. A dual-stack production design should follow current Azure IPv6 and service-specific guidance rather than treating the IPv4 examples as a complete dual-stack architecture.

Recognize overlap through ranges

Two prefixes overlap when at least one address belongs to both ranges. `10.60.0.0/16` overlaps `10.60.1.0/24` because the smaller block is inside the larger one. By contrast, `10.60.1.0/24` and `10.60.2.0/24` are adjacent in the broader space without sharing addresses. Comparing only the written starting addresses can miss the first overlap. The prefix length determines the actual coverage.

This becomes important when reviewing route tables. A broad route can match a destination while a more specific route also matches it. That is not necessarily an invalid overlap; routing deliberately uses specificity to select a path. Address-space overlap between networks intended for peering is a different issue. The same mathematical relationship can be useful in route selection and problematic in allocation, depending on the design context.

Keep an address allocation record

A useful allocation record includes the prefix, owning network, subscription, region, intended purpose, connected environments, and reserved growth. It should identify who can approve a change. An undocumented spare block may later be assigned by two teams, creating overlap only discovered when connectivity is needed. Central visibility reduces that risk without requiring every administrator to memorize the whole organization.

Changes should be checked against deployed resources and connected peers before implementation. Expanding a virtual-network address space can require related peering synchronization and route

review. Shrinking or replacing a subnet can be constrained by the resources already using it. An address plan is therefore a living technical dependency, not a diagram that can be edited freely after deployment.

CIDR gives the administrator a precise language for those decisions. The prefix identifies the range, the range exposes overlap, and the capacity calculation makes growth assumptions visible. Once the address plan is sound, subnets and IP configurations can turn it into a working resource layout.

A short calculation makes the reasoning concrete. For `10.60.1.64/26`, the six host bits provide sixty-four addresses, ending at `10.60.1.127`. Azure's five reserved addresses leave fifty-nine ordinarily assignable addresses before service-specific needs are considered. The first available address is `.68`, because `.64` through `.67` are reserved, and `.127` is also reserved. A proposed assignment of `.70` fits the range; `.130` belongs outside it. This example separates three checks that should always remain distinct: membership in the mathematical prefix, exclusion of platform reservations, and availability after actual resource assignments. Passing only the first check is not enough to select a valid unused address for a new interface.

6.2 Creating Subnets and Managing IP Addresses

A subnet is a placement and policy boundary

A subnet divides a virtual network's address space and provides a place for supported resources or service integrations. Network security groups, route tables, delegations, and other settings can attach to it where supported. The subnet's purpose should therefore be clear before resources are deployed. Combining workloads with incompatible routing or service requirements can make later changes difficult even when plenty of addresses remain.

A subnet does not create a complete security boundary by itself. Azure routing can connect subnets in the same virtual network, while security rules determine which traffic is allowed. An administrator should not assume that placing an application and database in different subnets automatically blocks access between them. Segmentation requires the relevant routing and filtering design. The subnet provides a useful attachment point for that design.

Some Azure services require a dedicated or delegated subnet. Delegation authorizes a supported service to use the subnet according to its requirements. A subnet reserved for such a service should not be treated as a general pool for unrelated virtual machines. The administrator should inspect the service's current requirements, including name, size, and coexistence restrictions, before committing the address block.

Account for Azure-reserved addresses

Azure reserves the first four addresses and the last address in every IPv4 subnet. In `10.60.1.0/24`, that leaves addresses from `10.60.1.4` through `10.60.1.254` for eligible assignments, subject to the service's requirements and existing allocations. The total mathematical block contains 256 addresses, while the ordinarily assignable count is 251. This difference matters when sizing a small subnet for growth.

The administrator should never select a reserved address simply because it appears unused in a resource list. The platform reservation is not represented as an ordinary VM network interface that can

be deleted to free space. It is part of the subnet's operation. A failed assignment to a reserved address should be corrected by choosing an available permitted address, not by attempting to override the platform behavior.

Capacity planning also needs to consider resources beyond the visible application instances. Internal load balancers, private endpoints, and service-managed components can consume addresses. Some services need spare capacity during upgrades or scaling. Counting only the current virtual machines can therefore understate the required subnet size. The resource inventory and service-specific requirements should both inform the plan.

Private addresses belong to Azure IP configurations

For a virtual machine, private addressing is configured through its Azure network interface and IP configuration. Dynamic allocation lets Azure choose an available address from the subnet. Static allocation reserves a chosen valid address for that configuration. The word dynamic does not mean the address changes every time a packet is sent; it describes the allocation method. The resource lifecycle determines when a dynamic assignment can change.

An administrator should use Azure's network configuration to establish a static private address where required. Hardcoding a different address only inside the guest operating system can create a mismatch with Azure's view of the interface. The VM can then become unreachable even though its portal settings look correct. Guest configuration and the Azure IP configuration should be kept consistent using the supported procedure for the operating system and workload.

A static private address can be useful for a service whose clients or dependent configurations require a stable endpoint. DNS often provides a better abstraction for clients than embedding the address everywhere. The administrator should distinguish the need for stable addressing from the need for a memorable name. A static IP does not remove the value of proper name resolution or the need to update dependent records after a replacement.

Public addresses provide a separate endpoint

A public IP resource can support internet-facing connectivity for compatible Azure resources. It is distinct from the private address assigned inside the subnet. A public address does not automatically grant a user access to the guest operating system, nor does it override security rules. It creates or supports a reachable endpoint path that still needs the appropriate routing, service listener, and filtering configuration.

Public IP behavior depends on supported SKU, allocation, region, and associated service. Older examples may use retired or changed combinations. The administrator should follow current documentation and use supported configurations rather than copying a legacy lab literally. This book avoids relying on obsolete public-IP defaults. A public endpoint should be introduced only when the workload's connectivity requirement justifies it.

Public addresses can also contribute to cost even when the associated application is idle. Lab cleanup should identify and remove unused public IP resources where appropriate. Deleting a VM does not always guarantee that every separately managed network resource has been removed. A resource-group inventory provides a useful check, but the administrator should still understand which resources the deployment created and which are shared.

Outbound connectivity is an explicit design choice

An Azure workload that must reach public endpoints needs a supported outbound method. Current Microsoft guidance states that API versions released after March 31, 2026 default new virtual networks to private subnets, with the portal already using private-subnet defaults. Existing networks are not automatically changed by that default transition. Administrators must inspect actual subnet settings instead of assuming that every VM receives implicit internet access.

Explicit outbound choices include NAT Gateway, appropriately configured Standard public Load Balancer outbound rules, or an intentionally assigned public IP where suitable. A routed firewall or network appliance design can provide another controlled path when configured correctly. These options have different operational and cost implications. The existence of a default route toward the internet does not by itself supply the required outbound translation or connectivity method.

Address or connectivity choice	Purpose	Does not automatically provide
Private IP configuration	Address inside the virtual network	Public internet exposure
Static private allocation	Stable selected private address	Correct DNS or an application listener
Public IP resource	Supported public endpoint association	Guest authorization or permissive firewall rules
Explicit outbound method	Deliberate public egress path	Unrestricted inbound access

This distinction affects common labs. Installing packages from a public repository requires outbound connectivity, even if the VM has no need for inbound public access. A lab that deploys a private VM and immediately runs internet-dependent installation commands can fail under modern defaults. The setup must either provide the required egress or avoid that dependency. The chapters use minimal configurations and identify such dependencies explicitly.

Create and inspect a subnet deliberately

A subnet creation procedure should confirm the parent virtual network, intended prefix, capacity, and required attached controls. The administrator then creates the subnet and reads back its properties before placing workloads there. The returned configuration should show the correct address prefix and relevant associations. This is a better acceptance check than relying only on a successful creation message.

For example, a subnet named `snet-app` under `vnet-compass-ch06` might use `10.60.1.0/24`. A second subnet can use `10.60.2.0/24` for a distinct routing or security purpose. Their names describe intended use, while the prefixes establish actual address placement. If the names are accidentally reversed, traffic follows the prefixes and attached controls, not the administrator's interpretation of the labels.

Troubleshoot address-related failures

If a new interface cannot receive an address, the administrator checks subnet capacity, reserved addresses, existing allocations, and service-specific restrictions. If a VM loses connectivity after a guest IP change, the Azure and guest configurations should be compared. If public access fails, the administrator checks the public association and filtering separately from the private address. Each symptom points to a different part of the address design.

The final network record should include subnet prefixes, delegations, attached route tables and security groups, important static assignments, and outbound design. These details make later changes predictable. An address list alone is insufficient because the same subnet also carries routing and policy relationships. Once those local relationships are understood, peering can connect separate virtual networks without losing their administrative identities.

Before changing a subnet's purpose, the administrator should inventory every attached resource and service relationship. An apparently empty address range can still have a delegation or a route-table association that reflects an existing design. Reusing the name without reviewing those settings can carry old behavior into a new workload. The intended subnet state should be described and verified as a whole.

6.3 Connecting Virtual Networks Through Peering

Peering connects address spaces

Virtual network peering creates private connectivity between supported Azure virtual networks. Traffic uses Microsoft's network infrastructure rather than requiring the resources to communicate through public internet endpoints. Each network retains its own resource identity and administrative settings. Peering establishes a connectivity relationship; it does not merge subscriptions, resource groups, or access roles into one administrative object.

The connected networks must have compatible, nonoverlapping address spaces for ordinary peering. If two networks use the same prefix, destination addresses become ambiguous. A peering request does not solve that conflict by translating one network automatically. The administrator must correct the address design or use another supported architecture appropriate to the situation. Nonoverlapping allocation is the simplest foundation for the peering exercises in this book.

Peering can connect networks in the same region or supported networks across regions. The regional relationship can influence cost and service constraints, so it belongs in the design review. A private backbone path is not the same as a promise that all applications experience identical latency. Workload behavior, region distance, service dependencies, and traffic volume still need consideration.

The relationship has two sides

A peering connection is represented from each virtual network's perspective. Both directions must be configured appropriately for the intended relationship. The administrator should inspect the state on each side rather than assuming that one successful creation command completed the whole connection. A one-sided or disconnected relationship can leave the apparent design incomplete even though one peering object exists.

Permissions are required for the relevant operations on both networks, either through one authorized principal or a coordinated process between owners. Networks in different subscriptions can have different administrative teams. The change request should identify both resource IDs and both owners. A short network name is not sufficient when the administrator must create a relationship across subscription boundaries.

Peering settings control aspects such as virtual-network access, forwarded traffic, and gateway use where supported. The administrator should enable only the features required by the design. Selecting every available option does not create a more reliable connection. It can introduce routing assumptions and trust relationships that the workload never needed. Each setting should have an explanation tied to the intended traffic path.

Peering is not automatically transitive

If network A is peered with network B and B is peered with C, A does not automatically gain ordinary peering connectivity to C. This nontransitive behavior is a central design fact. A hub network does not become a router merely because several spokes are peered with it. Spoke-to-spoke transit requires a deliberate supported design, such as routing through an appropriate network appliance or another documented connectivity mechanism.

The administrator should distinguish direct peering routes from forwarded traffic. A packet generated by a resource in the hub and a packet forwarded through the hub from another network are not the same case. Peering settings, routes, and the forwarding device's configuration must all support the intended transit. Enabling a forwarded-traffic option alone does not create a route or turn a normal VM into a functioning router.

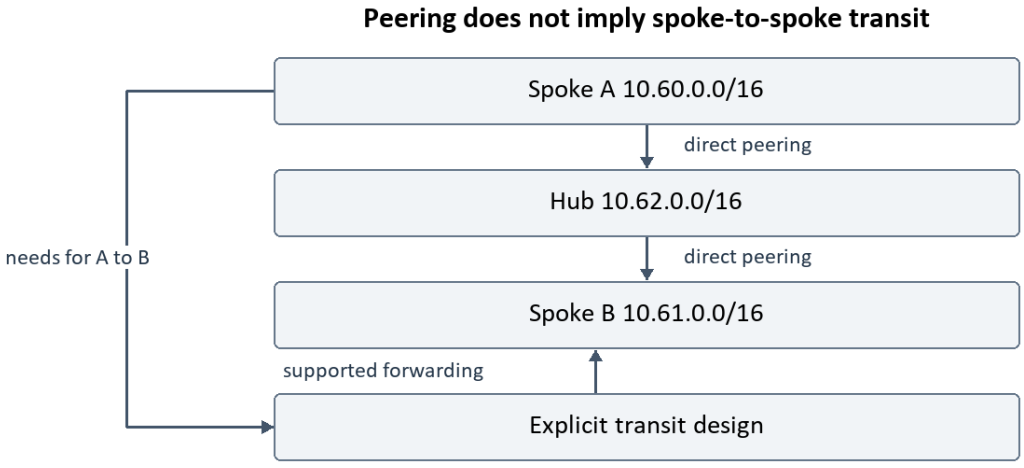


Figure 6.1 Peering connects networks without implicit transit

A-to-hub and hub-to-B peerings do not automatically create A-to-B transit. A supported routing and forwarding design must establish the intended path and return behavior.

The figure shows two spokes connected to a hub. The missing automatic spoke-to-spoke path is deliberate. An administrator reviewing such a diagram should ask how transit is implemented, what

device or service forwards the packet, and how the return path works. A line between a spoke and hub is not enough evidence that every spoke can reach every other spoke.

Routes and security still determine success

Peering introduces relevant connectivity routes, but security groups and other controls still apply. A connected peering state does not prove that a particular application port is allowed. The destination must also have a listener, the guest firewall must permit the request, and the return path must work. Peering is one dependency in the connection, not an end-to-end application health check.

Consider a client in `10.60.1.0/24` trying to reach a server in `10.61.1.0/24`. The administrator verifies connected peerings and the expected route, then checks the server's security rules and application port. If the route exists but the service refuses the connection, recreating the peering is unlikely to help. The failure has moved to another layer. Diagnosis should follow the evidence rather than repeatedly changing the most visible network object.

Name resolution also needs its own design. Peering does not automatically create every required DNS relationship between networks. A client may reach the server's private address while failing to resolve its application name. That pattern points toward DNS rather than the peering transport. The later DNS chapter develops private zones and links; here the important habit is to test address reachability and name resolution as separate observations.

Address-space changes need synchronization review

When a peered network's address space changes, the related peering configuration may need synchronization according to the supported workflow. The administrator should inspect both peers after the change and verify the resulting routes. A newly added subnet can remain unreachable if the remote side does not yet reflect the intended address space. The original peering success does not prove that later changes have propagated correctly.

The change record should include the old and new prefixes, overlap checks, affected peers, and verification results. It should also identify routes or security rules that reference the old range. A network address change can affect more than the peering object itself. Explicitly reviewing those references prevents a successful platform update from leaving partially broken application paths.

Gateway sharing is a deliberate extension

Peering can participate in supported gateway-transit designs, where one network uses a remote gateway under specific settings and constraints. The administrator should recognize the distinction between allowing gateway transit on one side and using a remote gateway on the other. Those options are not synonyms. Their prerequisites and limitations should be checked for the exact gateway and network design before configuration.

AZ-104 preparation benefits from understanding what the options mean without expanding into a full hybrid-network architecture course. The practical question is whether the resource needs direct peer connectivity, forwarded transit, or access through a shared gateway. Each has different dependencies. A simple lab should use direct peering with the minimum options needed so its behavior can be understood before additional transit features are introduced.

Cost and ownership remain separate

Peering can generate data-transfer charges according to the relevant regional and traffic arrangement. The absence of a gateway does not mean every peered traffic path is free. The administrator should review current pricing for the planned volume and geography. Cost responsibility should be clear when two teams own the connected networks. A technically successful shared connection can still create an ownership dispute if nobody agreed who monitors its usage.

Peering also does not grant Azure resource-management permission across the connection. A developer whose VM can reach another network does not thereby become Reader or Contributor on that network's subscription. Network access and administrative authorization remain different controls. This separation allows resources to communicate while their teams retain distinct management boundaries.

Verify a peering through several observations

A useful verification checks both peering states, address-space compatibility, relevant settings, and an actual approved application flow when test endpoints exist. The record includes source and destination addresses, protocol, port, and time. If no VMs are deployed in a low-cost exercise, the lab should honestly report configuration verification only. A connected peering state should not be presented as a measured end-to-end packet test that never occurred.

Cleanup removes both peering objects when the relationship is no longer needed, unless deleting the disposable networks removes them as part of an approved lifecycle. Shared networks require more care because other workloads may depend on the connection. The administrator should inventory that dependency before removal. Peering is easy to add, but its operational value comes from understanding the traffic it intentionally enables.

A peering inventory should include the remote network ID and the settings from both directions. Recording only the local friendly name can hide a connection to the wrong subscription or a stale environment. This is especially important when production and test networks use similar naming patterns. Comparing the full remote IDs before approval prevents a syntactically valid connection from linking the wrong operational boundaries.

6.4 Directing Traffic with Routes

A route chooses the next step for a destination

A route associates a destination prefix with a next hop. When a resource sends traffic, Azure uses applicable routing information to determine where that traffic should go. The route does not authenticate the application or decide whether the destination's firewall accepts the packet. It answers a narrower question about forwarding. Keeping that question separate makes route tables easier to interpret and prevents them from being mistaken for security rules.

Azure creates system routes for virtual networks and subnets. Additional features, such as peering and gateways, can contribute routes. Administrators can create user-defined routes in a route table and associate that table with a subnet. The effective routing result includes these sources according to

Azure's selection rules. Looking only at the custom route table can therefore miss an important system or learned route.

A route table's name has no effect on packet selection. A table named `to-firewall` can contain an incorrect next hop or be associated with the wrong subnet. The administrator should inspect the actual destination prefixes, next-hop types, addresses where required, and associations. Labels describe intended behavior; the effective route set determines what Azure actually does.

Longest matching prefix comes first

For ordinary route selection, Azure compares the destination address with matching prefixes and selects the most specific applicable match. A route for `10.61.1.0/24` is more specific than a route for `10.61.0.0/16`. Traffic to `10.61.1.20` matches both, but the `/24` describes the narrower range. Traffic to `10.61.2.20` does not match the `/24`, so a different route applies.

The default route, `0.0.0.0/0`, matches every IPv4 destination but has the least specificity. It is useful for destinations not covered by a more specific applicable route. Creating a default route to a firewall does not necessarily override every more specific path. The administrator must examine the effective table and Azure's documented preferred-route behavior. A broad route should not be described as a universal traffic interception mechanism.

When routes have the same destination prefix, Azure generally prefers a user-defined route over a BGP-learned route, and a BGP route over a system route. Azure also documents preferred system-route cases, including service-endpoint behavior, that require attention. The administrator should not apply a simplified mnemonic without checking those exceptions. Exact effective routes are the operational evidence for a specific destination.

Destination	Candidate routes	Ordinary specificity result
`10.61.1.20`	`10.61.0.0/16`, `10.61.1.0/24`	The `/24` is more specific
`10.61.2.20`	`10.61.0.0/16`, `10.61.1.0/24`	Only the `/16` matches
An address outside both ranges	Those prefixes plus `0.0.0.0/0`	The default route matches

This table teaches prefix reasoning, not the full route-source algorithm for every Azure service. In a real environment, next-hop types, route sources, peering, gateway propagation, and preferred routes must also be considered. The administrator should use the table to identify the matching prefixes, then inspect the actual effective result rather than stopping at a theoretical choice.

Next-hop types express different behavior

A virtual-appliance next hop sends traffic toward a specified private address, usually belonging to a firewall or network virtual appliance. The appliance must be reachable and configured to forward the intended traffic. A route does not install forwarding software or enable every necessary interface setting. If the next-hop VM behaves like an ordinary host, packets can arrive there and stop rather than continuing to the destination.

A None next hop discards matching traffic. This can be useful in a controlled routing design or a disposable demonstration of route intent. It should not be confused with a missing route-table entry. An explicit discard route is a deliberate forwarding decision. The administrator should document its purpose because a later operator may otherwise interpret the unreachable destination as an unexplained failure.

Other next-hop types support the platform's virtual-network, gateway, and internet paths where applicable. Their meaning depends on the actual routing configuration. A next hop labeled Internet does not guarantee that a private VM has a usable outbound translation method. The explicit outbound requirements discussed in Section 6.2 still apply. Routing and outbound connectivity are related prerequisites, not identical features.

Associate the table with the correct subnet

A user-defined route table affects a subnet through its association. Creating routes in an unattached table does not change a VM's path. This is a common source of confusion because the route objects can look correct in isolation. The administrator should inspect the subnet's route-table reference and then the effective routes on a relevant network interface. The association connects the intended rule to the actual source population.

One route table can serve multiple subnets where the design calls for identical routing, but those subnets may have different workloads and dependencies. A change to the shared table affects all its associations. The change review should list them before an update. Reusing a route table reduces duplication, yet it also creates a shared operational dependency that deserves clear ownership.

Gateway route propagation can influence the effective set when a gateway supplies learned routes. Disabling propagation is a design choice with consequences, not a generic troubleshooting step. The administrator should understand which routes would disappear and which workloads depend on them. Gateway-specific restrictions must be respected. A route-table toggle should not be changed simply because a path is difficult to explain.

The return path is part of the connection

Successful communication usually requires a usable path in both directions. A request can reach a server while the reply follows a different route and is dropped by a stateful device. This is often called asymmetric routing. The administrator should trace the return destination from the server's perspective, not assume that the forward path automatically establishes a matching return route. Different subnets can have different route tables.

For example, a client subnet sends traffic through a firewall to a server subnet, but the server's route returns directly to the client. Depending on the firewall's stateful processing and topology, the flow can fail or behave unpredictably. The correction may require consistent route design rather than an extra allow rule. A packet's direction and the source subnet's effective routes both matter.

Network address translation can further change the addresses seen by devices along the path. The administrator should record the original and translated addresses where the design uses translation. A security rule or diagnostic query that uses only the original source can be misleading after translation occurs. The route investigation should follow the packet as each relevant device actually sees it.

Inspect effective routes before changing them

An effective route view on a VM's network interface helps reveal the combined routes relevant to that interface. Network Watcher's next-hop capability can help identify the selected next hop for a destination under supported conditions. These tools answer routing questions more directly than a failed ping. Their prerequisites and resource state should be checked before interpreting a missing result as a network failure.

A useful route investigation records the source interface, destination IP, matching prefix, route source, next-hop type, and next-hop address where applicable. It also records the subnet association and any recent change. That evidence can explain why one destination works while another fails. The most specific route may differ even when the destinations share a similar-looking address.

A route correction should be narrow

Suppose a route incorrectly directs `10.61.0.0/16` to a retired appliance. The administrator confirms the intended current path and checks every subnet using the table. The correction updates or removes that specific route according to the approved design. Deleting the entire table would also remove unrelated routes and could create new failures. A precise diagnosis supports a precise change.

After the correction, the effective route is checked again and the intended application flow is tested. If the route is now correct but the connection still fails, the investigation continues to security rules, guest networking, DNS, or application behavior. A route fix is successful when the routing evidence matches the intended path. End-to-end success remains a separate, broader verification.

Routing becomes manageable when the administrator reasons from the destination and the source subnet rather than from resource names or diagrams alone. Prefix specificity, route source, association, next-hop readiness, and return behavior together explain the path. Those same observations form the foundation for systematic connectivity troubleshooting.

6.5 Tracing Connectivity Problems

Describe one failing flow precisely

A connectivity investigation should begin with one source, one destination, one protocol, and one port. “The network is down” is too broad to test. “The client at `10.60.1.4` cannot establish TCP to `10.61.1.4` on port `443`” gives the administrator a concrete flow. The time and observed error should also be recorded, because configuration and service state can change during an incident.

The source context includes the actual network interface and subnet. The destination can be a private address, public endpoint, or name that resolves differently depending on the client. The administrator should identify which form the application uses. A test against a raw address and a test against a hostname can exercise different behavior, especially when DNS directs the client toward a private endpoint or a public service address.

The protocol matters because not every service responds to every test. A failed ICMP ping does not prove that an HTTPS endpoint is unavailable. A service can block ICMP while accepting TCP on its

application port. The test should match the required application flow as closely as practical. Diagnostic convenience should not be confused with relevance to the reported problem.

Separate name resolution from transport

The first question for a hostname-based request is whether the name resolves to the intended address from the actual source environment. A correct record viewed from an administrator's laptop does not prove that a VM using custom DNS receives the same answer. DNS servers, private-zone links, forwarding, and cache state can affect the result. The investigation should capture the client's actual answer and resolver path.

If the name resolves incorrectly, changing network security rules for the intended address may have no effect because the client is contacting another destination. If the name resolves correctly, the administrator can continue to route and transport checks. This separation reduces unnecessary changes. It also prevents a temporary hosts-file entry from being mistaken for a complete DNS repair.

The application may require the hostname for certificate validation or virtual hosting, even when the address is known. A raw-IP test can confirm part of the transport path while failing at the application layer for a legitimate reason. The administrator should state what each test demonstrates. A successful TCP connection proves less than a successful authenticated application transaction, and that narrower result is still useful.

Trace the Azure path

The source subnet's effective routes determine the outgoing next hop. Peering state and address-space synchronization matter for cross-network traffic. Network security groups can filter the flow at applicable subnet and interface locations. A firewall or network appliance can add another forwarding and filtering layer. The administrator should draw the actual path and inspect each relevant control instead of assuming that one Azure object owns the whole connection.

The destination's return path must also be checked. A one-way packet capture or a successful next-hop lookup from the source cannot prove that replies return correctly. If stateful devices are involved, asymmetric routing can be especially significant. The investigation should compare forward and return routes and note any translation. A complete flow has two directions even when the user's complaint describes only the initial request.

For internet-bound traffic, the explicit outbound method must be identified. A private subnet with no configured egress can have a plausible-looking default route while lacking usable public connectivity. A failed package download in that environment should not immediately be blamed on DNS or the remote repository. The administrator first checks whether the network design supplies the required outbound path under the subnet's actual configuration.

Use Network Watcher for the question it answers

Network Watcher provides diagnostic capabilities for supported Azure infrastructure networking. IP flow verification can help evaluate whether a specified flow is allowed or denied by applicable security rules. Next-hop inspection addresses route selection. Connection troubleshooting examines supported

connectivity paths. The administrator should choose the tool that matches the current hypothesis and confirm its prerequisites, rather than run every tool and collect unexplained outputs.

An allowed result from an NSG-oriented diagnostic is not proof that the guest operating system accepts the connection. A correct next hop is not proof that the application process is listening. Each result narrows the failure domain. The strongest investigation combines those bounded observations with guest and application evidence. That approach produces an explanation rather than a pile of green status messages beside a still-failing application.

Diagnostic observation	What it supports	What remains possible
Intended DNS answer	Client resolved the expected destination	Route, filtering, or application failure
Correct next hop	Routing selects the intended path	Device forwarding or later filtering failure
NSG flow allowed	Evaluated NSG rules permit the specified flow	Guest firewall or missing listener
TCP connection succeeds	Transport to that endpoint works	TLS, authentication, or application error

This table helps keep conclusions proportionate to evidence. An administrator should not report that networking is fully verified after checking only one layer. Equally, a successful lower-layer test is valuable because it directs attention away from already verified components. The aim is to reduce uncertainty one observation at a time.

Inspect the guest and the application

A virtual machine can have correct Azure networking while its guest firewall rejects the port. The application might be stopped, listening only on the loopback address, or bound to a different port. The administrator should inspect the service process, listener, and guest firewall using the operating system's supported tools. Azure routing cannot make a process accept connections on an address it is not using.

The destination's response also matters. A connection refused response often suggests that the host was reached but no suitable listener accepted the request, while a timeout can have several causes. These observations are clues, not universal diagnoses. A firewall can reject actively or drop silently, and applications can fail in different ways. The administrator should combine the symptom with route, filter, and process evidence.

Authentication failures after a successful connection belong to another layer. An HTTP authorization response, for example, can demonstrate that a server received and processed the request enough to reject access. The network may be functioning while the identity or application permission is wrong. Returning to the distinction from Chapter 3 prevents a network team from opening unnecessary ports to solve an authorization problem.

Compare a working flow with a failing one

A controlled comparison can be highly informative. If one VM in a subnet succeeds and another fails, interface-specific configuration, guest state, or identity becomes more likely than a shared peering failure. If every VM in one subnet fails while another subnet succeeds, the route-table or NSG association becomes a stronger lead. The comparison should keep destination, protocol, port, and test time as similar as practical.

Recent changes provide another useful clue. A new route, changed security rule, DNS update, or endpoint configuration can explain the start of a failure. The administrator should inspect change records and activity evidence without assuming that the most recent change must be guilty. A hypothesis is useful when it leads to a targeted check. It becomes harmful when it replaces evidence and causes unrelated rollback.

Make one controlled correction

Once the failing control is identified, the administrator changes the narrowest relevant configuration and repeats the original test. Opening every port, deleting a route table, and adding a public IP simultaneously can make a connection work while hiding the cause and expanding exposure. A single controlled correction preserves the ability to explain why the result changed. Temporary diagnostic changes must be removed afterward.

The final record should state the original flow, failed layer, evidence, correction, verification, and cleanup. If the root cause remains uncertain, the record should say so and identify the next investigation. A plausible story should not be presented as a proven cause. Reliable networking work depends on reproducible observations, especially when several teams own different parts of the path.

The investigation follows a clear path: confirm the address and name, inspect routes and peerings, evaluate filters, verify the guest listener, and test the application. Chapter 7 examines the controls that deliberately permit or restrict those paths, using the same precise flow description.

Guided lab

Objective: Create two nonoverlapping virtual networks, establish two-sided peering, and inspect a route-table association without deploying virtual machines.

Prerequisites and permissions: Use an approved Azure public-cloud training subscription, Bash with Azure CLI, and resource-group plus network administration rights. Choose an approved region using `location='<approved-region>'`. The lab deploys a group, two basic virtual networks, subnets, peerings, and a route table. It deploys no VM, public IP, NAT gateway, firewall, or gateway. Basic VNets and route tables have no direct service charge; peering traffic can be chargeable, but this configuration-only lab creates no workload traffic. An automatically enabled Network Watcher resource may appear in its own managed group; do not delete shared Network Watcher resources as part of this lab.

1. Select the training subscription and create `rg-compass-ch06` with `az group create --name rg-compass-ch06 --location "$location"`.
2. Create the first network: `az network vnet create --resource-group rg-compass-ch06 --name vnet-compass-ch06-a --location "$location" --address-prefixes 10.60.0.0/16 --subnet-name snet-app --subnet-prefixes 10.60.1.0/24`.

3. Create the second: `az network vnet create --resource-group rg-compass-ch06 --name vnet-compass-ch06-b --location "$location" --address-prefixes 10.61.0.0/16 --subnet-name snet-app --subnet-prefixes 10.61.1.0/24`.
4. Set `a_id=$(az network vnet show -g rg-compass-ch06 -n vnet-compass-ch06-a --query id -o tsv)` and `b_id=$(az network vnet show -g rg-compass-ch06 -n vnet-compass-ch06-b --query id -o tsv)`. Confirm their subscription and group paths.
5. Create A to B using `az network vnet peering create -g rg-compass-ch06 --vnet-name vnet-compass-ch06-a -n a-to-b --remote-vnet "$b_id" --allow-vnet-access`. Create the reverse with `az network vnet peering create -g rg-compass-ch06 --vnet-name vnet-compass-ch06-b -n b-to-a --remote-vnet "$a_id" --allow-vnet-access`.
6. Inspect both peering lists and their states. The expected final state is Connected on both sides. This verifies configuration, not an application connection; no endpoint VM exists to send test traffic.
7. Create `rt-compass-ch06` with `az network route-table create -g rg-compass-ch06 -n rt-compass-ch06 --location "$location"`. Add a deliberate discard route using `az network route-table route create -g rg-compass-ch06 --route-table-name rt-compass-ch06 -n discard-test --address-prefix 192.0.2.0/24 --next-hop-type None`. The prefix is reserved for documentation and is different from both peered networks.
8. Associate the table with A's subnet: `az network vnet subnet update -g rg-compass-ch06 --vnet-name vnet-compass-ch06-a -n snet-app --route-table rt-compass-ch06`. Inspect the subnet reference and route definition. Explain that traffic matching the discard prefix would be dropped if a workload used that path. No packet result is claimed.
9. Delete `rg-compass-ch06` with `az group delete --name rg-compass-ch06 --yes`. Verify `az group exists --name rg-compass-ch06` returns `false`. The group deletion removes both lab networks, peerings, and route-table objects.

The lab is independent and has not been executed during authoring. It intentionally avoids internet-dependent installation commands and makes no assumption about implicit outbound access. The controlled variation is the explicit discard route, examined as configuration rather than a fabricated traffic test.

Chapter synthesis

Address ranges establish where resources can be placed. Subnets divide those ranges and attach routing and security behavior. Peering connects nonoverlapping networks without creating automatic transit or administrative access. Routes select next hops, while explicit outbound design supplies the required public egress path. Each element contributes one part of a working connection.

Troubleshooting succeeds when those parts are tested separately and then assembled into an explanation of one flow. Names, routes, filters, guest listeners, and application authorization all matter. A correct result at one layer narrows the investigation without proving every other layer. That discipline makes network security controls easier to apply in the following chapter.

Review questions

06-Q1. How many total IPv4 addresses are in a /26 prefix before platform reservations?

- **A.** 32
- **B.** 64
- **C.** 128
- **D.** 256

06-Q2. Which pair is suitable for ordinary peering address-space planning?

- **A.** 10.60.0.0/16 and 10.60.1.0/24
- **B.** 10.60.0.0/16 and 10.60.0.0/16
- **C.** 10.60.0.0/16 and 10.61.0.0/16
- **D.** Two identical ranges with different VNet names

06-Q3. In an Azure subnet 10.60.1.64/26, which listed address is the first ordinarily assignable address?

- **A.** 10.60.1.68
- **B.** 10.60.1.64
- **C.** 10.60.1.65
- **D.** 10.60.1.127

06-Q4. A private-subnet VM has a default route but no explicit outbound method. Which assumption is unsafe under modern defaults?

- **A.** The subnet can contain private addresses
- **B.** The route table can be inspected
- **C.** The VM can need DNS
- **D.** The VM will automatically reach public package repositories

06-Q5. A is peered with B, and B with C. What does ordinary peering alone establish?

- **A.** Automatic A-to-C transit
- **B.** No automatic A-to-C transit through B
- **C.** A shared Azure Owner role
- **D.** A single merged subscription

06-Q6. Routes 10.61.0.0/16 and 10.61.1.0/24 both match destination 10.61.1.20. Under ordinary prefix selection, which is more specific?

- **A.** The /16 because its number is smaller
- **B.** The route whose name comes first
- **C.** The /24
- **D.** Neither route can ever match

06-Q7. A user-defined route table contains correct entries but is not associated with the intended subnet. What is the likely consequence?

- **A.** Those entries do not govern that subnet merely because the table exists
- **B.** Every subnet automatically uses the newest table
- **C.** The table replaces every NSG
- **D.** The route grants application authorization

06-Q8. A diagnostic confirms NSG permission for a TCP flow, but the application still fails. Which next statement is most accurate?

- **A.** All networking and application behavior are proven healthy
- **B.** The user must receive Owner
- **C.** Every peering must be deleted
- **D.** Guest firewall, listener, routing, or application behavior can still require investigation

Further reading

- Azure Virtual Network overview
- Private IP addresses
- Virtual network peering
- Azure traffic routing
- Default outbound access changes
- Network Watcher overview

Chapter 7: Controlling What Can Connect

A network path becomes useful only when the right traffic can use it. Controlling What Can Connect builds on Chapter 6 by examining the controls that permit administrative sessions, application flows, and access to platform services. Network security groups filter traffic, application security groups simplify rule targets, Bastion supports private VM administration, and endpoint options shape service access. The administrator's task is to select the control that matches the required connection and verify its boundaries.

Learning objectives

The reader should be able to interpret NSG rules and their combined effect, use application security groups appropriately, explain Bastion's connection path and prerequisites, distinguish service endpoints from private endpoints, and diagnose private-service access without confusing DNS, networking, and identity authorization. The reader should also understand which controls leave public service access unchanged unless separately configured.

7.1 Using Network and Application Security Groups

An NSG filters a defined flow

A network security group, or NSG, contains rules that allow or deny network traffic for supported Azure resources. Rules describe direction, protocol, source, source port, destination, destination port, priority, and action. These fields define a flow more precisely than a statement such as “allow the application.” The administrator should translate the application's communication requirement into those fields before writing a rule.

For a web service, the destination port might be TCP 443, while the client source port is normally selected dynamically. A rule that unnecessarily fixes the source port to 443 can fail to match ordinary client connections. Source and destination ports describe different ends of the flow. Understanding that distinction avoids a common mistake made when a service name is copied into both fields without considering how connections are established.

NSGs can be associated with subnets and supported network interfaces. A subnet association applies a common filtering context to the relevant resources in that subnet. An interface association can add another layer for a particular VM interface. The administrator should choose a consistent design and avoid unnecessary duplicated rules. Multiple locations can be useful, but they also increase the amount of state that must be inspected during troubleshooting.

Priority determines which matching rule applies

NSG rules are evaluated in priority order, with lower numerical values taking precedence. The first applicable matching rule determines the allow or deny decision within that NSG and direction. A specific allow rule placed after a broader matching deny will not be reached. The administrator should read the ordered rule set rather than simply confirm that an allow rule exists somewhere in the list.

Custom rule priorities occupy a supported range, and priorities must be unique within the relevant rule collection and direction. The exact current limits should be checked when building automation. A useful numbering convention leaves gaps so related rules can be inserted without renumbering everything. The convention helps maintenance, but the actual numeric order controls behavior. A descriptive rule name cannot override its priority.

Default rules provide baseline behavior, including permitted virtual-network traffic and deny behavior for unmatched inbound traffic. They cannot simply be deleted as ordinary custom rules, but higher-priority custom rules can change the resulting decision for matching traffic. The administrator should inspect defaults rather than assuming an empty custom rule list means no traffic is allowed. Default behavior is still active behavior.

Service tags describe maintained address groups

A service tag represents a Microsoft-maintained group of IP prefixes for a defined service or network concept. It can reduce the need to maintain changing public address lists manually. The tag's meaning and supported regional scope should be checked in current documentation. A name that sounds broad, such as `VirtualNetwork`, may represent more than the address range an administrator first imagines because connected network context can influence it.

Service tags do not provide application identity authorization. Allowing traffic to a service tag establishes a network filtering decision, not permission to read data from every service instance covered by the tag. The target service's authentication and authorization still apply. Conversely, an appropriate data role does not override an NSG deny that prevents the network connection. Both controls can be required for the same request.

Application security groups express workload roles

An application security group, or ASG, groups supported network-interface IP configurations so NSG rules can refer to an application role instead of individual addresses. A web-tier ASG and a database-tier ASG can make a rule easier to understand: permit the required database port from the web group to the database group. The membership then connects the logical role to actual interfaces.

An ASG is not a Microsoft Entra security group. It groups network endpoints for filtering, not people for Azure role assignment or product licensing. The shared word group can mislead learners, but the objects participate in different systems. Adding a user to an Entra group does not place a VM interface in an ASG. The administrator must inspect the relevant network membership.

ASGs have scope and membership constraints that should be checked for the intended virtual-network design. They are not a universal cross-region or cross-network naming system for arbitrary endpoints. A rule using ASGs should be validated against the supported placement of its members. A clear logical model is useful only when the platform can evaluate it for the actual interfaces involved.

Control	Represents	Used by
NSG	Ordered allow and deny rules	Supported network filtering points

Control	Represents	Used by
ASG	A set of supported interface IP configurations	NSG source or destination selection
Service tag	Maintained service or network address prefixes	Supported security rules and services
Entra security group	Directory principals	Identity and authorization systems

This comparison helps prevent a naming shortcut from becoming a design mistake. The administrator should identify the object type and consuming service before deciding what membership means. A group can be useful in several parts of Azure, but those parts do not share one universal group mechanism. Each relationship must be configured and verified in its own system.

NSGs maintain connection state

NSGs are stateful. When an allowed flow is established, response traffic is handled according to the connection state, so an administrator does not normally create a mirror rule merely to permit the response. This is different from authorizing a new independent connection in the opposite direction. A server initiating a separate outbound connection still needs the applicable rules for that new flow.

Rule changes generally affect new connections rather than instantly terminating every established flow. A test that reuses an existing session can therefore appear to ignore a newly introduced restriction. The administrator should establish a fresh test connection when validating the rule change. The observation must distinguish an existing tracked flow from a new flow that exercises the current rules.

Stateful behavior does not eliminate the need for a valid route or a listening application. An NSG can allow a packet that never reaches a working service. The filtering result is one prerequisite in the connection. The path-oriented method from Chapter 6 still applies: destination, routing, filter, guest firewall, listener, and application behavior each need their own evidence when a failure remains.

Design a narrow rule from a requirement

Suppose a private application tier needs to reach a private data tier on one TCP port. The administrator identifies the actual source endpoints, destination endpoints, protocol, and destination port. A rule can use appropriate ASGs to express the two tiers. A broad allow from any source to any destination port would satisfy the one connection but also permit unrelated traffic. The rule should match the requirement's boundary.

The design should also consider what defaults already allow. If the goal is to restrict traffic within a virtual network, a specific allow may need to be paired with an appropriately ordered deny for other relevant traffic. Adding only an allow rule does not necessarily narrow a preexisting broad default allowance. The administrator must evaluate the complete ordered set, including defaults, before claiming that segmentation has been achieved.

Review and maintain rules

Each custom rule should have a purpose, owner, and reason for its source and destination breadth. Temporary diagnostic rules should have a removal condition. A rule named `AllowTest` that remains indefinitely can become an unexplained path months later. The review should compare the rule with actual application requirements and current ASG membership, not merely confirm that its syntax remains valid.

Unused or shadowed rules should be investigated before removal. A rule can appear unnecessary during a short observation window while supporting a periodic task. The owner should provide the requirement and evidence. Equally, a rule should not remain forever because nobody knows whether it is needed. A documented review process turns uncertainty into an action with an owner and a deadline.

NSGs are most effective when the rule set reads as a clear explanation of permitted flows. ASGs and service tags can make that explanation more maintainable, while priority and state determine the actual behavior. The next step is to combine the applicable filtering points and determine what a particular interface really permits.

7.2 Interpreting Effective Security Rules

A local allow is not the complete result

When an NSG applies at both a subnet and a network interface, traffic must satisfy the relevant rules at both locations. An allow in one NSG does not cancel a deny in the other. This is a frequent cause of confusion because an administrator may inspect the more visible rule set and stop there. Effective-security analysis follows the actual interface and its subnet to collect the complete filtering context.

For inbound traffic, the subnet NSG is evaluated before the network-interface NSG where both are present. For outbound traffic, the interface and then the subnet provide the relevant sequence. The operational result still requires permission through every applicable filtering point. The administrator should know the direction of the flow and inspect the correct inbound or outbound rules. A perfectly written inbound rule does not explain an outbound failure.

If no NSG is associated at a particular supported point, there is no NSG rule set there to inspect, but other controls can still apply. The absence of an interface NSG does not mean the VM is unfiltered if a subnet NSG applies. Conversely, a subnet with no NSG does not erase a restrictive interface NSG. The association inventory is the starting point for a meaningful effective-rule review.

Use the exact flow fields

A rule matches more than a port number. Source address, destination address, protocol, direction, and source port can all affect the result. The administrator should capture the full test flow before searching the rules. If the client reaches a service through address translation, the address visible at the filtering point may differ from the original client address. The investigation must use the packet as that control sees it.

Private and public paths can also expose different source or destination context. A test from an administrator's laptop may not match a rule intended for a private application subnet. The correct test

should originate from the approved source or use a diagnostic that models the specified flow. Otherwise, a denied test may simply demonstrate that the rule correctly excludes an unrelated source.

Application security group membership should be inspected for the actual interface IP configuration. A rule that references a database ASG will not match an interface that was never added to that group. The friendly resource name is irrelevant to membership. A VM called `db-server` does not automatically become part of the database ASG. This is another example of intent labels differing from configured relationships.

Read priorities within each NSG

The administrator evaluates the matching rule with the lowest priority number in each applicable NSG. Comparing a priority from one NSG directly with a priority from another does not create one combined global list. The rule sets are evaluated at their own points. An interface allow with priority `100` does not override a subnet deny with priority `200` merely because its number is smaller.

This distinction is easiest to see in a worked example. The subnet NSG allows TCP `443` from the application subnet, while the interface NSG denies that same flow. The connection is blocked at the interface layer. Reversing the arrangement also blocks the connection at the subnet layer. To permit the flow, the design must allow it through both applicable NSGs, subject to other controls on the path.

Subnet decision	Interface decision	NSG outcome for the flow
Allow	Allow	Permitted by those NSGs
Allow	Deny	Blocked
Deny	Allow	Blocked
Deny	Deny	Blocked

The first row should be read carefully. It means the evaluated NSGs permit the flow; it does not prove the route, guest firewall, or application works. A successful filter evaluation narrows the investigation. It should not be stretched into a general claim that the destination service is healthy or the caller is authorized to use its data.

Inspect the effective view

Azure provides an effective security rules view for supported network interfaces associated with VMs. It aggregates the relevant NSG information so the administrator can inspect the actual context. The VM and interface must satisfy the tool's prerequisites, including the relevant resource state. An unavailable view should be investigated as a tooling prerequisite before it is interpreted as evidence that no rules apply.

Network Watcher IP flow verification can help determine whether a specified flow is allowed or denied and identify the matching rule under supported conditions. The administrator should provide the correct direction, protocol, local and remote addresses, and ports. A diagnostic configured with reversed

endpoints can answer a different question from the original incident. Tool output is only as relevant as the flow submitted to it.

The effective view and the individual NSG definitions serve complementary purposes. The effective view helps locate the applied control. The original NSG provides the object that must be changed, with its scope, owner, and other associations. A shared NSG can affect many interfaces or subnets, so a correction should review the full association set before modifying a broad rule.

Established flows can complicate testing

Because NSGs are stateful, an existing connection can remain active after a rule change that affects new flows. An administrator should close the test session and establish a new connection when validating the new policy. Reusing a browser session or long-lived TCP connection can otherwise create a false impression that the deny rule has no effect. The test plan should state whether it checks a new or existing flow.

The same principle matters after an allow change. The client may retain an application error, DNS cache, or failed connection state unrelated to the current NSG decision. A fresh controlled test reduces those confounding factors. The administrator should record the configuration change time and the new test time so the evidence can be compared. Timing is part of the diagnosis, not incidental background.

A concrete troubleshooting sequence

A private web VM should accept HTTPS from an application subnet, but the connection times out. The administrator confirms the destination address and port, then inspects the subnet and interface NSG associations. The subnet rule allows the flow, but the interface NSG contains an earlier matching deny. The effective view and a matching diagnostic identify that rule as the filtering cause.

The correction is to modify the intended interface policy or remove an obsolete interface NSG according to the approved design. It is not to add a public IP or grant Owner to the connecting user. After the change, the administrator starts a new connection and verifies the result. If transport still fails, the next checks concern routing, the guest firewall, and the listener. One confirmed defect can coexist with another.

If the rule uses ASGs, membership is checked before changing the rule's breadth. A missing interface membership can explain why a narrow allow fails to match. Adding an Any source might make the test pass while hiding the membership error and allowing more traffic than intended. Correcting the intended relationship produces a safer and more understandable result.

Keep effective-rule reports useful

A useful report includes the source and destination flow, interface ID, subnet ID, associated NSGs, matching rules, priorities, and test time. It also names the owner of the rule that requires change. This is more actionable than a screenshot of an NSG with several green allow rows. The report should explain which rule matched and why, not merely show that rules exist.

The final verification should include a permitted flow and, where practical, a nearby flow that should remain blocked. For example, the approved source can reach HTTPS while an unapproved source does

not. Tests should use disposable or approved endpoints and avoid disruptive probing. The boundary test confirms that the correction did not broaden access beyond the requirement.

Effective-security reasoning is ultimately a combination of association, direction, matching fields, priority, and state. Once those elements are explicit, a deny becomes explainable and an allow becomes testable. The administrator can then choose a suitable administrative access path without opening VM management ports indiscriminately to the internet.

7.3 Providing Administrative Access with Bastion

Separate the administrator's path from public VM exposure

Azure Bastion provides managed connectivity for Remote Desktop Protocol and Secure Shell access to supported virtual machines. It allows an administrator to reach a VM through a controlled service path using the VM's private address, so the target VM does not need its own public IP for that connection. This reduces the need to expose management ports directly on every VM. It does not eliminate the need for guest credentials, authorization, or network controls.

The connection has distinct segments. The administrator reaches Bastion through the supported client experience, and Bastion reaches the target VM's private address on the relevant management port. A failure can occur on either segment. The administrator should identify which stage failed rather than treating the whole session as one opaque tunnel. The target VM's listener and guest firewall remain part of the second segment.

Bastion is a platform service, not an ordinary jump-host VM that the customer patches manually. Its supported capabilities depend on the chosen SKU and deployment model. Browser-based access, native-client features, scaling, and private-only options have specific availability and prerequisites. The administrator should check the current feature comparison before choosing a SKU. A feature seen in one tutorial should not be assumed to exist in every Bastion offering.

Understand the deployment requirements

For dedicated Bastion deployments other than the Developer offering, Microsoft requires a dedicated subnet named `AzureBastionSubnet`. Current guidance requires `/26` or larger for new dedicated deployments. The subnet must satisfy the documented placement rules and cannot be treated as a general application subnet. Reserving the address space in advance avoids discovering during deployment that the virtual network has no suitable contiguous block left.

Public-IP requirements depend on the offering. Dedicated public-facing deployments require the documented public IP configuration, while Developer and private-only options have different requirements. The administrator should not state that every Bastion deployment always needs the same public IP arrangement. The design should name its SKU and access model, then validate the corresponding prerequisites. Specificity prevents a generic architecture description from becoming an incorrect deployment recipe.

The dedicated Bastion resource can generate charges while deployed, even when no administrator is connected. Associated resources can also contribute to cost. A training exercise should therefore avoid deploying it casually and leaving it behind. The cost decision should be explicit, with a planned duration

and cleanup owner. A budget notification remains useful but does not automatically stop the service or cap its cost.

Network rules must support the service

NSGs can be used with Bastion's subnet and target VM subnets, but the required service traffic must be allowed. Microsoft documents ingress and egress rules needed for the selected Bastion configuration. Omitting platform communication can prevent connectivity or service maintenance. The administrator should use the complete current rule guidance rather than allowing only the visible browser port and assuming the rest is unnecessary.

The target VM needs the appropriate management port accessible from the Bastion path, normally the relevant SSH or RDP port for its configured service. A rule that permits that source is narrower than exposing the same port to the entire internet. However, the administrator should verify the actual source context and supported topology. Peered-network access and other extensions depend on the chosen capabilities and configuration.

Bastion does not grant a VM an unrestricted outbound internet path. A private VM that needs package repositories, updates, or external services still needs an explicit outbound design. A successful administrative session proves that the Bastion management path works, not that every public endpoint is reachable from the guest. This distinction is especially important under modern private-subnet defaults.

Identity and guest authorization remain necessary

The user needs appropriate Azure visibility and permissions for the resources involved in the connection, according to the documented Bastion workflow. The user also needs a supported guest authentication method. A person who can view the VM in the portal may not have valid SSH credentials or the appropriate guest login role for an identity-based login configuration. Portal access and operating-system access are separate checks.

Credential handling should follow the organization's approved process. Private keys and passwords should not be copied into a shared lab document or source file. The presence of Bastion does not make a weak or mishandled guest credential safe. The service improves the connection path, while account security and authorization still require sound administration. Both aspects should be tested before the VM is considered operable.

A connection-oriented checklist

A useful preparation sequence identifies the target VM, its private address, operating system, listening management service, and allowed source path. The administrator then confirms the Bastion resource's deployment state, SKU capability, subnet, and required network rules. Finally, the selected user and guest credentials are tested through the intended client. Each check corresponds to a specific dependency in the connection.

Dependency	Evidence
Bastion deployment	Correct SKU, state, subnet, and required IP resources

Dependency	Evidence
Platform communication	Complete required Bastion NSG rules
Target reachability	Private route and allowed management port
Guest readiness	Running SSH or RDP service and guest firewall
User access	Azure permissions plus supported guest authentication

The checklist separates resource deployment from operational readiness. A successfully created Bastion resource still depends on the target VM accepting the connection and credentials. A valid user can be blocked by a subnet rule. The administrator should record which dependency has been verified and which remains uncertain when a connection fails.

Diagnose the failed segment

Suppose the Bastion interface opens but cannot establish the VM session. The administrator checks the VM's power state, private IP, management listener, guest firewall, and target NSG rules. If several VMs fail through the same Bastion deployment, a shared service or subnet dependency becomes more likely. If only one VM fails, its interface and guest configuration deserve closer attention. The pattern helps locate the fault.

If the user cannot initiate the Bastion workflow at all, Azure permissions, supported client features, or Bastion deployment state may be relevant. Repeatedly resetting the VM password does not address a failure that occurs before guest authentication. The administrator should use the visible error and resource context to identify the stage. A successful connection from a different authorized user can provide a useful comparison without proving that the failing user's permissions are correct.

If authentication reaches the guest but is rejected, the account or key should be investigated through the approved operating-system process. Opening broader network access is unlikely to repair a wrong username or invalid key. The same layer distinction used throughout the book applies here: transport establishes a path, and authentication determines whether the guest accepts the identity on that path.

Operate and retire the access service deliberately

A shared Bastion deployment should have an owner, cost allocation, access-review process, and documented dependencies. Its subnet and public endpoint where applicable should be included in the network inventory. Changes to associated NSGs or peering can affect administrators across several workloads. The change review should consider that shared operational role rather than treating Bastion as a disposable accessory to one VM.

For a temporary deployment, cleanup should remove the dedicated Bastion resource and associated lab-only public IP after confirming that no other workload uses the service. Deleting one target VM does not necessarily remove the shared Bastion resource. Conversely, deleting a shared Bastion deployment can remove the administrative path for VMs that remain. The lifecycle boundary should match the actual consumers.

Bastion provides a useful way to manage private VMs, but its benefit comes from a complete connection design. The administrator knows which service mediates access, which network rules support it, which credentials the guest accepts, and which resources generate cost. That understanding supports secure operation without assuming that a managed access service makes every surrounding control unnecessary.

Client choice should also be recorded in the operating procedure. A browser-based session and a native-client workflow can have different SKU and permission prerequisites. A support engineer should not discover those differences during an outage. The approved connection method should be exercised in advance with the actual administrative account type and target operating system, while retaining an independently documented recovery path appropriate to the organization's requirements.

7.4 Connecting Services Through Service Endpoints

Extend network identity to a supported service

A virtual network service endpoint allows a supported Azure service to recognize traffic from a configured subnet through an optimized Azure network path. It is commonly used with the service's network access rules to permit specific virtual-network sources. The endpoint is configured on the subnet, while the service resource separately decides which network sources it accepts. Both sides of that relationship matter.

A service endpoint does not place a private IP address for the platform service into the subnet. The client continues to use the service's public endpoint name and address behavior. The service can restrict that public endpoint to selected virtual networks through its supported network controls. This differs fundamentally from a private endpoint, which creates a private interface representing a connection to a particular service resource.

The word endpoint can therefore hide two different designs. An administrator should state whether the requirement is to identify an allowed subnet at a public service endpoint or to reach a service through a private IP in a virtual network. Those requirements can lead to different choices. Selecting a feature because its name sounds private enough is not a substitute for understanding the actual connection path.

Configuration has a subnet side and a service side

Enabling a service endpoint on a subnet establishes the relevant service connectivity behavior for that subnet. It does not automatically configure every storage account or other service resource to trust the subnet. The target resource's network rules must also be set according to the intended restriction. An endpoint enabled on a subnet with an unrestricted service firewall may leave the service more broadly reachable than the requirement intended.

Conversely, adding a virtual-network rule at a service without satisfying the required endpoint configuration can fail validation or fail to provide the expected access. The administrator should inspect both the subnet's enabled service endpoint and the service's allowed network list. The exact service name, subnet resource ID, and target resource ID should be recorded. Similar subnet names in different networks are not interchangeable sources.

The service must support the endpoint capability, and regional or cross-region behavior can vary by service. Administrators should consult the current service-specific documentation before applying a pattern broadly. A successful storage endpoint design does not prove that another platform service supports identical settings. The general principle is shared, but the supported resource types and constraints belong to each service.

Routing changes deserve a maintenance plan

Enabling service endpoints can change the effective route and source identity used for traffic to the supported service. Existing connections can be affected during the transition. The administrator should consider the workload's tolerance for reconnecting and test the change in a representative environment. A subnet setting that looks small can alter the path used by every relevant client in that subnet.

Service-endpoint routes have special behavior in Azure's route selection. They should not be assumed to follow a generic default route through a network appliance. If an organization requires service traffic to traverse a central inspection device, the endpoint design must be evaluated against that requirement. The administrator should inspect the effective routes and current documentation rather than assume that a `0.0.0.0/0` route captures every destination.

This is a useful example of why routing and security design must be reviewed together. A service endpoint can improve the service's ability to recognize an approved subnet, while changing the path expected by a central firewall design. Neither benefit nor conflict should be hidden. The architecture should specify which control is responsible for the traffic and how that control is verified.

Network permission is not data authorization

A service network rule decides whether the request may reach the service through an accepted network source. It does not grant the caller permission to read or write data. A client can pass the network restriction and still receive an authorization error because its identity lacks the required data role or its credential is invalid. The administrator should preserve this distinction when diagnosing service access.

For storage, the actual credential path might use Microsoft Entra authorization, a shared access signature, or another supported mechanism. Each has its own permission semantics. Enabling a service endpoint does not convert a key-based request into an Entra-authenticated request. The network path and the authorization method should both be named in the design and test record. A success using one credential does not verify a different intended method.

Requirement	Service endpoint contribution	Separate requirement
Allow a selected subnet to a supported service	Supplies subnet-aware service connectivity	Configure the service's network rule
Read a protected object	Does not grant content permission	Appropriate identity or credential authorization
Give the service a private subnet address	Does not do this	Evaluate a private endpoint

Requirement	Service endpoint contribution	Separate requirement
Allow an on-premises client	Not automatically equivalent to a subnet-originating client	Design the supported hybrid access path

The last row prevents a common overextension. A service endpoint identifies the configured Azure subnet behavior; it should not be assumed to make arbitrary connected on-premises traffic appear as an approved local VM. Hybrid access requires a design supported by the service and connectivity path. Private endpoints are often relevant when a service must be reached through private addressing from connected networks, but their DNS and routing prerequisites still need attention.

A storage access example

An application subnet needs to access one storage account while the account's public endpoint is restricted to approved networks. The administrator enables the supported storage service endpoint on the subnet and adds the subnet to the storage account's virtual-network rules. The application continues to use the normal storage service name. Its identity receives only the required data permission at an appropriate scope.

Verification has several parts. The administrator checks the subnet endpoint setting, storage network rule, client route, and authorization method. An approved client should complete the intended operation. An unapproved network should fail according to the service's configured public-network restrictions. The test should not rely on an administrator's laptop succeeding through an unrelated firewall exception, because that would not prove the application subnet path works.

If public network access is disabled entirely on the service, a service-endpoint path to the public endpoint should not be assumed to remain usable. Service-specific settings determine the effect, and a private-endpoint design may be required for the intended private access. The administrator should read the actual network-access mode, not only the list of allowed subnets. A selected-network configuration and a disabled public endpoint are different states.

Troubleshoot the two-sided relationship

If a previously working client fails after endpoint activation, the administrator examines the changed routing and the target's allowed subnet rule. If a new subnet fails while an older subnet works, the comparison should include both endpoint enablement and resource-side authorization. The problem may be a missing subnet reference rather than an incorrect NSG. Changing broad security rules without checking the service network settings can introduce unnecessary exposure.

If networking appears correct but the service returns an authorization error, the administrator investigates the principal, token, key, or signature used by the client. The service's error and diagnostic evidence help distinguish network rejection from content authorization. A successful DNS lookup of the public service name is expected in this design; it is not proof that a private endpoint is missing. The chosen endpoint model determines what the name should resolve to.

Maintain endpoint settings with workload changes

When an application moves to a different subnet, the service's network rules and the new subnet's endpoint settings may need updates. Removing the old subnet rule too early can interrupt a staged migration; leaving it indefinitely can preserve unwanted access. The change plan should include both the transition and the final cleanup. The endpoint relationship is part of the application dependency inventory.

Service endpoint policies, where supported, can further constrain access to service resources, but they are a separate capability with specific scope and limitations. They should not be assumed to exist or behave identically for every service endpoint. The AZ-104 administrator should recognize the distinction and consult current guidance when the requirement calls for that control. A basic service endpoint alone does not express every desired data-exfiltration restriction.

The service-endpoint model is strongest when its boundary is stated clearly: an approved Azure subnet reaches a supported public service endpoint under resource-side network rules, while data authorization remains separate. That clarity makes it possible to compare it honestly with the private-endpoint model rather than treating the two as interchangeable names for the same feature.

7.5 Using Private Endpoints

Bring a service connection onto a private address

A private endpoint is a network interface with a private IP address from a virtual-network subnet that connects to a supported service through Azure Private Link. It represents a connection to a particular service resource and, where applicable, a particular subresource. Clients can reach the service through that private address when routing and DNS are configured correctly. The platform service itself is not converted into an ordinary VM inside the customer's subnet.

This design differs from a service endpoint. A service endpoint retains the service's public endpoint behavior while recognizing an approved subnet. A private endpoint creates a private connection address that can be used from the virtual network and supported connected networks. The choice should follow the required reachability and control model. Neither option automatically grants permission to the service's data.

Private endpoints consume subnet addresses and can generate charges for the endpoint and relevant traffic. DNS resources or other connectivity components can also contribute cost. A lab should account for these dependencies before deployment. The absence of a public address does not imply the absence of charges. The endpoint's lifecycle should be tied to the workload or shared service that actually uses it.

The target and subresource must be correct

A service can expose several subresources with distinct private-endpoint requirements. Azure Storage, for example, has separate service surfaces such as blob and file. A private endpoint for one surface should not be assumed to provide every other surface automatically. The administrator must identify the exact endpoint the client uses and create the supported connection for that service component.

The private endpoint connection also has an approval state. Depending on the caller's permissions and service arrangement, approval can be automatic or require action by the service owner. A private endpoint object can exist while its connection is not yet approved for use. Troubleshooting should inspect that state before changing DNS or granting broader data permissions. Object creation and usable service connection are separate observations.

The request should include the target resource ID, intended subresource, subnet, address requirements, DNS design, and owner. These details make the endpoint reviewable before it is created. A generic request to “make storage private” is incomplete because the storage service surface, client networks, public-access policy, and authentication method all affect the final behavior. The administrator should turn that request into a concrete connection design.

DNS directs clients onto the private path

Clients normally continue using the service's documented hostname. DNS must resolve that hostname appropriately from the client environment so traffic reaches the private endpoint. Azure Private DNS zones and virtual-network links are commonly used for this purpose, with service-specific zone names. The administrator should use the documented zone for the exact service rather than inventing a private suffix based on its display name.

For a storage blob connection, the relevant private-link DNS relationship differs from the file-service relationship. A correct record in the wrong service zone will not necessarily direct the client's actual hostname to the intended endpoint. The administrator should inspect the complete resolution chain from the application source. A portal view showing a private IP is insufficient if the client still resolves the public address.

Custom DNS and on-premises clients require an explicit forwarding or resolver design. Linking a private zone to one Azure virtual network does not automatically teach every external DNS server how to answer that name. The client's configured resolver must reach a supported path that can resolve the private zone. The DNS chapter develops those patterns further; the key dependency here is that the actual client receives the intended private answer.

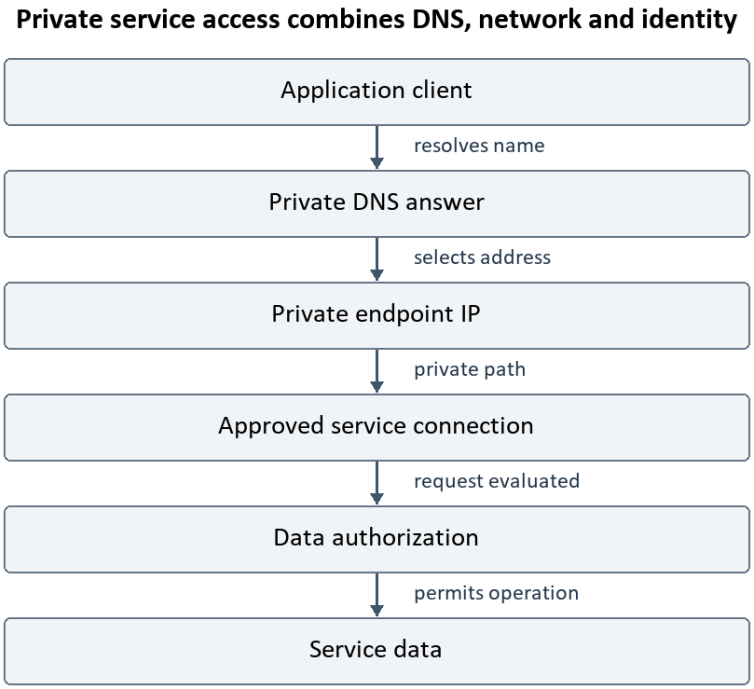


Figure 7.1 Private endpoint access needs DNS and authorization

The normal service hostname must resolve onto the intended private path. Endpoint approval and data authorization remain separate requirements, and public service access must be configured independently.

The figure separates DNS selection, private network reachability, endpoint approval, and data authorization. A failure in any one can prevent the application operation. The separation also explains why changing an access role cannot fix a hostname that still sends the client to a blocked public endpoint. The request must reach the intended service path before its authorization result can be interpreted correctly.

Public access is a separate setting

Creating a private endpoint does not automatically disable the service's public endpoint. Azure Storage explicitly documents this separation. If the requirement is private-only access, the administrator must configure the target service's public-network controls appropriately and test the effect. Otherwise, the new private path can exist alongside a still-available public path. A private endpoint is an added connection, not a universal public-access removal switch.

The public-access change should be staged with client validation. Disabling public access before every required client has a working private route and DNS path can cause an outage. The administrator should identify clients outside the linked network, automation agents, recovery processes, and administrative tools. A workload can have less obvious consumers that still use the public path. The migration is complete only when the required consumers are accounted for.

The target's data authorization remains active on the private path. A client needs the correct identity or credential and service permission. Private network placement is not proof of trusted application

identity. A compromised resource in a connected network should not receive broad data access merely because it can reach the private address. Network restriction and least-privilege authorization reinforce one another.

Network policies require current understanding

Older material often describes private endpoints as exempt from all NSG or user-defined routing behavior. Current Azure supports private-endpoint network policies under documented subnet settings and constraints. The administrator should inspect whether support is enabled for NSGs, routes, or both and follow the current service guidance. A blanket statement that NSGs can never affect private endpoints is no longer an adequate design rule.

Private-endpoint routes are specific, and steering them through an appliance requires the supported network-policy configuration and suitable route prefixes. A generic default route should not be assumed to override every private-endpoint path. The administrator must inspect the effective route and the documented prefix conditions. This is another case where a simplified routing mnemonic is insufficient for a service-specific behavior.

Changes to the subnet's private-endpoint policy setting can affect multiple endpoints in that subnet. The change review should inventory those endpoints and their consumers. A setting changed to fix one application can alter another application's path. Shared subnet design therefore needs a clear owner and consistent policy intent. The administrator should avoid treating private endpoints as isolated objects with no common network dependencies.

The policy setting applies specifically to private-endpoint behavior in that subnet; it does not replace the ordinary NSG behavior for unrelated VM interfaces there. This distinction should be preserved in change records. Otherwise, an operator may infer that changing the endpoint setting has enabled or disabled all filtering for every resource in the subnet, which is a broader and incorrect conclusion.

A layered troubleshooting example

A VM cannot read a blob after public access is disabled on the storage account. The administrator first resolves the blob hostname from that VM. If it returns a public address, the DNS path is wrong for the intended private-only design. The next checks concern the private DNS zone, record, link, and custom resolver configuration. Adding a storage data role will not repair the destination selected by DNS.

If the hostname resolves to the private endpoint's address, the administrator checks endpoint approval, routing, applicable network policies, and the service connection. If transport succeeds but the data request is rejected for authorization, the principal and data role are examined. The same user may succeed from one client and fail from another because the clients use different DNS or credential contexts. The investigation should capture both rather than assume identical behavior from a matching username.

Verify and clean up the complete relationship

A successful test should use the normal service hostname from each required client network and perform a harmless authorized operation. The administrator records the resolved address, endpoint connection state, and result. Where the design requires private-only access, an appropriate test from

outside the allowed private path should confirm the public restriction. The test must avoid using a different credential or network exception that changes the question being asked.

Cleanup should remove the private endpoint, obsolete DNS records or zone-group relationships, and lab-only network resources without deleting shared DNS zones or target services unnecessarily. A stale private DNS record can continue directing clients toward a removed address. The endpoint object and its name-resolution dependencies should therefore be reviewed together. Resource deletion alone is not proof that the client path has returned to the intended state.

Private endpoints provide a strong building block when the administrator designs the whole connection: exact service surface, approval, address, DNS, network policy, public-access setting, and data authorization. The feature becomes dependable through that combined design. A private IP shown in the portal is the beginning of the evidence, not the end of the verification.

Guided lab

Objective: Create a private subnet with an NSG, express a narrow HTTPS rule, and inspect how rule priority changes the intended result without exposing a VM or deploying paid endpoint services.

Prerequisites and permissions: Use an approved training subscription, Bash with Azure CLI, and resource-group plus network administration permissions. Set `location='<approved-region>'` after choosing an available region. The lab creates a resource group, basic VNet, subnet, NSG, and ASG. It creates no VM, public IP, Bastion host, private endpoint, NAT gateway, or charge-generating application service. These basic configuration objects have no direct service-consumption charge. No traffic test is claimed because no VM is deployed.

1. Select the training subscription. Create `rg-compass-ch07` with `az group create --name rg-compass-ch07 --location "$location"`.
2. Create the network with `az network vnet create -g rg-compass-ch07 -n vnet-compass-ch07 --location "$location" --address-prefixes 10.70.0.0/16 --subnet-name snet-app --subnet-prefixes 10.70.1.0/24`. No internet-dependent guest operation is required.
3. Create the NSG using `az network nsg create -g rg-compass-ch07 -n nsg-compass-ch07 --location "$location"`. Associate it with `az network vnet subnet update -g rg-compass-ch07 --vnet-name vnet-compass-ch07 -n snet-app --network-security-group nsg-compass-ch07`.
4. Add a narrow illustrative allow rule: `az network nsg rule create -g rg-compass-ch07 --nsg-name nsg-compass-ch07 -n AllowAppHttps --priority 200 --direction Inbound --access Allow --protocol Tcp --source-address-prefixes 10.70.2.0/24 --source-port-ranges '*' --destination-address-prefixes 10.70.1.0/24 --destination-port-ranges 443`. The source prefix is a planned test tier, not a deployed client.
5. Add a higher-numbered deny for other inbound traffic to the application range: `az network nsg rule create -g rg-compass-ch07 --nsg-name nsg-compass-ch07 -n DenyOtherAppInbound --priority 300 --direction Inbound --access Deny --protocol '*' --source-address-prefixes '*' --source-port-ranges '*' --destination-address-prefixes 10.70.1.0/24 --destination-port-ranges '*'`.

6. Inspect `az network nsg rule list -g rg-compass-ch07 --nsg-name nsg-compass-ch07 -o table` and confirm the subnet's NSG reference. Explain why the specified HTTPS flow matches priority 200 before 300. Explain why another new inbound flow to the application range would match the deny, assuming no earlier matching custom rule. Record these as predicted rule decisions, not observed packet results.
7. Introduce a controlled configuration fault by changing the deny's priority with `az network nsg rule update -g rg-compass-ch07 --nsg-name nsg-compass-ch07 -n DenyOtherAppInbound --priority 150`. Inspect the rules and explain why the same HTTPS flow now matches the deny first. Restore priority 300 with the same update command before continuing.
8. Create `asg-compass-ch07-app` with `az network asg create -g rg-compass-ch07 -n asg-compass-ch07-app --location "$location"`. Inspect its resource type. Record that an empty ASG has no interface members and grants no user role; the lab does not pretend that it is an active application tier.
9. Delete the lab group using `az group delete --name rg-compass-ch07 --yes`. Verify `az group exists --name rg-compass-ch07` returns `false`. Do not remove shared Network Watcher resources that Azure may enable automatically in the region.

The lab demonstrates configuration and rule-order reasoning only. A later approved VM exercise can use effective security rules and IP flow verification with real endpoints. Bastion and private endpoints are omitted from this low-cost lab because their complete exercises need paid resources, service-specific prerequisites, and explicit DNS design. No live lab action was executed during authoring.

Chapter synthesis

Network access is governed by several distinct relationships. NSGs evaluate flows, ASGs identify supported endpoint groups, and effective rules combine the filtering points that apply to an interface. Bastion supplies a managed administrative path to private VMs. Service endpoints identify allowed subnet access to supported service endpoints, while private endpoints create a private service connection that depends on DNS and approval.

No single control replaces all the others. An allow rule does not create a listener, a private endpoint does not automatically disable public access, and a private network path does not grant data permission. Administrators who keep those boundaries clear can explain both the intended design and the evidence from a failed connection. Chapter 8 adds name resolution and load distribution to make reachable services usable and resilient.

Review questions

07-Q1. A client connects to a server's HTTPS service. Which port assumption best matches an ordinary TCP client flow?

- **A.** Both source and destination ports must be 443
- **B.** Destination is 443 while the source port is typically dynamically selected
- **C.** Destination must be 22
- **D.** Source port is the subnet prefix length

07-Q2. Within one NSG direction, an allow at priority 200 and a broader matching deny at priority 150 apply. What is the decision?

- **A.** Allow because it is more specific regardless of priority
- **B.** Allow because its name contains Allow
- **C.** Deny because the lower numeric priority matches first
- **D.** Ignore both rules

07-Q3. What does an application security group collect for use in NSG rules?

- **A.** Supported network-interface IP configurations
- **B.** Microsoft 365 product licenses
- **C.** Subscription invoices
- **D.** Entra users for password reset

07-Q4. A subnet NSG allows a new flow while the target interface NSG denies it. What is the result?

- **A.** The subnet allow always wins
- **B.** The smallest priority across both NSGs becomes one global rule
- **C.** The flow bypasses the interface NSG
- **D.** The flow is blocked by the applicable deny

07-Q5. A dedicated Bastion deployment is planned using an offering other than Developer. Which subnet design matches current new-deployment requirements?

- **A.** Any application subnet with a /29 prefix
- **B.** A dedicated AzureBastionSubnet with /26 or larger address space
- **C.** A subnet containing all database VMs
- **D.** No subnet under any SKU

07-Q6. What does enabling a service endpoint on a subnet do by itself?

- **A.** Give the target service a private IP in the subnet
- **B.** Grant every user permission to its data
- **C.** Establish supported subnet-aware service connectivity, while target network rules remain a separate configuration
- **D.** Disable all public service endpoints automatically

07-Q7. A storage private endpoint is created successfully. Which additional action is needed if the requirement is private-only access?

- **A.** Configure and verify the storage account's public-network restriction separately
- **B.** Assume public access is already disabled
- **C.** Delete the storage data roles
- **D.** Rename the subnet to Private

07-Q8. A VM resolves a storage hostname to a public address after the account's public access is disabled. What should be investigated first for the intended private path?

- **A.** A broader Owner assignment
- **B.** The NSG rule description spelling
- **C.** The user's product license count
- **D.** Private DNS records, links, and the client's resolver path

Further reading

- Network security groups
- Application security groups
- Diagnose traffic filtering
- Bastion overview
- Bastion configuration settings
- Bastion NSG requirements
- Service endpoints
- Private endpoint overview
- Private endpoint DNS
- Private endpoint network policies
- Storage private endpoints

Chapter 8: Making Services Reachable and Reliable

A usable service needs more than an address. Its name must lead to the intended destination, traffic must reach a working instance, and failures must produce evidence that an administrator can interpret. This chapter connects the networking foundations to those operational results. Public naming, private naming, and load distribution are related, but each solves a different problem.

Learning objectives

- Host and verify public and private DNS records.
- Explain delegation, caching, virtual network links, and resolver selection.
- Distinguish the frontend, backend pool, rule, and probe of a load balancer.
- Choose probes that represent useful application health.
- Diagnose reachability through a sequence of testable hypotheses.

8.1 Publishing Names with Azure DNS

Naming a service without confusing its location

The Domain Name System, or DNS, is a distributed naming system. An application asks for an address associated with a name, receives an answer, and then attempts its connection. Those are separate operations. A correct answer does not prove that the destination accepts connections. A running web server does not prove that its name resolves correctly. Keeping that distinction clear prevents many mistaken repairs.

A fully qualified domain name describes a position in the DNS hierarchy. In `portal.example.com`, the label `portal` belongs beneath `example.com`. The owner of the parent domain can delegate responsibility for a child zone. Delegation tells resolvers which authoritative name servers hold the answers. An Azure public DNS zone is a managed location for those answers; creating the zone alone does not transfer authority from the existing DNS provider.

An organization must therefore coordinate two kinds of administration. Azure controls the zone and its record sets. The domain registrar or existing parent-zone operator controls the delegation that points toward that zone. If a newly created Azure zone contains perfect records but the parent still delegates elsewhere, ordinary internet clients continue receiving answers from the other provider. Editing the new zone repeatedly cannot repair an unchanged delegation.

The boundary also explains why hosting a zone differs from buying a domain. A registrar establishes the registration relationship for the name. DNS hosting publishes its records. Both services may be offered through a single commercial interface elsewhere, but they remain distinct responsibilities. A deployment checklist should identify the registration owner, renewal contact, authoritative provider, and person authorized to change delegation. These ownership details matter during an outage as much as technical settings.

Record sets describe different relationships

An A record associates a name with an IPv4 address. An AAAA record performs the corresponding role for IPv6. A CNAME establishes an alias to another DNS name. MX records describe mail-routing destinations, while TXT records carry text used for several purposes, including domain ownership verification. The record type is part of the question being asked. A successful A lookup says nothing about an unrelated TXT verification requirement.

Azure groups records with the same name and type into a record set. That distinction is useful when several addresses belong to one application name. The administrator manages the set rather than pretending that each address is a separate DNS name. A set can contain multiple appropriate values, subject to the rules of its record type. The returned list is not an application health check and should not be presented as equivalent to a managed load-balancing service.

The zone apex is the zone's own name, such as `example.com`, rather than a child such as `portal.example.com`. It already carries essential zone records. An ordinary CNAME cannot simply replace those required relationships at the apex. Azure alias record sets address selected integration scenarios by referring to supported Azure resources. The administrator should verify that the intended resource and record type support an alias instead of assuming every hostname can become an apex alias.

Consider a small application whose public load balancer has an assigned public address. An A record named `portal` can publish that address beneath the organization's zone. If the address is replaced, the record must be updated unless an appropriate alias relationship handles the association. The operational question is not merely whether a record can be created. It is how the record stays correct when the underlying resource changes.

Record choice	Useful purpose	Important boundary
A or AAAA	Publish an address for a name	An address change requires record maintenance unless another supported mechanism handles it
CNAME	Refer one name to another name	It is not a web redirect and has placement restrictions
TXT	Publish verification or descriptive text	It does not route application traffic
Supported Azure alias	Associate a record set with an Azure target	Support depends on the target and record type

Authority and caching explain different answers

An authoritative server answers for a zone it hosts. A recursive resolver works on behalf of a client, follows the hierarchy when necessary, and commonly caches results. The client's operating system or application may maintain another cache. Consequently, an administrator can change a record successfully while some clients continue using a previous answer. That observation is compatible with normal caching and does not automatically show failed replication.

The time to live, or TTL, tells caching systems how long an answer may be retained. It should be selected as an operational tradeoff. A shorter TTL can reduce the period during which old answers remain useful to caches, but it can increase query activity. A longer TTL can reduce repeated lookups while making planned address changes less immediate. Neither value changes an already established application connection.

A migration plan lowers the TTL sufficiently before the cutover if a shorter cache lifetime is needed. Lowering it at the same moment as the address change does not recall answers already cached with the former TTL. The plan also includes a rollback destination and evidence that both old and new endpoints can operate during the transition. DNS changes should be coordinated with certificates, firewall rules, application configuration, and public-address ownership.

Negative answers can be cached as well. A client that queried a name before its record existed may retain a not-found result for a period. An administrator who tests immediately after creating the record should distinguish a fresh authoritative query from the normal recursive path. Otherwise, a correct deployment can be mistaken for a broken one. Recording the queried name, record type, resolver address, and response makes comparisons meaningful.

A controlled publishing workflow

The following sequence separates configuration from verification. It applies whether the portal, Azure CLI, or infrastructure code creates the records. A test zone can be inspected directly without owning its public parent, but public internet delegation requires legitimate control of the real domain. Documentation examples should never imply that creating an Azure zone grants ownership of an existing domain name.

1. Confirm the intended domain and the team that controls its delegation.
2. Create the public DNS zone and record the authoritative name servers assigned to it.
3. Add the required record sets and inspect names, types, values, and TTLs.
4. Query an assigned authoritative server directly to verify the hosted answers.
5. Update the parent delegation through the domain's authorized management process.
6. Test through ordinary recursive resolvers and compare results with authoritative responses.
7. Monitor application connections separately from DNS lookup success.

For example, `nslookup portal.example.com <authoritative-name-server>` requests an answer from a particular server. The placeholder must be replaced with a server assigned to the actual zone. A successful response should contain the intended record value. The command does not prove that all recursive caches have refreshed, and it does not verify a TLS certificate. Each additional claim requires a corresponding test.

Change control belongs beside the records

DNS records deserve the same care as other production configuration. A broad permission to edit a zone can redirect an application, interrupt mail delivery, or invalidate ownership checks. Permissions should follow responsibility, and changes should be traceable to a planned outcome. Exported configuration or infrastructure code helps preserve intent, but a usable recovery process must also account for delegation outside Azure.

Suppose an application deployment succeeds but the public name resolves to an old address. The first check is the authoritative provider used by the client path. If it is the previous hosting provider, delegation is the likely issue. If Azure's authoritative server returns the new address while a recursive resolver returns the old one, caching becomes the leading explanation. If all resolvers return the new address but the application fails, investigation moves beyond DNS.

This layered diagnosis makes DNS manageable. A record is a statement about a name, a zone is a boundary of authority, delegation locates that authority, and caching affects how quickly changes appear. Each concept supplies a different piece of evidence. Reliable administration comes from checking those pieces in order instead of treating every unreachable website as a single DNS problem.

8.2 Resolving Names Privately

Private names serve a defined audience

A private DNS zone publishes names for clients that use an authorized private resolution path. It does not make a workload reachable by itself, and it does not encrypt application traffic. Its purpose is to give internal resources stable, meaningful names without publishing those names as public internet answers. That naming capability becomes especially useful when several virtual networks participate in one environment.

An internal application might use `inventory.internal.example.com` instead of embedding a virtual machine's private address in every configuration file. The name expresses a service role, while the record expresses its current location. When the service moves, the administrator can update the record without editing every dependent application. Caching still applies, so the change must be planned rather than assumed to be instant.

The audience for an Azure private zone is established through virtual network links and the DNS resolution architecture. A link makes the zone available through Azure's DNS resolution path for the linked virtual network. It is not a peering relationship. Two virtual networks can resolve records in a shared zone while lacking a route that allows their workloads to connect. Conversely, peered networks can exchange packets while private names fail because the zone is not available to the resolver path.

That distinction is valuable in design reviews. A diagram should show both data connectivity and name resolution rather than drawing one line labeled connection. Network peering, routes, security rules, and guest firewalls govern packet delivery. DNS links, resolver settings, forwarding rules, and records govern name answers. A working system needs both, but the failure evidence for each is different.

Linking does not mean registering everything

A virtual network link can enable automatic registration of eligible virtual machines. Registration creates and maintains records for those machines in the selected private zone. A resolution-only link permits lookup without automatically publishing machine records. These choices serve different purposes. A centrally managed service zone may intentionally use only explicit records, while a lab zone may benefit from automatic VM registration.

Automatic registration should not be described as a universal service-discovery mechanism. It does not infer the correct application hostname for every Azure resource. Private endpoints, managed services,

custom application aliases, and other resource types can have their own DNS integration requirements. An administrator should inspect what creates each record and what removes it. Otherwise, old manually created records can survive after a resource disappears.

The lifecycle of a record is an ownership decision. A platform-managed record should normally be changed through the platform relationship that owns it. A manually managed application alias should have a documented maintenance process. Mixing these responsibilities can lead to an administrator editing a record that automation later replaces. The apparent inconsistency is then a configuration ownership problem, not unreliable DNS behavior.

Resolver selection determines the actual path

Virtual machines obtain DNS server information through their network configuration. Azure-provided resolution supports common Azure private-zone scenarios. An environment can instead use custom DNS servers. Once that choice changes, the administrator must ensure that the custom service can resolve the required zones, either by hosting them or forwarding queries through an appropriate supported path. A zone link alone does not force arbitrary custom resolvers to consult Azure.

An application may also use a resolver setting independent of the operating system. Containers, appliances, and application libraries can complicate the path. When one application fails while another on the same machine succeeds, the difference may be resolver selection, caching, or the exact queried name. A useful diagnostic captures the real application query rather than relying only on a convenient command executed from a different context.

Azure DNS Private Resolver supports managed resolution patterns between Azure and other networks. Its endpoints and forwarding rules belong to the larger DNS design. For an AZ-104 learning environment, understanding the need for a resolver path is more important than turning this chapter into a hybrid architecture manual. The central question remains concrete: which server receives the query, and how can that server obtain an authoritative answer for the requested suffix?

Split-horizon naming needs deliberate boundaries

Split-horizon DNS means that the same name can produce different answers depending on the resolution context. An organization might return a public address to internet clients and a private address to internal clients. This can support a consistent application name while keeping internal traffic on private paths. It also creates a diagnostic obligation: the location and resolver used for each test must be recorded.

The existence of a private zone can change how names beneath that zone are answered. If the private zone is authoritative for a suffix but lacks a required record, a client may receive a negative result rather than the public record the administrator expected. The design must therefore account for all names needed by clients using that private view. Creating an overly broad private zone without planning its contents can break unrelated dependencies.

Private endpoints make this relationship particularly visible. A service's normal hostname can lead through aliases to a private-link-specific name, whose record supplies the endpoint's private address within the intended resolution context. Correct DNS directs the client toward the private endpoint. Correct network access then allows the connection. Correct identity and service authorization finally permit the requested operation. None of those layers substitutes for another.

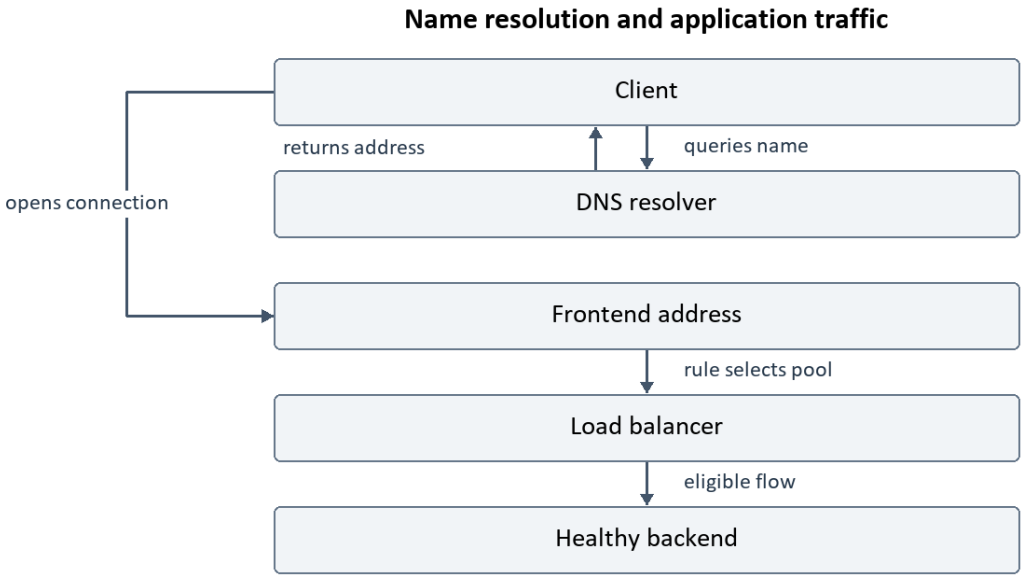


Figure 8.1 Names and traffic follow related but separate paths

DNS supplies an address; the client then opens a separate application connection. Rules and probe health select an eligible backend. A lookup alone does not prove application availability.

A worked diagnosis across two virtual networks

Consider an application VM in a client network and a service VM in a backend network. The networks are peered, and an explicit security rule permits the application's destination port. A connection to the backend's private address works. A connection using `inventory.internal.example.com` fails before it reaches the application. This evidence directs attention toward name resolution because the address-based test already establishes a useful baseline for connectivity.

The administrator checks the client's DNS server configuration and queries the full name. The private zone contains the expected record, but only the backend network has a zone link. If the client uses Azure-provided resolution, linking the client network to the zone addresses the missing resolution relationship. If it uses custom DNS, the administrator must also inspect forwarding. Adding more peering connections would not repair the missing name path.

Now reverse the evidence. The name returns the correct private address, but the application connection times out. The private-zone configuration is no longer the leading suspect. Effective routes, network security rules, the service's listening address, and guest firewall policy become more useful checks. Replacing the DNS record with another unverified address would discard evidence and introduce an additional variable.

Verification should reflect the intended clients

Testing a private zone from a laptop on an unrelated public network does not reproduce the intended environment. The test should run from a linked virtual network or from a client with a configured path

to an appropriate resolver. The query should use the full name first. Search suffixes can make short names convenient, but they can also hide which name was actually requested.

An administrator should verify positive and negative cases. A valid service name should return the expected address. An intentionally nonexistent name should fail in a predictable way. A client outside the intended private resolution path should not be assumed to receive the same answer. These tests reveal whether the design is scoped correctly, rather than merely proving that one machine happened to resolve one record.

Cleanup is also part of correctness. Deleting a test VM does not necessarily remove a manually created record, and removing a zone link can interrupt every dependent name lookup in that network. Before deleting a zone, the administrator should identify consumers, automation, and endpoint integrations. A name can appear unused in an infrastructure inventory while still being embedded in application configuration.

Private DNS is easiest to operate when four responsibilities remain visible: who owns the name, who maintains its record, which resolvers can answer it, and which networks can reach the returned address. That small model explains most everyday failures without requiring speculation. It also supports future changes because naming, resolution, and connectivity can be reviewed independently before they are brought together.

8.3 Designing Internal and Public Load Balancing

A stable destination in front of changing instances

A load balancer gives clients a destination that represents a group of backend instances. Instead of configuring each client with a list of virtual machines, the application exposes a frontend address. The load balancer then distributes eligible traffic according to its configuration. This arrangement separates the service's contact point from the individual machines doing the work. It can support both capacity growth and continued service when an instance becomes unavailable.

Azure Load Balancer operates at the transport layer for supported traffic. It uses network information rather than interpreting a web application's page paths or business meaning. That boundary distinguishes it from an application-aware reverse proxy. An administrator should choose the service because its behavior matches the requirement, not because every product with the phrase load balancing sounds interchangeable. A requirement for URL-path routing or TLS termination calls for a different evaluation.

A public load balancer has a public frontend address. An internal load balancer has a private frontend address accessible through an appropriate private network path. The backend machines do not need individual public addresses merely because the frontend is public. This separation is useful: clients reach the published service address, while administration and backend communication can remain on controlled private paths.

An internal frontend is not a special exemption from networking rules. The client still needs a route to the private address, and applicable security controls still matter. A user on an ordinary internet connection cannot reach that private destination just because its DNS name is publicly visible. Naming an address and providing a network path to it remain separate design decisions.

The frontend and backend pool have distinct jobs

The frontend IP configuration identifies the address presented to clients. A backend pool identifies the instances eligible to receive the associated traffic. A rule connects these components for a particular traffic pattern, and a health probe supplies information about backend availability. Describing those four objects separately makes a load balancer configuration easier to inspect. Each answers one question: where clients arrive, which instances serve them, what traffic is forwarded, and which instances appear healthy.

Backend membership is an infrastructure relationship, not evidence of application readiness. A VM can belong to the pool while its service is stopped, its application files are missing, or its guest firewall rejects connections. Conversely, a healthy application can be invisible to the load balancer if its network interface is not associated with the intended pool. Both conditions produce service failures, but they require different repairs.

The pool should contain instances that can correctly handle the same class of request. If one VM holds a newer application version with incompatible behavior, traffic distribution can create intermittent errors that resemble network instability. Consistent deployment becomes part of reliable load balancing. The administrator must coordinate images, extensions, configuration, secrets, and application dependencies rather than treating pool membership as the final deployment step.

Distribution is not a promise of perfect alternation

Load balancing commonly operates on connection flows. A flow includes properties such as source and destination addresses, source and destination ports, and protocol. The service uses these properties when selecting an eligible backend. Repeated browser refreshes therefore do not prove that every backend receives exactly the same number of requests. Persistent connections and client behavior influence what an observer sees.

An application test should create independent connections and inspect backend-identifying responses or server logs. Even then, equal traffic counts over a very small sample are not a meaningful acceptance criterion. The useful questions are whether healthy instances can receive traffic, unhealthy instances are excluded from new work as expected, and the service continues meeting its operational requirements. Statistical perfection is neither necessary nor a substitute for those checks.

Session persistence changes how a client is associated with a backend. It can help applications that maintain state on a particular instance, but it does not make that state durable. If the selected instance fails, another instance cannot reconstruct an in-memory shopping cart merely because a load balancer knows the original client address. Applications that need reliable state should use an appropriate shared or durable state design.

This is why adding a load balancer to two identical-looking VMs does not automatically create a resilient application. Data may still live on one local disk. A shared database may remain a single dependency. A deployment script may depend on an unavailable package source. Resilience requires examining the complete request path and the dependencies behind it, including components that do not appear inside the backend pool.

Inbound service and outbound access are different requirements

An incoming connection through the frontend and a backend-initiated connection to the internet are different network operations. A VM may successfully serve inbound traffic while being unable to download updates or call an external API. Conversely, a VM may reach the internet while the published application port remains blocked. The load balancer design must identify the intended outbound mechanism instead of relying on an accidental default.

Explicit outbound connectivity can be provided through an appropriate supported configuration, such as outbound rules or a separate network address translation design. The choice affects public source addresses, connection capacity, and operations. A lab that installs software from a public repository must state that dependency. Without it, an application deployment failure may be incorrectly blamed on the incoming load-balancing rule.

Design question	Public frontend	Internal frontend
Who can have a network path?	Internet clients, subject to the configured service and security controls	Clients with private connectivity to the frontend
What address is published?	Public IP address	Private IP address
Must each backend have a public IP?	No	No
Does the frontend grant application authorization?	No	No

Availability decisions should be visible in the design

If the application must survive the loss of an instance, the pool needs enough healthy capacity after that loss. Two instances each operating at nearly full capacity may technically provide duplication but little practical tolerance. A capacity review considers expected demand, the remaining instances during maintenance, and the time required to restore or add capacity. These are workload decisions rather than properties a load balancer can infer.

Placement matters as well. Availability zones and other compute availability choices affect which failures can remove several instances together. The load balancer's frontend configuration should be selected consistently with the intended resilience design and the capabilities available in the chosen region. A diagram labeled highly available is insufficient if all application state or all usable capacity still depends on one failure domain.

The service tier should also be current. New instructional deployments should use Standard Load Balancer rather than copying obsolete Basic examples. Historical screenshots can explain why an older environment looks different, but they should not become the basis for a new lab. Product retirements, supported combinations, and regional availability belong in the edition's technical review process.

Standard Load Balancer is secure by default for inbound traffic. The design must explicitly permit the required client traffic through applicable NSGs. A public frontend therefore does not mean that every

backend port is automatically open. This behavior should be included in deployment validation rather than discovered only after the application is announced as available.

Reviewing a concrete design

Consider an internal reporting application used by two departments. Its frontend should be private because the requirement is access from connected corporate networks, not public internet exposure. Two backend VMs run the same application release. A dedicated health endpoint reports readiness, and a rule forwards the application port. Private DNS publishes a memorable name for the frontend. The application stores persistent report data outside either VM's temporary local state.

The review asks whether both departments can resolve the name and route to its answer, whether the security rules admit their traffic, and whether the application accepts their identities. It then tests whether one backend can be withdrawn while the other serves new requests. Finally, it checks deployment and outbound dependencies. These checks turn a collection of Azure objects into a service whose behavior can be explained and repeated.

A reviewable load-balancing design names its audience, frontend, backends, health contract, capacity needs, and outbound path. Each item can be checked against configuration and observed behavior. With those decisions recorded, probes and rules become concrete expressions of the service requirement.

8.4 Configuring Probes and Traffic Rules

Health is a carefully chosen question

A health probe asks a limited question about a backend instance. The usefulness of its answer depends on how that question is designed. A network port accepting a connection demonstrates something different from an application endpoint confirming readiness. Neither automatically proves that every business operation will succeed. The administrator should define what the probe is intended to establish before choosing its protocol, port, and path.

A TCP probe tests whether a connection can be established to the specified listener. This is appropriate when that listener's availability is a useful indicator of service readiness. It does not examine the application's response body or confirm a database query. A process that accepts TCP connections while returning errors to all requests can pass such a probe. That limitation is not a defect; it is the boundary of the selected test.

HTTP and HTTPS probes request a configured path and evaluate the response according to the load balancer's probe behavior. They can support a dedicated readiness endpoint whose response reflects useful application conditions. The endpoint should be deliberately designed for automated checks. Redirecting it to a sign-in page or requiring an interactive user session makes the probe test authentication behavior rather than basic backend readiness.

For these HTTP-based probes, the intended healthy response is HTTP status `200`. A redirect, authorization failure, or missing-path response is not an equivalent success. This detail makes a direct probe-path test valuable: a browser may follow a redirect automatically and show a useful page, while the probe still rejects the original response. Inspect the status returned by the configured path itself.

A useful readiness endpoint avoids extremes

A probe that always reports success can leave broken instances receiving traffic. A probe that depends on every optional service can remove otherwise useful instances during a minor external failure. The readiness decision should represent whether that instance can accept the traffic assigned to it. A reporting server might remain useful when an optional recommendation service fails, but not when its essential report database is unreachable.

The endpoint should also be inexpensive. A health check runs repeatedly across instances, so an elaborate operation can create unnecessary load. It should not modify business data or trigger expensive work. If a probe performs a full transaction for every request, the monitoring mechanism itself can become part of the outage. A small readiness result backed by appropriately maintained internal state is often easier to operate.

The selected path must exist on every backend version. A deployment that changes `/health` to `/ready` while the probe still requests the former path can remove healthy application processes from service. Compatibility during rolling deployment therefore includes the readiness contract. The same applies to ports, protocol, and certificate requirements when HTTPS probes are used.

Probe reachability differs from client reachability

The probe originates through Azure's platform health-check mechanism. Network security rules and guest firewalls must allow the appropriate probe traffic. An NSG rule that allows only an administrator's workstation does not necessarily allow the probe. The `AzureLoadBalancer` service tag is relevant to NSG rules for this purpose. It identifies platform probe traffic, not every client request arriving through the load balancer.

Client traffic needs its own permitted path to the application port. A backend can pass its probe while legitimate clients are blocked, or clients can reach a backend directly while the platform probe is blocked. These are separate cases. Effective security rules, guest firewall rules, and application listening addresses should be inspected together. Opening every source and port is an uncontrolled workaround rather than a useful diagnosis.

The listening address is especially easy to overlook. A service bound only to the loopback interface may respond to a local test while refusing connections arriving through the VM's network interface. A local `curl` test can therefore establish that the process works without establishing network reachability. The administrator should compare local and remote tests and inspect the service's binding configuration.

Rules turn the design into a traffic contract

A load-balancing rule identifies the frontend, protocol, frontend port, backend port, backend pool, and associated health probe. Together these settings define the incoming traffic contract. A rule for one port does not expose every service on the VM. If the application listens on a different backend port, the rule must express that mapping correctly and the security policy must permit the actual destination behavior.

An inbound NAT rule serves a different purpose: it maps an incoming frontend combination toward a particular backend target rather than distributing the service across the pool in the same way as a load-balancing rule. That can be useful in selected management patterns, but it should not be confused with

application load distribution. Administrative exposure should be evaluated carefully, and private management approaches can avoid publishing management ports altogether.

An outbound rule addresses backend-initiated outbound communication. It is not a repair for an incorrect incoming application rule. Reviewing rules by direction helps avoid category errors: inbound service distribution, individual inbound mapping, and outbound translation are different functions. Their names may appear together in the portal, but their success criteria are not interchangeable.

Configuring a rule with expected evidence

The following conceptual procedure assumes an existing Standard Load Balancer, compatible frontend, backend pool, and application instances. Exact names are illustrative. The administrator should verify the installed CLI command help when adapting automation. The purpose is to make every association visible rather than relying on objects that happen to have similar names.

1. Confirm that each backend responds on the intended service port from an allowed test client.
2. Create a probe with the selected protocol, port, and application path where applicable.
3. Associate a load-balancing rule with the intended frontend and backend pool.
4. Attach the probe to that rule and inspect the resulting configuration.
5. Verify probe health before sending application test traffic through the frontend.
6. Test several independent connections and inspect backend evidence.
7. Withdraw one backend deliberately, then confirm that new traffic reaches the remaining healthy instance.

The expected result is a configuration whose references point to the intended resources and whose health signals match the application state. A successful configuration command alone is not proof of a healthy backend. Conversely, an initially unhealthy result after deployment can reflect application startup time. The test should record timestamps and allow the configured observation behavior to take effect before drawing a conclusion.

Timing and connection behavior require care

Probe intervals and failure thresholds influence how quickly health changes affect new traffic. Aggressive settings can react quickly but may interpret brief pauses as failures. Relaxed settings can reduce flapping while delaying removal of a failed instance. The correct choice depends on the workload's startup behavior, normal response variation, and tolerance for failed requests. Copying a default without understanding it leaves the operational tradeoff undocumented.

Removing an instance from eligibility for new connections does not necessarily end all existing sessions immediately. Connection behavior depends on protocol, service configuration, and the state of the application. Tests must distinguish an established connection from a newly created one. A browser with a persistent connection can continue showing the same backend after its probe status changes, which does not by itself prove that new traffic is being assigned incorrectly.

Maintenance should account for that distinction. An application can signal that it is not ready for new work while allowing existing work to finish, where its design supports that behavior. The team should define how long to wait and what evidence indicates completion. Simply stopping the VM first turns a planned maintenance event into an abrupt application failure.

A controlled failure teaches more than a green indicator

Suppose both backends serve a page successfully, but one has a probe path typo. Direct connections to that VM work, while the load balancer sends new connections only to the other instance. The useful evidence is the mismatch between application reachability and probe response. Correcting the path restores the health contract. Adding another VM or changing DNS would not address the configuration error.

Now suppose the probe is healthy on both machines but clients receive failures through the frontend. The next checks concern the load-balancing rule, client-traffic security rules, and application behavior on the mapped port. This example reinforces a central habit: each green signal has a limited meaning. Reliable operations come from combining several precise signals, not assigning universal meaning to one successful probe.

8.5 Diagnosing DNS and Load-Balancing Failures

Begin with an observable symptom

Troubleshooting starts by translating a broad complaint into a specific failed operation. An application described as unavailable might fail during name resolution, connection establishment, TLS negotiation, authentication, or a later business request. Those failures occur at different layers and produce different evidence. An administrator should capture the client location, exact hostname, protocol, destination port, time, and error before changing the environment.

The scope of the failure matters immediately. If every client fails, a shared component becomes more likely. If only one virtual network fails, its resolver, routes, and security policy deserve attention. If only one application operation fails after sign-in, DNS is less likely to be the primary cause. Scope does not prove a diagnosis, but it helps rank hypotheses and select efficient tests.

A recent change is a useful clue, not a verdict. A DNS update, certificate renewal, backend deployment, or NSG change can coincide with the outage without causing it. The administrator should state a testable hypothesis: for example, the client is receiving an old frontend address because its resolver cached the previous record. That statement suggests a direct comparison between authoritative and recursive answers.

Follow the request through distinct checkpoints

The first checkpoint is the name answer. A lookup should return the intended address or alias chain from the client's actual resolution context. If the answer is absent or unexpected, inspect the zone, record type, delegation, private-zone links, forwarding, and caching. Testing the same name from a different network can be informative, but the result should not be substituted for the failing client's evidence.

The second checkpoint is the connection to that address and port. A correct DNS response can lead to an unreachable address, an unconfigured frontend, or a blocked application port. The test should use the actual protocol where possible. A failed ping is not a complete test for a service delivered over TCP, and a successful TCP handshake is not a complete test for an HTTP application.

The third checkpoint is backend eligibility. The intended instances should belong to the correct pool and show health consistent with their application state. A VM's power state is not application health. A healthy probe is not proof that the frontend rule references that probe or pool. Configuration references should be inspected explicitly, particularly where several similar resources were created during repeated deployments.

The fourth checkpoint is application behavior. The service may reject a hostname, require a particular TLS certificate, or depend on a database unavailable to one backend. A load balancer distributes transport connections; it does not repair inconsistent application configuration. Comparing backend logs and direct tests can reveal whether the failure follows one instance, one request type, or all traffic through the frontend.

Use comparison tests carefully

A good comparison changes one meaningful variable. Testing the same request by address instead of name can help isolate DNS, but HTTPS applications may depend on the hostname for certificate validation and virtual hosting. The administrator must preserve the relevant application semantics when interpreting such a test. A certificate error caused by using a raw address does not prove the original named endpoint is broken.

Testing a backend directly can isolate frontend configuration, but it also bypasses part of the real path. Direct success establishes that the application can answer under those test conditions. It does not prove the load-balancing rule, probe, or client-facing security policy is correct. Recording what a test bypasses is as important as recording what it exercises.

Repeated frontend requests can expose a single faulty backend when some responses succeed and others fail. The strongest evidence associates each response with an instance through logs or a temporary diagnostic marker. A production response should not expose sensitive host details merely to simplify testing. Controlled lab pages can identify instances safely, while production investigations can use request identifiers and protected telemetry.

Observation	Leading checks	What it does not prove
Authoritative answer is correct; client answer is old	Resolver path and caches	Application availability
Name resolves; connection times out	Routes, rules, listeners, frontend	That DNS is defective
Backend responds directly; probe fails	Probe protocol, port, path, and permissions	That all client traffic is allowed
Probe is healthy; application errors persist	Rule mapping and application dependencies	End-to-end business readiness

Build a short evidence timeline

Distributed systems contain delays. DNS caches, role propagation, deployment completion, probe observation, and application startup do not all follow the same clock. A timeline prevents normal

transitions from being mistaken for contradictions. It records when the change was submitted, when Azure reported completion, when backend health changed, and when clients were tested. That sequence often explains apparently inconsistent results.

The timeline should include configuration versions or deployment identifiers where available. If two administrators change rules during an incident, a successful later test may be difficult to attribute. Controlled troubleshooting limits simultaneous changes and preserves the prior state. A rollback plan is most useful when it names the exact configuration to restore rather than merely promising to undo the last change.

Metrics and logs provide another view. Health-probe status and data-path observations can help distinguish unavailable backends from an inaccessible frontend. Guest logs show whether requests arrived and how the application handled them. Network tools can inspect effective security or connectivity. None of these sources should be treated as complete alone. Their value increases when timestamps and resource identities align.

Worked incident: a healthy pool with an unreachable service

An internal web application resolves to the expected private frontend address. Both backend probes report healthy. Users in one subnet cannot connect, while users in another subnet can. The evidence makes a globally broken application less likely. The administrator compares the source paths, effective security rules, and routes for the failing and working clients rather than rebuilding the backend pool.

The failing subnet is governed by a rule that permits probe traffic but omits the required client source range. The probe remains healthy because it tests from the platform's health-check source, not from the user's subnet. A narrowly scoped client-traffic rule repairs the intended path. The administrator then verifies the formerly failing client and confirms that unintended sources remain blocked.

The lesson is not to allow all traffic whenever a probe is green. It is to recognize that the probe and the user take different logical paths through the controls. A precise diagnosis preserves the security design while restoring the service. A broad allow rule might hide the symptom but remove the evidence needed to understand it.

Worked incident: a private endpoint with a public answer

A storage service has a private endpoint, but an application still resolves the normal service name through a public DNS path. The administrator inspects the actual alias chain and final address from the application network. The private-zone record exists, yet the application's custom resolver has no supported forwarding path to that zone. The resource deployment is present; the resolution integration is incomplete.

The repair addresses the resolver path and then verifies the returned private address. A second test checks network reachability, followed by a data operation using the intended identity. These separate tests prevent a DNS repair from being mistaken for complete storage authorization. The same method applies to many managed services because naming, routing, and authorization remain distinct layers.

Close the incident with a reusable explanation

An incident record should state the symptom, scope, evidence, cause, repair, verification, and remaining limitation. It should also identify how the problem could be detected earlier. A missing record may suggest a deployment validation check. A probe path mismatch may suggest a release compatibility test. An undocumented custom resolver may suggest an architecture inventory correction.

The final explanation should be specific enough that another administrator can reproduce the reasoning. Saying that the network was reset conveys little. Saying that clients used a resolver without the required private-zone forwarding relationship identifies both the failed component and the successful correction. That clarity is the foundation of dependable administration: evidence narrows the problem, a targeted change repairs it, and verification demonstrates the intended behavior.

Guided lab

Objective and boundaries

Create a nondelegated public DNS zone, inspect its authoritative answer, and demonstrate why record configuration differs from public delegation. This independent exercise does not create a web application or load balancer. Its documentation addresses represent test answers, not reachable services. The earlier load-balancer walkthrough supplies a separate configuration and diagnosis exercise; live backend failover requires an application environment.

The prerequisites are an Azure public-cloud subscription, Azure CLI in Bash, and `nslookup` on the test machine. Authenticate with `az login` if necessary and select the intended subscription with `az account set --subscription <subscription-id>`. Permission to create the resource group is required; within that group, DNS zone and record management permissions are required. Contributor at the necessary scope is sufficient for the management operations in this lab. No production zone or registrar setting is changed.

The charge-generating resources are the Azure DNS zone and DNS queries. Cloud Shell storage can incur separate charges if a storage-backed session is used. The selected location stores resource-group metadata; it is not a claim that public authoritative DNS is confined to that region. Review pricing before starting, and remember that a budget does not cap expenditure.

1. Set the location and create the isolated resource group and zone.

```
location="<supported-resource-group-location>"
group="rg-compass-ch08"
zone="training.example.com"
az group create --name "$group" --location "$location"
az network dns zone create --resource-group "$group" --name "$zone"
```

`example.com` is used as a documentation name. This zone is deliberately not delegated from its parent. Its existence in the subscription grants no ownership of that parent domain. The expected result is a zone resource with assigned authoritative name servers.

2. Add and inspect an A record.

```
az network dns record-set a add-record \
  --resource-group "$group" --zone-name "$zone" \
  --record-set-name portal --ipv4-address 192.0.2.10 --ttl 60
az network dns record-set a show \
  --resource-group "$group" --zone-name "$zone" \
  --name portal --output json
```

The expected configuration contains the documentation address and selected TTL. This is management-plane evidence that the record exists. It does not yet demonstrate a DNS answer.

3. Obtain an assigned authoritative server and query it directly.

```
server=$(az network dns zone show \
  --resource-group "$group" --name "$zone" \
  --query 'nameServers[0]' --output tsv)
nslookup portal.training.example.com "$server"
```

The expected authoritative response contains `192.0.2.10`. If the command is unavailable, install an appropriate DNS client on the test machine or use another machine that has one. Do not mistake a missing local utility for an Azure DNS failure.

4. Introduce a controlled error by querying `portla.training.example.com` against the same server. The misspelled name should not produce the intended record. Compare it with the successful query and explain why adding a load balancer would not fix a spelling error.
5. Query the correct name without specifying the authoritative server. Because the test zone has no public delegation, an ordinary recursive lookup is not expected to discover this Azure zone. Record the actual result without claiming a fixed response from every resolver. The contrast demonstrates the authority boundary.
6. Clean up the dedicated resource group after confirming that it contains only this lab's resources.

```
az resource list --resource-group "$group" --output table
az group delete --name "$group" --yes
az group exists --name "$group"
```

The final command should return `false` after deletion completes. If a lock or policy prevents deletion, inspect the failure and remove only a lab-created control that is safe to remove. Do not delete unrelated resources to force cleanup. No live Azure execution was performed during manuscript preparation.

Chapter synthesis

Names, addresses, health signals, and traffic rules form a chain of responsibilities. DNS identifies a destination within a particular resolution context. The network provides a path to it. A load balancer assigns eligible connections to backends, while probes provide limited evidence about readiness. Applications still own their data, identity checks, and business behavior. The administrator's strongest tool is a sequence of precise tests that keeps these responsibilities distinct.

Review questions

08-Q1. An Azure public DNS zone contains the correct A record. Its assigned authoritative server returns it, but ordinary internet resolvers still query the previous provider. Which change addresses the most likely cause?

- **A.** Add an NSG rule
- **B.** Update the parent delegation to the assigned Azure name servers
- **C.** Create a VM in the DNS zone
- **D.** Enable private-zone autoregistration

08-Q2. Two peered networks exchange application traffic by private IP. A client using Azure-provided DNS cannot resolve a record in a private zone linked only to the other network. What is the most direct correction?

- **A.** Add a public IP to the server
- **B.** Replace peering with a public load balancer
- **C.** Delete the existing private zone
- **D.** Link the client network to the private zone

08-Q3. An application should accept connections only from clients with private network connectivity. Which frontend matches that requirement?

- **A.** An internal load balancer with a private frontend
- **B.** A public load balancer without a backend pool
- **C.** A public DNS TXT record
- **D.** A public IP on every backend

08-Q4. An HTTP probe requests /health. The application redirects to /signin. Why can the backend remain unhealthy although a browser eventually shows a sign-in page?

- **A.** Browsers cannot access probed endpoints
- **B.** HTTP probes require public DNS
- **C.** The configured path must return the required probe success status rather than a redirect
- **D.** Every backend needs a public IP

08-Q5. A backend passes its probe, but clients from one subnet cannot connect. Other subnets work. Which investigation fits the evidence?

- **A.** Recreate every DNS record
- **B.** Compare the failing client's routes and effective security rules with a working client's
- **C.** Replace backend disks
- **D.** Remove all probes

08-Q6. A public load balancer serves inbound traffic, but backend VMs cannot contact an external package repository. What needs independent review?

- **A.** The explicit outbound connectivity design
- **B.** The public zone's TXT records
- **C.** The resource-group display name
- **D.** Only the inbound probe path

08-Q7. An administrator lowers a TTL while changing an A record. Some clients retain the former answer. Which explanation is accurate?

- **A.** Azure DNS cannot update A records
- **B.** TTL changes terminate existing TCP sessions
- **C.** All resolvers ignore TTL values
- **D.** Previously cached answers can retain their earlier cache lifetime

08-Q8. A VM answers a web request by private IP, but its HTTP probe is unhealthy. Which test directly checks the health contract?

- **A.** Purchase another domain
- **B.** Change a resource tag
- **C.** Request the configured probe port and path and inspect the original HTTP status
- **D.** Increase the DNS TTL

Further reading

- Azure DNS overview
- DNS zones and records
- DNS delegation
- Azure Private DNS
- Load Balancer components
- Load Balancer health probes
- DNS A-record CLI reference
- Load Balancer troubleshooting

Chapter 9: Choosing a Home for Cloud Data

Storage decisions begin with the data and its users. An administrator must understand how information is accessed, which failures it must survive, who may read it, and how it will move. An Azure storage account brings several settings together, but those settings do not answer every recovery or security question. This chapter develops a method for choosing an account and operating its foundational capabilities.

Learning objectives

- Select account capabilities from workload requirements.
- Compare local, zonal, and geographic redundancy.
- Explain object replication and its operational dependencies.
- Distinguish encryption from authorization and key custody from data access.
- Transfer data with explicit permissions and verifiable outcomes.

9.1 Selecting and Creating Storage Accounts

Start with the data access pattern

A storage account is an Azure resource that provides a namespace and configuration boundary for supported storage services. Depending on its type, it can provide services such as blobs, file shares, queues, and tables. The account is not merely a folder containing everything that happens to be called data. Its capabilities, endpoints, protection settings, and charging model influence how applications can use the information inside it.

Object storage suits applications that read and write objects through service APIs. A file share suits workloads that expect a file-system-style interface through a supported protocol. A queue holds messages for processing, while a table supports a particular structured storage model. Choosing among them begins with the application's interface and behavior. Renaming a document as a blob does not make a legacy application understand an object-storage API.

The administrator should describe the workload before selecting a portal option. Relevant questions include object size, access frequency, expected concurrent operations, protocol, retention, geographic use, and recovery needs. A collection of rarely accessed reports differs from a frequently updated shared workspace. A large total capacity also differs from a high transaction rate. Both can create performance or cost concerns, but they call for different analysis.

A general-purpose v2 account is a common starting point for ordinary Azure Storage workloads. Specialized account types serve particular performance or protocol requirements. Premium is not a universal switch that makes every storage service faster while preserving every feature. Supported services, redundancy options, tiering capabilities, and billing must be checked for the selected account type. A decision should name the capability required, not merely select the most expensive-looking choice.

Treat the account as a policy boundary

Several important settings apply at account scope. Resources sharing an account can therefore share constraints that would be awkward for unrelated workloads. If two applications need different redundancy strategies, security administration, or lifecycle ownership, separate accounts may make the boundary clearer. The opposite extreme also has costs: creating an account for every small object can produce an inventory that is difficult to govern and monitor.

The useful question is which resources should change together. A development environment and a production environment usually have different permission and cleanup requirements. Combining them can make a harmless lab deletion dangerous or force overly broad access. Similarly, data owned by separate teams may need independently reviewable network and encryption configurations. Account separation is a tool for expressing those boundaries, not an end in itself.

The account's resource group and subscription provide further management context. A resource group can organize related deployment resources, while Azure role assignments can be scoped at several levels. These management relationships do not automatically grant access to blob contents. An administrator who can view or configure the account may still lack a data role needed to upload a file through Microsoft Entra authentication.

Names and endpoints become application dependencies

Storage account names participate in service endpoints and must satisfy the service's naming and uniqueness rules. A globally unique suffix is therefore useful in a lab. The name should not contain secrets or personal information, since it can appear in URLs, logs, and configuration. A naming convention should balance recognizability with the limited character set and length allowed by the service.

The endpoint identifies both the account and the storage service. Blob and file endpoints are not interchangeable merely because they belong to the same account. A transfer tool or application must use the endpoint appropriate to its operation. When private endpoints are involved, DNS also becomes part of reaching that service correctly. A successful account creation does not establish every client's required resolution path.

An account name can become embedded in scripts, deployment templates, applications, and monitoring. Replacing an account is therefore a migration task, not a cosmetic rename. Before selecting a name or account type, the administrator should consider how dependencies will be discovered later. Infrastructure code and a small configuration inventory make future changes safer than scattered hard-coded URLs.

Creation should make defaults explicit

The portal offers useful defaults, but an operational design should record the choices that matter. These include region, account type, redundancy, network access, secure transport, anonymous access policy, identity strategy, and required data-protection features. Some settings can be changed later; others require a different deployment or data migration. Feature compatibility should be checked before application data accumulates.

A hierarchical namespace adds directory-oriented capabilities used by Azure Data Lake Storage scenarios. It should be selected because the workload needs those semantics, with attention to compatibility. It is not a harmless checkbox that improves every blob account. Features such as object replication and versioning have support boundaries that must be reviewed against namespace and account configuration.

Region selection also deserves a written reason. Data consumers, related compute resources, residency requirements, service availability, and recovery design can all influence the choice. The nearest-looking region is not always correct, and a region parameter in a lab is not a promise that every storage capability is available there. The deployment should fail visibly if a required capability is unavailable rather than silently selecting a different protection model.

A minimal creation workflow

The following sequence is more useful than memorizing the order of portal tabs. It can be implemented through the portal, CLI, PowerShell, or infrastructure code. Each step produces a decision or evidence that a reviewer can inspect. A lab uses an isolated account with small test data, while a production deployment also follows the organization's approval and operational processes.

1. Describe the data, protocol, access pattern, and owners.
2. Select the service and account type that support those requirements.
3. Choose the region and redundancy after checking availability and recovery needs.
4. Define network access and a data-plane authorization method.
5. Configure secure transport and required protection settings explicitly.
6. Create the account and inspect its actual properties.
7. Perform a small authorized data operation from the intended client path.
8. Record the configuration, costs to monitor, and cleanup or retirement process.

The seventh step is especially important. Resource deployment success demonstrates that Azure accepted the management configuration. It does not demonstrate that the application identity can reach and use the service. A small upload and read-back test exercises a different path. If that test fails, the administrator can investigate network access, identity, authorization, and endpoint selection without the noise of a large migration.

Compare options through requirements

Requirement	Decision to examine	Mistake to avoid
Existing application expects a file share	Supported file service and protocol	Assuming a blob container is a mounted file share
Ordinary mixed storage needs	General-purpose v2 capabilities	Selecting a legacy type from an old screenshot
Predictable low-latency object access	Appropriate premium object-storage option	Assuming every premium type supports every service

Requirement	Decision to examine	Mistake to avoid
Separate operational ownership	Account and resource-group boundaries	Granting broad account access to unrelated teams

Cost evaluation should include more than stored capacity. Transactions, retrieval, replication, network transfer, and optional protection features can matter. A low capacity charge can be offset by frequent retrieval or an unsuitable access pattern. The administrator should use current pricing information for the actual region and configuration rather than treating a book's illustrative calculation as a quote.

Diagnose creation and first-use failures separately

A creation failure may reflect a duplicate account name, unsupported region and SKU combination, policy denial, quota, or insufficient management permission. These are management-plane issues. The deployment error usually identifies the failed operation and provides a useful starting point. Retrying with random settings can produce an account that violates the intended design even if the retry succeeds.

A first-use failure after creation belongs to another investigation. The signed-in identity may lack a blob data role, the client may be outside allowed networks, or the URL may target the wrong service. A generic forbidden response should not trigger immediate account-key sharing. The administrator should determine which control rejected the request and repair that control with the narrowest appropriate change.

Choosing a storage account is therefore a small architecture exercise. The result should connect workload behavior to service capabilities, isolation boundaries, protection, and evidence of access. Once those foundations are correct, redundancy and replication can be evaluated as explicit recovery and distribution choices rather than as reassuring labels attached to an otherwise unexplained resource.

9.2 Choosing Redundancy

Define the failure that must be tolerated

Redundancy means keeping additional copies so that specified infrastructure failures do not necessarily destroy or interrupt access to the data. The phrase is incomplete without a failure scope. A failed drive, an unavailable datacenter, a zone outage, and a regional disaster are different events. Storage choices should be compared against those events rather than ranked only by the number of letters in a SKU name.

Durability and availability are related but different. Durability concerns preservation of data over time. Availability concerns whether the service can answer requests when needed. Data may remain safely stored while a particular access path is unavailable. An application may also be unavailable because its compute or identity dependency failed, even when storage remains accessible. A storage redundancy choice cannot by itself guarantee application continuity.

Locally redundant storage, or LRS, keeps copies within the primary location's local failure boundary. It protects against common hardware failures but does not provide the same isolation as copies

distributed across zones or regions. LRS can be suitable when the data is reproducible and the accepted outage or loss consequences are modest. It should be a deliberate economic and recovery decision, not the unnoticed default for every production account.

Zone-redundant storage, or ZRS, distributes copies across availability zones in the primary region where supported. This addresses a broader local failure scope. The application continues to use the storage service rather than selecting an individual zone's copy. The administrator still checks regional and account-type support. A region's support for availability zones in one service does not guarantee every storage configuration supports ZRS.

Geographic copies introduce another tradeoff

Geo-redundant storage, or GRS, adds replication to a secondary region. Geo-zone-redundant storage, or GZRS, combines zonal protection in the primary region with geographic replication. These choices address regional failure concerns, but geographic replication is asynchronous. A write acknowledged in the primary region may not yet exist in the secondary region when a disaster occurs.

That timing creates a recovery point consideration. Recovery point objective, or RPO, describes how much recent data loss the business can tolerate, usually expressed as a time window. The actual recovery position depends on replication progress and the event. An administrator should not promise zero data loss merely because the account has a geographic copy. The requirement must be compared with the documented behavior and available replication evidence.

Recovery time objective, or RTO, concerns how quickly service must be restored. Storage account failover, DNS updates, application reconnection, identity access, and dependent services can all contribute to that time. A geographic copy is an ingredient in the recovery plan, not the entire plan. Testing only that the redundancy property says GRS leaves the operational recovery sequence unproven.

Read-access variants add the ability to read from the secondary endpoint under the supported configuration. This can help an application that tolerates potentially older data. It does not make the secondary a second general-purpose writable primary. Applications must be designed to choose and handle the secondary path appropriately. A client that always uses the primary endpoint will not automatically gain every benefit merely because read access was enabled.

A secondary-read test should use information whose expected age is understood. A missing newly created object can be consistent with replication delay, while an older confirmed object supplies a better connectivity baseline. Monitoring the available replication information helps distinguish those cases. The application must also decide what to do when a secondary response is stale, since the storage setting cannot decide whether older business information is acceptable.

Compare the protection boundaries

Option family	Primary protection idea	Geographic copy	Secondary reading before failover
LRS	Local replication	No	Not applicable

Option family	Primary protection idea	Geographic copy	Secondary reading before failover
ZRS	Replication across supported zones	No	Not applicable
GRS	Local primary copies plus geographic replication	Yes	Requires an applicable read-access variant
GZRS	Zonal primary copies plus geographic replication	Yes	Requires an applicable read-access variant

The table is a conceptual comparison, not a complete support matrix. Account type, storage service, region, and feature combinations can limit the available selections. Azure Files, for example, should be evaluated using its own supported redundancy and failover guidance rather than assuming every blob-oriented capability applies. A production decision records those compatibility checks alongside the selected SKU.

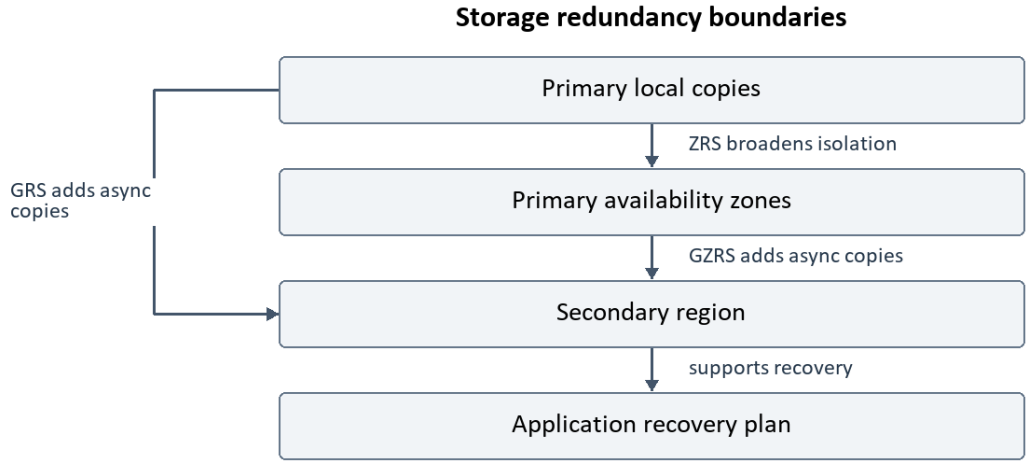


Figure 9.1 Redundancy protects different failure boundaries

Local, zonal, and geographic replication protect different infrastructure boundaries. Geographic replication is asynchronous; application recovery and protection against unwanted data changes require additional planning.

Redundancy does not preserve every earlier state

Copies maintained for infrastructure resilience generally follow authorized data changes. If an application overwrites a good object with bad content, redundancy can faithfully protect the new bad content. If an authorized deletion occurs, additional copies are not automatically a time machine. This is the central distinction between redundancy and data protection features designed for earlier states or deletion recovery.

Versioning, soft delete, snapshots, backup, and immutability address different aspects of that problem. Their detailed behavior matters. A backup can provide a recovery point independent of the current operational state. Versioning can retain earlier object versions. Soft delete can retain deleted items for a configured period. None should be selected by name alone; the administrator must understand the operation each feature protects and the restoration procedure it supports.

An accidental-deletion test therefore evaluates a different requirement from a regional-failure exercise. The first asks whether the desired item can be recovered after an authorized or mistaken modification. The second asks whether the application can continue or resume when its primary infrastructure location is unavailable. A complete storage design usually needs answers to both, even when only one event is considered likely.

Residency and dependency choices affect the answer

Geographic replication can place copies in a secondary region selected according to the service's regional pairing and supported configuration. The administrator must verify the actual locations before making data-residency commitments. A generic statement that data stays nearby is not adequate. Contracts, policy, and organizational requirements can depend on specific jurisdictions or permitted regions.

The storage location should also be considered alongside the rest of the application. A secondary storage copy has limited usefulness if the recovery environment lacks compute capacity, network paths, secrets, or authorized identities. A failover runbook should identify those dependencies and the order in which they become usable. This work connects storage administration to the broader recovery planning developed later in the book.

Changing redundancy requires a supported path

Some redundancy changes are available as configuration transitions; others involve migration constraints or service-managed conversion processes. The administrator should consult current guidance for the existing and target configuration. A change should not be treated as a reversible cosmetic edit. Its duration, costs, compatibility, and consequences for failover protection need review before execution.

Before a change, capture the account's current properties and the workloads that use it. Confirm whether applications depend on secondary endpoints, whether policies restrict the target configuration, and whether a recent recoverable copy exists. During the operation, track the service's reported state. Afterward, verify the intended redundancy and perform a small application operation. A portal notification alone does not validate every dependent workload.

A requirement-led example

A team stores generated thumbnails that can be recreated from protected originals. It accepts rebuilding them after a major outage and values a low operational cost. LRS may be a reasonable choice for that derived dataset if the documented requirements permit it. The same team stores original customer submissions in another account. Those originals need stronger resilience and a recovery method for accidental deletion, so the decision is evaluated separately.

Another application needs regional disaster recovery and can tolerate a limited amount of recent data loss. Geographic replication may support that requirement, with the exact supported option chosen after checking service and region capabilities. The recovery plan still specifies how the application reconnects, how replication status informs decisions, and how recovered data is validated. A stronger SKU name cannot replace those procedures.

The redundancy decision should satisfy the stated failure, residency, and recovery requirements at an acceptable cost. Its justification names the protected failure boundaries and the recovery work that remains. This gives the operations team a practical basis for testing the design and revisiting it when the workload changes.

9.3 Understanding Object Replication

Distribution is a different purpose from account redundancy

Object replication copies supported block-blob data between a source account and a destination account according to a policy. It can place data nearer consumers, feed processing in another location, or distribute selected results. Unlike an account redundancy choice, it creates an explicit relationship between independently managed accounts and containers. The administrator chooses what that relationship should accomplish and then monitors whether it is doing so.

The distinction affects ownership. Geographic redundancy is part of the storage account's resilience configuration. Object replication is a data-distribution arrangement with source and destination resources, permissions, policy, and application consumers. Those accounts can have their own operational settings, within feature compatibility requirements. Treating the two mechanisms as identical hides the additional decisions that a replication policy requires.

Suppose an application generates reports in one region and another workload reads them elsewhere. Replication can allow the second workload to consume a nearby copy after replication completes. It does not make each report appear at both locations at the same instant. If the consumer requires immediate confirmation of a newly written report, the workflow must account for replication status or use another design that satisfies that requirement.

Asynchronous behavior shapes the application contract

Asynchronous replication means the source operation and destination appearance are separated in time. A successful source upload does not by itself prove that a destination read will succeed. Applications should distinguish not yet replicated from permanently missing. Monitoring and retry behavior should reflect that possibility rather than immediately declaring data corruption when a recent object is absent at the destination.

The copy is also not an application-level transaction across several objects. A report may consist of a data file, an index, and a manifest. If consumers treat the presence of one file as proof that all related files are ready, they can observe an incomplete publication. A safer publication protocol uses explicit completion information and verifies the required objects before presenting the dataset as complete.

This is an example of a storage administrator contributing to application design without becoming the application developer. The administrator explains the service's consistency and timing boundaries. The

application team decides how its business operation tolerates them. A shared test then demonstrates the chosen contract using realistic object sizes and dependencies rather than only a single small file.

Prerequisites are part of the system

Object replication relies on supporting storage features. The source needs change feed, and both source and destination need blob versioning under the documented supported configuration. Change feed records relevant changes for processing, while versioning preserves object states used by the replication behavior. These are not incidental checkboxes: they influence compatibility, retained data, cost, and how later administrators can modify the accounts.

The supported object type matters. This feature is for block blobs, not a universal copier for file shares, append blobs, page blobs, and every Azure Storage service. Account type and namespace settings also have support boundaries. In particular, a design using hierarchical namespace must be checked against current object replication support rather than assumed compatible because both capabilities appear under Blob Storage.

An existing replication policy can constrain later changes to prerequisite features. Disabling versioning, deleting containers, or changing an account's recovery design should therefore include an inventory of replication relationships. An administrator who sees only the source application's dependency may overlook consumers in another region. The replication configuration belongs in the account's operational documentation and retirement checklist.

Policies define the intended relationship

A replication policy identifies the source and destination relationship and contains rules for container pairs and eligible objects. Filters can narrow the set of data that should be copied. The policy should express a business purpose clearly enough that a reviewer can explain why each destination needs the data. Replicating everything by default can create unnecessary cost and expand the locations where sensitive information exists.

The configuration workflow must establish the corresponding policy information on the participating accounts in the supported sequence. Copying a JSON example without replacing account and container identifiers can create a valid-looking but incorrect relationship. The administrator should inspect both sides after configuration and use a deliberately identifiable test object to verify the actual destination. Resource names alone are not sufficient evidence.

Existing data and new data also require an explicit decision. A filter based on when objects were created or the intended replication scope can affect whether older objects participate. A team expecting a full historical copy should verify that expectation before retiring a previous transfer process. The initial population should be measured and checked separately from ongoing replication of later changes.

Destination behavior deserves its own review

A destination container participating in replication is not simply a second ordinary writing location for the same application. The service imposes behavior intended to maintain the replication relationship. Consumers should use the destination according to its supported role, and writers should not assume an

active-active multiwriter system. A design requiring independent edits at both locations needs a conflict and synchronization model that object replication alone does not supply.

Version identifiers and lifecycle behavior also deserve attention. A copied object state should not be assumed to retain every source-side identifier exactly. Applications that use version-specific references must understand the destination's behavior. Similarly, deletion and version retention interact with replication. An administrator should test the operations the application actually performs rather than drawing conclusions from the word copy.

The destination's network and authorization controls remain important. Replication does not imply that every consumer should receive broad data access. A service relationship can move objects successfully while a reader receives an authorization failure. Those outcomes are compatible: one concerns replication, the other concerns the reader's identity and permitted operation. Troubleshooting should keep them separate.

Validate the relationship with a small dataset

An effective test begins with a small set of distinctive objects. Include an ordinary object, an object outside the intended filter, and a later update to an included object. Record the source properties and expected destination behavior before starting. This creates a test of the policy's meaning rather than merely demonstrating that some bytes appeared somewhere.

The administrator then inspects replication status using supported properties and verifies destination content after completion. A time delay alone is not sufficient proof of success. If a copy has not completed, the next checks include prerequisites, policy consistency, object eligibility, and service restrictions. Recreating accounts immediately can destroy the evidence needed to identify the original mistake.

1. Confirm account types, namespaces, and feature prerequisites.
2. Create or identify the intended source and destination containers.
3. Configure the policy and review both participating account relationships.
4. Upload a small included object and a deliberately excluded object.
5. Inspect replication status and compare destination content after completion.
6. Test an update and document version-related behavior.
7. Record monitoring, cost, retention, and retirement responsibilities.

Cost follows retained and transferred data

Replication can create charges for operations, transfer, destination storage, and supporting features. Versioning can retain states that remain billable after the current object changes. Change feed also has storage and processing implications. An estimate should therefore consider change rate and retention, not just the size of the current dataset. A frequently rewritten object can create a different cost pattern from a similarly sized static archive.

Lifecycle policies at the source and destination should be reviewed together. A destination can be useful for a different access pattern, but a policy that removes required data too early defeats the replication purpose. Retention rules should name the consumer's recovery or processing requirement. Otherwise, a cost optimization can silently become a data-availability incident.

Troubleshooting a missing report

A consumer reports that yesterday's report exists at the source but not the destination. The administrator first confirms the exact account, container, and blob name. The policy filter then reveals that only names beginning with `published/` are included, while the report was uploaded beneath `drafts/`. The absence is consistent with the policy. The repair belongs to the publishing workflow or the approved filter, not the network firewall.

In another case the correct object is eligible, but versioning was never enabled on the destination. The prerequisite failure prevents the intended relationship from operating. These examples show why object replication should be explained as a system of explicit dependencies. When purpose, eligibility, timing, and ownership are documented, missing data can be investigated through concrete checks rather than assumptions about invisible background copying.

9.4 Protecting Data Through Encryption

Encryption protects a particular boundary

Encryption transforms readable data into a form that requires the appropriate cryptographic process and key material to recover. It is an important protection, but its value depends on where it is applied and who can authorize decryption. Azure Storage encrypts persisted data through service-side encryption. Applications normally interact with the service without implementing that storage-at-rest process themselves.

At-rest encryption differs from protection in transit. A secure transport connection protects communication between a client and the service. At-rest encryption protects persisted information within the storage design. An account can have encrypted stored data while an application still needs appropriate transport settings. The administrator should review both boundaries rather than using the single word encrypted as a complete security assessment.

Encryption also differs from authorization. A user who is legitimately authorized to read an object receives readable data through the service. If that permission is too broad, at-rest encryption does not correct it. The service is performing the authorized operation. Protecting information therefore requires appropriate identity, roles, network policy, data classification, and encryption together.

Microsoft-managed keys simplify the default operation

With Microsoft-managed keys, the platform manages the keys used by the storage encryption service. This is the default operational model for new storage accounts. It allows an organization to benefit from encryption without creating its own key lifecycle for each account. The administrator still owns the account's access configuration and the business decision about whether the default key-management model meets its requirements.

The default should not be described as unencrypted or inherently unsuitable merely because the organization does not hold the key itself. The relevant distinction is control and responsibility. Some organizations have a requirement for customer-controlled key rotation, separation of duties, or a particular audit trail. Others prefer the platform-managed model because it reduces operational dependencies while meeting their policy.

An evidence-based review asks what requirement a different key model satisfies. If no requirement changes, adding another key-management system can increase complexity without improving the application's practical protection. If a requirement does exist, the implementation should include availability and recovery as well as key creation. A cryptographic setting that makes essential data unavailable through poor operations is not a successful design.

Customer-managed keys add custody and dependencies

Customer-managed keys place key control in an organization-managed key service such as Azure Key Vault or Managed HSM, using the supported storage integration. The storage service needs an identity and appropriate access to use the key. The key service's configuration therefore becomes a dependency of storage operations. Permissions, network access, deletion protection, and key lifecycle must all be considered.

The identity used by the storage service is different from an application identity that reads blobs. Granting a human administrator a data role on the storage account does not grant the storage service permission to unwrap a customer-managed key. Conversely, allowing the storage service to use a key does not give arbitrary users permission to read data. These relationships should be drawn separately in the design.

Key rotation changes the key material or version used under the supported integration. The process should preserve access to data protected under the required previous material and follow the service's rotation behavior. An administrator should not delete an old key simply because a new version exists. Recovery and retention requirements must be checked first, and a small data access test should follow a controlled rotation.

The review should identify whether the integration tracks a versionless key reference or a particular key version. That detail affects how a new version becomes active and what the rotation procedure must verify. A change record should include the key identifier and version relationship, never the secret key material. This gives operators enough evidence to diagnose the dependency without exposing the cryptographic value itself.

Key availability is part of data availability

A disabled, deleted, inaccessible, or improperly permissioned key can prevent the service from completing required encryption operations. That failure may appear to an application as a storage problem, but the cause can reside in the key-management dependency. Monitoring should therefore include key-service changes and the identity relationship, not only storage capacity or request rates.

The recovery plan should identify who can restore access and how. An emergency that removes every authorized key administrator can be as damaging as an accidental deletion. Separation of duties should prevent inappropriate access while preserving a controlled recovery route. The design must satisfy both goals; simply removing permissions until nobody can act is not a workable security strategy.

Key vault protection settings and retention requirements are not optional background details in a customer-managed-key design. They help prevent an irreversible loss of required key material. The administrator should use current Microsoft guidance for the exact key service and storage configuration. A book example can explain the relationship, but it cannot safely substitute a universal set of settings for every organization's recovery requirements.

Customer-provided keys and encryption scopes are distinct

A customer-provided key can be supplied with particular supported Blob Storage requests. This is different from configuring an account to use a customer-managed key held in Key Vault. The client becomes responsible for retaining and supplying the correct material according to the supported operation. Losing that material can make the data unusable. The two similar names describe different integration and operational models.

Encryption scopes allow supported blob data to use defined encryption boundaries within an account. They can help express separate encryption requirements for different data sets. They do not replace data authorization or make every account feature compatible automatically. The administrator should verify which operations, defaults, and scope-enforcement choices apply before presenting them as a general tenant-isolation solution.

Infrastructure encryption can add another encryption layer for supported configurations. Its purpose and deployment requirements should be evaluated against a specific assurance requirement. More layers do not remove the need for correct role assignments, secure application behavior, or recoverable key management. The explanation should remain precise about what additional layer is provided and what threats remain outside its scope.

Model	Key-management responsibility	Operational question
Microsoft-managed	Platform manages service keys	Does this model satisfy the organization's policy?
Customer-managed	Organization controls a supported external key resource	Can storage reliably use and recover the required key?
Customer-provided	Client supplies key material for supported requests	Can every required operation obtain the correct key securely?

Protect secrets in tools and evidence

Keys, tokens, and connection strings should not be pasted into manuscript examples, source control, or troubleshooting transcripts. A diagnostic record needs the configuration relationship and relevant error, not the secret value. Logging full request URLs can expose a shared access signature. Copying a command history into a ticket can expose credentials even when the application itself handles them carefully.

Managed identity and Microsoft Entra authorization can reduce the need to distribute long-lived secrets in supported scenarios. They do not eliminate the need for permission review. An identity with excessive access is still excessive access. The administrator should favor narrowly scoped permissions and document which principal performs each data and key operation.

Diagnose a key-related interruption methodically

Suppose blob reads begin failing after a key-service change while the storage endpoint and network path remain unchanged. The administrator examines the storage encryption configuration, the identity

used for key access, and recent key or permission events. If the relevant key was disabled, restoring its intended availability through the approved process may resolve the interruption. Generating a new storage account key would address a different credential and miss the cause.

After repair, verification should include a representative read and write, not merely a green configuration status. The incident record should state which dependency failed and how recurrence will be detected. This distinguishes the account's access credentials from its encryption key dependency and gives future administrators a usable explanation.

Encryption is strongest operationally when its boundaries are clear. At-rest protection, secure transport, data authorization, and key custody each answer different questions. A capable administrator can explain those differences, verify the chosen relationships, and recover from a key-management failure without improvising with sensitive material. That clarity makes encryption a maintained control rather than a checkbox whose consequences are understood only during an outage.

9.5 Moving Data with Storage Explorer and AzCopy

A transfer is a data operation with consequences

Moving data sounds simple until the source contains changing files, the destination already contains objects, or the identity lacks the required operation. A reliable transfer plan identifies source, destination, selection rules, overwrite behavior, verification, and cleanup. It also distinguishes copying from moving. A successful copy does not automatically authorize deleting the original, and deletion should follow a separately verified migration decision.

Azure Storage Explorer provides a graphical interface for browsing and managing supported storage resources. It is useful for inspecting a small dataset, reviewing object properties, or performing interactive operations. AzCopy provides a command-line transfer tool suited to repeatable copy and synchronization work. The choice depends on the task's size, repeatability, and need for automation rather than a belief that graphical tools are inherently safer.

Both tools operate within authorization and network constraints. Signing in establishes an identity; it does not grant every data permission. A management-plane role that can configure an account does not necessarily allow reading its blobs. A permitted data identity can still be blocked by account network policy. These independent controls explain why a resource can appear in a tool while its contents remain inaccessible.

Choose the credential model deliberately

Microsoft Entra authorization is useful for supported operations because it associates access with an identity and role assignments. The administrator should select the correct tenant and account context, then verify the relevant data role at an appropriate scope. Role propagation can take time. A newly assigned permission should be tested after allowing the supported propagation behavior, with a fresh authentication context when needed.

A shared access signature, or SAS, can delegate limited access through a signed token. It should be constrained to the needed resources, operations, and period. Anyone holding a usable token can exercise its granted capability within the other applicable controls. A SAS URL is therefore sensitive

even though it resembles an ordinary address. It should not be included in screenshots, source files, or support messages unnecessarily.

A failed transfer using a SAS also requires checking its validity period and allowed operations. A token that permits writing may not permit listing, and a token that has expired does not become valid because the user also owns the resource group. Clock assumptions and copied URL characters can complicate diagnosis. The operator should inspect nonsecret token properties carefully and issue a correctly scoped replacement through the authorized process when necessary.

Account keys provide broad capabilities and deserve careful handling. A tool that can list or use account keys may appear to work even when the intended Entra data permission is missing. That can conceal an authorization design error. Tests should state which credential path they use. An example using `--auth-mode login` helps make the intended identity-based data operation explicit for Azure CLI commands that support it.

Understand the paths before copying

Local paths, container URLs, object URLs, and wildcard patterns express different selections. A missing slash or an unexpected wildcard can change the destination layout or copy more data than intended. Before running a large operation, test with a small directory whose contents are easy to recognize. Inspect the resulting hierarchy rather than assuming it matches the source merely because the command completed.

Blob names can include slash characters that tools display as virtual folders. That display does not always imply ordinary filesystem directory semantics. A local application may also rely on metadata or permissions that do not map directly to the destination service. A migration plan should identify which properties must survive and verify that the selected tool and options preserve them for the particular source and destination combination.

Names and character handling matter across operating systems. A set of objects acceptable in storage may be awkward on a destination filesystem. Case sensitivity can also differ between systems and application expectations. Testing representative filenames early can reveal problems before a large transfer is underway. These checks are part of data correctness, not cosmetic cleanup after the migration.

Copy and synchronization are not interchangeable

A copy operation transfers selected content according to its options. Synchronization compares source and destination and attempts to make the destination reflect the selected source state. Depending on options, synchronization can include deletion. That makes source selection and deletion policy especially important. Running a destructive synchronization against the wrong empty source can remove valid destination content.

The administrator should inspect the proposed operation with supported dry-run or listing capabilities where available, then use a small test scope. Exclusions, timestamps, overwrite behavior, and delete-destination settings should be explicit in an automation review. A command copied from another environment may have assumptions that are inappropriate for the current dataset. The safest repeatable process records those assumptions alongside the command.

```
azcopy login --tenant-id "<tenant-id>"
azcopy copy "./sample.txt" \
  "https://<account>.blob.core.windows.net/<container>/sample.txt"
```

This illustrative Bash command uses an authenticated identity and a specific destination object. The identity needs appropriate blob data permissions, the account must allow the client's network path, and the source file must exist. The expected outcome is a completed transfer recorded by AzCopy. It is not an assertion that the command was executed during preparation of this book.

Measure success beyond the exit message

A transfer tool reports useful job status, but the verification plan should match the business risk. For a small text file, downloading it and comparing contents can be enough. For a larger dataset, compare expected object counts, lengths, selected hashes where supported, and representative application reads. No single comparison establishes every property. A matching file count does not prove matching contents.

Job logs and plan files can help diagnose failed transfers and resume supported work. They should be retained long enough to explain the outcome, with sensitive information protected. A partial failure should not be reported as a successful migration merely because most bytes moved. The summary should identify completed, failed, and skipped items and the reason each category exists.

If the source changes during copying, the destination may represent several points in time. Applications that require a consistent dataset need a quiescing, snapshot, version-selection, or application-level coordination strategy appropriate to the source. The administrator should not claim transactional consistency from a tool that copies independent objects while writers remain active. A final catch-up pass may help, but it must be part of a defined cutover process.

Performance tuning follows correctness

Transfer throughput depends on the client, network, storage service, object sizes, concurrency, and throttling. Increasing concurrency can improve utilization in some cases and worsen contention in others. The administrator should establish a correct small transfer first, then measure a representative workload. Tuning a command with an incorrect destination simply makes the mistake happen faster.

Costs can include storage operations, retrieval from a cooler tier, and network transfer. A test that reads every archived object can have consequences very different from listing names. The transfer plan should identify expensive operations before execution and use current pricing. An unexpectedly slow job can also reflect rehydration or service limits rather than a local network defect.

Symptom	Useful first evidence	Likely area to investigate
Resource visible, data forbidden	Credential mode and data role	Authorization
Connection cannot reach endpoint	DNS answer and allowed network path	Connectivity
Some objects skipped	Job summary and overwrite options	Selection semantics

Symptom	Useful first evidence	Likely area to investigate
Content differs after copy	Source changes and verification method	Consistency and integrity

A controlled migration sequence

An administrator first inventories a small report directory and creates a destination container with narrowly scoped access. A test file is uploaded, downloaded, and compared. The full directory is then copied with explicit selection and overwrite options. Failed items are reviewed rather than silently ignored. The application is pointed to the new location only after its representative reads succeed.

The original dataset remains available through the agreed rollback period. Deleting it is a separate step performed after verification and ownership approval, not an automatic consequence of the tool reporting success. Temporary credentials are revoked or allowed to expire according to the design, and temporary resources are removed. This sequence makes data movement auditable: the team can explain what was selected, what arrived, how it was checked, and when the old copy ceased to be required.

Guided lab

Objective and prerequisites

Create an isolated general-purpose v2 account, grant a specific identity blob data access, upload a small text file, and verify a downloaded copy. This lab uses Azure CLI in Bash and public service endpoints with authenticated access. It assumes no previous chapter resources. It does not configure object replication or customer-managed encryption keys.

The operator needs permission to create the resource group and storage account, plus permission to assign an Azure role at the account scope. Contributor alone does not grant role-assignment authority. If an administrator prepares the role assignment, the learner needs the resulting Storage Blob Data Contributor permission for the data operations. The principal identifier is a Microsoft Entra object ID, not an email address or an application client ID.

Storage capacity and operations generate charges. Downloads can create transfer charges depending on the path. Cloud Shell storage can be charged separately. Use a small file, select a supported region, and remove the dedicated group after verification. A budget is a notification mechanism, not a spending cap. No live Azure commands were executed during authoring.

1. Authenticate, select the intended subscription, and set explicit values. Replace the account placeholder with a globally unique name using only lowercase letters and numbers within the service's name-length limit.

```
location="<supported-region>"
group="rg-compass-ch09"
account="<globallyuniqueaccountname>"
principal="<learner-entra-object-id>"
az group create --name "$group" --location "$location"
az storage account create --name "$account" --resource-group "$group" \
  --location "$location" --kind StorageV2 --sku Standard_LRS \
  --https-only true --min-tls-version TLS1_2 \
  --allow-blob-public-access false
```

Anonymous blob access is disabled. The lab still uses the account's public network endpoint, subject to subscription policy. If policy requires a private endpoint or selected networks, use an approved client path and adapt the network configuration before data operations. Do not bypass organizational policy to complete a sample.

2. Inspect the account and assign data access at its scope.

```
scope=$(az storage account show --name "$account" \
  --resource-group "$group" --query id --output tsv)
az storage account show --name "$account" --resource-group "$group" \
  --query
'{kind:kind,sku:sku.name,httpsOnly:enableHttpsTrafficOnly,encryption:encryption.keySource}'
az role assignment create --assignee-object-id "$principal" \
  --assignee-principal-type User --role "Storage Blob Data Contributor" \
  --scope "$scope"
```

The expected properties include the selected account kind, LRS SKU, secure transport requirement, and platform key source. Allow role propagation before testing. The assignment command is for a user principal; use the correct principal type if adapting the lab to an application identity.

3. Create a container and a small local text file, then upload it using the signed-in identity.

```
az storage container create --account-name "$account" \
  --name samples --auth-mode login
printf '%s\n' 'Compass storage verification' > compass-ch09-source.txt
az storage blob upload --account-name "$account" --container-name samples \
  --name sample.txt --file compass-ch09-source.txt --auth-mode login
az storage blob download --account-name "$account" --container-name samples \
  --name sample.txt --file compass-ch09-downloaded.txt --auth-mode login
cmp compass-ch09-source.txt compass-ch09-downloaded.txt
```

A successful `cmp` produces no differences and exits successfully. That verifies this file's downloaded bytes; it does not validate every possible future transfer. Inspect the blob's properties in Storage Explorer as an optional graphical comparison using the same identity.

4. Introduce a controlled selection error by attempting to download `missing.txt`. Record the error and distinguish a nonexistent blob from an authorization failure. Restore the correct name and repeat the successful read. Do not change permissions to repair a misspelled object name.

5. Clean up after inspecting the dedicated group's contents.

```
az resource list --resource-group "$group" --output table
az group delete --name "$group" --yes
az group exists --name "$group"
rm -- compass-ch09-source.txt compass-ch09-downloaded.txt
```

Delete only the two exact local lab files. The resource-group existence check should return `false` when Azure deletion completes. If cleanup fails, inspect the reported lock, policy, or permission issue and resolve only controls belonging to the lab. Record any remaining charge-generating resources.

Chapter synthesis

An account choice connects data behavior with service capabilities and management boundaries. Redundancy addresses infrastructure failure, object replication distributes eligible data, encryption protects defined boundaries, and transfer tools move content under explicit permissions. Each capability needs its own verification. Their combined value comes from a documented design that explains both protection and limitations.

Review questions

09-Q1. A new operator can create a storage account but receives an authorization failure when uploading with Microsoft Entra authentication. Network access is permitted. What should be checked first?

- **A.** The applicable blob data role
- **B.** The account's display label
- **C.** The number of DNS TXT records
- **D.** Whether the operator owns a public domain

09-Q2. Which requirement most directly distinguishes ZRS from LRS?

- **A.** Automatic recovery from every accidental deletion
- **B.** Distribution across supported availability zones in the primary region
- **C.** An independently writable secondary region
- **D.** Elimination of application backups

09-Q3. Why must a disaster-recovery plan consider recent data loss when using geographic storage replication?

- **A.** Encryption is disabled in the secondary region
- **B.** All secondary copies are always empty
- **C.** Geographic replication is asynchronous
- **D.** A secondary copy uses the application's local disk

09-Q4. Which prerequisite set matches supported block-blob object replication?

- **A.** Change feed only on the destination
- **B.** No versioning on either account
- **C.** An Azure Files share on both accounts
- **D.** Change feed on the source and blob versioning on both accounts

09-Q5. A report is outside the prefix filter of a replication rule. It never appears at the destination. What best explains the result?

- **A.** The report is ineligible under the policy
- **B.** Every report needs a public IP
- **C.** The destination must disable encryption
- **D.** Replication requires the reader to be an account Owner

09-Q6. What additional operational dependency does customer-managed storage encryption introduce?

- **A.** A requirement that every blob be public
- **B.** The storage service's supported access to the external key resource
- **C.** A rule that all consumers use account keys
- **D.** A guarantee that data authorization is unnecessary

09-Q7. An AzCopy job reports some files skipped. What is the best next action before declaring a migration complete?

- **A.** Delete the source immediately
- **B.** Assume skipped files are corrupt
- **C.** Review the job summary, selection and overwrite options, then verify destination content
- **D.** Disable all storage network rules

09-Q8. Which statement correctly separates redundancy from version-based recovery?

- **A.** Redundancy prevents every permitted overwrite
- **B.** Versioning chooses the account's geographic region
- **C.** Redundancy always retains every previous business state
- **D.** Redundancy protects infrastructure copies; version-based recovery can preserve earlier object states

Further reading

- Storage account overview
- Storage redundancy
- Object replication
- Storage encryption
- AzCopy
- Storage Explorer
- Assign blob data roles
- Storage account CLI
- Blob CLI

Chapter 10: Managing the Life of Blob Data

Data changes after its first upload. A frequently read document can become an old record, a valid file can be overwritten, and a container can be deleted by mistake. Blob administration therefore includes organization, cost, retention, and recovery over time. This chapter connects those responsibilities through explicit rules and small, verifiable operations.

Learning objectives

- Organize blobs using containers, names, properties, metadata, and tags.
- Select access tiers from retrieval and retention requirements.
- Read lifecycle policies as operational rules with defined scope.
- Identify and recover earlier object versions.
- Distinguish blob deletion recovery from container deletion recovery.

10.1 Organizing Containers and Blobs

Objects have addresses and behavior

A blob is an object stored in Azure Blob Storage. Its address includes the account's service endpoint, a container, and the blob name. The container groups objects and provides a useful scope for administration. This model resembles folders enough to feel familiar, but ordinary object storage does not necessarily behave like a desktop filesystem. Applications use service operations to read, write, list, and manage objects.

In a flat namespace, a slash inside a blob name can create the appearance of folders in tools. For example, `reports/2026/summary.csv` can be displayed as a nested structure while remaining an object name. That naming pattern is useful for listing and lifecycle filters. It should not be mistaken for every directory behavior provided by a hierarchical filesystem. Namespace choice affects supported operations and features.

The first organizational decision is therefore whether the application needs object names or directory semantics. An administrator should not enable hierarchical namespace solely to make the portal display attractive folders. The workload's APIs, analytics requirements, access controls, and feature compatibility should drive the choice. This chapter's ordinary versioning and recovery examples use a compatible flat-namespace account unless stated otherwise.

Containers should reflect an administrative boundary

A container can group data with a shared purpose, owner, or access pattern. Examples include published reports, inbound submissions, and processed results. Those groupings make permission review and lifecycle targeting easier. A container containing unrelated datasets with conflicting retention requirements can still be managed, but it increases the risk that a broad rule affects the wrong objects.

Creating too many containers can produce its own problems. An application that creates a new container for every minor event may require complex discovery, permission, and cleanup logic. A balanced design uses containers for meaningful boundaries and object names or tags for finer classification. The rule should be easy to explain to the team that will maintain the data after the original author leaves.

Container names participate in URLs and have service-specific naming restrictions. Blob names have different rules and can be more expressive. A convention should be checked against both sets of requirements rather than copied from a resource-group naming policy. It should also avoid personal details or secrets, since names can appear in logs and diagnostics even when the content remains private.

Choose the blob type for the operation

Block blobs are widely used for documents, media, and other object data. Their upload model supports assembling an object from blocks, which helps tools transfer large content efficiently. Append blobs are designed around appending data, a pattern relevant to selected logging scenarios. Page blobs support page-oriented random updates and have different use cases. These types are not merely labels for the same behavior.

A workload's write pattern helps identify the appropriate type. An application that replaces complete report files differs from a writer that appends records. A feature supported for block blobs should not be assumed to support append or page blobs. Access-tier changes and object replication are examples where type matters. An exam question that names a blob type is supplying a constraint, not decorative detail.

The administrator should inspect actual object properties when diagnosing a feature mismatch. A container can contain objects whose behavior differs because of their type, tier, versions, or protection settings. Assuming all objects in the same account are equivalent can lead to repeated failed operations. A small inventory of these properties often explains why one object behaves differently from another.

Properties, metadata, and tags serve different roles

System properties describe aspects of the stored object, such as its size and content-related headers. The content type helps a client interpret the bytes. If a CSV file is served with an unexpected content type, the browser's behavior may differ from the application's intention. Correct data bytes and correct response properties are both part of a usable object.

User-defined metadata attaches application information to the object. It can record a processing stage or another descriptive value within the service's supported constraints. Blob index tags provide another mechanism for classifying and finding supported objects and can participate in selected lifecycle filtering. Metadata and index tags should not be treated as interchangeable because their query capabilities and operational behavior differ.

An organization should define who maintains classification values. A lifecycle rule that relies on a tag is only as reliable as the process that sets the tag. If one uploader omits it, the object may remain outside the intended rule. If another uploader assigns the wrong value, data may be tiered or deleted incorrectly. Classification is therefore a data-quality responsibility as well as a storage feature.

Mechanism	Typical purpose	Review question
Container	Group objects under a useful administrative scope	Do these datasets share ownership and access needs?
Name prefix	Organize and select related objects	Is the naming convention stable and unambiguous?
Metadata	Attach application-specific descriptions	Which process writes and updates the values?
Index tags	Support classification and supported filtering	Can missing or incorrect tags affect retention?

Private content needs explicit access decisions

The presence of an object URL does not mean its content is public. Anonymous access requires the relevant supported configuration and should be disabled when the workload does not require it. Identity-based access, SAS delegation, and account credentials are different authorization models. Their details are developed in the storage-access chapter, but organization should already reflect the intended audience.

A public dataset and confidential uploads should not be mixed casually under a broad access configuration. Even where individual operations remain controlled, confusing boundaries make review harder. A clear container purpose allows an administrator to answer who should read, who should write, and who should administer the objects. The scope of an access grant should match that answer.

An object's name can disclose information even when its bytes are protected. Names such as a person's full identity or a sensitive case description can appear in application logs, monitoring, and error messages. A neutral identifier with controlled metadata is often easier to manage. The goal is not to make every name meaningless, but to avoid putting sensitive content into a widely repeated address.

Updates require a concurrency decision

Two writers can attempt to update the same object. Without a concurrency strategy, the later successful write can replace the earlier result. Applications can use supported conditional operations, such as conditions based on an entity tag, to detect unexpected changes. An entity tag, commonly called an ETag, is a value representing a particular resource state for such comparisons. It should not be interpreted as a universal content hash.

Leases provide another supported coordination mechanism for selected operations. They can prevent competing writers from changing a protected object unless the required lease information is supplied. A lease is not a backup and does not prove the content is correct. It controls operations under its supported rules. When a maintenance task fails because of an active lease, deleting the entire container is not an appropriate first response.

The administrator's role is to recognize these controls and coordinate with application owners. A failed overwrite can be intentional protection rather than a service defect. Diagnostic records should include

the operation, response code, object state, and relevant conditions without exposing credentials. This allows the application team to distinguish an expected conflict from a genuine availability problem.

Build an inventory that supports later decisions

An object inventory should answer more than how many files exist. Useful attributes include container, prefix, owner, blob type, size, tier, modification time, classification, and protection requirements. The inventory can be produced through supported listing or inventory features appropriate to the scale. Its purpose is to guide lifecycle, recovery, and cost decisions with evidence rather than assumptions.

For a small report collection, a manual review may reveal that active and obsolete reports share one prefix. Before adding an automatic deletion rule, the team can introduce a clear published-versus-temporary distinction. That organizational improvement prevents the lifecycle policy from having to guess business meaning from age alone. Structure makes later automation safer.

Good blob organization therefore gives every object a usable address, an appropriate type, a clear owner, and a predictable classification. It also recognizes that writes, reads, and deletion are controlled operations with consequences. Once those foundations exist, access tiers and lifecycle policies can express the data's changing value without relying on ambiguous folder appearances or undocumented conventions.

10.2 Selecting Access Tiers

Cost depends on how data is used

An access tier expresses a tradeoff between the cost of keeping data and the cost or speed of accessing it. A dataset that is read constantly has different economics from a record retained for occasional review. Choosing a tier only by its capacity price can therefore produce a higher total bill or an unacceptable retrieval delay. The administrator should begin with the workload's observed and expected behavior.

Hot, cool, and cold are online tiers for supported block-blob scenarios. Online means the data remains available for ordinary access without first completing an archive rehydration process. The tiers differ in their intended access patterns and charging characteristics. Moving to a cooler online tier does not mean moving to an offline tape-like workflow, but it does change the economic assumptions that should be checked.

Archive is an offline tier. Reading archived content requires rehydration into an online state through a supported operation. This introduces delay and additional planning. An application that needs immediate access cannot treat archive as an inexpensive substitute for hot storage merely because both are described as durable. The retrieval requirement must be stated before selecting the tier.

Model the complete cost pattern

Stored capacity is only one part of the calculation. Operations, data retrieval, network transfer, tier changes, and early-deletion charges can all matter. A dataset repeatedly moved to a cooler tier and then read heavily may cost more than keeping it in a tier suited to active use. The administrator should compare plausible usage patterns using current pricing for the actual configuration.

Minimum retention economics are especially important. Certain tiers impose charges when data is deleted, overwritten, or moved too soon after entering the tier. These are billing rules, not always technical prohibitions against the operation. A successful deletion can therefore still create a charge. The team should distinguish what the service permits from what the chosen pricing model makes economical.

At the editorial baseline, standard general-purpose v2 examples use minimum storage periods of 30 days for cool, 90 days for cold, and 180 days for archive when evaluating early-deletion charges. These values must be rechecked against current pricing and the actual account configuration before a cost commitment. A seven-day temporary export is therefore a poor candidate for a cooler tier chosen solely for its lower capacity rate.

Illustrative calculations should make assumptions explicit. A monthly report archive may contain a large amount of data but receive few reads. A support investigation can suddenly retrieve a large subset. The estimate should include both normal behavior and plausible bursts. A cost plan based only on the quietest month is not a useful basis for a service with unpredictable investigations.

Defaults and explicit choices can differ

An account-level default access tier can influence objects that inherit the default, while an explicitly assigned object tier records a deliberate choice for that object. An administrator should inspect actual blob properties instead of assuming every blob follows the current account default. Changing a default and changing each object's explicit tier are not the same operation.

The distinction matters during migration and troubleshooting. An account may show a cool default while several actively used objects remain explicitly hot. That can be intentional. Conversely, an uploader may set a tier that defeats the organization's expected default behavior. The deployment and upload process should document whether it relies on inheritance or makes explicit per-object choices.

Account type, blob type, redundancy, and feature combinations constrain available tier operations. A premium account's performance model is not identical to the standard account's hot-to-archive lifecycle. The administrator should check the current support matrix before designing a transition. A policy that describes an unsupported transition cannot be repaired merely by waiting longer.

Archive needs a retrieval workflow

Before archiving data, the team should define who can request restoration, how urgency is chosen, where the restored content becomes available, and how completion is detected. The workflow should also specify whether the original archived object remains or whether its tier is changed. These alternatives affect cost, references, and later lifecycle behavior. A restoration request is an operational event, not an ordinary immediate read.

Rehydration timing depends on the supported operation and selected priority. The book should not promise a fixed completion time for every dataset. Instead, the administrator monitors the object's state and communicates an estimate based on current service guidance. A user-facing process should distinguish request accepted from data ready. Confusing those stages can cause applications to retry reads unnecessarily.

Applications should not repeatedly initiate equivalent restoration work while a prior request is pending. The workflow needs an identifiable request, status tracking, and a completion check. For a large set of archived objects, it should also handle partial completion and failures. Operational discipline matters because the cost and time of retrieval are part of the archive decision.

Data behavior	Tier direction to evaluate	Main concern
Frequent reads and updates	Hot	Avoid unnecessary access and early-change costs
Infrequent access with immediate retrieval	Cool or cold	Compare retrieval charges and expected retention
Rare access with flexible retrieval time	Archive where supported	Plan rehydration and recovery timing
Uncertain or changing access pattern	Measure first; evaluate supported automation	Avoid oscillating transitions and hidden costs

Retention and access are different requirements

A record can require long retention while still needing immediate retrieval. That does not automatically make archive appropriate. A legal or operational request might require a response faster than rehydration can provide. The administrator must combine the retention period with the retrieval service expectation. The longest-lived data is not always the coldest data.

Similarly, a frequently accessed object may have a short business lifetime. It can remain hot until deletion if that matches the workload. Forcing every object through hot, cool, cold, and archive creates a decorative lifecycle rather than an economic one. Each transition should have a reason supported by access evidence and the relevant billing model.

Protection features can retain additional states in different tiers. Previous versions and snapshots can consume capacity even when the current object is small or deleted. A cost review should include those retained states. Otherwise, the team may repeatedly change the current object's tier while the largest part of the bill comes from unmanaged history.

Verify a tier decision with real evidence

Metrics and application logs can show access frequency, operation types, and changing demand. The administrator should choose an observation period that reflects the workload's cycle. A report read only at quarter end can look unused during an ordinary week. Business context is necessary to interpret the telemetry. Automated recommendations should be reviewed against that context.

A small representative trial can validate assumptions without moving the entire dataset. The team records baseline costs and retrieval behavior, applies the proposed tier to a controlled subset, and compares results. A trial should avoid data whose recovery requirements cannot tolerate experimentation. The outcome can support a broader policy or reveal that the original access assumptions were wrong.

Troubleshoot inaccessible or unexpectedly costly data

If an application cannot read an object, first inspect its actual tier and state. An archived object awaiting rehydration differs from an online object denied by authorization. Retrying credentials will not make offline content immediately readable. Conversely, moving a blob to hot will not grant a missing data role. The observed error should guide which control is examined.

If costs rise after a cooler-tier transition, compare retrieval volume, operation counts, and early-change activity. The transition may have succeeded technically while failing economically. The correction might involve keeping an active subset hot, separating datasets, or revising lifecycle timing. It should not be reduced to a general claim that cool storage is expensive; the workload and pricing model interact.

Access tiers are most useful when the organization can explain the expected access pattern, retrieval deadline, retention period, and total cost. Those four facts turn a tier label into a maintainable decision. As the data's use changes, the decision can change too, supported by measurements and a controlled transition rather than an assumption that older always means cheaper to retrieve.

10.3 Automating Lifecycle Decisions

Turn a business rule into a testable policy

A lifecycle policy automates supported actions on objects that match defined conditions. It can move eligible data to another tier or remove data when its retention requirement ends. The automation is useful because a growing account can contain more objects than a person can review individually. Its risk is equally clear: a mistaken rule can apply the wrong decision consistently across a large dataset.

The policy should begin as a sentence in ordinary language. For example, published monthly reports that have not changed for an agreed period can move to a cooler tier, while temporary exports can be deleted after their approved lifetime. Those are two different data classes with different consequences. Combining them into one broad age-based rule would discard the business distinction that makes the actions safe.

The administrator then translates the sentence into filters, conditions, and actions. Filters identify the objects in scope. Conditions decide when an eligible object qualifies. Actions specify the supported operation. Reviewing those components separately helps reveal errors. A correct age threshold cannot compensate for a filter that includes confidential records intended for longer retention.

Filters deserve the most careful review

A prefix filter selects names according to the service's documented syntax, including the container relationship. A blob index tag filter selects objects using classification values. These mechanisms can be combined under their supported logic. The policy should be evaluated against actual example names and tags, including objects that must remain outside its scope. An empty or overly broad filter can be more dangerous than a visibly invalid one.

Naming conventions can make selection understandable. If temporary exports live beneath a dedicated container or stable prefix, the rule can express their lifecycle clearly. If temporary and permanent content are mixed under arbitrary names, the policy may depend on unreliable metadata conventions.

Improving organization before automation can reduce both policy complexity and the chance of deleting valuable content.

Filters are not a substitute for permissions. A writer allowed to alter classification values may be able to change whether an object qualifies for lifecycle processing. That can be legitimate, but the responsibility should be intentional. The team should decide which system classifies data and how classification changes are reviewed. A retention rule is only reliable when its inputs are reliable.

Time conditions need a precise meaning

Creation time, last modification time, and last access time describe different events. A report created long ago but updated yesterday has a different modification age from its creation age. A report never modified after publication may still be read daily. Selecting the wrong time property can therefore move active data or retain obsolete data for longer than intended.

Access-based conditions require the relevant access-time tracking capability and its documented behavior. The administrator should verify what operations update the tracked value and how that value is maintained. An application that checks metadata frequently may not behave like one that reads content, depending on the feature's semantics. The rule must be based on the actual signal, not an intuitive interpretation of the phrase last accessed.

The threshold should also reflect the workload's cycle. A policy based on a short quiet period can misclassify quarterly reports as abandoned. A retention obligation may begin at publication or contract completion rather than the blob's most recent technical write. If the service's built-in time condition does not match the business event, the design needs an appropriate classification or application process rather than a misleading approximation.

Current data and historical states need separate attention

The current blob, previous versions, and snapshots are different lifecycle targets. A rule that handles only current objects may leave old versions consuming storage indefinitely. A rule that deletes history too aggressively may remove the recovery window the organization expected. Both current-state cost and historical protection should appear in the policy review.

The retention plan should name the longest plausible interval between a mistake and its discovery. If a bad overwrite is commonly noticed during a monthly reconciliation, retaining only a few days of versions does not meet that recovery need. The policy's cost savings must be compared with the resulting loss of recovery capability. That tradeoff belongs to the data owner, not to an unexplained default.

Other controls can affect lifecycle actions. Immutability, legal holds, leases, and feature compatibility may prevent or change an operation's outcome under their supported rules. A failed or delayed action should be investigated rather than met with removal of every protection feature. The administrator needs to understand why the object exists and which control has precedence for the requested operation.

Read a policy as executable intent

```
{
  "rules": [
    {
      "enabled": true,
      "name": "cool-published-reports",
      "type": "Lifecycle",
      "definition": {
        "filters": {
          "blobTypes": ["blockBlob"],
          "prefixMatch": ["reports/published/"]
        },
        "actions": {
          "baseBlob": {
            "tierToCool": {"daysAfterModificationGreaterThan": 45}
          }
        }
      }
    }
  ]
}
```

This illustrative rule targets supported block blobs whose names fall beneath the specified container and prefix. It moves qualifying current blobs to cool after the stated modification-age condition. It does not delete data, manage previous versions, or prove that the reports are no longer used. The number is an example chosen for teaching; a real policy must use the organization's approved requirement and current pricing implications.

The administrator should explain each field before deploying the rule. A name helps identify the rule but does not determine its scope. The enabled flag controls whether it participates. The filter determines eligible objects. The action and condition define what happens and when. This reading method is more reliable than treating a JSON document as an opaque block copied from a sample.

Execution is periodic rather than an immediate timer

Lifecycle processing is not a per-object stopwatch that guarantees action at the exact threshold moment. Policy changes and processing take time, and the account's scale and workload can influence completion. A newly eligible object remaining unchanged immediately after policy creation does not by itself prove failure. The administrator should consult current execution guidance and inspect state over an appropriate interval.

Disabling or deleting a policy also has documented processing behavior. Removing the policy definition should not be assumed to cancel every action already underway instantly. A change plan should account for work in progress and verify the resulting object state. This is particularly important for deletion actions, where reacting after the first unexpected deletion may be too late to prevent all effects.

Validate policy scope before trusting automation

A review dataset should include an eligible object, an ineligible object, and a boundary case. The administrator records the expected action for each. Names, tags, types, and timestamps should be inspected explicitly. Although a small test cannot reproduce every large-account condition, it can reveal mistaken filters or unsupported actions before they affect a broad population.

1. Obtain the data owner's retention and retrieval requirements.
2. Inventory representative current objects and historical states.
3. Write the intended policy in plain language.
4. Translate it into narrow filters, conditions, and supported actions.
5. Review examples that must match and examples that must not match.
6. Deploy to an isolated test scope and monitor over the supported processing period.
7. Verify object properties, costs, and recovery consequences before broader use.

The result should be recorded as observed behavior, not merely as a valid policy document. A schema-valid policy can still encode the wrong retention rule. Conversely, a correctly scoped policy may need time before its first action occurs. Validation needs both configuration review and outcome evidence.

Diagnose a rule that appears inactive

Suppose a report remains hot after a rule was added. The administrator first checks whether its name matches the prefix, whether its blob type is eligible, and whether its actual modification age exceeds the condition. The report may have been rewritten by a nightly process, resetting the relevant time. In that case the policy is behaving consistently with its condition, while the business assumption was wrong.

Another object may qualify but use an unsupported account or tier combination. Waiting longer will not repair that incompatibility. The correction is to choose a supported design that still meets the requirement. Lifecycle administration therefore combines business interpretation, storage feature knowledge, and evidence of actual object state. Automation becomes dependable when all three agree.

10.4 Recovering Earlier Blob Versions

History changes the recovery question

Without retained history, an overwrite can leave only the latest object state available through the normal name. Versioning preserves earlier states under the feature's supported behavior. This changes the recovery question from whether the old bytes exist anywhere to which retained version represents the desired state. The administrator still needs a method to identify, inspect, and restore that state safely.

A current version is the state returned by an ordinary read of the blob name. Earlier versions have their own version identifiers and can be addressed explicitly. The identifier distinguishes a historical object state from the current name. Applications and tools should preserve it accurately when selecting a recovery source. Guessing from a visually similar timestamp can restore the wrong content.

Versioning is configured for compatible accounts and operations. It is not universally available across every namespace or storage feature combination. The administrator should verify support before relying on it in a recovery requirement. Enabling a setting after an overwrite cannot recreate history that was never retained. Protection must be active before the event it is intended to address.

Automatic versions differ from deliberate snapshots

Versioning creates history as supported writes occur. A snapshot is an explicitly created point-in-time state. Both can support recovery, but their creation models and management differ. An application taking a snapshot before a planned change has a known checkpoint. An account with versioning can retain a sequence of changes without requiring each writer to create that checkpoint manually.

The organization should avoid accumulating overlapping protection mechanisms without a reason. Versions, snapshots, soft delete, and backup can complement one another, but each creates retention and recovery work. A design review should state which failure each feature addresses and how old states are removed when no longer required. Otherwise, the account can become expensive while the restore procedure remains unclear.

Earlier versions are not ordinary editable working copies. A recovery process typically uses a selected prior state to create or restore the current state through a supported operation. The distinction preserves history rather than changing what an old version means. It also allows the team to inspect a recovery candidate before making it the live object.

Identify the desired state before restoring

An incident report should establish when the unwanted change occurred, which blob was affected, and what content is expected. Application logs, deployment records, and version listings can narrow the candidates. A timestamp is useful evidence but may not be enough. Several writes can occur close together, and the latest version before an alert is not always the last correct business state.

For example, a pricing file may be rewritten several times during an import. The version immediately before the final error could already contain partial or incorrect data. The administrator should compare candidate contents with a trusted business reference. Restoring the most recent historical version blindly can produce a technically successful operation with the wrong business result.

The inspection should be performed without overwriting the current object first. Downloading a candidate to an isolated local file or copying it to a separate review location can allow comparison. Access to historical content remains sensitive; old versions can contain information removed from the current file. The same authorization and handling discipline should apply to recovery evidence.

Control writers during recovery

If an application continues writing while an administrator restores an earlier state, the restored object can be overwritten again immediately. A recovery plan should coordinate with the writer or use supported concurrency controls. The team needs to decide whether to pause processing, change a deployment, or reject conflicting writes. Storage recovery cannot repair an application that keeps reproducing the same bad output.

The administrator should also capture the current state before replacement when it may be needed for investigation. An unwanted file can still contain evidence about the failure. Preserving it under controlled access avoids choosing between recovery and diagnosis. The retention period for incident evidence should follow the organization's requirements rather than remaining indefinitely by accident.

A supported restoration sequence

1. Confirm the account, container, blob name, and protection configuration.
2. Identify the unwanted change and coordinate with active writers.
3. List retained versions and select a candidate using recorded evidence.
4. Read the candidate separately and verify its content.
5. Use a supported copy or promotion operation to make the intended content current.
6. Confirm that the operation completed and read the normal blob name again.
7. Resume the application and verify the business operation that consumes the object.

The expected outcome is that a normal read returns the chosen content and the application can use it correctly. A completed copy request is not always the same as a completed asynchronous copy. Tools should inspect operation status where the selected method requires it. The final verification should include properties important to the application, such as content type or metadata, rather than checking only that a file exists.

Deleting the current blob can leave history

With versioning enabled, deleting a blob's current state can leave earlier versions available under the supported behavior while no current version remains. A normal listing or read may therefore suggest that the blob is gone even though historical content still exists. The administrator must use a view or listing option that includes versions. Recreating an empty object with the same name does not by itself restore the desired earlier contents.

Soft delete adds another layer when versions themselves are deleted. Its behavior should be evaluated together with versioning. The correct recovery operation depends on whether the missing item is the current blob, a retained version, a soft-deleted version, or an entire container. These are different states, and a single generic undelete instruction can be misleading.

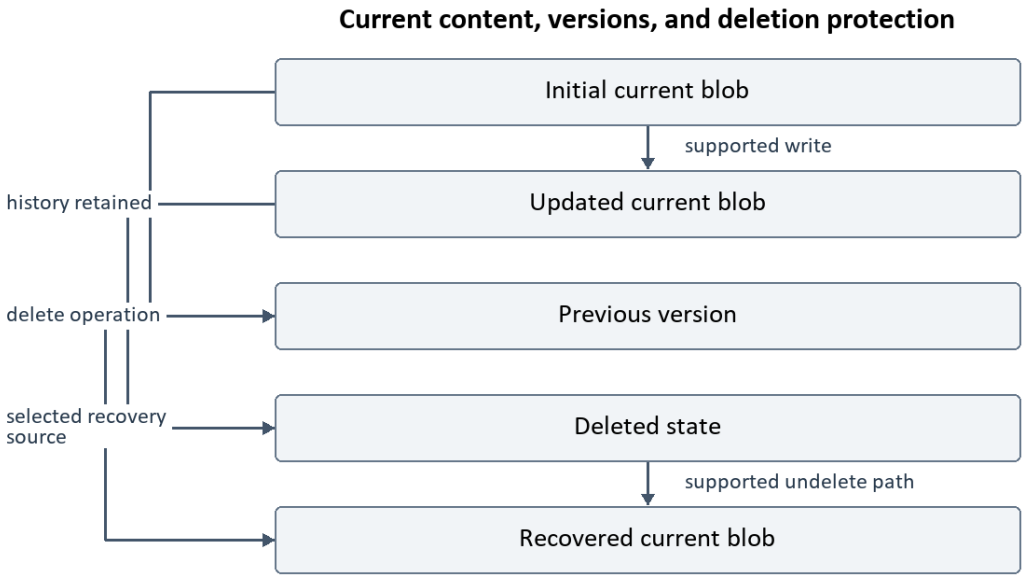


Figure 10.1 Current content and retained history

Versioning and soft delete retain different states. The valid restoration path depends on account compatibility and enabled features; container or account deletion has a broader protection boundary.

Retention is a budget and recovery decision

Every retained version can contribute to storage usage. A small file rewritten frequently can accumulate many versions, while a large static file may have only one. Capacity planning should consider write frequency and retention policy. The current blob's size is not a complete measure of the account's historical storage footprint.

Lifecycle rules can manage previous versions according to an approved retention requirement. The rule must leave enough history to cover realistic detection and response times. If the business discovers errors during a monthly review, a very short retention window is inconsistent with that process. A policy should be tested against how incidents are actually found, not how quickly the team hopes they will be found.

Reducing history also affects investigation. Older versions can help determine when a corruption began or compare several business states. That value must be balanced with cost and data-retention obligations. Some information should not be retained indefinitely. A mature design treats both unnecessary deletion and unnecessary retention as outcomes that need control.

Versioning has a limited protection boundary

Versioning does not automatically protect against deletion of the entire storage account or container. Those operations remove a broader resource boundary than an ordinary object overwrite. Container soft delete, account-level safeguards, and appropriate backup designs address different parts of that risk. An administrator should resist describing any one feature as complete protection for all data-loss scenarios.

A management lock can help prevent certain management-plane deletions, but it does not generally prevent all data-plane modifications to blobs. Permissions also remain important because a principal with broad rights may be able to delete historical states. Recovery design should therefore consider who can change protection settings and who can remove the protected data.

Verify recovery before the incident

A small lab should write known content, overwrite it with a different value, inspect the retained versions, and recover the earlier content. This tests the exact tool and permission path the team expects to use later. The exercise should record which version was selected and what the final normal read returned. A screenshot of the enabled versioning setting is not equivalent evidence.

A versioning exercise succeeds when the selected historical content becomes current and the application can use it. The record of that exercise should identify the source version, coordination with writers, restoration result, and final read. Those details make the procedure usable during a real overwrite incident.

10.5 Using Soft Delete for Blobs and Containers

Deletion protection has a scope and a clock

Soft delete retains deleted data for a configured period so that supported recovery operations remain possible. It changes deletion from an immediately final event into an event with a recovery window. That window is valuable only if the loss is detected and the restoration occurs before retention expires. The administrator should choose the period from operational requirements rather than accepting a default without discussion.

Blob soft delete and container soft delete protect different resource scopes. Blob protection concerns eligible objects and associated states under its documented behavior. Container protection concerns a deleted container and its contents. Enabling one should not be presented as enabling the other. An accidental container deletion requires container-level recovery capability, while a single deleted blob in an otherwise intact container needs the appropriate blob-level protection.

Neither feature is a general recovery mechanism for a deleted storage account. The account is the broader boundary that hosts both the data and its in-account protection features. Account deletion safeguards and an appropriate backup design therefore remain relevant. A complete protection discussion should name the exact deletion operation being considered before recommending a feature.

Retention should match discovery and response

The team needs enough time to notice the loss, investigate it, obtain authorized access, and restore the correct data. A short retention period may work for immediately monitored workflows but fail for records reviewed infrequently. The period should include realistic weekends, handoffs, and incident triage. Optimistic detection assumptions create a recovery promise the operations process cannot fulfill.

Retention also has a cost. Soft-deleted data remains stored during its protected period and can contribute to charges. A workload that deletes large amounts of temporary data each day can retain a substantial hidden population. The administrator should include that population in capacity and cost

reviews. The correct response is an approved retention decision, not secretly disabling protection to reduce a bill.

Changes to retention settings have documented effects on existing and future deleted objects. An administrator should not assume that extending a setting retroactively rescues every item approaching expiry. The actual deletion time and applicable retention state need inspection. Similarly, disabling a feature should not be assumed to remove all already retained data immediately. Operational procedures should follow the current service behavior.

Deleted objects may be hidden from ordinary views

A standard listing usually emphasizes active content. Soft-deleted objects can require an explicit option or view to appear. This makes an inventory of only active blobs insufficient for deciding whether recovery is possible. The administrator should know how the chosen portal view, CLI command, or SDK exposes deleted states before an incident occurs.

The distinction between absent and hidden is important during troubleshooting. A normal read failing with not found can be consistent with a recoverable soft-deleted object. The next step is to inspect protected states, not immediately create replacement content. Reusing names too quickly can complicate recovery, especially when an entire deleted container must be restored under its original name.

Permissions for recovery also matter. The person who notices a missing object may not have authority to restore it. The incident process should identify an appropriate operator and scope rather than distributing account keys broadly. A tested data role and a documented tool path make recovery faster while retaining accountability.

Blob recovery depends on the active protection combination

In a compatible account without versioning, soft delete can preserve eligible deleted or overwritten content according to the service's behavior. With versioning enabled, current and previous versions interact with deletion differently. The administrator must inspect which states exist and choose the correct restoration procedure. A universal instruction to call undelete and expect the old content to become current can conceal important distinctions.

An overwrite, deletion of the current blob, and explicit deletion of a prior version are separate operations. The protection configuration determines what historical or soft-deleted states remain after each. A recovery test should include the operations the application actually performs. If the application normally deletes and recreates objects, testing only a simple overwrite leaves a gap.

Snapshots can add another state to the inventory. A blob with snapshots may require those snapshots to be considered during deletion and recovery. The administrator should inspect the relationships rather than delete snapshots merely to make an error disappear. They may be the recovery evidence the business needs. Tool output should be read as a description of state, not an obstacle to bypass.

Container recovery is a whole-container operation

Container soft delete preserves a deleted container and its contents for the supported retention period. Restoration returns that container through the supported operation. It is not a mechanism for selecting

one deleted blob from an active container. The scope distinction helps prevent an administrator from searching the container recovery view for a single-object incident it was not designed to resolve.

The original name matters. If a new active container has already been created with the deleted container's name, that name can block restoration of the earlier container under the documented rules. An incident procedure should therefore pause automatic recreation where possible until recovery options are understood. Otherwise, a well-intentioned deployment script can complicate the restoration process.

If a replacement already exists, preserve and inventory its data before considering any change to it. Deleting the replacement blindly can create a second incident. The recovery operator should identify the deleted container instance, compare both datasets, and plan a controlled reconciliation. The goal is to recover the intended business content while preserving any valid writes that occurred after the original deletion.

The contents restored reflect the preserved container state. The team should verify the expected blobs, versions, and snapshots after restoration, then test application access. A restored container may still be inaccessible to a client because of a separate network or permission change. Recovery of data and recovery of the application path are related but distinct verification tasks.

Incident	Relevant protection to inspect	Separate concern
One blob deleted	Blob soft delete and version history	Correct current-state restoration
Entire container deleted	Container soft delete	Original name availability
Blob overwritten incorrectly	Versions or supported overwrite protection	Choosing the correct earlier content
Storage account deleted	Account safeguards and appropriate backup	In-account soft delete is insufficient

A careful recovery workflow

1. Stop or coordinate the process that caused the unwanted deletion.
2. Record the account, container, object name, deletion time, and observed error.
3. Inspect active, versioned, and soft-deleted states using the correct view.
4. Confirm that the required state remains inside its recovery window.
5. Select the operation appropriate to a blob, version, or whole container.
6. Restore and verify content, properties, and application access.
7. Document the cause and adjust prevention or detection controls as needed.

The first step prevents a recovered object from being deleted again immediately. A lifecycle policy, cleanup job, or application bug can repeat the original action. Recovery without addressing the active cause may produce a confusing cycle in which the administrator sees content reappear and vanish. The incident timeline should connect restoration attempts with the process that continues modifying the data.

Deletion tests should be isolated and reversible

A learning exercise should use a dedicated container with unmistakable test content and a confirmed retention setting. The operator lists the object, deletes it, inspects the protected state, and restores it. A separate container-level exercise deletes and restores a different dedicated container. Combining the two tests too early can obscure which feature produced the recovery.

The exercise should also include a negative case in the explanation: content deleted before protection was enabled is not recreated by turning the setting on afterward. Likewise, content beyond its retention window cannot be promised recoverable through that feature. These boundaries are essential to honest incident communication. A protection setting is not evidence that every historical loss can be reversed.

Soft delete is most effective when paired with clear ownership, suitable retention, limited permissions, and tested restoration. Its value is time: time to discover an error and act before deletion becomes permanent. The administrator must use that time well, with the right scope, an identified recovery state, and evidence that the restored information once again supports the intended work.

Guided lab

Objective and preparation

Demonstrate blob soft delete and container soft delete separately in a new, isolated account. The blob portion deliberately leaves versioning disabled so that its undelete behavior is easy to observe. This is a teaching configuration, not a recommendation to omit versioning from production. No archive transition or automatic deletion policy is applied.

Use Azure CLI in Bash, an authenticated subscription, and a supported public-cloud region. The operator needs account-management permissions and role-assignment authority; the learner needs Storage Blob Data Contributor for the data operations. Contributor alone does not authorize role assignments. Use a dedicated group and a globally unique storage name. Storage capacity, transactions, retained deleted data, and possible transfer create charges. Cloud Shell storage can create separate charges. Budgets do not prevent spending.

1. Create the account with anonymous access disabled.

```
location="<supported-region>"
group="rg-compass-ch10"
account="<globallyuniquelowercasename>"
principal="<learner-entra-object-id>"
az group create --name "$group" --location "$location"
az storage account create --name "$account" --resource-group "$group" \
  --location "$location" --kind StorageV2 --sku Standard_LRS \
  --https-only true --min-tls-version TLS1_2 --allow-blob-public-access false
scope=$(az storage account show --name "$account" --resource-group "$group" \
  --query id --output tsv)
az role assignment create --assignee-object-id "$principal" \
  --assignee-principal-type User --role "Storage Blob Data Contributor" --scope "$scope"
```

Allow role propagation and use a client network permitted by the account and organization. A policy requiring private access must be satisfied through an approved path before continuing.

2. Enable both deletion protections and inspect the actual settings.

```
az storage account blob-service-properties update \
  --account-name "$account" --resource-group "$group" \
  --enable-versioning false --enable-delete-retention true --delete-retention-days 7 \
  --enable-container-delete-retention true --container-delete-retention-days 7
az storage account blob-service-properties show \
  --account-name "$account" --resource-group "$group"
```

The expected configuration shows blob and container retention enabled for the lab period and versioning disabled. If organizational policy enforces versioning, preserve that policy and use a separate approved exercise designed for the combined behavior. Do not silently assume the simpler configuration was applied.

3. Create a container and upload known content.

```
az storage container create --account-name "$account" --name recoverylab --auth-mode login
printf '%s\n' 'Recover this exact content' > compass-ch10.txt
az storage blob upload --account-name "$account" --container-name recoverylab \
  --name sample.txt --file compass-ch10.txt --auth-mode login
az storage blob delete --account-name "$account" --container-name recoverylab \
  --name sample.txt --auth-mode login
az storage blob list --account-name "$account" --container-name recoverylab \
  --include d --auth-mode login --output json
```

The listing including deleted states should expose the deleted object. An ordinary active listing should not be interpreted as proof that no recoverable state exists.

4. Restore the blob and compare the downloaded content.

```
az storage blob undelete --account-name "$account" --container-name recoverylab \
  --name sample.txt --auth-mode login
az storage blob download --account-name "$account" --container-name recoverylab \
  --name sample.txt --file compass-ch10-restored.txt --auth-mode login
cmp compass-ch10.txt compass-ch10-restored.txt
```

The expected comparison succeeds without differences. This proves the small file's restored bytes under the lab configuration, not every protection combination.

5. In the portal, open this account's container list and delete only `recoverylab`. Display deleted containers, select the deleted `recoverylab` instance, and use the restore action. Do not create a replacement container with the same name. Verify that `sample.txt` returns and can be downloaded. This second exercise tests container-level recovery separately from the first.
6. As a controlled troubleshooting variation, hide deleted items immediately after a deletion and compare that view with the view that includes them. Explain why the active-only view is insufficient evidence of permanent loss. Restore before continuing.
7. Remove the dedicated group and exact local files after recording results.

```
az resource list --resource-group "$group" --output table
az group delete --name "$group" --yes
az group exists --name "$group"
rm -- compass-ch10.txt compass-ch10-restored.txt
```

The existence check should eventually return `false`. Resolve only lab-specific locks or policy issues if cleanup fails, and record anything still generating charges. The account deletion ends this isolated exercise; in-account soft delete is not a recovery promise for that account deletion. All live execution remains not executed during manuscript preparation.

Chapter synthesis

Blob administration follows the data through several states: organized current content, economical retained content, historical versions, and recoverable deleted items. Each state has different operations and costs. Naming and classification make automation possible, while carefully chosen protection windows make recovery practical. A reliable administrator verifies both what remains accessible and what can be restored.

Review questions

10-Q1. A tool displays reports/2026/file.csv as folders in a flat-namespace blob account. What does that display prove?

- **A.** The blob name contains a useful prefix structure
- **B.** The account automatically has hierarchical namespace enabled
- **C.** The object is an Azure Files share
- **D.** Every filesystem directory operation is supported

10-Q2. A dataset needs infrequent but immediate reads. Which choice is least consistent with that requirement?

- **A.** Cool online storage where supported
- **B.** Archive without a planned rehydration allowance
- **C.** Cold online storage where supported
- **D.** Hot storage

10-Q3. A lifecycle rule uses time since modification, but a nightly process rewrites the blob. Why might the object never qualify?

- **A.** The container name grants immunity
- **B.** Lifecycle rules operate only on public data
- **C.** Each rewrite changes the condition's relevant time
- **D.** Encryption disables all lifecycle processing

10-Q4. Which review best protects against a lifecycle rule affecting the wrong dataset?

- **A.** Review only whether the JSON parses
- **B.** Use the shortest available retention
- **C.** Delete all prior policies without inspection
- **D.** Test filters against matching, nonmatching, and boundary examples

10-Q5. An incorrect overwrite is discovered while the application continues writing. What belongs in the recovery plan?

- **A.** Coordinate or pause the writer and validate a historical candidate before promotion
- **B.** Restore repeatedly without changing the writer
- **C.** Delete all versions before investigating
- **D.** Assume the newest prior version is always business-correct

10-Q6. Which feature directly addresses accidental deletion of an entire container within its configured recovery window?

- **A.** Only a lower access tier
- **B.** Container soft delete
- **C.** A public DNS alias
- **D.** Only a shorter blob name

10-Q7. A deleted container's original name has been reused by a new active container. What is the appropriate response?

- **A.** Delete the replacement immediately without inspection
- **B.** Assume the old data is now merged automatically
- **C.** Preserve and inspect the replacement, then plan recovery and reconciliation around the name constraint
- **D.** Disable all account encryption

10-Q8. Why can a storage bill remain substantial after the current versions of many blobs are removed?

- **A.** Deleted current blobs always become free instantly
- **B.** All storage charges are fixed regardless of content
- **C.** DNS queries necessarily dominate every account
- **D.** Versions, snapshots, or soft-deleted states may still be retained

Further reading

- Blob Storage introduction
- Access tiers
- Lifecycle management
- Lifecycle policy structure
- Blob versioning
- Blob soft delete
- Container soft delete
- Manage soft-deleted blobs
- Enable and restore soft-deleted containers

Chapter 11: Sharing Files and Governing Storage Access

Shared files combine familiar application expectations with cloud identity and network controls. A successful deployment must support the right protocol, admit the right clients, authorize the right operations, and preserve useful recovery states. This chapter focuses on classic Azure file shares managed through the Microsoft.Storage resource provider, especially SMB. Other Azure Files resource models and protocols have separate support boundaries that must be checked before applying these procedures.

Learning objectives

- Choose and operate an Azure file share for a defined workload.
- Separate SMB authentication, share permissions, and file permissions.
- Recover file content from snapshots and distinguish share soft delete.
- Explain account keys, SAS delegation, and stored access policies.
- Diagnose storage access through independent network and authorization checks.

11.1 Creating and Operating File Shares

A file interface preserves application expectations

Azure Files provides managed file shares through supported file protocols. It can help applications that expect shared files operate without maintaining a conventional file-server operating system. The service manages the underlying storage infrastructure, but the administrator still chooses the share configuration, access model, network path, protection, and monitoring. Managed infrastructure does not mean that every application requirement is automatically satisfied.

Server Message Block, or SMB, is a common choice for Windows-oriented shared-file workloads and is also supported by appropriate clients on other operating systems. Network File System, or NFS, serves different application and client requirements. The protocols have different identity, security, and feature considerations. A deployment must select a supported protocol and configuration rather than assuming the same share accepts every client method interchangeably.

The current Azure Files product includes classic shares in storage accounts and a separate resource model for selected scenarios. The account, snapshot, and soft-delete procedures in this chapter refer to classic shares. This explicit boundary matters because a similarly named feature in another resource model may have different configuration and recovery behavior. The administrator should identify the resource provider before following a runbook.

Start with the workload's file behavior

A file share should be sized and selected from the application's operations. Relevant characteristics include total capacity, file size distribution, concurrent clients, metadata operations, throughput, latency, and input/output operations per second. A workload opening many small files can behave differently from one streaming a few large files. Capacity alone does not predict the required performance.

Applications may also depend on file locking, permissions, path conventions, and supported protocol features. A migration test should exercise representative behavior, not merely copy a file and declare compatibility. An application that works on a local disk can still have assumptions that fail on a shared network filesystem. The administrator should involve the application owner in defining a useful acceptance test.

Client location influences performance and connectivity. A share used by compute in the same region has a different path from a share mounted across a distant network. Latency can make many small sequential operations slow even when available bandwidth appears high. The design should account for round trips and application behavior before increasing provisioned capacity in the hope that every delay is a storage-size problem.

Account and billing choices shape operation

Classic file shares use a compatible storage account and supported billing model. Provisioned and usage-oriented arrangements express capacity and performance differently. The administrator should review current Azure Files guidance for the selected offering rather than carrying assumptions from an older premium-versus-standard comparison. Product names and billing models evolve, while the underlying questions remain capacity, performance, resilience, and cost.

A provisioned amount can affect charges even when the share contains little data. A usage-based arrangement can include transaction and capacity charges that change with application activity. Snapshots and soft-deleted shares can also contribute to usage or provisioned-account considerations. A cost estimate should include the actual service model and retained states, not just the bytes visible in the active root directory.

The share quota or provisioned capacity is also not always identical to a performance guarantee. The relationship depends on the offering. An administrator should inspect the selected share's properties and current performance model. A generic instruction to enlarge the share whenever latency increases can be expensive and ineffective if the bottleneck lies in client networking or metadata-heavy application behavior.

Separate creation from mounting

Creating a share is a management operation. Mounting it establishes a client data path using a supported protocol and authentication method. A successful management deployment does not prove that the client can resolve the endpoint, reach the required port, authenticate, or access a directory. Each part needs verification. The distinction is particularly useful when the portal shows a healthy share but a workstation cannot connect.

For SMB, the endpoint normally uses the account's file-service hostname and the share name. The client should use the supported hostname so that DNS, authentication, and service behavior align. Replacing it with an arbitrary alias or raw address can introduce authentication or certificate-related problems. Private-endpoint designs should preserve the intended name while resolving it through the correct private path.

Network environments can block SMB traffic before it reaches Azure. An on-premises firewall, internet provider, or local client policy may prevent the required connection. A DNS lookup alone does not

establish protocol reachability. A connectivity test to the intended destination port is a useful early check, followed by an authenticated mount. The test should be performed from the actual client network.

A deployment sequence with visible evidence

1. Confirm the application protocol, file behavior, performance, and recovery requirements.
2. Select a supported account, share offering, region, and redundancy configuration.
3. Create the classic share with an explicit capacity or quota decision.
4. Configure the intended identity source and permissions, or a clearly bounded lab credential method.
5. Establish DNS and network access from the client.
6. Mount the share using the supported endpoint and protocol.
7. Create, read, modify, and remove a dedicated test file according to the user's intended permissions.
8. Record monitoring, backup, and retirement responsibilities.

The seventh step should include a negative test where practical. A read-only user should be able to read the permitted file and fail to modify it. A user outside the approved group should not gain access merely because the client reaches the endpoint. Testing only an administrator with broad credentials can hide the very permission boundaries the design was meant to establish.

Understand what a graphical browser proves

The portal or Storage Explorer can access file data using a credential path that differs from an SMB-mounted application. A successful graphical upload therefore does not necessarily validate identity-based SMB access. The administrator should record which method was used. This prevents a common mistake in which a key-authorized browsing session is presented as proof that ordinary users have correct share and directory permissions.

REST operations and mounted filesystem operations can also differ in their supported authorization methods and semantics. A script designed for one should not be assumed to work unchanged through the other. The service documentation for the exact operation is the source of truth. A tool's ability to display a file does not imply support for every file-system feature or identity path.

Successful check	Evidence supplied	Additional check still needed
Share creation	Management configuration accepted	Client network and data access
DNS lookup	Endpoint name resolves	Protocol connection
TCP connection	Network path reaches a listener	Authentication and permissions
Administrator upload	That credential can perform the operation	Intended user's effective rights

Operate the share as an application dependency

Monitoring should reflect both capacity and experience. Usage trends help predict growth, while latency, transaction patterns, and errors help identify workload pressure. Client-side observations remain important because a service metric cannot describe every local filesystem operation or network delay. A useful incident record correlates the application symptom with share metrics and the client's location.

Open file handles can affect operations such as deletion, replacement, or application maintenance. An administrator should coordinate with the workload before forcing closure of handles. A file that cannot be removed may be in legitimate use rather than corrupted. The operational procedure should identify the owning process and the business consequence of interruption, particularly for shared application data.

Azure File Sync can support selected hybrid scenarios by caching and synchronizing file content with Windows servers. It adds another set of components and operational responsibilities. This chapter does not require a sync deployment for a basic share. The important boundary is that a cloud share, a synchronized server endpoint, and a local cached file are related resources with distinct state and troubleshooting evidence.

Diagnose a slow or failed mount

If a client cannot mount the share, the administrator checks name resolution, network reachability, authentication prerequisites, and authorization in that order. If mounting succeeds but opening a particular directory fails, file-level permissions become a stronger suspect than DNS. If access works but feels slow, the investigation compares latency, workload pattern, client placement, and selected performance capacity.

An effective file-share deployment ends with a documented application test and a clear account of the credential and network path used. That evidence makes later incidents easier to classify. The team can distinguish a share configuration problem from a client-path problem and can change capacity or permissions for a specific reason instead of treating all file failures as the same storage issue.

11.2 Applying Identity-Based File Access

Authentication and permissions form separate stages

Identity-based SMB access associates file operations with an authenticated identity instead of sharing a broad storage account key among users. The service uses a supported identity source and Kerberos-based authentication. After authentication, authorization determines which share and file operations are allowed. A successful sign-in or ticket exchange does not automatically grant access to every directory.

Kerberos uses tickets to establish an authenticated relationship without sending the user's password to the file service for each request. The client must meet the requirements of the selected identity source and configuration. Time, naming, domain or device state, and connectivity can matter. The administrator should understand these prerequisites before interpreting an access-denied message as a simple missing Azure role.

The account's identity-source configuration is an architectural choice. Supported approaches include Active Directory Domain Services, Microsoft Entra Domain Services, and Microsoft Entra Kerberos under their documented scenarios. They do not have identical client, identity, and network requirements. The current support guidance should be checked for the actual client operating system and identity type, especially where a scenario is in preview.

Match the identity source to the environment

An organization with existing AD DS identities and appropriate domain connectivity may use that integration for its file shares. The design includes the storage account's identity relationship and the required user or group representation for share-level assignments. The administrator should coordinate with the directory team rather than creating an unrelated identity system solely because the storage resource resides in Azure.

Microsoft Entra Domain Services provides a managed domain environment for supported scenarios. Clients and users must satisfy that environment's requirements. It is not the same thing as the ordinary Microsoft Entra tenant used for every Azure portal sign-in. Confusing those products can lead to a plan that assumes domain capabilities exist when they have not been deployed.

Microsoft Entra Kerberos supports cloud-oriented and hybrid scenarios with its own identity and client requirements. Those requirements have evolved, so a book should not apply an old universal rule about every supported identity or operating system. The administrator should verify the current scenario, device state, authentication policy requirements, and preview status before deployment. The principle remains choosing a supported path for the actual users.

Share-level access is the outer authorization boundary

Azure role assignments can grant supported SMB share-level permissions. The role should match the user's required operations and be scoped narrowly, commonly to the relevant share where appropriate. A broad management role is not a substitute for a file data role. Likewise, a role that permits data access should not be expanded into subscription administration merely to make a mount succeed.

Share-level permission establishes an outer allowance. Directory and file permissions then control access inside the share. Both layers matter. If the share-level role permits writing but the target directory's access control list allows only reading, the user cannot assume the broader role overrides the directory restriction. Effective access is determined through the applicable layers, not by choosing whichever permission looks most generous.

Groups can make these assignments easier to govern. A role associated with a business group is usually clearer than many undocumented individual grants. Group membership still needs ownership and review. A user moved to another department can retain access if the old group membership is never removed. Storage access is therefore part of the identity lifecycle rather than a one-time deployment checkbox.

File and directory permissions express local structure

Access control lists, or ACLs, describe permissions on files and directories for the supported SMB identity model. They can distinguish read, modify, and administrative operations at a finer level than the

share. Inheritance can simplify management across a directory tree, but it can also carry unexpected rights into a newly created folder. Administrators should inspect effective permissions on representative paths rather than only the share root.

A useful structure might permit a department to modify its working directory while reading a published reference directory. The share-level role permits the necessary class of access, and the directory ACLs express the finer distinction. A test should use actual representative users and files. Testing only the person who configured the ACL can miss privilege differences that ordinary users experience.

Changing ACLs requires an appropriate administrative path. Granting every user elevated permission to simplify one setup operation defeats the intended separation. A dedicated setup or maintenance identity can perform the necessary changes, after which ordinary users retain their narrower access. The operations record should identify who can alter permissions and how emergency corrections are handled.

Distinguish identity problems from authorization problems

An authentication failure means the service could not establish the expected identity under the configured method. Possible causes include unsupported client state, incorrect identity-source setup, naming issues, or ticket problems. An authorization failure means an established identity lacks permission for the requested operation or path. The visible error may not always make the distinction obvious, so diagnostic evidence is important.

The administrator should confirm the client identity, device or domain state, endpoint name, and selected account identity source. Then review the relevant share-level role and directory ACL. Role propagation and cached credentials can delay the apparent effect of changes. Repeatedly adding broader roles without refreshing the correct authentication context can make the final permission set unnecessarily permissive.

Existing SMB connections can also use credentials different from those the administrator expects. A client may retain a connection established with an account key or another identity. A test intended to validate a user's Kerberos permissions must not silently reuse that broader connection. The procedure should establish a clean, understood connection context before drawing conclusions.

A worked effective-access example

A finance group receives a share-level role suitable for modifying files. Inside the share, the `published` directory grants that group read access, while `working` grants modification rights. A finance user can edit a worksheet in `working` but cannot replace a file in `published`. That result is consistent with the design. Expanding the Azure role is not the right fix if the directory was intentionally read-only.

Now suppose the same user cannot open either directory. The administrator checks whether the user is represented in the correct identity source, whether group membership is effective, whether the client authenticated through the intended method, and whether the share assignment applies to the correct scope. If the user can mount but only one nested path fails, the investigation narrows toward that path's ACL and inheritance.

Layer	Question	Example evidence

Layer	Question	Example evidence
Client and identity source	Can this client authenticate as the intended identity?	Supported client state and Kerberos context
Share role	Is this identity allowed the required share operation?	Effective Azure role assignment
Directory or file ACL	Is this operation allowed on this path?	Effective permissions and inheritance
Network	Can the protocol reach the service?	DNS and connection test

Keep protocol boundaries explicit

The identity-based SMB model described here should not be copied wholesale to NFS. NFS authorization and client behavior have their own design. REST access also has supported identity and privileged-operation options that differ from a mounted SMB user session. An administrator should state the protocol in every access requirement and test plan. Without that detail, the phrase file access can conceal several different mechanisms.

The use of identity-based access also does not eliminate all powerful credentials from the environment automatically. Account keys and roles that can retrieve them may still exist unless governed appropriately. A complete review considers both the intended user path and alternative administrative paths. Otherwise, carefully designed ACLs can coexist with widely shared broad credentials that bypass the intended user model.

Identity-based file access succeeds when the correct user authenticates through a supported method and receives exactly the required operations at both share and path levels. The proof includes successful permitted actions and failed prohibited actions. That evidence is more useful than a statement that identity integration is enabled, because it describes the actual boundary experienced by the application's users.

11.3 Protecting Shares with Snapshots and Soft Delete

A snapshot records a share at a moment

A share snapshot is a read-only point-in-time state of a supported Azure file share. It can help recover a file that was changed or removed after the snapshot. The snapshot belongs to the share's protection design and has an identifiable time. It should not be confused with an independently editable duplicate share or with a promise of application-consistent recovery for every workload.

The snapshot captures storage state. An application may have pending writes, related files updated at different moments, or data held in memory. If a consistent application checkpoint is required, the team must coordinate the application appropriately. Taking a snapshot while a multi-file update is underway can preserve a state that needs application recovery logic. The administrator should ask what consistency the application expects before scheduling snapshots.

Snapshots are useful before a planned change because they provide a known recovery reference. The change record can identify the snapshot time, application state, and verification performed before the update. If the update damages a file, the operator has a concrete candidate rather than searching through arbitrary old copies. A snapshot still needs a tested restoration process to be operationally valuable.

Restore the needed content carefully

An individual file can be inspected or copied from a suitable snapshot through supported tools. The administrator should first recover it to a separate review location when practical. This allows content comparison without immediately replacing the current file. The recovery candidate may be older than desired, and the current file may contain valid changes made after the snapshot.

A restoration decision should identify which changes must be preserved. Replacing an entire directory to recover one damaged file can remove unrelated recent work. The narrowest suitable restoration reduces that risk. If a whole-share recovery is required, the team should plan it as a coordinated operation using the supported method for the actual offering rather than assuming every snapshot supports a universal instant rollback button.

File properties and permissions can matter as much as content. A restored file that has the correct bytes but inappropriate access can still fail the business requirement. The verification should therefore include the intended user's read or modify operation, important attributes, and the application's own acceptance test. A successful copy notification is only one piece of that evidence.

Snapshots need retention and inventory

Snapshots consume storage according to the service's supported billing behavior. Their cost depends on retained changes and the selected share model, not merely the number of snapshot labels in a list. A frequently modified dataset can retain significant historical data. The administrator should review snapshot growth alongside active share usage and avoid assuming that read-only history is free.

Retention should reflect both recovery needs and data-management obligations. A snapshot taken before every deployment can accumulate indefinitely unless an owner defines its lifetime. A short retention period may be insufficient if errors are discovered only during later business reconciliation. The policy should specify when snapshots are taken, how long they remain, who can remove them, and how successful recovery is demonstrated.

Manual snapshots can support learning and selected operational tasks, while Azure Backup can manage protection schedules and recovery under its supported model. The choice should depend on the required policy, reporting, and recovery capabilities. A handful of manually created snapshots without monitoring is not equivalent to a managed backup process with defined retention and tested restores.

Share deletion is a broader event

A snapshot does not make the parent share immune to deletion. Share snapshots are associated with the base share, and deletion behavior must be considered at that boundary. If the incident removes the entire share, the administrator needs the appropriate share-level recovery capability. This differs from recovering a file while its parent share still exists.

For classic Azure file shares, soft delete retains a deleted share for a configured period. A supported undelete operation can restore the share and its contents, including its retained snapshots, to the preserved state. The feature applies to the share scope. It does not serve as an individual-file recycle bin for a file removed from an otherwise active share.

That scope distinction is easy to test. Deleting `budget.xlsx` inside a share does not delete the share itself, so share soft delete is not the direct recovery mechanism. A suitable snapshot or backup can provide the earlier file. Deleting the entire `finance` share invokes the share-level protection question. The runbook should begin by identifying which operation occurred.

The recovery window is an operational commitment

Soft-delete retention begins from the relevant deletion event under the service's rules. The operator needs enough time to detect the problem and restore the share before permanent removal. The retention decision should include weekends, alert handling, and the time needed to identify the correct deleted share. A default period is a starting configuration, not proof that the organization's response process fits inside it.

Changing retention settings should be reviewed against existing deleted shares and current service behavior. It should not be assumed to retroactively extend every recovery window. Billing and quota effects can also differ by share model. A retained provisioned share may continue to affect capacity accounting in ways that matter when creating replacements. Inspect the actual offering's documentation rather than applying a blob-storage assumption.

Soft delete is an in-account protection. It does not guarantee recovery after deletion of the entire storage account. Management locks can help prevent selected management-plane deletions, while backup and broader recovery controls address other risks. The team should describe a layered design rather than presenting one feature as a complete answer to every deletion scenario.

A practical protection comparison

Event	Protection to evaluate	Verification target
One file overwritten	Snapshot or suitable backup	Earlier bytes and required permissions
One file deleted	Snapshot or suitable backup	Correct file restored into the active share
Entire classic share deleted	Share soft delete within retention	Share, contents, and snapshots restored
Account deleted	Account safeguards and appropriate independent recovery	Rebuilt service and validated data

The comparison separates recovery scope from preventive controls. A lock can reduce accidental management deletion but does not create an earlier version of an overwritten file. A snapshot can

preserve file content but does not prevent an authorized user from making a bad change. Permissions reduce exposure but cannot replace recovery planning for every mistake.

Test the whole recovery path

1. Create a dedicated test share and verify its protection settings.
2. Upload a small file with recognizable content and record its properties.
3. Create a snapshot after the write completes.
4. Change the active file and retrieve the earlier content from the snapshot.
5. Compare the recovered bytes and test the intended user's access.
6. In a separate exercise, delete the dedicated share and restore it through share soft delete.
7. Verify that both the active content and expected snapshots are present after recovery.

The test record should identify the exact snapshot and deleted-share instance used. A list of several similar timestamps can be confusing during an incident. Clear names, times, and content markers make the exercise reproducible. The operator should also record how long each stage took without presenting that one measurement as a universal recovery guarantee.

Diagnose an unsuccessful restore

If the desired file is absent from a snapshot, confirm that it existed and had been written before the snapshot was taken. A later upload cannot appear in an earlier point-in-time state. If the snapshot contains the file but restoration fails, inspect permissions, destination state, open handles, and the selected tool's operation. These checks distinguish an unavailable recovery source from a blocked recovery action.

If an entire share cannot be undeleted, inspect the applicable retention state, name conflicts, resource model, and permissions. A runbook for classic shares cannot be assumed valid for a different provider model. The operator should preserve evidence and avoid creating more replacement resources until the recovery path is understood. Hasty recreation can complicate names, quotas, and application references.

File protection works when the team can name the recovery source, select the correct scope, and demonstrate usable restored content. Snapshots supply historical states; soft delete supplies a window for recovering a deleted share. Combined with appropriate permissions and a tested backup strategy, they allow routine mistakes to be repaired through a known procedure instead of an improvised search for copies.

11.4 Managing Keys, SAS, and Stored Access Policies

Credentials differ in scope and lifetime

Azure Storage supports several authorization mechanisms, and they should be chosen deliberately. Identity-based authorization associates access with a principal and permissions. Account keys provide a powerful shared credential for supported operations. Shared access signatures delegate a defined capability through a signed token. The right choice depends on the service, protocol, client, and operational requirement.

An account key is not a user's personal permission. Anyone holding a usable key can perform the supported operations authorized by that credential within other applicable controls. Sharing it with an application or person creates a secret-distribution responsibility. The account owner needs to know where the key is stored, how it is rotated, and what fails if it is replaced.

Management roles that can retrieve keys deserve particular attention. A person may lack a direct data role yet be able to obtain a key through management permissions. That alternative path can undermine an access model that reviews only data-role assignments. A complete review considers key-listing permissions, application configuration, and secret stores as well as ordinary user roles.

Rotate keys through a coordinated process

Storage accounts provide keys that support a staged rotation process. The administrator should identify which credential each consumer uses before regeneration. Regenerating a key while an application still depends on it can cause immediate failures. A rotation plan changes consumers to the alternate valid credential, verifies them, regenerates the unused key, and then completes the intended transition.

The process should not assume that every consumer appears in one configuration file. Scheduled jobs, desktop tools, backup software, and forgotten scripts can retain credentials. Inventory and monitoring help expose those dependencies. A rotation is successful when the intended applications work with the new arrangement and the retired credential no longer remains an undocumented dependency.

Key material should be handled through approved secret storage and protected deployment mechanisms. It should not be printed in ordinary logs or committed to source control. A troubleshooting transcript needs the fact that key-based authorization was used, not the key value itself. If exposure occurs, the response should consider every consumer of the affected credential and coordinate rotation rather than merely deleting the visible log line.

A SAS delegates a bounded capability

A shared access signature, or SAS, carries signed information describing permitted access. Depending on its type and supported service, it can restrict resources, operations, validity period, protocol, and other conditions. The client presents the token with its request. The signature allows the service to determine whether the token was constructed using the appropriate signing authority and whether the request fits its constraints.

The token should be treated as a credential. A URL containing a SAS can grant access to whoever obtains it, subject to the token and other service controls. Placing the URL in an email, browser history, log, or issue tracker can therefore disclose access. The delegation design should minimize both its scope and its useful lifetime, while keeping enough time for the legitimate task to complete.

A SAS is not a mechanism for bypassing storage network restrictions. A valid token can still fail when the request comes from a disallowed network or uses an unsupported protocol. Likewise, a network-allowed client can fail because the token lacks the required permission. Troubleshooting should inspect these independent conditions rather than assuming one successful condition overrides all others.

Understand the signing and policy models

A service SAS delegates access within a storage service and is signed with the account key. An account SAS can cover supported operations across services and resource types at broader scope. A user delegation SAS uses a delegation key obtained through Microsoft Entra authorization under the supported service and API model. Current support should be checked for the intended service and tool rather than applying an outdated assumption that every SAS type is available everywhere.

The administrator should prefer a design that avoids distributing account keys when the supported identity-based alternative meets the requirement. That preference does not remove the need to constrain the resulting SAS. A token signed through an identity-based process can still be overly broad or long-lived. The review should examine both how the token is signed and what it permits.

Stored access policies provide server-side constraints for supported service SAS relationships. A policy is defined on an appropriate container-like resource, such as a blob container or file share. A service SAS can refer to that policy so that supported policy changes affect its access. Account SAS and user delegation SAS do not use stored access policies in the same way; they are not interchangeable policy references.

Design revocation before issuing access

An ad hoc SAS contains its relevant constraints in the token. Revoking it may require a broader action, such as changing the signing-key relationship, depending on its type and configuration. A service SAS associated with a stored access policy offers a policy-management path for changing or withdrawing the delegated access. The exact revocation method should be documented before the token is distributed.

Revocation can affect more than one client. Several service SAS tokens may rely on the same stored policy, and many applications may rely on the same account key. A response plan should identify that shared dependency. An emergency action that stops an exposed token but also interrupts essential unrelated workloads may still be necessary, but its consequence should be understood and managed.

Short validity periods reduce the duration of exposure but require a renewal mechanism. A batch job that runs longer than its token's lifetime can fail partway through a transfer. The administrator should estimate the task duration, handle retries, and avoid issuing a needlessly long token merely to hide unreliable scheduling. Expiration is part of the workflow contract, not an afterthought.

Mechanism	Main advantage	Operational responsibility
Identity-based role	Access linked to a principal	Review role scope and identity lifecycle
Account key	Broad compatibility for supported operations	Protect, inventory, and rotate a powerful secret
Ad hoc SAS	Limited delegated capability	Plan expiry and the appropriate revocation method

Mechanism	Main advantage	Operational responsibility
Service SAS with policy	Supported server-side constraint management	Track policy consumers and change effects

Read a failed SAS request as evidence

A forbidden request can result from expiry, a not-yet-valid start time, missing operation permission, wrong resource scope, changed signing key, or another applicable control. The administrator should capture the operation and nonsecret token properties without logging the signature. The error's request identifier and timestamp can help correlate service diagnostics. Blindly regenerating a broader token can conceal the original cause.

For example, a token allowing read access to one blob does not necessarily allow listing the entire container. A graphical tool that tries to browse first may fail even though a direct read of the permitted object would succeed. The correct fix depends on the requirement. If only one object should be shared, changing the client workflow may be preferable to granting broad listing access.

Time conditions can also surprise operators. A token created with an immediate start time can be affected by clock assumptions across systems. The issuing workflow should follow current guidance and use a deliberate validity window. It should not remove expiration entirely to avoid investigating timing. A clear issuance record helps determine which token a client actually used when several similar URLs exist.

Keep file protocol limitations visible

A SAS usable for a supported REST operation is not automatically a credential for mounting an SMB share. SMB authentication has its own supported methods. Similarly, a role used for a privileged FileREST operation should not be described as equivalent to an ordinary user's directory ACL rights. Every example should name the protocol and operation so that the credential's meaning stays clear.

A credential inventory should identify the authorized operation, consumer, permitted location, lifetime, and withdrawal method. This gives the administrator a practical basis for rotation and incident response. A connection test demonstrates immediate access; the inventory explains how that access remains controlled after deployment.

11.5 Restricting Storage Network Access

Network admission and authorization answer different questions

A storage network control decides whether a request from a particular path is admitted to the service endpoint. Authorization decides whether the request's identity or credential can perform the operation. Both must succeed. Allowing a subnet does not grant its users permission to read every object, and granting a data role does not make a blocked network path usable.

This distinction explains many confusing failures. An administrator can manage a storage account through Azure Resource Manager while a data request from the same workstation is denied by the

account's data endpoint policy. Management and data traffic use different operations and controls. The portal's ability to display account settings is therefore not proof that its data browser can reach the service successfully.

The network design should begin with the intended clients. These might include Azure VMs, managed applications, an on-premises server, an administrator's workstation, and a recovery environment. Each has a source path and a protocol. An allowlist assembled only from the first development machine is unlikely to support the full application. Conversely, allowing every network to avoid inventory work defeats the intended restriction.

Public endpoints can be restricted

A public service endpoint can remain publicly addressed while access is limited to selected networks or supported resource exceptions. IP rules identify applicable public source addresses or ranges. Virtual network rules identify supported subnets using the service-endpoint relationship. The exact options and considerations depend on the storage service, so Azure Files deployments should use the Files-specific networking guidance as well as the account's general concepts.

An IP rule should use the source address the service actually observes. A workstation behind corporate network address translation does not present its private desktop address to a public service. A changing outbound address can also break an apparently correct allowlist later. The administrator should identify the stable egress path and coordinate changes with the network team rather than repeatedly adding whatever address happened to work once.

Rules intended for public source addresses should not be used as a substitute for a virtual-network design where the service uses a different source identity. Traffic from Azure services can follow paths that do not behave like a random internet client. Current service documentation explains those boundaries. A precise design uses the supported rule type for the actual source instead of assuming every packet can be admitted by its familiar private address.

Service endpoints preserve the service's public endpoint

A service endpoint associates a subnet with access to a supported Azure service over the Azure network. The storage account's virtual network rule then permits the selected subnet under the supported configuration. Both sides matter. Enabling an endpoint on a subnet does not automatically grant access to every storage account, and adding a subnet rule without the required endpoint configuration can leave the intended path incomplete.

For SMB clients, verify the required TCP 445 path in addition to the account's network rule. An HTTPS portal operation on TCP 443 tests a different path. Recording both results prevents a successful browser upload from being used as evidence that an SMB mount should work through an upstream firewall.

The service's endpoint remains a public service endpoint in this model. A service endpoint does not allocate a private IP address for the storage service inside the subnet. That distinction affects DNS, exposure review, and how on-premises clients connect. An administrator should choose it because its access model fits the requirement, not because the word endpoint sounds equivalent to Private Link.

The region and endpoint type can influence supported access. The administrator should verify the current Microsoft.Storage or cross-region service-endpoint configuration appropriate to the design. A

disaster-recovery environment may need its own prepared network relationship. Waiting until regional failover to discover that the secondary application's subnet is not permitted can turn a storage recovery into an application outage.

Private endpoints establish a private service path

A private endpoint gives a supported storage subresource a private address in a virtual network. Clients use the normal service name through a DNS configuration that resolves appropriately to that private path. The endpoint is associated with a particular service subresource, such as files or blobs. Creating a blob private endpoint does not automatically create a file-service endpoint for the same account.

DNS is therefore central to the deployment. The administrator should inspect the full resolution result from the actual client. A private endpoint resource can exist while clients still resolve the service through a public path. Private DNS zones, virtual network links, and custom resolver forwarding must align. Chapter 8's separation of name resolution from packet connectivity applies directly here.

Creating a private endpoint does not necessarily disable all public access automatically. Public network settings must be reviewed separately against the intended policy. Some environments require private-only access; others retain a restricted public path for selected clients. The chosen result should be explicit and tested from both an allowed and a disallowed location.

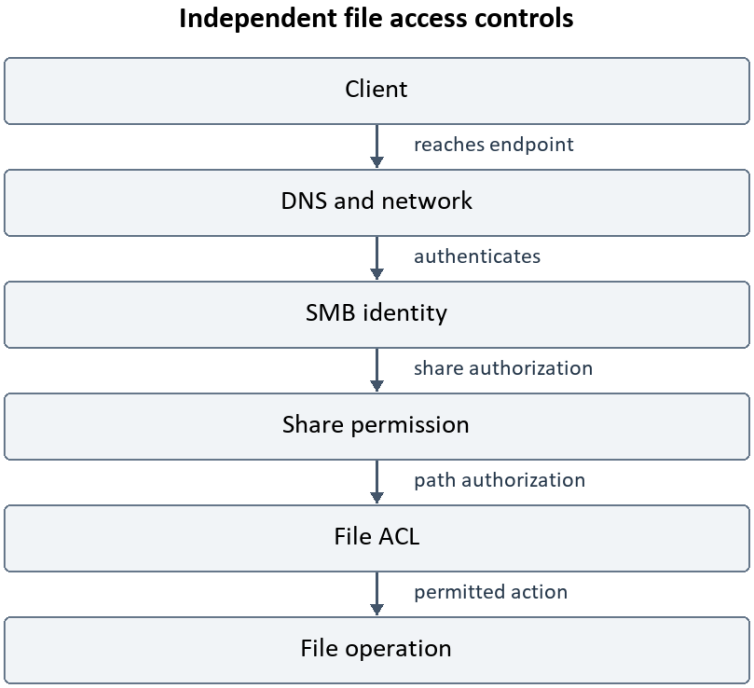


Figure 11.1 Storage access requires several independent checks

Identity-based SMB access requires a usable network path, supported authentication, share-level permission, and applicable file permissions. A key-authorized REST test follows a different credential path.

Trusted exceptions need a defined reason

Trusted-service and resource-instance exceptions support specific integration scenarios. They should not be interpreted as a universal permission for every Azure workload or every identity in the tenant. The administrator should identify the exact service and operation that needs the exception and verify its supported authorization requirements. An exception that solves one integration may still leave another service blocked, correctly.

The review should also consider whether a narrower supported design is available. Exceptions are not inherently wrong; they are configuration decisions that need ownership and evidence. A short comment explaining the consumer and required operation is more useful than a checkbox enabled because a tutorial used it. That explanation becomes valuable when the integration is retired or moved.

File protocol access adds client-path requirements

SMB file access needs a permitted protocol path from the client as well as storage-side admission. A local firewall, corporate edge, or service provider can block the connection before Azure evaluates the account rule. Private connectivity can provide an appropriate alternative path in a designed environment, but it still requires routing and DNS. A storage firewall rule cannot repair every upstream network restriction.

An authenticated HTTPS data operation through a tool can work while an SMB mount fails. Those requests use different protocol paths and may use different credential methods. The administrator should not treat the first success as proof that the second path is valid. A diagnostic table should list endpoint name, resolved address, port, credential model, and observed result for each test.

Change restrictions without losing the evidence

Before tightening access, inventory active clients and establish an approved administrative test path. Apply the intended rules, then verify each required consumer. A short maintenance plan should describe the expected denied cases as well. If the account is supposed to reject an unapproved workstation, that failure is a successful control test rather than an incident.

1. Identify service endpoints and the actual client network paths.
2. Choose public restrictions, service endpoints, or private endpoints according to requirements.
3. Configure DNS and routing dependencies before the final restriction.
4. Verify data authorization with the intended identity.
5. Apply the network policy and test approved consumers.
6. Test a controlled unapproved path and record the expected denial.
7. Preserve the configuration and remove obsolete exceptions after review.

Diagnose a forbidden request without broadening everything

A client with the correct data role receives a forbidden response after a network change. The administrator confirms its resolved endpoint and actual source path. If the account's rule no longer permits that path, the correction belongs to the network design. Assigning Owner would add management power without fixing the admission failure. Conversely, if the path is permitted but the credential lacks data rights, adding another IP rule is ineffective.

The strongest storage access design makes those two conclusions easy to reach. Each client has a known network route and a known authorization method. Tests verify both independently, and exceptions have named owners. This allows the administrator to restore legitimate access with a targeted change while preserving the boundary that keeps unrelated clients and credentials out.

Guided lab

Objective and prerequisites

Create a classic SMB share, preserve a known file in a snapshot, and recover its earlier content. Then inspect share soft-delete configuration and perform a separate deleted-share recovery. This lab uses the portal's supported key-authorized file-data path for an isolated account. It does not demonstrate a user's identity-based SMB mount or ACL behavior; those require a prepared supported identity environment and client.

The operator needs permission to create the resource group, account, and share, and permission to retrieve the account key for the portal data operation. A role such as Contributor at the dedicated lab scope supplies relevant management permissions, including key retrieval, but its broad key capability is precisely why this is an isolated teaching account. Do not grant that role to ordinary file-share users as a substitute for identity-based access.

Use a subscription that permits a classic general-purpose v2 account and a supported region. The client's HTTPS path must be permitted. The account, active files, snapshots, retained deleted share, and transactions can generate charges. Depending on the selected offering, capacity settings can affect billing even when little data is stored. Review the offering's displayed cost before creation. A budget does not cap spending.

1. Create `rg-compass-ch11` in the chosen region. Create a uniquely named classic StorageV2 account using an appropriate low-capacity lab configuration, LRS where supported, secure transport, and anonymous blob access disabled. Keep this account separate from existing application data. Record the actual account and share billing model.
2. Open the account's classic file-share management view. Confirm file-share soft delete is enabled with a seven-day lab retention period. Read the displayed configuration rather than assuming the default. Create an SMB share named `compassfiles` with the smallest appropriate supported lab capacity or quota. Avoid selecting a different resource-provider model merely because its creation page also says file share.
3. On the local computer, create a text file named `compass-ch11.txt` containing `First verified version`. Upload it to the share through the portal's file-data interface. Use the authorized key-based path offered by the interface and avoid copying the key into notes, screenshots, or source files. Confirm the uploaded file's name and content by downloading it to a separate temporary location.
4. Create a share snapshot and record its timestamp. Then change the local file to `Second working version` and upload it over the active file using the interface's explicit overwrite action. Confirm that a fresh download from the active share contains the second value. The snapshot should remain a read-only record of the earlier state.

5. Browse the recorded snapshot, locate `compass-ch11.txt`, and download its earlier content to a separately named local file. Compare it with the expected first value. Restore that content to the active share through a supported file copy or upload operation, explicitly confirming the overwrite. Download the active file once more and confirm the first value is now current. This verifies a single-file recovery rather than claiming an instant whole-share rollback.
6. For a controlled troubleshooting variation, try to find the second value inside the snapshot taken before that update. Explain why the snapshot correctly contains the first value. A later write cannot appear in an earlier point-in-time state. Record the snapshot time and the two content values as evidence.
7. Perform the share-level test separately. Delete only the dedicated `compassfiles` share, including its snapshots as required by the interface's confirmation. Display soft-deleted shares, select the correct deleted instance, and restore it within the retention window. Do not create a new share with the same name first. Verify the active file and expected snapshot are available again.
8. Inspect the dedicated group's resources, then delete `rg-compass-ch11` after recording the lab results. Confirm that the group no longer exists. Remove only the local text files and downloads created by this exercise, and clear any copied credential from the clipboard or temporary notes. Account deletion ends this isolated lab; share soft delete is not protection against deleting the parent account.

If policy blocks the key-based portal data path, do not relax that policy. Use an approved prepared identity/REST path with the required permissions and record that changed credential model. A successful management operation is still insufficient evidence for the data operation. No live Azure resources were created or modified during manuscript preparation, and all lab execution remains unverified until performed in an actual environment.

Chapter synthesis

A shared file service has several boundaries: protocol support, client reachability, authentication, share permission, path permission, and recovery scope. Account keys and SAS tokens create additional credential relationships that must remain visible. Snapshots preserve earlier file content, while classic-share soft delete protects a deleted share for a limited period. Clear boundaries allow access and recovery problems to be repaired without granting unnecessary power.

Review questions

11-Q1. A portal upload to a file share succeeds using an account key. What does this establish?

- **A.** The key-authorized data operation worked
- **B.** Every user's SMB ACL is correct
- **C.** Every on-premises network permits TCP 445
- **D.** The share has an independent backup

11-Q2. A user has a share-level role allowing modification, but the target directory's ACL permits only reading. What is expected?

- **A.** The Azure role always overrides the ACL
- **B.** The user remains limited by the applicable directory permissions
- **C.** The directory automatically becomes public
- **D.** The storage account changes its identity source

11-Q3. An individual file is deleted while its parent classic share remains active. Which protection should be inspected for earlier content?

- **A.** Only share soft delete
- **B.** The public DNS TTL
- **C.** A suitable share snapshot or backup
- **D.** The account's display name

11-Q4. Which SAS type can use a stored access policy under the described model?

- **A.** Any account SAS
- **B.** Any user delegation SAS
- **C.** Every SAS without qualification
- **D.** A supported service SAS

11-Q5. Why should consumers be inventoried before regenerating an account key?

- **A.** Applications still using that key can lose access
- **B.** Key regeneration automatically migrates every client
- **C.** Keys are personal user roles
- **D.** Key regeneration changes all file names

11-Q6. A blob private endpoint exists for an account. Does that alone provide the account's file service a private endpoint?

- **A.** Yes, all service subresources are always included
- **B.** No, the intended storage subresource needs its own supported endpoint configuration
- **C.** Only if the account name is short
- **D.** Only if a SAS has no expiration

11-Q7. An HTTPS upload works from a workstation, but its SMB mount fails before authentication. Which test is especially relevant?

- **A.** Whether the blob lifecycle rule is enabled
- **B.** Whether the file has an index tag
- **C.** Whether the client's TCP 445 path is permitted
- **D.** Whether an older snapshot has been deleted

11-Q8. A SAS permits reading one blob but not listing its container. A browsing tool fails during listing. What is the best interpretation?

- **A.** The storage account must be corrupt
- **B.** The token should always be replaced with an account key
- **C.** All network controls must be removed
- **D.** The tool's listing operation exceeds the token's granted capability

Further reading

- Azure Files introduction
- SMB identity-based authentication
- Share snapshots
- Classic file-share soft delete
- SAS overview
- Stored access policies
- Storage key management
- Azure Files networking
- Storage public-endpoint controls

Chapter 12: Bringing Virtual Machines into Service

A virtual machine supplies a configurable operating environment, but an operational server consists of more than its compute allocation. Disks, network interfaces, access controls, identity, images, and maintenance all contribute to the result. This chapter follows a VM from its initial design through storage, sizing, encryption, and movement. The aim is a server whose dependencies and behavior can be explained before a change is made.

Learning objectives

- Select an image, size, network, and operating model for a VM.
- Distinguish operating-system, data, temporary, and ephemeral storage.
- Evaluate and perform a disruptive resize with appropriate checks.
- Explain and configure supported encryption at host.
- Separate administrative moves from regional relocation.

12.1 Selecting and Deploying a Virtual Machine

Choose a VM for the control it provides

An Azure virtual machine provides an operating environment whose guest operating system and installed software remain under the customer's administration. Azure manages the physical infrastructure, while the organization manages its workload configuration, patching, accounts, and application behavior. This division makes VMs useful when a workload needs operating-system control or software that does not fit a more managed hosting service.

The choice should begin with requirements rather than habit. An application that needs a particular operating system, agent, filesystem layout, or server package may justify a VM. Another application may be easier to operate through a managed application service. The administrator should understand the reason for using a VM because that reason determines which responsibilities the team must continue performing after deployment.

A VM specification should describe the workload, operating system, processor architecture, memory, disk behavior, network access, identity, and availability requirement. It should also identify who owns patching and recovery. Without those decisions, the deployment wizard can create a running server whose maintenance responsibilities remain undefined. A green power-state indicator is not an operational handover.

Images establish the starting state

An image supplies the initial operating-system content and, in some cases, preinstalled software. Marketplace images, organizational images, and specialized disk-based deployments serve different purposes. The administrator should select a trusted publisher and a supported release, then record the exact image reference used. A floating latest version can be convenient for exploration but makes reproduction harder when the publisher updates the image.

Processor architecture and image generation must be compatible with the chosen VM size and security configuration. An image intended for one architecture cannot be assumed to run on another merely because both are listed as Linux. Drivers, boot mode, application binaries, and licensing may constrain the choice. Compatibility should be verified before a deployment script is reused across different VM families.

Generalized images are prepared to create new instances with distinct machine-specific settings. Specialized images preserve an existing machine state and require different handling. The administrator should not generalize a production machine casually or assume that capturing a disk produces a clean reusable image. An image pipeline needs preparation, validation, and a clear statement of what identities and secrets are excluded.

A VM is connected to several Azure resources

The compute resource references an operating-system disk, network interface, and other configuration. Data disks, public IPs, network security groups, load balancers, and identity relationships can add further dependencies. These resources may have separate lifecycles and charges. Deleting the VM should therefore trigger a review of associated resources rather than an assumption that every cost disappeared.

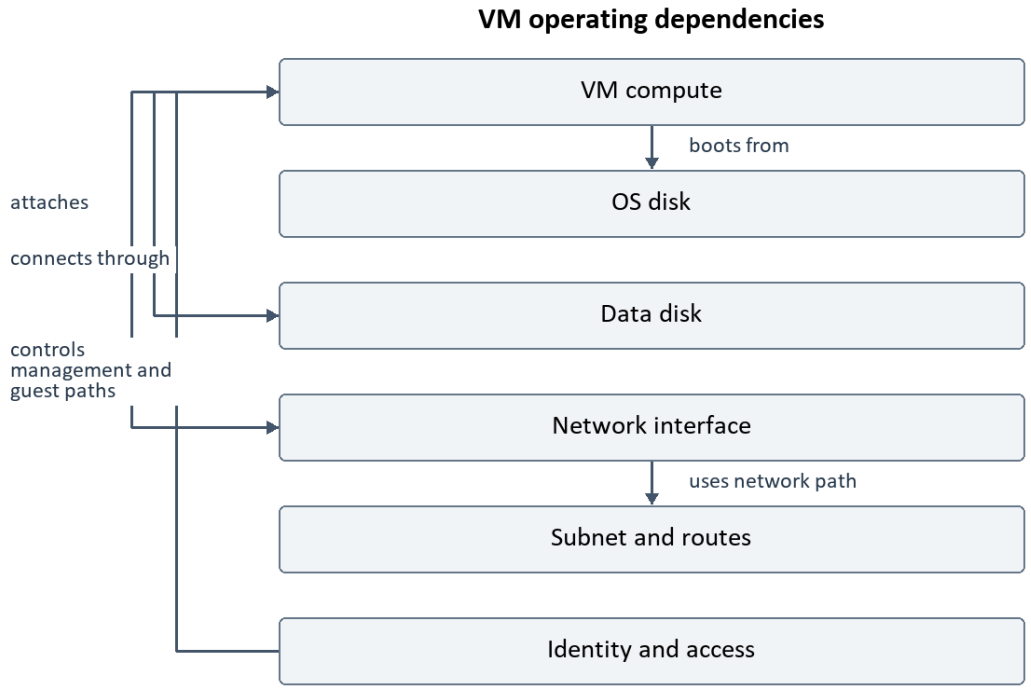


Figure 12.1 A VM and its operating dependencies

A VM references independently managed resources. Persistent storage, network paths, and management or guest identities require separate verification; power state alone is not application readiness.

A network interface connects the VM to a subnet and carries IP configuration. The subnet belongs to a virtual network with its own routes and DNS settings. Network security groups can apply at subnet or

interface scope. The VM's guest firewall and application listener add more controls. A successful deployment does not guarantee that the intended management or application connection is permitted through all of them.

Public access is a choice, not a requirement for every VM. A VM can use private administration through a supported Bastion or connected-network path. If a public management endpoint is deliberately used in a lab, its source should be restricted and its lifetime short. Publishing an administrative port to every address merely to simplify the first connection creates unnecessary exposure and poor habits.

Outbound connectivity must be designed explicitly

Many server setup tasks need outbound access: downloading packages, retrieving configuration, contacting identity services, or returning extension results. New network designs should not rely on implicit default outbound behavior. The administrator should select a supported explicit outbound mechanism and verify it for the subnet and workload. NAT Gateway, appropriate load-balancer outbound configuration, or a controlled network appliance are possible design components.

The chosen mechanism affects the source address observed by external services, connection capacity, cost, and troubleshooting. A package installation that times out may indicate a missing outbound path rather than a broken image. Similarly, an extension or Run Command operation can depend on the guest agent and required connectivity. A deployment test should separate platform resource creation from successful guest configuration.

The lab at the end of this chapter avoids installing packages or running guest commands. It inspects management-plane state only and makes that limitation explicit. A production readiness test needs more: controlled management access, required outbound connectivity, and an application-level check. This distinction keeps the book's unexecuted examples from claiming evidence they do not supply.

Administrative access needs a credential plan

Linux deployments commonly use SSH public-key authentication for administrative access. The private key should remain protected on the authorized client, while the public key can be supplied during deployment. Windows access uses its own supported credential and identity options. Credentials should not be embedded in templates or printed in command output. A placeholder in an example is not an instruction to store a real password in source control.

The local administrator account is not the same as an Azure RBAC role. A user allowed to start a VM through the management plane does not automatically have a guest operating-system account. Conversely, a local guest administrator does not automatically have subscription permissions. Where Azure-integrated login is used, its configuration and roles must be verified explicitly. These identity boundaries are important during both onboarding and incident response.

Managed identity gives a VM-associated workload a supported identity for accessing other Azure resources without distributing an ordinary application secret. The identity still needs appropriate permissions on the target. Enabling it does not grant access to every storage account or key vault. The deployment should record which resource uses the identity and which operations are authorized.

Create with reproducibility in mind

1. Record the workload and operating-system requirements.
2. Select a supported image, size, region, and availability placement.
3. Define persistent storage and the intended data locations.
4. Create or select the subnet, DNS configuration, and explicit outbound design.
5. Configure administrative access, identity, and narrowly scoped network rules.
6. Deploy the VM and inspect the resulting resource relationships.
7. Validate guest health, required connections, and application behavior.
8. Register patching, monitoring, backup, ownership, and cleanup responsibilities.

Infrastructure code can make these choices repeatable, but a template does not automatically make them correct. The deployment should use parameters for environment-specific values while preserving security and compatibility requirements. A reviewer should be able to identify which resources are created and what each one costs. Hidden defaults make later troubleshooting harder.

Interpret deployment and power states correctly

Provisioning state describes the outcome of an Azure management operation. Power state describes whether the VM is running, stopped, or deallocated under the reported state. Guest health and application health are additional questions. A VM can be running while its web service has failed. A deployment can complete while an optional application setup script later reports an error.

Stopping the guest operating system and deallocating the Azure compute allocation are not identical billing actions. A stopped-but-allocated VM can continue incurring compute charges. Deallocation releases the compute allocation, while persistent disks and other resources can continue generating charges. A lab cleanup guide should therefore use explicit deallocation or deletion and inspect associated resources rather than relying only on an operating-system shutdown.

Diagnose the first unsuccessful connection

If a VM is running but cannot be reached, confirm the destination name and address, the client's route, effective security rules, the guest firewall, and the listening service. A failed SSH connection can also reflect the wrong username or key. The error stage helps distinguish a network timeout from an authentication rejection. Recreating the VM before checking these facts can discard useful evidence.

Handover should include verified configuration, guest state, access paths, and operational ownership. The record gives the team the information needed to patch, resize, protect, and recover the machine. A running power state remains useful evidence, but it is only one part of that handover.

12.2 Managing Disks

Persistent and temporary storage have different promises

An operating-system disk contains the VM's boot environment. Data disks provide additional block storage for application data and other persistent uses. Temporary storage, where present, is local working space whose contents must not be treated as durable across relevant platform events. The

administrator should identify each storage type inside the guest before choosing where important information is written.

A familiar drive letter or device name can be misleading. A temporary disk may appear as an ordinary filesystem, making it easy for an application to store valuable content there accidentally. The configuration should use deliberate paths and document which locations are disposable. Data that cannot be reconstructed should reside on an appropriately protected persistent storage service, not on a location selected only because it has free space.

Ephemeral OS disks are another specialized option for supported workloads and sizes. They place the operating-system state on local storage under a model suited to replaceable instances. They are not equivalent to a conventional persistent managed OS disk. A stateful server that depends on preserving local operating-system changes needs a design review before using them. Rebuildability is the central requirement.

Managed disks simplify infrastructure ownership

Azure managed disks provide block-level storage resources for VMs. The administrator selects a supported type, size, and performance configuration while Azure manages the underlying storage allocation. This avoids managing a separate storage account for each ordinary managed-disk deployment. It does not remove the need to plan capacity, performance, encryption, backup, or deletion behavior.

Disk offerings differ in latency, throughput, operation capacity, cost, and supported features. Standard HDD, Standard SSD, Premium SSD, Premium SSD v2, and Ultra Disk are examples of offerings with different use cases and constraints. The administrator should compare current capabilities for the actual VM size, region, and workload. A name containing premium is not enough to establish compatibility or optimal performance.

The VM itself can impose aggregate disk throughput and input/output limits. Adding a faster disk may not improve the application if the VM's limit or another bottleneck is already reached. Disk and compute sizing should therefore be reviewed together. The measurements should distinguish the individual disk's behavior from the VM's total storage path and the application's request pattern.

Attach the correct disk before touching its contents

Attaching a disk in Azure makes a block device available to the guest. The operating system may still need to discover, initialize, partition, format, and mount it. These are separate steps with different risks. A newly attached empty disk can be prepared for use, while an existing disk containing valuable data must not be formatted simply because it appears unfamiliar.

The administrator should correlate the Azure disk resource, attachment information, and guest device before running destructive disk commands. Device ordering can change, and a command copied from another VM can target the wrong device. Stable identifiers and careful inspection are preferable to assuming that the newest disk always appears under a particular device name. The procedure should stop when the identity is uncertain.

On Windows, disk management tools can identify offline or uninitialized disks and their sizes. On Linux, block-device listing and filesystem tools reveal devices, partitions, and mounts. The exact commands

depend on the operating system and filesystem. A generic Azure attachment example should not pretend that one formatting command is correct for every guest. The book's lab inspects attachment without formatting.

Plan data layout for operation and recovery

Separating application data from the OS disk can make management and recovery clearer. It can support a different performance selection, easier capacity growth, and an explicit data-protection policy. The separation does not automatically create redundancy or backup. A single data disk remains a resource that needs an appropriate recovery strategy for deletion, corruption, and application mistakes.

Applications with several related disks may require coordinated snapshots or application-aware backup to capture a consistent state. Independently snapshotting disks during active writes can produce states from different moments. The administrator should understand the application's consistency requirements and use a supported protection method. A collection of individually successful disk snapshots is not necessarily a recoverable database.

Caching options can affect performance and data behavior for supported disk configurations. The choice should follow the workload and service guidance. A setting that helps read-heavy data may be inappropriate for a different write pattern. The administrator should avoid changing caching during an incident without considering application requirements and the possible disruption of the configuration change.

Storage location	Typical role	Operational boundary
Persistent OS disk	Boot and operating-system state	Protect and maintain as part of the server
Managed data disk	Application data	Initialize carefully and back up appropriately
Temporary local disk	Reconstructable scratch data	Contents can be lost during relevant events
Ephemeral OS disk	Replaceable operating-system instance	Requires a rebuildable workload design

Expansion includes an Azure and a guest stage

Increasing a managed disk's capacity does not necessarily extend the guest partition and filesystem automatically. The administrator first performs a supported Azure disk expansion and then uses the correct guest procedure to use the new space. The operation should include a recovery plan and a check of supported online or offline behavior for the selected disk and operating system.

The guest verification should compare the block device's capacity with the partition and filesystem capacities. If Azure shows a larger disk while the application still sees the old free space, the missing step may be inside the guest. Repeatedly increasing the Azure disk can create cost without solving the problem. The observed layers should be compared before making another change.

Shrinking disks is not the reverse of expansion in ordinary managed-disk administration. A reduction can require migration to a new disk or another supported process. The administrator should not assume that a capacity selection can always be undone without moving data. Planning a small lab disk is therefore different from promising that a large production disk can be reduced later with one click.

Deletion and detachment are different operations

Detaching a data disk removes its connection to the VM while leaving the disk resource available. Deleting the disk removes the resource and its data under the applicable protection model. A VM's disk delete option can influence what happens when the VM is deleted. The administrator should inspect those settings explicitly before cleanup or server retirement.

A detached disk can continue generating charges. It may be a forgotten lab resource or an intentionally preserved recovery artifact. An inventory should distinguish those cases through ownership and retention information. Automatically deleting every unattached disk can destroy valid recovery data. Conversely, retaining every disk forever can create substantial avoidable cost and unnecessary data exposure.

Before deleting an apparently unused disk, inspect its resource tags, creation history, snapshots, and change records. Confirm whether it is a rollback copy or part of a recovery investigation. A useful retirement record names the owner who confirmed disposal and the recovery period that has ended. This provides a defensible reason for deletion instead of relying solely on the absence of a current VM attachment.

Diagnose disk-related symptoms with layered evidence

If a newly attached disk does not appear in the guest, verify the Azure attachment, VM compatibility, and guest rescan state. If it appears but cannot be mounted, inspect its partition and filesystem state without formatting it. If the application reports low space, compare the filesystem with the underlying device and Azure capacity. Each symptom belongs to a different layer.

If latency is high, examine disk metrics, VM limits, queueing, caching configuration, and application request size. An application performing many small synchronous writes can behave differently from a sequential copy benchmark. The administrator should use representative measurements and avoid upgrading a disk based only on a vague report that the server feels slow.

Disk management becomes dependable when resource identity, guest layout, persistence, performance, and recovery are all documented. The administrator can then attach, expand, detach, and retire storage without confusing a temporary workspace with protected data or an Azure configuration change with a completed filesystem operation.

12.3 Choosing and Changing VM Sizes

A size is a bundle of capabilities

A VM size defines a combination of processor, memory, storage, networking, and other capabilities. The number of virtual processors is only one part of that bundle. Two sizes with the same processor count can have different memory, disk support, network capacity, local storage, and processor architecture. An

administrator should compare the required capabilities rather than treating size names as a simple small-to-large ladder.

General-purpose, compute-optimized, memory-optimized, storage-optimized, and accelerator-oriented families serve different workload patterns. The categories help narrow the search, but the actual SKU specifications remain important. An application with high memory pressure may benefit more from a different memory ratio than from adding processors indiscriminately. A storage-bound workload may need a different disk and VM combination rather than more CPU.

The selected image and application must also support the processor architecture and security features of the target. Resizing is not a universal migration between every hardware family. Local temporary storage, disk controller, networking features, and generation constraints can affect the supported transition. Current compatibility guidance should be checked before promising an in-place change.

Measure the bottleneck before changing capacity

CPU utilization can indicate pressure, but it is not the whole performance story. Memory exhaustion, disk latency, network limits, application locks, or a slow external dependency can produce similar user complaints. The administrator should collect measurements that correspond to the observed symptom. A server with low CPU can still be overloaded in another resource dimension.

Guest metrics are often needed to understand memory and filesystem behavior. Platform metrics describe valuable infrastructure signals but do not automatically reveal every application constraint. Logs, performance counters, and application telemetry can show whether work is waiting on storage, memory, or a remote service. A resize recommendation should name the limiting resource and the expected improvement.

Observation should cover a representative period. A quiet hour does not describe a month-end processing peak, and a brief spike does not necessarily justify permanently doubling capacity. The workload's schedule and response-time requirements provide context. The team can then compare a larger fixed VM, a different family, a scheduled change, or a scale-out design where the application supports it.

Quota and capacity are different constraints

Subscription quota limits how much of a resource the subscription is permitted to allocate. Regional or zonal capacity concerns whether the service can supply the requested allocation at that time and location. Having sufficient quota does not guarantee available capacity. Increasing quota does not necessarily resolve a placement-specific allocation failure.

The administrator should inspect the error and the available size options for the existing deployment. A target size may not be available on the current hardware cluster, even if the region supports it generally. Deallocation can permit a different placement, but it introduces disruption and does not guarantee that every desired allocation will succeed. A change plan should include an alternative or rollback path.

Reservations and savings arrangements affect pricing under their terms; they should not be confused with every form of capacity assurance. Capacity reservation is a distinct concept where supported. The administrator should use precise terminology when discussing cost commitments and allocation

requirements. A finance decision and a resilience decision can be related without being the same product feature.

Resizing a running VM is disruptive

Changing the size of a running VM causes a restart under the documented operation. Some transitions require deallocation first. The application must therefore be prepared for interruption, and the change should be scheduled according to its service requirements. A portal's ability to offer the target size does not mean the application experiences a seamless change.

The plan should account for temporary storage and address behavior during deallocation or placement changes. Important data must not depend on disposable local storage. Network references should use deliberately configured identities and addresses rather than assumptions about incidental allocations. Exact address preservation depends on the resource configuration and supported operation, so it should be verified before the change.

Stateful applications deserve extra care. In-flight work may need to drain, services may need an orderly shutdown, and a valid recovery point should exist. A restart can expose an application that was never configured to start automatically. The resize test should therefore include post-boot service behavior rather than ending when Azure reports the VM running.

Prepare a change with explicit checkpoints

1. Record the current SKU, image, disks, network interfaces, and security profile.
2. Measure the workload and identify the capacity constraint.
3. Check target compatibility, regional availability, quota, and current resize options.
4. Confirm a suitable recovery point and the application's shutdown procedure.
5. Notify affected owners and execute the required restart or deallocation sequence.
6. Apply the supported resize and confirm successful allocation.
7. Verify guest resources, application startup, data access, and performance.
8. Record the result and retain a tested rollback approach until acceptance.

The allocation check is important because a requested model change can differ from the resources actually running after a failed resize. The administrator should inspect the operation result and instance evidence, not merely the requested size shown in configuration. A failed operation should be resolved or rolled back explicitly before reporting that capacity was increased.

Use current size discovery rather than hard-coded assumptions

```
az vm list-vm-resize-options \
  --resource-group rg-compass-ch12 --name vm-compass-ch12 --output table
az vm show --resource-group rg-compass-ch12 --name vm-compass-ch12 \
  --query hardwareProfile.vmSize --output tsv
```

The first command reports resize options relevant to the existing VM under the service's current conditions. The second reads the configured size. Neither command proves application performance or guarantees a future allocation. A production script should inspect errors and operation status rather than selecting the first returned SKU automatically. The intended target must still meet the workload and compatibility requirements.

Compare scale up with scale out

Scaling up changes the capabilities of an individual VM. Scaling out adds instances and distributes work, where the application supports that model. Scaling up can be simpler for a tightly coupled stateful application, but it leaves the workload dependent on an individual instance's lifecycle. Scaling out can improve capacity and resilience but requires shared state, traffic distribution, and consistent deployment.

The choice should be driven by application architecture. Adding a second VM does not automatically share an in-memory session or synchronize a local database. A larger VM does not protect against every instance failure. The next chapter develops availability and scale sets, while this section establishes why a size change is only one capacity tool.

Symptom	Candidate investigation	Unhelpful shortcut
High CPU during useful work	Processor capacity and application efficiency	Increase disk size without evidence
Memory pressure and paging	Memory capacity and application allocation	Assume low CPU means sufficient capacity
Storage queueing	Disk and VM storage limits	Add processors without checking I/O
Allocation failure	Quota, placement, and current availability	Repeatedly retry without reading the error

Right-size after the change as well

Post-change monitoring should compare the same workload and metrics used to justify the resize. If the limiting condition remains, the original diagnosis may have been incomplete. The administrator should not treat a successful resource change as proof of a successful performance change. A larger bill without improved service is useful evidence that the next investigation should take another direction.

Downsizing also requires validation. A VM that appears underused most of the time may support infrequent peaks, recovery tasks, or batch work. The team should evaluate those needs and the disruption of changing size. The goal is an appropriate operating envelope, not the smallest SKU that can boot the operating system during a quiet test.

Diagnose a failed resize without losing control

If the operation fails, capture the error, requested SKU, prior SKU, and current power and instance state. Determine whether the failure concerns compatibility, quota, or capacity. Do not assume the VM now has the requested resources simply because its model displays that size. Verify actual allocation and restore a known supported configuration if necessary.

A successful sizing process connects evidence to a change and then checks the result. It treats restarts, allocation constraints, guest behavior, and cost as part of the operation. That discipline allows an administrator to improve performance without confusing a new SKU label with a healthier application.

12.4 Configuring Encryption at Host

Identify the data boundary being protected

Managed disks use service-side encryption for persisted data. Encryption at host extends protection to data on the VM host, including supported temporary storage and disk caches, and keeps data encrypted as it flows to the storage service under the feature's design. The distinction matters because a disk's persisted copy and data temporarily present on the host are different parts of the storage path.

An administrator should describe the requirement in those terms. A statement that disks are encrypted can be true while leaving the host-level question unanswered. Conversely, enabling host encryption does not replace application authorization or protect data after an authorized process reads it into memory. Encryption features have boundaries, and the design should identify the boundary that each one addresses.

The feature is configured through the VM or scale-set security model for supported configurations. It is not the same as installing a guest operating-system encryption extension. This keeps the operational process distinct from Azure Disk Encryption, which has its own architecture and compatibility constraints. Old deployment instructions should be reviewed rather than combined with host encryption indiscriminately.

Verify compatibility before enabling the setting

Supported VM sizes, disk configurations, and subscription prerequisites must be checked. Some older sizes do not support the feature. A size available in the region is not automatically a size supporting every security capability. The administrator should inspect the relevant SKU capability and current Microsoft guidance for the exact deployment instead of assuming a common family name is sufficient.

The subscription may require the documented EncryptionAtHost feature registration. Registration is a management operation requiring appropriate permission and can take time to complete. The deployment should confirm the registered state before using the setting. A failed registration or pending state is not repaired by repeatedly recreating the VM with the same unsupported request.

Feature registration should be checked in the same subscription that will host the VM. A successful check in a development subscription says nothing about a separate production subscription. Record the subscription identifier with the validation evidence and verify the active CLI or PowerShell context before any change. This simple context check prevents an otherwise correct procedure from modifying or inspecting the wrong environment.

Existing Azure Disk Encryption history is an important restriction. Microsoft documents incompatibility with VMs or scale sets that currently or previously used Azure Disk Encryption under the relevant feature rules. The administrator should check that history rather than looking only for a currently installed extension. A migration plan may require a supported new deployment instead of an in-place toggle.

Understand the key relationships

Host encryption uses platform-managed or customer-managed key relationships according to the supported disk configuration. Temporary and ephemeral operating-system storage have their documented platform-managed behavior, while disk caches follow relevant disk encryption choices. The administrator should avoid a broad claim that every host component uses a customer key merely because one data disk does.

Customer-managed disk encryption introduces resources and identity relationships such as a disk encryption set and access to the appropriate key service. Those dependencies must be available for the VM to operate under the supported design. The key-management principles from Chapter 9 apply: custody, rotation, permissions, deletion protection, and recovery all matter. Enabling encryption does not remove the need to operate the key dependency.

The identity used for key access differs from the VM application's permissions to read a blob or query a database. A managed identity on the VM does not automatically authorize a disk encryption set to use a key. The architecture should show each principal and target separately. This prevents broad role grants made in the hope that some unspecified identity will gain the missing permission.

Plan existing-machine changes as maintenance

For an existing supported VM, enabling encryption at host requires the documented deallocation and reallocation sequence. The administrator should prepare for interruption and verify that important data is not stored only on temporary storage. The application shutdown procedure, backup state, and restart behavior should be reviewed before the change. A security improvement should not be implemented through an unplanned availability incident.

The change record should capture the original security profile and selected disk encryption configuration. After enabling the setting and restarting, verify both the management property and the workload. A property showing true is evidence of configuration, while successful boot and application tests establish additional operational behavior. Both are needed for a complete acceptance record.

For scale sets, changing the model can affect new and existing instances differently. Existing instances may require their own supported update or reallocation process. An administrator should not assume every running instance gained host encryption the moment the scale-set model changed. The next chapter's distinction between desired model and instance state is relevant here as well.

A reviewable configuration workflow

1. Confirm the required host-level protection and applicable policy.
2. Inspect the VM's current size, image, disks, encryption history, and security profile.
3. Check supported SKU capabilities and subscription feature registration.
4. Verify key-service dependencies if customer-managed disk encryption is used.
5. Establish a recovery point and schedule the required downtime.
6. Deallocate and apply the supported host-encryption configuration.
7. Reallocate, inspect the resulting property, and test the workload.
8. Record evidence and any remaining encryption or access requirements.

This sequence separates eligibility from execution. It also gives the operator a stopping point if the existing VM is incompatible. The correct response to an unsupported configuration is a migration or alternative design, not weakening an unrelated security control or suppressing the deployment error.

```
az feature show --namespace Microsoft.Compute --name EncryptionAtHost \
  --query properties.state --output tsv
az vm show --resource-group rg-compass-ch12 --name vm-compass-ch12 \
  --query securityProfile.encryptionAtHost --output json
```

These read commands inspect prerequisite and VM configuration state. A missing or false property should be interpreted in the context of the actual API and resource, not silently reported as enabled. The commands do not expose key material and do not execute the disruptive configuration change. Their output should be retained with the deployment record where appropriate.

Compare related protection mechanisms accurately

Mechanism	Primary focus	What it does not establish
Managed-disk service encryption	Persisted disk data	Every host-local protection requirement
Encryption at host	Supported host-local data and disk path	Application authorization or backup
Guest encryption model	Encryption within the guest's supported design	Compatibility with every host-encryption scenario
Secure application transport	Data moving between application endpoints	Recovery from deletion or a compromised authorized application

The table is not a ranking from weak to strong. Each mechanism addresses a defined boundary, and some combinations have restrictions. The administrator should select a supported design that satisfies the requirement and can be operated reliably. Adding every available encryption label without checking compatibility can prevent deployment or complicate recovery.

Diagnose an encryption-related failure

If deployment rejects host encryption, inspect the feature registration, SKU capability, and existing encryption configuration first. The error may indicate an unsupported size or a prerequisite that has not completed. Changing the network security group cannot repair those conditions. The investigation should stay close to the control that rejected the operation.

If a VM fails to start after a customer-key-related change, inspect the key resource, its availability, and the permissions of the identity used by the disk encryption integration. A disabled or inaccessible key can be the relevant dependency. Generating a new SSH key would affect guest login, a different layer. Clear naming of each key and identity avoids those category mistakes.

Preserve evidence without exposing secrets

An encryption review should record resource identifiers, selected key-management mode, feature state, and validation results. It should not contain private keys, account secrets, or exported cryptographic values. Most troubleshooting questions can be answered from relationships and status. Secret material in a ticket adds exposure without usually improving diagnosis.

The acceptance decision should state exactly what was verified. The team can confirm that a supported VM is configured for host encryption and that its workload operates after the required maintenance. It should not claim independent cryptographic testing or complete protection against every threat. Precise evidence makes the control trustworthy and keeps future changes grounded in its actual operating boundaries.

12.5 Moving VMs Between Resource Groups, Subscriptions, and Regions

Three moves solve different problems

Moving a VM to another resource group changes its management organization. Moving it to another subscription changes its subscription context under the supported resource-move rules. Moving it to another region changes where the workload runs and generally requires a relocation or migration workflow. These are not variations of one simple location-field edit. The administrator must identify the intended result before choosing a procedure.

A resource group's location stores metadata for that group; it does not force all contained resources to run in that region. Moving a VM into a group whose metadata location differs does not relocate the VM's compute or disks. This distinction is essential in exam questions and real change requests. A request for geographic relocation cannot be satisfied by changing the resource-group container alone.

Administrative moves can preserve the running workload under supported conditions, but their effects still require planning. Resource identifiers change because they include subscription and resource-group information. Scripts, monitoring, policy, access assignments, and external references can depend on those identifiers. A move that leaves the application running can still disrupt administration or automation afterward.

Inventory the dependency graph

A VM references disks and network interfaces, and those interfaces reference subnet and network configuration. Public IPs, security groups, availability resources, identities, extensions, backup, and other services can add dependencies. The move procedure must account for the supported relationship among all relevant resources. Selecting only the compute object because it has the server's familiar name can leave the validation incomplete.

Some resources must move together, some can remain, and some configurations prevent the requested move. Current resource-type support and service-specific guidance should be checked before scheduling. A list of generic move support is a starting point, not a substitute for examining the actual VM deployment. Marketplace plans, encryption dependencies, and recovery integrations can introduce additional conditions.

The inventory should also include dependencies outside Azure Resource Manager's automatic validation. An external monitoring system may store the old resource ID. A script may target a hard-coded group. A budget or policy assignment may apply differently in the destination. These relationships can be invisible to a portal move wizard but still matter to the team operating the server.

A resource ID should therefore be treated as a configuration dependency rather than a permanent business identifier. Applications and operations tools that store it need an update path. Record both the old and new IDs in the move change record and use them to search managed automation references. Avoid exporting secrets while collecting this inventory; the reference relationship is the relevant evidence.

Subscription moves need destination readiness

Supported moves between subscriptions require compatible tenant context and the documented prerequisites. An ordinary resource move is not a mechanism for transferring a VM across Microsoft Entra tenants. If the business requirement includes tenant change, the migration scope is broader and needs a separate identity and service plan. Treating it as a routine subscription move risks missing fundamental dependencies.

The destination subscription should have the required resource providers registered, sufficient quota, appropriate policies, and authorized operators. Billing and governance consequences should be understood. A resource that complied with the source environment can encounter different allowed locations, SKU restrictions, or tag requirements in the destination. Validation should reveal those issues before the move window.

Role assignments deserve explicit review. Permissions inherited from the source group or subscription do not automatically express the intended access in the destination. Resource-scoped assignments and references can also require attention under the supported move behavior. The administrator should inventory and recreate or adjust required access deliberately rather than assuming that every person retains exactly the same rights.

Validate before committing

Azure provides move validation under the supported resource-management workflow. The administrator should use it to identify dependency and eligibility problems before the operation. A validation success is useful but does not prove every application or external automation dependency is ready. The full change plan combines platform validation with the organization's own checks.

1. State whether the goal is organization, subscription transfer, or regional relocation.
2. Inventory the VM, dependencies, external references, and operational integrations.
3. Verify current move support and source/destination prerequisites.
4. Prepare permissions, policy, quota, provider registration, and recovery evidence.
5. Run the supported validation and resolve specific failures.
6. Execute the move in the agreed window and monitor completion.
7. Update references and verify application, monitoring, backup, and operator access.
8. Retain the change record and remove obsolete configuration only after acceptance.

During a move, management operations can be restricted on involved scopes according to the service's behavior. The team should avoid overlapping deployments or unrelated changes that complicate the operation. A coordinated window makes failure evidence easier to interpret and reduces the chance that another process changes a dependency while it is being moved.

Regional relocation rebuilds an operating context

A VM's region is tied to its placement and regional resources. Relocation can use supported tools such as Azure Resource Mover or an appropriate replication and migration approach, depending on the scenario. The process prepares target resources, transfers or replicates data, tests the target, and performs a controlled cutover. It is closer to moving an application environment than rearranging resource-group membership.

The target region needs a compatible VM size, image, disk configuration, network, and capacity. Not every SKU or feature exists in every region or zone. Target addresses and names may differ, and external allowlists may need changes. The administrator should prepare these dependencies before transferring the live workload. A copied disk without a usable network and identity environment is not a completed relocation.

Data consistency and downtime need an explicit agreement. Replication can reduce the final transfer work but does not remove every cutover decision. The team must decide when to stop or redirect writes, how to verify the final state, and what conditions justify rollback. Running both source and target as independent writers without a conflict plan can damage application data.

Test without confusing source and target

A target test should isolate the relocated workload sufficiently to avoid accidental production side effects. A server booting successfully can still send notifications, process jobs, or connect to production dependencies. The test plan should identify those behaviors and control them. This is especially important when a copied configuration contains scheduled tasks or integration credentials.

The verification should include more than login. Test application requests, storage access, identity, required outbound connections, monitoring, and backup registration. Compare the target's effective configuration with the intended design rather than assuming all source settings transferred automatically. Regional network and service differences often appear only when the application exercises a specific dependency.

Cutover and cleanup are separate decisions

After the target passes its tests, the team performs the agreed cutover, such as changing a traffic destination or application configuration. DNS caching and existing connections can affect the transition. The source should remain in an appropriate preserved state until the rollback period ends, subject to security and cost requirements. Deleting it immediately after the first successful target response removes an important recovery option.

Duplicate resources can generate charges during migration. The plan should identify which source and target resources remain intentionally and when each can be removed. Old disks, public IPs, recovery artifacts, and temporary networks should be reviewed individually. A broad deletion of the original

group is safe only after every contained resource has been confirmed retired and no longer required for rollback.

Diagnose a rejected or incomplete move

A rejected administrative move may identify an unsupported resource type, missing dependency, policy restriction, or destination prerequisite. The administrator should address that specific issue and rerun validation. A regional relocation failure may instead concern replication, target capacity, network configuration, or application readiness. These are different investigations, even when both requests were described casually as moving a VM.

The successful result is a workload operating in the intended administrative or geographic context with its access, monitoring, and recovery responsibilities intact. Resource location, resource identity, and application state must all be checked. That approach prevents a completed management operation from being mistaken for a completed migration.

Guided lab

Objective and preparation

Deploy a small Linux VM with no public management endpoint, inspect its configuration, attach an empty managed data disk, and deallocate it. This independent exercise verifies management-plane configuration only. It deliberately does not run guest commands, install packages, format disks, or claim application readiness. Guest-agent operations and package installation require a separately designed explicit outbound path.

Prerequisites are Azure CLI in Bash, an authenticated subscription, a supported region and VM SKU, sufficient compute quota, and permission to create compute, disk, and network resources in the dedicated group. Supply an existing SSH public-key file or create a dedicated lab key. Protect the private key. The operator needs management permissions; this exercise does not grant guest access through the network.

The VM allocation, OS disk, data disk, and any optional diagnostic storage can generate charges. The isolated VNet, subnet, NIC, and NSG also form part of the resource inventory even where their basic configuration has no separate charge. Do not add NAT Gateway, Bastion, or a public IP merely to complete this management-only lab. Those services would add scope and potential charges. Budgets do not cap spending.

1. Select supported parameters and create an isolated network. Use a VM size and image that are compatible and available to the subscription. The image alias below is an illustrative Ubuntu choice; verify it resolves to a supported image for the selected architecture.

```
location="<supported-region>"
size="<available-compatible-small-vm-size>"
group="rg-compass-ch12"
vm="vm-compass-ch12"
az group create --name "$group" --location "$location"
az network vnet create --resource-group "$group" --name vnet-compass-ch12 \
  --address-prefixes 10.12.0.0/16 --subnet-name workload \
  --subnet-prefixes 10.12.1.0/24
az network nsg create --resource-group "$group" --name nsg-compass-ch12
az network nic create --resource-group "$group" --name nic-compass-ch12 \
  --vnet-name vnet-compass-ch12 --subnet workload \
  --network-security-group nsg-compass-ch12
```

No public IP or inbound management allow rule is created. The subnet's implicit outbound behavior is not relied upon. This lab does not use that path for verification.

2. Create the VM using the existing NIC and an SSH public key.

```
az vm create --resource-group "$group" --name "$vm" --location "$location" \
  --nics nic-compass-ch12 --image Ubuntu2204 --size "$size" \
  --admin-username compassadmin --ssh-key-values "<path-to-public-key>" \
  --storage-sku StandardSSD_LRS
```

The expected outcome is a successfully provisioned VM and managed OS disk. If the request fails because of SKU, image, policy, security-type, or capacity constraints, inspect the error and choose a compatible approved configuration. Do not report the VM as created until the operation actually succeeds.

3. Inspect the configured and runtime state.

```
az vm show --resource-group "$group" --name "$vm" \
  --query
'{size:hardwareProfile.vmSize,osDisk:storageProfile.osDisk.name,nics:networkProfile.networkInterfa
ces[].id}'
az vm get-instance-view --resource-group "$group" --name "$vm" \
  --query instanceView.statuses --output table
```

The output should identify the selected size, OS disk, NIC reference, and reported instance states. It does not establish that an application is installed or that SSH is reachable. The absence of a public path is intentional.

4. Create and attach a small empty data disk, then inspect its attachment.

```
az disk create --resource-group "$group" --name disk-compass-ch12-data \
  --location "$location" --size-gb 8 --sku StandardSSD_LRS
az vm disk attach --resource-group "$group" --vm-name "$vm" \
  --name disk-compass-ch12-data --lun 0
az vm show --resource-group "$group" --name "$vm" \
  --query storageProfile.dataDisks --output json
```

The expected evidence is a data-disk reference with the chosen logical unit number. No filesystem is created. A guest administrator would need a controlled access path and an exact device-identification procedure before initializing it.

5. For a controlled troubleshooting exercise, compare the attached disk's Azure size with the absence of any guest filesystem evidence in this lab. Explain why attachment does not prove formatting or mounting. Then inspect resize options without applying a disruptive change.

```
az vm list-vm-resize-options --resource-group "$group" --name "$vm" --output table
az vm deallocate --resource-group "$group" --name "$vm"
az vm get-instance-view --resource-group "$group" --name "$vm" \
  --query instanceView.statuses --output table
```

The final state should report deallocation after the operation completes. Disk charges can continue. Do not confuse deallocation with deletion of the server's storage.

6. Inspect and delete the dedicated resource group to remove the lab resources.

```
az resource list --resource-group "$group" --output table
az group delete --name "$group" --yes
az group exists --name "$group"
```

The existence check should return `false` after completion. Remove only a dedicated lab SSH key if one was created and is no longer needed; preserve any preexisting key. If policy or locks prevent cleanup, record remaining resources and resolve only the lab-specific restriction. No live Azure execution was performed during authoring.

Chapter synthesis

A VM's operating result depends on its image, size, disks, network, identities, and maintenance process. Disk changes require guest verification, resizing requires disruption planning, host encryption requires compatibility checks, and movement requires dependency review. The administrator should distinguish desired configuration from allocated resources and working application behavior. Each change is complete only when the relevant outcome has been verified.

Review questions

12-Q1. A VM reports Running. Which conclusion is justified?

- **A.** The compute power state is running; application health still requires verification
- **B.** Every application dependency is healthy
- **C.** All data disks are formatted
- **D.** The VM is protected by a successful backup

12-Q2. An attached managed data disk appears in Azure, but the application cannot use a filesystem on it. What should be checked next?

- **A.** Whether a public DNS zone exists
- **B.** The guest's device identification, partition, filesystem, and mount state
- **C.** Only the resource group's tags
- **D.** Whether the VM has the largest available size

12-Q3. A VM needs more capacity. Which statement about resizing a running VM is accurate?

- **A.** It is always nondisruptive
- **B.** It always changes the region
- **C.** It causes a restart, and some changes require deallocation
- **D.** It automatically fixes every application bottleneck

12-Q4. Which additional boundary does encryption at host address beyond persisted managed-disk encryption?

- **A.** All application authorization decisions
- **B.** Every backup retention requirement
- **C.** Public DNS delegation
- **D.** Supported host-local data and disk-cache paths

12-Q5. A VM is moved to a resource group whose metadata location is another region. What happens to the VM's compute region from that move alone?

- **A.** It remains in its existing region
- **B.** It automatically relocates to the group's region
- **C.** It is converted to a container
- **D.** Its disks are automatically archived

12-Q6. A VM cannot install packages after deployment into a new network. No explicit outbound design was configured. What is a relevant first investigation?

- **A.** Increase the DNS TTL for an unrelated zone
- **B.** Verify the VM's required explicit outbound path and destination access
- **C.** Delete every data disk
- **D.** Assign the local administrator subscription Owner

12-Q7. Which storage location should not hold the only copy of irreplaceable application data?

- **A.** A managed data disk with appropriate protection
- **B.** A suitable external persistent data service
- **C.** The VM's temporary local disk
- **D.** A verified backup location

12-Q8. What must be reviewed before a supported cross-subscription resource move?

- **A.** Only the VM's display name
- **B.** Only the guest wallpaper
- **C.** Whether every file has the same extension
- **D.** Tenant compatibility, dependencies, destination prerequisites, and access/reference changes

Further reading

- Virtual machine overview
- Managed disks
- VM sizes
- Resize a VM
- Encryption at host
- Resource move prerequisites
- Move support by resource type
- Regional VM relocation
- Explicit outbound connectivity

Chapter 13: Keeping Compute Available

Availability depends on where instances run, how work reaches them, and what happens when capacity disappears. Additional VMs can help, but only if the application can use them and the remaining environment continues to function. This chapter examines failure isolation, zones, scale sets, capacity rules, and diagnosis. Each topic is connected to observable service behavior rather than a resource count alone.

Learning objectives

- Explain fault domains and update domains in availability sets.
- Design a workload across zones without confusing placement with replication.
- Operate scale-set models and instances using the appropriate orchestration mode.
- Choose bounded scaling rules that reflect application demand.
- Diagnose allocation, health, configuration, and autoscale failures separately.

13.1 Understanding Availability Sets

Reduce correlated infrastructure failures

An availability set is a logical grouping that helps Azure distribute related VMs across supported fault and update domains. Its purpose is to reduce the chance that one underlying infrastructure event affects every instance together. It does not create additional VMs automatically or make an application understand how to use them. The administrator must deploy enough instances and configure the application to continue when one is unavailable.

A fault domain groups resources that can share a physical infrastructure dependency, such as power or networking. Placing instances across fault domains reduces exposure to one shared failure. An update domain groups instances for planned maintenance behavior. These concepts address different events: unplanned infrastructure failure and coordinated platform maintenance. They should not be treated as two names for the same placement unit.

The number of supported domains and related storage placement depends on the environment and service configuration. A deployment should use current regional guidance rather than memorizing a universal count. The important administrative question is how the intended instances are distributed and which failures can still affect them together. A count without that explanation provides little assurance.

More than one instance is necessary for useful redundancy

Putting one VM into an availability set does not create another application instance. If that VM stops, there is no second server to handle requests. A useful design deploys multiple compatible instances and provides a way for clients or work queues to use healthy capacity. The set influences placement; the application architecture determines whether that placement can preserve service.

For a web application, a load balancer can distribute connections among healthy servers. For a worker application, a queue can allow surviving workers to continue processing. A database cluster may require its own supported replication and quorum configuration. These are different application patterns. An availability set does not automatically supply the traffic distribution, data synchronization, or consensus behavior they need.

Capacity must be sufficient after a failure. Two instances each running near their limit can leave the remaining server overloaded when one disappears. The design should specify how much capacity must survive maintenance or failure and whether additional capacity can arrive quickly enough. Redundancy that exists only on a diagram may fail under the first realistic workload test.

Understand the protection boundary

Availability sets provide infrastructure isolation within their supported scope. They do not provide the same separation as deploying across availability zones. Shared datacenter-level dependencies can still affect multiple fault domains. The administrator should compare the failure requirement with the available placement options and use an availability set when its boundary fits the workload and environment.

An operating-system crash, bad application release, or common configuration error can affect every instance regardless of fault-domain placement. If all servers receive the same faulty script, their physical separation does not stop that script from failing. Reliable deployment and rollback remain necessary. Infrastructure diversity and software correctness are complementary controls.

Data dependencies can also collapse the apparent redundancy. Several web VMs may all depend on one unprotected database VM. Their availability set protects the web tier's placement while the single database remains the service's weak point. A failure review should trace the entire request path rather than stopping after counting front-end instances.

Plan placement before building the workload

Availability choices should be made during deployment planning. Changing an existing VM's placement relationship is not a routine cosmetic edit. Supported migration paths can evolve, and some scenarios require recreating or moving resources through a specific process. The administrator should check current guidance for the existing configuration instead of assuming a portal field can be changed safely in place.

Managed disks integrate with supported availability-set storage isolation. The disk placement relationship matters because a compute instance without its persistent storage is not useful. The administrator should use compatible managed configurations and inspect regional support. Mixing old unmanaged-disk assumptions into a modern deployment can produce misleading conclusions about resilience and management responsibility.

The set and its instances should have meaningful names and ownership metadata. An inventory should show which application tier each instance serves and why they belong together. Grouping unrelated servers merely because they share a project can obscure the intended failure model. The logical group should represent resources whose simultaneous loss would affect the same service requirement.

Maintenance is an application event

Planned platform maintenance is one reason update domains matter, but the organization's own maintenance also needs coordination. Patching every VM at the same time can defeat the availability architecture. The operations process should withdraw one portion of capacity, perform maintenance, verify health, and continue only when enough capacity remains. A placement feature cannot prevent a script from restarting all servers simultaneously.

Application readiness should be checked before returning an instance to service. A VM that has rebooted may still be loading data, warming caches, or completing configuration. The health signal used by traffic distribution should represent readiness for the assigned work. Otherwise, the load balancer can send requests to a machine whose power state is running but whose application is not ready.

The maintenance procedure should also define what happens when verification fails. Continuing the rollout to every remaining instance can convert a partial problem into a complete outage. A pause condition, rollback method, and responsible operator should be documented. This operational discipline supplies the software counterpart to the infrastructure isolation.

Concept	Main concern	What must be added
Fault domain	Shared infrastructure failure	Enough usable instances and protected dependencies
Update domain	Planned platform maintenance grouping	Coordinated application maintenance and health checks
Availability set	Logical placement relationship	Traffic or work distribution
Application resilience	Continued useful service	Data design, capacity, testing, and recovery

Inspect actual placement and state

The administrator should inspect instance information and the configured availability-set relationship after deployment. A template containing an availability-set resource does not prove that each intended VM references it. Resource IDs and instance properties provide stronger evidence. The inspection should also verify that the expected number of VMs exists and that their applications are healthy.

A failed allocation can occur when a requested size or placement combination is unavailable. The response should identify whether the issue is quota, capacity, or compatibility. Repeatedly attempting a different size without checking the workload's requirements can produce inconsistent instances. A supported alternative should be reviewed and recorded rather than selected randomly until deployment succeeds.

Test continued service, not only placement

A controlled test can stop or withdraw one test instance while requests continue through the intended service entry point. The observer records which requests succeed, how health changes, and whether

remaining capacity stays within its operating limits. This is not a simulation of every physical failure, but it tests a useful part of the application's response to losing an instance.

The test should avoid irreversible data changes and should run in an environment authorized for disruption. Stopping a production database simply to demonstrate availability is not an appropriate beginner exercise. A dedicated lab service with known state allows the team to learn the method safely. The book's management-only lab later in this chapter does not claim to perform this application test.

1. Identify the service entry point and the minimum acceptable capacity.
2. Verify that multiple instances are healthy before the test.
3. Withdraw one selected test instance through a controlled operation.
4. Observe client behavior and the health of the remaining instances.
5. Restore the instance and verify readiness before returning traffic.
6. Record failures, recovery time observed, and the limits of the test.

Choose a current design deliberately

Microsoft recommends evaluating Flexible Virtual Machine Scale Sets for modern availability scenarios because they combine placement and broader management capabilities. Availability sets remain important for understanding existing environments and AZ-104 administration. The choice should be based on workload needs, supported features, and migration constraints rather than automatically repeating the oldest deployment pattern.

An availability set is useful when its contribution is described precisely: it reduces selected correlated infrastructure risks for the VMs actually placed in it. The application still needs compatible instances, usable surviving capacity, reliable data, and controlled maintenance. That explanation gives an administrator a sound basis for deciding whether the design is sufficient or whether zonal distribution is required.

13.2 Distributing Workloads Across Zones

A zone is a failure-isolation choice

Availability zones are separate physical locations within a supported Azure region, with independent infrastructure such as power, cooling, and networking. Deploying appropriate resources across zones can reduce the effect of a zone-level failure. The administrator should distinguish a zonal resource, placed in a selected zone, from a zone-redundant service that manages distribution across zones under its own design.

A single zonal VM remains one VM. Selecting a zone does not duplicate its operating system or application into every other zone. To maintain service through a zone loss, the application generally needs usable instances and dependencies in other zones or another supported recovery design. A resource labeled Zone 1 is evidence of placement, not evidence of automatic failover.

A regional VM without an explicit zonal placement should not be assumed immune to zone failure. Its underlying location is not a substitute for a deliberate multi-zone architecture. The administrator should use supported availability options and inspect the actual configuration. An unspecified placement can leave important failure behavior unknown.

Design the whole application across the boundary

The web tier, data tier, traffic distribution, identity dependencies, and network appliances all contribute to availability. Distributing only the web servers can leave the application dependent on a single-zone database or firewall. A zone review should trace every required dependency and ask what remains usable if the selected zone disappears. This produces a more realistic design than a diagram containing several zone labels.

Data replication is especially important. A VM in another zone needs access to the correct application state, not merely the same software image. The data service or application must provide a supported replication and consistency model. Sharing a copied disk from yesterday may support recovery under one requirement but cannot satisfy a promise of immediately current data without further design.

Load distribution also needs to align with the zone design. A suitable frontend and health configuration should direct work to healthy instances that remain available. DNS alone does not provide transport-level backend health for an ordinary static address record. The administrator should use the appropriate service and understand how its own zone configuration affects the failure path.

Capacity must survive the expected loss

If the design assumes continued service after losing one zone, the remaining zones need enough capacity to handle the required workload or a reliable way to add it. An architecture that spreads all normal capacity evenly but has no headroom can become overloaded during failure. The capacity plan should state the degraded-service target rather than assuming normal performance continues automatically.

Allocating more instances during a regional incident can encounter capacity constraints. The organization should evaluate whether the recovery requirement needs capacity assurance or preallocated headroom. Autoscale rules cannot force unavailable capacity into existence. A resilience plan should distinguish the desired instance count from a guaranteed ability to allocate it during stress.

The cost of extra capacity should be compared with the business impact of reduced service. This is an explicit tradeoff. A low-cost development environment may accept rebuilding after a zone event, while a critical transaction service may need warm capacity across zones. The administrator's role is to make the tradeoff visible and implement the agreed requirement accurately.

A capacity calculation should name the assumed failure. If three equal groups normally share the work and one group disappears, the surviving groups must absorb the lost share or the service must reduce demand. That arithmetic should be tested with the actual application, since throughput does not always scale linearly. Shared databases, connection pools, and external rate limits can become the new bottleneck when surviving instances work harder. The plan should identify the acceptable degraded behavior before an outage forces an improvised choice.

Check regional and zonal support

Not every VM size, disk offering, or related service supports every region and zone combination. The deployment should verify the chosen SKU and dependencies together. A region that offers availability zones does not guarantee that an arbitrary specialized VM size is available in each one. Quota and current capacity remain additional considerations.

Logical zone labels are subscription-scoped representations of physical placement. When coordinating resources across subscriptions, the administrator should use supported zone mapping guidance rather than assuming that the same number always denotes the same physical location. This matters when latency or failure alignment depends on actual placement. A convenient label is not a universal physical address.

Zonal placement changes can require a supported move or redeployment process. Current preview capabilities may offer additional recovery behavior for eligible VMs, but they should be clearly distinguished from the generally available core design. This book's baseline relies on explicit multi-instance architecture rather than promising that every standalone VM can be moved automatically during a zone outage.

Account for latency and data-transfer behavior

Separating instances across zones can change communication patterns. Applications with frequent synchronous exchanges may be sensitive to latency, and cross-zone traffic can have pricing implications under current terms. The administrator should measure the application and review current pricing rather than assume physical separation is free in every dimension. Resilience and performance must be evaluated together.

Placement should not force an application into an unsupported data-consistency pattern. A database cluster may have explicit latency and quorum requirements. A file-based application may assume local locking behavior. The team should use the application's supported multi-zone architecture rather than distributing machines first and discovering compatibility problems later.

Plan maintenance with failure headroom

A multi-zone deployment still needs coordinated software updates. If one zone is unavailable, the maintenance process may need to pause rather than remove additional healthy capacity. The rollout plan should define a minimum healthy population and stop conditions. This combines infrastructure status with application health before each maintenance step.

Health checks should observe useful service readiness in each zone. A shared dependency failing everywhere can make all backends unhealthy simultaneously. That signal is valuable because it reveals a common dependency, but blindly recreating every VM can make diagnosis harder. The administrator should correlate health across zones and identify what those instances share.

Design claim	Evidence needed
Instances span zones	Actual placement of the intended VMs
Service survives one-zone loss	Healthy remaining path and sufficient capacity
Data remains usable	Supported replication and tested consistency behavior
Recovery is fast enough	Observed procedure and dependency readiness

A worked zonal design review

A reporting service uses application instances in two zones and a data service with an appropriate zone-redundant configuration. A Standard Load Balancer frontend is selected consistently with the intended availability design. The application stores no irreplaceable state on an instance's temporary disk. Its deployment process can create an equivalent replacement from a recorded image and configuration.

The review then asks what happens if either application zone becomes unavailable. It checks remaining compute capacity, frontend behavior, data access, outbound dependencies, and operational access. It also checks whether the monitoring system will distinguish application failure from expected loss of one instance group. These questions turn the placement arrangement into a testable service requirement.

Test a controlled approximation honestly

A lab can withdraw all application instances representing one zone and observe the remaining service path. This tests application distribution and capacity behavior under that simulated loss. It does not reproduce every consequence of an actual zone outage, such as correlated failures in unrelated dependencies. The test report should describe that limitation rather than claiming complete disaster validation.

The team should record the baseline, withdrawal action, observed client errors, remaining capacity, and restoration sequence. If the service fails, the evidence should identify whether traffic distribution, data, identity, or capacity was the limiting factor. A failed test is useful when it exposes a specific design gap before a real event.

Zonal resilience comes from a coordinated application design, not from selecting several placement values independently. Each resource's failure boundary, data behavior, and capacity role must fit the whole. An administrator who can explain those relationships can distinguish genuine continued service from a collection of geographically separated machines that still depend on one fragile component.

13.3 Operating Virtual Machine Scale Sets

Manage a population through a model

A Virtual Machine Scale Set manages a group of VM instances under a defined orchestration model. It can support consistent deployment, capacity changes, and availability placement. The scale set does not make an arbitrary application horizontally scalable by itself. The workload must tolerate multiple instances, instance replacement, and the way traffic or jobs are distributed among them.

The model describes desired configuration, including relevant image, size, disk, network, and extension settings. Instances are the actual machines created or managed under that model. Those two states can differ temporarily or intentionally. An administrator should inspect both the desired model and the instances rather than assume a successful model update immediately changed every running machine.

This distinction resembles infrastructure code and deployed resources. The model expresses intent, while operation status and instance properties show what exists. A scale set can have the desired capacity recorded while some instances are still provisioning or have failed. Counting the configured target is not the same as counting healthy application workers.

Choose orchestration mode deliberately

Flexible orchestration works with standard Azure VM resources and provides broad management flexibility for supported workloads. Microsoft recommends it for many modern scale-set scenarios. Uniform orchestration uses a common VM profile and manages scale-set child instances through its specific APIs. Both can support scaling, but their resource identities, management operations, and feature integrations differ.

The mode is selected when the scale set is created and should not be treated as a reversible display preference. Migration between models needs a supported plan. An administrator should identify the workload's management needs, instance variation, backup and recovery integrations, and automation before choosing. A tutorial using one mode is not a universal recommendation for every application.

Tool commands must match the mode. Uniform instances are commonly inspected through scale-set VM operations, while Flexible instances use standard VM resource management where appropriate. A script that assumes one resource shape can fail against the other. The operations documentation should state the orchestration mode near its commands rather than burying that assumption in a deployment parameter.

Consistency is a deployment property

A common image and configuration help ensure that newly added instances can serve the same workload. Manual changes made only inside one VM create drift. When that instance is replaced, the change can disappear; when another instance receives traffic, behavior can differ. A scale set works best when the application can be reproduced through an image, configuration process, and controlled external dependencies.

Extensions and configuration scripts can install or configure software, but they introduce dependencies such as package sources, secrets, identity permissions, and outbound connectivity. A script that succeeds on the first instance can fail later if a package version disappears or a token expires. Repeatability requires explicit versions and a reliable configuration source, not merely reusing the same command text.

The application should keep durable state in an appropriate external or replicated system. Local instance state can be lost during replacement or scale-in. A worker should acknowledge a job only after the required durable result is stored, according to its application protocol. A web server should not make irreplaceable customer state depend on one disposable instance's memory or temporary disk.

Instance health is more than provisioning success

Provisioning status describes whether infrastructure and configuration operations completed. Application health describes whether an instance can accept useful work. A VM can pass provisioning while its process crashes or its database connection fails. Scale-set operations that depend on health need an appropriate supported health signal, such as application health reporting or a relevant load-balancer probe, according to the selected feature and mode.

The health contract should be designed with the same care described in Chapter 8. A signal that always returns success allows broken instances to remain in service. A signal that treats every optional

dependency as fatal can remove too much capacity. The administrator should work with the application team to define readiness and verify it during startup, normal operation, and failure.

Automatic repairs can respond to unhealthy instances under supported configurations. They should not be enabled as a substitute for understanding persistent failures. If every new instance receives the same bad configuration, repeated repairs can create a cycle of replacements without restoring service. Repair logs and application evidence should be reviewed together to identify whether the cause is instance-specific or common to the model.

Model updates need an upgrade strategy

Upgrade policies determine how supported model changes reach instances. Manual, automatic, and rolling behaviors have different operational consequences and feature requirements. The administrator should select a supported policy that fits the application's availability needs. A manual policy requires explicit instance updates, while a rolling approach needs meaningful health checks and sensible limits on unavailable capacity.

Automatic upgrade mode can disrupt all instances together; its name does not imply a gradual rollout. Rolling upgrades use batches. Flexible orchestration requires the Application Health extension for rolling upgrades, whereas Uniform supports that extension or an appropriate load-balancer health probe. These prerequisites should be verified before a model update is scheduled.

A model change should be tested on a controlled subset before broad application. The rollout needs stop conditions, verification, and rollback. If the first updated instances fail health checks, continuing to update the remaining healthy population is dangerous. The operations process should preserve enough known-good capacity to serve the application while the problem is investigated.

Image updates and application updates also need clear version tracking. A machine built from a newer image may differ in packages, drivers, or security settings even when the application version is unchanged. The administrator should record both image and configuration versions. A reproducible rollback needs those details, not just the name of the scale set.

A useful release record also identifies the health test and the acceptance threshold used during deployment. Otherwise, an operator can reproduce the software while overlooking a changed readiness path. Configuration consistency includes the evidence used to decide that an instance is ready to receive work.

Scale-in is a deletion decision

Increasing instance count adds capacity, subject to quota and allocation. Decreasing it removes instances according to the supported scale-in behavior. The application must prepare for that removal. Long-running jobs, active sessions, and local temporary results need a draining or retry strategy. A scale-in rule that saves compute cost while losing valid work does not meet the service requirement.

Instance protection can control selected scale-set actions for particular instances under supported configurations. It is useful for special maintenance or operational needs, but it should be tracked. Protecting too many instances can prevent the group from reaching the intended lower capacity. The administrator should know why each exception exists and when it can be removed.

Operational object	Question to ask
Scale-set model	What configuration and capacity are intended?
Instance inventory	Which machines actually exist?
Provisioning status	Which infrastructure operations completed?
Application health	Which instances can accept useful work?
Upgrade and repair policy	How are changes and failures handled?

A practical operating workflow

1. Select the orchestration mode from application and management requirements.
2. Record a reproducible image and configuration model.
3. Define placement, network access, explicit outbound connectivity, and identity.
4. Create a small initial population and validate each instance.
5. Connect the application to its traffic or work-distribution mechanism.
6. Configure health, upgrade, repair, and scaling behavior with clear limits.
7. Test scale-out, scale-in, and a controlled failed-instance case.
8. Monitor model drift, failed operations, and application outcomes.

The test should use the actual workload path. A count changing from two to three is useful infrastructure evidence, but it does not demonstrate that the third instance received configuration or became healthy. The observer should follow the instance from allocation through readiness. This reveals delays that matter when autoscale is expected to react to demand.

Diagnose inconsistent instances

If one instance behaves differently, compare image version, applied model, extension status, environment configuration, and application logs. A manual change or incomplete upgrade may explain the difference. Reimaging the instance can remove the symptom, but the team should first capture evidence and determine whether the model will reproduce the same fault. Otherwise, the replacement can fail identically.

A scale set is valuable because it makes a population manageable, not because it removes every server responsibility. The administrator still needs consistent configuration, correct state handling, useful health, and controlled changes. With those elements in place, the group can add and remove capacity predictably while preserving the application's operating contract.

13.4 Matching Capacity to Demand

Scaling is a feedback process

Autoscale adjusts the number of supported resource instances according to configured conditions or schedules. For VM scale sets, it is a horizontal capacity mechanism: it adds or removes VMs rather than changing each VM's size. The administrator chooses the signals, thresholds, limits, and timing. Those choices form a feedback process whose behavior should be tested against the application.

The first question is what resource pressure actually limits useful work. CPU can be a good signal for a compute-bound service. Queue depth can indicate work waiting for a worker pool. Request latency can reveal a service-level symptom but may also reflect a shared database that more web servers cannot repair. A useful metric should have a plausible relationship to the capacity being changed.

Some metrics require guest or application instrumentation. A metric named in a design document does not exist automatically in Azure Monitor with the required dimensions and sampling. The administrator should confirm the source, namespace, dimensions, aggregation, and freshness before using it in a rule. Missing telemetry can make a well-written threshold ineffective.

Set a safe operating envelope

Minimum, maximum, and default counts define part of the scale setting's behavior. The minimum should preserve the required baseline and availability posture. The maximum limits expansion but must still be achievable within quota and capacity. When metrics cannot be read, a population below the default can expand to that count; a population already above it is not reduced merely to reach the default. This fallback should preserve safe service capacity during a telemetry problem.

A maximum count is not a complete spending cap. Other resources continue generating charges, and a large permitted population can still be expensive. Budgets provide notifications rather than automatically preventing every allocation. The organization should align capacity bounds, cost monitoring, and the service requirement. Setting the maximum too low can protect a narrow compute estimate while allowing an avoidable application outage.

The minimum also affects fault tolerance. A group that scales down to one instance loses the benefits of having several ready workers, even if it can later expand. The workload may accept that in a development environment. A production requirement for continuous service during one-instance failure may need a higher baseline. The rule should reflect the agreed service level.

Thresholds require aggregation and time

A metric's instantaneous value differs from its value aggregated over a time window. A short spike may not justify adding capacity if the workload normally absorbs it. A sustained high value can indicate a real need. The rule's aggregation and duration determine which behavior triggers action. The administrator should read the complete rule rather than focusing only on a percentage threshold.

Aggregation across instances matters as well. An average can hide one overloaded instance, while a maximum can react to an isolated hot spot. The appropriate choice depends on how work is distributed.

If one client is pinned to a single backend, adding instances may not reduce that backend's load. The scaling design and traffic-distribution design must be reviewed together.

With multiple rules, any satisfied scale-out rule can request expansion, while all relevant scale-in rules must be satisfied before contraction. For example, low CPU alone should not permit removal when another configured scale-in condition still indicates pressure. These directional rules prevent a reader from assuming that every rule is combined identically.

Queue-based metrics require attention to their evaluation semantics, including whether a value is considered relative to the current instance count. A queue of many small jobs differs from the same number of long-running jobs. Processing time and arrival rate help interpret backlog. A rule should be validated with representative work rather than a synthetic count that has no relationship to the application's tasks.

Avoid oscillation between expansion and contraction

Scale-out and scale-in thresholds should leave enough separation to avoid repeatedly adding and removing capacity near one boundary. This separation is often called hysteresis. Cooldown periods also give the system time to stabilize after an action. Without these controls, new capacity can reduce the metric briefly, trigger immediate removal, and recreate the original pressure.

Startup time influences those settings. A VM may take time to allocate, boot, obtain configuration, and become ready. A rule that evaluates too aggressively during that period can request more instances before the previous ones contribute. The administrator should measure time to useful capacity and account for it in thresholds, schedules, and maximum growth behavior.

The measurement should include successful work, not only a boot timestamp. A worker that has started its process but has not loaded required reference data may contribute no useful throughput yet. Record the interval from scale request to the first completed representative operation. Repeating that measurement under realistic conditions provides a better basis for cooldown and scheduled preparation than assuming every instance starts as quickly as the first empty lab VM.

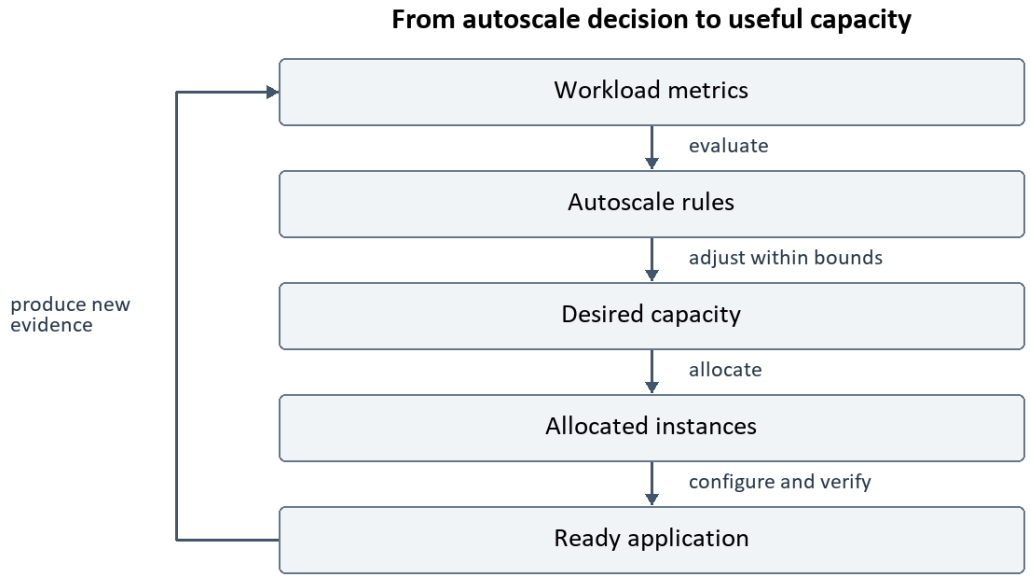


Figure 13.1 Autoscale decisions and useful capacity

A successful scaling decision is only the start. Allocation, configuration, and readiness must succeed before new instances perform useful work; metrics then describe the changed workload.

Scale-in should be conservative enough to preserve active work and required headroom. The application may need to drain sessions or finish jobs before removal. A low CPU value can occur while a worker waits on an essential external operation, so it does not always mean the instance is disposable. The selected signal should reflect the application's actual lifecycle.

Use schedules for predictable demand

If demand follows a known schedule, scheduled capacity can prepare the population before the increase arrives. This can avoid waiting for reactive rules to observe pressure and provision new instances. The schedule should specify its time zone and account for relevant daylight-saving behavior. A business opening time interpreted in the wrong zone can create capacity at the wrong moment every day.

Metric-based rules can still help within scheduled profiles where supported. The administrator should understand which profile is active and how its bounds and rules interact. Overlapping assumptions can make an otherwise reasonable policy confusing. A runbook should explain the expected profile during ordinary hours, peak periods, and exceptional events.

Predictive capabilities can assist suitable cyclical workloads, but they also depend on data and supported conditions. They should not be presented as a substitute for an explicit capacity requirement or tested application startup. A forecast can improve timing while still being constrained by quota, availability, and configuration failures.

Read and test a policy as a complete system

Policy element	Operational question
Metric and dimensions	Does this signal describe the workload being scaled?
Aggregation and window	Is the pressure sustained and interpreted correctly?
Action	How much capacity changes per decision?
Cooldown	Can new capacity become useful before another decision?
Bounds	Is the permitted range safe, sufficient, and affordable?
Scale-in behavior	Can removed instances finish or transfer their work?

1. Measure a representative baseline and time to readiness.
2. Choose a signal that corresponds to the limiting resource.
3. Set minimum, maximum, and default counts from the service requirement.
4. Define separated expansion and contraction conditions with suitable timing.
5. Confirm the metric is present and the intended profile is active.
6. Apply controlled load and observe evaluation, allocation, and readiness.
7. Remove the load and verify safe scale-in without losing work.
8. Review cost, latency, failures, and unexpected oscillation.

The test record should include each stage. A rule can evaluate true while allocation fails. Allocation can succeed while the new VM never becomes healthy. A healthy VM can remain unused because traffic distribution is wrong. Observing only the final count misses these distinctions and can lead to an incorrect diagnosis of the autoscale engine.

Manual changes interact with the controller

An operator who manually changes capacity while autoscale remains enabled is acting alongside an active controller. The service can bring the population back within the configured bounds or respond to its current rules. A manual change should therefore include a deliberate decision about the active policy. Otherwise, the team may interpret the controller's expected correction as unexplained interference.

During an incident, temporary bounds or disabled automation may be appropriate under the operations process. The change should have an owner and an end condition. Leaving emergency capacity or disabled scale-in indefinitely can create cost and reliability problems later. The incident record should specify when normal control is restored and how that restoration is verified.

Good scaling matches useful capacity to actual demand within a defined operating envelope. It depends on telemetry, application behavior, allocation, and timing together. The administrator's task is to make those relationships explicit and measurable so that the system expands for a reason, contracts safely, and reveals why it could not do either.

13.5 Diagnosing Availability and Scaling Problems

Separate the controller from the workload

A complaint that autoscale is broken can describe several different failures. The metric may be missing, the rule may not qualify, the active profile may impose a limit, allocation may fail, or the new instances may be unhealthy. An availability complaint can similarly involve placement, traffic distribution, capacity, or a shared dependency. The first task is to identify which stage failed.

The administrator should record the desired capacity, actual instance count, healthy instance count, active autoscale profile, and relevant time window. These values answer different questions. A desired count of four with two provisioned instances points toward allocation or provisioning. Four provisioned instances with one healthy application worker points toward guest configuration or application health. A rule that never requested four requires a different investigation.

The timeline should include recent deployments, policy changes, metric observations, scale actions, and application errors. Azure activity records and autoscale evaluation history can help explain what the controller attempted. Instance and application logs explain what happened afterward. Correlating those sources avoids blaming the wrong layer because its error was the most visible.

Check whether the rule actually qualifies

Begin with the active profile and its schedule. A correct rule in an inactive profile cannot cause the expected action. Verify the time zone, current bounds, and any temporary incident changes. A maximum equal to the current count prevents further expansion even when the workload is busy. That is a policy outcome, not a mysterious allocation failure.

Next inspect the exact metric used by the rule. Confirm its resource, namespace, dimensions, aggregation, and data freshness. A chart displayed with different aggregation can look above the threshold while the rule's own evaluation remains below it. The administrator should reproduce the rule's interpretation rather than compare it with a visually convenient but different graph.

Duration and cooldown also affect action timing. A brief spike may not satisfy the required window, and a recent action may still be inside cooldown. These conditions should be checked before changing thresholds. Reducing every delay during an incident can create oscillation or excessive allocation requests without addressing the real bottleneck.

Distinguish a decision from an allocation

If evaluation requests more capacity, inspect the resulting operation. Quota, SKU availability, zone constraints, policy, or resource limits can prevent allocation. The error should identify a concrete cause or category. A controller cannot create capacity outside subscription permission or service availability. Increasing the threshold does not repair a failed allocation request that already occurred.

The desired model can remain different from the actual population while an operation fails or is incomplete. An administrator should not report success based solely on the model's count. List instances and inspect provisioning state. If a subset failed, record which instances and what operation failed. This evidence supports a targeted correction rather than deleting the entire group.

New instances also need supporting resources and configuration. An exhausted subnet address range, inaccessible image, failed extension, or unavailable secret can interrupt provisioning. These dependencies can fail only at scale-out time even when existing instances continue working. A system that has not added an instance for months may contain a stale deployment path that ordinary traffic never exercises.

Follow a new instance to readiness

After allocation, the VM must boot, configure its application, reach required dependencies, and report healthy. Each stage can have its own failure. Explicit outbound connectivity is especially important for package downloads and guest-agent communication. A VM that lacks the necessary egress path can appear allocated while its configuration remains incomplete.

The administrator should inspect extension results and guest logs through an authorized path. Repeatedly reimaging the instance without preserving failure output can remove the most useful evidence. If the same failure occurs across new instances, examine the common image, script, permissions, and network configuration. If it affects one instance only, compare its state with a known-good peer.

A load balancer's probe can reveal readiness problems but cannot diagnose every cause. A wrong path, blocked probe source, missing listener, or failed dependency can all mark an instance unhealthy. Direct application tests, probe-path tests, and effective security checks help narrow the reason. A VM power state of running does not contradict an unhealthy application probe.

Diagnose scale-in with equal care

When capacity does not decrease, inspect the minimum bound, active profile, metric evaluation, cooldown, and instance-protection settings. A minimum of three means the group should not contract below three through that policy. Protected instances can also constrain removal. The administrator should identify why those controls exist before changing them.

If scale-in occurs but users lose work, the problem may be the application's draining or state model rather than the controller's decision. Long-running jobs need supported retry or checkpoint behavior. Sessions may need external state or graceful termination. The capacity rule should be adjusted only after understanding how instance removal interacts with the workload.

Oscillation suggests a feedback problem. Thresholds too close together, inappropriate aggregation, insufficient cooldown, or a signal unrelated to useful demand can cause repeated expansion and contraction. The timeline should show whether new capacity had time to become ready and whether its arrival changed the metric as expected. That evidence guides a specific policy correction.

Observation	Most useful next check
No scale action recorded	Active profile, metric, threshold, duration, bounds
Action requested but instances absent	Allocation error, quota, capacity, policy, dependencies

Observation	Most useful next check
Instances exist but are unhealthy	Image, extensions, egress, application and probe evidence
Instances healthy but overloaded unevenly	Traffic distribution and session behavior
Scale-in loses work	Draining, state, retries, and removal policy
Population oscillates	Threshold separation, cooldown, and metric suitability

Investigate correlated failures across placement boundaries

If instances in several fault domains or zones fail at the same time, look for shared dependencies and common changes. A bad image rollout, expired credential, DNS problem, or unavailable database can affect physically separated VMs. Their placement may be working as designed while the application still fails. The investigation should not assume that every simultaneous failure proves Azure ignored the placement configuration.

Conversely, if one zone's instances fail while others remain healthy, the team should inspect zone-related service health and the application's remaining capacity. A resilient design may continue serving with reduced headroom. The operator should avoid initiating unrelated maintenance that removes more capacity. Incident actions should preserve the surviving path while restoring the failed portion.

Worked incident: new capacity never becomes useful

A worker pool receives a growing queue and requests two additional instances. Azure allocates both, but queue processing does not improve. The instances report failed configuration extensions because their subnet has no approved outbound path to the package source. Existing workers still run software installed earlier. The failure appears only when fresh capacity is needed.

The repair is to establish the intended supported deployment dependency or use a prepared image that removes that particular download requirement, according to the design. Raising the maximum count would only create more unusable instances. After correction, the team creates a fresh test instance and follows it through readiness and successful job processing. This verifies the repaired path rather than only the edited network setting.

Close with an explanation that can prevent recurrence

An incident record should distinguish the trigger, controller decision, allocation outcome, instance configuration, and application effect. The corrective action should include a validation test for the failure discovered. A stale image dependency suggests regular fresh-instance tests. A wrong metric dimension suggests policy review. A state-loss incident suggests an application drain or retry improvement.

Correlate timestamps and resource identifiers across those records. A scale-set operation identifier, an affected VM identifier, and an application request identifier describe different levels of the same event. Preserve their relationship without treating one as a substitute for the others. This allows another

administrator to establish whether the failing request reached a newly created instance, an existing instance, or no backend at all, and prevents unrelated errors from becoming a misleading explanation.

Availability and scaling become manageable when each claim has observable evidence. Placement shows isolation, instance inventory shows allocation, health shows readiness, and application tests show useful service. The administrator combines those observations into a causal explanation, makes a targeted correction, and verifies that the workload can complete its required operations.

Guided lab

Objective and limits

Create a small Uniform scale set, inspect its model and instances, and change its capacity manually. Uniform is selected explicitly here to demonstrate its distinct instance-management API. Flexible orchestration is the recommended evaluation starting point for many new production designs; the lab's selection is an instructional boundary rather than a blanket production recommendation.

This exercise has no application, load balancer, public IP, guest command, or package installation. It demonstrates allocation and model inspection, not application availability, zonal failover, or metric-triggered autoscale. No implicit outbound connectivity is required for its management-plane observations. A real application would need its own explicit outbound, health, and traffic-distribution design.

Prerequisites are Azure CLI in Bash, a supported region and compatible small VM SKU, sufficient quota for three instances, an SSH public-key file, and permissions to create compute and network resources. Compute instances and their OS disks generate charges; optional diagnostic resources can add charges. The temporary increase to three instances increases cost. Review prices first and remove the group afterward. Budgets do not cap expenditure.

1. Create the isolated network and NSG.

```
location="<supported-region>"
size="<available-compatible-small-vm-size>"
group="rg-compass-ch13"
setname="vmss-compass-ch13"
az group create --name "$group" --location "$location"
az network vnet create --resource-group "$group" --name vnet-compass-ch13 \
  --address-prefixes 10.13.0.0/16 --subnet-name workload \
  --subnet-prefixes 10.13.1.0/24
az network nsg create --resource-group "$group" --name nsg-compass-ch13
```

2. Create two Uniform instances with manual model-upgrade policy and no public frontend.

```
az vmss create --resource-group "$group" --name "$setname" \
  --location "$location" --orchestration-mode Uniform \
  --image Ubuntu2204 --vm-sku "$size" --instance-count 2 \
  --admin-username compassadmin --ssh-key-values "<path-to-public-key>" \
  --vnet-name vnet-compass-ch13 --subnet workload --nsg nsg-compass-ch13 \
  --load-balancer "" --public-ip-address "" \
  --upgrade-policy-mode Manual --storage-sku StandardSSD_LRS
```

Verify the installed CLI accepts the empty-string arguments used to suppress automatic frontend creation. If the local shell or tool version handles them differently, use the portal or an explicit deployment definition to preserve the stated no-public-frontend design. Inspect the resource inventory rather than assuming a default produced the intended result.

3. Read the desired model and list the actual instances.

```
az vmss show --resource-group "$group" --name "$setname" \
  --query '{mode:orchestrationMode,capacity:sku.capacity,upgrade:upgradePolicy.mode}'
az vmss list-instances --resource-group "$group" --name "$setname" \
  --query '[].{id:instanceId,name:name,state:provisioningState}' --output table
```

The expected model reports Uniform orchestration and a capacity of two. The instance list should eventually show two successfully provisioned instances. If fewer appear or one fails, inspect the operation error and instance status before continuing. The target count alone is not proof of successful allocation.

4. Increase capacity and observe the new instance.

```
az vmss scale --resource-group "$group" --name "$setname" --new-capacity 3
az vmss list-instances --resource-group "$group" --name "$setname" \
  --query '[].{id:instanceId,name:name,state:provisioningState}' --output table
```

Record the new instance identifier and the observed provisioning outcome. Do not describe it as a healthy application server because no application health contract exists in this lab. A production test would continue through configuration and readiness.

5. Return capacity to two and inspect which instance remains or is removed according to the service's supported behavior.

```
az vmss scale --resource-group "$group" --name "$setname" --new-capacity 2
az vmss list-instances --resource-group "$group" --name "$setname" --output table
```

As a controlled reasoning variation, compare a hypothetical autoscale profile with minimum two and maximum two against one with minimum two and maximum four. Explain why the first cannot expand through that policy even if a CPU condition is true. Do not claim that manually changing capacity tested autoscale evaluation.

6. In the scale set's scaling interface, inspect the available autoscale configuration fields without enabling an untested production policy. Draft a lab rule using CPU above 70 percent for a sustained ten-minute window to add one instance, and CPU below 35 percent for ten minutes to remove one, with a five-minute cooldown and bounds of two to four. These values are teaching examples, not workload recommendations. Record which metric aggregation the interface uses and what a real controlled-load test would need to verify.

7. Inspect and remove the dedicated resources.

```
az resource list --resource-group "$group" --output table
az group delete --name "$group" --yes
az group exists --name "$group"
```

The final check should return `false` after deletion completes. Preserve any preexisting SSH key; delete only a dedicated disposable lab key if appropriate. Record remaining resources if a lock or policy blocks cleanup and resolve only the lab-specific issue. The manuscript's commands and expected results were not executed against a live Azure subscription.

Chapter synthesis

Placement, capacity, and application behavior are separate parts of availability. Fault domains and zones reduce selected shared risks. Scale sets manage an instance population, while autoscale responds to defined signals within bounds. Useful service still depends on consistent configuration, durable state,

working dependencies, and meaningful health. Diagnosis follows those layers from the controller's decision to the application's result.

Review questions

13-Q1. Two application VMs are correctly placed in an availability set. What additional condition is needed for the service to continue when either VM becomes unavailable?

- **A.** The application must distribute work to usable remaining capacity and preserve required state.
- **B.** Both VMs must use the same private IP address.
- **C.** The availability set must replicate the application's database automatically.
- **D.** The VMs must be stored in different resource groups.

13-Q2. An administrator creates one VM explicitly in Zone 1. Which statement accurately describes this choice?

- **A.** Azure has created identical VMs in every supported zone.
- **B.** The VM has a selected zonal placement, but the application still has only one instance.
- **C.** The VM's data is automatically synchronously replicated to a second region.
- **D.** The application is guaranteed to remain available during a zone outage.

13-Q3. A scale-set model reports a desired capacity of four, but only three instances have successfully provisioned. What should the administrator conclude?

- **A.** All four application instances are healthy because the model update succeeded.
- **B.** The missing instance must have been removed by a load-balancer probe.
- **C.** Desired capacity differs from actual allocation, so the failed or pending operation needs investigation.
- **D.** The scale set must be recreated immediately without collecting evidence.

13-Q4. Which statement distinguishes Flexible and Uniform scale-set orchestration?

- **A.** Flexible orchestration cannot support autoscaling.
- **B.** Uniform and Flexible instances always have identical resource APIs.
- **C.** The orchestration mode is an ordinary display setting that can be switched freely.
- **D.** Flexible uses standard VM resources, while Uniform manages scale-set child instances through mode-specific operations.

13-Q5. A VM scale set has an active autoscale profile with minimum two and maximum two. It currently has two instances, and its CPU scale-out condition is satisfied. What is the expected constraint?

- **A.** The profile cannot increase the count beyond its maximum of two.
- **B.** The maximum is ignored whenever a CPU rule is true.
- **C.** Each existing VM automatically changes to a larger size.
- **D.** The budget automatically adds a third instance.

13-Q6. A worker population can add VMs quickly, but each new worker needs several minutes before completing a job. Which measurement best informs cooldown and scheduled preparation?

- **A.** Only the time at which the model's desired count changes.
- **B.** The time from the scale request to a representative operation completed by the new capacity.
- **C.** The length of the resource group's display name.
- **D.** Only the time at which a VM first reports a running power state.

13-Q7. Existing workers process jobs, but newly scaled-out workers fail while downloading startup packages. Their subnet has no intended explicit outbound path. What is the most appropriate next investigation?

- **A.** Raise the maximum instance count until some downloads happen to work.
- **B.** Assume the old workers prove the current startup dependency path is healthy.
- **C.** Inspect startup errors and establish the supported outbound or prepared-image design needed for fresh instances.
- **D.** Delete application logs before replacing every worker.

13-Q8. A worker keeps an unfinished job's only result on local temporary storage. What must be addressed before allowing automated scale-in?

- **A.** Rename the scale set to include the word production.
- **B.** Add a DNS record for the temporary disk.
- **C.** Assume a successful infrastructure deletion means the job completed.
- **D.** Provide durable state and a supported drain, completion, or retry strategy for work on removed instances.

Further reading

- Availability sets
- VM availability options
- Scale sets overview
- Orchestration modes
- Upgrade policies
- Application health extension
- Azure Monitor autoscale
- Autoscale practices
- Autoscale troubleshooting
- Scale-set CLI

Chapter 14: Making Deployments Repeatable

A working environment is easier to maintain when its intended configuration can be inspected before anyone changes it. The virtual machines and networks developed earlier in the book provide the context for this chapter. Infrastructure as code records their configuration in files that can be reviewed, compared, and deployed again. The central skill is reading what a deployment will do, including the consequences of apparently small edits.

Learning objectives

- Read the structure and expressions of an Azure Resource Manager template.
- Recognize parameters, resources, dependencies, modules, and outputs in Bicep.
- Change a deployment without confusing resource identity with its settings.
- Distinguish compilation, validation, what-if analysis, deployment, and operational verification.
- Use exported templates and decompilation as starting points that require review.

14.1 Reading ARM Templates

A description of the desired environment

An Azure Resource Manager template, usually shortened to ARM template, is a JSON document describing Azure resources and their intended properties. Resource Manager evaluates the document and sends operations to the relevant resource providers. A storage account is handled by the storage provider; a virtual network is handled by the network provider. The template brings those definitions together without turning them into a series of portal clicks. Understanding that division helps explain why one deployment can contain operations with different validation rules and different completion times.

The term declarative describes this approach. A template declares what should exist. An imperative script instead describes a sequence of commands to perform. Both approaches are useful, and an administrator commonly uses them together. A script might select a subscription, invoke a deployment, and check the resulting resources. The template describes those resources. A declaration does not remove the need for judgment: an incorrect desired state can be reproduced very reliably. The first responsibility of the reviewer is therefore to understand the intended outcome, rather than simply confirm that the JSON parses.

Most introductory templates contain a schema declaration, a content version, parameters, variables, resources, and outputs. The schema describes the shape of the document and its deployment scope. The content version is the author's version label; changing it does not automatically create a new Azure resource. Parameters accept values supplied at deployment time. Variables calculate or collect values used inside the template. Resources are the declarations Azure applies. Outputs return selected results after deployment, such as a resource ID or endpoint name. Optional sections and newer language features exist, so these headings are a reading framework rather than a list of mandatory fields.

Reading a resource declaration

Start with a resource's type, API version, name, and scope. Together they explain what Azure object the declaration addresses. The resource type contains a provider namespace and a type path. For example, `Microsoft.Storage/storageAccounts` identifies a storage account resource. The API version chooses a version of the provider's management contract. It is not the version of the Azure CLI, the operating system, or the storage data itself. A template can legitimately use different API versions for different resource types. The reviewer should check each version against the features and properties used in that declaration.

The name becomes part of the resource ID, which also includes the subscription and relevant scope. A changed name usually addresses a different resource instead of renaming the existing object. This distinction is easy to miss when editing a parameter called `environmentName`. If several resource names are calculated from that parameter, changing it can result in several new resources. Azure services also impose their own naming requirements. A string that satisfies JSON syntax might still violate a storage account's naming rules or conflict with an existing global name. Template validity and resource name availability are separate questions.

Location identifies the region for resource types that have a regional location. The location of a resource group stores management information and does not force all resources within that group into the same region. A template often uses the group's location as a convenient default, but that is an authoring choice. A resource requiring a different location needs an explicit value. Service availability, capacity, and policy can constrain the choice. Reading a location parameter means tracing its actual use, not assuming that a single parameter automatically governs every resource in a large file.

The `properties` object holds settings specific to the resource type. Other fields, such as `sku`, `kind`, and `tags`, can sit outside it. Their positions are determined by the resource provider's schema. A storage account's SKU describes a performance and redundancy option; tags identify management metadata. Placing a valid-looking property at the wrong level is still an error. When the structure becomes unfamiliar, consult the resource type reference rather than copying a neighboring declaration. Similar service names do not imply identical management schemas.

Following values through the file

Expressions in an ARM template are commonly enclosed in square brackets. They can reference parameters, variables, functions, resource IDs, and runtime information. A value such as `[parameters('location')]` means that Azure evaluates an expression rather than using those characters as a literal region. A reviewer should trace expressions until the operational value is clear. If a network address comes from a parameter file, the review is incomplete without that parameter file. The template and its deployment inputs form the actual proposed configuration.

Consider this small resource fragment. It illustrates the reading pattern and is not a complete deployment document.

```
{
  "type": "Microsoft.Storage/storageAccounts",
  "apiVersion": "2023-05-01",
  "name": "[parameters('storageName')]",
  "location": "[parameters('location')]",
  "sku": { "name": "Standard_LRS" },
  "kind": "StorageV2",
  "properties": {
    "supportsHttpsTrafficOnly": true,
    "minimumTlsVersion": "TLS1_2",
    "allowBlobPublicAccess": false
  }
}
```

The declaration addresses a general-purpose storage account with locally redundant standard storage. It requires secure transfer, establishes a minimum TLS setting, and prevents enabling anonymous blob access. Those properties describe selected management controls. They do not assign an administrator permission to read blobs, create private connectivity, or upload content. This separation matters when evaluating a deployment result. A successfully created account can still deny a data operation because its network path or data-plane authorization has not been configured for the caller.

Dependencies and deployment order

A template may contain explicit dependencies between resources. A dependency tells Resource Manager that one operation must wait for another. It is not a permanent network connection or an access permission. For example, a virtual machine can require a network interface that in turn references a subnet. Those relationships create an ordering requirement during deployment. Adding dependencies everywhere makes the file harder to read and can unnecessarily serialize independent work. Conversely, removing a required dependency can expose an operation to a resource that does not yet exist or is not ready for the requested reference.

Reading dependencies is especially useful when troubleshooting partial failures. A failed parent operation can prevent dependent resources from being attempted. Several failures in the deployment record may therefore have a single underlying cause. Begin with the first meaningful provider error and inspect its target resource. A network policy denial, unavailable SKU, or naming conflict can explain a much longer list of unsuccessful downstream operations. The final deployment status summarizes the result; the individual operation details explain how that result arose.

Outputs deserve the same scrutiny as inputs. An output is convenient for passing a resource ID to another command, but deployment outputs can be viewed by people with access to the deployment record. Secrets should not be placed in ordinary outputs. Marking an input secure protects specific handling of that input; it does not make every later use of the value safe. An output that combines a secret with an ordinary string can still reveal it. Prefer returning identifiers and nonsecret configuration, and obtain sensitive values through the service's intended authorization path when needed.

Deployment history is also a management record with a scope and access controls. It can help explain which values were supplied and which operations failed, but it should not be treated as an unlimited archive of every configuration change. Portal edits and other management operations can occur outside a particular template deployment. When investigating an unexpected value, compare the deployment record with current resource properties and the relevant activity records. That comparison separates what the deployment requested from what the resource currently reports. It also prevents a misleading conclusion that the most recent visible template must have caused every difference in the environment.

A useful review sequence

Read a template from the deployment boundary inward. First identify the target scope and the subscription or tenant in which the operation will run. Next examine resource names, types, locations, and the actual parameter values. Then trace dependencies and settings that affect security, cost, availability, or data retention. Finally inspect outputs and compare the intended resources with the existing environment. This sequence turns a dense JSON document into a set of operational decisions. It also exposes missing information before any provider receives a write request.

For example, a requested change might say that a test network needs one additional subnet. A review that checks only the new subnet object can miss the fact that the virtual network declaration no longer contains an existing subnet. The reviewer must inspect the complete desired configuration of the parent resource and its children. Infrastructure as code makes a change visible, but the visible unit can be larger than the sentence in the change request. The important question is which resource properties will be reapplied when the deployment executes.

14.2 Understanding Bicep Files

A clearer authoring language for the same platform

Bicep expresses Azure resource declarations with a syntax designed for infrastructure authoring. It compiles into an ARM template, and Resource Manager remains the deployment engine. Moving from JSON to Bicep changes how the definition is written and reviewed. It does not bypass role assignments, Azure Policy, resource locks, regional availability, or resource-provider validation. An administrator who understands the management hierarchy already understands an important part of Bicep's operating context. The language describes requests made within that hierarchy.

A basic Bicep file contains parameters, variables, resource declarations, and outputs. It can also contain modules and other language constructs. Parameters declare inputs with types such as strings, integers, booleans, arrays, and objects. Variables hold calculated values. A resource declaration assigns a symbolic name to an Azure resource type and API version. The symbolic name is an identifier used within the file; it is separate from the Azure resource's actual `name` property. Confusing those names can cause unnecessary concern when a harmless refactor changes a local symbol but leaves resource identity unchanged.

The distinction is visible in a declaration beginning `resource trainingStorage 'Microsoft.Storage/storageAccounts@2023-05-01'`. Here `trainingStorage` is the local symbolic name. A later `name: storageName` property sets the Azure name from a parameter. Renaming the symbol while updating its references does not rename the storage account. Changing `storageName` can address a different account. Reviewers should inspect the emitted resource name and scope whenever a proposed change appears to rename a resource, rather than judging the effect from the local identifier alone.

Parameters and their contracts

Parameter declarations can include default values and decorators. A decorator adds information or constraints to a declaration. Descriptions explain how a value should be chosen; allowed-value lists and

length constraints help reject unsuitable inputs. These controls are useful because a parameter is an interface between the author and the person deploying the file. A parameter named `location` is clearer when its description states that service availability must be checked. A storage name parameter is safer when it documents its uniqueness requirement and expected characters.

Constraints should reflect what the deployment actually supports. An allowed-value list containing three SKUs means the author intentionally supports those options; it does not establish that Azure offers only three choices. A broad string parameter can expose more flexibility while shifting validation to the provider. Neither approach is universally superior. A learning template benefits from a narrow, explained interface. An organization-wide module may require a broader interface with additional checks. Good parameter design reduces accidental variation without hiding an assumption that future maintainers need to understand.

Defaults also deserve scrutiny. A location default of `resourceGroup().location` is convenient for a resource-group deployment. It should not be interpreted as an instruction to relocate existing resources when their group metadata changes. A default name based on a deterministic function can help repeated deployments address the same object. A random name produced outside the template can make every run create something new. Both designs can be legitimate, but a repeatable lab needs a stable naming decision until cleanup is complete. The deployer's input remains part of the desired state.

References and implicit ordering

Bicep references resources through their symbolic names. An expression using `trainingStorage.id` obtains that resource's ID. A property reference can establish an implicit dependency, allowing the compiler to create the appropriate ordering information. This makes the relationship easier to see than a manually written resource ID scattered through a JSON document. It also reduces the chance of changing a name in one place and forgetting a reference elsewhere. A clear reference communicates both where a value comes from and why two declarations are related.

An `existing` resource declaration describes a resource that is already present. It enables the file to refer to that resource without declaring that it should be created by this deployment. Existing declarations are useful for shared networks, monitoring workspaces, or other independently managed infrastructure. They do not verify ownership or automatically grant access. If the referenced resource is absent, outside the expected scope, or inaccessible to an operation, a later request can still fail. The author should document the dependency and ensure that validation includes the intended target environment.

Explicit `dependsOn` remains available, but it should solve an actual ordering requirement. It is usually unnecessary when a direct symbolic reference already establishes the dependency. A common beginner mistake is to put every resource after the first one in a long dependency chain. That creates a sequence that looks reassuring but obscures which resources genuinely depend on each other. Independent storage and virtual network resources can ordinarily deploy in parallel. A resource that consumes another resource's output requires a more specific relationship.

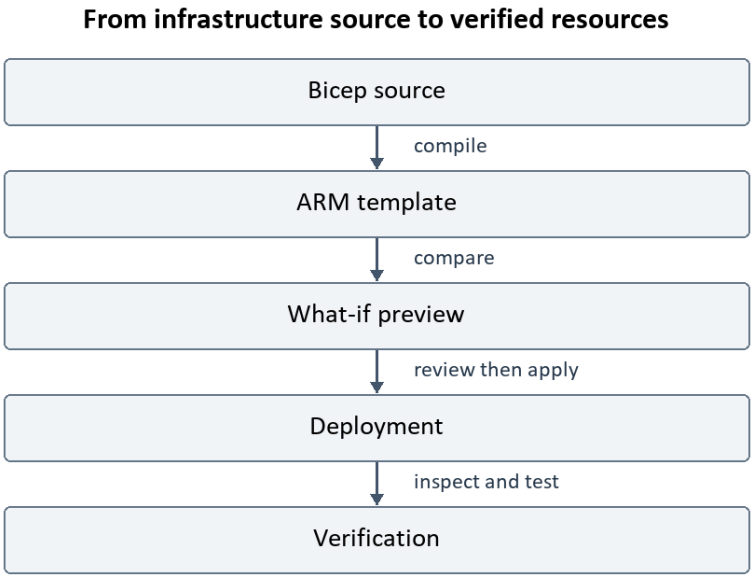


Figure 14.1 From source definition to verified resources

Compilation checks the source representation. What-if predicts changes in the target scope. Deployment applies the definition, and verification checks the resulting resources and intended task.

The diagram separates authoring, compilation, preview, deployment, and verification. Each stage answers a different question. A well-formed Bicep file can compile while still requesting an unavailable regional SKU. A deployment can succeed while the application using its resources fails a connectivity test. Treating those stages separately produces a more accurate record of what has actually been checked.

Modules and boundaries

A Bicep module is a reusable deployment definition called from another Bicep file. Modules can group resources around a meaningful responsibility, such as a network or a monitoring foundation. Inputs pass values into the module, and outputs return selected information to the caller. A module's interface should make its dependencies visible. For example, a module that creates a network interface in a shared subnet needs the subnet's identity, not a vague parameter that silently selects a production network.

Small modules are useful only when their boundaries make the system easier to understand. Dividing every single resource into a separate file can force the reader to open many files to understand one change. Conversely, a huge file that combines unrelated lifecycles becomes difficult to review safely. A practical boundary groups resources that are created, changed, and retired together. The network definition might have a different owner and lifecycle from an application deployment. Passing a subnet ID between them makes that separation explicit while preserving the connection.

Deployment scope matters when modules cross resource-group or subscription boundaries. The parent file's target scope and a module's assigned scope must fit the operations being performed. Having write access in one resource group does not grant permission in another. The caller also needs the relevant

deployment permissions at each destination. A module that worked in a personal sandbox can therefore fail in an organization with separated responsibilities. The error should lead to a review of the intended identity and scope, rather than an immediate request for unrestricted access.

Reading expressions without guessing

Bicep uses expressions for string interpolation, conditional values, loops, and functions. A string containing `${location}` substitutes a value; a boolean condition can determine whether a resource declaration is included. Loops can create several similar declarations from an input collection. These features reduce repetitive text, but they also mean that the visible source is a recipe for a set of resources rather than a simple one-to-one inventory. Review the evaluated names and count when changing a loop's input or a conditional resource.

Conditions require attention during updates. Setting a deployment condition to false means the resource is omitted from that deployment definition. In an ordinary incremental deployment, omission does not generally delete an existing independent resource. A user expecting the toggle to remove an optional resource may therefore be disappointed. Cleanup should be an explicit lifecycle decision. This is particularly important for resource locks: turning off the conditional declaration in a template is not a reliable procedure for removing a lock already created by an earlier incremental deployment.

Type checking and editor assistance improve feedback but do not replace provider documentation. A property can be syntactically acceptable while its combination with another setting is invalid. Some restrictions depend on the selected SKU, region, or existing state. Compiler warnings should be investigated, especially where a resource type lacks detailed type information. Suppressing a warning might be justified after checking the provider contract, but it should not become a way to hide uncertainty. A clean compile is one piece of evidence about the file.

The administrator's reading goal

After reading a Bicep file, an administrator should be able to describe its resources, locations, inputs, important security choices, dependencies, and outputs in ordinary language. That description is more valuable than memorizing every punctuation mark. It provides a basis for reviewing a proposed change and comparing the deployment result with the request. The next section applies this reading skill to modifications, where a single parameter can affect names, capacity, permissions, or several resources at once.

14.3 Changing Parameters and Resource Definitions

Treat a parameter change as a configuration change

Parameters are often described as a convenient way to reuse a template. They also determine the exact resources and settings a deployment addresses. Changing a parameter can be as consequential as changing the resource declaration itself. A parameter controlling a tag is different from a parameter controlling a virtual machine's name, storage redundancy, or subnet address prefix. The administrator should classify the effect before deployment. Is the change metadata, capacity, identity, connectivity, access, or data protection? That classification suggests the verification and recovery work the change needs.

A parameter file stores deployment inputs separately from the main definition. This supports the same resource design in development and test while keeping environment-specific names and locations explicit. The parameter file should be versioned and reviewed with the definition it belongs to. Keeping it separate does not make it less important. An approved template deployed with an unreviewed parameter file is a different proposal. Review records should identify both files and any command-line overrides, because the effective input is what Resource Manager receives.

Sensitive input requires special treatment. Passwords, tokens, and connection strings should not be committed as ordinary parameter values. Secure parameters influence how Resource Manager handles sensitive inputs, but surrounding tools and scripts can still expose them through command history, logs, or debugging output. A file containing an actual secret is still sensitive even if the receiving parameter is marked secure. Prefer managed identity and service-native authentication when appropriate. Where a secret remains necessary, use a supported protected reference or a secure delivery mechanism and limit its visibility throughout the workflow.

Understand the boundary of a deployment

An incremental deployment preserves resources outside the template's declared set, but it does not mean that every omitted property of an included resource is preserved. Resource definitions describe the intended resource configuration. Reapplying a declaration can reset properties that are not supplied, depending on the resource provider and property. This distinction explains why a seemingly narrow template can have broader effects than expected. A reviewer must inspect the complete declaration of a resource being updated and consult the provider's update behavior where the outcome is uncertain.

For a virtual network, subnet definitions are a particularly important example. A network declaration needs to represent the subnets it owns consistently. Maintaining some subnets inline and others through separate deployment definitions can create conflicting views of the same parent configuration. A later redeployment may attempt to remove or change a subnet that another deployment expects. The problem is an unclear ownership boundary. Decide which definition owns the virtual network's subnet collection and avoid having independent files silently compete over it.

Complete deployment mode has different deletion behavior from incremental mode and is not the default choice for this book. Microsoft recommends incremental deployments and points authors to deployment stacks for managed resource deletion. Existing complete-mode material still appears in older examples, so administrators should recognize what it means before using it. A lab that creates two resources does not justify applying a deletion-oriented mode to a resource group containing unrelated work. Lifecycle management needs a deliberately selected boundary and a preview of what would be affected.

Change	Main question before deployment	Useful verification
Tag value	Which resources receive the updated metadata?	Inspect resulting tags
Resource name	Does the change address a new resource?	Compare full resource IDs

Change	Main question before deployment	Useful verification
SKU or size	Is the target supported and is interruption expected?	Inspect capacity and workload health
Network rule	Which paths become allowed or denied?	Test an intended and an unintended path
Retention setting	Which data can expire under the new rule?	Inspect policy scope and recovery behavior

Make changes easy to explain

A useful change set has a clear purpose and a small enough boundary to review. Changing naming conventions, resource locations, security rules, and deployment tooling at once makes unexpected results harder to diagnose. Related changes should remain together when they must be applied consistently, but unrelated cleanup can wait. For example, adding one storage firewall rule should not also rename every parameter. The functional difference should be visible in a review without forcing the reviewer to separate it from extensive formatting changes.

Comments should explain a decision or a constraint that is not obvious from the syntax. A comment stating that a storage account is a storage account adds little. A comment explaining why a sample uses locally redundant storage, or why a particular network range must not overlap another network, helps a future reader. Comments also become stale, so they need review when the configuration changes. A contradictory comment can mislead an administrator who is trying to understand an unfamiliar deployment under time pressure.

Defaults are not a substitute for environment-specific decisions. A sample can reasonably default an optional lock to false so its cleanup procedure remains simple. A production deployment may require that lock, but enabling it changes the permissions and removal steps needed later. The person changing the default should trace those consequences through the lab or runbook. An interface is complete when it explains both how to turn a feature on and how to manage its resulting state. Otherwise, an apparently helpful switch becomes a source of operational surprise.

Work through a concrete modification

Suppose the companion file creates a virtual network and one subnet. The original subnet prefix is `10.44.1.0/24` within `10.44.0.0/16`. A request adds a second subnet for a future workload. The administrator first checks that the new range fits within the virtual network and does not overlap an existing subnet. The change then adds the second subnet to the authoritative subnet collection. It does not alter the virtual network's resource name, because the request concerns a new subnet inside the existing network.

Next, the administrator compares the new definition with the existing environment. If an additional subnet was created manually after the original deployment, the file may already be out of date. The proposed change must resolve that drift before redeployment. Either the file should incorporate the manually created subnet, or its ownership should be moved into a clearly separate supported pattern.

Ignoring it because it is absent from the source does not make it irrelevant. Actual state and intended state need to be reconciled before the update is considered safe.

A similar review applies to storage settings. Changing a network access control to deny traffic can be technically valid while breaking a data transfer job. The deployment author should identify clients, authentication methods, and network routes before making the change. A resource management operation can still succeed while a blob read fails afterward. Verification should therefore include a representative data request from the intended client. The review must distinguish a denied request caused by network controls from one caused by missing data-plane authorization.

Drift and emergency changes

Configuration drift is a difference between the maintained definition and the actual environment. Drift can come from a portal edit, a separate deployment, an automated policy action, or a provider change. It is not always evidence of careless administration. Emergency repairs sometimes need immediate action through the most reliable available tool. What matters afterward is whether the intended configuration is updated or the emergency change is deliberately reversed. Leaving an undocumented difference means a future deployment can unexpectedly restore an earlier setting.

An effective incident record includes the resource ID, changed property, previous value when available, new value, reason, and verification result. That record gives the infrastructure author enough information to reconcile the source. It should not contain passwords or unrelated personal data. A small team can use a straightforward change log; a larger organization can connect the same information to its formal review process. The underlying requirement is continuity between the environment that exists and the files that claim to describe it.

Define recovery before applying the change

Reverting a file does not guarantee that every resource change can be undone. Some properties are immutable, some updates require replacement, and deleted data may need a separate recovery mechanism. A previous template can restore configuration only where the service supports the requested transition. For a change affecting data or availability, identify the recovery path before deployment. That can include a backup, a maintenance window, a retained earlier application release, or a tested alternative route. Infrastructure history is valuable evidence, but it is not itself a backup of every resource's contents.

The practical standard is that another administrator can read the proposed difference and explain its intended effect, affected resources, verification method, and recovery limits. Achieving that standard does not require a long document for every tag update. It requires detail proportional to the consequence of being wrong. Once the change is understandable, compilation and service-side checks can examine it more precisely. Those checks provide additional evidence rather than transferring responsibility away from the author.

14.4 Validating and Running Deployments

Separate the checks

Deployment confidence comes from several checks that answer different questions. Local compilation asks whether the Bicep source can be translated into a valid deployment template. Template validation checks the submitted definition and, depending on the selected validation level and provider behavior, additional deployment prerequisites. What-if compares a proposed deployment with the target environment and predicts changes. The deployment operation requests the actual changes. Operational verification then checks whether the resulting resources support the intended task. Calling all of these activities testing conceals important differences in what has been established.

Local compilation is useful early because it catches language errors without creating Azure resources. A missing brace, an invalid property name recognized by the type system, or an incorrect type can be corrected before any service request. Compilation also produces the ARM representation that Resource Manager will receive. The administrator can inspect that output when learning how a Bicep expression becomes a template expression. Generated JSON should normally be treated as a build artifact when Bicep is the source of truth, so edits are made in the source rather than split across two competing definitions.

```
az bicep build --file companion/ch14-deploy.bicep
```

This command requires the appropriate Azure CLI and Bicep tooling. A successful local build is evidence about compilation only. It does not demonstrate that the selected subscription has quota, the selected region supports a resource option, or the signed-in identity can deploy the resources. The distinction is especially important when reviewing a file built on a developer's machine and later deployed by a different identity. The deployment identity and environment need their own checks.

Establish context explicitly

Before running a service-side operation, inspect the active account and subscription. A familiar resource-group name can exist in more than one subscription. Commands that depend on an implicit current context are therefore easy to direct at the wrong environment. Use an explicit subscription identifier in a script or select and confirm the intended context before continuing. Record nonsecret resource IDs rather than relying only on friendly names. The deployment's scope is part of its input and should be visible in the execution record.

Permissions must cover both the deployment operation and the resources it creates or changes. Contributor access is commonly sufficient for ordinary resource deployments within its assigned scope, but it does not automatically permit role assignments. Locks also have their own authorization requirements. A template containing a role assignment or a lock can therefore need permissions beyond those needed for its network and storage resources. The correct response to an authorization failure is to identify the denied action and its scope. Granting unrestricted access without understanding the request is a poor diagnostic method.

Azure Policy can deny a configuration even when role-based access control permits the write. Resource provider registration, regional constraints, and service quotas can also prevent deployment. These controls answer different questions: permission concerns who may act; policy concerns what configurations are allowed; capacity concerns what the environment can currently support. The error

details often identify which control applied. Keeping these distinctions in mind prevents an administrator from trying to repair a policy denial by repeatedly changing role assignments.

Preview the proposed result

What-if is a preview, not a write operation on the proposed resources. It is particularly useful for noticing a new resource name, an unexpected deletion, or a property change outside the intended request. Its output can also contain limitations or unresolved expressions. Those messages require attention. A clean-looking summary does not justify assuming that an unanalyzed resource is unchanged. Use a current supported tool version, read diagnostics, and investigate important differences before deployment. If the result cannot fully resolve a critical part of the design, obtain another form of evidence for that part.

```
az deployment group what-if \
  --resource-group rg-compass-ch14 \
  --template-file companion/ch14-deploy.bicep \
  --parameters @companion/ch14-parameters.json
```

The resource group must already exist for this resource-group deployment. The parameter file needs a region and a globally available storage account name selected for the exercise. What-if uses Azure and requires suitable access; it is different from local compilation. Store a reviewed summary of material proposed changes when a deployment is significant. That record makes it possible to compare what was expected with the operation that later occurred, especially when a parameter value changes between review and execution.

Run the deployment and inspect operations

After the proposal is understood, use a descriptive deployment name and apply the intended files. A deployment name identifies the deployment record, not the resources themselves. Reusing or changing that name does not determine whether a storage account is replaced; the resource declarations determine that. A useful name connects the operation to the change being made. For the lab, a short name such as `compass-ch14-initial` is sufficient. In a larger environment, a release identifier or change reference can make deployment history easier to follow.

```
az deployment group create \
  --name compass-ch14-initial \
  --resource-group rg-compass-ch14 \
  --template-file companion/ch14-deploy.bicep \
  --parameters @companion/ch14-parameters.json
```

The command requests real resources and can result in charges. The output should be checked for the provisioning result and relevant outputs. If it fails, inspect the deployment's operations rather than rerunning the command immediately. Some resources may have succeeded before another failed. Repeating the same deployment can be appropriate after the underlying problem is corrected, but the existing partial state should first be understood. A failed overall deployment is not proof that nothing was created.

In PowerShell, `Test-AzResourceGroupDeployment` and `New-AzResourceGroupDeployment` provide corresponding validation and deployment workflows. Their parameter syntax differs from Azure CLI, so examples should remain clearly separated by shell. A command copied from Bash can fail in PowerShell because continuation characters, quoting, or variable syntax differ. Such failures occur before Azure

processes the requested resource definition. Check the shell error first instead of treating every command failure as a cloud service problem.

Deployment names and correlation information help connect the command to service-side evidence. Preserve the relevant operation identifier when seeking assistance or reviewing a failure. Avoid copying an entire debug log into a broadly shared document without examining it, because diagnostic output can contain environment details or sensitive inputs from surrounding tooling. A concise record of scope, deployment name, failed resource, provider error, and corrective action is often more useful than a large unfiltered transcript. The evidence should be sufficient for another administrator to reproduce the reasoning without exposing information unrelated to the problem.

A quota failure illustrates why rerunning without investigation is ineffective. If the requested capacity exceeds the subscription's available quota in a region, a syntactically correct template remains unable to deploy there. The administrator can choose an approved smaller request, use another supported region where the design permits it, or follow the organization's quota process. Changing the API version at random does not resolve the capacity decision. The chosen correction should update the deployment inputs and its review record, so the successful run still corresponds to an understood proposal.

Verify the resources and the task

Verification starts with the management plane. Confirm the expected resource IDs, locations, types, and important properties. Inspect the subnet collection and storage network controls in the companion exercise. Confirm that the optional lock is absent when disabled. Then verify a representative operational outcome if the design includes one. A template that creates a storage account alone does not establish that a particular user can read its data. That additional task needs the appropriate role and allowed network path, which should be tested separately when included in the design.

Redeploying an unchanged definition is a useful learning exercise. The expected outcome is that the same intended resources remain, rather than a second set appearing under new names. This demonstrates the role of stable resource identity. It does not promise that every provider operation is free of side effects or that all resources are perpetually reconciled in the background. Resource Manager applies a deployment when requested. It is not automatically watching the file on a laptop and correcting every later portal change.

Finally, record what was actually verified. A statement such as "compiled locally and reviewed with what-if" is more useful than an unqualified claim that a template was tested. If a lab has not been executed in Azure, that limitation should remain visible in the authoring record. Clear evidence supports responsible reuse: the next administrator can see which checks were performed and which remain necessary in the target environment. The result is a repeatable process with understandable boundaries.

14.5 Exporting Templates and Converting ARM to Bicep

Use exports to learn from existing resources

An existing environment can help an administrator understand a resource's management representation. Azure can export template information from supported resources or from deployment history. These sources answer related but different questions. An export of current resources attempts to describe their configuration as it exists. A template associated with a deployment records the

definition used for that deployment. Later portal edits may make those two views differ. Before relying on an export, identify which view was obtained and when it was captured.

Exported templates are useful starting points for learning and reconstruction. They are not guaranteed to be complete, portable, or ready for immediate redeployment. Some settings may be absent, some resource types may have export limitations, and generated definitions may contain environment-specific names or references. Secret values are not a dependable part of an export. A template describing a storage account does not contain the blobs stored in it. Recreating management configuration and restoring workload data are separate tasks, even when both are needed during recovery.

This distinction prevents a common misunderstanding about infrastructure as code. A configuration export can help recreate a virtual machine resource, but it does not automatically capture every application setting inside the operating system or every file on its disks. Those elements need their own configuration and data protection methods. The export should be evaluated against the actual recovery objective. If the requirement is to rebuild an application service, the administrator must account for application code, dependencies, identities, certificates, network relationships, and data as well as the Azure resource declarations.

Inspect the export before conversion

Begin by listing the resources in the export and comparing them with the intended scope. Look for references to shared resources outside that scope, such as a subnet in another resource group. Those references can be correct, but they reduce portability and require permissions or prerequisites elsewhere. Replace environment-specific assumptions with explicit parameters where appropriate. Do not parameterize every value simply to make the file appear reusable. A parameter should represent an intentional variation that the deployer is expected to choose.

Generated templates can be verbose because they include properties that an author would omit or organize differently. Some properties describe observed state rather than a useful desired-state choice. Review them against the current resource type reference. Removing a field should be a reasoned edit, especially when it affects encryption, access, availability, or identity. A shorter file is easier to maintain only if it still describes the required configuration accurately. Cosmetic simplification cannot compensate for accidentally weakening a security control.

Check API versions and naming expressions next. An export can use literal names that are appropriate for the original environment but collide when deployed elsewhere. If the purpose is to manage the same resources, preserving names can be essential. If the purpose is to create a separate test environment, names and dependencies need a deliberate transformation. These are different tasks. A reviewer should be able to state whether the result is an adoption of existing resources, a replacement design, or a new environment based on the original.

Decompile without assuming perfection

Decompilation converts an ARM JSON template into Bicep source. The command provides a practical bridge for an organization with existing templates or for an administrator learning how the two formats relate. The conversion is not guaranteed to produce the author's ideal structure. Some constructs may require correction, warnings may identify uncertain translations, and generated symbols may need

clearer names. Read the result as code that needs review, rather than treating conversion success as a certification of the deployment's design.

```
az bicep decompile --file exported-template.json
```

The command writes a Bicep file beside the input. Use a separate working copy when a file with the expected output name already exists; overwriting a maintained source file should be an explicit decision. After conversion, run the Bicep build command and examine its diagnostics. The compiler can identify syntax or type problems introduced or exposed by the conversion. It cannot decide whether an exported address range fits the organization's network plan or whether a reference points to the intended subscription.

A round trip from JSON to Bicep and back to JSON does not need to preserve every character or property ordering. Equivalent deployment behavior is the important objective. Formatting, symbol names, and expression forms can change while the resulting configuration remains equivalent. Conversely, files that look similar can behave differently if a default, scope, condition, or dependency changes. Review the actual resource identities and values, and use validation and what-if in an isolated target environment to examine the result.

Refactor around responsibilities

Once the converted file compiles, improve its structure in small, understandable changes. Rename confusing symbolic identifiers without altering Azure resource names. Add parameter descriptions and appropriate validation. Replace repeated expressions with well-named variables where that improves clarity. Group resources around their lifecycle and consider modules only when the boundary helps. Preserve a working reference version so each change can be compared with the original export. A large rewrite performed before the first successful validation makes conversion errors harder to distinguish from design changes.

Take particular care with dependencies. An export may contain explicit ordering that is redundant after symbolic references are introduced. Removing an unnecessary dependency can make the file clearer, but removing a real dependency can change deployment behavior. Evaluate the relationship instead of applying a blanket rule. A resource ID supplied as a plain string might not establish the dependency that a direct symbolic reference would establish. The reviewer needs to understand how the referenced value is produced and whether the referenced resource is created in the same deployment.

Outputs are another opportunity to improve the interface. An export may include values that are not useful to a future deployer, while omitting a resource ID needed by the next step. Return the smallest useful set of nonsecret information. Avoid exposing keys just because they can be queried during deployment. An infrastructure definition is easier to reuse when its outputs communicate stable identifiers and endpoints. Secrets should remain behind the relevant authentication and authorization controls instead of becoming casual deployment artifacts.

Compare in a controlled environment

A converted definition should first be exercised in a designated practice or test scope that contains no unrelated resources. Choose unique names, a supported region, and a clearly identified cleanup boundary. Examine what-if output before deployment. After deployment, compare important properties with the original design, rather than relying only on the success status. If the goal is adoption of existing

resources, perform a particularly careful preview because the deployment will address resources already in use. The consequences of an incorrect default are greater in that situation.

A practical comparison includes resource count, full IDs, locations, dependencies, tags, network restrictions, identity assignments, encryption choices, and retention settings relevant to the workload. It also includes an operational test. A web application can exist with the correct name while pointing to the wrong storage account. A virtual network can contain the expected subnets while lacking a route required by its clients. Configuration comparison narrows uncertainty; a representative task tests whether the relationships work together.

Maintain one authoritative source

After a converted and reviewed Bicep definition becomes the maintained source, record that decision. Retain the original export as reference material, but do not allow both versions to evolve independently without a defined process. Otherwise, a later administrator may deploy an older JSON file and unintentionally restore obsolete settings. The repository or project folder should identify the source definition, its parameter files, generated artifacts, and deployment instructions. Clear organization prevents mistakes that language features cannot detect.

The same discipline applies to future portal changes. If an urgent change is made directly, update the maintained source or document why the difference should remain. Exporting again can assist comparison, but a new export should not silently replace reviewed source with a large generated file. Infrastructure as code becomes valuable through continued ownership. The files remain understandable, the target scope remains explicit, and the review process explains what will change before Azure is asked to apply it.

Guided lab

Deploy and inspect a small foundation

The exercise uses `companion/ch14-deploy.bicep` and `companion/ch14-parameters.json` to create a virtual network, one subnet, and a storage account in `rg-compass-ch14`. It does not deploy a virtual machine or upload data. Public network access to storage is disabled, and no private endpoint is created, because this exercise checks management configuration rather than data transfers. The optional resource-group lock is disabled by default. Storage usage and operations can incur charges; any later workload added to the network has its own costs. This lab has not been executed in a live Azure subscription during manuscript preparation.

Prerequisites are an Azure practice subscription, Azure CLI with Bicep support, permission to create the resource group, and Contributor or equivalent deployment permissions within it. Creating or deleting the optional lock requires suitable lock permissions as well. Use a region where the selected services are available. Select a globally available lowercase storage name meeting Azure's storage naming requirements. The supplied name is an example and must be replaced.

1. Inspect the two companion files. Confirm that the subnet prefix is contained within the virtual network prefix, that the storage account disallows anonymous blob access, and that `enableLock` is false.

2. Edit the parameter file to select the intended region and a unique storage name. Keep the resource-group and virtual-network names consistent with the exercise.
3. Confirm the Azure account context with `az account show`. Select the intended subscription explicitly with `az account set --subscription SUBSCRIPTION_ID`.
4. Create the group using `az group create --name rg-compass-ch14 --location REGION`, replacing `REGION` with the selected region.
5. Compile the Bicep file locally. Resolve compiler errors before continuing.
6. Run the what-if command in section 14.4. Expect the proposed creation of the defined network and storage resources, with no unrelated resources affected. Investigate any unexpected difference.
7. Run the deployment command only after reviewing the proposal. Inspect the deployment result and resource IDs.
8. In the portal, inspect the network address space, subnet prefix, storage secure-transfer setting, and minimum TLS setting. Confirm that no data access has been granted merely by creating the account.
9. Run what-if again without changing the inputs. Explain any reported differences before deciding whether another deployment is appropriate.

For a controlled failure, make a copy of the parameter file and set the storage SKU to a value outside the allowed list. Run validation or what-if with the copied file. The expected outcome is rejection of the invalid input, not successful creation of a different account. Restore the valid input and keep the original resource identities unchanged. This exercise demonstrates parameter validation without intentionally weakening access controls.

Cleanup is the deletion of the dedicated resource group after its contents have been inspected. If a lock was enabled, remove that specific lock first using an identity with permission to do so. Confirm the selected subscription and group name, then run `az group delete --name rg-compass-ch14 --yes`. Verify that the group no longer exists. Do not reuse the cleanup command for a group containing unrelated work.

Chapter synthesis

A deployment definition is useful when another administrator can understand what it creates, how its inputs affect the result, and how the outcome will be checked. ARM and Bicep describe the same Azure management system through different authoring forms. Stable resource identity, explicit scope, careful updates, and honest verification records make that system easier to operate. Exports and conversions can accelerate authoring, but maintained source still needs deliberate ownership.

Review questions

14-Q1. A Bicep author renames a local resource symbol but preserves its type, Azure name, scope, and properties. What change is expected in the deployed resource identity?

- **A.** A new resource group is created
- **B.** No identity change is caused by the symbol rename
- **C.** The resource receives the symbol as its Azure name
- **D.** The existing resource is deleted

14-Q2. A Bicep file compiles successfully on a laptop. Which conclusion is justified?

- **A.** The deployment identity has all Azure permissions
- **B.** The selected region has available capacity
- **C.** The application has passed its health test
- **D.** The source can be translated into an ARM template

14-Q3. An incremental deployment includes an existing resource but omits some previously configured properties. What should the reviewer do?

- **A.** Review provider update behavior and include the intended full configuration
- **B.** Assume every omitted property is always preserved
- **C.** Switch automatically to complete mode
- **D.** Delete the resource before every deployment

14-Q4. A what-if result reports that a critical expression could not be evaluated. What is the best next action?

- **A.** Treat the resource as unchanged
- **B.** Assume the deployment is impossible
- **C.** Investigate the unresolved part and obtain additional evidence before applying it
- **D.** Ignore all other what-if results

14-Q5. A template successfully creates a storage account. A user still cannot read its blobs. What explains why deployment success is insufficient?

- **A.** Storage accounts cannot be deployed with Bicep
- **B.** Data access also depends on its authorization and network path
- **C.** The deployment name must match the user name
- **D.** Every storage read requires subscription Owner

14-Q6. An optional lock was created by an earlier incremental deployment. Its condition is now false. What cleanup assumption is unsafe?

- **A.** The existing lock can remain
- **B.** Lock deletion needs suitable permissions
- **C.** The exact target scope should be checked
- **D.** Setting the condition false necessarily deletes the existing lock

14-Q7. An exported template has been decompiled into Bicep. What should happen before it becomes maintained deployment source?

- **A.** Review conversion diagnostics, dependencies, environment-specific values, and deployment behavior
- **B.** Assume conversion establishes production readiness
- **C.** Remove all properties to shorten the file
- **D.** Treat the export as a backup of all application data

14-Q8. A deployment fails after successfully creating one of its resources. What is the best first response?

- **A.** Assume no resources exist
- **B.** Repeat unchanged until it succeeds
- **C.** Inspect operation errors and the partial state before correcting the cause
- **D.** Rename every resource

Further reading

- Bicep overview
- ARM deployment modes
- Bicep what-if operations
- Decompiling ARM templates into Bicep

Technical references checked November 13, 2026. Service availability and tooling behavior should be rechecked before running the exercise.

Chapter 15: Running Containers with Confidence

Virtual machines place an operating system under the administrator's direct care. Containers package an application and its dependencies so that a compatible hosting service can run them. The network, identity, storage, and deployment principles already developed still apply. The administrator's task is to understand what the package contains, what the hosting service supplies, and which responsibilities remain with the workload owner.

Learning objectives

- Distinguish an image, a running container, a registry, and a hosting service.
- Manage a registry without confusing resource administration with image access.
- Deploy and diagnose Azure Container Instances.
- Explain Container Apps environments, ingress, revisions, and replicas.
- Select resource allocations and scaling rules from observed workload behavior.

15.1 Understanding Images, Containers, and Hosting Choices

The package and the running process

A container image is a packaged filesystem and associated metadata used to start an application process. It commonly includes application code, libraries, runtime components, and startup instructions. A running container is an execution of that image with a writable layer and a configured environment. The image is therefore closer to a prepared application package than to a running server. Several containers can start from the same image while receiving different environment variables or connection settings. Keeping the package separate from the running instance makes deployment and diagnosis easier to reason about.

An image is usually assembled from layers. Reusing layers can reduce repeated transfers and storage, but an administrator should not assume that every deployment starts instantly because a related image was used earlier. Startup includes retrieving required content, allocating resources, launching the process, and passing any configured health checks. Large images and slow initialization can both increase the delay. When a workload responds slowly after being idle, investigate those stages separately. A slow image download requires a different remedy from an application that spends several minutes loading its own data.

An image name normally identifies a registry, a repository, and a tag or digest. The registry hosts the content. The repository groups related image versions. A tag is a human-readable reference such as `release-2026-11`; a digest identifies particular content. A tag can be moved to another image unless controls prevent that. The word `latest` is simply a conventional tag, not proof of when the content was built or whether it is suitable for deployment. Reproducible releases require a naming and retention practice that lets an administrator determine exactly which image ran.

Configuration and state

Container configuration often includes environment variables, a command, resource requests, ports, and references to secrets or storage. These values can change the behavior of the same image considerably. An administrator diagnosing a difference between two instances should compare both the image identity and the supplied configuration. A correct image pointed at the wrong endpoint is still an incorrect deployment. Configuration should be recorded alongside the deployment definition, with sensitive values handled through the hosting service's protected mechanisms rather than embedded in a publicly visible image.

The container's writable filesystem should generally be treated as temporary unless a particular persistence mechanism is explicitly configured and understood. A container can be replaced during an update, a restart, a scale operation, or a host event. Data written only inside that transient instance may disappear. Durable application state should use an appropriate external service or supported persistent volume. This is the same recovery distinction seen with virtual machine disks, but replacement is especially normal in container-based hosting. The application design must expect instances to come and go.

Temporary storage is still useful. A process can use it for a cache, downloaded intermediate files, or a calculation that can safely restart. The important question is whether losing the data causes unacceptable loss or inconsistency. A cache should be reconstructible. A completed order or a user's uploaded document normally needs durable storage. Assigning a mount does not automatically make the application reliable either; concurrent access, file locking, latency, and permissions need to match the workload. Persistence is a design choice with behavior that must be tested.

Networking remains a separate concern

An application listens on a port inside its container. The hosting service must direct traffic to that port, and any external exposure must be configured intentionally. A common deployment failure is a mismatch between the application's listening port and the host's configured target port. Another is an application listening only on a local interface that the hosting path cannot reach as expected. Reading startup logs and the application's own configuration often resolves these failures more quickly than changing unrelated network security groups.

Public accessibility does not mean anonymous access is appropriate. The hosting service can expose a network endpoint while the application still requires authentication and authorization. Conversely, application authentication does not remove the need to consider network exposure. A private administrative application might need an internal endpoint, suitable DNS, and an allowed network path in addition to user authentication. The familiar layers remain separate: name resolution finds the destination, networking carries the request, transport security protects it, and application controls determine whether the requested action is allowed.

Choosing a hosting service

Azure Container Instances provides a direct way to run container groups without managing virtual machines. It suits exercises and workloads where a straightforward container execution model is sufficient. Azure Container Apps adds application-oriented capabilities such as revisions, ingress, and declarative scaling. A virtual machine running a container engine gives more direct control over the host

but also returns host maintenance to the administrator. Azure Kubernetes Service supports broader orchestration needs, but detailed cluster operation lies outside this book's core scope.

Hosting approach	Main administrative focus	Useful fit
Container Instances	Group configuration, startup, resource allocation, and completion	A bounded task or a simple container workload
Container Apps	Environment, revisions, ingress, and scaling	An application or event-driven service with managed scaling
Virtual machine	Host operating system and container runtime as well as the workload	A requirement for host-level control

Service selection should follow the workload's operating needs. A task that reads a file, produces a result, and exits has different lifecycle requirements from an HTTP service that must remain available. A background worker reacting to queue depth benefits from a trigger that measures outstanding work. A legacy application requiring a particular host configuration may not fit a managed container service without changes. The administrator should identify these requirements before comparing feature lists. A service is suitable when its supported behavior matches the application, not merely because it can start an image.

Compatibility and responsibility

Container images target operating systems and processor architectures. A host must support the image's combination. An image built on an ARM-based development computer is not automatically usable on a service expecting an x64 image. Similarly, a Windows container image cannot simply be treated as a Linux image. Multi-architecture image manifests can help select compatible variants where the hosting service supports them, but the administrator still needs to check the actual platform requirements. An image pull can succeed while startup fails because the content does not match the execution environment.

The platform manages selected infrastructure tasks, but the application owner remains responsible for the image's contents and behavior. An outdated library inside an image is not necessarily repaired when the hosting service updates its own infrastructure. The image needs a rebuild and a controlled redeployment. A practical maintenance record identifies the source image, build process, application version, and deployment target. It should be possible to trace a running workload back to an approved artifact. This traceability makes vulnerability response and rollback more dependable.

Diagnose the lifecycle in order

When a container is unavailable, separate image retrieval, process startup, health, connectivity, and application behavior. A registry authentication failure occurs before the application can start. A process that exits immediately can indicate a bad command, missing configuration, or successful completion of a batch job. A running process with an unreachable endpoint suggests checking ports, ingress, and the application's listening behavior. An endpoint returning an application error requires examination of application logs and dependencies. These stages prevent a generic "container failed" report from becoming an unfocused search.

Useful evidence includes the exact image reference, revision or group identifier, start time, exit code, restart count, resource settings, and a short relevant log excerpt. Avoid putting secrets or complete customer requests into a shared troubleshooting record. A clear report should explain the failed stage and the next test. For example, "the image was pulled, but the process exited after rejecting an absent environment variable" is actionable. It directs attention to configuration rather than storage redundancy or external DNS.

The remainder of the chapter follows this lifecycle. The registry supplies an identifiable package. Container Instances runs a group with explicit resources and restart behavior. Container Apps adds application revisions and managed scaling. In each case, successful administration means observing the workload's actual behavior and choosing settings that support it. Packaging an application changes how it is delivered; it does not eliminate the need to understand its dependencies, data, or failure modes.

15.2 Managing Azure Container Registry

Keep the registry's role clear

Azure Container Registry stores container images and related artifacts. It is not the service that runs the application. Creating a registry establishes a place to distribute images; a separate host must retrieve and execute them. This boundary helps distinguish a registry problem from a runtime problem. If the image cannot be pulled, examine registry identity, permissions, network access, and image reference. If the image has been pulled and the process crashes, inspect the process and its configuration. Both services can be healthy while the application package itself is faulty.

A registry contains repositories, and repositories contain image versions or other artifacts. Repository names can include slash-separated paths to organize content, but those names should not be mistaken for a filesystem permission hierarchy. Access behavior depends on the registry's configured authorization model. A useful naming scheme identifies the application or component and leaves version information to tags or digests. For example, `operations/report-worker:release-17` communicates a component and a release. It does not by itself prove that the release was approved or that its content cannot change.

Tags make images easier for people to reference, while digests provide a content identity. An operational record should preserve enough information to identify the actual artifact deployed. If the same tag is reused for new content, a later deployment might retrieve different bytes under the same visible tag. This complicates incident comparison and rollback. Use distinct release tags or digest-based references according to the organization's process, and retain required prior artifacts. Deleting the only available copy of a previous release can make a planned rollback impossible.

Separate management and repository access

Resource management permissions determine who can create or configure the registry. Image permissions determine who can push or pull its content. They are related through Azure's identity and authorization system but are not interchangeable operations. An administrator should identify the action a caller needs: reading a registry's properties, uploading an image, downloading an image, or administering repository settings. Granting broad management access because an image pull failed can create unnecessary exposure without addressing the intended data action correctly.

Azure Container Registry supports different permission modes, including role-based access control and a mode with repository-level attribute-based access control. The applicable roles depend on the mode. Older examples frequently use `AcrPull` and `AcrPush`; those examples should not be applied blindly to every registry configuration. Inspect the registry's permission mode and current Microsoft role guidance before assigning access. The principle remains stable even as role names and supported conditions evolve: the runtime normally needs to retrieve approved images, while a build system needs permission to publish them.

An individual administrator can authenticate interactively using a Microsoft Entra identity. An unattended workload should use an identity suitable for its hosting service, preferably a managed identity where supported. A service principal can serve other automation scenarios, with its credentials managed through an appropriate lifecycle. Avoid sharing one person's interactive credentials across a deployment pipeline. The identity recorded in access logs should meaningfully identify the actor, and its permissions should fit the work it performs.

An access review should preserve the registry's selected permission mode in its evidence. In a registry using repository-level ABAC permissions, use the repository roles documented for that mode rather than assuming a legacy registry-wide pull role applies unchanged. Conditions can further restrict a role assignment's repository access. An identity might therefore pull one repository successfully and receive a denial from another in the same registry. Test the exact repository required by the workload. A successful login alone establishes neither permission to every repository nor permission to publish new content.

Managed identity and network access

A managed identity removes the need to place a long-lived registry password directly in a supported Azure workload configuration. It does not automatically grant image access. The identity still needs the appropriate registry or repository permission, and the hosting service must be configured to use that identity for the pull. The identity used to deploy the resource may differ from the identity used by the running host. Distinguishing them is essential when a deployment succeeds but image retrieval fails afterward.

Networking is another independent requirement. A private registry endpoint or restricted public access can prevent a host from reaching the registry even when the identity is authorized. The host needs a supported network route and name resolution for the registry endpoints it uses. Test from the actual hosting context where possible. A successful pull from an administrator's laptop does not prove that a private application environment can reach the same service. Likewise, a network timeout should not be diagnosed as an incorrect password without evidence.

The registry's administrative account provides a shared credential with broad registry access. It can simplify certain demonstrations, but it is a poor default for a multiuser production process. Shared credentials reduce attribution and require careful rotation wherever they are stored. This chapter's lab uses a public Microsoft image for runtime deployment so that image hosting can be studied without enabling an administrative registry account. A separate registry inspection demonstrates the management structure. A private-image exercise should add a supported managed-identity path deliberately rather than distributing a registry password in copied commands.

Create and inspect before publishing

When creating a registry, choose a globally available name, region, and supported service tier. The tier determines available capabilities and capacity characteristics, so a selection should follow actual requirements. A small learning registry does not need every advanced feature. Requirements such as private connectivity or replicated distribution need checking against current tier support. Avoid memorizing a price or quota from an old screenshot. Review the current configuration choices and cost information in the target environment before deployment.

The following command illustrates registry creation. It creates a chargeable Azure resource and must be run only in the designated practice group after a unique name is chosen.

```
az acr create \
  --resource-group rg-compass-ch15 \
  --name REGISTRY_NAME \
  --sku Basic \
  --admin-enabled false
```

After creation, inspect the login server, permission mode, and access configuration. A registry name used in management commands is not necessarily the complete image hostname. Image references include the registry's login server followed by the repository and tag or digest. Copy the actual login server from the resource rather than reconstructing it from memory. This habit becomes more important when services introduce different endpoint forms or when a design includes private connectivity and DNS overrides.

Publishing and retention

Publishing an image involves creating or selecting the artifact, assigning its registry reference, authenticating the publishing identity, and pushing it. The build environment and runtime environment can have different tools and permissions. Azure CLI registry login commonly interacts with a local Docker client, so a workstation needs the appropriate local runtime for that workflow. A Cloud Shell session should not be assumed to have a running Docker daemon. If the selected workflow requires local Docker, state that prerequisite explicitly rather than treating a local tooling failure as an Azure permission problem.

Retention is an operational decision. Keeping every historical artifact can increase storage and clutter, while aggressive deletion can break deployments that still reference older content. Tags, manifests, and shared layers have relationships that matter during cleanup. Deleting a tag is not always equivalent to deleting all underlying content, and deleting an artifact can affect consumers that still expect it. Establish a retention policy around deployment history, supported releases, and recovery requirements. Before removing content, identify which active or recoverable deployments depend on it.

Troubleshoot a failed pull

For a failed pull, inspect the exact registry hostname, repository path, and reference first. A typo or deleted tag is a different problem from denied access. Then identify the pulling identity and the registry's permission mode. Confirm the relevant assignment and allow for propagation after a change. Next inspect network reachability and DNS from the host's environment. Finally check platform compatibility and image availability. This sequence follows the request through the systems that must cooperate to deliver the image.

Evidence should identify whether the failure was name resolution, connection, authentication, authorization, missing content, or platform mismatch. A broad change that enables public access and administrative credentials simultaneously may make a pull work while concealing the original cause. A better repair changes the specific failed requirement and verifies the intended least-privilege route. The registry then remains a controlled source of identifiable application artifacts instead of becoming an unexplained exception in the environment's access design.

15.3 Operating Container Instances

Understand the container group

Azure Container Instances, or ACI, runs containers without requiring the administrator to provision and maintain a virtual machine host. Its central resource is a container group. A group defines an execution boundary with shared settings such as networking and a lifecycle. Supported multi-container groups can place related processes together, such as an application and a helper. Those containers are distinct processes, but grouping them means their resource and operational relationships must be understood together. A group should represent a deliberate workload unit, not a collection of unrelated applications placed together for convenience.

The operating system and image architecture must be supported by the selected ACI configuration. Feature availability differs between Linux and Windows containers, so an example written for Linux should not be assumed to apply unchanged to Windows. The lab uses a Linux sample image. Before running a different image, verify its platform requirements and the service's current support. A message about an unsupported image platform is not repaired by increasing memory or changing a DNS label. The requested executable content must match the hosting environment.

Each container needs an image reference and enough configuration to start its main process. CPU and memory settings describe the resources requested for execution. Environment variables can supply nonsecret configuration, while sensitive values should use the supported protected mechanisms. A command override can change what the image runs, but an incorrect override can prevent an otherwise valid image from starting. Inspect the image's expected startup behavior before replacing it. A container whose main process exits is finished from the host's perspective, even if a background process was launched incorrectly inside it.

Restart behavior follows the task

ACI restart policy expresses what should happen when a container process exits. `Always` is appropriate for a continuing service that should be restarted after exit. `OnFailure` can suit a task that should retry unsuccessful exits while remaining complete after success. `Never` preserves a completed or failed execution without an automatic restart. The policy should match the application's meaning of success and failure. Using a continuing-service policy for a successful one-time job can repeatedly rerun work that was intended to happen only once.

Exit codes are therefore evidence about the application lifecycle. An exit code indicating failure can result from a missing file, an invalid environment variable, an unavailable dependency, or application logic. It does not automatically mean that ACI itself is malfunctioning. Inspect the process logs and the configured command. If the application completes successfully under a batch-oriented restart policy, a

terminated state can be the expected result. Monitoring should distinguish that expected completion from an unexpected loss of a long-running service.

Retries also have business consequences. A container that retries a partially completed task can create duplicate output unless the application handles repetition safely. Infrastructure restart policy does not supply application-level transaction guarantees. A worker processing a message should use the relevant service's acknowledgment and retry semantics. A file transformation job should distinguish completed output from partial output. The administrator need not write all application logic, but should understand whether restarting the process is a safe recovery action for that workload.

Configure the network path

A public container group can expose selected ports and receive an address or DNS name according to the configuration. The application must listen on the port that is exposed. A DNS label helps clients find a group, but it is not an authentication mechanism and should not be treated as a permanent application identity without considering the resource lifecycle. The lab uses the returned public address for a brief demonstration and deletes the group afterward. A production endpoint needs a more deliberate naming, transport security, and availability design.

ACI can also participate in supported virtual-network configurations. That design requires attention to subnet delegation, available addresses, outbound connectivity, and the current feature combinations supported by the service. A private address prevents direct public exposure but does not automatically make every dependency reachable. The container might need to contact a registry, storage service, or application endpoint. Confirm those paths and their DNS behavior. A private workload can fail during startup if it cannot retrieve its image or required configuration.

The network lessons from earlier chapters still apply. Check whether a failure occurs before or after name resolution, whether the destination address is expected, and whether the relevant traffic is allowed. An application binding to the wrong port creates an endpoint failure even when Azure networking is correctly configured. Start with the process's reported listening address and port, then compare the group configuration. This is often more efficient than changing routing or security rules without evidence that they are involved.

Logs and interactive diagnosis

Container logs commonly capture output written to standard output and standard error. They are the first place to look for startup messages and application exceptions. A well-designed containerized application writes useful operational information through those streams. An application that writes only to an undocumented internal file can be harder to diagnose after the container is replaced. Logging should include enough context to identify the failed operation while excluding secrets and sensitive payloads. More logging is not always better if it obscures the meaningful event or creates unnecessary cost.

The following Azure CLI commands inspect an existing lab group. They do not create a new group.

```
az container show --resource-group rg-compass-ch15 --name aci-compass-ch15
az container logs --resource-group rg-compass-ch15 --name aci-compass-ch15
```

An interactive command can help diagnose a running container when the image contains a suitable shell or executable. Minimal images often omit common troubleshooting tools, so a failed attempt to launch a

shell does not prove that the application is broken. Use the image's supported capabilities and service logs. Avoid making an unrecorded repair inside a running container and assuming it will survive replacement. The durable correction belongs in the image or deployment configuration, followed by a controlled redeployment.

Storage and external dependencies

ACI supports persistent storage integrations for supported configurations, including Azure Files mounts. The mount introduces additional requirements: the storage resource must exist, access must be authorized, network connectivity must work, and the mount settings must match the service's expectations. A successful container start does not by itself prove that the application can write to the intended directory. Verify a representative file operation when storage is part of the workload, then confirm that the data remains after an appropriate restart or replacement test.

Mounting a shared filesystem also changes how several containers interact. Two processes writing the same file can conflict even when the mount is healthy. File permissions and application behavior need review. The administrator should identify which path is temporary, which path is durable, and which service is responsible for backup. A container image is not the backup of the data written after startup. The recovery design should protect the external data and preserve the configuration needed to reconnect a replacement workload to it.

Group events complement application logs. An event describing image retrieval or resource provisioning can explain why no application output exists yet. Conversely, a clear application exception after startup indicates that the host progressed beyond image retrieval. Preserve the event time and compare it with the process logs instead of merging them into an undifferentiated failure message. When several containers share a group, identify the specific container whose status or logs are being inspected. A healthy helper process does not prove that the main application process is healthy, and their resource settings can differ.

Resource use and cleanup

A container group can consume resources while an application is idle. Choose a restart policy and lifecycle appropriate to the task, and monitor the actual resource state. Stopping or deleting the group affects execution, but separately provisioned storage, monitoring, or networking resources can continue to incur costs. The cleanup boundary should list those dependencies. A resource group dedicated to a self-contained lab provides a practical removal boundary, provided it contains no unrelated resources and any deletion protections are understood.

A useful ACI review asks whether the group is running the intended image, using the expected command and environment, receiving suitable resources, and exposing only the intended endpoint. It also asks what happens after success, failure, and deletion. Those questions make the lifecycle explicit. Container Apps builds on the same application concepts while adding revisions and scaling behavior, which are useful when a workload needs more than a single group's execution lifecycle.

15.4 Deploying Container Apps

Begin with the environment

Azure Container Apps provides a managed platform for containerized applications. An environment supplies shared infrastructure for one or more container apps, including networking and observability integration according to its configuration. An individual app defines the workload, its image, configuration, ingress, and scaling settings. A revision captures a version of revision-scoped application configuration. A replica is a running instance of a revision. Keeping environment, app, revision, and replica distinct makes portal views and diagnostic messages much easier to interpret.

The environment is an important design boundary. Apps placed within it can share infrastructure and networking assumptions, so unrelated security or lifecycle requirements should be considered before grouping them. A practice environment can be small and temporary. An organizational environment needs a deliberate network design, capacity model, logging destination, and ownership. Creating several apps in one environment does not mean that their application permissions are automatically correct. Each app still needs appropriate identities, secrets, and access to its dependencies.

Container Apps supports different workload and capacity arrangements. The administrator should select the supported environment and workload profile that match the application's resource and isolation requirements. Exact resource combinations and availability can change, so check the options offered in the target region. A sample's CPU and memory values illustrate one configuration, not a promise that every possible pair is valid. If deployment rejects a resource combination, consult the selected profile's requirements before increasing values at random.

Make ingress explicit

Ingress is the managed entry path for traffic into a container app. It can be configured for external or internal exposure where supported by the environment design. The target port must correspond to the port on which the application listens. HTTP-oriented ingress can provide a managed HTTPS endpoint, while other supported protocols have their own requirements. The endpoint shown by Azure should be inspected rather than constructed from assumptions about its hostname. A reachable platform endpoint is the beginning of application verification, not the end.

External ingress allows requests from outside the environment through the configured path. It does not mean every application operation should be anonymous. Authentication and application authorization remain necessary where the workload requires them. Internal ingress narrows reachability but still depends on the caller's network context and name resolution. A client outside the allowed context can fail to connect even though the revision is healthy. Diagnose endpoint reachability separately from the application's decision to accept or reject an authenticated request.

An application that listens on a different port from the configured ingress target commonly appears unavailable after a successful deployment. Inspect the image's documentation and startup logs, then compare the target port. Health probes can also fail when their path or expected response does not fit the application. Correct the configuration based on the process's actual behavior. Opening additional public ports without understanding the application's listener is unlikely to solve the underlying mismatch and can unnecessarily broaden exposure.

Revisions preserve application versions

A revision records a particular version of revision-scoped configuration. Changing an image or other revision-scoped settings creates a new revision rather than editing an existing revision in place. Application-scoped settings can affect the app more broadly. The administrator should distinguish these categories when planning a change, especially for secrets, identity, ingress, and traffic behavior. A setting that affects several revisions can limit how independently an older revision can be used during recovery. A revision is a useful release boundary, but it is not a complete snapshot of every external dependency.

Single-revision mode simplifies ordinary replacement of one application version with another. Multiple-revision mode allows more than one revision to remain active and supports intentional traffic distribution between versions. This can enable a staged release, but it introduces operational responsibilities. Both versions need compatible dependencies and sufficient capacity. Traffic percentages should reflect a deliberate test plan, and the observed behavior should be monitored. Leaving several revisions active unintentionally can complicate costs, diagnosis, and the interpretation of which code handled a request.

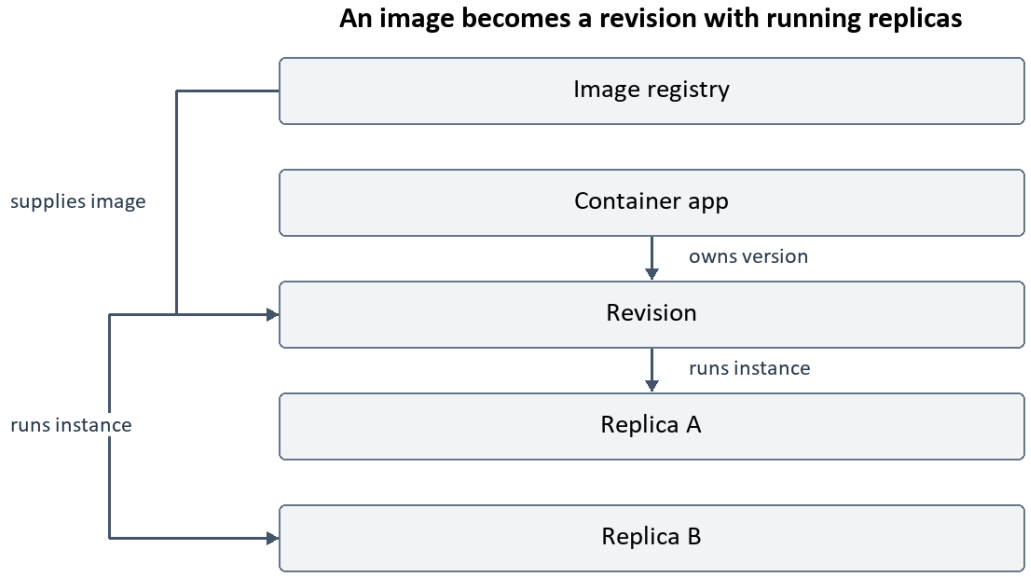


Figure 15.1 Image delivery and application replicas

A revision captures versioned configuration using an image. The platform changes its replica count according to configured scaling. Registry access and runtime access to other services remain separate permissions.

The image registry supplies the artifact used by a revision. The revision then has one or more replicas according to its scaling settings. A new image reference can create a new revision, but it does not update the old image's contents or repair external data automatically. This separation makes release history understandable. A report of an error should identify the revision and image that handled the request, especially while multiple revisions receive traffic.

Configuration, secrets, and identity

Environment variables supply application settings such as a nonsecret service endpoint or operating mode. Sensitive values should use supported secret references or identity-based access. A secret's existence in the app configuration does not guarantee that the application consumes the intended value; the variable or reference must be connected correctly. Changes to secrets also require attention to how active revisions obtain updated values. Follow the service's current restart or revision guidance rather than assuming that every running process immediately rereads a changed setting.

Managed identity can support access to services such as a private image registry or another Azure dependency. The identity must be enabled, selected where the feature requires it, and assigned the appropriate target permissions. These steps are independent of the administrator's own permission to create the app. A successful portal creation operation therefore does not establish that the app can pull its image or read its data. Verify the runtime identity's task through the application's actual execution path, and inspect authorization failures at the target service when necessary.

Avoid baking environment-specific secrets into an image. Images can be copied, retained, and inspected by people with registry access. Removing a secret from a later layer or a later image does not make earlier distributed copies safe. The deployment process should provide configuration through the appropriate mechanisms, and the application should avoid printing it. A useful startup message identifies the selected operating mode and nonsecret endpoint without exposing credentials. This gives administrators evidence while preserving the intended security boundary.

Observe rollout and health

A new revision passes through provisioning and running states. Those states help identify whether a failure occurred while preparing the revision or after it started processing work. Image pull failures, startup failures, failed health probes, and resource pressure can produce different symptoms. Inspect the revision details and logs, then the individual replica where relevant. A single failing replica within a larger revision can be masked by healthy responses from other replicas. Aggregate availability and per-replica health both matter during a rollout.

For a controlled release, record the previous healthy revision, the new image identity, the changed settings, and the verification criteria. Test the new version through a supported endpoint or traffic assignment before increasing exposure. If behavior is unacceptable, return traffic to a known healthy configuration where that remains compatible with external state. Rolling back application code does not reverse a destructive database migration or restore deleted data. Recovery planning must consider the full workload, not just the revision selector.

Traffic distribution and replica count answer different questions. A revision can be assigned a share of incoming traffic while its replica count changes within the configured limits. A revision receiving no traffic might still matter because it remains active or supports another execution path. Inspect the app's traffic configuration alongside the active revision list when diagnosing an unexpected response. Testing a revision-specific endpoint and testing the app's ordinary endpoint can exercise different routing decisions. Record which endpoint was used so that a successful test is interpreted at the correct scope.

Keep the deployment understandable

The portal is useful for seeing an app's relationships, while CLI or template-based deployment makes the configuration reproducible. Use the same names, region, and resource relationships consistently across methods. A convenience command that creates an environment and monitoring resources automatically should be examined so that its resulting resources and costs are understood. The administrator remains responsible for the environment it creates. For learning, inspect the resource group after deployment and explain the purpose of each resource before considering the exercise complete.

Container Apps is most useful when its managed lifecycle matches the application. Revisions provide a clear record of versioned configuration, ingress provides a defined entry path, and replicas provide units of capacity. The next section connects those units to actual workload demand. A scaling rule should express a measurable reason to add or remove capacity, with limits and observations that make the behavior predictable enough to operate.

15.5 Managing Container Resources and Scaling

Start with measured demand

Container resource settings should follow the work performed by the application. CPU-intensive processing, memory-heavy caching, and waiting on an external service produce different resource patterns. An application that spends most of its time waiting for a database may not improve when given more CPU. A process that exceeds its memory allocation can terminate even when CPU use is low. Observe latency, throughput, resource use, failures, and dependency behavior together. One metric rarely explains the whole performance problem.

Vertical scaling changes the resources available to an execution unit, such as its CPU and memory allocation. Horizontal scaling changes the number of running units. These strategies can complement each other, but they are not interchangeable. A single task requiring a larger memory working set may need a larger allocation. Many independent requests may benefit from additional replicas. Before scaling horizontally, confirm that the application can safely run several copies. Shared local state, exclusive file access, and assumptions about one active worker can prevent additional replicas from providing the expected benefit.

Resource requests and limits also interact with the hosting service's supported combinations, available quota, and selected workload profile. A configuration can be reasonable in principle but invalid for the chosen environment. Check the current service constraints during planning, and avoid copying a CPU-memory pair from a different hosting model. The deployment should make the intended values explicit. If an application needs a different capacity class, treat that as an architectural change with cost and operational consequences rather than a small unexplained parameter adjustment.

Scaling Container Instances

For the basic ACI container-group model used in this chapter, the administrator chooses the group's resource configuration and lifecycle. Increasing CPU or memory for a workload is a configuration operation with potential restart or replacement consequences that need checking against current support. Running additional groups requires coordination of work and network endpoints. The simple

lab group should not be mistaken for a complete application autoscaling design merely because it runs on managed infrastructure. Advanced ACI capabilities and orchestration integrations have their own requirements outside this exercise.

A batch task offers a useful example. If the task reads independent input files, several groups might process separate files safely. If every group reads the same uncoordinated work list, adding groups can duplicate processing instead of increasing useful throughput. The application needs a work distribution mechanism, such as a queue with appropriate processing semantics. Infrastructure supplies execution capacity; it does not automatically divide arbitrary application work. The administrator should verify that each additional execution unit contributes useful work and handles failure consistently.

The restart policy also influences observed resource use. An `Always` policy can keep restarting an application that exits after completing a short job. The resulting pattern may look like repeated failures or unexplained compute consumption. Check whether the task is intended to finish and choose its policy accordingly. For a continuing service, repeated restarts require investigation into the exit cause. Increasing memory is justified when evidence indicates memory pressure; it is not a general repair for a missing configuration value or an invalid startup command.

Scaling Container Apps replicas

Container Apps uses declarative rules to scale replicas. The rule can respond to supported signals such as HTTP activity, an event source, CPU, or memory. Minimum and maximum replica settings bound the ordinary desired scale behavior for a revision. A minimum above zero keeps baseline capacity available, while a supported scale-to-zero arrangement can reduce idle execution. The maximum protects against uncontrolled growth but can also become the reason a busy application stops meeting its performance objective. Select both values intentionally and observe whether the workload reaches either bound frequently.

HTTP scaling is appropriate for an application whose demand arrives through HTTP requests. A queue-driven worker is better represented by a signal related to pending work than by incoming web traffic it never receives. Event-source scaling can require credentials or managed identity permissions to read the relevant signal. If the scaler cannot access its source, the application might remain at an unsuitable replica count even though messages accumulate. Diagnose the scaling rule's access path separately from the worker's access to process the messages, because they can be configured through different references.

CPU- or memory-based scaling has a particular limitation: those signals depend on running replicas. Such rules do not support scaling an app from zero in the same way as an external event trigger. A design expecting zero idle replicas and awakening on work needs a supported external signal or request path. This is a useful example of choosing a metric according to the entire lifecycle, not only its behavior while the app is busy. An otherwise sensible threshold cannot wake a workload when no running process exists to produce the measured utilization.

Avoid unstable or misleading rules

A scaling rule should be sensitive enough to respond to demand without reacting to every brief fluctuation. The platform's scaling behavior includes evaluation and timing characteristics, and the application has its own startup and shutdown delays. If a new replica takes time to become ready,

demand can increase faster than useful capacity appears. A small minimum capacity may be justified for a latency-sensitive workload. The decision should compare the cost of warm capacity with the consequence of slower response after inactivity, rather than assuming that scale-to-zero is always the best outcome.

More replicas can also overload a downstream dependency. A database, storage account, API, or network path has its own limits and performance characteristics. Scaling the caller can increase connection count or request rate until the dependency becomes the bottleneck. An administrator should examine end-to-end throughput and error rates after scaling. If application replicas are mostly waiting on a saturated dependency, adding more can worsen contention. Capacity planning needs to include the services that perform the useful work behind the application endpoint.

Scaling configuration changes also belong in release history. A new threshold can alter behavior even when the application image remains identical. Compare a previous and a current revision's resource allocation, minimum and maximum replicas, and trigger settings before attributing a performance change entirely to code. If the trigger depends on an external service, verify its authentication reference and measurement scope as well. An incorrect queue name can make a healthy worker appear idle to the scaler. The remedy is to correct the observed signal, not to leave a large fixed replica count as an unexplained workaround.

Observe a controlled scaling test

A scaling test should use a defined workload, a bounded duration, and a known maximum resource exposure. Record the initial replica count, relevant rule, image version, and dependency configuration. Apply a representative amount of work and observe both the scale decision and the application's results. A rising replica count is not itself proof of success. The test should show whether response time, completion rate, or backlog improves without an unacceptable increase in failures. Stop the test and confirm that the environment returns to its intended state.

Do not use an uncontrolled public load generator against an endpoint simply to watch a graph move. The exercise should be confined to an authorized practice workload with a small, deliberate demand. Even a modest test can create logs, network traffic, and downstream operations that incur charges. Include those resources in the test's cleanup and cost review. A maximum replica setting limits one dimension of capacity; it does not cap every possible charge across the environment. Budgets and alerts provide visibility rather than a guaranteed spending stop.

Diagnose capacity with several observations

Suppose latency increases while CPU is low and replicas have reached their configured maximum. Raising the maximum might help, but the evidence is incomplete. Inspect concurrent work, memory, dependency latency, and errors first. The application could be waiting on a slow external service, or each replica might enforce a low internal concurrency limit. If additional replicas successfully handle independent work and the dependency has capacity, scaling out is reasonable. If every replica waits on the same bottleneck, the repair lies elsewhere. The scaling decision should be connected to a causal explanation.

Record the selected configuration and the observation that supports it. A useful note states that a particular resource allocation processed the representative workload within its target, with observed

headroom and a defined scale limit. It does not claim that the same setting is optimal forever. Application versions and workloads change. Repeat focused measurements after material changes, and use monitoring to identify when the earlier assumption no longer holds. Container administration becomes more dependable when capacity decisions remain understandable and revisable.

Guided lab

Compare a container group with a container app

This exercise deploys a public Microsoft sample image to ACI and Container Apps, then compares their lifecycle and configuration. It has not been executed in a live subscription during manuscript preparation. Use a dedicated `rg-compass-ch15`, a supported region, permission to create the selected resources, and current portal access. The exercise can incur ACI compute, Container Apps, registry, and monitoring charges. No production data or secrets are needed.

1. Create the dedicated resource group in the selected subscription and region. Review its empty resource inventory.
2. Create a Basic Azure Container Registry with a globally available name and its administrative account disabled. Inspect its login server and permission mode. No private image push is required for this exercise.
3. In the portal, create a Linux container instance named `aci-compass-ch15` using the public sample image `mcr.microsoft.com/azuredocs/aci-helloworld:latest`. Verify that the image remains available and supported before proceeding. Configure one virtual CPU, 1.5 GiB memory where the regional options support it, a public endpoint on port 80, and the `Always` restart policy. Review the creation summary and costs.
4. After deployment, inspect the group state, image reference, public address, and logs. Open the returned endpoint and record the sample response. Do not submit sensitive information to the public demonstration endpoint.
5. Create a small Container Apps environment in the same practice group using a supported consumption configuration. Record any monitoring resource created with it.
6. Create `ca-compass-ch15` using the public sample image `mcr.microsoft.com/k8se/quickstart:latest`. Verify current availability first. Enable external HTTP ingress, set the target port to 80, and use a supported small CPU-memory combination offered by the portal. Set minimum replicas to zero and maximum replicas to two for this HTTP demonstration.
7. Inspect the app's generated URL, initial revision, and replica view. Open the URL and compare the lifecycle terminology with ACI. Record which resource owns ingress and which configuration controls replica count.
8. Temporarily set an incorrect ingress target port in the app's ingress configuration. Observe the endpoint or health failure, inspect the app and revision details and logs, then restore the correct target port. Ingress is application-scoped, so do not assume this change creates an independently reversible revision. Do not change registry credentials or network exposure as a speculative repair.
9. Confirm that the repaired application responds, and inspect the group's complete resource inventory before cleanup.

The expected learning outcome is a working sample endpoint on each hosting service and a documented explanation of the controlled port mismatch. Image availability, regional capacity, and portal wording can change, so an unavailable sample or option should be resolved against current Microsoft quickstart guidance rather than replaced with an unknown public image.

Delete the dedicated resource group after confirming that it contains only exercise resources. This removes the container group, app, environment, registry, and monitoring resources placed in that group. If a shared or separately created monitoring resource was selected, it will not necessarily be removed with the group; inspect its ownership before deleting anything. Verify removal and stop any local test process used during the exercise.

Chapter synthesis

Reliable container operation begins with an identifiable image and an explicit execution configuration. Registry permissions, host networking, application startup, persistent data, and scaling are separate requirements that must work together. ACI makes a container group's lifecycle visible, while Container Apps adds revisions and managed replicas. Resource decisions become defensible when they are connected to observed application behavior rather than to the appearance of a successful deployment.

Review questions

15-Q1. Two deployments use the same mutable image tag a month apart. Why might they run different content?

- **A.** A tag can be reassigned to another image
- **B.** A repository cannot contain more than one image
- **C.** An Azure region changes every image automatically
- **D.** A digest is always a random display name

15-Q2. A Container App deployment succeeds but its managed identity cannot pull a private image. Which check is most relevant?

- **A.** Whether the administrator has a larger VM
- **B.** Whether the app has a longer display name
- **C.** Whether the pulling identity has the correct repository permission and network path
- **D.** Whether the image contains a desktop interface

15-Q3. An ACI task should stop after a successful exit and retry an unsuccessful exit. Which restart policy fits?

- **A.** Always
- **B.** OnFailure
- **C.** Never
- **D.** ReadOnly

15-Q4. A Container App has healthy process logs but its HTTP endpoint fails after the ingress target port is changed. What is the best first check?

- **A.** Delete the registry
- **B.** Increase the storage redundancy
- **C.** Grant every user Owner
- **D.** Compare the target port with the application's listening port

15-Q5. A worker must scale from zero when queue messages arrive. Why is a CPU-only rule insufficient?

- **A.** No running replica exists to produce the CPU signal at zero
- **B.** Queues cannot hold work
- **C.** Every Container App requires a public endpoint
- **D.** CPU allocation determines registry permissions

15-Q6. A container writes completed business records only to its temporary writable filesystem. What is the main risk?

- **A.** The image tag becomes longer
- **B.** The registry automatically encrypts the records twice
- **C.** Replacement can lose the records
- **D.** The application cannot start

15-Q7. A registry uses repository-level ABAC permissions. What should an administrator do before copying an older AcrPull example?

- **A.** Enable a shared admin account for every user
- **B.** Check the permission mode's applicable roles and repository conditions
- **C.** Remove all authentication
- **D.** Assume a successful login permits every repository

15-Q8. Adding replicas raises database errors without improving throughput. What is the best next action?

- **A.** Keep doubling replicas indefinitely
- **B.** Delete all logs
- **C.** Assume the deployment failed to create replicas
- **D.** Investigate downstream capacity and connection behavior

Further reading

- Registry and image concepts
- Container Registry authentication
- Container Instances overview
- Container Apps overview
- Container Apps scaling
- Container Apps revisions

Technical references checked November 13, 2026.

Chapter 16: Hosting Applications with App Service

App Service supplies a managed hosting platform for web applications and APIs. The administrator configures the hosting resources, application settings, network paths, release process, and recovery arrangements while the platform manages much of the underlying infrastructure. The container and deployment concepts from earlier chapters remain useful, but App Service introduces its own boundaries. The plan supplies capacity; the app supplies a hosted workload; slots provide separate release endpoints.

Learning objectives

- Distinguish an App Service plan from the applications and slots using it.
- Select capacity and scaling behavior while considering shared workloads.
- Connect a custom DNS name and verify its certificate binding.
- Separate inbound restrictions from outbound virtual-network integration.
- Use deployment slots and backups with explicit recovery expectations.

16.1 Creating Plans and Applications

The plan is a capacity boundary

An App Service plan defines compute capacity and hosting characteristics for apps assigned to it. Important choices include region, operating system, pricing tier, instance size, and instance count. An app is a separate resource that uses that capacity. This distinction explains why creating another app in an existing plan can have different cost and performance consequences from creating another plan. Apps sharing a plan can share its compute resources. Their combined workload matters when evaluating capacity, even if each app has a separate name and configuration page.

A plan is therefore more than a billing label. It is a boundary for performance, features, and some scaling decisions. A small test app placed beside a busy production app can consume resources from the same underlying allocation. Deployment slots and background jobs can also contribute demand. An administrator should inventory what runs in the plan before changing it. A view focused on one app can hide the other workloads affected by a scale operation or a change to the plan's tier.

The pricing tier determines both capacity characteristics and available capabilities. Features such as deployment slots, networking options, and backup behavior depend on supported tiers and configurations. A missing portal option may therefore reflect the selected plan rather than missing user permissions. Check both possibilities. The smallest tier that can start an application is not automatically the smallest tier that supports its operational requirements. A release process requiring slots needs a tier supporting slots even when the application's raw CPU demand is modest.

Choose the operating model

App Service can host supported application stacks and custom container workloads. A code-based deployment uses a platform-provided runtime appropriate to the application. A container deployment

supplies more of the runtime package through an image. The choice affects responsibility for runtime components and the deployment process. A custom image still needs maintenance when its libraries become outdated. A managed runtime reduces some packaging work but requires the application to fit its supported versions and behavior. Confirm those requirements with the application owner before creating the resource.

Windows and Linux plans support different combinations of runtimes and features. A plan's operating-system choice is not a setting to guess from the administrator's laptop. It follows the application's requirements and the intended hosting configuration. A static file example can be deliberately simple, but a production application may depend on a particular framework, native library, architecture, or startup command. Record these requirements so a replacement app can be created consistently. A working endpoint is easier to reproduce when the host assumptions are explicit.

Region selection affects latency, service availability, networking relationships, and the location of the plan's compute. It should align with the workload's data and connectivity requirements. Putting an app in the same resource group as a database does not place them in the same region or create a private connection. The administrator must inspect the actual locations and paths. A resource group organizes management; it does not automatically establish performance or network proximity between its members.

Create the app with clear ownership

The application name must satisfy the service's naming requirements and availability rules. The platform provides a default hostname for reaching the app. Use the actual hostname shown on the resource rather than assuming its exact format. A custom domain can be added later. The default endpoint is useful for initial verification because it separates application deployment from custom DNS configuration. If the default endpoint fails, repairing a custom CNAME is unlikely to solve the underlying application startup problem.

Assign tags and a resource-group location consistent with the environment's management conventions. Record which team owns the code, who maintains the plan, and which identity deploys changes. These responsibilities can be held by one person in a lab, but they are distinct operations. An identity allowed to deploy application content should not automatically receive broad rights over unrelated resources. Use the permissions needed for the selected deployment method and scope, and keep those choices visible in the operating instructions.

Application settings provide values to the application environment. They can hold nonsecret configuration such as an operating mode or service URL. Connection settings and secrets require appropriate handling, potentially through managed identity and Key Vault references where supported. A setting's name must match what the application expects. Creating a value called `StorageEndpoint` does not help an application that reads `STORAGE_URL`. Configuration errors often look like application failures, so compare the configured names and values with the deployment's documented requirements.

Deploy content deliberately

Deployment methods include supported package deployment, source integration, and container image configuration. Choose a method that preserves a clear relationship between the released artifact and the running app. A package should contain the expected application root rather than an extra enclosing

directory that shifts all paths. A framework application may need a build step or a deployment artifact produced by its build process. The administrator should know whether the selected method builds source or simply copies a prepared package. Assuming the wrong behavior can produce an app with files present but no runnable entry point.

For a small static Windows-hosted demonstration, an archive containing `index.html` at its root is sufficient to show content deployment. A command such as the following targets an existing app and uploads the archive. It assumes Azure CLI is authenticated to the intended subscription and that the caller has deployment permission.

```
az webapp deploy \
  --resource-group rg-compass-ch16 \
  --name APP_NAME \
  --src-path site.zip \
  --type zip
```

This example does not establish a complete build pipeline. It demonstrates the boundary between creating the hosting resource and deploying its content. In an operational process, preserve the package version or checksum, relevant settings, and deployment result. That information helps determine what changed when a new version behaves differently. Recreating the app resource alone is not enough if the application artifact has been lost or its build cannot be reproduced.

Inspect startup and configuration

After deployment, inspect both the platform result and the application response. A successful upload means the content transfer or deployment operation completed; it does not prove every application path works. Open the default endpoint, check the returned content, and examine application or platform logs when necessary. A framework mismatch, missing environment variable, incorrect startup command, or inaccessible dependency can prevent the app from functioning. The error's stage should guide the next test. Do not change DNS when the application process has not started successfully.

The app's configuration page includes settings that influence process behavior, transport, health, and deployment. Change them intentionally and document values that matter to the workload. Some changes restart the app. A restart can be harmless in a short lab but significant for a production service, especially if the application stores session state only in memory. The operational design should account for restarts and replacement rather than assuming a process remains continuously alive forever. Durable state belongs in suitable services with their own protection arrangements.

Understand what stopping an app does

Stopping an app prevents it from serving its normal workload, but it does not necessarily stop charges for the dedicated plan that supplies capacity. The plan remains provisioned until it is changed or removed according to its supported lifecycle. Other apps or slots may still use it. A cleanup procedure should therefore identify the app, its slots, and the plan separately. Deleting a practice app while leaving an unused paid plan can create continuing costs that are easy to overlook in a resource list.

Resource	What it represents	What to inspect

Resource	What it represents	What to inspect
App Service plan	Hosting capacity and feature tier	Region, operating system, size, count, shared apps
App	Application configuration and endpoint	Runtime, content, settings, identity, network access
Deployment slot	Separate live release endpoint	Content, slot-specific settings, traffic and swap behavior

Establish a useful operating baseline

Before calling the initial deployment complete, record the plan's capacity, the app's runtime and endpoint, the deployed artifact, and a simple health verification. Identify its external dependencies and the credentials or identities used to reach them. This baseline need not be lengthy, but it should let another administrator distinguish an infrastructure problem from a configuration or application problem. A short record of known working behavior is particularly useful when several teams share responsibility for the service.

The next section considers capacity changes. The plan's shared nature remains important throughout that discussion. Scaling one hosting boundary can affect several apps, and improving the web tier does not automatically improve its database or storage dependency. A dependable capacity decision connects the observed workload to the resource that is actually limiting it.

16.2 Scaling Application Capacity

Distinguish size from count

Scaling up changes the App Service plan's capacity class or tier. This can provide more resources per instance or unlock capabilities required by the workload. Scaling out changes the number of instances available under the selected plan and scaling model. The distinction is similar to virtual-machine and container scaling, but the plan can host several apps and slots. An administrator should identify the entire affected set before changing capacity. A plan-level decision made to help one app can alter cost and behavior for every workload sharing that plan.

The default shared-plan behavior means apps use the plan's instances, while supported features such as per-app scaling or automatic scaling can refine placement and allocation. Do not assume every environment uses the same mode. Inspect the actual plan and app settings, then apply the relevant documented behavior. For an introductory exercise, a single app in a dedicated practice plan keeps the relationship clear. In a shared environment, the inventory and scaling mode belong in the change review because they determine who is affected.

Increasing instance size can help an application whose work requires more memory or processing capacity within each execution instance. Increasing instance count can help when requests can be distributed across independent workers. Neither approach automatically fixes inefficient queries, unavailable dependencies, or a configuration error. Before scaling, identify the symptom and the likely

limiting resource. A graph showing high CPU during slow responses is evidence worth investigating, but it should be connected to application activity rather than treated as a complete explanation by itself.

Read the workload as a system

Useful observations include request rate, response time, error rate, CPU, memory, connection behavior, and dependency latency. Compare them over the same time interval. A plan can show high overall demand because another app in the plan is busy. An app can show poor response time while plan CPU is low because it waits on a remote service. A short spike and a sustained shortage suggest different responses. The administrator should understand which measurements describe the plan, which describe one app, and which describe an external dependency.

Application behavior also determines whether additional instances are safe. Session state held only in one worker's memory can cause inconsistent behavior when requests reach different workers. Background tasks can run more than once if each instance performs the same uncoordinated work. Applications that rely on a local file as a shared lock can fail under distribution. These are design concerns, not reasons to avoid all scaling. They need a suitable state and coordination mechanism before instance count becomes the main capacity control.

Connection limits and downstream capacity can constrain useful scale. More web instances can open more database connections and increase request pressure on storage or external APIs. If the dependency cannot absorb that demand, web-tier scaling can increase errors. Examine the end-to-end result after a change. The goal is successful work completed within an acceptable response time, not simply a larger number of workers. The same principle applied to container replicas also applies to App Service instances.

Manual scaling provides a baseline

A manual capacity change is useful for understanding how the workload responds before automating the decision. Record the original size and instance count, the observed problem, and the expected improvement. Select a supported target, review cost implications, and apply the change through the plan's scaling controls. Then compare the same measurements under a representative workload. A controlled test should not rely on unrelated traffic fluctuations to establish success. If demand changed dramatically at the same moment, the result may not isolate the effect of scaling.

Scaling can involve process movement or restart behavior, and applications should tolerate platform maintenance and replacement regardless of the selected method. Plan a production change around the service's availability requirements. A successful scale operation is not proof that every application session survived or that every dependency remained reachable. Check the user-facing task and relevant logs after the operation. For a practice app containing only static content, the check is simple. A transactional application needs a more meaningful verification sequence.

Scaling down requires at least as much attention as scaling up. The smaller tier must support the app's current features, slots, and networking requirements. A reduction that removes a needed feature may be blocked or require configuration changes first. Lower capacity also reduces headroom. Examine recent demand and expected upcoming workload before deciding that an idle period represents normal use. A system quiet overnight can still need its daytime capacity unless a controlled schedule manages that difference.

Autoscale expresses an operating rule

Azure Monitor autoscale can change instance count according to configured metrics or schedules for supported App Service plans. A profile defines capacity boundaries and rules. Minimum, maximum, and default counts have distinct purposes, and each should be selected deliberately. A metric rule evaluates a signal over a time window and requests a scale action when its condition is met. A schedule can apply a profile during a known period. The administrator should be able to explain the rule in ordinary language before enabling it.

A rule such as adding capacity after sustained high CPU has an implicit assumption: CPU pressure is a useful indicator of insufficient web capacity for that workload. If slow responses are caused by a database outage, the rule might add instances without helping. Metric choice should follow diagnosis. Similarly, a scale-in rule must leave enough capacity to avoid rapid oscillation between adding and removing instances. Evaluation windows and cooldown behavior help moderate reactions, but an unsuitable threshold remains unsuitable even when wrapped in a careful schedule.

Schedules are useful for predictable patterns, such as a known daily processing period. They should include the intended time zone and consider calendar changes. A schedule that was correct in one region or season can be wrong elsewhere if time assumptions are hidden. Record why the schedule exists and what should happen outside it. A default capacity should support an acceptable baseline when a more specific scheduled profile does not apply. Avoid treating a scheduled reduction as a guaranteed cost saving without verifying that it actually occurs and preserves required service.

Automatic scaling is a distinct feature

App Service also offers automatic scaling capabilities in supported configurations. These should not be casually equated with a manually authored Azure Monitor autoscale rule. Their configuration and operating model differ. Check which feature is enabled in the target plan and follow its current documentation. A troubleshooting report that simply says "autoscaling is on" is incomplete. It should identify the mechanism, applicable limits, and the app or plan affected. This avoids applying the expected behavior of one feature to another.

Costs should be considered at the capacity boundary. Additional instances or a larger tier increase the resources provisioned for the plan according to its pricing model. Monitoring, data transfer, and dependencies can add separate costs. A maximum instance count bounds one aspect of scaling, but it is not a total spending cap. A budget alert can help identify unexpected usage; it does not automatically make the plan stop at the budget value. Review actual usage after a scaling change and remove unnecessary test capacity promptly.

Diagnose an unexpected scale result

If the plan does not scale when expected, inspect the enabled scaling mechanism, active profile, time window, metric scope, and available capacity range. A rule may be observing a different resource or aggregation than intended. The plan may already be at its maximum count. A schedule may select another profile. Provider limits or quota can prevent a requested action. The relevant activity and autoscale records help distinguish a condition that never triggered from an action that triggered but failed. Those are different repair paths.

If the plan repeatedly scales in and out, examine whether the thresholds leave a stable operating range and whether the metric responds strongly to instance count. Removing an instance can immediately raise average utilization enough to trigger scale-out again. Longer evaluation and cooldown choices can help, but the right configuration depends on the workload's response. Test a bounded change and observe several cycles before declaring the problem solved. A rule should produce explainable behavior under both rising and falling demand.

The final capacity record should state the selected mechanism, normal range, trigger rationale, dependencies considered, and evidence of useful performance. That record gives future administrators a starting point when application versions or demand change. Scaling is an operating decision that can be revised as evidence improves. It works best when the team knows what the chosen signal represents and what successful service looks like after capacity changes.

16.3 Configuring Domains, Certificates, and TLS

Name resolution and application binding

A custom domain gives an application a name chosen by its owner, such as a subdomain under an organization's registered domain. Two separate configurations are involved. DNS directs clients toward the service, and App Service associates the hostname with the intended app. A DNS record alone does not establish that the app accepts the hostname, and an application binding alone does not make public DNS resolve correctly. This separation is useful during troubleshooting because the failure can be located before or after the request reaches App Service.

The domain owner must be able to create the verification and routing records required by the current App Service workflow. A subdomain commonly uses a CNAME record pointing to the app's designated hostname, while other mappings can involve an A record and verification records. The exact values should be obtained from the app's custom-domain configuration. Do not copy an address from an unrelated app or assume an old screenshot still describes the correct endpoint. The service's presented requirements connect the DNS configuration to the specific resource being bound.

A domain verification record demonstrates control of the name for the binding process. It is different from the record that directs ordinary application traffic. Keeping those purposes distinct avoids removing a verification record as apparently redundant. Domain ownership and routing both matter to the lifecycle of a custom name. When an app is retired, review its DNS records and bindings together. Leaving a name pointing at a released service endpoint can create a dangling reference and an operational or security problem later.

Work through the mapping in order

Begin with a working default application endpoint. This establishes that the app responds before custom DNS is introduced. Next identify the intended custom hostname and confirm that its zone can be edited. Obtain the required records from App Service, apply them at the authoritative DNS provider, and validate the mapping through the service. Then test DNS resolution from an appropriate client and request the application using the custom name. A local cached result can lag behind a recent change, so distinguish cache behavior from the authoritative record's actual value.

DNS time to live influences how long resolvers can retain a previous answer. It does not promise that every client changes its behavior at an exact moment. Some applications maintain connections or their own caches. Plan a production cutover with this propagation behavior in mind. Reduce uncertainty by verifying the new target before changing traffic, preserving a recovery path where appropriate, and monitoring requests afterward. A DNS edit can be quick to submit while its practical effect unfolds across clients over time.

Certificates identify the server name

Transport Layer Security, or TLS, protects the connection and allows the client to validate the server's identity through a certificate. For a custom hostname, the certificate must cover that name and be presented through the correct binding. A certificate for one hostname does not automatically secure a different hostname merely because both reach the same app. The client evaluates the name it requested. This is why a site can work at its platform-provided HTTPS address while showing a certificate error at a newly added custom domain.

App Service supports several certificate acquisition and import paths, including managed certificates where the scenario meets their requirements. A managed certificate reduces renewal administration, but its eligibility and validation requirements must remain satisfied. Custom certificate uploads or Key Vault imports have their own ownership and renewal responsibilities. Choose a method according to the domain, application, and operational requirements. A certificate that was valid during initial setup can later expire or fail renewal if its prerequisites change. Monitor the lifecycle rather than treating issuance as a one-time task.

A binding connects the hostname and certificate within the app's supported TLS configuration. Server Name Indication, or SNI, allows the hostname requested by a client to select the appropriate certificate on shared infrastructure. IP-based binding has different characteristics and requirements. Most ordinary modern-client scenarios can use SNI, but the administrator should understand the selected option and its compatibility needs. If an IP-based choice changes the address used for the app, the DNS mapping must be reviewed accordingly. The certificate workflow and network address workflow are related but not identical.

Managed certificate requirements have changed over time, including issuance and renewal changes introduced in 2025. An older working procedure should therefore be checked against current guidance when a domain is added or its routing changes. This does not mean that every existing certificate needs replacement. It means the administrator should verify the actual eligibility and renewal path for the chosen hostname. Preserve the validation records and network conditions required by that path, and investigate renewal warnings before the certificate expires.

Verify what the client sees

The portal can show that a binding exists, but a real client test should verify the certificate presented for the intended hostname. Inspect the covered names, expiration, trust chain, and connection result. A bound certificate can still be expired or unsuitable. A request routed through another service, proxy, or gateway can present that service's certificate rather than the app's direct binding. Identify the actual path before deciding which certificate needs repair. Testing the direct endpoint and the public application endpoint can answer different questions.

HTTPS-only settings direct the app toward encrypted client access, but application behavior also matters. A page can contain absolute links or resources using insecure schemes. An application behind a reverse proxy can generate incorrect redirects if it does not interpret forwarded scheme information correctly. These issues are not solved solely by uploading a certificate. Verify a representative application flow, including redirects and embedded resources, so the user-facing result matches the intended transport policy. A successful connection to an empty root page is a limited test.

Minimum TLS settings control the oldest protocol version accepted by the relevant endpoint. Select a supported secure configuration consistent with the organization's requirements and client compatibility. Avoid weakening the setting to accommodate an unidentified client failure. First inspect the client's actual protocol and certificate error. A trust failure, hostname mismatch, or missing intermediate certificate is different from an unsupported protocol version. Repairing the specific cause preserves the intended security level and produces a more useful incident record.

Separate application and deployment endpoints

App Service also has deployment and management-related endpoints. Their access and TLS settings can differ from the public application path. A successful website request does not guarantee that a deployment agent can reach the publishing endpoint. Conversely, a successful content upload does not prove that users receive the correct certificate at the custom domain. When an error report mentions HTTPS, identify the exact hostname and operation. The same app can have several relevant endpoints with different clients and network restrictions.

Certificate material includes sensitive components. A public certificate can be shared for validation, while a private key must be protected. A password-protected PFX file still needs careful storage and distribution. Do not include private certificate files or their passwords in an ordinary deployment repository. Use supported protected import and access mechanisms, and restrict who can export or replace the material. The operator who can configure a binding does not necessarily need unrestricted access to every certificate in an organization's vault.

Diagnose a custom-domain failure

A practical diagnostic sequence begins with the exact hostname the user requested. Resolve it and confirm that the result matches the intended routing design. Check that the app contains the corresponding hostname binding. Inspect the certificate presented along the actual request path. Then examine the HTTP response and application logs. A name that resolves incorrectly is a DNS problem; a hostname mismatch is a certificate or binding problem; an authenticated application rejection is a different layer. Keeping the sequence clear prevents a broad change from masking the real cause.

For example, an administrator adds a CNAME and immediately sees a certificate warning at the custom hostname. If the default HTTPS hostname still works, the app and its basic hosting path may be healthy. The next check is whether the custom name has an appropriate certificate binding, not whether the plan needs more CPU. If a valid binding exists, inspect the certificate actually returned to the client and any proxy in the path. Evidence at each layer narrows the next action.

Document the custom hostname, authoritative DNS zone, record purpose, certificate source, renewal owner, and verification method. This information is especially valuable when the app is moved or retired. DNS names often outlive individual app deployments, while certificates have their own lifecycles.

Managing those relationships explicitly keeps a familiar application address trustworthy even as the hosting configuration changes underneath it.

16.4 Managing Application Networking

Follow the direction of the connection

App Service networking becomes much clearer when every requirement names the connection's direction. Inbound traffic travels from a client to the application. Outbound traffic travels from the application to a dependency. A configuration that improves one direction does not automatically change the other. Virtual-network integration primarily supports outbound connectivity from the app into or through a virtual network. A private endpoint supplies an inbound private access path to the app. Confusing those features is a common cause of designs that appear private on a diagram but remain publicly reachable.

Consider an app that must read from a privately accessible data service. It needs an outbound path to that service, suitable DNS resolution, and the service's required authorization. Adding an inbound private endpoint to the app does not by itself provide that outbound path. Conversely, integrating the app with a virtual network does not automatically prevent public clients from reaching its public endpoint. The requirements should be written as separate statements: which clients may reach the app, and which dependencies the app must reach. Each statement then maps to the appropriate controls.

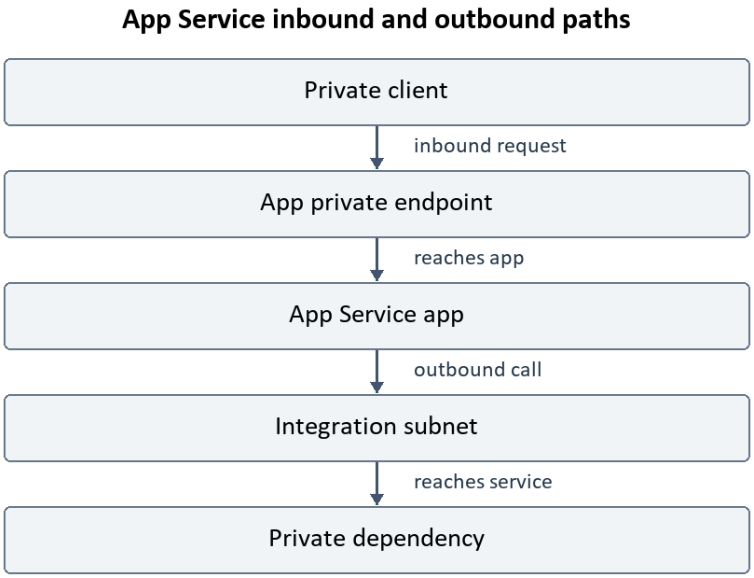


Figure 16.1 Inbound access and outbound integration

A private endpoint handles a supported inbound private path. Virtual-network integration handles outbound app connectivity. Each path needs correct DNS and network configuration; application and data authorization remain separate.

The diagram deliberately separates client access from dependency access. Application authentication and service permissions still apply on both paths where required. Private connectivity narrows the

route through which an operation can occur; it does not grant the caller permission to read data or perform an application action.

Restrict inbound public access

Access restrictions can limit requests reaching an app through its applicable public endpoint. Rules should describe known clients or supported service paths, with priority and unmatched-action behavior understood. An administrator should review the complete rule set rather than adding an allow rule in isolation. A broader earlier rule can change which request reaches the intended rule. The effective result should be tested from both an allowed and a disallowed source. A rule list that looks reasonable is weaker evidence than a verified request outcome.

Publishing endpoints need their own consideration. The main site's restrictions and the deployment endpoint's restrictions can have separate configuration or inheritance behavior. A deployment pipeline might need a permitted path that ordinary users do not need. Locking down the website can therefore leave publishing unexpectedly exposed, or locking down publishing can break an otherwise healthy release process. Identify the deployment agent's network context and configure it deliberately. Do not broadly reopen the app because one publishing operation failed without determining which endpoint was blocked.

An IP-based restriction depends on the source address Azure sees, which can be a proxy or outbound gateway address rather than the individual workstation's private address. A changing home or mobile connection is a poor foundation for an unexplained permanent rule. For a controlled lab, inspect the current public source address and clearly label the rule temporary. For production, use a supported stable access design. The rule should express a known path, not an address copied from a machine's local network adapter.

Use private endpoints with correct DNS

A private endpoint associates the supported service with a private address reachable through the selected network design. Clients must resolve the application name to the intended private path. A missing or incorrect private DNS configuration can leave a client using the public endpoint even when the private endpoint exists. Inspect the resolved address from the actual client network. The existence of a private endpoint resource is therefore only one part of verification. Routing, name resolution, endpoint approval, and application behavior must also align.

Creating a private endpoint does not necessarily disable every public access path automatically. Review the app's public access setting and intended restrictions explicitly. Likewise, a slot can have its own networking requirements, and private endpoint configuration should not be assumed to clone or swap with application content. A release design that uses private staging must provide the staging endpoint's own path and DNS where required. Test both production and staging access rather than inferring one from the other.

Integrate outbound access

Virtual-network integration connects supported App Service configurations to a suitable subnet for outbound communication. The subnet must meet the service's delegation, address capacity, and regional requirements. Capacity planning should allow for the plan's supported scaling and platform

operations. A subnet that barely accommodates the initial deployment can become a constraint when the plan scales. Check current sizing guidance for the selected configuration rather than relying on a generic subnet size copied from another service's example.

An integrated app can reach resources through the virtual network according to the configured routing and DNS behavior. Network security groups and route tables on the integration subnet can influence outbound paths. A user-defined route to an appliance introduces that appliance into the app's dependency path. The appliance must be healthy and configured to handle the traffic. An app that worked before integration can lose access to a public dependency if the new route or DNS configuration blocks it. The change review should list both private and public destinations the app needs.

Routing all applicable outbound traffic through the integration path is a separate configuration decision from simply establishing integration. The exact settings depend on the App Service feature and workload configuration. Inspect the current routing options and document which traffic is expected to follow the virtual network. A security design that assumes all traffic passes through a firewall should verify that assumption through observed connections and the supported configuration. The presence of a subnet association alone is insufficient evidence of complete egress control.

DNS affects dependency access

An app reaching a private endpoint for storage or another service needs name resolution consistent with that private endpoint. If the hostname resolves to a public address while public access is denied, the request can fail even though the private endpoint and permissions are correct. Diagnose the resolved address and route before changing authentication. A failure message may present as a connection timeout or service denial rather than explicitly stating that DNS is wrong. The administrator needs to connect the observed symptom with the intended path.

Custom DNS servers introduce their own availability and forwarding requirements. A server that resolves internal names but cannot resolve required public service names can break application startup or external API calls. The integration subnet's DNS design should support the full set of destinations. Test a private dependency and a required public dependency after a DNS change. This avoids declaring success after one internal hostname resolves while another essential service becomes unreachable. Document the resolver path so future troubleshooting does not depend on memory.

Identity remains independent

A private network path does not replace the identity used to access a dependency. An app can reach a storage account's private endpoint and still receive an authorization error because its managed identity lacks a data role. An app can also have the correct role and fail because the endpoint is unreachable. Check the network and authorization evidence separately. A useful error record includes the destination, operation, runtime identity, observed address, and service response without exposing a token or secret.

Managed identity is particularly helpful when an app needs service access without storing a long-lived credential. Enable the intended identity, assign the appropriate target permissions at a suitable scope, and configure the application to use it. Confirm that the application actually requests tokens for the correct service. Merely turning on managed identity in the portal does not rewrite application code or replace every connection string automatically. The application's authentication implementation and the platform configuration must agree.

Troubleshoot from the failing client

For inbound failures, begin at the client that cannot reach the app. Inspect its name resolution, path, TLS result, and application response. For outbound failures, inspect from the app's execution context or the service's supported diagnostics. A laptop on a different network can produce a misleading successful test. Compare a known permitted path with the failing path and change one relevant condition at a time. This preserves evidence and avoids weakening several controls merely to make an endpoint respond.

The completed network record should show inbound exposure, publishing access, private endpoint relationships, outbound integration, routing, DNS, and dependency authorization. It can be a small diagram accompanied by concise settings and tests. The purpose is to make the actual request path understandable. That clarity becomes especially valuable during a release, when a new slot or configuration may use a different identity or endpoint even though the application code appears unchanged.

16.5 Using Deployment Slots and Backups

A slot is a live application endpoint

A deployment slot is a separate live hosting endpoint associated with an app. It allows a candidate release to be deployed and tested before it becomes the version served through production. Slots require a supported App Service plan tier. They also use the plan's resources, so a staging workload can affect shared capacity. A slot is not simply a folder of inactive files. It can run code, contact dependencies, generate logs, and incur downstream activity. Its configuration and access should reflect that operational reality.

When a slot is created, selected configuration can be cloned from another slot. Cloning helps establish a starting point but does not guarantee that every relevant resource relationship is copied. Identity, private endpoint, and other platform settings have specific behavior that must be reviewed. A staging app should not unintentionally send test messages to a production queue or modify a production database. Examine its connection settings and runtime identity before running tests. The application can be safe to expose while still being unsafe to execute against the wrong dependency.

Each slot has its own hostname, which supports direct verification of the candidate release. A test should confirm the intended version and representative application behavior. A response at the root page might be insufficient for a service whose important work involves authentication, storage, or database access. Use a test that exercises relevant dependencies without corrupting production data. Record the version being tested and its configuration. Otherwise, a later slot swap can appear successful even though the pre-release test was performed against a different artifact or endpoint.

Understand what a swap changes

A slot swap exchanges supported content and configuration between slots according to App Service's swap behavior. Some settings move with the release, while others remain associated with the slot. App settings and connection strings can be marked as deployment-slot settings so that they remain with their environment. Other platform settings have defined non-swapping behavior. The administrator must review the actual list for the app's configuration. A swap should not be treated as an exchange of every property visible in the portal.

Environment-specific connection information is a common reason for slot settings. A staging release may use a test dependency while production uses the production dependency. During a swap, the intended settings need to remain associated with the correct environment. This requires more than marking every setting sticky without thought. Settings that genuinely belong to the application release should move consistently with that release. Classify each important value by whether it describes the code version or the environment in which that version runs.

The swap workflow includes preparation and warming behavior intended to make the transition reliable, but application compatibility still matters. A candidate release that requires an incompatible data schema can break after traffic moves. A background process may run during warm-up and perform work before ordinary users reach the slot. Review these behaviors with the application owner. The platform can coordinate a hosting transition; it cannot infer whether a business operation is safe to execute twice or whether an external database migration can be reversed.

Plan the return path

If a release behaves badly, swapping back can return the earlier application content to production where the previous slot remains available and compatible. This is useful, but it is not a universal rollback of all system state. Data written by the new release remains unless a separate recovery action addresses it. External services may have changed. A revoked secret or modified network rule may affect both versions. The recovery plan should name which changes a reverse swap can undo and which require another procedure.

Preserve the previous known-good artifact even when slots are used. A slot can be overwritten by a later deployment, and retaining only the live state of one staging endpoint makes release history fragile. The artifact repository and infrastructure definition should support rebuilding a known release. Slots improve the transition and verification process; artifact retention preserves the ability to recreate earlier versions. Both are useful parts of a dependable release practice.

Separate application backup from release history

App Service backup protects supported application content and configuration through its backup mechanisms. Automatic and custom backups differ in configuration, storage, retention, and supported scenarios. Automatic backups are available in supported tiers without the same storage configuration required for custom backups. Custom backups can be requested or scheduled and use a configured storage destination. Check current support, limits, and exclusions for the app's operating system and deployment model. A feature being available in the portal does not mean it captures every dependency the application uses.

A backup of the app is not automatically a backup of every database, storage account, message queue, or external service it contacts. Those dependencies need their own protection. Linked database backup support has limitations and a published retirement direction, so a durable design should rely on the database service's appropriate native protection rather than assume App Service will protect it indefinitely. The administrator should inventory what is included in a backup and what remains outside it. Recovery planning depends on that boundary.

Backup storage and credentials also need care. A custom backup destination must be reachable and authorized for the required operation. A stored access token can expire, causing scheduled backups to

fail even though the app continues serving requests. A restrictive network change can break the backup path. Monitor the backup result rather than assuming that a configured schedule is still producing recovery points. A successful app health check and a failed backup can coexist for a long time if nobody watches the backup operation.

Restore into a deliberate target

Restoring a backup can overwrite a target app or slot. The target's current content and configuration can be affected, and App Service stops the target during the restore operation. A separate recovery slot or app can reduce the risk of testing directly over a working production site. Select the destination explicitly, verify that it contains no work that must be preserved, and understand which settings will be restored. The restore's success status should be followed by application verification before traffic is directed to it.

A useful restore test confirms more than the presence of files. Check the expected version, relevant settings, identity and network relationships, and a representative application task. Some relationships may need reconfiguration outside the restored content. Compare the recovered result with the documented recovery objective. If the app cannot reach its database because a private endpoint or role assignment is absent, the backup may still have restored its supported content correctly while the overall service remains incomplete. This distinction identifies the missing recovery steps.

Record recovery objectives in practical terms

The recovery point objective describes the acceptable amount of data loss measured in time, while the recovery time objective describes the acceptable time to restore service. These objectives must account for the app and its dependencies together. An hourly app backup does not establish an hourly recovery point for a database protected on a different schedule. A quick content restore does not establish rapid service recovery if DNS, identities, or external connections take longer to restore. Measure the complete procedure during a controlled test.

A recovery runbook should identify the backup source, selection criteria, target, required permissions, dependencies, verification, and cleanup. It should explain what to do when the latest recovery point is unsuitable. For example, the most recent backup may contain a configuration mistake discovered only later. Earlier retained points can be necessary. Retention choices should reflect how long a problem may remain undetected as well as how frequently backups run. More frequent backups do not compensate for insufficient retention of a usable state.

Combine release and recovery evidence

Slots and backups solve related but different problems. A slot supports controlled release and rapid return to an available earlier version. A backup supports recovery of captured content and configuration when the needed state is no longer present in a live slot. Neither replaces source control, artifact retention, or protection of external data. A mature operating record connects these mechanisms so that an administrator can select the right response to a failed release, accidental overwrite, or wider service loss.

After each significant release, confirm that production serves the intended version, staging remains appropriately configured, backup operations are healthy, and unnecessary test artifacts have been

removed. These checks close the gap between a successful change operation and a service ready for continued use. The following lab practices the distinction with small static pages so that the effect of deployment, swap, and restore can be observed without introducing database complexity.

Guided lab

Release and recover a small static application

Use a dedicated `rg-compass-ch16`, a region supporting the selected App Service configuration, and a globally available app name. Contributor or equivalent permissions are needed for the resources and deployment operations. This exercise uses a paid Windows Standard plan so that deployment slots are available. The plan continues to incur charges while provisioned, including when an app is stopped. Custom backup storage and monitoring can add charges. The lab has not been executed in Azure during manuscript preparation.

1. Create a Windows App Service plan named `asp-compass-ch16` using a supported Standard size, such as S1 where available. Create a code-based web app in that plan. Select a supported runtime; the demonstration serves only static HTML through the Windows hosting environment.
2. Create a local `index.html` containing `<h1>Compass release one</h1>`. Place this file at the root of `site.zip`, without an enclosing folder. Use the deployment command in section 16.1 with the actual app name. Verify the default HTTPS endpoint displays release one.
3. Create a `staging` deployment slot. Inspect its separate hostname and settings. Add a harmless app setting named `COMPASS_ENVIRONMENT` with different values in production and staging, and mark it as a deployment-slot setting in both places.
4. Change the local HTML to `<h1>Compass release two</h1>`, rebuild the archive, and deploy it to the staging slot by adding `--slot staging` to the deployment command. Verify staging shows release two while production still shows release one.
5. Review the swap operation from staging to production. After performing it, verify production shows release two and staging holds the previous content. Inspect the harmless setting in each slot's configuration to confirm it remained associated with the intended slot. The static HTML does not read or display that setting.
6. Swap back and verify production again shows release one. Explain why this did not restore any external database: none was included in the demonstration.
7. In Backups, inspect the app's available automatic recovery points. If none is available yet, configure an on-demand custom backup to a dedicated storage container using the current portal workflow, supported access authorization, and an appropriate expiry for any required token. Keep anonymous blob access disabled. Confirm that the storage network configuration permits the chosen backup path; a valid token alone does not provide connectivity. Record the backup result and destination.
8. Create an empty `recovery` slot and restore a suitable backup into that slot, explicitly selecting it rather than production. Verify the restored HTML and review which configuration was recovered. Do not treat the operation as successful solely because the restore job completed.
9. Compare the plan, production app, staging slot, recovery slot, and backup storage. Record what each mechanism preserved and what each operation changed.

For a controlled diagnosis, deploy an archive with the HTML inside an extra folder to the staging slot. Observe that the expected root content is missing or unchanged according to deployment behavior. Inspect the archive layout, correct it, and redeploy the valid package. This demonstrates a content-root error without weakening transport or access settings.

Remove the dedicated resource group after verifying it contains only exercise resources. If backup storage was created outside that group, inspect and remove only its exercise container or account as appropriate. Confirm that the paid App Service plan has been removed, not merely stopped through its app. Delete local archives when no longer needed, keeping the source example if useful for later practice.

Chapter synthesis

App Service operations depend on clear boundaries between plan capacity, application configuration, network paths, and release state. DNS and certificates establish a trustworthy name, while inbound and outbound networking solve different connectivity requirements. Slots make release transitions observable, and backups protect supported recoverable state. Each mechanism becomes dependable when its limits and verification steps are part of the operating record.

Review questions

16-Q1. Three apps use the same dedicated App Service plan under the default shared placement model. What should be reviewed before increasing plan capacity?

- **A.** Only the selected app's display name
- **B.** Only the DNS zone
- **C.** The demand and requirements of all apps and slots sharing the plan
- **D.** Whether the registry contains Linux images

16-Q2. A paid App Service app is stopped but its dedicated plan remains provisioned. What cost expectation is appropriate?

- **A.** Plan charges can continue
- **B.** Every Azure charge immediately stops
- **C.** The plan is automatically deleted
- **D.** Backup storage becomes free

16-Q3. An app needs outbound access to a private database. Which feature directly addresses the App Service outbound virtual-network path?

- **A.** A custom hostname alone
- **B.** An inbound private endpoint alone
- **C.** A deployment-slot swap
- **D.** Virtual-network integration

16-Q4. A custom domain resolves to the app, but the browser reports a hostname mismatch in the certificate. What is the most relevant check?

- **A.** Increase instance count
- **B.** Inspect the certificate binding and names covered by the presented certificate
- **C.** Change the storage access tier
- **D.** Delete application logs

16-Q5. A staging connection string must remain associated with staging after a swap. What configuration is relevant?

- **A.** Rename the resource group
- **B.** Raise the maximum instance count
- **C.** Mark the connection string as a deployment-slot setting
- **D.** Remove the staging hostname

16-Q6. After scaling out, web instances wait on a saturated database and errors increase. What should the administrator investigate?

- **A.** Database capacity and the application's connection behavior
- **B.** Only the web app's friendly name
- **C.** Whether the certificate has a longer subject name
- **D.** How to add instances without a limit

16-Q7. A reverse slot swap restores older code after a failed release. Which statement remains true?

- **A.** Every database change is reversed automatically
- **B.** All external service settings are restored
- **C.** The old code guarantees compatibility with changed data
- **D.** External data changes may need a separate recovery procedure

16-Q8. A private endpoint exists for an app, but a client resolves the hostname to a public path. What should be examined next?

- **A.** The app's font choices
- **B.** Private DNS resolution from that client's actual network
- **C.** Whether the slot name contains a digit
- **D.** The number of HTML headings

Further reading

- App Service plans
- Scaling App Service
- Custom-domain TLS bindings
- Virtual-network integration
- Deployment slots
- App Service backup and restore

Technical references checked November 13, 2026.

Chapter 17: Seeing What the Environment Is Doing

An Azure environment becomes operable when its behavior can be observed and explained. Monitoring connects the configuration decisions in earlier chapters with evidence from running resources. A chart can reveal pressure, a log can explain an operation, and a connectivity test can isolate a broken path. The administrator needs to choose the right signal and understand its limits before using it to make a change.

Learning objectives

- Interpret metrics with their units, dimensions, aggregations, and time windows.
- Configure relevant log collection and investigate records with KQL.
- Distinguish alert rules, action groups, and alert processing rules.
- Use curated monitoring views while understanding their data prerequisites.
- Select Network Watcher tools for a specific connectivity question.

17.1 Interpreting Azure Monitor Metrics

A measurement needs context

A metric is a numerical observation associated with time and a monitored resource or other defined scope. Examples include processor utilization, request counts, and storage latency. The number becomes meaningful only when its unit, interval, aggregation, and dimensions are understood. A value of fifty could represent a percentage, a count, or milliseconds. A chart without those details can create a persuasive but incorrect impression. The first step in reading any metric is identifying exactly what was measured and how it was summarized.

Azure collects many platform metrics automatically. They provide a useful starting point without installing software inside every resource. Guest operating-system metrics and application-specific measurements can require additional configuration. The absence of a particular metric therefore does not always mean that the resource is inactive. It may mean the required collection path has not been enabled. A virtual machine's host-level processor metric and a process-level memory measurement come from different observation boundaries. The administrator should know which boundary is relevant to the question being investigated.

In Metrics explorer, the selected resource and metric namespace determine which measurements are available. The time range determines the period displayed. Time granularity determines how observations are grouped into chart points. The aggregation determines the calculation performed within each group. These choices can change the appearance of the same underlying activity substantially. Before comparing screenshots or reporting a trend, confirm that they use compatible scopes and settings. Two charts with different aggregation windows can appear contradictory while both accurately summarize their selected data.

Aggregation changes the question

Average describes the typical measured value within the chosen interval, while maximum highlights the largest observed value. Minimum can identify the lowest point. Sum adds values where that operation is meaningful, such as a count of transactions over an interval. Count can describe the number of measurements rather than the sum of their values. Not every aggregation is appropriate or available for every metric. Choose the statistic that answers the operational question instead of accepting a default simply because it produces a smooth line.

Suppose a processor is heavily used for a short period and mostly idle afterward. A long-window average can hide the brief pressure, while a maximum can preserve evidence that the peak occurred. Neither proves whether users experienced a problem. Compare response time or task completion during the same interval. A short peak can be harmless if work completes within its objective. Sustained moderate utilization can be problematic if another constrained resource causes delay. Metrics need a relationship to useful service behavior, not only a threshold inherited from another environment.

Changing time granularity also changes how much detail remains visible. A day-long chart grouped into large intervals is useful for broad patterns but less useful for investigating a failure lasting a few minutes. Zoom into the incident interval and use an appropriate granularity. Then widen the range to see whether the event was unusual. This movement between detail and context helps distinguish a one-time event from a regular pattern. Keep the original incident time in a consistent time zone so that log records and charts can be compared accurately.

Dimensions reveal differences inside a total

A metric can have dimensions that separate observations by categories such as response type, API operation, or instance. Splitting a chart by a useful dimension can reveal that an overall healthy average hides one failing component. For example, a storage account's total transactions might appear steady while one response category shows throttling. An app's overall response time might hide a slow instance. Inspect the available dimensions and choose a split that tests a plausible explanation. Splitting indiscriminately can produce a crowded chart that obscures the result.

Filters determine which dimension values contribute to the displayed series. A chart filtered to successful requests should not be reported as if it describes all requests. Likewise, a view of one instance is not the same as a plan-wide view. Record important filters with the finding. A concise note such as "maximum latency for the selected API over the incident interval" is more informative than "latency was high." It tells another administrator which observation can be reproduced and which conclusions still require additional evidence.

Missing data is not zero

A gap in a chart can mean that no observation was available, not that the measured value was zero. Collection delay, resource state, unsupported metrics, filtering, or a configuration problem can create gaps. Treating absence as a healthy zero can conceal an outage in the monitoring path itself. Investigate whether the resource should have produced data during the interval and whether other signals were received. A stopped virtual machine and a running machine with a failed guest agent can both lack a particular guest signal for very different reasons.

Ingestion and display are not instantaneous. A recently generated event or measurement may take time to appear. Before concluding that collection is broken, check the expected pipeline and a reasonable time range. Repeatedly changing configuration during normal delay can create a more confusing situation. At the same time, indefinite waiting is not a diagnosis. Use evidence from the source, collection configuration, and destination to determine where the expected data is missing. The monitoring system has components that can be inspected like any other system.

Compare related signals

Processor use, memory pressure, disk activity, and network traffic can help explain a virtual machine's behavior, but their relationship matters. High processor use during a scheduled calculation may be expected. Low processor use with slow requests can indicate waiting on storage or a remote service. A storage account's end-to-end latency and server latency can help distinguish processing time from other portions of a request path, but the exact metric definitions must be consulted. Do not subtract unrelated measurements or compare statistics produced over different populations without checking their meaning.

Observation	Useful next question	Evidence to compare
High CPU	Is useful work increasing or stalled?	Throughput, errors, response time
Storage latency increase	Which operation and response type are affected?	API dimensions, request failures, client behavior
Missing guest metric	Was collection configured and the agent healthy?	DCR association, agent state, other signals
Uneven instance behavior	Is one worker different?	Per-instance metrics, release and configuration

A baseline describes the environment's usual behavior under known conditions. It can reveal whether a current value is unusual, but it should not become a permanent assumption. A new application release, changed user population, or scheduled batch job can alter the normal pattern. Record why a baseline period is representative. Comparing a holiday with a busy weekday without accounting for demand can produce a false performance conclusion. The administrator should combine historical comparison with knowledge of the workload's actual operating schedule.

Units deserve a second check when information moves between tools. Bytes, kibibytes, and decimal kilobytes express related but different quantities. A rate per second is different from a total over an interval. A percentage can describe utilization of a limit rather than an absolute capacity value. Preserve the unit in notes and exported charts so that a later reader does not compare unlike measurements. If a service documents a metric as a count of requests, do not relabel it as a count of users without a separate relationship that supports that interpretation.

Chart presentation can also influence perception. An automatically adjusted vertical axis can make a small change look dramatic, while a broad fixed scale can hide a meaningful variation. Read the axis and underlying values before describing a trend. Compare charts using compatible scales when that comparison matters. The administrator's conclusion should follow the measurements and workload

effect, not the visual steepness of a line. This is especially useful when a monitoring screenshot is shared without the interactive controls that would reveal its configuration.

Turn a chart into an operational finding

A useful metric finding states the resource, interval, measurement, aggregation, and consequence. It also distinguishes observation from explanation. "The selected instance showed repeated high processor peaks during slow requests" is an observation. "An inefficient application loop caused the peaks" requires additional evidence. Logs, profiling, or a controlled test may establish that explanation. Keeping those stages separate prevents a plausible hypothesis from becoming an unsupported fact in an incident report.

Metrics are especially useful for narrowing where to look next. They can identify the time of a change, the resource under pressure, or a pattern worth investigating. They rarely explain every detail of an individual request or management action. Logs provide that additional context. The next section follows the path from collection configuration to a query whose result can support a specific operational conclusion.

17.2 Collecting Logs and Asking Questions with KQL

Choose the source before the query

Logs are records describing events, operations, or observations with fields that can be searched and analyzed. Azure Monitor Logs stores collected data in tables, commonly within a Log Analytics workspace. A workspace provides a data and management boundary with access, retention, and table settings. Creating the workspace does not automatically collect every log from every Azure resource. The source must be configured through its supported collection path. A query cannot recover an event that was never collected or retained in the queried destination.

The activity log records many Azure management operations, such as resource creation and configuration changes. Resource logs describe supported operations within a service, often including data-plane activity. Guest operating-system logs and application logs have additional collection requirements. These categories answer different questions. A virtual machine start operation can appear in the activity log, while an application exception inside that machine belongs to another source. A missing event in one category should not be treated as proof that the event did not occur anywhere.

The activity log is not a complete authentication and authorization audit trail. Requests rejected before a management operation begins may not generate an activity-log entry. Identity sign-ins and other security events have their own sources. This distinction matters when investigating a denied request. An empty activity-log search does not prove that nobody tried to access the resource. Identify which service processed or rejected the request and which logs that service produces. The monitoring source should match the operation being investigated.

Configure the collection path

Diagnostic settings route selected platform logs and supported metrics to destinations such as a Log Analytics workspace, storage account, or event hub. The available categories depend on the source

resource. Choose categories that support the operating requirement rather than enabling every possible source without considering volume, cost, and sensitive content. The destination must exist and meet the relevant access and network requirements. A diagnostic setting is a configuration for a particular source; it is not a universal switch for all resources sharing its group.

Data collection rules, or DCRs, describe collection and routing behavior for supported monitoring paths. They can be associated with resources that use the rule. A DCR is configuration, not a storage destination through which every record physically passes. For virtual-machine guest collection, the agent, rule, association, destination, and supported connectivity must fit together. Installing an agent without the intended rule association can leave needed data absent. A rule pointing to a different workspace can make the data appear missing when it is simply being queried in the wrong place.

Collection should be verified with a known event or observation. Generate a small, harmless operation appropriate to the source, then confirm the resulting record after allowing for ingestion delay. Record its time and resource ID. This establishes that the source, configuration, and destination work together. It is stronger evidence than a portal message saying the diagnostic setting was saved. The same pattern is useful after changing a destination or tightening network access. A configuration operation can succeed while the intended telemetry path remains incomplete.

Read the table schema

Kusto Query Language, or KQL, expresses questions over the collected records. A query commonly begins with a table name and passes results through operators separated by a pipe character. Operators filter rows, select columns, summarize values, or sort results. The table's actual schema determines which columns are available. Start by inspecting a few records and the schema in the editor. A query copied from another workspace can fail because that workspace uses a different table or collection model, not because the language itself is broken.

The following example assumes that subscription activity-log records have been routed to the selected workspace's `AzureActivity` table. It lists recent operations for the chapter's practice group. If the table has not received data, configure and verify collection first.

```
AzureActivity
| where TimeGenerated > ago(1h)
| where ResourceGroup =~ "rg-compass-ch17"
| project TimeGenerated, OperationNameValue, ActivityStatusValue, ResourceId
| order by TimeGenerated desc
```

The time filter narrows the investigation. The resource-group comparison is case-insensitive. `project` keeps only the columns needed for the question, and ordering places the newest records first. The query does not change resources or delete data. Its result still requires interpretation: an operation can produce more than one record as it progresses. Counting rows is not automatically the same as counting unique user actions. Inspect correlation and operation fields when the distinction matters.

Summarize without losing meaning

Summarization turns many records into a smaller result. A count grouped by operation can show which operations dominate an interval. Binning timestamps creates time buckets for trend analysis. The example below counts records rather than claiming to count unique deployments. Its label should preserve that distinction.

```
AzureActivity
| where TimeGenerated > ago(6h)
| where ResourceGroup =~ "rg-compass-ch17"
| summarize Records=count() by OperationNameValue, bin(TimeGenerated, 15m)
| order by TimeGenerated asc
```

A summary can hide the details needed to explain an unusual value. Return to the underlying records when a bucket stands out. Check whether a repeated operation generated several status records, whether retries occurred, and whether the query includes the intended resource scope. A large count does not establish malicious activity or a service fault by itself. The result becomes evidence when it is connected to the recorded operations and the surrounding context.

Column types influence query behavior. A field containing structured data may need to be inspected or converted before it can be compared with a plain string. A numeric value stored as text does not automatically behave like a number in every expression. Start with the schema and sample records, then make conversions explicit where needed. Avoid building a complex query around assumptions about a column's shape. A readable sequence of filtering, selection, and aggregation is easier to validate than a dense expression that performs several unexplained conversions at once.

Query performance matters when an investigation searches large volumes. Apply a relevant time filter and selective resource conditions early, then return the columns needed for the question. This keeps the result manageable and reduces unnecessary processing. A broad search across every table can help discover an unknown source, but it is a poor default for a saved operational query. Once the table is identified, use a focused query with clear assumptions. The same discipline improves alert reliability because the rule evaluates a stable, purposeful result rather than a costly exploratory search.

Scope, time, and access

Opening Logs from a resource can set a narrower scope than opening the workspace directly. The same query text can therefore return different records depending on the editor's scope and the caller's access. Check the scope shown in the interface before interpreting an empty result. Also examine the selected time range and any time filter in the query. A portal time selection and a query condition can interact in ways that exclude the desired event. Keep the incident timestamp and time zone explicit.

Log access can reveal operational and personal information. Grant access according to the data needed for the role, and avoid collecting secrets in application logs. A query result exported to a file becomes another copy of that information with its own handling requirements. For this book's exercises, use harmless resource changes and avoid entering real personal or production data. The objective is to understand the collection and query path, not to gather the largest possible dataset.

Diagnose no results systematically

If a query returns no rows, first distinguish a valid empty result from a query error. Then confirm the table exists and has records in a wider appropriate interval. Check the source resource, collection category, destination workspace, and rule associations. Verify that the expected operation actually occurred after collection was enabled. Allow for documented ingestion delay, and inspect whether retention or filtering removed the relevant data. Change one filter at a time so the cause becomes clear. Removing every filter immediately can produce an overwhelming result without explaining the original failure.

Save useful queries with a clear purpose and their prerequisites. A query called "recent management changes in the practice group" is easier to reuse than an unexplained collection of operators. Include the source table and the meaning of important fields in the operating notes. Queries become dependable tools when their assumptions are visible. The next section uses those observations to define alerts that identify conditions requiring action.

17.3 Building Alerts, Action Groups, and Processing Rules

Begin with the response that is needed

An alert should identify a condition that warrants a defined response. A rule that produces frequent notifications without an owner or next action quickly loses value. Before selecting a metric or writing a query, state the operational concern: a service is unavailable, a backup has failed, a resource has sustained pressure, or an important management operation occurred. Then identify the signal that can reveal that condition. This sequence connects monitoring configuration to useful work instead of accumulating alarms simply because a metric is available.

An alert rule specifies a scope, signal, and condition. Depending on the alert type, it can include an evaluation frequency, time window, dimensions, and threshold behavior. An action group defines notifications or automated actions associated with an alert. An alert processing rule modifies action-group handling for matching fired alerts, often according to a scope, filter, or schedule. These are separate resources with separate purposes. Changing an email recipient does not necessarily change the condition that fires, and suppressing notifications does not necessarily prevent an alert record from being created.

Choose the alert type

Metric alerts evaluate numerical signals and suit conditions such as sustained resource pressure or a request error measure. Log search alerts evaluate query results and support more detailed questions over collected records. Activity log alerts respond to matching management or health events. The right type follows the signal and required timing. A log query that depends on delayed ingestion can behave differently from a platform metric condition. An event-based alert for resource deletion asks a different question from a metric alert for high processor use.

Static thresholds compare a signal with an explicitly chosen value. Dynamic threshold features, where supported, use observed patterns to determine unusual behavior. Dynamic behavior still requires understanding the metric, training requirements, and workload pattern. It should not be treated as an automatic substitute for a service objective. A consistently poor response time can become normal historical behavior while still being unacceptable to users. The alert design should connect the platform's condition with what the team actually considers a problem.

Dimension selection can create separate alert evaluations for different parts of a resource population. This is useful when one failing instance would be hidden by an overall average. It can also increase the number of alert instances and notifications. Choose the split deliberately and review its cost and operational volume. An alert for every possible dimension value can overwhelm the responder. A narrowly filtered condition can miss a newly added instance if its value was not included. The configuration should match how the monitored population changes over time.

Understand state and human response

The alert condition reflects the monitored signal's state, such as fired or resolved. The user response records how a person is handling the alert, such as acknowledging or closing it. Those are different concepts. Acknowledging an alert does not repair the underlying resource. Resolving the measured condition does not prove that someone reviewed the incident. Keeping these states separate helps an operations team distinguish current service health from the progress of its investigation and follow-up work.

Stateful and stateless alert behavior can affect repeated notifications. A stateful rule tracks whether its condition remains active and has defined resolution behavior. A stateless rule can produce alerts repeatedly when its evaluation meets the condition. The relevant options and semantics depend on the alert type. Inspect them before deciding whether repeated messages are duplicates or expected behavior. A noisy notification pattern might be a rule design issue rather than repeated independent service failures.

Design the action group

An action group can send notifications or invoke supported automation. Its name and short display information should help recipients identify its purpose. A learning exercise should use an address controlled by the reader and avoid paging a shared production on-call team. Test the notification path through the supported test capability where available, then test the real rule separately. A successful action-group test proves that a notification route can work; it does not prove that the alert's condition, scope, or data collection is correct.

Automated responses need clear authorization and failure handling. A webhook or runbook can perform consequential operations, so it should validate the alert context and act within a defined scope. A repeated alert should not cause an unsafe repeated change. For example, an automation that restarts a service needs to consider whether the condition is actually repaired by restarting and how repeated attempts are limited. This book's basic lab uses a notification-only action group so that alert behavior can be learned without introducing an automated resource mutation.

Notification delivery can fail independently of alert creation. Incorrect recipients, endpoint authentication, network restrictions, downstream service availability, or rate limits can affect delivery. Inspect the alert record and action details before concluding the rule did not fire. A responder who checks only an inbox may miss a correctly created alert whose notification was suppressed or unsuccessful. The operating procedure should distinguish detection, routing, delivery, and response as separate stages with their own evidence.

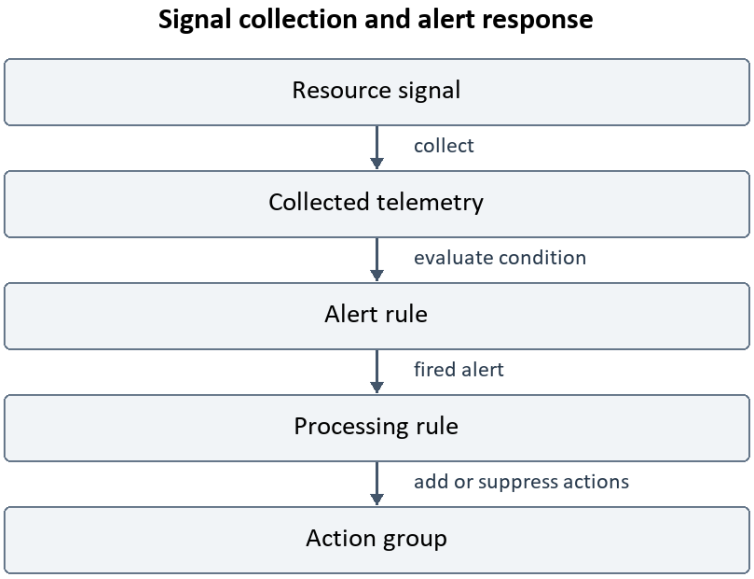


Figure 17.1 From collected signal to an operational response

Collection, rule evaluation, and notification are distinct stages. A processing rule can suppress matching action groups without preventing creation of the underlying alert. Inspect each stage when a notification is missing.

The diagram places processing rules between alert creation and action handling. This makes their role visible: they can alter which actions run for matching alerts while preserving the fact that the monitored condition occurred. The resulting record is valuable during maintenance, when a known temporary condition should not wake an operator but still needs to be visible during review.

Suppress actions with a defined scope

An alert processing rule can suppress action groups during a planned maintenance window for matching resources and alerts. It can also add action groups according to an agreed routing policy. Use the narrowest scope and filters that express the requirement. A subscription-wide suppression created for one virtual machine's maintenance can hide notifications for unrelated resources. Set the schedule explicitly and verify its time zone. A rule left enabled indefinitely after a one-time task becomes a silent operational gap.

Suppression is not the same as disabling collection or disabling the alert rule. The condition can still be evaluated and recorded even when actions are suppressed. This supports later review of what happened during the maintenance period. Some alert categories have specific processing-rule limitations, so check the supported behavior for the source involved. For example, Azure Service Health alerts are not affected by alert processing rules. A general suppression assumption should not be applied to every type of Azure notification.

Test the full chain

A meaningful alert test creates a harmless condition that should meet the rule, then follows the evidence from signal to alert to action. Record the event time, allow for collection and evaluation delay, and inspect the alert details. Confirm the recipient receives the expected message and can identify the affected resource and next action. Then clear the condition and inspect resolution behavior where supported. This validates more than a button that sends a sample notification. It tests the actual relationship between the environment and the rule.

For a log alert, run its query manually over a suitable interval and inspect the returned rows before enabling the rule. Confirm whether the rule measures row count, a numeric column, or another supported result. A query that already summarizes a count can be misinterpreted if the rule then counts the single summary row instead of the numeric result. The alert configuration must match the query's shape. A small test dataset makes this distinction easier to observe and prevents a condition that always fires or never fires for the wrong reason.

Maintain useful alerting over time

Review alerts that fire frequently without action. The condition may need better filtering, a longer observation window, or a different signal. It may also reveal a real recurring problem that deserves repair. Do not silence a useful alert solely because it is inconvenient. Review its purpose and the evidence. Conversely, an alert that has never fired might be well designed for a rare event or completely broken. Periodic controlled tests establish which explanation is true.

Every important alert should have an owner, purpose, scope, severity rationale, and response reference. Severity should communicate business impact rather than the author's anxiety about a number. A high processor value on a disposable practice VM is not automatically equivalent to a customer-facing outage. When resources are retired or moved, update the rules and action routes that refer to them. Alerting remains trustworthy when the configuration follows the environment's actual lifecycle.

17.4 Using Insights for Compute, Storage, and Networks

Curated views are starting points

Azure Monitor Insights and related curated views organize telemetry around common resource-management questions. They can make it easier to compare health, performance, capacity, and failures without constructing every chart from scratch. Their convenience does not remove the need to understand the underlying data. A panel can be empty because a required collection path is absent, a resource is outside the selected scope, or no relevant activity occurred. The administrator should identify the source and prerequisites before interpreting a visual state as healthy or unhealthy.

Curated experiences evolve as Azure's monitoring platform changes. Older material may describe a view or agent that has been replaced or supplemented. The useful skill is recognizing the relationship between resource, collection configuration, telemetry destination, and visualization. A portal label can change while that relationship remains. When following an older exercise, consult current Microsoft guidance rather than installing an obsolete agent simply to reproduce a screenshot. The book's objective is sound administration under the current service model.

Monitor a virtual machine from host to guest

Azure supplies host-level VM metrics that describe observations from the platform's perspective. These include useful signals about processor, disk, network, and availability behavior depending on the resource. Guest monitoring adds information from inside the operating system and application environment. The two views complement each other. A host metric can identify a machine under pressure, while guest information can identify the process or operating-system behavior involved. Neither should be assumed to contain every detail of the other.

Current VM monitoring guidance distinguishes enhanced monitoring choices, including newer OpenTelemetry-based metrics and a classic logs-based metrics experience. The supported choice should be made according to the deployment and current guidance. Guest monitoring uses the appropriate agent and collection configuration, while destinations and query behavior depend on the selected experience. This matters when a query from a classic workspace-based example does not find data in a differently configured environment. Confirm the telemetry path before concluding that monitoring failed.

The Azure Monitor agent and its configuration need a healthy installation, an appropriate identity and connectivity path, and the intended collection association. A portal onboarding workflow can create several of these pieces, but they should still be inspectable. Review the agent or extension state, DCR associations, destination, and expected records or metrics. If only host metrics are visible, guest collection may not yet be configured. If guest data disappears after a network change, inspect the required endpoint access as well as the agent's local status.

Interpret performance with workload knowledge

A VM performance view can show which machines are busy and how their demand changes over time. Start with the incident interval and compare relevant metrics, then inspect guest detail where needed. A machine with sustained high disk latency might be limited by the disk configuration, its throughput pattern, or an application issuing inefficient requests. A machine with low free memory might be functioning normally with an effective cache, or it might be under harmful pressure. The operating system and application evidence determine which interpretation fits.

Dependency-oriented views, where supported and configured, can help reveal connections between processes and endpoints. Treat them as observations within their collection scope, not as a guaranteed complete architecture inventory. A rarely used path may not appear during the selected interval. Encrypted traffic can still reveal a connection without exposing its business meaning. The administrator should combine observed relationships with the documented application design. A diagram is useful when its absence and presence are interpreted with the collection window in mind.

Use Storage insights to narrow the problem

Storage insights organizes storage account observations around performance, availability, failures, and capacity. A view across several accounts helps identify which resource deserves closer inspection. A resource-level view then exposes more detail about the selected service and operation. Start with the affected account and time interval rather than assuming that every account in a subscription shares the same problem. An account's total activity can also hide differences between blob, file, queue, and table workloads.

Transaction dimensions can separate operation types and response categories. A rise in authorization failures suggests a different investigation from a rise in throttling or server errors. Capacity trends answer another question: whether stored data is growing and how that affects cost or service planning. A capacity increase does not necessarily explain latency, and a latency increase does not imply data loss. Keep the dimensions of the investigation distinct while looking for relationships supported by evidence.

Comparing end-to-end and server latency can help identify where additional delay may occur, provided the metric definitions, aggregation, and scope are compatible. Client behavior, network conditions, and request size can influence observed results. A high average over one operation should not be compared casually with a maximum over all operations. Drill into the relevant API and response type. If the metrics identify an unusual category, use resource logs or client-side evidence to understand specific requests within it.

Understand the data behind storage views

Many storage platform metrics are collected automatically, so useful charts can exist without a custom diagnostic setting. Detailed resource-log analysis has separate collection requirements. The existence of a Storage insights workbook does not prove that every data operation has been retained in a Log Analytics table. If the investigation needs individual request records, confirm that the relevant service's logs were enabled and routed before the event. A newly enabled setting normally cannot reconstruct historical data that was never collected.

A storage access failure can originate from authentication, authorization, network rules, or request semantics. The monitoring view helps identify the response category, but the repair needs the actual context. An administrator should not rotate account keys merely because a request failed. The client may use Microsoft Entra authorization, or a firewall may be rejecting the path. Identify the authentication method and network route, then correlate the service response with the client's operation. A focused repair preserves working clients and avoids unnecessary credential changes.

Inspect network health across resources

Network-focused monitoring views combine resource health and telemetry for components such as virtual networks, load balancers, gateways, and application gateways where supported. They help locate a resource or path associated with an incident. A healthy resource state does not guarantee that a particular packet is allowed or that an application listens at the destination. Network configuration, guest firewalls, DNS, and application state still need examination. The view is a starting point for a targeted diagnostic, not a replacement for one.

For a load-balanced workload, compare frontend behavior, backend health, probe results, and backend application evidence. If all probes fail, inspect the probe protocol, port, path, and the backend's actual response. If only one backend fails, compare its configuration and recent changes with healthy peers. A general network health indicator can miss a misconfigured application endpoint. The administrator should move from the aggregate view to the smallest failing relationship that can be tested safely.

Customize views without hiding assumptions

Workbooks and dashboards can preserve useful scopes, charts, and queries for a team. Give a customized view a clear purpose and show its time range and filters. A display that always selects

yesterday's incident interval can mislead someone expecting current health. Threshold colors should reflect the team's operating expectations rather than be treated as universal definitions of acceptable service. Document any important calculated value so another administrator can inspect how it is derived.

Access to the visualization and access to the underlying telemetry are related but separate concerns. A shared workbook does not necessarily grant every viewer permission to read all its data. A reader may see missing panels because their scope differs from the author's. Test the intended viewing role and avoid exporting sensitive logs into a broadly shared static report as a workaround. A useful operational view respects the same access boundaries as the services it describes.

Move from observation to a testable hypothesis

Insights is most effective when it narrows the next question. A VM panel can identify memory pressure worth investigating inside a process. A storage view can identify one API with unusual failures. A network view can identify one backend that fails a probe. Each finding should lead to a testable hypothesis and a targeted check. This approach avoids making broad configuration changes merely because a dashboard contains a red indicator. The next section develops the network tools that support those precise checks.

17.5 Investigating with Network Watcher and Connection Monitor

Select a tool for the question

Network Watcher provides diagnostic and monitoring capabilities for Azure infrastructure networking. Its tools answer different questions about topology, security rules, routes, connectivity, and traffic. Choosing the right tool begins with a precise failure statement. "The application is slow" is too broad. "A VM in the application subnet cannot establish a TCP connection to the database endpoint on its configured port" identifies a source, destination, protocol, and operation. That statement supports a useful diagnostic sequence and makes the result easier to interpret.

Network Watcher is a regional capability, and individual tools can have agent, permission, resource-state, or storage prerequisites. Azure can enable it automatically in relevant regions, but availability of the service does not prove every diagnostic is configured. Check the tool's current requirements before running it. Some operations involve a VM extension or data capture that can produce sensitive information. Use a scoped test on an authorized workload, and record what is being measured. A diagnostic should be designed to reduce uncertainty without collecting unrelated traffic.

Start with topology and effective configuration

A topology view helps identify resource relationships, such as network interfaces, subnets, and virtual networks. It is useful for orientation in an unfamiliar environment. It should not be mistaken for proof that every relationship is currently carrying traffic successfully. A connected resource can still be blocked by a security rule, an incorrect route, a guest firewall, or an application that is not listening. Use topology to locate the intended path, then inspect the effective configuration along that path.

Effective security rules combine applicable network security group rules at the relevant interface and subnet. They help explain which configuration Azure evaluates for the VM. Rule priority, direction, protocol, addresses, and ports all matter. A rule that appears to allow traffic can be overridden by a higher-priority matching deny. Compare the actual packet characteristics with the effective rules rather than searching only for a rule whose name sounds relevant. Friendly rule names are descriptions; matching fields determine behavior.

Use IP flow verify for filtering questions

IP flow verify evaluates whether a specified packet would be allowed or denied by applicable network security rules for a supported VM context. It can identify the rule responsible for the result. This is a focused question about security-rule evaluation. It does not prove that the destination application is running, that DNS resolved correctly, or that every device along the path permits the connection. A result of allowed narrows the investigation; it does not complete an end-to-end application test.

Supply the correct direction and endpoints. For an outbound client connection, the destination port is commonly the service port, while the source port is a client-side ephemeral port. Reversing those fields can test a different packet from the one that is failing. Use the actual addresses after name resolution and account for the path being examined. If the result identifies an unexpected deny, compare that rule with the intended policy before modifying it. The repair should preserve the reason for the surrounding restrictions.

Inspect routing with next hop

Next hop helps determine where Azure expects to send traffic toward a destination from a supported VM. The result can reveal an unexpected virtual appliance route, an Internet path, a virtual-network path, or another applicable next-hop type. It is valuable when a custom route table changes traffic flow. A route existing in a table does not prove it is the effective route for every destination. Prefix selection, route propagation, and other routing behavior determine the path actually selected.

An expected next hop does not prove that the next device will forward the traffic successfully. A virtual appliance can be stopped, misconfigured, or unable to return the packet. The destination can have a missing return route. Continue the investigation along the identified path. For example, if traffic is routed to a firewall, inspect the firewall's applicable policy and logs. Replacing the route with a direct path may make a test work while bypassing an intended security control. Diagnose the failing component rather than removing it without understanding the design.

Test connectivity at the right layer

Connection troubleshoot provides a targeted connectivity test for supported source and destination combinations. The result can help locate configuration or reachability issues along the path. Choose the protocol and port that represent the application's requirement. A successful ICMP test does not establish that TCP on the application port works. An HTTP success at one path may not prove that an authenticated transaction works. Match the diagnostic to the layer being investigated, and state the scope of its conclusion in the incident record.

If an IP-based test succeeds while a hostname-based request fails, inspect DNS resolution from the source environment. If TCP succeeds but the application returns an error, inspect TLS, authentication,

and application logs. If no connection can be established, inspect security rules, routes, destination listening state, and guest firewalls. This progression avoids treating every failure as a network-security-group problem. The network can deliver packets successfully while the application rejects their contents for a legitimate reason.

Use Connection Monitor for ongoing evidence

Connection Monitor repeatedly tests defined source-to-destination relationships and records measurements such as reachability and latency according to the configured protocol. It is useful for intermittent problems that a single manual test can miss. Define a source that represents the actual workload path, a relevant destination, and appropriate test settings. A test from a VM in another region or subnet may measure a different experience. Monitoring should observe the path whose behavior matters to the service.

Sources require the appropriate supported monitoring setup, such as the Network Watcher extension for Azure VMs. Destinations can be supported resources or addresses and hostnames, and they do not always require an agent. The exact prerequisites depend on the source and test type. Inspect them in current guidance before enabling a monitor. The retired classic Connection Monitor should not be used as the basis for a new deployment. Existing old instructions need to be translated into the current supported resource model.

Test groups organize related endpoints and configurations. A meaningful group can represent an application tier's dependency path or a set of regional checks. Thresholds should reflect the expected behavior of that path and protocol. A latency value between distant regions should not be judged by the same expectation as a local subnet connection without considering the design. When a monitor reports a problem, compare it with application incidents and recent configuration changes. Correlation helps determine whether the measured path explains the user-facing symptom.

Capture traffic only when needed

Packet capture can provide detailed evidence when simpler configuration and connectivity checks are insufficient. It can also collect sensitive payloads or metadata and generate substantial files. Filter the capture to the relevant addresses, ports, protocol, and bounded duration where the tool supports those controls. Store the result in an authorized destination and handle it according to the data it contains. A full unrestricted capture should not be the default response to every network issue. Start with the smallest observation that can answer the question.

Traffic-flow logging provides another form of evidence, with supported modern virtual-network flow logging preferred over obsolete instructions for new NSG flow-log deployments. Confirm current service lifecycle guidance and feature support before enabling a collection path. Flow records and packet captures have different levels of detail. A record showing an allowed flow does not establish the success of every application transaction, while a packet capture may still need application context to explain an error. Select the evidence according to the investigation rather than its apparent richness.

Close the diagnostic loop

After a configuration repair, repeat the original failing test from the same relevant source and compare the result. Then test an intentionally disallowed path to ensure the change did not broaden access

beyond the requirement. Record the exact changed rule or route and the evidence supporting the repair. A service that becomes reachable after several simultaneous edits is harder to operate because the effective cause remains unknown. A focused correction produces both a working path and an explanation another administrator can use.

Remove temporary captures, test rules, and unnecessary monitors after the exercise, while preserving a concise authorized record of the finding. Monitoring has a lifecycle and can incur storage or ingestion costs. A diagnostic created for a one-time incident should either become an intentionally maintained control or be retired. This completes the chapter's central progression: collect the right signal, interpret it accurately, and use it to support a specific operational decision.

Guided lab

Follow a management event through logs and alerting

This exercise creates a small monitoring workspace and observes harmless changes to a dedicated resource group. It requires a practice subscription, permission to configure subscription activity-log export and monitoring resources, and access to query the destination workspace. Do not change an organization's existing diagnostic settings. Add a clearly named temporary setting only in a subscription where that change is authorized. Log ingestion, retention, and alert evaluation can incur charges. No live execution occurred during manuscript preparation.

1. Create `rg-compass-ch17` and a Log Analytics workspace named `law-compass-ch17` in a supported region. Record the workspace resource ID and inspect its pricing and retention settings.
2. From the subscription Activity log's diagnostic settings, add a temporary setting named `compass-ch17-activity` that sends the Administrative category to this workspace. Record the setting's subscription-level location so it can be removed later.
3. Add or update a harmless tag on `rg-compass-ch17`, such as `exercise=monitoring`. Record the time. For the later resource-group write alert, use the resource group's supported create-or-update operation, such as `az group update --name rg-compass-ch17 --tags exercise=monitoring`, and confirm its actual operation name in Activity log. Do not deliberately attempt unauthorized access to generate a test event; authentication and authorization rejections do not reliably produce the activity-log records used here.
4. Inspect the event in the subscription Activity log. After allowing for export and ingestion delay, run the first query in section 17.2 from the workspace. Confirm the resource group, operation, status, and time. If no result appears, follow the collection and scope checks before changing the query indiscriminately.
5. Create a notification-only action group using an address controlled by the reader. Use its supported test function, then confirm receipt. Do not add shared production recipients or automated resource-changing actions.
6. Create an activity log alert for the practice group's resource-group write operation, selecting the operation offered by the portal and a successful status. Associate the test action group. Change the harmless tag again and inspect the alert record and notification separately.
7. Add a narrowly scoped, short-lived alert processing rule that suppresses action groups for this exercise alert during a defined window. Repeat the tag change. Verify that the matching alert can

still exist while its action is suppressed. Inspect processing details instead of treating a missing email as proof of a missing alert.

8. Remove or expire the processing rule and repeat the event once more to confirm ordinary action handling resumes. Save the query and a concise record of the observed stages.

For metric practice, inspect a platform metric on an existing authorized practice resource, such as host CPU on the VM from Chapter 12. If that VM was removed, recreate its documented practice configuration before this optional check and include it in cleanup. Compare average and maximum over the same interval, then change granularity and explain what changed in the chart. Do not invent values for an empty or unavailable metric.

Cleanup must remove the subscription-level `compass-ch17-activity` diagnostic setting before deleting the workspace, then remove the exercise alert, action group, processing rule, and dedicated resource group. A subscription diagnostic setting is outside the group's deletion boundary. Remove only the setting created for this exercise. Confirm that no temporary suppression remains active and that any optional recreated VM resources are removed according to their own lab instructions.

Chapter synthesis

Monitoring is a chain of evidence with distinct collection, interpretation, detection, and response stages. A graph needs an aggregation and scope; a query needs collected data; an alert needs a useful condition and an observable action path. Insights and Network Watcher help narrow questions, but their results must be interpreted within the layer they measure. Reliable administration follows a finding with a targeted test and records what the evidence actually establishes.

Review questions

17-Q1. A five-minute CPU peak disappears when a chart uses a long-window average. What should the administrator do before concluding there was no pressure?

- **A.** Delete the VM
- **B.** Assume averages always preserve peaks
- **C.** Disable monitoring
- **D.** Inspect a suitable granularity and maximum alongside service behavior

17-Q2. A new Log Analytics workspace contains no storage request records. What is the most relevant initial check?

- **A.** Whether the workspace name is short
- **B.** Whether the relevant resource logs were enabled and routed to that workspace
- **C.** Whether the storage account has more tags
- **D.** Whether every user is Global Administrator

17-Q3. During maintenance, an alert processing rule suppresses the exercise action group. What result is possible?

- **A.** The alert record exists but the notification action is suppressed
- **B.** All source logs are deleted
- **C.** The monitored resource stops automatically
- **D.** The alert condition cannot be evaluated

17-Q4. A VM has host metrics but no guest performance data. What should be inspected?

- **A.** Only the VM display name
- **B.** The storage account's access tier
- **C.** The guest monitoring agent, configuration, association and destination
- **D.** The user's email signature

17-Q5. IP flow verify reports that a packet is allowed. Which conclusion is still unsupported?

- **A.** The tested security-rule evaluation allowed the packet
- **B.** A different packet could match another rule
- **C.** The result narrows the investigation
- **D.** The destination application must be running and accepting the request

17-Q6. A log query summarizes a count into one output row. The alert is configured to count output rows. What should be checked?

- **A.** Whether the rule should instead evaluate the numeric count column
- **B.** Whether the workspace should be deleted
- **C.** Whether KQL changes resource configuration
- **D.** Whether DNS uses a shorter hostname

17-Q7. A client fails intermittently when connecting to a specific dependency. Which tool is designed to collect ongoing evidence about that source-to-destination path?

- **A.** A resource naming policy
- **B.** A one-time screenshot of tags
- **C.** Connection Monitor
- **D.** A deployment slot swap

17-Q8. A request rejected before an ARM management operation begins is absent from Activity log. What is the correct interpretation?

- **A.** Nobody attempted the request
- **B.** Activity log is not a complete authentication and authorization audit source
- **C.** Every Azure log must be empty
- **D.** The workspace retention is necessarily zero

Further reading

- Metric aggregation
- Diagnostic settings
- Data collection rules
- Log Analytics tutorial
- Activity log
- Azure Monitor alerts
- Alert processing rules
- Current VM monitoring guidance
- Storage insights
- Network Watcher
- Connection Monitor

Technical references checked November 13, 2026.

Chapter 18: Recovering When Something Fails

Monitoring reveals that something has gone wrong; recovery restores a useful service or a usable copy of data. The administrator needs both capabilities. A backup configuration is incomplete until its recovery points can be found, restored, and verified. Disaster recovery adds the problem of operating in another location with the necessary networks, identities, and dependencies. This chapter turns those requirements into a deliberate recovery process.

Learning objectives

- Choose between Recovery Services vaults and Backup vaults for supported workloads.
- Connect backup schedules and retention to recovery requirements.
- Select and verify an appropriate restore operation.
- Explain Azure Site Recovery replication, test failover, failover, and reprotection.
- Interpret backup jobs, reports, and alerts without overstating recoverability.

18.1 Choosing Recovery Services Vaults and Backup Vaults

Start with the protected workload

Azure Backup uses different vault resource types for supported protection scenarios. A Recovery Services vault supports workloads including Azure virtual machines and SQL Server workloads inside Azure VMs. A Backup vault supports other data-protection scenarios, including supported blob and disk protection offerings and newer workload integrations. The correct choice follows the workload's current support matrix. The two vault names do not describe interchangeable containers into which any backup can be placed. An administrator should identify the data source and recovery operation before creating the vault.

A vault organizes protection configuration, recovery information, security settings, and operational views. The exact relationship between a vault and stored data depends on the workload and backup tier. Some protection uses operational snapshots or source-side capabilities, while other protection stores recovery data in a vault-backed location. A Backup vault resource existing in a group does not by itself prove that an isolated copy of every protected byte exists inside that vault. Read the specific workload's protection model and identify where recovery data resides.

This distinction is particularly important when comparing a snapshot-based operational recovery option with vaulted backup. A snapshot can provide fast recovery for a supported scenario, but its location and dependency on the source environment affect the failures it protects against. Vaulted protection can provide a different isolation and retention model. Neither label should be interpreted without the workload details. The administrator needs to explain which failures the selected design can survive and which source or regional dependencies remain.

Place the vault deliberately

Vault location, subscription, and resource-group organization should follow supported workload requirements and administrative ownership. For ordinary Azure VM backup, the Recovery Services vault is selected in a supported relationship with the VM's region and subscription. Restore options can have additional constraints or capabilities, including supported cross-region or cross-subscription scenarios. Do not assume that a flexible restore feature removes the initial protection requirements. Check the workload's current matrix for the actual combination being configured.

The vault's storage redundancy is a design decision. Locally redundant, zone-redundant, and geo-redundant options provide different resilience characteristics where supported. Their availability and change restrictions depend on the vault and workload. Select the intended option before enabling protection when the service requires that sequence. A replication label alone does not guarantee that an administrator can immediately restore in any chosen region. Cross-region recovery depends on supported features, configuration, and recovery-point availability in the secondary location.

Keep management organization distinct from recovery isolation. Placing a vault in a different resource group can help apply separate permissions and lifecycle controls. It does not automatically establish geographic separation or a complete security boundary. The actual protection comes from the service's data isolation, access controls, retention protections, and supported storage configuration. A separate group is useful when it expresses ownership and reduces accidental joint deletion, but its name is not evidence of a tested recovery design.

Separate administration from recovery authority

Backup roles define permissions for viewing, configuring, and operating protection. Restore operations can also require permission to create or modify target resources outside the vault. An identity that can inspect recovery points may not be able to create a restored VM in the target group. An identity that can manage a VM may not be authorized to delete protected backup data. This separation is useful because production access and backup destruction should not automatically be the same authority. Define the roles according to the actual duties.

The vault's own identity and access to related resources can matter for particular workloads and encryption configurations. Follow the current workload guidance for required assignments rather than granting broad access to every vault identity. A failure can involve the operator's permission, the vault or service identity's permission, or a target resource restriction. The error details and operation context help distinguish those actors. A role assigned to the human administrator does not necessarily repair an authorization failure performed by the backup service.

Understand deletion protection

Soft delete preserves deleted backup state for a recovery interval instead of immediately making deletion permanent. Its purpose is to provide a chance to recover from accidental or malicious removal. Current Azure Backup deployments increasingly enforce soft delete as part of the service's secure-by-default behavior. The exact rollout and vault-type behavior should be checked in the target region. A cleanup guide that assumes soft delete can always be disabled is not reliable. Protection settings can intentionally prevent immediate removal of backup data or the vault.

Immutability protects recovery points against supported operations that would reduce their protection or remove them prematurely. Enabling immutability and locking the setting have different consequences; a locked immutable configuration can be irreversible. This is a serious lifecycle decision, not a checkbox to select casually during a disposable lab. The protection is valuable when it matches an organization's retention requirements and governance. The exercise in this chapter does not require locking immutability. Readers should inspect the feature and understand its consequences before using it in a real environment.

Multiuser authorization and related security controls can separate sensitive backup operations from ordinary backup administration where supported. Their value comes from requiring an additional authorized role or process for high-impact actions. The design must also include a legitimate recovery path for administrators during an incident. A protective control that nobody understands can delay restoration or cleanup at the worst time. Document the identities, scopes, and approvals required by the selected service configuration, and test the permitted operational path without weakening the protection.

Encryption and key dependencies

Backup data is encrypted through the service's supported mechanisms. Platform-managed keys provide a default managed approach, while customer-managed key configurations add control and key-management responsibilities where supported. A customer-managed key must remain accessible through the required identity and vault arrangement. Disabling or losing access to a key can affect recovery even when backup data remains present. The key's lifecycle therefore belongs in the recovery plan. Protecting the backup while neglecting the key dependency can create a false sense of security.

A vault does not remove the need to consider source encryption and restore compatibility. Encrypted VMs or disks can have additional requirements for a supported restore method. The available portal choices depend on the source and recovery point. Review the relevant support matrix before promising a particular recovery path. A procedure tested with an unencrypted practice disk may not apply unchanged to a differently encrypted production workload. The important habit is to test the configuration actually used by the protected service.

Decision	What it affects	What to verify
Vault type	Supported workloads and operations	Current workload support matrix
Region and redundancy	Placement and recovery-location options	Supported target and available recovery points
Access roles	Who can configure, restore, or delete	Operator and service identities at each scope
Soft delete and immutability	Removal and retention behavior	Current enforcement and irreversible choices
Key management	Ability to use encrypted recovery data	Key availability and authorized access

Operational and vaulted protection can also expose different deletion boundaries. A source-side snapshot may remain associated with a resource group that an administrator can manage directly, while vaulted data follows the backup service's protected lifecycle. Review the exact workload design before treating either copy as independent of the source administrator. The protection objective should state which accidental or malicious actions the recovery data must survive. That objective helps select isolation and retention controls instead of assuming that the word backup alone establishes the required separation.

Create an inventory that supports recovery

Record the vault, protected items, policies, source resources, identities, and intended restore destinations. This inventory should make it possible to locate the right recovery points even if the source workload is unavailable. Names alone can be ambiguous, so include resource IDs and region information. Keep the record in a place accessible during the failure being planned for. A runbook stored only on the VM it is meant to restore is an avoidable dependency. The administrator needs the instructions and credentials through an independent supported path.

Finally, identify the cleanup implications before creating a practice vault. Removing the source VM does not remove its backup data. Stopping future backups can retain earlier recovery points, and deleting backup data can place it into soft-deleted state. Retained data and associated resources have their own lifecycle and potential charges. The correct cleanup record can therefore include a pending retention-expiry task instead of claiming immediate deletion. A transparent inventory is more useful than a broad resource-group deletion command that fails against protected dependencies.

18.2 Creating Backup Policies

Translate a recovery requirement into a schedule

A backup policy defines when supported backups occur and how recovery points are retained. The policy should follow an explicit recovery requirement. The recovery point objective, or RPO, describes the acceptable amount of lost recent data measured in time. The recovery time objective, or RTO, describes the acceptable time to restore useful service. A backup schedule influences potential data loss, but the actual latest usable recovery point determines what can be recovered after a failure. A scheduled job that repeatedly fails does not satisfy an objective merely because its interval looks appropriate.

Consider a small internal workload whose owner can accept losing changes since the previous night's backup. A daily schedule might fit its data-loss requirement if jobs reliably complete and the application is captured in a usable state. Another workload with frequent important transactions may need a more frequent supported backup pattern or complementary protection. The administrator should not choose the same policy for both simply to reduce the number of policies. Shared policies are useful when requirements are genuinely shared, not when differences are ignored.

Frequency and retention solve different problems

Backup frequency determines how often recovery opportunities are created. Retention determines how long selected points remain available. Frequent backups with short retention can protect recent mistakes while failing to provide an older clean state after a problem is discovered late. Long retention

with infrequent backups can preserve history while allowing substantial loss between points. Both dimensions should be chosen intentionally. The question is not only how quickly failure will be noticed, but how far back a reliable state may need to be found.

Policies can include different retention patterns for daily, weekly, monthly, or other supported recovery points. These patterns help balance recent recovery detail with longer-term history. The available options depend on the workload and policy type. Read the effective policy rather than inferring it from a friendly name such as `gold` or `daily`. A policy called daily can still have retention choices that do not meet the workload's requirement. The operating record should show the actual schedule, time zone, and retention behavior.

Understand policy and workload compatibility

Azure VM backup supports policy types with different capabilities, such as Standard and Enhanced policies. Their supported schedules, snapshot behavior, and VM compatibility differ. Select the type appropriate to the VM's configuration and the intended recovery requirements. Newer VM security configurations or disk arrangements can require a particular supported policy. Avoid copying an older policy example without checking the current support matrix. The portal's available choices and Microsoft's workload guidance should agree with the selected design.

An existing policy is an interface shared by every protected item assigned to it. Editing that policy can affect several workloads. Before changing frequency or retention, list the assigned items and evaluate their requirements. A change intended to reduce cost for a temporary VM can unintentionally weaken protection for an important server using the same policy. Create a separate policy when the lifecycle or retention requirements differ materially. Clear ownership makes policy reuse helpful instead of dangerous.

Choose timing with the workload in mind

The backup schedule should consider when the application is busy, when maintenance occurs, and how long backup operations usually take. A scheduled time is not a promise that every operation finishes at that exact moment. Data volume, change rate, source health, and service behavior influence completion. Monitor the duration and latest usable point. A backup that consistently extends into a busy period may need a different schedule or investigation into the workload's behavior. Do not treat longer duration as harmless without checking whether protection objectives are still met.

Time-zone settings must be explicit. A policy interpreted in one time zone can run at an unexpected local time for an administrator elsewhere. Daylight-saving changes can further complicate assumptions. Record the schedule's actual configured zone and compare job timestamps consistently. This matters when correlating a backup with application maintenance or a data corruption event. Selecting the wrong point because two logs use different time references can undermine an otherwise successful restore.

Capture a usable state

Recovery points can have different consistency characteristics. A crash-consistent point reflects a state similar to an abrupt stop, while an application-consistent point uses supported coordination to capture a state intended to be usable by the application. The exact behavior depends on the workload and its integration. Application consistency should not be assumed merely because the backup job has a

successful status. Inspect the recovery point type and the application's supported recovery behavior. Some workloads can recover correctly from crash-consistent storage; others need more specific coordination.

For virtual machines, the guest agent, backup extension, operating system, and application components can influence capture behavior. A failing extension or an unhealthy application consistency mechanism can affect the job. Diagnose the actual error rather than repeatedly increasing retention. Retention preserves points that exist; it does not repair the process that creates them. A supported, healthy source configuration is part of the policy's practical effectiveness even though it is not fully described by the schedule fields.

Separate snapshots and vaulted retention

Some backup workflows use an operational snapshot stage and a vault storage stage with different purposes and retention settings. A nearby snapshot can support a faster restore path, while retained vault data provides another recovery path. The administrator should identify which recovery point tier is being selected and what it requires. The existence of one tier does not guarantee every restore method is available in another region. A cross-region recovery can depend on replicated vault data rather than the source-side snapshot.

The policy should be evaluated against both restoration speed and protection depth. Keeping operational snapshots longer can improve a particular recovery workflow but can affect cost. Longer vaulted retention can preserve useful history but does not necessarily make a large restore instantaneous. The workload owner should understand the tradeoff in practical terms: how much data might be lost, which points remain, and how long the tested recovery procedure takes. A label such as enhanced is not a substitute for those measured outcomes.

Review changes to protection

Reducing retention or stopping protection can affect recovery availability. Security features such as immutability can block reductions that would remove protected points. A policy change must respect those controls and the organization's requirements. Do not disable protection merely to make a requested edit succeed. Determine whether the edit is appropriate and supported. If a temporary lab cannot be removed immediately because retention is enforced, record the remaining state and follow its supported lifecycle instead of trying undocumented deletion workarounds.

Stopping protection with retained data and stopping protection while deleting data are different decisions. The first stops future protection while preserving existing recovery points according to the service's behavior. The second begins the removal process, potentially including soft deletion. The administrator must select the intended outcome explicitly. An app retired from production may still need retained recovery history. A disposable exercise may no longer need it, but its enforced protection period can still delay final removal. The source resource's lifecycle and backup data's lifecycle are related but separate.

An on-demand backup has retention behavior that should be inspected separately from assumptions about the next scheduled point. The portal can request a retain-until date or apply supported policy behavior according to the workload. Record the selected value for the exercise. Creating a point immediately is useful for testing, but it should not accidentally extend a disposable lab's retained data

far beyond the intended period. Conversely, a production recovery test should not shorten protection merely to simplify cleanup. The policy and security controls remain authoritative for data that must be preserved.

Verify the policy by observing a job

After assigning a policy, confirm that the intended item is protected and that a backup actually completes. An on-demand backup can provide an initial recovery point without waiting for the next scheduled time where supported. Inspect the resulting point's timestamp, type, and retention information. Then verify that the next scheduled operation behaves as intended when the exercise or environment permits. The policy's configuration is an intention; job and recovery-point evidence show whether the intention is being fulfilled.

A useful policy review therefore includes its assigned population, schedule, retention, consistency expectations, latest successful point, and recent failures. It should also identify the tested restore method. This record makes protection understandable to another administrator and exposes gaps before an incident. The next section uses those recovery points to perform restoration, where selecting the correct target and verifying the application are as important as starting the restore job.

18.3 Performing and Verifying Restores

Define the recovery task

A restore should begin with a clear statement of what must be recovered. The task might be one accidentally deleted file, a damaged disk, an entire virtual machine, or an application service in a new location. Those tasks can use different recovery methods even when they rely on related backup data. Restoring an entire VM to recover one small file can create unnecessary downtime and overwrite valid recent work. Conversely, copying one file back may be insufficient when the application requires a consistent set of data and configuration. Select the smallest supported operation that satisfies the actual recovery requirement.

Identify the incident time and the last known usable state. The newest recovery point is not always the best choice. If corruption began before the most recent backup, that point can contain the same problem. Compare recovery timestamps with application events and the owner's observations. Where the failure's onset is uncertain, a restore into an isolated target allows inspection without overwriting the source. This preserves options while the administrator determines which point contains the required data. Recovery should reduce uncertainty before making irreversible replacements.

Inspect available recovery points

The protected item's recovery-point list provides timestamps and relevant types or tiers. Read those details rather than choosing by position alone. A point may be crash-consistent or application-consistent, and its restore capabilities can depend on the workload and storage tier. Some points may be available only through particular restore workflows. Confirm that the selected point meets the intended task and target constraints. A timestamp establishes when the captured state relates to the source; it does not prove the contents are correct for the incident being repaired.

Check the source's supported configuration and the chosen restore option. Azure VM backup offers methods such as creating a new VM, restoring disks, or replacing supported existing disks, with constraints that vary by VM and encryption configuration. A cross-region restore has additional requirements. The portal may expose different options for different recovery points. Treat those differences as a reason to consult the support guidance, not as a reason to improvise an unsupported conversion of the backup. The recovery path should remain within the workload's documented capabilities.

Prefer a separate target for verification

When practical, restore to a separate resource or isolated network first. This allows examination of the recovered state while preserving the current source. Use a distinct name and a clearly identified target group. The restored resource can incur compute, disk, network, and monitoring charges, so include it in the exercise inventory. An isolated target also prevents a recovered application from unexpectedly joining production processing or sending duplicate messages. The restored operating system may still contain startup tasks and credentials, so network isolation alone is not the whole application-safety plan.

For a VM restore, inspect the target network, subnet, public exposure, size, disks, and identities. A recovered disk does not automatically recreate every resource relationship exactly as intended. A chosen restore workflow may create basic resources while requiring further configuration. Check load-balancer membership, DNS, role assignments, managed identities, and service access where relevant. The restore job's success should be interpreted within its documented scope. A working VM resource is one component of a recovered application, not proof that all external relationships have been restored.

Understand disk restoration choices

Restoring disks provides flexibility to attach recovered data to an existing machine or construct a VM with deliberate settings. It can be useful when the original VM configuration needs adjustment or a direct create-VM option is unsuitable. The administrator must then manage the resulting disks and any generated deployment information. Verify disk identity and contents before attachment to a sensitive workload. Attaching the wrong recovered disk can confuse mount points or application configuration even when the storage operation itself succeeds.

Replacing existing disks can be appropriate for a supported repair but has more direct impact on the current VM. The workflow can retain or create additional disk state according to the service's behavior, and the administrator must inspect what remains afterward. Do not assume original disks disappear or that all new disks are needed indefinitely. Record the disk IDs before and after restoration. Cleanup should remove only items confirmed unnecessary after successful recovery, preserving a reasonable return path during verification.

File recovery has its own boundaries

File-level recovery can retrieve selected files from a supported VM recovery point without restoring the entire machine. The service may provide a script or other mechanism to expose the recovery data temporarily. Follow the documented platform and security requirements, inspect the source of any recovery script, and use a suitable recovery host. A script downloaded for a restore can contain access information and should be handled as sensitive. Remove temporary mounts or sessions after the operation using the supported cleanup procedure.

Copy recovered files to a separate location before overwriting current files when the application permits that approach. Compare names, timestamps, sizes, and a checksum or application-level content check where useful. File presence alone is a weak test. A document can exist but contain the wrong version; a configuration file can be syntactically valid but reference an obsolete endpoint. The application owner should confirm that the recovered state addresses the incident. The administrator supplies the recovery mechanism and evidence, while domain knowledge helps establish that the correct business data has returned.

Monitor the restore operation

Restore jobs can take time and can involve several stages. Inspect progress and errors through the relevant backup or vault view. If a job fails, determine whether any target resources were created before the failure. A VM creation step can fail because a size is unavailable while recovered disks remain present. Repeating the entire operation without inspecting partial state can create additional resources and costs. Use the job details to identify which stage needs correction and preserve the relevant operation information for further investigation.

Required permissions can span the vault and target scope. The operator may need to read recovery points and request restore, while the service needs access to create or use target resources. Storage staging or encryption dependencies can introduce additional requirements. A permission failure should be traced to the specific actor, action, and scope. Granting a role only on the vault will not necessarily permit resource creation in another group. Likewise, broad target-group access does not automatically authorize every backup operation.

Verify the recovered service

Verification should be planned before the restore so that success is not defined after seeing the result. For a simple lab VM, create a harmless marker file before backup and record its content. After changing the marker, restore the earlier point into an isolated target and compare the recovered file. This demonstrates that the selected point represents an earlier state. It does not demonstrate application consistency for a database or a complete business recovery. Keep the test's conclusion proportional to the workload actually exercised.

For an application, useful checks include startup, authentication, a representative read, a controlled write where safe, and connectivity to dependencies. Confirm that the application uses the intended data source rather than accidentally reconnecting to the original production service. A restored web server can appear healthy because it still reads a live database, concealing that the database was never included in the test. Document which components were recovered and which were reused. This makes the measured recovery time meaningful.

Measure the result and preserve evidence

Record when the recovery decision began, when the restore operation started, when it completed, and when the service passed verification. These intervals distinguish service execution time from the full operational recovery time. Time spent locating credentials, choosing a point, or fixing a missing network can dominate the result. A realistic RTO assessment includes those activities. The backup service's job duration is useful, but it is not automatically the time a user waited for the service to become usable again.

Also record the actual recovered point relative to the incident. That difference helps assess the achieved data-loss window. A job can complete quickly while recovering an older point than the workload's objective permits. The operation is technically successful but does not meet the recovery requirement. This distinction should lead to improving protection frequency, reliability, or the wider design rather than presenting every successful restore as an unqualified success.

Clean up without losing the required state

After verification, decide whether the recovered target becomes the active service or remains a test artifact. If it becomes active, update the operating inventory, monitoring, backup protection, and access relationships. If it is a test, remove temporary compute and disks after preserving the authorized evidence. Do not delete the backup points simply because one test succeeded. Their retention follows the policy and recovery requirement. The test confirms that a path worked for a particular point and configuration; future recoverability still depends on continued protection and periodic review.

18.4 Planning Site Recovery and Regional Failover

Replication and backup serve different needs

Azure Site Recovery supports replication and recovery of supported workloads so they can run at a recovery location after a disruption. For Azure-to-Azure VM scenarios, it coordinates replication to another region and the creation of a recovered VM during failover. This can support a shorter service-recovery path than rebuilding everything from a backup alone. It does not replace backup retention. Replication can carry an unwanted change or corruption toward the replica, while retained backups provide historical recovery choices. A sound design considers both continued service and recovery of an earlier usable state.

The RPO reflects the age of usable replicated data relative to the source, while the RTO includes the time needed to make the service usable at the destination. Replication health and lag influence the first objective. Target capacity, networking, dependencies, and operational procedures influence the second. A protected status is useful evidence, but it is not a guarantee of zero data loss or instant failover. The administrator needs to inspect actual replication health and test the complete recovery path.

Prepare the destination before an incident

The recovery region needs a supported network design, suitable VM capacity, and access to required dependencies. The target subnet, address plan, security rules, and routing should be chosen deliberately. A restored VM can be running but unreachable because its expected DNS name still points to the source region. It can also reach users while failing to reach a database or identity dependency. Document the intended application path at the recovery location and identify which components are replicated, precreated, or restored through another mechanism.

Region selection must satisfy current Site Recovery support requirements for the source and destination, including operating system, disk, encryption, and network configuration. Some VM features have specific constraints. Check those requirements before promising that an existing workload can simply be protected without modification. The replication setup can create supporting resources such as cache

storage and target disks. Include those resources in the design and cost inventory. They remain part of the protection system even while the target VM is not serving production traffic.

Source health also matters. The required replication components and connectivity must remain functional. A network rule that blocks replication traffic can increase lag while the source application continues working normally. Monitoring should detect that loss of recovery readiness before an outage occurs. Treat replication as a service path that needs its own health checks. The absence of a current customer-facing problem does not mean the disaster-recovery arrangement is healthy.

Use recovery points deliberately

Failover workflows offer supported recovery-point choices, such as a recent processed point or an application-consistent point. The newest available option can require different processing time from a point already prepared. An application-consistent point can be older than the latest replicated data. The choice balances the incident's needs, data consistency, and recovery time. Read the timestamp and available disks associated with the point. A disk added after an earlier point was created will not magically appear in that older recovered state.

The right point depends on the cause of the incident. A regional infrastructure outage may favor the most recent usable replicated state. An application corruption event can require a point before the corruption reached the replica. That second scenario might be better served by backup recovery depending on retention and consistency requirements. An administrator should not assume that the default selected point is correct for every failure. The recovery decision needs the incident timeline and the workload owner's understanding of usable state.

Test failover proves the recovery path

A test failover creates a test recovery instance without replacing the production workload's ordinary role. It should use an isolated nonproduction network designed for the exercise. Isolation avoids address conflicts and reduces the risk that a test copy performs production work. The test still creates real resources and can incur charges. Confirm the target configuration, launch the supported test workflow, and inspect the resulting VM and application. A successful test job establishes less than a successful application check, so both are needed.

Test isolation must consider outbound behavior as well as inbound exposure. A recovered server may start scheduled tasks, send email, process a queue, or connect to a production database using its copied configuration. Prevent those unintended effects through the test design. Use harmless test data or controlled dependency substitutes where appropriate. Record any dependency deliberately unavailable during the drill so that its absence is not confused with a replication failure. The test should make its boundaries clear enough to support an honest conclusion.

After the drill, use Site Recovery's supported test-failover cleanup operation. This records the completion of the exercise and removes the associated test resources according to the workflow. Inspect for any manually added resources that the cleanup does not own. Preserve the test notes, recovery-point timestamp, verification results, and measured duration. Leaving a test failover open indefinitely can interfere with future drills or create unnecessary running resources. The exercise is complete only when its result and cleanup state are understood.

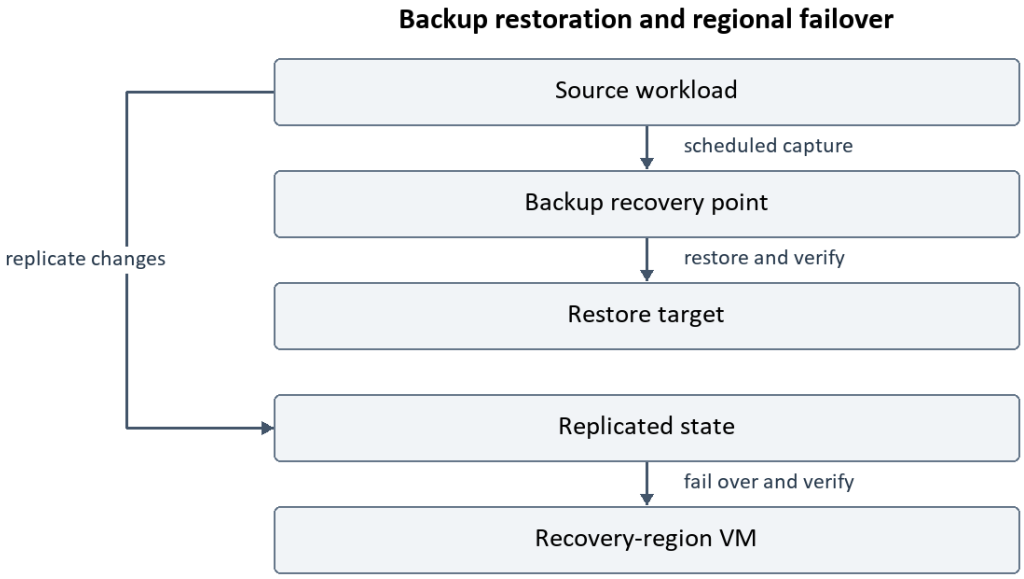


Figure 18.1 Protection and recovery paths

Retained backups preserve recoverable historical state. Site Recovery maintains a supported regional replication path. Both need verified target configuration, and replication does not replace retention of earlier usable data.

The diagram separates retained backup recovery from a regional replication path. Both require verification at their target. A healthy replication stream supports failover readiness, while a retained recovery point supports restoration of captured state. Their different purposes explain why an organization can need both even when the same VM is involved.

Fail over with an incident-specific decision

A real failover changes the service's active location and should follow the documented incident decision. Inspect the source's availability, replication health, chosen point, and target readiness. Where supported and appropriate, request a source shutdown before failover to reduce the risk of concurrent active copies and improve data consistency. A failed source shutdown does not establish that all final writes were replicated. The administrator should report the actual state rather than promise zero loss. A regional outage can prevent the source from participating in an orderly transition.

Prevent split operation in which both source and target independently process the same workload. This may require network, DNS, application, or operational controls beyond the VM failover itself. Traffic should move through the intended mechanism, and dependencies should be checked in the recovery region. Confirm that the recovered service uses the correct identity and endpoints. A VM successfully booted in another region can still require additional steps before users should be directed to it.

Commit and reprotect

After a successful failover and verification, the workflow includes a decision to commit the selected recovery state where applicable. Understand the consequence before committing, because it changes

the available recovery workflow. The exact steps depend on the scenario and current service behavior. The operating runbook should identify the verification required before that decision. Committing merely because the portal job is green can overlook an application that booted but cannot complete its essential task.

Reprotection establishes replication from the now-active recovery location back toward the intended protection location according to supported configuration. Without it, the workload can be running after failover while lacking the expected ongoing recovery protection. Inspect the new replication direction, target resources, and health. A later return to the original region is another controlled operation that needs readiness and verification. Failback is not simply deleting the recovered VM and turning the old machine back on, because the active data has changed since the original outage.

Exercise the entire runbook

A useful disaster-recovery runbook includes the trigger for invoking it, decision authority, prerequisites, recovery-point selection, target network, dependency order, traffic changes, verification, commit, reprotection, and cleanup. It should be accessible when the source region is unavailable. Test the sequence with the people and permissions expected to use it. A technically correct instruction can still fail operationally if the required identity cannot sign in or the team cannot locate the necessary resource IDs during an incident.

Measure what the drill actually demonstrated. A single-VM test confirms a narrower capability than a full multi-tier recovery. If a database, DNS change, or user access path was simulated rather than exercised, record that limitation. These notes identify the next improvement without diminishing the value of the completed test. Recovery confidence grows through specific evidence and corrected gaps, not through a general claim that replication is enabled.

18.5 Interpreting Recovery Reports and Alerts

Monitor protection as an operating service

Backup and disaster recovery need ongoing monitoring even when the protected applications are healthy. A source can serve users while its backups fail, its replication falls behind, or its recovery credentials become unusable. The administrator should therefore monitor both current production health and recovery readiness. These are related but different service objectives. A daily backup-success message can be reassuring, but it should be connected to an inventory showing that every required workload is actually included.

Begin with the protected population. Compare the resources that require protection with the items configured in the relevant vaults or recovery views. A report listing successful jobs cannot reveal an unprotected VM that never generated a job unless the reporting design also checks coverage. New resources, migrations, and retired workloads can change that population. Integrate protection review into the resource lifecycle. A deployment should not be considered operationally complete until its protection requirement has been satisfied or an explicit exception has been recorded.

Read jobs beyond their final label

A backup job records an operation and its result. Inspect the item, operation type, start and end times, error details, and resulting recovery point where relevant. A successful job establishes that the service completed its defined operation. It does not prove that an application-level restore has been tested. A warning can indicate a limitation worth addressing even when usable protection exists. A failed job requires examination of its cause and the age of the last successful point. The urgency depends partly on how far the workload has moved beyond its acceptable data-loss window.

Repeated failures can have different causes: unhealthy source agents, unavailable dependencies, permission changes, encryption-key access, unsupported configuration changes, or network restrictions. Use the error code and affected stage to select the investigation. Reassigning a policy without understanding the failure can obscure evidence and leave the root cause unchanged. A corrected job should be followed by a successful backup and inspection of the new recovery point. Marking an alert closed is not a substitute for restoring protection.

Interpret freshness and retention

Recovery-point freshness measures how recently usable state was captured. It should be compared with the workload's requirement, not merely with the calendar day. A backup completed last night may be acceptable for one system and far too old for another. Reports should make the relevant timestamps and time zone clear. A successful on-demand backup after several missed scheduled jobs can restore current protection while leaving a historical gap. That gap may still matter for recovery from a problem discovered later.

Retention reports show whether the expected historical points remain available. Compare the effective policy and actual points when investigating a concern. A policy changed recently may have different effects on existing and future recovery points depending on the workload. Security settings can prevent reductions that would remove protected data. The administrator should consult current behavior before interpreting every retained point as a cost error or every absent point as a service failure. The policy, change history, and item state provide the necessary context.

Use reporting destinations deliberately

Azure Backup reports use collected diagnostic data and Azure monitoring capabilities to provide longer-term views across protection resources. Configuring a report usually involves a destination such as Log Analytics and the appropriate diagnostic categories. The supported workloads and tables vary by reporting experience. A report opened immediately after configuration may lack historical data that was never exported. Allow for ingestion and reporting delay, and verify that the intended vaults are included. A blank workbook is not proof that no backups exist.

The reporting workspace has its own access, retention, and cost settings. A backup administrator may need permission to read the report data without needing authority to delete protected points. Keep that separation explicit. Reports can include resource identifiers and operational details that should not be shared indiscriminately. Export a concise result appropriate to its audience rather than an unrestricted dataset when a summary is sufficient. The same care used for general logs applies to recovery telemetry.

Interpret capacity and cost trends

Backup storage growth can reflect more protected items, larger source data, greater change rates, longer retention, or a different protection tier. A rising graph does not identify the cause by itself. Compare item count, policy changes, and workload activity over the same period. Reducing retention to make the graph fall can violate a recovery requirement. First establish whether the growth is expected and whether the selected protection model remains appropriate. Cost optimization should preserve the required recovery capability rather than optimize away the reason for keeping backups.

Retained data after source retirement is another common source of confusion. A deleted VM can still have backup data intentionally preserved in a vault. That state can be correct for a retention requirement or unnecessary for a disposable exercise. The inventory should state the reason and the planned lifecycle. A resource that no longer appears in the VM list is not necessarily absent from backup billing or recovery views. Review the protected item and retention state directly before declaring cleanup complete.

Configure alerts around useful conditions

Backup and recovery services provide built-in and configurable alerting for supported events. Route relevant alerts through an action group or supported notification configuration to a responsible operator. The recipient should understand the affected item, failure, latest usable point, and next diagnostic step. Avoid relying on a notification sent to a former administrator's address. Ownership changes should include a review of backup and recovery action routes, just as they include access permissions and application support contacts.

Severity should reflect the consequence of the gap. One transient failed job followed by a successful retry can have a different impact from repeated failures leaving a critical workload outside its RPO. The alert response should inspect current protection before deciding what action is needed. A historical failure message can remain relevant to audit even after readiness is restored. Distinguish the current condition from the investigation record so that reports neither overstate an ongoing outage nor conceal an unresolved pattern.

Monitor replication readiness separately

Site Recovery monitoring includes replication health, lag, protected-item state, and failover or reprotection jobs. A healthy backup does not prove replication is healthy, and healthy replication does not prove a historical backup restore is available. Review each mechanism according to its purpose. If replication becomes unhealthy after a source change, inspect new disks, network rules, credentials, extensions, or other configuration relevant to the supported scenario. A source VM's normal application response can conceal a broken replication dependency.

After failover, verify that protection has been reestablished in the intended direction. A report showing a successful failover job can distract from the fact that the active VM is not yet replicating back. The incident is not fully resolved until the agreed operating state includes both service availability and appropriate recovery readiness. Record any temporary exception with an owner and a concrete next action. This prevents an emergency configuration from silently becoming the permanent unprotected state.

Review restore evidence

Reports of successful backup jobs should be supplemented by records of actual restore tests. A useful test record identifies the source, point, target, method, duration, and verification result. It also states which dependencies were exercised and which were simulated. A test that restored a file correctly is valuable evidence for that task, but it does not demonstrate a complete regional application recovery. Maintain the distinction so that the next test targets the remaining uncertainty rather than repeating an already proven small step indefinitely.

Test frequency should follow the workload's importance and rate of change. A major application migration, encryption change, network redesign, or backup-policy change can justify a new targeted restore test. The objective is not to perform expensive full recovery drills without reason. It is to verify the aspects of the design that changed or remain uncertain. A focused test can reveal a missing permission or inaccessible key before a real incident makes the discovery costly.

Close protection gaps with evidence

A recovery review should end with a clear statement of coverage, current freshness, known exceptions, tested capabilities, and corrective actions. Avoid claiming that every service is recoverable solely because a dashboard is green. Identify the evidence behind the conclusion. If an exercise was only reviewed statically, say so. If a real restore passed its checks, preserve the relevant result without exposing sensitive contents. Honest evidence allows the next administrator to build on the work instead of repeating assumptions.

The combined monitoring and recovery discipline is straightforward: know what must be protected, verify that usable points exist, test the required restoration path, and watch for changes that invalidate the design. That discipline connects the whole book's topics. Identities permit the action, networks carry it, storage preserves the data, compute runs the recovered workload, and monitoring confirms what happened.

Guided lab

Restore a known file state from a VM backup

Use a disposable VM and data only. This lab has not been executed in Azure during manuscript preparation. It requires a supported practice VM, a Recovery Services vault in the supported source relationship, permissions to configure backup and restore into the target group, and access to inspect the guest. A vault-level backup role alone may not grant permission to create target compute and network resources. Compute, disks, snapshots, backup storage, and monitoring can incur charges.

1. Create `rg-compass-ch18-source` and `rg-compass-ch18-restore`. Recreate the small supported VM configuration from Chapter 12 in the source group, using the chosen region and a private administrative access path. Name it `vm-compass-ch18`. This is an explicit dependency on that chapter's VM setup, not an assumption that its earlier lab still exists.
2. In the guest, create a harmless text file named `compass-recovery.txt` containing `checkpoint-one`. Record its location and content. Do not place passwords or real business data in the file.

3. Create or select a dedicated Recovery Services vault named `rsv-compass-ch18` in a supported region and subscription relationship. Inspect redundancy and security defaults before enabling protection. Do not lock immutability for this disposable exercise.
4. Enable VM backup with a supported policy type for the VM's actual security and disk configuration. Select a supported schedule and the shortest retention suitable for the exercise without weakening enforced vault protections. Record the effective settings and expected retained-data lifecycle rather than assuming a named default has particular values.
5. Run an on-demand backup and wait for its successful completion and a usable recovery point. Inspect the timestamp and consistency information. Then change the marker file to `checkpoint-two` on the source VM.
6. Select the earlier point and restore to a new VM in the restore group using a distinct name and an isolated target network. Inspect the available restore options and support constraints before proceeding. Do not replace the source VM's disks for this first test.
7. Access the restored VM through a supported controlled path. Verify that the recovered marker contains `checkpoint-one`, and confirm the target is the restored resource rather than the original source. Record the restore duration and the separate time required to complete verification.
8. Inspect backup and restore job details, the protected item's points, and the resource inventory. Identify any extra disks, staging storage, or network resources created by the chosen restore workflow.
9. Create an empty Backup vault named `bv-compass-ch18` in the practice scope and inspect its offered workload types. Compare that selection with the Recovery Services vault without enabling an unrelated protection scenario merely to populate the interface.

For a controlled diagnostic, select a recovery point whose timestamp is earlier than the marker-file creation and explain why the marker would not be present. Do not perform an unnecessary second full restore merely to prove the timestamp relationship. The first restore supplies the practical evidence; this comparison develops recovery-point selection.

Cleanup has two parts. First, remove the restored VM and all confirmed exercise-only compute, disks, interfaces, and staging resources. Then stop protection for the disposable source using the intended supported data-removal option, understanding that deleted backup data can remain soft-deleted. Remove the source VM and its exercise resources after backup protection state is accounted for. Follow the vault's supported dependency-removal workflow; do not assume that deleting the group or disabling soft delete is possible.

If enforced soft delete or retention prevents immediate vault removal, record the remaining items, expiry or review date shown by the service, and any ongoing charges requiring monitoring. Complete final removal when the service permits it. Do not claim that cleanup is finished while protected or soft-deleted state remains. This delayed-removal possibility is part of the exercise's cost and lifecycle planning.

Chapter synthesis

Recovery is demonstrated by usable restored state, not by configuration alone. Vault selection, policy design, permissions, retention protections, and target readiness all affect the result. Site Recovery adds a regional operating path that must be tested and reprotected after failover. Clear records of actual

recovery points, tested procedures, and unresolved limitations make the environment easier to restore when ordinary operations are disrupted.

Review questions

18-Q1. An administrator must protect an ordinary supported Azure VM with Azure Backup. Which vault type is used for that workflow?

- **A.** A Key Vault secret container
- **B.** A Recovery Services vault
- **C.** A container registry repository
- **D.** A Log Analytics workspace alone

18-Q2. A backup runs frequently but retains points only briefly. A corruption problem is discovered after all clean points expire. Which policy dimension was insufficient?

- **A.** Resource naming
- **B.** Portal color selection
- **C.** VM display language
- **D.** Retention history

18-Q3. The latest backup contains the same corruption as the source. What is the best recovery-point decision?

- **A.** Investigate an earlier usable point and verify it in a suitable target
- **B.** Always restore the latest point because it is first in the list
- **C.** Delete every older point
- **D.** Assume backup cannot help any corruption incident

18-Q4. A Site Recovery test failover will start a copy of a production VM. Which target choice best supports a controlled drill?

- **A.** The production subnet without reviewing duplicate behavior
- **B.** An arbitrary public endpoint
- **C.** An isolated nonproduction network with controlled dependency access
- **D.** The source VM's name and address without checks

18-Q5. A failover succeeds and the recovered VM now serves users. What recovery-readiness step remains important?

- **A.** Assume all future failures are covered automatically
- **B.** Reprotect and verify replication in the intended direction
- **C.** Delete the source region's records immediately
- **D.** Disable all monitoring

18-Q6. A report lists successful backup jobs for every configured item. What additional check is needed to assess coverage?

- **A.** Whether all item names have the same length
- **B.** Whether every job started at midnight
- **C.** Whether the workbook uses a dark theme
- **D.** Whether every workload requiring protection is actually configured

18-Q7. A practice vault cannot be deleted because protected data remains soft-deleted under enforced retention. What is the appropriate response?

- **A.** Record remaining state and complete removal through the supported lifecycle when permitted
- **B.** Promise immediate cleanup is complete
- **C.** Use undocumented deletion bypasses
- **D.** Assume deleting the source VM removes every recovery point

18-Q8. A restore job completes in ten minutes, but users regain service after an hour of network and application repair. Which duration best represents the observed full service recovery time?

- **A.** Only the ten-minute job duration
- **B.** Zero because the job succeeded
- **C.** The full interval until the service passed its required verification
- **D.** The backup retention period

Further reading

- Recovery Services vaults
- Backup vaults
- Azure VM backup
- Azure VM restore options
- Secure-by-default soft delete
- Immutable vaults
- Site Recovery test failover
- Site Recovery failover and reprotection
- Backup reports

Technical references checked November 13, 2026. Protection defaults and cleanup behavior must be checked in the target vault and region before running the exercise.

Chapter 19: Operating a Complete Azure Environment

The previous chapters separated services so their behavior could be examined carefully. This project brings several of them together. A Linux virtual machine will use a managed identity to write a small blob through a private endpoint. The environment will collect storage logs, demonstrate a controlled access failure, and recover earlier blob contents. Each result must be supported by evidence from the actual execution.

Learning objectives

- Turn a small operational requirement into resources, permissions, and verification criteria.
- Deploy a repeatable environment with explicit network and identity boundaries.
- Distinguish deployment success from successful data access.
- Diagnose a controlled permission failure without weakening unrelated controls.
- Restore known contents from a retained version and remove the practice environment.

19.1 Turning Requirements into a Deployment Plan

Define the service to be demonstrated

The project represents a small internal processing workload. Its compute identity must write and read a text object in a designated container. Storage must not accept public-network data access. The workload must reach storage through a private endpoint, and the access decision must use Microsoft Entra authorization rather than a stored account key.

The test object contains a fixed sentence followed by a newline. Its purpose is to provide an unambiguous verification result. A successful HTTP response alone is weaker evidence than retrieving the expected contents. The probe compares the returned bytes with the expected teaching data and reports whether they match.

The workload is deliberately small. It does not serve inbound application traffic, require a load balancer, or use a container platform. Those services have already been taught separately. Adding them here would increase cost and dependencies without improving the identity-to-storage test that defines the project.

The recovery requirement is equally specific. After the current blob contents are deliberately changed, an earlier retained version must supply the known original contents. The recovery operation will write those contents as the new current version. It demonstrates content recovery from version history; it does not claim to restore every metadata property or an entire application.

Identify the resources and boundaries

The companion Bicep file creates one virtual network with separate workload and private-endpoint subnets. A network security group blocks inbound connections to the workload subnet. The VM receives

a Standard public IP address for an explicit outbound connectivity method, while its inbound ports remain blocked by the security group.

This outbound arrangement is a deliberate learning tradeoff. The VM agent needs suitable connectivity to report Run Command results. The public IP provides an explicit outbound mechanism without adding a NAT gateway to this small exercise. A production design may use centralized egress, a NAT gateway, or another controlled method. The lab does not equate a public IP with permission for inbound access.

The storage account disables public network access, anonymous blob access, and Shared Key access. A Blob private endpoint provides the data path. A private DNS zone and virtual-network link allow the VM to resolve the ordinary storage hostname to the private endpoint's address. The application continues using the normal service hostname.

The VM has a system-assigned managed identity. A Storage Blob Data Contributor role assignment at the evidence container gives that identity the data operations needed by the exercise. The assignment does not grant the VM authority to change the whole Azure environment. Code running on the VM can use its managed identity, so access to the VM remains a meaningful security boundary.

A Log Analytics workspace receives Blob service read, write, and delete logs through a diagnostic setting. These records provide additional evidence about operations and failures. The existence of the workspace is not evidence that a particular request has already arrived in it. Generate activity and allow for ingestion before drawing conclusions.

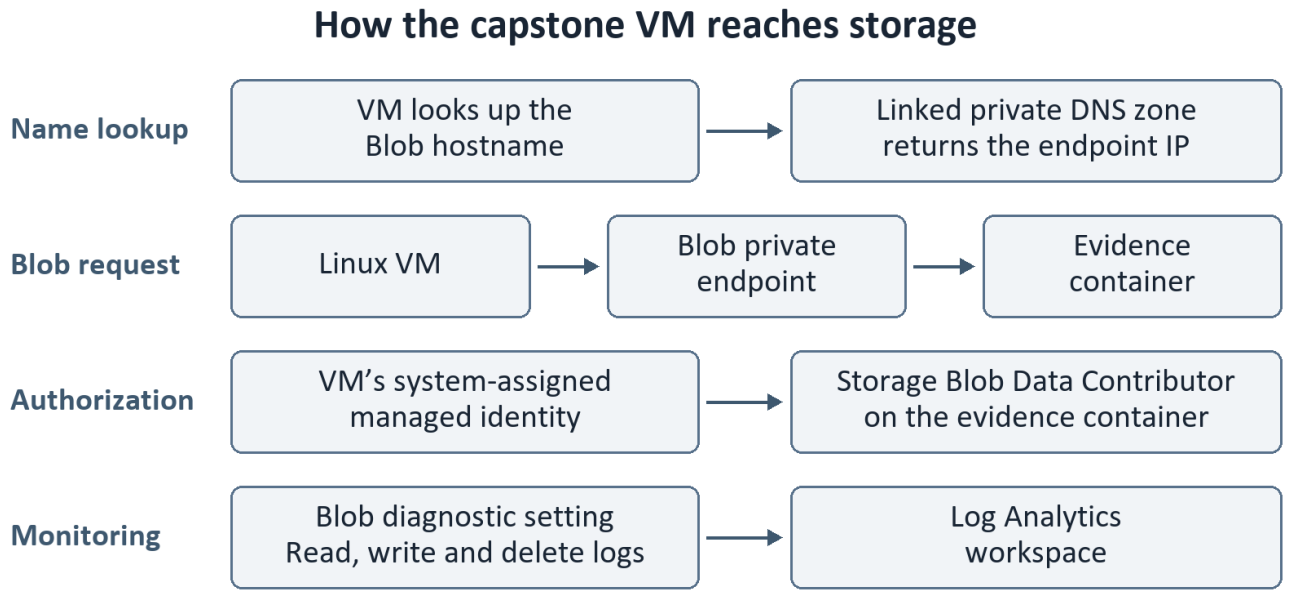


Figure 19.1 How the project VM reaches Blob Storage

Record prerequisites before deployment

The deployment identity needs permission to create the resources and to create the container-scoped role assignment. Contributor alone does not supply role-assignment write permission. Use an appropriately authorized practice identity or have a designated administrator perform the assignment. An organizational deny assignment or policy can still constrain the deployment.

The subscription must support the selected region, VM size, image, private endpoint, and workspace. The default size in the template is a modest example, not a guarantee of capacity. Check quota and availability, then pass a supported size if required. The template's Ubuntu image reference is explicit so the operating-system assumption can be inspected.

Provide an existing SSH public key. The deployment uses it to configure the guest account and disables password authentication. The private key must remain outside the parameter file and deployment record. Although the lab does not open SSH, a valid public-key configuration is still required for the selected VM provisioning design.

The storage account name must be globally available and meet the service's naming rules. Select a distinct lowercase name rather than copying one from a printed example. The template validates length; Azure also checks allowed characters and global availability. A local compilation cannot establish that a chosen name is still free.

Separate local checks from cloud checks

Compile the template before deploying it. A successful compilation establishes that Bicep can produce an ARM template with the checked syntax and type information. It does not prove that the subscription has sufficient quota, the identity can create a role assignment, or a policy permits the resources.

An Azure deployment validation or what-if operation evaluates additional aspects in the selected context. What-if previews expected changes and can help identify an unintended target. It is not a guarantee that every runtime service condition will remain unchanged through deployment. Read its output and investigate unexpected changes.

The companion files include the Bicep source, compiled ARM template, Python probe, and a PowerShell wrapper for Run Command. Local syntax and logic checks are recorded separately from live execution. The edition's verification record must remain honest about which checks have actually been performed.

Establish acceptance criteria

Write the expected evidence before the first creation command. This prevents a partial success from quietly becoming the new definition of success. The following matrix links each requirement with a meaningful observation.

Requirement	Evidence from the exercise
Correct deployment target	Resource identifiers contain the intended subscription and practice group
Private Blob path	The VM resolves the Blob hostname to the private endpoint address
Identity-based data access	The VM obtains its managed-identity token and completes the known write/read operation

Requirement	Evidence from the exercise
Public storage path disabled	Account configuration shows public network access disabled; an outside data test is rejected where performed
Monitoring enabled	Relevant generated operations appear in the configured storage log destination
Content recovery	A read fails after deliberate change, then passes after restoration from the recorded earlier version
Cleanup complete	The dedicated group is absent and separately retained resources have been accounted for

Do not infer one row from another. A private address does not prove the identity is authorized. A role assignment does not prove DNS works. A successful read does not establish that diagnostic logs have been delivered. The project is complete only when its required observations have been collected or a limitation is explicitly documented.

19.2 Establishing Identity and Governance

Prepare the practice boundary

Use a dedicated resource group, rg-compass-ch19. Inspect the selected subscription and confirm that the group name is unused or belongs exclusively to this exercise. If another environment already uses it, choose a distinct supported name and pass that name consistently to the deployment and probe wrapper.

Tag the group with Purpose=Training, Book=Compass, and Chapter=19. The template tags its deployed resources explicitly. This avoids assuming that resource-group tags automatically propagate to members. The tags help identify the resource set, while the deployment record and resource identifiers provide the authoritative inventory.

Before proceeding, inspect applicable location and resource-type policies. A policy denial is a useful indication that the design must fit the organization's allowed configuration. Do not disable a broad policy to make a training deployment succeed. Select a permitted environment or adapt the parameters and design under the appropriate authority.

A budget can provide notifications about spending in the chosen scope, but it does not automatically stop resources. Review current prices for the selected region and services before deployment. The VM, OS disk, public IP, private endpoint, storage, and log ingestion can incur charges. Keep the environment only as long as the exercise requires.

Inspect the identities involved

There are two main identities in the project. The deployment identity creates and configures resources. The VM's managed identity performs the blob data operation. These identities should not be treated as interchangeable merely because the deployment identity created the VM.

The system-assigned identity exists in association with the VM. The template refers to its principal identifier when assigning the data role. The principal type is declared as ServicePrincipal in the role-assignment resource. This communicates the kind of principal and avoids requiring a directory name lookup to identify it.

The role assignment is scoped to the evidence container. It gives the VM the data operations needed for creating, reading, changing, and recovering the teaching blob. It does not grant a subscription-wide resource-management role. If the workload needed only reads, a narrower data role would be the appropriate design.

The learning choice of a system-assigned identity is convenient because the VM and its workload share a lifecycle. A user-assigned identity would have a separate resource lifecycle and different reuse considerations. The project does not require that added independence.

Examine the deployment's permission requirements

Creating a role assignment is an administrative capability distinct from deploying a VM or storage account. A failure at the role-assignment stage can leave other resources successfully created. Inspect the deployment operations rather than assuming that a failed overall deployment rolled everything back.

If an authorized administrator supplies the role separately, preserve the same principal, role definition, and container scope. Record the resulting assignment identifier. The controlled failure exercise later removes only that known assignment, so a precise identifier matters.

The built-in role's name is human-readable, while the role-definition identifier is the durable reference used in the template. The relevant identifier is recorded in the source and can be checked against Microsoft's built-in role documentation. A role name typed incorrectly in a manual command is a different problem from an assignment made at the wrong scope.

An inherited broader grant can affect a negative test. If the VM identity receives equivalent storage permission from another scope, removing the project's container assignment will not necessarily remove its effective access. Before interpreting the failure exercise, inspect applicable assignments and account for any additional authorization path.

Deploy with explicit parameters

Create a local parameter file with the companion New-CompassParameters.ps1 helper. It accepts the selected region, globally available storage name, supported VM size, and an existing SSH public-key file. It refuses to overwrite an existing output file and rejects private-key content. The public key is not a private credential, but configuration files should still be reviewed before sharing.

```
./companion/New-CompassParameters.ps1 -StorageName "<unique-storage-name>" -SshPublicKeyPath
"<path-to-public-key>" -Location westeurope
```

Use a clear deployment name so that its operations and outputs can be located later. The commands below illustrate the flow in Bash. Replace the subscription value and prepare the parameter file before execution.

```
az account set --subscription "<practice-subscription-id>"
az account show --output table
az group create --name rg-compass-ch19 --location westeurope --tags Purpose=Training Book=Compass
Chapter=19
az deployment group what-if --resource-group rg-compass-ch19 --name compass-ch19 --template-file
companion/ch19-capstone.bicep --parameters @ch19-parameters.local.json
az deployment group create --resource-group rg-compass-ch19 --name compass-ch19 --template-file
companion/ch19-capstone.bicep --parameters @ch19-parameters.local.json
```

The location in the example must be changed if another region was selected. Inspect the context and what-if output before the creation command. An existing resource group with unrelated contents is not a suitable deployment target.

After completion, read the deployment outputs. They include the VM name, VM principal identifier, storage account name, workspace name, role-assignment identifier, and private-endpoint identifier. Record the values that support subsequent tests. Outputs are references to inspect; they are not a substitute for testing the resources' behavior.

Account for partial success

If the deployment fails, inspect which operation failed and which resources already exist. A name collision, unavailable VM size, missing permission, or policy denial calls for a different correction. Recreating the group without reading the operation details can repeat the same failure and make the evidence harder to follow.

A parameter correction can often be applied through another incremental deployment, but review the proposed changes first. A different storage name creates a different resource. A changed network address range may affect existing members. The repeatability of a template does not make every parameter change harmless.

Keep the governance record short and specific. It should identify the subscription, group, allowed region, deployment identity capability, VM identity, data-role scope, and cost-bearing resource types. This record connects the project's technical configuration to the requirement it was intended to satisfy.

19.3 Connecting Storage and Compute

Verify the network relationships

The VM's network interface belongs to the workload subnet. The private endpoint belongs to the endpoint subnet. Both subnets are inside the same virtual network. The workload subnet's inbound deny rule prevents unsolicited inbound sessions to the VM, while stateful handling allows responses to permitted outbound requests.

The public IP attached to the VM's interface provides an explicit outbound connectivity method. It is not used as the Blob service destination. The Blob hostname should resolve to the private endpoint address from the VM. Distinguishing the management and outbound requirements from the storage data path prevents misleading conclusions about what “private” means in this design.

Check the private endpoint connection state in the storage account or endpoint resource. An endpoint resource can exist while its service connection is not approved or ready. The target subresource must be blob, because the project uses Blob operations. An endpoint for a different storage service would not satisfy the same data path.

Inspect the private DNS zone, its virtual-network link, and the endpoint's DNS zone group. The project uses the service's private-link zone for Blob endpoints. The virtual-network link makes that zone available to the VM's normal Azure DNS resolution path. A different custom DNS design would need an appropriate forwarding or resolution arrangement.

Understand what the probe does

The Python probe runs inside the VM. It resolves the ordinary Blob hostname, requests a managed-identity token from the VM metadata endpoint, and sends an HTTPS request to the Blob service. Its dependencies are deliberately small: Python's standard library, a functioning VM identity, DNS, the network path, and the correct data permission.

The metadata token request is made directly without a configured HTTP proxy. The token's audience is Azure Storage. The script keeps the token in memory and does not print it. A token intended for another service would not be an appropriate credential for this Blob request.

In write mode, the probe writes verification.txt in the evidence container and reads it back. It prints the written version identifier, HTTP status values, and the content comparison result. The version identifier is needed for the later recovery exercise. The token and authorization header must not be copied into the practice record.

In read mode, the probe retrieves the current blob and compares it with the known expected contents. Read mode is useful for testing access without adding another version. A successful read shows that the specific identity, path, and operation worked at that time. It does not imply that all possible storage operations have been tested.

Run the first data test

Use the companion PowerShell wrapper from a workstation with Azure CLI available and authenticated. The wrapper supplies the selected subscription explicitly, builds a short Run Command payload, and asks the VM agent to run the probe. The wrapper accepts the exact storage account name rather than discovering one by a partial display-name match.

```
./companion/Invoke-CompassProbe.ps1 -SubscriptionId "<practice-subscription-id>" -StorageAccount
"<actual-storage-name>" -Mode write
```

Replace both placeholders. Run Command executes in the guest through the VM agent; it does not require opening SSH for this test. The VM agent must be healthy, and the VM must have suitable outbound connectivity to return results. This control path is separate from the Blob private endpoint.

Read both stdout and stderr in the returned result. A successful management request does not prove that the Python program completed successfully. The expected guest output includes private endpoint resolution, successful write and read statuses, a version identifier, and Content verification PASSED.

If the identity role was just assigned, allow for propagation and rerun the probe before concluding that the assignment is wrong. The metadata service may return a cached token; a new invocation does not force token renewal. Keep the scope and principal checks in view while waiting. Repeatedly granting broader roles is not an appropriate response to an unexamined delay.

Verify from more than one source

Compare the address printed by the probe with the private endpoint's network-interface address. The match supports the intended DNS path. If the address is public, investigate zone linking and the VM's DNS configuration before changing storage authorization.

Read the storage account's publicNetworkAccess and allowSharedKeyAccess properties. They should reflect the restrictions declared in the template. The probe's use of a bearer token and the container role assignment explain its authorization path. The test does not need an account key.

Review the VM's identity principal identifier and compare it with the assignment principal. A human deployment identity's successful data request would not prove that the VM identity works. Running the probe inside the VM makes the actor explicit.

Inspect the container and blob protection settings through the management plane. Versioning should be enabled, along with the selected soft-delete retention settings. The protection configuration must exist before the deliberate change if the intended earlier version is to be retained.

Observe the generated operations

Open the Log Analytics workspace and query StorageBlobLogs after generating the probe activity. A short query can show the operation names, status codes, authentication type, and caller address. Time filtering reduces unrelated records, but an overly short window can hide delayed ingestion.

```
StorageBlobLogs
| where TimeGenerated > ago(1h)
| project TimeGenerated, OperationName, StatusCode,
          AuthenticationType, CallerIpAddress
| order by TimeGenerated desc
```

If the table has no records, inspect the Blob service diagnostic setting, destination workspace, enabled categories, generated activity, and elapsed ingestion time. An empty result is not proof that no requests occurred. It might indicate that the query targets the wrong workspace or that collection was configured after the event.

Do not broaden the query into a search of unrelated organizational data. The dedicated workspace and resource names make the project's evidence easier to isolate. Record only the fields needed to explain the test result.

After successful execution and verification, the environment demonstrates a complete relationship: a compute identity performs a data operation through an intended network path, while the storage service produces observable records. The next section changes one part of that relationship to test whether the administrator can identify the resulting failure.

19.4 Investigating Deliberately Introduced Faults

Choose one controlled fault

The project's primary fault is removal of its container-scoped data-role assignment. This changes authorization without changing DNS, routing, or the VM identity itself. Because the starting state has already been verified, the difference can be interpreted more clearly.

Record the assignment identifier from deployment outputs and inspect it before deletion. Confirm the principal is the VM's managed identity, the role is Storage Blob Data Contributor, and the scope is the evidence container. Remove only that assignment in the disposable project. Do not remove a broader inherited assignment belonging to another workload.

The action requires role-assignment management permission. If the learner lacks that permission, the designated administrator can perform the controlled change. The learning record should identify who made the change and which exact assignment was affected.

Form a prediction

Before rerunning the probe, state what should remain unchanged. The VM should still resolve the Blob hostname through the private endpoint. The VM identity should still be able to request its token. The Blob operation should eventually be denied when no equivalent effective grant remains and the authorization change has propagated.

This prediction distinguishes authorization from connectivity. A DNS failure or timeout would not match the intended fault and would need a separate explanation. A successful data operation might mean that propagation has not completed or that another assignment still grants access.

Allow for the platform's permission propagation behavior. Do not claim an immediate denial as a guaranteed outcome. Use a fresh probe invocation and inspect effective assignments. The result should be interpreted in the context of token and authorization state rather than treated as a mechanical timer.

Collect the failing evidence

Run the probe in read mode so the test does not attempt to create another version. Inspect the printed DNS addresses, the service error category, and the request identifier when one is supplied. The helper deliberately avoids printing access tokens or response bodies that might contain unrelated data.

Compare the failure with the successful baseline. The intended observation is a storage authorization failure while the private endpoint continues resolving correctly. The same resource and operation should be used so that the comparison isolates the changed permission.

Check the storage logs when the generated event has been ingested. The request's status and authentication information can support the explanation. The absence of a log row at the first query does not invalidate the guest observation; inspect collection and time range before drawing that conclusion.

Restore the intended assignment

Redeploy the unchanged Bicep template to restore the missing assignment, or have the authorized administrator recreate the same principal-role-scope relationship. Review what-if output first. The template's deterministic assignment name supports restoring the intended resource rather than accumulating arbitrary duplicate grants.

After the assignment is restored and effective, run read mode again. The expected content comparison should pass. Record both the restored relationship and the successful data request. A deployment success message alone is not enough to close the fault exercise.

The correction should remain narrow. Enabling Shared Key access or public network access would change other controls without explaining the missing data permission. The project is designed to show that a specific authorization problem can be corrected while its network restrictions remain intact.

Investigate an unexpected outcome

If the request still succeeds after removal, inspect additional assignments at the container, account, resource-group, and higher relevant scopes. Confirm that the probe is running on the intended VM. Confirm that the removed assignment was the one associated with that VM's current principal identifier.

If token acquisition fails, inspect whether the VM's identity is still enabled and whether the metadata endpoint is reachable from the guest. That is a different failure from a Blob authorization rejection. The token request uses the VM-local metadata service; the Blob request uses the storage hostname.

If name resolution produces a public address, inspect the private DNS link, zone records, endpoint connection, and guest DNS configuration. Do not change roles to correct a name-resolution issue. Establishing which layer failed is the purpose of the exercise.

If Run Command itself fails, inspect VM agent health and outbound connectivity. A failed control channel cannot provide a valid observation of the Python probe's result. Restore the ability to execute and observe before interpreting storage behavior.

Turn the fault into a usable record

Write the diagnosis as a small causal account. State the previously working operation, the exact change, the observed failure, the evidence that ruled out other layers, and the successful verification after correction. Avoid a vague conclusion such as “permissions were wrong.”

The record should identify the missing container data-role assignment and explain why DNS and identity acquisition were still functioning. It should also acknowledge any propagation delay or additional assignment that affected the test. These details make the result reproducible.

A useful troubleshooting habit emerges from the comparison. Begin with a known operation and its dependencies, alter one controlled condition, and examine the result against a prediction. Production incidents are rarely so tidy, but the same reasoning helps separate causes from coincidental changes.

19.5 Demonstrating Recovery and Completing Cleanup

Preserve the known recovery point

The first successful write printed a version identifier. Record it exactly. The identifier names the retained version that contains the known teaching data. A timestamp noted separately in a notebook is helpful context, but it is not interchangeable with the service's version identifier.

Before the destructive variation, confirm that versioning remains enabled and that the earlier version is available. The account's soft-delete settings provide additional protection for relevant deletion cases, but this exercise focuses on an overwrite and a retained version. The recovery mechanism should match the action being tested.

The helper restricts its operations to evidence/verification.txt. This narrow target makes the exercise easier to reason about and limits the consequence of a typing error. Do not adapt the damage mode to organizational data or a shared production container.

Change the current contents

Run the helper in damage mode. It writes a different fixed teaching string to the same blob and reports that the deliberate change completed. The helper does not delete the account, container, or all versions.

```
./companion/Invoke-CompassProbe.ps1 -SubscriptionId "<practice-subscription-id>" -StorageAccount
"<actual-storage-name>" -Mode damage
./companion/Invoke-CompassProbe.ps1 -SubscriptionId "<practice-subscription-id>" -StorageAccount
"<actual-storage-name>" -Mode read
```

The following read should retrieve data successfully but fail the expected-content comparison. This is an important distinction. Connectivity and authorization can both work while the data is unsuitable for the intended operation. A successful HTTP status therefore does not prove data correctness.

Record the comparison failure and the fact that the current object still exists. This demonstrates the actual recovery problem. The administrator needs the earlier valid contents, not merely a way to reconnect to the current blob.

Restore the earlier contents

Run restore mode with the recorded good version identifier. The helper reads that version, checks that it contains the known expected teaching data, and only then writes those contents as the current blob. It reads the current blob again to verify the result.

```
./companion/Invoke-CompassProbe.ps1 -SubscriptionId "<practice-subscription-id>" -StorageAccount
"<actual-storage-name>" -Mode restore -VersionId "<recorded-good-version-id>"
```

The helper refuses to perform the write if the selected version does not contain the expected teaching data. This prevents choosing an arbitrary version and reporting a successful restoration solely because an API accepted the request. The example is intentionally specific to the known small text object.

The operation restores contents by reading an earlier version and writing a new current version. It does not restore every metadata field, access-control arrangement, or related application state. A production recovery procedure must identify those additional requirements and test them separately.

Record the restored content result and the newly written version identifier. The original good version and the new current version are different versioned states even though their contents match. This distinction helps explain why recovery can preserve history rather than erasing the evidence of the unwanted change.

Review the operational evidence

Return to the monitoring workspace and examine the project's write and read activity. Compare the time of the original write, deliberate change, and restoration with the relevant records. Do not infer byte-for-byte contents from an operation log; use the probe's explicit content comparison for that evidence.

Summarize which outcomes were demonstrated. The private endpoint path, managed-identity authorization, controlled permission failure, restoration of access, and content recovery each require their own observation. If a platform condition prevented one test, document that condition and leave the corresponding outcome unverified.

The project does not demonstrate a complete VM restoration or regional Site Recovery failover. Those procedures belong to the backup and recovery chapter's exercises. Avoid calling this small content recovery a disaster-recovery certification for an entire environment.

Remove the practice resources

Inventory the dedicated resource group before deletion. Compare its contents with the deployment record and any manual additions. The project should contain only resources created for this exercise. An unexpected resource requires investigation before the group is removed.

Remove any lock that was deliberately added during an optional governance exercise, after confirming its exact scope and purpose. The base capstone template does not add a lock. A lock at another scope may require a separate authorized action; do not remove an organizational safeguard merely to simplify cleanup.

Delete the dedicated group and wait for completion. Then verify that it no longer exists in the intended subscription. The VM, disk, public IP, endpoint, network, storage account, and workspace are within that group. Review the service-specific retention behavior of any soft-deleted or retained resources, rather than assuming that a portal disappearance immediately proves every cost or retention condition has ended.

The system-assigned identity follows the VM lifecycle. Any manually created users, groups, budgets, or assignments outside the project group require their own inventory and cleanup decision. The base template does not create directory users. Keep this distinction visible if the project has been extended.

Complete the handover record

A final project record should contain the requirements, resource identifiers, selected region and size, permission relationship, observed network path, successful and failing probe results, recovery evidence, and cleanup confirmation. Remove tokens and credentials. Preserve source files and a sanitized record so the learning can be repeated without keeping billable resources.

A completed project connects its original requirement to observed behavior and a tested correction. The handover record should make those connections easy to follow. That record provides a practical foundation for operational work and the exam-focused reasoning in the final chapter.

Project completion checklist

- The deployment context and resource inventory match the practice design.
- The VM uses its own managed identity at the intended container scope.
- Blob DNS resolves to the private endpoint from the VM.
- The known write/read succeeds without an account key.
- Generated storage activity appears in the selected monitoring destination.
- A controlled authorization fault is explained and corrected.
- Earlier known contents are recovered and verified.
- The dedicated environment is removed or its intentional retention is recorded.

Further reading

- Managed-identity token use on Azure VMs
- Blob authorization with Microsoft Entra ID
- Storage built-in roles
- Linux VM Run Command
- Explicit and default outbound access
- Blob versioning

Chapter 20: Turning Understanding into Exam Readiness

The final assessment brings the book's subjects together as written decisions. It asks the reader to interpret requirements, identify the relevant scope, and distinguish plausible alternatives. Its purpose is to reveal what needs further study. It is an original learning assessment and does not reproduce or predict the live certification exam.

Learning objectives

- Review coverage using specific skills rather than chapter-completion marks alone.
- Identify the actor, scope, operation, and constraint in a technical question.
- Complete an original assessment without relying on answer recognition.
- Use explanations to distinguish a misconception from a reading or execution error.
- Build a revision plan from evidence gathered across questions and practical work.

20.1 Reviewing Coverage Across the Five Domains

Use the objective map as an evidence map

The AZ-104 objectives establish the baseline scope for this book. Appendix A maps each objective to its primary teaching section. The mapping identifies where to study, but the reader must add evidence of understanding. Reading a section is an activity; explaining a decision or completing a suitable operation is a result.

For each objective, record one of three states: not yet studied, understood with assistance, or demonstrated independently. These states are learning labels rather than official certification categories. They help separate familiarity from reliable application. A skill can move backward if a later exercise reveals that an earlier explanation was incomplete.

The evidence should fit the skill. An objective concerning interpretation needs an explanation of what the configuration means. An objective concerning deployment needs a configuration and an observed result. An objective concerning recovery needs a restoration test appropriate to the stated recovery requirement. A screenshot of a setting cannot support every kind of claim.

The five domains have different published weight ranges. This assessment uses fourteen identity and governance questions, eleven storage questions, fourteen compute questions, twelve networking questions, and nine monitoring and recovery questions. That distribution lies within the planning baseline's domain ranges. These are domain allocations for this original assessment, not predictions of the number or arrangement of questions on a live exam.

Distinguish coverage from depth

A reader may have encountered every service name while still lacking important distinctions. Examples include resource access versus data access, a service endpoint versus a private endpoint, availability

versus recovery, and a scale-up operation versus a scale-out operation. These distinctions deserve focused review because they often determine which action meets a requirement.

Use a short explanation test. State what the feature controls, where it applies, what prerequisite it needs, and what it does not establish. For example, a private endpoint provides a private network interface for a service connection. Its existence does not by itself establish that a particular identity has permission to read the service's data.

A complete explanation need not be long. Precision matters more than reciting an entire chapter. If the explanation relies on an undefined word such as “secure,” replace it with the exact behavior: which traffic is allowed, which identity can perform which operation, or which earlier state can be recovered.

Check practical readiness separately

Assessment questions can expose reasoning errors, but they do not demonstrate command execution. The practice record should also show context selection, resource inspection, controlled changes, expected failures, and cleanup. A learner who understands a diagram may still need practice reading a real resource identifier or interpreting a deployment error.

Use the integrated project to inspect relationships between domains. Its identity assignment, private DNS path, storage settings, monitoring destination, and recovery test are connected. Failure to explain one relationship identifies a narrower revision target than the broad conclusion that “Azure is difficult.”

Do not delay all practical work until every question can be answered perfectly. A small exercise can settle an uncertainty more clearly than repeated reading. At the same time, avoid treating a single successful lab run as permanent mastery. Change one harmless input or requirement and explain how the outcome should differ.

20.2 Reading Technical Questions Carefully

Identify what the question actually asks

Begin with the required outcome. A question may ask for the least privilege, the smallest configuration change, a supported service choice, or the cause of a failure. These are different tasks. An option can be technically capable and still fail the stated requirement because it grants unnecessary authority or changes more than necessary.

Next identify the actor. A human user, group, service principal, managed identity, and unauthenticated client do not follow identical access paths. If a question specifies the identity used for an operation, do not silently substitute an administrator's identity when considering the answer.

Then identify the scope. A role at subscription scope affects a different set of resources from a role on a single container. A DNS zone linked to one virtual network is not automatically available through every possible DNS arrangement. A policy assignment's location in the hierarchy matters when interpreting its effect.

Finally identify constraints. The requirement might forbid public data access, require a particular recovery point, preserve an existing resource, or limit an operation to one group. Mark these constraints before evaluating options. They often explain why a plausible service is unsuitable.

Separate sufficient conditions from partial conditions

A question can describe several correct configuration facts while leaving one necessary condition unsatisfied. A VM may exist, have a network interface, and run its operating system while an application remains unreachable. Those facts narrow the problem but do not establish the missing condition.

Evaluate what each option would prove. Changing DNS can address name resolution, but it does not grant a missing data role. Adding a role can authorize an operation, but it does not create a network path. Recreating a resource can change its identity and identifiers, potentially introducing more problems.

When the question asks for a diagnostic action, prefer the observation that distinguishes the remaining explanations. A read of effective security rules can be more informative than a broad change to an NSG. A review of the active subscription can be more appropriate than creating a resource that appears missing.

Avoid importing unstated assumptions

Use the information given. If an option requires an additional service, permission, or configuration that is not present, account for that dependency. Do not assume that every subscription has unlimited capacity or that every identity holds broad administrative roles.

The same discipline applies to feature names. A backup-related name does not establish that the service protects every workload. A tier name does not imply every scale feature. A network control should be interpreted according to its documented behavior, not according to a general impression of security.

Some real-world problems remain ambiguous until more information is collected. The assessment questions are written to provide a single best answer within their stated conditions. If a question seems to have two answers, first inspect the constraints and the exact operation. Record a genuine ambiguity for review rather than forcing an unsupported certainty.

Make uncertainty useful

Before checking an answer, mark confidence as high, medium, or low. A correct answer with low confidence identifies a topic worth revisiting. A wrong answer with high confidence identifies a misconception that may affect several other decisions. These patterns are more informative than a total alone.

Write a brief reason for the chosen option. The reason should mention the decisive condition, not merely repeat the option's wording. For example, “the principal needs a Blob data role at the container scope” explains more than “B is the storage answer.”

If a question takes too long, identify the unresolved distinction and move on during a timed practice attempt. Return to that distinction afterward. The purpose of timing is to observe decision fluency; it should not encourage guessing without later examination.

20.3 Completing an Original Practice Assessment

Complete the following sixty questions without reading Section 20.4 first. Each question has one best answer under its stated conditions. Record the selected letter, confidence, and a short explanation. Use a separate answer sheet so that the questions remain reusable after a meaningful interval.

For a first attempt, use an untimed session and focus on complete reasoning. For a later attempt, set a study-session time limit appropriate to the reader's pace. This book does not present that limit as the live exam's duration. Check Microsoft's current certification information for the actual exam arrangements.

Avoid using the answer explanation as a hint before committing to a choice. The initial choice and reasoning are the evidence needed for useful revision. Changing an answer after viewing its rationale should be recorded as a correction, not counted as an independently correct response.

EX-Q01. An organization has one Microsoft Entra tenant and two Azure subscriptions associated with it. An employee is created in the tenant. What must be considered before that employee can manage a resource group in the second subscription?

- **A.** Creating the account automatically grants Contributor on every associated subscription
- **B.** The employee needs a second directory account merely because the subscription differs
- **C.** An appropriate Azure role assignment is needed at the resource group or an inherited scope
- **D.** The resource group must be moved into the employee's directory account

EX-Q02. A supplier needs access to one project resource group. The organization wants the supplier represented as an external user while granting only the required Azure permissions. Which approach best fits?

- **A.** Invite or provision the external identity through the supported guest workflow and assign a suitable scoped Azure role
- **B.** Make the supplier Global Administrator because external users cannot receive scoped resource access
- **C.** Share an employee's password so the supplier uses an existing identity
- **D.** Move every project resource into a publicly accessible storage account

EX-Q03. A team receives access through an Azure role assigned to an Entra security group. A new employee joins the team and needs the same established access. What should the administrator examine first?

- **A.** Create a direct Owner assignment before inspecting group membership
- **B.** Replace the existing security group with a new group for the employee
- **C.** Assume that adding the employee to any Microsoft 365 group grants the same Azure role
- **D.** Check whether the employee is correctly included in the intended group and its membership model

EX-Q04. A user has a Microsoft Entra account and Reader access to an Azure resource group but cannot use a licensed Microsoft service. Which distinction should guide the investigation?

- **A.** Reader automatically includes every product license
- **B.** Product licensing and Azure resource authorization are separate requirements
- **C.** A resource-group tag grants service entitlement
- **D.** A management lock supplies missing licenses

EX-Q05. Self-service password reset is enabled for a selected group. A member has never registered the required recovery methods. What should be checked before declaring the service unavailable?

- **A.** Grant Azure User Access Administrator so the user can reset the password
- **B.** The user's inclusion, registration state, and the configured authentication methods
- **C.** Create another Azure subscription for password-reset requests
- **D.** Assign Contributor on the resource group that contains the user's VM

EX-Q06. An operator must read Azure resource configuration across a project group and must not change it. Which built-in role is the most appropriate starting point among these choices?

- **A.** Owner
- **B.** Contributor
- **C.** User Access Administrator
- **D.** Reader

EX-Q07. A user receives Contributor at a resource group's parent subscription. An administrator assigns Reader directly on one resource in that group. What should be expected from ordinary additive role assignments, assuming no relevant deny or other restriction?

- **A.** The Reader assignment does not remove the inherited Contributor permission
- **B.** The lower-scope Reader assignment always replaces every inherited role
- **C.** The subscription assignment stops applying because the resource has a direct assignment
- **D.** The resource becomes inaccessible until its tags are removed

EX-Q08. A user can manage a storage account's Azure configuration but receives a denial when reading blobs through Microsoft Entra authorization. The network path is confirmed valid. What is the most relevant next check?

- **A.** Whether a ReadOnly management lock should be removed to permit a blob read
- **B.** Whether the user needs the Entra Global Reader role instead of storage data permission
- **C.** Whether the user has the required storage data-plane role at an applicable scope
- **D.** Whether enabling anonymous access is required whenever Entra authentication is used

EX-Q09. Several subscriptions need a common governance assignment inherited from a shared organizational level. Which Azure structure is intended to group subscriptions for this purpose?

- **A.** A blob container
- **B.** A virtual network subnet
- **C.** An availability set
- **D.** A management group

EX-Q10. A resource has a CanNotDelete management lock. A Contributor attempts to delete it and is blocked. Which interpretation is correct?

- **A.** Contributor always overrides management locks once the user obtains another token
- **B.** The lock restricts a supported management operation despite the user's ordinary resource permission
- **C.** CanNotDelete blocks every property update as well as deletion
- **D.** The same lock guarantees that an authorized client cannot delete a blob through the data plane

EX-Q11. An organization must prevent new deployments that violate an approved location rule. Which mechanism most directly expresses and enforces that configuration requirement?

- **A.** A naming convention documented only in a spreadsheet
- **B.** A cost budget alert
- **C.** An appropriately scoped Azure Policy assignment with an enforcing effect
- **D.** A Reader assignment to the deployment identity

EX-Q12. A policy assignment identifies existing resources that need a supported deployIfNotExists remediation. What should be checked before starting the remediation task?

- **A.** The assignment's managed identity and permissions needed for the corrective deployment
- **B.** Only the human administrator's ability to read the compliance report
- **C.** Whether switching the policy effect to audit will perform the same deployment
- **D.** Whether an assignment's managed identity automatically receives every required role without review

EX-Q13. A subscription budget reaches its configured notification threshold. What should the team expect from the budget alone?

- **A.** Every resource is automatically deleted
- **B.** All charges are guaranteed to stop at the budget amount
- **C.** A configured notification or action path can be triggered, but the budget is not itself a spending cap
- **D.** The subscription moves to a different tenant

EX-Q14. Azure Advisor recommends reducing a VM's capacity. The VM handles a known monthly processing peak that is not represented in the recent quiet period. What is the best response?

- **A.** Apply every recommendation without reviewing workload requirements
- **B.** Delete all performance history
- **C.** Assume recommendations can never be useful
- **D.** Evaluate the recommendation against the full workload pattern and verify any chosen change

EX-Q15. A storage design requires resilience across availability zones within a supported region. Which redundancy option most directly matches that stated placement requirement?

- **A.** LRS
- **B.** ZRS
- **C.** A single local VM disk without replication
- **D.** A lifecycle rule moving blobs to another tier

EX-Q16. An administrator configures object replication for a supported blob scenario. What must still be considered separately when protecting against accidental deletion or corruption?

- **A.** Historical recovery requirements and the appropriate retention or backup mechanisms
- **B.** Whether the second copy alone guarantees that every previous clean state remains recoverable
- **C.** Whether replication automatically supplies an independently tested application restore procedure
- **D.** Whether replica availability makes version and deletion-protection settings unnecessary

EX-Q17. A storage account uses customer-managed encryption keys. The team plans to disable the relevant key version without examining dependent resources. What is the principal concern?

- **A.** Disabling the key transparently migrates the account to Microsoft-managed keys
- **B.** Only requests that explicitly name the key version can be affected
- **C.** The service automatically re-encrypts all existing data with an unrelated available key
- **D.** The service may lose the key access needed to use protected data

EX-Q18. AzCopy can authenticate to a storage account, but a transfer from a particular workstation times out. The same operation works from an allowed network. What should be investigated next?

- **A.** Rotate credentials immediately because every timeout proves token expiration
- **B.** Increase the destination account's retention period to permit the network transfer
- **C.** The failing client's network route, DNS resolution, and storage network restrictions
- **D.** Change blob access tiers because authentication bypasses endpoint routing

EX-Q19. A team considers moving infrequently read blobs to a lower-cost storage tier. Which evaluation is necessary before making the change?

- **A.** Only the per-gigabyte storage rate, because access and retention cannot change the total cost
- **B.** Access frequency, retrieval needs, supported latency, and transaction or early-deletion cost implications
- **C.** Whether the lowest-cost tier always provides the same immediate retrieval behavior as Hot
- **D.** Whether moving to a lower tier automatically changes the account's redundancy option

EX-Q20. A lifecycle policy is intended to affect only blobs under the archive-input prefix. What should be reviewed before enabling its deletion action?

- **A.** The rule's filters, conditions, target data, and retention requirement
- **B.** Only the rule's display name, because it determines the matching blob prefix
- **C.** Whether enabling the policy creates a guaranteed preview and approval prompt before each deletion
- **D.** Whether the policy can replace all required protection for previous versions and snapshots

EX-Q21. Blob versioning was enabled before an application overwrote a document with incorrect content. The account and container still exist. Which recovery approach most directly addresses the mistake?

- **A.** Change the account's region
- **B.** Rotate every storage account key
- **C.** Locate and recover the appropriate earlier blob version using a supported version-recovery operation
- **D.** Delete the container and assume its newest contents become correct

EX-Q22. Container soft delete was enabled and a container was deleted within its retention window. What should be verified before promising recovery?

- **A.** Whether creating an empty container with the old name automatically restores all retained contents
- **B.** Whether the same retention window also guarantees restoration of a deleted storage account
- **C.** Whether changing a container's display metadata extends every expired recovery window
- **D.** The deleted container's recoverable state, applicable retention, and supported restore procedure

EX-Q23. Azure Files identity-based SMB access is configured through a supported directory model. A user has the required share-level authorization but cannot open one folder. Network connectivity is healthy. What should be inspected next?

- **A.** The folder and file permissions applicable to that identity
- **B.** Whether the storage account uses a larger capacity quota
- **C.** Whether adding Azure Reader on the share automatically replaces every filesystem permission
- **D.** Whether changing the storage redundancy option grants folder access

EX-Q24. A service SAS was issued using a stored access policy on a supported storage resource. The team needs to invalidate that policy-linked access without distributing a replacement account key to every client. What is the most targeted option?

- **A.** Rename the resource group
- **B.** Change or revoke the referenced stored access policy according to the supported behavior
- **C.** Grant the client Owner
- **D.** Assume the SAS cannot be revoked before its printed expiry under any circumstances

EX-Q25. A blob client has a valid Entra data role but reaches the storage account over a network path excluded by the account firewall. What outcome should be expected?

- **A.** The role automatically bypasses all storage network rules
- **B.** TLS automatically changes the client's source address
- **C.** The request can still be denied because authorization and network access are separate requirements
- **D.** The account must grant anonymous blob access to repair the role

EX-Q26. A VM deployment fails because the selected size is unavailable in the chosen region or subscription context. The template parses correctly. What is the best next action?

- **A.** Assume a successful local Bicep build proves quota and regional capacity are sufficient
- **B.** Assign the deployment identity Owner to guarantee physical capacity
- **C.** Retry indefinitely without examining the deployment error or selected size
- **D.** Inspect regional size availability, quota, and supported alternatives that meet the workload requirement

EX-Q27. A VM is stopped and deallocated, but its managed data disks remain. Which expectation is correct?

- **A.** The retained disks continue to exist and can continue to incur storage charges
- **B.** Deallocation automatically converts every managed disk to a free archived object
- **C.** The remaining disks are equivalent to separately retained Azure Backup recovery points
- **D.** A deallocated VM guarantees that all resources associated with it have stopped incurring charges

EX-Q28. A running VM must be resized. The target size is supported for the workload, but the available resize path can involve interruption. What should the change plan include?

- **A.** A guarantee that changing a VM size never restarts or deallocates the VM
- **B.** A capacity change without checking target disk, networking, or architecture compatibility
- **C.** A review of availability requirements, the supported resize procedure, and post-change workload verification
- **D.** Deletion of existing recovery points as a prerequisite for every supported resize

EX-Q29. A team is evaluating encryption at host for an Azure VM. What should be verified before applying it?

- **A.** Whether guest full-disk encryption automatically enables the Azure encryption-at-host setting
- **B.** Whether the selected VM configuration and size support the feature and its enablement requirements
- **C.** Whether assigning Key Vault Reader alone enables host encryption for every VM size
- **D.** Whether changing the VM's network security group substitutes for encryption at host

EX-Q30. A supported VM and its dependencies are moved to another resource group in the same subscription. What should not be assumed from that management move alone?

- **A.** The resource IDs may need review because their group component changes
- **B.** Dependencies and move validation matter
- **C.** Permissions inherited from the destination scope need review
- **D.** The VM's physical deployment region automatically changes

EX-Q31. A supported application must remain available if one availability zone in its Azure region fails. Which deployment decision most directly supports that requirement?

- **A.** Distribute suitable redundant application instances across supported availability zones
- **B.** Place all redundant application instances in the same availability zone
- **C.** Add a CanNotDelete lock to a single VM and rely on it for zone resilience
- **D.** Place one VM in an availability set and assume it now spans several zones

EX-Q32. A scale set can add instances, but each instance processes the same uncoordinated work file and creates duplicate results. What must be addressed before relying on scale-out?

- **A.** Increasing the maximum instance count without changing how work is claimed
- **B.** Whether the application safely distributes and coordinates work across instances
- **C.** Changing the autoscale threshold alone so duplicate work starts at a lower CPU value
- **D.** Assigning a different private IP address to each instance and assuming that partitions the input file

EX-Q33. An administrator changes a Bicep parameter used in several resource name properties. What should be examined before deployment?

- **A.** Whether incremental deployment mode guarantees that changed resource names update the old resources in place
- **B.** Whether keeping Bicep symbolic names unchanged preserves every deployed resource identity
- **C.** Whether the changed values address new resource identities rather than updating the intended existing resources
- **D.** Whether reusing the deployment-history name prevents creation of resources with new names

EX-Q34. A Bicep build succeeds locally, but what-if reports an unresolved critical expression. What is the best interpretation?

- **A.** The application has passed every operational test
- **B.** The unresolved resource is guaranteed unchanged
- **C.** The definition must be deployed immediately to make the warning disappear
- **D.** Compilation passed, but additional investigation is needed before the critical proposed change is understood

EX-Q35. A team exports an ARM template from an existing resource group and decompiles it to Bicep. Which statement is accurate?

- **A.** The result is a starting point that needs review of configuration, dependencies, and conversion diagnostics
- **B.** The export necessarily contains every application's data and secrets
- **C.** Decompilation proves that a production redeployment is safe
- **D.** The generated file can replace all backup procedures

EX-Q36. A registry changes from RBAC-only permissions to repository-level ABAC mode. An existing runtime identity can no longer pull its image using an old AcrPull assignment. What should be reviewed?

- **A.** Keep AcrPull unchanged because all RBAC-only registry roles remain effective in ABAC mode
- **B.** The applicable repository roles and conditions for the registry's selected permission mode
- **C.** Give the runtime identity Azure Reader and assume management visibility includes image data access
- **D.** Enable the registry admin user permanently instead of reviewing the intended managed-identity permissions

EX-Q37. A Container App worker must scale from zero when a queue receives messages. Which design best matches that requirement?

- **A.** Only a CPU threshold measured by running replicas
- **B.** A permanently stopped app with no trigger
- **C.** A supported queue-based scaling rule with access to its event source and appropriate replica limits
- **D.** A larger registry SKU without a scaling rule

EX-Q38. A staging slot uses a test database, while production must retain its production database connection after a swap. What should be configured and verified?

- **A.** Make both slots use the test database permanently
- **B.** Delete the production slot before every release
- **C.** Assume every setting always stays with its slot automatically
- **D.** Mark the environment-specific connection as a deployment-slot setting and verify the swap behavior

EX-Q39. An App Service app needs to call a private storage endpoint. An inbound private endpoint has been created for the app, but no outbound integration was configured. Which next step addresses the missing direction?

- **A.** Configure supported virtual-network integration and verify outbound routing, DNS, and storage authorization
- **B.** Assume the app's inbound private endpoint automatically handles every outbound call
- **C.** Add a custom domain without reviewing connectivity
- **D.** Increase only the deployment-slot count

EX-Q40. Two new virtual networks must eventually be peered. Their proposed address spaces are 10.20.0.0/16 and 10.20.8.0/21. No resources have been deployed. Which design change should happen first?

- **A.** Create a higher-priority allow rule in each network security group
- **B.** Assign nonoverlapping address spaces before deploying the networks
- **C.** Place both networks in the same resource group while retaining the ranges
- **D.** Enable service endpoints on every subnet

EX-Q41. An otherwise empty Azure IPv4 subnet uses 10.30.1.0/27. Ignoring additional service-specific reservations, how many addresses remain after Azure's standard subnet reservations?

- **A.** 32
- **B.** 30
- **C.** 28
- **D.** 27

EX-Q42. An administrator creates a public Azure DNS zone for a registered domain and adds the correct web record. Public resolvers still query the previous DNS provider. The administrator controls the registrar account. What should be checked and corrected?

- **A.** The registrar's delegation must reference the authoritative name servers assigned to the Azure DNS zone
- **B.** The web server must have the DNS Zone Contributor role
- **C.** The public zone must be linked to every client virtual network
- **D.** The web record must be copied into a private DNS zone instead

EX-Q43. Virtual networks A and B are peered, and B and C are peered. There is no direct A-to-C peering, gateway transit design, or routing appliance. Why should an administrator not assume that A can reach C through B?

- **A.** Peering supports only networks in one resource group
- **B.** Peering always requires a public IP address on every VM
- **C.** Virtual-network peering is not automatically transitive
- **D.** Peering permits only DNS traffic

EX-Q44. A subnet's effective route sends a destination prefix to a network virtual appliance. The appliance VM is running, but packets do not reach the destination. Which next investigation most directly addresses the forwarding path?

- **A.** Change the destination route's display name without inspecting its next hop
- **B.** Check appliance IP forwarding, its operating-system forwarding or inspection configuration, and the return route
- **C.** Assume a running appliance VM necessarily forwards packets because the route exists
- **D.** Add the same route to an unrelated subnet and assume it overrides the source subnet's path

EX-Q45. An inbound HTTPS connection reaches a subnet protected by an NSG. The VM's network interface also has an NSG. The subnet NSG permits the flow, while the NIC NSG denies it. Assume the application is listening and routing is correct. What is the expected result?

- **A.** The connection succeeds because the subnet rule is evaluated first
- **B.** The connection succeeds because any allow rule overrides all denies
- **C.** Azure ignores both NSGs because the application is listening
- **D.** The connection is blocked because the flow must be permitted by both applicable NSGs

EX-Q46. In one NSG, an inbound deny rule at priority 150 and an inbound allow rule at priority 300 match the same new TCP flow. No other rule with a lower priority number matches. Which result follows?

- **A.** The allow rule wins because 300 is the larger number
- **B.** The allow rule wins because TCP is stateful
- **C.** The deny rule wins because 150 is evaluated before 300
- **D.** Both rules are ignored because their priorities differ

EX-Q47. Several VM network interfaces in one virtual network belong to the same application tier. An administrator wants NSG rules to refer to that tier without listing each private IP address. Which feature best fits this requirement?

- **A.** An application security group with the relevant network interfaces assigned
- **B.** An Azure role assignment granting Reader to the application team
- **C.** A management group containing the application's subscription
- **D.** A resource lock on the network interfaces

EX-Q48. An administrator needs supported interactive RDP access to an Azure VM that has no public IP address. An appropriately configured Azure Bastion deployment can reach the VM's private address. What does Bastion provide in this design?

- **A.** A replacement for the VM's operating-system account permissions
- **B.** An automatic exemption from all applicable NSG rules
- **C.** An administrative connection path to the VM's private IP without requiring a public IP on that VM
- **D.** A guarantee that the VM is patched and free of malicious software

EX-Q49. A storage account permits a subnet through a virtual-network rule, and that subnet has the appropriate service endpoint enabled. Which statement correctly describes the endpoint model?

- **A.** The storage service now has a private endpoint NIC in that subnet
- **B.** The service endpoint supports access to the service's public endpoint with the subnet identified for the service's network rules
- **C.** Every request bypasses storage authorization because its source is trusted
- **D.** All clients on the internet inherit the subnet's allowed status

EX-Q50. A VM should access Blob Storage through an approved private endpoint. The storage account blocks public network access, but the VM resolves the normal blob hostname to a public address. What should be investigated first?

- **A.** Grant the VM identity Owner without changing how its hostname resolves
- **B.** Create a private endpoint for the File service instead of reviewing the Blob endpoint's DNS path
- **C.** Assume an approved private endpoint forces every custom resolver to return its private address
- **D.** The private DNS zone records, virtual-network links, and any custom DNS forwarding used by the VM

EX-Q51. An Azure Load Balancer backend is unhealthy. The load-balancing rule uses TCP port 443, but the configured HTTP probe requests /health on port 8080. The application serves HTTPS on 443 and has no listener on 8080. What is the most direct correction?

- **A.** Configure a supported probe that matches a real health listener and permit the probe's required network path
- **B.** Change the public DNS record while leaving the probe unchanged
- **C.** Give the VM's administrator account Owner on the subscription
- **D.** Assume the frontend rule automatically makes the mismatched probe healthy

EX-Q52. A VM experiences brief CPU spikes, but a chart showing a one-hour average looks moderate. Which change best investigates whether short peaks are being hidden?

- **A.** Delete the VM's diagnostic settings
- **B.** Change the resource group's region label
- **C.** Inspect a supported finer time granularity and compare suitable aggregations such as maximum and average
- **D.** Treat the hourly average as proof that saturation never occurred

EX-Q53. A new Log Analytics workspace contains no expected resource logs. The administrator created the workspace but did not configure diagnostic settings or any required collection rules. What is the best next action?

- **A.** Configure the supported collection path for the specific data source and destination, then verify ingestion
- **B.** Assume the workspace automatically collects every log in the subscription
- **C.** Assign Owner to every application user
- **D.** Increase retention before establishing collection

EX-Q54. During planned maintenance, a team wants matching supported Azure Monitor alerts to remain visible while suppressing their action-group notifications for a defined window. Which configuration best matches that requirement?

- **A.** Delete the action group permanently
- **B.** Delete the underlying resource's metrics
- **C.** Remove all alert rules for the subscription
- **D.** Create a scoped alert processing rule that suppresses actions during the maintenance window

EX-Q55. Network Watcher IP flow verify reports that the applicable NSG rules allow a specific VM flow. The application's TCP connection still fails. What conclusion is justified?

- **A.** The application must be healthy because the NSG allows traffic
- **B.** The NSG decision is favorable, but routing, guest firewall, listener, and application behavior still need investigation
- **C.** The DNS answer must be correct because IP flow verify passed
- **D.** The service's data-plane permissions have been verified automatically

EX-Q56. An administrator must configure Azure Backup for a supported Azure virtual machine. Which vault family is used for this VM protection scenario?

- **A.** Any Log Analytics workspace, because it retains operational data
- **B.** A Key Vault, because VM disks are encrypted
- **C.** A Recovery Services vault with the required regional and subscription relationship
- **D.** A Backup vault selected solely because its name contains the word Backup

EX-Q57. A backup policy retains daily recovery points for 30 days. The last successful backup is three days old because recent jobs failed. Which statement best describes the current recovery position?

- **A.** The available successful recovery points determine the actual recoverable age; the configured daily schedule has not guaranteed a one-day RPO
- **B.** The 30-day retention setting guarantees a recovery point from every hour
- **C.** The daily policy proves that yesterday's backup exists even when its job failed
- **D.** Increasing retention alone will create the missing recovery points

EX-Q58. An application database was corrupted at 14:00. Several earlier recovery points are available. Before replacing production, what is the strongest initial restore approach?

- **A.** Restore the newest point without checking whether it contains the corruption
- **B.** Delete production immediately to ensure there is no comparison target
- **C.** Assume a successful backup job proves the restored application will be usable
- **D.** Restore an appropriate pre-corruption point to an isolated supported target and validate the application and required data

EX-Q59. A Site Recovery test failover creates a test VM in a separate virtual network. The application can still reach production queues through an allowed outbound path. What should be corrected before running the application's normal workers?

- **A.** Replace every recovery point with a newer one regardless of the test objective
- **B.** Extend test isolation to outbound dependencies so the test copy cannot process production work unintentionally
- **C.** Assume separate virtual-network names prevent all application communication
- **D.** Commit a production failover to make the test network trustworthy

EX-Q60. A recovery report shows that every listed backup job succeeded. An administrator discovers that a newly deployed critical VM does not appear in the protected-item inventory. What is the most accurate assessment?

- **A.** Successful jobs prove that all critical resources are protected automatically
- **B.** The missing VM is protected because it is in the same subscription as a vault
- **C.** Job success describes configured jobs; coverage must be reconciled against the resource inventory and protection requirements
- **D.** The report can be corrected by renaming the VM without configuring protection

20.4 Learning from Answer Explanations

Interpret the result

Count correct responses overall and within each domain. Also count high-confidence mistakes and low-confidence correct responses. These additional counts reveal where familiarity is fragile. The assessment's percentage is a learning measure, not a conversion to Microsoft's scaled exam score or a promise of certification readiness.

Classify each mistake by its principal cause. A knowledge gap means that the relevant behavior was not understood. A scope error means that a known behavior was applied to the wrong boundary. A reading error means that a stated condition was missed. An execution gap means that the concept was familiar but its practical operation remained unclear.

More than one cause can contribute, but choose a specific first correction. “Study networking again” is broad. “Compare private DNS zone linking with public DNS delegation, then test name resolution from the intended client” is actionable. A narrow correction makes later verification possible.

Read why each alternative fails. An incorrect option may be a useful solution to another problem. Understanding that different problem improves discrimination between services. The aim is to build a precise relationship between requirement and action rather than memorize a list of favored answers.

Verify changed understanding

After reviewing a rationale, explain the concept without looking at the answer. Change one condition and predict whether the answer would change. For example, a request for read-only blob access should lead to a different permission decision from a request to modify resource configuration.

Use a small lab where the distinction is observable. Inspect two role scopes, query an endpoint's resolution, compare a current blob with an earlier version, or read a deployment's parameters. Choose a test that directly addresses the mistaken assumption and remains within a dedicated practice boundary.

Record the corrected explanation and evidence. A revision entry should show what was believed, why it was incomplete, and what observation now supports the improved understanding. This creates a compact record of learning that is more useful than copying the full answer rationale.

Answer key and rationales

The explanations below refer to the exact conditions in their questions. Product behavior outside those conditions may require additional decisions. Use the mapped teaching section to revisit the underlying concept.

EX-Q01. An organization has one Microsoft Entra tenant and two Azure subscriptions associated with it. An employee is created in the tenant. What must be considered before that employee can manage a resource group in the second subscription?

Answer: C. C is correct because the directory identity and permission to manage Azure resources are separate concerns. A creates an automatic grant that does not follow from account creation. B is unnecessary when the same tenant identity can receive the appropriate assignment. D treats a directory account as a resource container, which it is not. The administrator should examine the intended action, role, scope, and inherited assignments before deciding that additional access is required.

EX-Q02. A supplier needs access to one project resource group. The organization wants the supplier represented as an external user while granting only the required Azure permissions. Which approach best fits?

Answer: A. A separates establishing the external identity from granting permission to the project resources. The role should match the supplier's duties and be scoped appropriately. B grants broad directory authority and is not required for ordinary scoped resource administration. C removes individual attribution and exposes another person's credentials. D changes neither the identity requirement nor the relevant management authorization. External collaboration should retain an identifiable principal, explicit access, and a process for reviewing or removing that access when the engagement ends.

EX-Q03. A team receives access through an Azure role assigned to an Entra security group. A new employee joins the team and needs the same established access. What should the administrator examine first?

Answer: D. D checks the actual mechanism used for the established grant: membership in the security group named by the role assignment. A adds an excessive separate grant before diagnosing the existing path. B replaces a working access structure unnecessarily. C confuses membership in an unrelated group with membership in the assigned principal. After verifying the correct membership and expected propagation, the administrator should inspect the role's allowed actions, assignment scope, and the specific operation if access still fails.

EX-Q04. A user has a Microsoft Entra account and Reader access to an Azure resource group but cannot use a licensed Microsoft service. Which distinction should guide the investigation?

Answer: B. B is correct because an identity's product entitlement is distinct from an Azure RBAC assignment on resources. The administrator should inspect the required service license and its assignment conditions rather than increasing the Azure role without evidence. A incorrectly combines licensing with resource permissions. C and D are governance mechanisms with different purposes: tags organize metadata, and locks restrict supported management operations. Neither creates a product entitlement. The actual service and its current licensing requirements determine the appropriate repair.

EX-Q05. Self-service password reset is enabled for a selected group. A member has never registered the required recovery methods. What should be checked before declaring the service unavailable?

Answer: B. B examines the prerequisites of the configured reset experience. Inclusion in the selected group does not establish that the user registered enough permitted recovery methods. A grants Azure access-management capability rather than enabling the user's password-reset authentication. C introduces an unrelated resource-management boundary. D grants resource administration without satisfying the reset requirements. The administrator should also consider the account's identity configuration and any applicable licensing, then verify the supported reset path using the methods actually registered by the user.

EX-Q06. An operator must read Azure resource configuration across a project group and must not change it. Which built-in role is the most appropriate starting point among these choices?

Answer: D. D matches the stated requirement to inspect management-plane resource configuration without making changes. Owner and Contributor permit broad resource management beyond that task, while User Access Administrator concerns access-management duties rather than ordinary read-only inspection. The choice does not imply permission to read every service's underlying data. If the operator also needs blob contents, database records, or another data-plane operation, that requirement must be assessed separately with the appropriate service authorization and network path.

EX-Q07. A user receives Contributor at a resource group's parent subscription. An administrator assigns Reader directly on one resource in that group. What should be expected from ordinary additive role assignments, assuming no relevant deny or other restriction?

Answer: A. A follows the additive nature of ordinary Azure role assignments and inheritance. Assigning a narrower role does not act as an explicit reduction of an existing broader grant. B and C incorrectly treat the direct assignment as an override. D introduces unrelated metadata. To reduce the user's effective permissions, the administrator must address the broader assignment or its scope rather than layering Reader on top. The question excludes deny assignments and other restrictions so that the ordinary inheritance behavior is unambiguous.

EX-Q08. A user can manage a storage account's Azure configuration but receives a denial when reading blobs through Microsoft Entra authorization. The network path is confirmed valid. What is the most relevant next check?

Answer: C. C checks the data authorization needed by the stated Entra-authenticated blob operation. A targets a management lock, which does not supply blob data permissions. B confuses a directory role with Azure Storage data access. D incorrectly treats public anonymous access as a prerequisite for authenticated access. The administrator should inspect the actual principal, data role, applicable scope, and propagation. Management-plane administration and a confirmed network path are useful evidence, but neither alone establishes permission to read the requested blob.

EX-Q09. Several subscriptions need a common governance assignment inherited from a shared organizational level. Which Azure structure is intended to group subscriptions for this purpose?

Answer: D. D provides a management hierarchy above subscriptions where supported governance and access assignments can be applied and inherited. A organizes blob data inside storage, B partitions network address space, and C concerns VM availability placement. Those resources do not provide a subscription governance hierarchy. The administrator should still review the management group's scope and descendants before assigning policy or access, because inheritance can affect every subscription beneath the selected level rather than only the subscription currently visible in the portal.

EX-Q10. A resource has a CanNotDelete management lock. A Contributor attempts to delete it and is blocked. Which interpretation is correct?

Answer: B. B explains how a management lock can prevent deletion despite a role that ordinarily permits the operation. A incorrectly makes Contributor an override for the lock. C confuses the intended restrictions of CanNotDelete and ReadOnly. D extends management-plane locks to data-plane deletion protection they do not provide. The administrator should inspect the lock's exact scope and inheritance and follow an authorized removal process if deletion is intended; additional resource permissions alone do not solve the observed restriction.

EX-Q11. An organization must prevent new deployments that violate an approved location rule. Which mechanism most directly expresses and enforces that configuration requirement?

Answer: C. C applies a configuration rule at the intended scope and can enforce supported deployment restrictions. A communicates a convention but does not enforce resource location. B observes spending rather than location compliance. D removes broad write capability instead of allowing compliant deployments and denying noncompliant ones. The policy definition, effect, parameters, exemptions, and scope must be reviewed together. An assignment with an audit effect would report relevant noncompliance but would not provide the preventive behavior requested in the question.

EX-Q12. A policy assignment identifies existing resources that need a supported deployIfNotExists remediation. What should be checked before starting the remediation task?

Answer: A. A checks the identity that performs the corrective deployment and whether its actual grants authorize the required changes. B verifies report visibility rather than the remediation identity's write permissions. C changes enforcement behavior and does not perform the equivalent corrective deployment. D assumes permission assignment that depends on the definition, tooling, and configuration. The administrator should also review the remediation target population and intended changes. A noncompliance result is evidence of evaluated state, not proof that a successful repair is authorized.

EX-Q13. A subscription budget reaches its configured notification threshold. What should the team expect from the budget alone?

Answer: C. C describes the distinction between cost monitoring and a hard limit on consumption. The team must inspect the configured notification or automation and its actual behavior. A and B incorrectly promise automatic resource removal or guaranteed spending cessation. D is unrelated to budget evaluation. Budget data and alerts can also have processing delay, so a threshold should be part of an active cost-management process. Any automated corrective action needs its own explicitly configured and tested scope.

EX-Q14. Azure Advisor recommends reducing a VM's capacity. The VM handles a known monthly processing peak that is not represented in the recent quiet period. What is the best response?

Answer: D. D treats the recommendation as useful input that must be connected to the actual operating requirement. A can remove capacity needed for the monthly peak. B destroys evidence needed for a sound decision. C discards potentially valuable guidance. The administrator should review representative performance, cost, availability requirements, and the recovery path for a resize. A quiet observation period can support an investigation, but it does not establish that the workload's maximum required capacity has permanently fallen.

EX-Q15. A storage design requires resilience across availability zones within a supported region. Which redundancy option most directly matches that stated placement requirement?

Answer: B. B distributes supported storage replicas across availability zones in the region. A provides local redundancy within its placement model rather than the stated zone distribution. C does not supply the requested managed storage redundancy. D changes data-tier lifecycle behavior and is not a redundancy choice. The administrator must still confirm that the selected account type, service, and region support ZRS. Zone resilience also does not automatically satisfy a separate requirement for recovery from a region-wide disruption.

EX-Q16. An administrator configures object replication for a supported blob scenario. What must still be considered separately when protecting against accidental deletion or corruption?

Answer: A. A distinguishes copying data from preserving and validating recoverable historical state. B promises every earlier state merely because another copy exists. C assumes a recovery procedure has been tested by the replication service. D discards protection settings without considering the specific failure. The administrator should examine replication behavior alongside versioning, retention, deletion protection, and backup requirements. A second copy can improve the design, but recovery from an unwanted change depends on which usable states remain available and how they can be restored.

EX-Q17. A storage account uses customer-managed encryption keys. The team plans to disable the relevant key version without examining dependent resources. What is the principal concern?

Answer: D. D identifies the storage service's dependency on the configured customer-managed key. A assumes an automatic fallback that should not be relied on. B ignores that ordinary data requests can depend on the service's ability to use the encryption key. C invents an automatic migration to an arbitrary key. Before disabling a version, the administrator should verify the actual configuration, supported rotation behavior, permissions, and recovery path. Key operations can affect availability even though the application never submits that key in its blob request.

EX-Q18. AzCopy can authenticate to a storage account, but a transfer from a particular workstation times out. The same operation works from an allowed network. What should be investigated next?

Answer: C. C follows the observed difference between the successful and failing client paths. A mistakes a timeout for proof of expired credentials, despite the stated authentication evidence. B changes data retention rather than connectivity. D incorrectly treats authentication as a replacement for a viable route and endpoint. The administrator should compare the resolved address and permitted path from both locations, including any private endpoint design. Replacing credentials without evidence can disrupt working clients while leaving the network failure unchanged.

EX-Q19. A team considers moving infrequently read blobs to a lower-cost storage tier. Which evaluation is necessary before making the change?

Answer: B. B evaluates the workload's required behavior and total cost rather than just the stored-capacity price. A omits access charges and any applicable minimum-retention implications. C assumes equivalent retrieval behavior across tiers, which is especially unsuitable when considering Archive. D confuses data access tier with redundancy. The administrator should consult current pricing and supported tier capabilities for the account and region, then compare expected reads, writes, retention, and recovery needs before applying a manual change or lifecycle rule.

EX-Q20. A lifecycle policy is intended to affect only blobs under the archive-input prefix. What should be reviewed before enabling its deletion action?

Answer: A. A examines the matching conditions and actions that determine which data can be deleted. B mistakes the display name for a selection filter. C assumes an interactive approval step in an automated lifecycle process. D treats lifecycle management as universal historical protection, although the treatment of current blobs, previous versions, and snapshots must be designed explicitly. The administrator should verify that the intended prefix and other conditions select only the approved population and that deletion timing respects the data owner's retention requirements.

EX-Q21. Blob versioning was enabled before an application overwrote a document with incorrect content. The account and container still exist. Which recovery approach most directly addresses the mistake?

Answer: C. C uses the historical state preserved by versioning for the specific overwritten object. A changes an unrelated placement concern and is not the direct recovery method. B changes credential access without restoring content. D risks further loss and does not establish which version is correct. The administrator should inspect the version timestamp and content, choose the point preceding the bad write, and verify the recovered document through the intended application. Existing version history is useful only when the correct retained version is selected.

EX-Q22. Container soft delete was enabled and a container was deleted within its retention window. What should be verified before promising recovery?

Answer: D. D checks the actual retained item and the recovery operation applicable to that deletion. A confuses creating a replacement container with restoring the deleted container. B extends container protection to a different resource deletion scenario. C assumes metadata can revive expired recovery state. The administrator should verify that the item remains recoverable, use the supported restore workflow, and inspect the resulting contents. Configuration before the incident enables protection, but the current service state is the evidence needed before promising recovery.

EX-Q23. Azure Files identity-based SMB access is configured through a supported directory model. A user has the required share-level authorization but cannot open one folder. Network connectivity is healthy. What should be inspected next?

Answer: A. A examines filesystem authorization, which can restrict access after share-level authorization succeeds. B changes capacity rather than the folder's access decision. C incorrectly treats management-plane Reader as a replacement for file and folder permissions. D confuses resilience configuration with authorization. The administrator should examine the actual identity, the attempted file operation, and applicable explicit and inherited permissions. Since the supported identity model and network path are already established, changing those unrelated controls would not directly investigate the stated folder-specific denial.

EX-Q24. A service SAS was issued using a stored access policy on a supported storage resource. The team needs to invalidate that policy-linked access without distributing a replacement account key to every client. What is the most targeted option?

Answer: B. B targets the policy to which this service SAS is linked, allowing its effective permissions or validity to be changed through the supported policy mechanism. A does not control SAS authorization. C adds unrelated management access. D ignores the specific policy-linked design stated in the question. This reasoning should not be generalized to every SAS type, because user delegation SAS and account SAS have different capabilities. The administrator must identify the token type and policy relationship before choosing a revocation method.

EX-Q25. A blob client has a valid Entra data role but reaches the storage account over a network path excluded by the account firewall. What outcome should be expected?

Answer: C. C reflects the need for both an allowed path and suitable data authorization. A overstates the role's effect, while B invents network behavior unrelated to TLS. D weakens a separate access control and does not fix the intended private or restricted route. The administrator should verify the resolved endpoint and configure the supported network path required by the workload. Once connectivity is permitted, the existing role must still authorize the requested operation at an applicable scope.

EX-Q26. A VM deployment fails because the selected size is unavailable in the chosen region or subscription context. The template parses correctly. What is the best next action?

Answer: D. D investigates the service-side conditions identified by the failure. A confuses source validation with available regional capacity and subscription quota. B grants more authority but cannot guarantee capacity for a requested size. C repeats the same inputs without distinguishing a transient allocation issue from an unsupported choice or quota constraint. The administrator should inspect the exact error, use a supported alternative or an appropriate quota request, and review the resulting cost, compatibility, and performance before changing the deployment parameters.

EX-Q27. A VM is stopped and deallocated, but its managed data disks remain. Which expectation is correct?

Answer: A. A distinguishes released compute allocation from persistent managed disks that remain deployed. B invents a free archival conversion. C confuses current disks with independently managed backup recovery points. D extends the compute state to unrelated charges such as retained storage or addresses. The administrator should inspect all associated resources when estimating ongoing cost or completing cleanup. Resources needed for restart or retention should be preserved deliberately, while unneeded disks and other billable resources require their own appropriate removal decisions.

EX-Q28. A running VM must be resized. The target size is supported for the workload, but the available resize path can involve interruption. What should the change plan include?

Answer: C. C connects the capacity change to its operational consequences and verifies the application afterward. A overstates availability during supported resize paths. B skips compatibility checks that can matter when changing VM families or capabilities. D invents a universal backup-deletion requirement and unnecessarily removes recovery options. The administrator should check available target sizes, understand any required stop or deallocation step, plan the resulting interruption, and verify workload behavior. A successful management-plane resize is not proof that every application requirement remains satisfied.

EX-Q29. A team is evaluating encryption at host for an Azure VM. What should be verified before applying it?

Answer: B. B checks feature support and the required enablement procedure for the actual VM configuration. A treats guest encryption and a platform host setting as interchangeable. C confuses a key-vault management permission with compute feature support and configuration. D changes network filtering rather than storage-path encryption. The administrator should consult current support guidance, understand any required VM state transition, and verify the resulting setting. Encryption features protect different parts of the system, so a related security control does not prove this one is enabled.

EX-Q30. A supported VM and its dependencies are moved to another resource group in the same subscription. What should not be assumed from that management move alone?

Answer: D. D is the unsupported assumption: moving management scope is not the same as relocating the workload to another region. A, B, and C identify legitimate review concerns. The administrator should validate supported resources and dependencies, examine references and access at the destination, and verify the workload after the move. A regional relocation is a separate process with its own data, network, capacity, and downtime considerations. Treating a resource-group move as regional migration can leave the actual resilience or location requirement unmet.

EX-Q31. A supported application must remain available if one availability zone in its Azure region fails. Which deployment decision most directly supports that requirement?

Answer: A. A distributes execution capacity across the failure boundary named by the requirement. B leaves every instance exposed to the same zone loss. C protects certain management operations rather than physical availability. D misinterprets an availability set and does not create multiple instances across zones. The application, data, traffic distribution, and other dependencies must also tolerate the failed zone. Compute placement supports resilience, but application continuity requires more than selecting a zone value on otherwise independent VM resources.

EX-Q32. A scale set can add instances, but each instance processes the same uncoordinated work file and creates duplicate results. What must be addressed before relying on scale-out?

Answer: B. B addresses the application's work-distribution and coordination behavior. A can multiply duplicate processing by creating more workers. C changes when scaling occurs without making the shared work safe. D confuses distinct network identities with ownership of application tasks. The team should design appropriate claiming, retry, and duplicate-handling behavior, then verify that additional instances increase useful completed work. Infrastructure scaling supplies capacity, but it does not automatically partition an arbitrary business input or make repeated processing harmless.

EX-Q33. An administrator changes a Bicep parameter used in several resource name properties. What should be examined before deployment?

Answer: C. C examines the effect of the parameter values on deployed resource names and full resource IDs. A incorrectly treats incremental mode as a renaming mechanism. B confuses local Bicep symbols with Azure resource identity. D gives the deployment-history name control over identities defined inside the template. The administrator should evaluate the produced names at their scopes and compare them with existing state and what-if output. A small parameter edit can create multiple new resources when it changes the names used to identify them.

EX-Q34. A Bicep build succeeds locally, but what-if reports an unresolved critical expression. What is the best interpretation?

Answer: D. D preserves the different scope of compilation and environment-aware preview. A claims application verification that did not occur. B turns an unknown into an unsupported conclusion. C treats execution as a substitute for understanding a potentially consequential change. The administrator should inspect the unresolved dependency or expression, use current diagnostics, and obtain appropriate additional evidence. What-if is valuable, but its analysis limitations need to be read alongside its proposed changes rather than ignored because the source compiled successfully.

EX-Q35. A team exports an ARM template from an existing resource group and decompiles it to Bicep. Which statement is accurate?

Answer: A. A describes the appropriate use of exported and converted infrastructure definitions. B confuses management configuration with workload contents and sensitive data. C overstates what language conversion proves. D ignores the separate need to preserve recoverable data. The administrator should validate the converted source, inspect environment-specific references and supported properties, and test the intended deployment behavior. A useful export can accelerate authoring, but maintained infrastructure source and application recovery require additional evidence and deliberate ownership.

EX-Q36. A registry changes from RBAC-only permissions to repository-level ABAC mode. An existing runtime identity can no longer pull its image using an old AcrPull assignment. What should be reviewed?

Answer: B. B addresses the registry's changed authorization model. Legacy roles such as AcrPull are not honored in the ABAC-enabled mode, so supported repository roles and conditions must be reviewed. A assumes backward behavior the selected mode does not provide. C confuses management visibility with repository data access. D introduces a shared credential while avoiding the intended identity design. Verify the actual pulling principal, repository, and applicable conditions, then test the image pull from the runtime through its supported network path.

EX-Q37. A Container App worker must scale from zero when a queue receives messages. Which design best matches that requirement?

Answer: C. C provides an external work signal that can support the desired lifecycle, together with the authorization and capacity bounds needed to use it. A lacks a running CPU signal at zero replicas. B supplies no mechanism to start work. D changes image storage capacity rather than application scaling. The administrator should verify the scaler's identity and queue scope, the worker's own processing permissions, and the application's ability to handle retries and multiple replicas safely before relying on the resulting scaling behavior.

EX-Q38. A staging slot uses a test database, while production must retain its production database connection after a swap. What should be configured and verified?

Answer: D. D makes the environment-specific intent explicit through the supported slot-setting behavior. A breaks the production dependency requirement. B removes the controlled release path and can cause unnecessary outage. C overgeneralizes because settings have defined swapping and non-swapping behavior. The administrator should inspect all important configuration, test the candidate release safely, and verify both content and retained environment settings after the swap. A reverse swap also does not automatically undo database changes made by the new application version.

EX-Q39. An App Service app needs to call a private storage endpoint. An inbound private endpoint has been created for the app, but no outbound integration was configured. Which next step addresses the missing direction?

Answer: A. A addresses the app-to-dependency path. App Service virtual-network integration supports outbound connectivity, while the app's private endpoint supplies an inbound private path. B confuses those directions. C changes naming for client access without establishing the required route. D adds release endpoints rather than dependency connectivity. After integration, the administrator must still confirm that storage resolves to the intended private address and that the runtime identity has permission for the requested data operation; network reachability does not replace service authorization.

EX-Q40. Two new virtual networks must eventually be peered. Their proposed address spaces are 10.20.0.0/16 and 10.20.8.0/21. No resources have been deployed. Which design change should happen first?

Answer: B. B resolves the address overlap: the entire proposed /21 lies within the /16. Ordinary virtual-network peering requires nonoverlapping address spaces. A changes traffic filtering but cannot disambiguate overlapping destinations. C changes resource organization without changing network addresses or peering requirements. D gives supported services a subnet-based access path rather than repairing inter-network addressing. Planning the change before resources exist also avoids later dependencies on allocated private addresses, name resolution, route tables, and application configuration.

EX-Q41. An otherwise empty Azure IPv4 subnet uses 10.30.1.0/27. Ignoring additional service-specific reservations, how many addresses remain after Azure's standard subnet reservations?

Answer: D. D follows from 32 total IPv4 addresses in a /27 minus Azure's five reserved addresses. A counts every address and ignores the platform reservation. B subtracts only the network and broadcast addresses familiar from some other network environments. C subtracts four rather than five. This calculation gives the general available address count, not a promise that every Azure service can use all remaining addresses; delegated services, deployment models, and growth requirements can impose additional sizing considerations.

EX-Q42. An administrator creates a public Azure DNS zone for a registered domain and adds the correct web record. Public resolvers still query the previous DNS provider. The administrator controls the registrar account. What should be checked and corrected?

Answer: A. A connects the domain's public delegation chain to the Azure-hosted zone. Creating records in an undelegated zone does not redirect public recursive resolvers. B concerns management authorization rather than authoritative DNS selection. C describes a private-zone relationship and is not the delegation mechanism for a public zone. D would not publish the required answer to internet clients. After correcting delegation, the administrator should verify the assigned name servers and allow for previously cached DNS answers and delegation information.

EX-Q43. Virtual networks A and B are peered, and B and C are peered. There is no direct A-to-C peering, gateway transit design, or routing appliance. Why should an administrator not assume that A can reach C through B?

Answer: C. C describes the missing relationship. Two peering connections through a common network do not automatically create a routed path between the other networks. A incorrectly makes resource-group placement a peering boundary. B confuses private connectivity with public addressing. D incorrectly limits the supported traffic to name resolution. The administrator must design an appropriate direct peering or supported transit arrangement, then verify routes, forwarding requirements, and security controls; a diagram shaped like a hub does not itself configure transit.

EX-Q44. A subnet's effective route sends a destination prefix to a network virtual appliance. The appliance VM is running, but packets do not reach the destination. Which next investigation most directly addresses the forwarding path?

Answer: B. B tests the behavior that a route entry cannot establish: receiving, processing, and forwarding packets with a viable return path. A changes a label without repairing forwarding. C treats VM availability as evidence of correct router or firewall configuration. D changes another subnet rather than the path used by the source. Effective routes show the selected next hop, but forwarding must also work at the appliance and operating-system layers. Applicable filtering and return-path decisions remain part of the end-to-end investigation.

EX-Q45. An inbound HTTPS connection reaches a subnet protected by an NSG. The VM's network interface also has an NSG. The subnet NSG permits the flow, while the NIC NSG denies it. Assume the application is listening and routing is correct. What is the expected result?

Answer: D. D reflects the combined filtering requirement when subnet and NIC NSGs both apply. A mistakes evaluation order for an override. B invents a global allow-wins rule across separate NSGs. C confuses the application's listening state with network authorization. Investigating effective security rules for the actual source, destination, protocol, and port helps identify the blocking rule. A listener is necessary for a successful application connection, but it does not make a denied network flow acceptable to Azure's filtering controls.

EX-Q46. In one NSG, an inbound deny rule at priority 150 and an inbound allow rule at priority 300 match the same new TCP flow. No other rule with a lower priority number matches. Which result follows?

Answer: C. C applies the NSG evaluation order: a lower numeric priority is evaluated first, and the first matching rule determines the result. A reverses that order. B uses statefulness to claim an override that it does not provide for the described new flow. D misunderstands distinct priority values, which establish rule order. The administrator should review the actual matching tuple and rule direction before changing priorities, because a broad earlier rule can unintentionally shadow a more specific later rule.

EX-Q47. Several VM network interfaces in one virtual network belong to the same application tier. An administrator wants NSG rules to refer to that tier without listing each private IP address. Which feature best fits this requirement?

Answer: A. A allows supported NSG rules to express a workload grouping through application security group membership instead of maintaining individual address lists. B grants management visibility to identities and does not define packet-filter membership. C organizes subscriptions for governance. D prevents certain management operations without defining permitted network flows. The group must be used in an appropriate security rule, and membership must be maintained as interfaces change; merely creating the group does not automatically open connectivity for the application.

EX-Q48. An administrator needs supported interactive RDP access to an Azure VM that has no public IP address. An appropriately configured Azure Bastion deployment can reach the VM's private address. What does Bastion provide in this design?

Answer: C. C states the relevant connectivity benefit of Bastion. A is incorrect because the administrator still needs an appropriate authentication method and authorization on the target. B is incorrect because the required network path and security rules must support the connection. D assigns operating-system maintenance and security assurance to a connection service. The exact Bastion SKU, client method, deployment prerequisites, and identity model should match the desired administrative experience; removing the VM's public IP does not remove every other access requirement.

EX-Q49. A storage account permits a subnet through a virtual-network rule, and that subnet has the appropriate service endpoint enabled. Which statement correctly describes the endpoint model?

Answer: B. B distinguishes service endpoints from Private Link. The service endpoint supplies the supported subnet relationship to the service's public endpoint; the storage network configuration must permit that relationship. A instead describes a private endpoint's network interface. C incorrectly removes data authorization, which remains necessary. D extends a subnet-specific rule to unrelated internet clients. Administrators should separately verify endpoint enablement, the storage account's network rules, name resolution, and the caller's credentials or role assignment when diagnosing a request.

EX-Q50. A VM should access Blob Storage through an approved private endpoint. The storage account blocks public network access, but the VM resolves the normal blob hostname to a public address. What should be investigated first?

Answer: D. D addresses the observed selection of a public address instead of the intended private endpoint address. A adds management authority without fixing DNS. B targets a different storage subresource. C assumes that endpoint approval automatically configures every client resolver. The administrator should examine the actual resolution chain, including custom forwarding and virtual-network links. Once the normal Blob hostname resolves to the intended private address, routing and data authorization must still permit the specific operation; fixing DNS establishes one part of the path.

EX-Q51. An Azure Load Balancer backend is unhealthy. The load-balancing rule uses TCP port 443, but the configured HTTP probe requests /health on port 8080. The application serves HTTPS on 443 and has no listener on 8080. What is the most direct correction?

Answer: A. A aligns the health test with a real backend service and its permitted probe path. B changes client name resolution without fixing the backend probe failure. C changes management permissions unrelated to the health listener. D incorrectly treats frontend rule configuration as proof of backend health. The administrator should choose the probe protocol, port, and any path deliberately, then verify the resulting health state. A reachable application port and a meaningful application health response answer related but distinct operational questions.

EX-Q52. A VM experiences brief CPU spikes, but a chart showing a one-hour average looks moderate. Which change best investigates whether short peaks are being hidden?

Answer: C. C examines how aggregation and time buckets shape the displayed evidence. A removes a collection configuration without answering the metric question. B changes unrelated metadata and does not relocate resources. D treats an average as if it described every instant. A maximum can reveal a peak, while an average describes the bucket's aggregate behavior; neither alone supplies complete application impact. The administrator should correlate the relevant period with workload latency or errors and confirm the metric's supported aggregation and sampling characteristics.

EX-Q53. A new Log Analytics workspace contains no expected resource logs. The administrator created the workspace but did not configure diagnostic settings or any required collection rules. What is the best next action?

Answer: A. A addresses the missing connection between a source and the destination workspace. Different signals use different supported mechanisms, such as diagnostic settings or agent collection governed by data collection rules. B assumes collection that has not been configured. C broadly changes permissions without establishing the data pipeline. D governs preservation of ingested data rather than making absent data arrive. After configuration, the administrator should allow for ingestion delay, confirm the expected table and time range, and investigate permissions or collection status if necessary.

EX-Q54. During planned maintenance, a team wants matching supported Azure Monitor alerts to remain visible while suppressing their action-group notifications for a defined window. Which configuration best matches that requirement?

Answer: D. D separates alert creation from the action response and can apply the required scope, conditions, and schedule. A removes a shared response configuration and can affect unrelated alerts after maintenance. B does not provide the requested alert behavior. C prevents the desired evidence from being generated. The administrator should verify that the alert type supports the processing rule and that the schedule uses the intended time zone. A suppression rule reduces notifications; it does not make the underlying resource healthy.

EX-Q55. Network Watcher IP flow verify reports that the applicable NSG rules allow a specific VM flow. The application's TCP connection still fails. What conclusion is justified?

Answer: B. B preserves the boundary of the diagnostic result. An allowed NSG decision answers a filtering question rather than proving the entire application path. A assumes a listening and functioning service that the test did not verify. C treats an IP-based flow evaluation as a complete DNS test. D confuses network filtering with service authorization. The administrator should use the observed destination and port to continue through effective routes, reachability tests, guest configuration, and application evidence until the failing layer is identified.

EX-Q56. An administrator must configure Azure Backup for a supported Azure virtual machine. Which vault family is used for this VM protection scenario?

Answer: C. C matches Azure VM backup to its supported vault family and acknowledges the required resource relationship. A stores monitoring data rather than VM recovery points. B manages supported secrets and cryptographic objects, not VM backup policies and recovery operations. D ignores the workload-specific distinction between Recovery Services vaults and Backup vaults. Administrators should check the currently supported workload, region, redundancy, and policy capabilities before deployment; a vault's display name is not evidence that it supports a particular protection method.

EX-Q57. A backup policy retains daily recovery points for 30 days. The last successful backup is three days old because recent jobs failed. Which statement best describes the current recovery position?

Answer: A. A distinguishes desired policy from successful protection evidence. Retention describes how eligible recovery points are preserved, while successful job completion and recovery-point availability establish what can actually be restored. B confuses retention duration with backup frequency. C substitutes the configured schedule for the failed execution result. D cannot recover data that was never captured by simply changing future retention. The administrator should investigate the failures, identify the latest usable recovery point, and communicate the resulting exposure against the workload's recovery objectives.

EX-Q58. An application database was corrupted at 14:00. Several earlier recovery points are available. Before replacing production, what is the strongest initial restore approach?

Answer: D. D combines time selection with a controlled validation step before production replacement. A can reproduce the corruption if the newest point includes the unwanted state. B removes useful evidence and increases recovery risk before a replacement is verified. C equates successful capture with complete application recovery. Validation should include relevant dependencies, consistency, access, and the business transactions needed after restoration. Isolation also helps avoid duplicate processing or unintended communication with production services while the recovered copy is being tested.

EX-Q59. A Site Recovery test failover creates a test VM in a separate virtual network. The application can still reach production queues through an allowed outbound path. What should be corrected before running the application's normal workers?

Answer: B. B addresses the remaining risk: network separation by name does not prevent an allowed outbound connection to a production dependency. A changes recovery-point selection without resolving duplicate processing. C assumes isolation beyond what the configuration provides. D changes production recovery state rather than making the test safe. A useful drill verifies recoverability while controlling both inbound and outbound effects, credentials, scheduled tasks, and application behavior. The test should be cleaned up through the supported workflow after its evidence has been recorded.

EX-Q60. A recovery report shows that every listed backup job succeeded. An administrator discovers that a newly deployed critical VM does not appear in the protected-item inventory. What is the most accurate assessment?

Answer: C. C separates job reliability from protection coverage. A gives successful executions responsibility for resources that were never enrolled. B mistakes subscription membership for a backup assignment. D changes a label instead of creating and verifying the required protection. A complete recovery review compares the intended inventory with protected items, checks recent usable recovery points, and considers restore validation. This avoids a misleading green dashboard that accurately reports the jobs it knows about while overlooking an important unprotected workload.

20.5 Building a Focused Revision Plan

Start with evidence, not a calendar

A revision plan should follow the gaps found in the assessment and practice record. Identify the few concepts whose misunderstanding affects several domains. Scope, identity, DNS, deployment dependencies, and recovery evidence often have this cross-cutting effect. Improving one of these can make several service-specific questions easier to interpret.

Group revision tasks by the decision they support. One session might compare permissions for resource management and blob data. Another might trace a connection through name resolution, routing, and filtering. A third might compare replication, versioning, backup, and regional failover against different failure requirements.

For each task, define a completion condition. Reading a chapter is not enough by itself. Require an independent explanation, a small successful operation, or a correct answer to a new question with reasoning. Completion conditions keep revision from becoming repeated exposure without demonstrated progress.

Use a repeatable study cycle

Begin a session by recalling the relevant concept before consulting the book. Write what the feature does and the condition that determines its use. Then compare that explanation with the source section. The difference reveals the exact gap.

Perform a small practice operation if it can clarify the gap. Keep the environment modest and record the active context. Observe the intended result and, where appropriate, a controlled failure. Cleanup belongs to the session rather than an indefinite later task.

Finish with a fresh decision. Change a permission scope, network condition, workload requirement, or recovery objective in the example. Explain which action now meets the requirement. This tests whether the concept transfers beyond the original wording.

Return to the topic after an interval. A correct immediate answer can reflect short-term recall of the explanation. A later independent explanation provides stronger evidence that the distinction has been retained. Adjust the interval and session length to the learner's schedule.

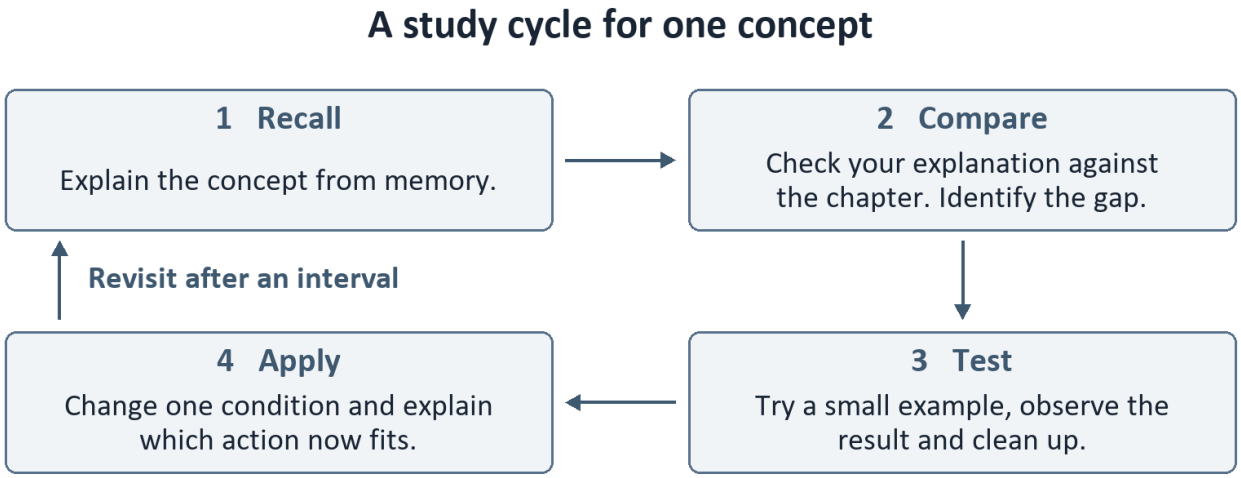


Figure 20.1 A repeatable study cycle

Keep a concise revision table

Topic	Evidence of the gap	Next action	Completion evidence
Resource and data permissions	Correct role family chosen at the wrong plane	Compare management Reader with Blob data roles	Explain and demonstrate the intended data read
Private service access	Endpoint exists but hostname resolves publicly	Inspect DNS zone and client resolution path	The intended client resolves the endpoint's private address
Recovery selection	Replication chosen for accidental overwrite recovery	Compare version history and replication	Restore known earlier contents and explain the boundary
Deployment troubleshooting	Permission changes attempted for a policy denial	Read operation-level deployment errors	Identify the denying control and a permitted correction

The entries are examples of useful structure, not a claim that every reader has these gaps. Replace them with observations from the actual assessment. Keep the table short enough to revisit at the start of a session.

Decide when to broaden practice

After narrow gaps are corrected, return to an integrated task. A learner can understand services individually while still missing the order in which dependencies must be established. The capstone provides a defined environment for checking that integration.

Explain the environment to another person or produce a concise handover record. Identify the actor, permission scope, network path, storage protection, monitoring evidence, and cleanup boundary. An unexplained join between components marks a useful next revision target.

Before scheduling an exam, review the current official objectives and exam information. This edition uses an explicit baseline, while the live certification can change. Compare any changes with the objective map and study the newly relevant material through current Microsoft documentation.

A useful revision session ends with a decision that can be explained and a result that can be checked. Keep the resource scope explicit when applying the lesson to another task. Administrative work continues to depend on these habits after an assessment is complete.

Further reading

- AZ-104 study guide and objectives
- Azure Administrator Associate certification

Conclusion: Beyond the First Deployment

The first resource-group exercise established a small discipline. Identify the active context, make a deliberate change, inspect the result, and account for cleanup. Later chapters applied that discipline to identities, networks, data, workloads, monitoring, and recovery. The resources became more complex, but the reason for each step remained clear.

An Azure administrator works with relationships. A role connects an identity to permitted operations at a scope. A route connects a destination with a next hop. A private endpoint connects a service to a network boundary. A recovery policy connects retained information with an intended recovery outcome. Understanding these relationships makes individual settings easier to interpret.

This understanding is visible in the way a task is described. “The application cannot reach storage” is a starting observation. A more useful account identifies the client identity, destination hostname, resolved address, network path, authorization mechanism, and attempted operation. Each additional fact narrows the investigation. Precision creates direction without requiring an immediate guess.

The value of a deliberate boundary

The book repeatedly returned to scope because scope determines where a decision applies. A permission assigned too broadly can exceed a requirement. A permission assigned too narrowly can leave the intended operation unavailable. A policy or lock can affect resources below its assignment point in ways that a resource-level inspection alone does not reveal.

Other boundaries require the same attention. A tenant is an identity context. A subscription is a resource-management and billing boundary. A region is a placement choice with service and resilience implications. A subnet is a network division. These boundaries coexist, but they do not replace one another.

An administrator who can name the relevant boundary can usually ask a better next question. If the problem concerns a blob read, inspect the data operation and its access path. If it concerns a deployment denial, inspect the management request and applicable controls. If it concerns recovery, identify the failed state and the retained information capable of restoring it.

The practical benefit is smaller, more informed change. A narrow correction is easier to evaluate and explain. It also preserves controls unrelated to the actual problem. Removing a broad restriction because one operation fails can turn an unresolved diagnosis into a larger operational risk.

Evidence that matches the promise

The book began with the difference between a resource existing and a resource meeting its purpose. That difference remains important after deployment. A virtual machine's running state does not prove application health. An alert rule's existence does not prove that the intended recipient receives a useful notification. A backup configuration does not prove that the required data can be restored.

Evidence should match the promise being made. A successful read supports a claim about a particular identity, operation, object, and path at a particular time. A restored file supports a claim about the

recovered contents. A broader claim requires broader testing. This is not a reason to avoid conclusions; it is a way to make conclusions dependable.

The integrated project used a known text object because its correctness could be checked directly. The initial write established a baseline. A controlled role change exposed an authorization dependency. A deliberate content change separated data correctness from connectivity. Restoration from a retained version then demonstrated a specific recovery result.

Each observation has a limit. A completed run of the project does not establish that every possible workload is resilient or that an entire organization can recover from a regional failure. It demonstrates the outcomes actually tested in that run. This habit of stating the boundary of evidence is part of sound administration.

Keep the environment understandable

Technical environments accumulate history. Resource names reflect earlier projects. Permissions remain after a role changes. A diagnostic setting can continue sending data to an old destination. A configuration that was once appropriate can become difficult to explain when its owner or purpose is no longer visible.

Good administration includes keeping that purpose recoverable. Use names and tags that help identify resources, while recognizing their limits. Maintain a concise inventory of important dependencies and owners. Keep deployment sources aligned with the configuration they are intended to describe. Record why an exception exists and when it should be reviewed.

A useful operating record is neither an unfiltered transcript nor a polished claim without detail. It contains the requirement, target, action, observation, and remaining limitation. Another administrator should be able to identify the resource and understand what was verified. Sensitive values should not be included merely because they appeared in output.

Cleanup belongs to the same discipline. Temporary resources become confusing when their expected lifetime is forgotten. A practice group that remains active after its exercise can continue incurring charges. A role assignment outside that group can remain after its original purpose disappears. The end of a task should include an explicit decision about what remains.

Continue learning through changed conditions

Azure will change after this edition. Interfaces will move, services will gain capabilities, and certification objectives will be revised. The useful response is to preserve the underlying reasoning while checking the current implementation. A familiar principle should not become an excuse to ignore a changed service boundary.

When documentation changes, identify what kind of change occurred. A renamed interface may leave the operation unchanged. A new authorization mode may alter the appropriate role assignment. A changed default network behavior may invalidate an old deployment assumption. These changes deserve different amounts of attention.

Use small experiments to understand unfamiliar behavior. State a prediction, choose a controlled environment, and define the observation that would confirm or challenge it. Keep the test narrow

enough that its result can be interpreted. Record a surprise rather than smoothing it into the expected story.

The same approach strengthens revision. A missed question can reveal a distinction that was understood only superficially. Return to the requirement and identify which condition changed the answer. Then test that distinction in a new example. Learning becomes more durable when it transfers beyond one remembered question.

Carry the learning into everyday work

The administrator does not need to recall every command from memory. It is more valuable to know which operation is needed, which context it requires, and how to verify its effect. Documentation, command help, and reusable scripts support that work. Their usefulness depends on a clear question.

Repeatable deployment sources make intended configuration visible. Monitoring makes selected aspects of operation observable. Recovery procedures make restoration actionable. None of these removes the need for judgment. They provide better material for that judgment when their assumptions and limitations are understood.

Collaboration also benefits from clarity. An application team can respond more effectively to a precise connectivity observation than to a vague request to “fix Azure.” A security team can evaluate an access requirement more effectively when the principal, operation, and scope are named. A manager can understand a recovery decision when its outcome and evidence are expressed plainly.

Certification preparation gives a useful structure to this learning, but the work continues after an exam. New workloads introduce new constraints. Existing workloads change. A dependable administrator keeps the connection between requirement, configuration, and observed behavior intact as those conditions evolve.

The next deployment can begin with a familiar set of questions. What outcome is required? Which identity will act? What resource and scope are involved? Which dependencies must work? What observation will demonstrate success? Asking them deliberately gives the administrator a clear direction, even when the surrounding environment is new.

Appendix A: AZ-104 Objective Coverage Map

This map uses the skills measured from April 17, 2026. Objective wording is paraphrased for navigation. The teaching sections provide the explanation; the practical checks below provide focused ways to demonstrate each capability. A chapter lab or review question may reinforce several rows without exercising every variation. Record checks as unexecuted until evidence is collected in an authorized practice environment.

The domain ranges in this baseline are identity and governance 20–25 percent, storage 15–20 percent, compute 20–25 percent, networking 15–20 percent, and monitoring and recovery 10–15 percent. Review the Microsoft AZ-104 study guide for changes before scheduling an exam.

Identity and governance

Objective	Sections	Practical check
I01 Create users and groups	2.2, 2.3	Create a temporary user and security group, verify object IDs and membership, then remove them.
I02 Maintain user and group properties	2.2, 2.3	Change one permitted property and verify the exact object before and after the update.
I03 Administer Entra user licenses	2.4	Inspect available licenses, assignment prerequisites, and the resulting service-plan state.
I04 Manage external users	2.5	Trace invitation, redemption, assigned resource access, and removal for a test guest.
I05 Configure self-service password reset	3.2	Verify eligible users, registration, allowed methods, and a permitted reset test.
I06 Use built-in Azure roles	3.3	Choose a built-in role whose actions match a specific administrative requirement.
I07 Assign roles at different scopes	3.4	Assign a limited role at a practice group and inspect its inherited effect on a member resource.

Objective	Sections	Practical check
I08 Interpret access assignments	3.5	Explain effective access using the principal, role, scope, inherited grants, and applicable denies.
I09 Implement Azure Policy	5.1, 5.2, 5.3	Evaluate a practice assignment, inspect compliance, and distinguish evaluation from remediation.
I10 Configure resource locks	4.5	Apply a delete lock to an empty practice group, observe denial, and remove the exact lock.
I11 Manage resource tags	4.4	Inspect group and resource tags separately and verify an explicit resource-tag update.
I12 Manage resource groups	4.3	Create, inventory, and remove a dedicated empty group in the intended subscription.
I13 Manage subscriptions	4.2	Compare selected subscription context, directory association, and applicable administration boundaries.
I14 Use budgets, cost alerts, and Advisor recommendations	5.4, 5.5	Inspect a scoped budget and explain its alert behavior without assuming a spending cap.
I15 Configure management groups	4.1	Read an authorized hierarchy and predict the scope of an assignment above a subscription.

Storage

Objective	Sections	Practical check
S01 Configure storage firewalls and network access	11.5	Compare the intended client's allowed network path with a deliberately disallowed path.
S02 Create and use SAS tokens	11.4	Design a minimal-scope, time-limited SAS and explain its allowed operation and exposure.

Objective	Sections	Practical check
S03 Configure stored access policies	11.4	Relate a supported service SAS to a stored access policy and describe revocation behavior.
S04 Manage storage account keys	11.4	Explain a staged key-rotation sequence that accounts for clients and key-signed SAS tokens.
S05 Configure identity-based Azure Files access	11.2	Verify the supported identity model, share permission, and file-system permission for a client.
S06 Create and configure storage accounts	9.1	Inspect account kind, region, performance offering, security settings, and resulting endpoint.
S07 Configure storage redundancy	9.2	Choose a supported redundancy option for an explicit failure and data-location requirement.
S08 Configure object replication	9.3	Check source and destination prerequisites, policy rules, and evidence of replication.
S09 Configure storage encryption	9.4	Inspect the configured key-management choice and explain required access dependencies.
S10 Use Storage Explorer and AzCopy	9.5	Transfer a small test object with the intended identity and verify destination contents.
S11 Create and configure Azure Files shares	11.1	Create a classic practice share, inspect its settings, and explain the client protocol.
S12 Create and configure Blob containers	10.1	Create a private container and verify its name, authorization path, and access setting.
S13 Configure storage access tiers	10.2	Compare access, retrieval, retention, and cost consequences before changing a test blob's tier.

Objective	Sections	Practical check
S14 Configure soft delete for blobs and containers	10.5	Test blob deletion and container deletion separately with the relevant protection enabled.
S15 Configure Azure Files snapshots and soft delete	11.3	Identify the recovery object and demonstrate recovery from a supported snapshot or deleted-share state.
S16 Configure blob lifecycle management	10.3	Evaluate a rule's filters and conditions against sample objects without assuming immediate execution.
S17 Configure blob versioning	10.4	Record a good version, overwrite a test blob, and recover the known earlier contents.

Compute

Objective	Sections	Practical check
C01 Interpret ARM templates and Bicep files	14.1, 14.2	Trace parameters, resources, dependencies, and outputs in the companion template.
C02 Modify an ARM template	14.3	Change a parameterized property and inspect the resulting template and proposed resource change.
C03 Modify a Bicep file	14.3	Change an allowed parameter or resource property and compile the revised file.
C04 Deploy resources using ARM or Bicep	14.4	Validate the intended scope, review what-if, deploy, and inspect resource-level results.
C05 Export ARM templates and convert ARM to Bicep	14.5	Inspect an export or decompilation for missing settings and repair it before reuse.
C06 Create virtual machines	12.1	Verify image, size, identity, network, disk, guest access, and provisioning evidence.

Objective	Sections	Practical check
C07 Configure encryption at host	12.4	Check size and subscription prerequisites, inspect the setting, and explain its encryption boundary.
C08 Move VMs across groups, subscriptions, or regions	12.5	Compare supported move and regional relocation procedures with their dependency requirements.
C09 Manage virtual-machine sizes	12.3	Check available sizes and capacity, plan downtime, and inspect the resized VM.
C10 Manage virtual-machine disks	12.2	Inspect disk roles and attachment state before a permitted attach, resize, or detach exercise.
C11 Use availability zones and availability sets	13.1, 13.2	Compare the intended failure boundary with the actual VM placement and dependencies.
C12 Deploy and configure scale sets	13.3, 13.4	Inspect the instance model, capacity, health, and scaling behavior of a practice design.
C13 Create and manage Container Registry	15.2	Verify registry mode, image reference, and the identity's supported push or pull authorization.
C14 Provision Container Instances	15.3	Deploy a small container and inspect its state, logs, endpoint, and resource requests.
C15 Provision Container Apps	15.4	Inspect environment, revision, image, ingress, and identity settings for a small app.
C16 Size and scale ACI and Container Apps	15.5	Compare explicit resource requests and each hosting service's supported scaling model.
C17 Provision App Service plans	16.1	Choose an appropriate region, operating system, and tier for the required app features.

Objective	Sections	Practical check
C18 Scale App Service plans	16.2	Distinguish tier changes from instance-count changes and account for shared-plan workloads.
C19 Create App Service applications	16.1	Inspect app runtime, plan relationship, deployment configuration, and a working endpoint.
C20 Configure App Service certificates and TLS	16.3	Verify the certificate, hostname binding, and intended TLS settings.
C21 Map custom DNS names to App Service	16.3	Trace domain verification, DNS records, and the app's hostname binding.
C22 Configure App Service backups	16.5	Check supported tier and backup scope, then identify an appropriate restoration test.
C23 Configure App Service networking	16.4	Distinguish inbound restrictions or private endpoints from outbound virtual-network integration.
C24 Configure deployment slots	16.5	Compare slot settings and swap behavior, then verify the intended app configuration.

Networking

Objective	Sections	Practical check
N01 Create virtual networks and subnets	6.1, 6.2	Plan nonoverlapping CIDR ranges and inspect the created subnet boundaries.
N02 Configure virtual-network peering	6.3	Verify both peering directions and explain which additional connectivity is not automatic.
N03 Configure public IP addresses	6.2	Inspect SKU, allocation, association, and the separate traffic-permission controls.

Objective	Sections	Practical check
N04 Configure user-defined routes	6.4	Relate a route table and subnet association to effective next-hop selection.
N05 Troubleshoot connectivity	6.5	Trace the client, destination, DNS, route, filtering, listener, and return path.
N06 Configure NSGs and application security groups	7.1	Create a narrowly scoped practice rule and inspect its source and destination grouping.
N07 Evaluate effective NSG rules	7.2	Explain the combined NIC and subnet rules that permit or deny a specified flow.
N08 Implement Azure Bastion	7.3	Check supported SKU, network requirements, target reachability, and authorized guest access.
N09 Configure service endpoints	7.4	Relate subnet service-endpoint configuration to the service's network access rule.
N10 Configure private endpoints	7.5	Verify target subresource, connection approval, private address, and client DNS.
N11 Configure Azure DNS	8.1, 8.2	Distinguish public delegation and records from private-zone links and resolution.
N12 Configure internal or public load balancers	8.3, 8.4	Trace frontend, backend pool, probe, and traffic rule for the intended flow.
N13 Troubleshoot load balancing	8.5	Compare probe health, service listener, security rules, and client-to-frontend behavior.

Monitoring and recovery

Objective	Sections	Practical check

Objective	Sections	Practical check
M01 Interpret Azure Monitor metrics	17.1	Explain metric scope, aggregation, time grain, and dimension before interpreting a chart.
M02 Configure Azure Monitor log collection	17.2	Verify collection source, diagnostic or collection configuration, and destination.
M03 Query and analyze logs	17.2	Run a scoped KQL query and interpret time range, records, and missing-data limits.
M04 Configure alerts, action groups, and processing rules	17.3	Trace signal evaluation, action routing, and a suppression or processing condition.
M05 Use Insights for VMs, storage, and networks	17.4	Inspect required telemetry and explain the selected resource's actual observations.
M06 Use Network Watcher and Connection Monitor	17.5	Choose the diagnostic observation that distinguishes the suspected connectivity failure.
M07 Create Recovery Services vaults	18.1	Match a supported workload and region to a Recovery Services vault.
M08 Create Backup vaults	18.1	Match a supported data source to a Backup vault and its required permissions.
M09 Configure backup policies	18.2	Connect schedule and retention to the stated recovery requirement.
M10 Perform Azure Backup and restoration	18.3	Create a usable recovery point and verify a restored workload or selected data.
M11 Configure Site Recovery	18.4	Inspect replication prerequisites, target dependencies, and readiness evidence.
M12 Perform regional Site Recovery failover	18.4	Plan a suitable test, isolate test networking, and verify the recovered operation.

Objective	Sections	Practical check
M13 Interpret backup reports and alerts	18.5	Distinguish protected-item configuration, job outcomes, recovery points, and actual restore evidence.

Appendix B: Azure CLI and PowerShell Reference

These examples are lookup aids for concepts taught in the chapters. Select the intended subscription and inspect the target before changing resources. Replace placeholders with verified values. Bash and PowerShell syntax must not be mixed, and the context used by Azure CLI must not be assumed to match an existing Az PowerShell or Microsoft Graph session.

Context and inventory

Purpose	Azure CLI	Azure PowerShell
Inspect active context	`az account show`	`Get-AzContext`
List accessible subscriptions	`az account list`	`Get-AzSubscription`
Select subscription	`az account set --subscription ID`	`Set-AzContext -Subscription ID`
List resource groups	`az group list`	`Get-AzResourceGroup`
Inspect a group	`az group show --name NAME`	`Get-AzResourceGroup -Name NAME`
List group members	`az resource list --resource-group NAME`	`Get-AzResource -ResourceGroupName NAME`

An inventory is scoped by the account's permissions and the selected context. An absent result does not by itself prove that a resource never existed. Check filters, identifiers, and access before creating a replacement.

Resource-group lifecycle

Use a dedicated empty group when practicing this sequence. The creation commands can update some properties of an existing group, so inspect the name before reuse.

```
lab_group="rg-compass-reference"
lab_region="westeurope"
az group create --name "$lab_group" --location "$lab_region"
az group show --name "$lab_group" --output json
az resource list --resource-group "$lab_group" --output table
# Delete only after verifying this group contains no unrelated work.
az group delete --name "$lab_group" --yes
az group exists --name "$lab_group"
```

```powershell
$labGroup = "rg-compass-reference"
$labRegion = "westeurope"
New-AzResourceGroup -Name $labGroup -Location $labRegion
Get-AzResourceGroup -Name $labGroup
Get-AzResource -ResourceGroupName $labGroup
# Inspect the confirmation target before accepting removal.
Remove-AzResourceGroup -Name $labGroup
```

Read before changing access

Use exact principal identifiers and scopes. A list scoped to one resource should be interpreted with the tool's inheritance and filtering behavior in mind; it is not automatically an exhaustive effective-permissions analysis.

```
az role assignment list --scope "<resource-scope>" --include-inherited --output table
az role definition list --name "Reader" --output json
Get-AzRoleAssignment -Scope "<resource-scope>"
Get-AzRoleDefinition -Name "Reader"
```

A management Reader role is different from a storage data role. Directory roles also belong to a different authorization system from Azure resource RBAC. Use Chapter 3 to interpret the relationship rather than inferring it from a familiar name.

Deployment inspection

Local compilation produces a template without provisioning resources. What-if and deployment validation use the selected Azure context and can require permissions. A successful deployment must still be followed by resource and functional checks.

```
az bicep build --file companion/ch14-deploy.bicep
az deployment group what-if --resource-group "<group>" --template-file companion/ch14-deploy.bicep
--parameters @parameters.local.json
az deployment group list --resource-group "<group>" --output table
az deployment operation group list --resource-group "<group>" --name "<deployment>" --output json
```

VM and storage observations

```
az vm show --resource-group "<group>" --name "<vm>" --show-details
az vm get-instance-view --resource-group "<group>" --name "<vm>"
az storage account show --resource-group "<group>" --name "<account>"
az network nic show --resource-group "<group>" --name "<nic>"
```

The management reads above do not demonstrate guest application health or authorized blob access. Use an appropriate client and data operation for the latter. Explicitly select Entra authorization where supported rather than allowing a data command to obtain an account key unexpectedly.

Use help deliberately

Inspect command help before relying on an unfamiliar option. Help is especially useful for accepted enum values, output structure, optional extensions, and the command's resource scope. A CLI or module version can affect available flags even when the underlying service supports an operation.

```
az group create --help
az vm create --help
```

```
Get-Help New-AzResourceGroup -Full
Get-Command Get-AzResourceGroup
```

Keep reusable scripts parameterized and inspect their target identifiers. A short command can still have a broad consequence when its scope is a subscription or management group. The ease of entering a command says little about the size of its effect.

Appendix C: ARM and Bicep Reading Reference

Read a deployment file as a description of intended resource state. First identify its target scope, then trace inputs, resource declarations, dependencies, and outputs. Resource API versions and supported properties belong to the specific resource type. A valid file can still request a configuration that the selected subscription cannot deploy.

Element	Question to answer
Target scope	Which management boundary receives the deployment?
Parameter	What must the caller supply, and what default or validation applies?
Variable	Which value is derived inside the template?
Resource declaration	Which resource type, name, API version, and properties are requested?
Existing resource	Which object must already exist at the stated scope?
Module	Which separately defined deployment is being invoked?
Dependency	Which resource or value must be available before another operation?
Output	Which nonsecret result is useful to the caller?

The chapter 14 companion template provides a small example. Its network and storage account are management resources. It does not upload data or grant a user blob access. Its optional lock must be removed explicitly before cleanup; setting the lock parameter to false in an incremental deployment does not automatically delete the previously created lock.

The chapter 19 template adds a VM identity, container-scoped data role, private endpoint, DNS integration, and diagnostic destination. Those references explain dependencies that would be harder to track in a long sequence of independent creation commands. They do not remove the need to verify runtime behavior.

Review changes before deployment

Compare the proposed configuration with the current environment. A parameter change can update an existing resource, create a different resource, or require a replacement procedure. Resource names and scopes determine identity. Do not assume that changing a name merely relabels an existing object.

Pay attention to secrets and output values. A secure parameter type reduces exposure in relevant deployment handling, but a later output or log statement can still reveal a sensitive value. The supplied

templates require no private key or embedded password. The VM parameter accepts SSH public-key text only.

An exported template is a starting description, not a guaranteed complete recovery package. Exports can omit unsupported configuration, contain resource-specific values that need parameters, and exclude application data or external dependencies. Decompilation from ARM to Bicep also requires review before the result becomes a maintained source.

Inspect failures at the appropriate level

A top-level deployment failure can contain several successful resource operations and one failed operation. Read the operation details. A syntax issue, missing action permission, policy denial, name collision, unavailable SKU, and runtime dependency failure require different corrections.

After correction, review the next deployment's proposed changes. Repeated deployment should be intentional, especially where a previous attempt left partial state. Keep the deployment name, scope, inputs, and meaningful error details in the practice record.

Appendix D: KQL and Troubleshooting Reference

Kusto Query Language expresses questions about collected data. The query's usefulness depends on the source, table, time range, and collection configuration. An empty result can mean that no matching event occurred, but it can also mean that the event was never collected, has not arrived, or is outside the query's scope.

Start with a narrow observation

The following examples assume the named tables exist in the selected workspace. Table availability depends on the relevant collection configuration. They are teaching queries, not evidence that this edition queried a live environment.

```
AzureActivity
| where TimeGenerated > ago(24h)
| project TimeGenerated, OperationNameValue, ActivityStatusValue,
          ResourceGroup, Caller
| order by TimeGenerated desc
```

Use this pattern to inspect management activity collected into the workspace. The activity record concerns management operations. It does not replace a service's data-operation logs.

```
Heartbeat
| where TimeGenerated > ago(1h)
| summarize LastSeen=max(TimeGenerated) by Computer
| order by LastSeen asc
```

A recent heartbeat indicates that the relevant reporting path produced a record. It does not prove that every application on the machine is healthy. A missing heartbeat requires investigation of the agent, configuration, network, and collection scope as well as the machine itself.

```
StorageBlobLogs
| where TimeGenerated > ago(1h)
| summarize Requests=count() by OperationName, StatusCode
| order by Requests desc
```

This aggregation helps identify the distribution of observed Blob operations and statuses. It does not reveal the correctness of a file's contents. The capstone's content comparison supplies that separate evidence.

Read a query in order

Identify the source table, then inspect filters. Check whether the time expression matches the event being investigated. Read projections to see which fields remain visible. Read aggregations to understand which detail has been collapsed into a count, average, maximum, or other summary.

Grouping by an unexpected dimension can make a result look more fragmented than the underlying question requires. Omitting an important dimension can conceal differences between resources or outcomes. Choose dimensions because they help distinguish an explanation, not simply because they are available.

Troubleshoot with a question

Observation	Next discriminating check
Resource is absent from a portal list	Verify directory, subscription, filters, and exact resource identifier
Management request is denied	Inspect actor, action, scope, inherited permissions, policy, and error category
Name resolves to an unexpected address	Inspect client resolver, authoritative records, private-zone links, and forwarding
TCP connection fails	Inspect route, effective filtering, destination listener, and return path
HTTPS responds but data access fails	Inspect the actual identity, token audience, operation, and data permission
Alert does not notify	Separate signal evaluation, alert state, action group, and processing rules
Backup is configured but restore is unavailable	Check supported workload, completed job, usable recovery point, and restore prerequisites

Avoid making several unrelated changes before collecting another observation. If a change succeeds, its value depends on knowing which condition it corrected. Keep a successful baseline when one exists and compare the failing state against it.

Appendix E: Lab Preparation and Cleanup Reference

Each chapter lab states its own requirements. This appendix provides a common preparation record so that the lab's instructions can be applied to a known environment. Complete the record using actual values rather than relying on the defaults in a copied command.

Before the exercise

Record the intended tenant, subscription, resource group, and region. Confirm the selected tool's active context. Identify the exact permission needed for both management and data operations. If the lab creates role assignments, remember that ordinary resource Contributor permission does not include that capability.

Read the resource inventory and likely charges. A free credit balance does not make every service free. A budget does not automatically stop spending. Check current prices, service tiers, and regional availability for resources that will remain running.

Identify the cleanup boundary. A dedicated resource group is useful, but directory objects and assignments at higher scopes may be outside it. A backup or retention feature can create additional cleanup steps. Record these before the first creation action so they are not forgotten at the end.

During the exercise

Keep a small evidence record with the intended operation, exact target, expected result, observed result, and relevant identifier. Mark expected output as expected until it has been observed. Do not copy tokens, passwords, private keys, or unnecessary personal data into the record.

Treat a failed step as an opportunity to locate the failing layer. Shell parsing, authentication, authorization, policy, networking, service capability, and application behavior are distinct possibilities. Read the first meaningful error rather than assuming that every failure needs a broader role.

If a lab requires a dependency from an earlier exercise, recreate and verify it explicitly. Do not assume that a resource survived cleanup or that an earlier identity assignment remains in place. Independent verification makes the exercise repeatable.

At cleanup

List resources inside the practice group and compare them with the expected inventory. Investigate unfamiliar objects before deletion. Inspect locks and other controls without removing unrelated organizational safeguards. Remove only the resources and assignments created for the authorized exercise.

Wait for deletion or deallocation operations to complete and inspect the resulting state. A stopped guest is different from a deallocated VM, and retained disks or public IP resources can remain relevant to cost. Data-retention and vault behaviors may also require a separate review.

Inventory items created outside the group, including test identities, role assignments, budgets, DNS delegation changes, and downloaded credentials. Remove or deliberately retain each item, and record anything that remains with its reason.

Glossary

These definitions summarize the meanings used in this book. Service-specific prerequisites and limitations remain in the relevant teaching sections.

Access tier. A storage classification that changes access and cost characteristics. A tier choice must account for retrieval behavior, expected use, and any applicable minimum retention or rehydration conditions.

Action group. A reusable collection of actions associated with Azure Monitor alerts. A configured action group does not prove that a particular alert condition fired or that its notification reached the recipient.

Activity log. A record of Azure management-plane events. Service data operations require the appropriate service-specific logs; they are not automatically equivalent to activity-log entries.

Application security group. A logical grouping used to express supported network-security rules for application-related network interfaces. It helps describe the intended source or destination without relying solely on individual IP addresses.

ARM template. A JSON deployment description used by Azure Resource Manager. It declares resources and their configuration, with parameters and expressions supporting reusable deployment.

Authentication. The process of establishing an identity. Successful authentication does not by itself establish permission to perform a particular operation.

Authorization. The decision about whether an identity may perform an operation on a target under the applicable permissions and controls.

Availability set. A VM placement arrangement that distributes supported infrastructure dependencies across fault and update domains. It does not create additional application instances or provide regional recovery by itself.

Availability zone. A physically separated failure boundary within a supported region. Service behavior differs between zonal placement and zone-redundant configurations.

Azure CLI. A command-line tool whose commands begin with az. The tool's selected context and the surrounding shell syntax both affect how an operation is performed.

Azure Policy. A governance service that evaluates resource configurations against assigned rules. Effects such as audit, deny, modify, and deployment of related configuration have different behaviors and prerequisites.

Azure Resource Manager. Azure's principal resource deployment and management layer. It evaluates applicable management requests and passes supported operations to the relevant resource provider.

Azure role. A definition of permitted Azure resource-management or data operations. A role assignment also needs a principal and scope before it grants the defined permissions.

Backup policy. A configuration that defines supported backup scheduling and retention behavior for a protected workload. A policy assignment is not evidence that a usable recovery point already exists.

Backup vault. An Azure Data Protection resource used for supported backup data sources. Its supported workloads differ from those of a Recovery Services vault.

Bastion. An Azure service that provides supported administrative connectivity to virtual machines without requiring a public IP on each target VM. SKU, network, identity, and guest-access requirements still apply.

Bicep. A declarative language for Azure resource deployment. Its compiler produces an ARM template; compilation alone does not validate subscription permissions, quota, or all runtime conditions.

Blob. An object stored in Azure Blob Storage. Its content, metadata, access tier, versions, and protection settings are related but distinct aspects of administration.

Budget. A cost-management configuration used to track spending against a defined amount and trigger supported notifications. A budget does not automatically impose a hard spending cap.

CIDR. Classless Inter-Domain Routing notation, which represents an IP address range using an address and prefix length. Overlap and subnet containment matter when connecting networks.

Container. In Blob Storage, a grouping of blobs. In compute, an isolated application execution unit based on an image. The surrounding subject determines which meaning applies.

Container image. A packaged filesystem and configuration used to start a container. An image reference can identify a registry, repository, and tag or digest.

Container Registry. An Azure service for storing and distributing container artifacts. Registry authorization mode and the operation being attempted affect the appropriate permission.

Control plane. Another term commonly used for the management plane. It concerns resource configuration and lifecycle rather than the service's ordinary data operations.

Data collection rule. A configuration that describes supported telemetry collection and routing. Its association and the relevant agent or service determine which data can reach the destination.

Data plane. The service interface through which data is used, such as reading a blob. Data authorization can differ from the permission used to configure the resource.

Deployment slot. A separate App Service deployment environment within supported plan capabilities. A slot swap changes supported application configuration and traffic relationships; it is not a general database rollback.

Diagnostic setting. A resource configuration that routes supported logs and metrics to selected destinations. It must be applied at the appropriate resource or subresource scope.

DNS. Domain Name System, which resolves names using records and a resolution path. Public delegation and private-zone linking solve different naming problems.

Effective access. The permissions that actually apply to an identity at a target after considering relevant assignments, inheritance, conditions, and deny behavior.

Encryption at host. A VM encryption capability that extends encryption protection to supported host-side storage paths. Availability and prerequisites must be checked for the selected VM configuration.

Endpoint. An address or service connection through which an operation is attempted. Endpoint existence, network reachability, and authorization are separate conditions.

External user. A user from outside the directory's ordinary internal membership context who is represented through a supported collaboration arrangement. Invitation alone does not grant every resource permission.

Fault domain. A grouping associated with shared infrastructure failure dependencies. Availability designs distribute resources across relevant failure domains where supported.

Health probe. A test used by a service such as a load balancer to evaluate backend suitability. Probe success is narrower than complete application correctness.

Idempotence. The property of an operation reaching the same intended result when repeated under the stated conditions. It does not mean that every changed parameter or external side effect is harmless.

Identity. The representation of a person, application, or resource acting in a system. Credentials and tokens support authentication, while roles and other controls determine authorization.

Inheritance. The application of a setting or assignment through a hierarchy according to that feature's rules. Different Azure controls do not necessarily inherit in the same way.

Initiative. A group of Azure Policy definitions managed together for a governance purpose. Assignment parameters and individual policy effects still require inspection.

KQL. Kusto Query Language, used to query supported telemetry datasets. Table selection, time filters, dimensions, and aggregation determine what a result means.

Lifecycle rule. A storage rule that applies supported actions to objects meeting specified filters and conditions. Evaluation is asynchronous rather than an immediate reaction to saving the rule.

Load balancer. A service that distributes supported network flows across configured backends. Frontends, backend pools, probes, and traffic rules must fit the intended connection.

Lock. A management control that restricts supported resource changes or deletion at its scope. It is not a universal data-access or data-deletion control.

Log Analytics workspace. A destination and query boundary for supported Azure Monitor log data. Workspace existence does not establish that every source has been configured to send data.

Managed disk. An Azure resource that represents storage attached to a VM. Its role, performance, encryption, attachment state, and lifecycle require separate decisions.

Managed identity. An identity associated with an Azure resource through a supported platform mechanism. It can obtain tokens without an application storing a conventional secret, but it still needs authorization.

Management group. A hierarchy element used to organize subscriptions and apply supported governance or access controls above individual subscriptions.

Management plane. The set of operations used to configure and manage resources. It is distinct from service data operations and from a guest operating system's internal administration.

Metric. A numerical observation associated with a resource or service. Aggregation, time grain, dimensions, and the observation's definition affect interpretation.

Microsoft Entra ID. Microsoft's cloud identity service. A tenant contains directory objects and identity configurations; directory roles and Azure resource roles have different purposes.

NAT gateway. An Azure service that provides explicit outbound connectivity for supported subnet resources. It does not by itself expose an inbound application endpoint.

Network interface. A resource that connects a VM to a subnet and associated addressing configuration. Effective connectivity also depends on routes, filtering, DNS, and the workload.

Network security group. A set of priority-ordered network traffic rules applied at supported subnet or network-interface scopes. Applicable rules and stateful behavior must be considered together.

Object replication. A Blob Storage replication feature that copies supported objects between configured source and destination accounts under defined prerequisites and rules.

Peering. A connection between virtual networks. Peering configuration does not automatically provide every transitive route or resolve all DNS and security requirements.

Principal. An identity object to which a role or permission can be assigned, such as a user, group, service principal, or managed identity's service principal.

Private endpoint. A network interface that provides private connectivity to a supported service subresource. Approval, DNS, network reachability, and data authorization remain relevant.

Provisioning state. The status of a resource creation or configuration operation. A successful provisioning state does not necessarily demonstrate application health.

Public IP address. An address resource associated with supported public connectivity scenarios. Its presence does not independently permit traffic through security controls.

RBAC. Role-based access control. Azure RBAC connects a principal, role definition, and resource scope to determine supported permissions.

Recovery point. A retained state from which a supported restoration can be performed. Its existence and suitability must be checked against the recovery requirement.

Recovery point objective. The acceptable amount of data loss expressed as a time interval. It is a requirement for a recovery design, not an automatic guarantee from a feature name.

Recovery Services vault. A resource used for supported Azure Backup and Site Recovery scenarios. Workload support and configuration differ from those of a Backup vault.

Recovery time objective. The acceptable time to restore the required operation after a disruption. Dependencies and procedures affect whether a design can meet it.

Region. An Azure geographic deployment area with specific service and resilience capabilities. A resource's region and its resource group's metadata location are separate concepts.

Resource group. A subscription-level management grouping for related resources. It provides a useful lifecycle and assignment scope but does not describe every operational dependency.

Resource identifier. A structured identifier locating an Azure resource in its management hierarchy. It is more precise than a display name that can repeat in another scope.

Resource provider. The Azure service component exposing a resource namespace, resource types, and supported operations through the management model.

Route. A rule used to determine the next hop for traffic to a destination prefix. Effective route selection depends on the applicable route sources and precedence.

SAS. Shared access signature, a token that grants specified access to supported storage resources under defined conditions. Its signing method and service type affect capabilities and revocation.

Scale out. Increase the number of workload instances. It differs from increasing the capacity of each instance and requires suitable application and service behavior.

Scale up. Increase an instance's capacity or service tier. Its effect, availability implications, and supported features depend on the service.

Scope. The boundary at which an operation or assignment applies. Common Azure RBAC scopes include management groups, subscriptions, resource groups, and individual resources.

Service endpoint. A subnet feature used with supported services and their network rules. It differs from a private endpoint and does not create a private service network interface.

Service principal. A directory representation of an application's identity in a tenant. It can receive resource permissions appropriate to the application's operations.

Site Recovery. An Azure service for supported replication and failover scenarios. Replication readiness is one component of a complete recovery plan.

SKU. A specific service offering, size, or tier. Feature availability, price, capacity, and regional support can depend on the selected SKU.

Snapshot. A retained point-in-time representation supported by a particular service. Its recovery scope and consistency properties depend on that service.

Soft delete. A protection feature that retains supported deleted objects for a configured period. Different object types and services have distinct restoration behavior.

SSPR. Self-service password reset. Eligibility, registration, authentication methods, licensing, and hybrid writeback requirements determine the supported reset path.

Stored access policy. A server-side policy used with supported service SAS arrangements. It is not a universal mechanism for controlling every SAS type.

Subnet. A subdivision of a virtual network's address space. Service delegation, routes, security groups, and endpoint configuration can affect its use.

Subscription. An Azure resource-management and billing boundary associated with an identity tenant. Access to one subscription does not imply access to another.

Tag. A name-value metadata pair used for organization and supported reporting or automation. Tags are not a security boundary and do not universally inherit.

Tenant. An instance of Microsoft Entra ID providing a directory and identity boundary. It is distinct from an Azure subscription or resource group.

TLS. Transport Layer Security, used to protect communication. Certificates, hostname bindings, protocol settings, and client behavior determine the resulting connection.

Token audience. The service or resource for which an access token is intended. A token acquired for one audience is not generally a substitute for a token for another service.

User-defined route. A route configured in a route table and associated with a subnet. It participates in effective routing rather than replacing every route indiscriminately.

Versioning. Retention of distinguishable versions of a supported object. An earlier version can help recover from an unwanted change when it has been preserved.

Virtual machine scale set. A service for operating a group of VMs through a supported orchestration model. Capacity, instance health, updates, and scaling require explicit configuration.

Virtual network. An Azure private-network boundary with an address space and subnets. Connectivity to other networks or services depends on configured relationships and controls.

What-if. A deployment preview that reports expected resource changes in the selected scope. It helps review intent but does not guarantee every runtime outcome.

Appendix F: Chapter Review Answers

Chapter 1

01-Q1. A learner can inspect a storage account's properties but cannot read a blob using an Entra token. What should be examined first?

Answer: A. Reading management properties and reading blob data are different operations. The relevant data permission and network path must be checked. Display settings and dashboards do not authorize access, and matching group and account regions is not a general data-access prerequisite.

01-Q2. A single VM is placed in an availability zone. Which conclusion is justified?

Answer: B. A zonal placement identifies a failure boundary. It does not create additional application instances, configure regional failover, or provide recovery from data deletion. Those outcomes require other arrangements.

01-Q3. A resource group is located in one region. What does that establish about a VM inside it?

Answer: C. The group's location concerns its metadata. Member resources have their own placement where applicable. Neither required co-location nor replication follows from the group location, and VMs do have regional placement.

01-Q4. Two subscriptions contain a resource group named rg-training. What best distinguishes the exact target of a change?

Answer: D. The identifier locates the group in its subscription. Names can repeat across subscriptions, while browser position and dashboard history are navigation conveniences rather than target identifiers.

01-Q5. The portal shows an empty resource list immediately after a successful deployment in another session. What is the best first action?

Answer: B. An empty view can result from context or filters. Recreating resources can produce duplicates, while removing locks or broadening access changes unrelated controls before the cause is known.

01-Q6. A Bash example fails in PowerShell before an Azure request is sent. What should be checked first?

Answer: A. Parsing occurs in the shell before Azure evaluates the request. Policy, replication, and probes do not correct a local syntax mismatch. The example must use syntax appropriate to the selected shell.

01-Q7. An empty practice resource group was created successfully. Which observation verifies that the intended subscription was targeted?

Answer: C. The subscription segment of the returned identifier is direct evidence of target context. A resource name, portal branding, and an open terminal do not identify the subscription that received the change.

01-Q8. Before deleting a practice resource group, its inventory reveals an unfamiliar VM. What is the appropriate action?

Answer: D. The cleanup boundary must contain only resources authorized for removal. An unfamiliar resource needs investigation. A training label does not establish ownership, and renaming or selectively deleting the VM does not resolve that uncertainty.

Chapter 2

02-Q1. Which identifier is most suitable for confirming that an automation script is targeting the intended Microsoft Entra directory?

Answer: B. The tenant ID identifies the directory. A display name is a friendly label, department describes a user property, and a resource-group name identifies a resource-management container rather than a directory.

02-Q2. A user can manage virtual machines as an Azure Contributor but cannot create a cloud-only directory user. What best explains the difference?

Answer: C. Azure Contributor grants resource operations within its scope, not user-administration authority in Entra. Subscriptions are not required per user, Contributor is a role rather than a product license, and VM location does not authorize directory creation.

02-Q3. An administrator deletes an account and later creates a new account with the same visible name. Why should old access references be investigated?

Answer: A. The new object has a distinct object ID, so references to the old object do not become references to the new one merely through a matching name. Display-name reuse is not the controlling issue, roles do not automatically transfer by name, and users can have multiple memberships.

02-Q4. A synchronized user's department repeatedly returns to its former value after a cloud-side edit. What is the best next check?

Answer: D. The authoritative source can overwrite a downstream value. VM state and public IPs do not explain directory property synchronization, and broad privilege is not a substitute for editing the correct source.

02-Q5. A group should automatically reflect a reliable department attribute. Which design best matches that requirement, assuming required licensing and supported rules?

Answer: C. Dynamic membership can evaluate the trusted attribute. Unreviewed assigned membership does not automate department changes, a name alone adds no members, and a role for every tenant user is neither the requested group behavior nor suitably scoped.

02-Q6. A licensed group contains another group as a member. Users exist only inside the nested group. What should the administrator expect from group-based licensing?

Answer: B. Group-based licensing does not expand nested groups into licensed users as assumed here. Immediate universal assignment is incorrect, licenses do not become Azure roles, and licensing does not convert account types.

02-Q7. An employee is moving between two groups that provide the same required product. Which sequence best avoids an avoidable entitlement gap?

Answer: A. Adding and verifying the new assignment before removing the old source preserves continuity. Recreating an account changes identity, removing both sources creates a gap, and a display-name change does not establish entitlement.

02-Q8. An invited external user authenticates successfully but cannot read an Azure resource group. Which check most directly addresses the remaining resource-access question?

Answer: D. Resource access depends on the local principal's effective Azure authorization. A matching organization name does not grant access, the resource tenant generally does not administer the external identity's home password, and a tag is not an access assignment.

Chapter 3

03-Q1. A user signs in successfully but a resource update returns an authorization error. Which statement is most accurate?

Answer: B. Authentication and authorization answer different questions. A successful sign-in does not prove the update is permitted. A wrong password would concern authentication, SSPR does not grant resource write access, and recreating the tenant is unrelated and excessive.

03-Q2. A workload uses its managed identity to read blobs. The human deployer has Owner but the workload is denied. Which principal's data permissions should be investigated first?

Answer: C. The workload's managed identity is the caller. The deployer's Owner role does not automatically transfer to it. Unrelated guests and the tool publisher are not the stated caller, and a display name is not a principal.

03-Q3. An ordinary user belongs to an SSPR-enabled pilot group but has no acceptable registered verification methods. What is the most direct missing prerequisite?

Answer: A. SSPR requires registered methods that satisfy its policy. Contributor, public IPs, and resource groups do not supply those verification methods.

03-Q4. Cloud-only pilot users reset passwords successfully, while identities whose passwords are managed on premises fail. Which investigation best matches the pattern?

Answer: D. The differing password authority points to hybrid dependencies such as writeback. Azure scope does not repair password recovery, a group rename does not change authority, and deleting successful accounts removes evidence without fixing the failure.

03-Q5. A principal needs to inspect a resource group's configuration without changing resources. Which candidate is the narrowest appropriate choice among these?

Answer: B. Reader at the specified group matches configuration inspection with a narrow scope. Owner and Contributor allow unnecessary operations and broader reach; User Access Administrator addresses authorization management rather than the stated inspection task.

03-Q6. A user has Contributor at subscription scope and Reader on one child resource group. What does the child Reader assignment do?

Answer: C. RBAC grants are generally additive; a narrow Reader assignment does not subtract broader Contributor permission. It neither changes the definition into a data role nor blocks subscription access.

03-Q7. An action is excluded through NotActions in one role, but another applicable role grants it. In the absence of a deny or other restriction, what is the correct interpretation?

Answer: A. NotActions subtracts from that role's allowed actions; it is not a deny assignment overriding other grants. Role names are not ordered this way, roles remain valid, and password changes do not resolve the permission-set interpretation.

03-Q8. A user has the required management action but the error identifies a read-only resource lock. What is the appropriate next focus?

Answer: D. The lock is a separate management control. More RBAC privilege does not make it irrelevant, password reset addresses authentication, and a storage data role does not remove a management lock.

Chapter 4

04-Q1. What is the principal purpose of an Azure management group?

Answer: B. Management groups organize subscriptions for inherited governance. They do not host disks, replace resource-group lifecycle containers, or perform product-license assignment.

04-Q2. A subscription is moved to another management-group branch. Which consequence must be reviewed?

Answer: C. The new ancestry can change inherited policy and access. It does not inherently move physical resources, grant every user Owner, or alter resource-type naming rules.

04-Q3. Which evidence most directly confirms the CLI's selected resource-management target?

Answer: A. Subscription and tenant IDs establish the current context. Window appearance, personal names, and extension count do not identify the selected subscription.

04-Q4. A subscription has sufficient configured quota for a VM size. What should the administrator still recognize?

Answer: D. Assigned quota is not a reservation of physical capacity. It also does not grant authorization or override policy; regional support and allocation conditions still apply.

04-Q5. Which grouping best matches a resource-group lifecycle boundary?

Answer: B. A shared lifecycle gives the group a coherent management purpose. Arbitrary initials or resource types can combine unrelated owners, and identical private addresses are not a resource-group design requirement.

04-Q6. A resource group has an Environment tag. What is the correct expectation for contained resource tags by default?

Answer: C. Resource tags do not automatically inherit from the group; a deliberate supported process is needed. Tags do not grant network isolation or create locks.

04-Q7. A team wants to prevent accidental deletion while retaining ordinary permitted configuration updates. Which lock is the closer fit?

Answer: A. CanNotDelete targets deletion while allowing otherwise permitted updates. ReadOnly is more restrictive, a tag is descriptive metadata, and an unrelated user's Reader role does not protect everyone else's operations.

04-Q8. A storage account has a management delete lock. Which claim would be incorrect?

Answer: D. Management locks do not provide universal data-plane deletion protection. The other statements describe supported management-lock relationships; blob data requires its own protection controls.

Chapter 5

05-Q1. What causes a policy definition to govern a resource population?

Answer: B. An assignment applies the definition. A tag, user Reader assignment, or viewing the definition does not activate policy governance.

05-Q2. A deployment caller has sufficient RBAC permission, but a deny policy rejects the requested region. What is the correct response direction?

Answer: C. The request violates a property rule, so the configuration or approved exception process must address it. Password and unrelated data roles do not correct the region, and broad resource privilege does not ordinarily bypass policy.

05-Q3. What is an initiative?

Answer: A. An initiative groups related policy definitions. It does not cap spending, replace authorization assignments, or route network traffic.

05-Q4. A dashboard shows a favorable aggregate compliance percentage. What should a reviewer still inspect?

Answer: D. An aggregate can hide significant failures or changes in evaluated population. Names and interface appearance do not establish compliance, and color alone lacks diagnostic detail.

05-Q5. Which policy effects are central to supported remediation of existing resources?

Answer: B. Modify and deployIfNotExists support the described remediation tasks. Audit observes rather than repairs, Reader and Contributor are roles, and the final pair are lock levels.

05-Q6. A remediation task lacks permission to deploy its required related resource. Which identity's actual assignments must be checked?

Answer: C. The assignment's managed identity performs supported remediation changes. Guests and display names are not the actor, and the observer's permission does not prove the remediation identity has access.

05-Q7. What does a Cost Management budget do by itself when its threshold is crossed?

Answer: A. Budgets notify and track; they do not inherently stop resource consumption, delete resources, cap invoices, or reserve capacity.

05-Q8. Advisor recommends downsizing a lightly used standby VM. Which response is best?

Answer: D. The workload requirement and evidence determine whether the proposed change is suitable. Automatic application or label-based dismissal avoids that analysis, and estimated savings are not guarantees.

Chapter 6

06-Q1. How many total IPv4 addresses are in a /26 prefix before platform reservations?

Answer: B. A /26 leaves six host bits, so 2^6 gives 64 addresses. The other counts correspond to different prefix sizes and do not include the stated calculation.

06-Q2. Which pair is suitable for ordinary peering address-space planning?

Answer: C. The 10.60 and 10.61 /16 ranges do not overlap. The first pair overlaps by containment, and the other pairs repeat a range; names do not resolve address ambiguity.

06-Q3. In an Azure subnet 10.60.1.64/26, which listed address is the first ordinarily assignable address?

Answer: A. Azure reserves the first four and last addresses. .64 through .67 and .127 are reserved, making .68 the first ordinarily assignable address, subject to existing allocations and service requirements.

06-Q4. A private-subnet VM has a default route but no explicit outbound method. Which assumption is unsafe under modern defaults?

Answer: D. Private-subnet defaults require explicit outbound connectivity for public endpoints. A default route alone is insufficient. Private addresses, route inspection, and DNS needs remain valid observations.

06-Q5. A is peered with B, and B with C. What does ordinary peering alone establish?

Answer: B. Peering is not automatically transitive. A transit design needs additional supported routing and forwarding. It also does not merge subscriptions or grant resource-management roles.

06-Q6. Routes 10.61.0.0/16 and 10.61.1.0/24 both match destination 10.61.1.20. Under ordinary prefix selection, which is more specific?

Answer: C. The /24 fixes more network bits and covers a smaller matching range. Numeric size of the written prefix length is not a preference for broader routes, names do not decide selection, and both prefixes match.

06-Q7. A user-defined route table contains correct entries but is not associated with the intended subnet. What is the likely consequence?

Answer: A. The subnet association applies the custom table. Creation alone does not attach it globally, route tables do not replace filters, and routing does not grant application permission.

06-Q8. A diagnostic confirms NSG permission for a TCP flow, but the application still fails. Which next statement is most accurate?

Answer: D. An NSG result verifies that control only. Other path and application conditions remain possible. Broad roles or deleting peerings are not justified by this evidence.

Chapter 7

07-Q1. A client connects to a server's HTTPS service. Which port assumption best matches an ordinary TCP client flow?

Answer: B. The service listens on destination 443 while the client normally uses an ephemeral source port. Mirroring 443 is unnecessary, 22 is ordinarily SSH, and CIDR length is unrelated to transport ports.

07-Q2. Within one NSG direction, an allow at priority 200 and a broader matching deny at priority 150 apply. What is the decision?

Answer: C. NSG evaluation uses the first matching rule in priority order. Priority 150 precedes 200; names and an assumed specificity preference do not override that order, and the matching rules are not ignored.

07-Q3. What does an application security group collect for use in NSG rules?

Answer: A. An ASG groups supported network endpoint configurations. Licensing, billing, and directory-user recovery use different systems.

07-Q4. A subnet NSG allows a new flow while the target interface NSG denies it. What is the result?

Answer: D. Traffic must satisfy both applicable NSGs. Their rules do not form one global priority list, and a subnet allow does not bypass the interface filter.

07-Q5. A dedicated Bastion deployment is planned using an offering other than Developer. Which subnet design matches current new-deployment requirements?

Answer: B. Dedicated non-Developer deployments require the specifically named dedicated subnet and current minimum size. A small arbitrary subnet, shared application placement, or universal no-subnet claim is incorrect.

07-Q6. What does enabling a service endpoint on a subnet do by itself?

Answer: C. Service endpoints establish the supported subnet connectivity relationship; service-side rules and data authorization still need configuration. They do not create a private endpoint IP or universally disable public endpoints.

07-Q7. A storage private endpoint is created successfully. Which additional action is needed if the requirement is private-only access?

Answer: A. Creating the endpoint does not automatically block public access. The target's public-network control must be configured and tested. Removing data roles breaks authorization, while names do not implement network restrictions.

07-Q8. A VM resolves a storage hostname to a public address after the account's public access is disabled. What should be investigated first for the intended private path?

Answer: D. The client is selecting the wrong address path, so DNS configuration is the immediate lead. Broader resource permissions, descriptions, and product license counts do not change name resolution.

Chapter 8

08-Q1. An Azure public DNS zone contains the correct A record. Its assigned authoritative server returns it, but ordinary internet resolvers still query the previous provider. Which change addresses the most likely cause?

Answer: B. B changes which authoritative provider the public hierarchy identifies. A affects packet filtering, not authority. C is unrelated to delegation. D concerns private-zone registration and cannot redirect public delegation.

08-Q2. Two peered networks exchange application traffic by private IP. A client using Azure-provided DNS cannot resolve a record in a private zone linked only to the other network. What is the most direct correction?

Answer: D. D supplies the missing resolution relationship for the stated resolver path. A does not expose the zone to the client. B changes traffic design without fixing naming. C removes valid records unnecessarily.

08-Q3. An application should accept connections only from clients with private network connectivity. Which frontend matches that requirement?

Answer: A. A provides a private contact point, with routes and security still required. B is publicly addressed and lacks serving backends. C does not carry application traffic. D creates public endpoints rather than the required private frontend.

08-Q4. An HTTP probe requests /health. The application redirects to /signin. Why can the backend remain unhealthy although a browser eventually shows a sign-in page?

Answer: C. C identifies the mismatch: HTTP probes expect status 200, while browsers can follow redirects. A is false. B is not a probe prerequisite. D is unnecessary behind a load balancer.

08-Q5. A backend passes its probe, but clients from one subnet cannot connect. Other subnets work. Which investigation fits the evidence?

Answer: B. B examines the differing client path; probe and client traffic can be governed differently. A ignores working clients and established naming evidence. C does not match the scoped symptom. D discards useful health evidence without repairing connectivity.

08-Q6. A public load balancer serves inbound traffic, but backend VMs cannot contact an external package repository. What needs independent review?

Answer: A. A addresses backend-initiated traffic, distinct from incoming service connections. B does not establish outbound routing or translation. C does not affect packet flow. D affects inbound backend eligibility rather than providing outbound connectivity.

08-Q7. An administrator lowers a TTL while changing an A record. Some clients retain the former answer. Which explanation is accurate?

Answer: D. D explains why TTL reduction is planned before cutover. A is false. B confuses caching with established connections. C is an unjustified generalization and does not explain normal caching.

08-Q8. A VM answers a web request by private IP, but its HTTP probe is unhealthy. Which test directly checks the health contract?

Answer: C. C tests the exact endpoint and required response. A changes domain ownership, not readiness. B affects metadata. D changes caching and cannot repair the health endpoint.

Chapter 9

09-Q1. A new operator can create a storage account but receives an authorization failure when uploading with Microsoft Entra authentication. Network access is permitted. What should be checked first?

Answer: A. Management permission does not automatically grant blob data operations. The data role is relevant; a display label, TXT record count, and domain ownership do not grant upload permission.

09-Q2. Which requirement most directly distinguishes ZRS from LRS?

Answer: B. ZRS addresses zonal infrastructure failures. Neither option automatically reverses authorized deletion, supplies a second writable region, or removes the need for appropriate recovery controls.

09-Q3. Why must a disaster-recovery plan consider recent data loss when using geographic storage replication?

Answer: C. Asynchronous replication means recent primary writes may not yet be present at the secondary. Encryption remains supported, secondaries are not necessarily empty, and the copies are service-managed rather than application-local disks.

09-Q4. Which prerequisite set matches supported block-blob object replication?

Answer: D. The source change feed and versioning on both sides support object replication. Destination-only change feed is insufficient, disabling versioning violates prerequisites, and Azure Files is not the block-blob replication feature.

09-Q5. A report is outside the prefix filter of a replication rule. It never appears at the destination. What best explains the result?

Answer: A. The filter determines eligibility. Objects do not need public IPs, disabling encryption is neither required nor appropriate, and a reader's Owner role does not change which objects the policy selects.

09-Q6. What additional operational dependency does customer-managed storage encryption introduce?

Answer: B. The storage identity must be able to use the required key. Customer-managed encryption does not require anonymous data access or account-key credentials, and it does not replace data authorization.

09-Q7. An AzCopy job reports some files skipped. What is the best next action before declaring a migration complete?

Answer: C. The job report and configured semantics explain skipped items and guide validation. Source deletion is premature, corruption is unproven, and removing network controls does not explain intentional skip behavior.

09-Q8. Which statement correctly separates redundancy from version-based recovery?

Answer: D. D identifies distinct purposes. A is false because permitted changes can be replicated. B confuses version history with location. C incorrectly treats current-state replication as unlimited historical retention.

Chapter 10

10-Q1. A tool displays reports/2026/file.csv as folders in a flat-namespace blob account. What does that display prove?

Answer: A. A is the supported interpretation. A folder-like display does not establish hierarchical namespace, create a file share, or guarantee filesystem semantics.

10-Q2. A dataset needs infrequent but immediate reads. Which choice is least consistent with that requirement?

Answer: B. Archive is offline and requires rehydration. The other listed online tiers can support immediate reads, though their cost suitability differs.

10-Q3. A lifecycle rule uses time since modification, but a nightly process rewrites the blob. Why might the object never qualify?

Answer: C. C explains the mismatch between the business age and technical modification age. Container naming alone, anonymous access, and encryption do not create the asserted restrictions.

10-Q4. Which review best protects against a lifecycle rule affecting the wrong dataset?

Answer: D. D validates scope. Parsing checks syntax rather than business meaning; short retention and deleting policies do not establish correct object selection.

10-Q5. An incorrect overwrite is discovered while the application continues writing. What belongs in the recovery plan?

Answer: A. A prevents repeated overwrites and verifies the recovery source. B leaves the cause active. C destroys evidence. D ignores the possibility that several previous writes were already wrong.

10-Q6. Which feature directly addresses accidental deletion of an entire container within its configured recovery window?

Answer: B. Container soft delete protects the container scope. Tiering changes access economics, and DNS or naming length does not retain deleted containers.

10-Q7. A deleted container's original name has been reused by a new active container. What is the appropriate response?

Answer: C. C respects the original-name restoration constraint while protecting new valid data. A can cause further loss, B invents automatic merging, and D is unrelated and inappropriate.

10-Q8. Why can a storage bill remain substantial after the current versions of many blobs are removed?

Answer: D. D identifies billable retained states. A ignores retention, B misstates usage-based charging, and C confuses unrelated services and makes an unsupported universal claim.

Chapter 11

11-Q1. A portal upload to a file share succeeds using an account key. What does this establish?

Answer: A. Only the credential and path actually tested are established. Key-based portal access does not validate user SMB permissions, upstream SMB reachability, or backup.

11-Q2. A user has a share-level role allowing modification, but the target directory's ACL permits only reading. What is expected?

Answer: B. Share and path authorization both apply. A broader outer allowance does not erase a narrower directory restriction, and neither public access nor identity-source changes follow automatically.

11-Q3. An individual file is deleted while its parent classic share remains active. Which protection should be inspected for earlier content?

Answer: C. Snapshots or backup can recover individual files. Share soft delete addresses deletion of the entire share; DNS and display names do not retain file content.

11-Q4. Which SAS type can use a stored access policy under the described model?

Answer: D. Stored access policies apply to supported service SAS relationships. Account SAS and user delegation SAS are ad hoc and do not use that policy mechanism.

11-Q5. Why should consumers be inventoried before regenerating an account key?

Answer: A. A identifies the dependency risk. Clients are not universally migrated automatically, account keys are shared credentials rather than personal roles, and regeneration does not rename files.

11-Q6. A blob private endpoint exists for an account. Does that alone provide the account's file service a private endpoint?

Answer: B. Private endpoints target service subresources. Blob configuration does not automatically establish Files private access; name length and token expiry do not alter that relationship.

11-Q7. An HTTPS upload works from a workstation, but its SMB mount fails before authentication. Which test is especially relevant?

Answer: C. SMB uses TCP 445, a different path from HTTPS. Lifecycle rules, blob tags, and snapshot deletion do not establish SMB transport reachability.

11-Q8. A SAS permits reading one blob but not listing its container. A browsing tool fails during listing. What is the best interpretation?

Answer: D. D distinguishes an operation mismatch from service failure. The correct remedy depends on the required workflow; broad keys or removal of network controls are not justified by this evidence.

Chapter 12

12-Q1. A VM reports Running. Which conclusion is justified?

Answer: A. A limits the claim to observed power state. Running does not establish application dependencies, guest filesystem setup, or backup success.

12-Q2. An attached managed data disk appears in Azure, but the application cannot use a filesystem on it. What should be checked next?

Answer: B. Azure attachment and guest preparation are separate. B examines that boundary; DNS, tags, and an arbitrary larger SKU do not initialize or mount the correct disk.

12-Q3. A VM needs more capacity. Which statement about resizing a running VM is accurate?

Answer: C. C reflects the disruptive operation. Resizing does not inherently relocate the VM or guarantee a performance fix; compatibility and bottleneck evidence remain necessary.

12-Q4. Which additional boundary does encryption at host address beyond persisted managed-disk encryption?

Answer: D. D identifies the host-level storage boundary. Encryption does not decide application permissions, preserve backups, or manage DNS authority.

12-Q5. A VM is moved to a resource group whose metadata location is another region. What happens to the VM's compute region from that move alone?

Answer: A. Resource-group metadata location does not relocate member resources. The other choices invent unrelated transformations.

12-Q6. A VM cannot install packages after deployment into a new network. No explicit outbound design was configured. What is a relevant first investigation?

Answer: B. Package downloads need a usable outbound path. Unrelated DNS changes, disk deletion, and broader management permission do not establish that connectivity.

12-Q7. Which storage location should not hold the only copy of irreplaceable application data?

Answer: C. Temporary local storage can lose contents during relevant events and is intended for reconstructable data. The other choices can form part of a persistent protection design, though each still needs correct configuration.

12-Q8. What must be reviewed before a supported cross-subscription resource move?

Answer: D. D covers the actual move requirements. Display details and filename extensions do not establish resource eligibility, destination readiness, or operational continuity.

Chapter 13

13-Q1. Two application VMs are correctly placed in an availability set. What additional condition is needed for the service to continue when either VM becomes unavailable?

Answer: A. A is correct because placement alone does not provide traffic distribution, state protection, or sufficient surviving capacity. B creates an address conflict rather than redundancy. C describes a feature availability sets do not provide. D changes management organization and does not establish runtime availability.

13-Q2. An administrator creates one VM explicitly in Zone 1. Which statement accurately describes this choice?

Answer: B. B is correct: a zonal VM is placed in one zone and is not automatically duplicated. A confuses placement with replication. C confuses a compute location with a separate storage and regional replication design. D requires a broader application and recovery architecture that has not been described.

13-Q3. A scale-set model reports a desired capacity of four, but only three instances have successfully provisioned. What should the administrator conclude?

Answer: C. C is correct because model intent and actual instance state are separate observations. A also confuses provisioning with application health. B invents a deletion action; a load-balancer probe controls traffic eligibility rather than proving an allocation outcome. D discards useful evidence and is not justified by the limited information.

13-Q4. Which statement distinguishes Flexible and Uniform scale-set orchestration?

Answer: D. D captures the administrative resource-model distinction. A incorrectly excludes a supported Flexible capability. B ignores the different instance resource models and operations. C is wrong because the choice is made during creation and changing models needs a supported migration plan.

13-Q5. A VM scale set has an active autoscale profile with minimum two and maximum two. It currently has two instances, and its CPU scale-out condition is satisfied. What is the expected constraint?

Answer: A. A is correct because autoscale acts within the profile's inclusive bounds. B reverses the purpose of the maximum. C confuses horizontal autoscale with a vertical resize. D attributes resource creation to a budget, which monitors expenditure and does not supply capacity.

13-Q6. A worker population can add VMs quickly, but each new worker needs several minutes before completing a job. Which measurement best informs cooldown and scheduled preparation?

Answer: B. B measures useful capacity through allocation, startup, configuration, and readiness. A observes controller intent without proving an instance exists. C has no causal relationship to readiness. D proves power state but can omit configuration, dependency initialization, and application readiness.

13-Q7. Existing workers process jobs, but newly scaled-out workers fail while downloading startup packages. Their subnet has no intended explicit outbound path. What is the most appropriate next investigation?

Answer: C. C addresses the observed provisioning dependency and preserves evidence. A creates more failing capacity without repairing the path. B overlooks that existing instances may have been configured earlier and no longer need that download. D removes diagnostic evidence and expands disruption without identifying the cause.

13-Q8. A worker keeps an unfinished job's only result on local temporary storage. What must be addressed before allowing automated scale-in?

Answer: D. D protects useful work when capacity is removed. A changes a label without changing behavior. B cannot make temporary local state durable. C confuses infrastructure success with an application transaction and risks losing the unfinished result.

Chapter 14

14-Q1. A Bicep author renames a local resource symbol but preserves its type, Azure name, scope, and properties. What change is expected in the deployed resource identity?

Answer: B. B is correct because a local symbolic identifier is distinct from the deployed resource name and scope. A does not follow from a symbol rename. C confuses the symbol with the name property. D describes no requested lifecycle operation.

14-Q2. A Bicep file compiles successfully on a laptop. Which conclusion is justified?

Answer: D. D states the scope of compilation. A requires authorization checks in Azure. B depends on regional service capacity and quota. C requires deployment and an operational test; compilation does not run the application.

14-Q3. An incremental deployment includes an existing resource but omits some previously configured properties. What should the reviewer do?

Answer: A. A is correct because incremental resource preservation does not mean every unspecified property is preserved. B is the common mistaken interpretation. C introduces deletion behavior without solving the configuration question. D is unnecessary and can destroy data or availability.

14-Q4. A what-if result reports that a critical expression could not be evaluated. What is the best next action?

Answer: C. C responds to the actual uncertainty. A converts an unknown into an unsupported conclusion. B is too strong because analysis limitations do not necessarily prevent deployment. D discards useful information without addressing the unresolved expression.

14-Q5. A template successfully creates a storage account. A user still cannot read its blobs. What explains why deployment success is insufficient?

Answer: B. B distinguishes management-plane creation from data-plane access. A is false because Bicep supports storage account resources. C invents a relationship between unrelated names. D grants excessive management access and does not establish the correct data-plane authorization.

14-Q6. An optional lock was created by an earlier incremental deployment. Its condition is now false. What cleanup assumption is unsafe?

Answer: D. D is unsafe because omission in an ordinary incremental deployment does not automatically remove the earlier resource. A describes a possible result. B correctly identifies authorization requirements. C is an appropriate precaution before cleanup.

14-Q7. An exported template has been decompiled into Bicep. What should happen before it becomes maintained deployment source?

Answer: A. A describes the needed review. B overstates what conversion proves. C can remove required configuration. D confuses resource configuration with workload data, which needs separate protection.

14-Q8. A deployment fails after successfully creating one of its resources. What is the best first response?

Answer: C. C uses the deployment record and actual state to diagnose the failure. A ignores partial success. B does not resolve a persistent error. D can create additional resources and obscure the original problem.

Chapter 15

15-Q1. Two deployments use the same mutable image tag a month apart. Why might they run different content?

Answer: A. A explains mutable tags. B is false because repositories contain versions. C invents automatic image modification. D confuses a content identifier with an arbitrary label.

15-Q2. A Container App deployment succeeds but its managed identity cannot pull a private image. Which check is most relevant?

Answer: C. C checks the actual runtime requirements. A does not change runtime registry authorization. B is unrelated to access. D is not needed for a container image pull.

15-Q3. An ACI task should stop after a successful exit and retry an unsuccessful exit. Which restart policy fits?

Answer: B. B matches failure-only restarts. A can rerun successful completed work. C does not automatically retry failures. D is not an ACI restart policy.

15-Q4. A Container App has healthy process logs but its HTTP endpoint fails after the ingress target port is changed. What is the best first check?

Answer: D. D addresses the change and symptom. A removes an unrelated dependency. B does not repair a port mismatch. C is excessive and unrelated to the application's listener.

15-Q5. A worker must scale from zero when queue messages arrive. Why is a CPU-only rule insufficient?

Answer: A. A explains the signal limitation. B is false. C is not a requirement for every background worker. D incorrectly links capacity with authorization.

15-Q6. A container writes completed business records only to its temporary writable filesystem. What is the main risk?

Answer: C. C follows from transient instance storage. A and B are unrelated. D is not inevitable; the application can run while using an unsafe persistence design.

15-Q7. A registry uses repository-level ABAC permissions. What should an administrator do before copying an older AcrPull example?

Answer: B. B respects the configured authorization model. A broadens access unnecessarily. C removes required controls. D confuses authentication with authorization to specific content.

15-Q8. Adding replicas raises database errors without improving throughput. What is the best next action?

Answer: D. D examines the likely bottleneck. A can worsen contention and costs. B removes evidence. C ignores the observed replica change and does not explain database errors.

Chapter 16

16-Q1. Three apps use the same dedicated App Service plan under the default shared placement model. What should be reviewed before increasing plan capacity?

Answer: C. C reflects shared compute and plan-level effects. A does not reveal capacity demand. B concerns name resolution rather than shared execution resources. D is unrelated unless a specific hosted workload depends on those images, and still would not replace the plan inventory.

16-Q2. A paid App Service app is stopped but its dedicated plan remains provisioned. What cost expectation is appropriate?

Answer: A. A correctly distinguishes the app's stopped state from provisioned plan capacity. B ignores other resources and pricing boundaries. C is not the effect of stopping an app. D invents a change to separate storage charges.

16-Q3. An app needs outbound access to a private database. Which feature directly addresses the App Service outbound virtual-network path?

Answer: D. D provides the relevant outbound path, subject to routing, DNS and authorization. A changes naming. B addresses inbound private access to the app. C changes release state and does not establish the required network path.

16-Q4. A custom domain resolves to the app, but the browser reports a hostname mismatch in the certificate. What is the most relevant check?

Answer: B. B addresses the TLS identity mismatch. A changes capacity without changing the certificate. C is unrelated to the hostname binding. D removes evidence and cannot repair TLS.

16-Q5. A staging connection string must remain associated with staging after a swap. What configuration is relevant?

Answer: C. C expresses the setting's environment-specific behavior. A does not control swapping. B changes capacity. D interferes with testing and does not define configuration movement.

16-Q6. After scaling out, web instances wait on a saturated database and errors increase. What should the administrator investigate?

Answer: A. A examines the observed dependency bottleneck. B and C are unrelated to the symptom. D can worsen contention and cost without increasing useful throughput.

16-Q7. A reverse slot swap restores older code after a failed release. Which statement remains true?

Answer: D. D identifies the boundary of a slot swap. A and B attribute effects outside the swap mechanism. C assumes application compatibility without checking the external state.

16-Q8. A private endpoint exists for an app, but a client resolves the hostname to a public path. What should be examined next?

Answer: B. B checks the path the client actually uses. A, C and D do not determine service name resolution or private endpoint routing.

Chapter 17

17-Q1. A five-minute CPU peak disappears when a chart uses a long-window average. What should the administrator do before concluding there was no pressure?

Answer: D. D examines the aggregation effect and operational consequence. A is unrelated and destructive. B is false because averaging can conceal brief peaks. C removes evidence rather than interpreting it.

17-Q2. A new Log Analytics workspace contains no storage request records. What is the most relevant initial check?

Answer: B. B checks the required collection path. A and C do not enable data collection. D confuses directory administration with the source and destination configuration and grants unnecessary privilege.

17-Q3. During maintenance, an alert processing rule suppresses the exercise action group. What result is possible?

Answer: A. A describes processing-rule action handling. B changes data retention, which is unrelated. C is not implied by notification suppression. D confuses suppression with disabling the alert rule.

17-Q4. A VM has host metrics but no guest performance data. What should be inspected?

Answer: C. C checks guest collection prerequisites. A, B and D do not explain whether guest telemetry was configured or delivered.

17-Q5. IP flow verify reports that a packet is allowed. Which conclusion is still unsupported?

Answer: D. D exceeds the tool's security-rule scope. A describes the observed result. B recognizes that packet fields matter. C correctly treats the result as one part of an end-to-end diagnosis.

17-Q6. A log query summarizes a count into one output row. The alert is configured to count output rows. What should be checked?

Answer: A. A addresses the mismatch between query shape and alert measurement. B discards the monitoring destination. C is false for the analysis query described. D has no relationship to the count interpretation.

17-Q7. A client fails intermittently when connecting to a specific dependency. Which tool is designed to collect ongoing evidence about that source-to-destination path?

Answer: C. C repeatedly tests a defined connection path. A controls naming rather than connectivity. B captures no ongoing path measurements. D changes an application release and does not provide continuous network evidence.

17-Q8. A request rejected before an ARM management operation begins is absent from Activity log. What is the correct interpretation?

Answer: B. B respects the source's documented boundary. A treats absent evidence as proof of no attempt. C generalizes one source to unrelated logs. D invents a retention cause without evidence.

Chapter 18

18-Q1. An administrator must protect an ordinary supported Azure VM with Azure Backup. Which vault type is used for that workflow?

Answer: B. B is the Azure VM backup vault for this workflow. A manages keys and secrets, not VM recovery points. C stores application artifacts. D stores telemetry and does not itself provide VM backup.

18-Q2. A backup runs frequently but retains points only briefly. A corruption problem is discovered after all clean points expire. Which policy dimension was insufficient?

Answer: D. D explains the missing older usable state. Frequency creates points but does not preserve them indefinitely. A, B and C do not determine how long recovery points remain available.

18-Q3. The latest backup contains the same corruption as the source. What is the best recovery-point decision?

Answer: A. A connects point selection to the incident timeline. B ignores known content. C removes potentially useful recovery options. D is too broad because an earlier retained clean state can address the problem.

18-Q4. A Site Recovery test failover will start a copy of a production VM. Which target choice best supports a controlled drill?

Answer: C. C reduces address conflicts and unintended production activity while supporting planned tests. A, B and D introduce unnecessary exposure or duplicate identity risks and do not establish a controlled recovery environment.

18-Q5. A failover succeeds and the recovered VM now serves users. What recovery-readiness step remains important?

Answer: B. B restores ongoing protection for the active workload. A overstates failover's effect. C can remove useful recovery information before a controlled return plan exists. D hides readiness problems.

18-Q6. A report lists successful backup jobs for every configured item. What additional check is needed to assess coverage?

Answer: D. D detects resources absent from the protected population. A and C are cosmetic. B is not a universal requirement; schedules should follow workload needs.

18-Q7. A practice vault cannot be deleted because protected data remains soft-deleted under enforced retention. What is the appropriate response?

Answer: A. A respects the protection boundary and provides an honest cleanup record. B is false while state remains. C undermines supported controls. D confuses source and backup lifecycles.

18-Q8. A restore job completes in ten minutes, but users regain service after an hour of network and application repair. Which duration best represents the observed full service recovery time?

Answer: C. C includes the operational work needed to restore useful service. A measures only one component. B ignores the outage. D describes historical point availability rather than recovery duration.

Subject Index

References identify numbered teaching sections, so they remain usable across print and digital layouts.

- **Access tiers:** 10.2
- **Action groups:** 17.3
- **Application security groups:** 7.1
- **ARM templates:** 14.1, 14.3, 14.4, 14.5
- **Availability sets:** 13.1
- **Availability zones:** 1.2, 13.2
- **AzCopy:** 9.5
- **Azure Backup:** 18.1, 18.2, 18.3
- **Azure Bastion:** 7.3
- **Azure CLI:** 1.5
- **Azure DNS:** 8.1, 8.2
- **Azure Files:** 11.1, 11.2, 11.3
- **Azure Monitor:** 17.1, 17.2, 17.3, 17.4
- **Azure Policy:** 5.1, 5.2, 5.3
- **Azure PowerShell:** 1.5
- **Bicep:** 14.2, 14.3, 14.4, 14.5
- **Blob recovery:** 10.4, 10.5, 19.5
- **Budgets:** 5.4
- **CIDR:** 6.1
- **Container Apps:** 15.4, 15.5
- **Container Instances:** 15.3, 15.5
- **Container Registry:** 15.2
- **Deployment slots:** 16.5
- **Diagnostic settings:** 17.2
- **Encryption at host:** 12.4
- **External users:** 2.5
- **Groups:** 2.3
- **KQL:** 17.2
- **Licenses:** 2.4
- **Load balancing:** 8.3, 8.4, 8.5
- **Locks:** 4.5
- **Managed identities:** 19.2, 19.3
- **Management groups:** 4.1
- **Microsoft Entra ID:** 2.1, 2.2, 2.3, 3.1
- **Network security groups:** 7.1, 7.2
- **Network Watcher:** 17.5

- **Private endpoints:** 7.5, 19.3
- **RBAC:** 3.3, 3.4, 3.5
- **Redundancy:** 9.2
- **Resource groups:** 1.3, 4.3
- **Routes:** 6.4
- **SAS:** 11.4
- **Scale sets:** 13.3, 13.4
- **Service endpoints:** 7.4
- **Site Recovery:** 18.4
- **Soft delete:** 10.5, 11.3
- **SSPR:** 3.2
- **Storage encryption:** 9.4
- **Storage Explorer:** 9.5
- **Subscriptions:** 4.2
- **Tags:** 4.4
- **TLS:** 16.3
- **Versioning:** 10.4, 19.5
- **Virtual machines:** 12.1, 12.2, 12.3, 12.4, 12.5
- **Virtual network peering:** 6.3

Additional Questions and Answers

These original short-answer questions extend the chapter reviews with practical diagnosis and reasoning. Each answer states what the evidence supports and which check should follow. They are learning exercises, not recalled certification-exam items. The chapter references identify the fuller explanations used for review.

Identity and Governance

Q1. A departed employee's direct Reader assignment is removed, but the account can still inspect a resource group. What should the administrator investigate?

The administrator should identify the actual signed-in principal and inspect every applicable access path at the resource group, including group membership and inherited assignments. Removing one direct grant does not remove another grant that authorizes the same operation. The review should locate the assignment's originating scope and its owner, then remove only the access that is no longer justified. A fresh session and a bounded read test verify the final state. References: Chapters 2–3.

Q2. A returning employee receives a newly created account with the same sign-in name as a deleted account. Why might the old application assignments no longer work?

The new account has a new directory object identity. Matching a sign-in or display name does not make old references point to that new object. The administrator should compare object IDs, distinguish supported restoration from recreation, and inventory the assignments that must be reestablished through approval. Reusing a name is not evidence that access transferred correctly. The verification should test the new principal rather than an administrator's broader session. References: Chapters 2–3.

Q3. A user loses a project membership but retains a product license. Does that prove group-based licensing failed?

No. Another licensed group or a direct assignment can still provide the entitlement. The administrator should inspect the product's assignment sources and enabled service plans, then confirm which source remains effective. Removing the project membership may have worked exactly as intended. Any cleanup should preserve unrelated approved entitlement and consider processing time. A product label alone does not explain every source of the license or the user's resulting application access. Reference: Chapter 2.

Q4. A user is eligible for password reset but has lost the only registered verification method. Why is an Azure role assignment the wrong repair?

Azure resource roles govern resource operations, not the user's ability to prove identity during password recovery. The account needs the approved recovery or method-replacement process, with proper identity verification. The administrator should inspect the applicable SSPR policy, registered methods, and password authority, then follow the support exception path. Assigning Contributor or Owner neither restores the missing method nor safely establishes the user's identity. The incident should preserve that separation. Reference: Chapter 3.

Q5. A resource moved into another group continues running, but an operations script cannot find it. Which evidence should be compared?

The administrator should compare the script's stored resource ID with the resource's current ID and scope. A move can change the management path while the service continues operating. The investigation should also review inherited access, monitoring references, and other dependencies affected by the destination group. Recreating a similarly named resource would obscure the original issue. Updating the correct reference and retesting the script provides a narrower, more defensible repair. Reference: Chapter 4.

Q6. A team assigns an audit policy and expects existing noncompliant resources to disappear. What misunderstanding should be corrected?

Audit records whether applicable resources meet the policy condition; it does not delete or repair those resources. The administrator should explain the definition's effect, assignment scope, and evaluation timing, then determine the approved response to the findings. Supported remediation requires an appropriate effect and configuration, while deletion is a separate lifecycle decision. The compliance report should identify the actual failed rule rather than implying that assignment automatically corrected the environment. Reference: Chapter 5.

Q7. A temporary policy exemption expires while the resource still violates the standard. What should the owner have planned beforehand?

The owner should have defined an exit condition and review date: correct the resource, obtain a justified renewed exception, or retire the affected workload. Expiration does not itself mean Azure deletes the resource. Normal policy evaluation and the assigned effect become relevant again for the applicable state and requests. The administrator should inspect the exact assignment and current compliance evidence, then follow the approved change process without broadening the exception unnecessarily. Reference: Chapter 5.

Q8. A budget alert arrives after a lab VM was stopped. What should be checked before concluding that the alert is incorrect?

The administrator should confirm the reporting scope, period, cost-data timing, and the VM's actual power state. Related resources such as disks, addresses, backups, or gateways can continue producing charges, and recent cleanup may not yet appear in cost data. The remaining inventory should be reconciled with the lab's expected resources. A budget is a notification mechanism, not an automatic shutdown or invoice cap, so cleanup still requires explicit verification. References: Chapters 5 and 12.

Storage

Q9. A storage account uses geographic redundancy, but a user accidentally overwrites an important blob. Why is redundancy alone an incomplete recovery answer?

Redundancy protects against defined infrastructure failures by maintaining copies of data; it does not inherently provide the historical application state needed after every mistaken write. The administrator should inspect available versions, snapshots, soft-delete protection, or backups and select a usable point before the overwrite. The exact recovery mechanism depends on what was enabled and retained. A resilience label should never be substituted for a tested procedure that restores the required content. References: Chapters 9–10.

Q10. An application needs a document immediately, but the only copy is in the archive tier. What planning error may have occurred?

Archive requires a supported rehydration process before ordinary access, so it does not match an assumption of immediate online retrieval. The administrator should examine the application's recovery and access-time requirements alongside storage and retrieval costs. A lower storage rate can be unsuitable when retrieval delay violates the workload's need. The repair should follow the supported rehydration path and then revise the tiering rule so future placement reflects actual access expectations. Reference: Chapter 10.

Q11. A lifecycle rule removes old previous versions while current blobs remain. Why should the rule still receive a recovery review?

Previous versions can be the historical states needed after an unwanted overwrite. Removing them changes recovery options even if every current object remains readable. The administrator should compare the rule's version scope, age conditions, and filters with the agreed retention requirement. Cost reduction is valid only when the remaining recovery coverage is acceptable. A test should verify which objects and versions qualify before the rule is expanded to important production data. Reference: Chapter 10.

Q12. New blobs appear at an object-replication destination, but older expected blobs do not. Which configuration should be inspected first?

The administrator should inspect the replication policy's source and destination container rules, filters, and eligibility conditions, including the treatment of existing data. A working flow for newly created objects does not prove that every historical object qualifies. Replication status and destination content should be checked for representative missing objects. Recreating the accounts would be premature and could destroy useful evidence. The intended historical population should be verified separately from ongoing replication. Reference: Chapter 9.

Q13. AzCopy can inspect a storage account but fails when downloading a protected blob with Entra authentication. What two independent areas need review?

The administrator should verify the caller's data-plane authorization for the blob and the actual network path to the storage service. Management-plane visibility does not establish blob-read permission, while a correct data role does not bypass a firewall or incorrect private DNS path. The account, tenant, target URL, authentication method, and error should be recorded. A successful test with an account key would not prove that the intended Entra authorization works. References: Chapters 9 and 11.

Q14. A shared access signature appears in a copied troubleshooting URL. What should the response consider besides deleting the visible message?

The signature is a credential, so removing one visible copy does not prove that it was never used or retained elsewhere. The administrator should identify its scope, permissions, expiry, and signing method, then use the supported revocation or containment approach for that SAS type. Stored access policies apply only to supported cases and should not be assumed universal. The investigation should also remove secret-bearing logging practices and replace the exposed delegation where needed. Reference: Chapter 11.

Q15. A user authenticates to an Azure file share but cannot open one folder. Why might adding another share-level role be insufficient?

Identity-based file access can involve both share-level authorization and file or directory permissions. Successful authentication and an appropriate share assignment do not automatically grant every operation on every path. The administrator should inspect the intended identity, share access, and the folder's applicable permissions, then compare a working path with the failing one. A broad role change can leave the actual directory restriction unchanged while granting unnecessary access elsewhere. Reference: Chapter 11.

Q16. A file was deleted inside an existing share. Why should the administrator distinguish share soft delete from a file snapshot?

Share soft delete addresses recovery of a deleted share, while a snapshot can provide an earlier state of file content within the share under supported conditions. The administrator should identify whether the lost object is the share itself or a file inside it, then inspect the available recovery points and retention. Choosing a feature based only on the words soft delete can lead to the wrong recovery procedure and an inaccurate promise about recoverability. Reference: Chapter 11.

Compute

Q17. A VM deployment succeeded and its power state is Running, but the application is unavailable. What has actually been proven?

The evidence establishes that the management operation completed and the VM is running in the reported state. It does not prove the guest service is healthy, listening on the expected address, reachable through the network, or authorized to access its dependencies. The administrator should inspect guest and application evidence, test the required flow, and separate startup configuration from resource provisioning. Reporting the exact verified layer avoids treating a deployment status as application acceptance. Reference: Chapter 12.

Q18. A VM is deallocated after a training session. Why can the subscription still incur charges for that exercise?

Deallocation changes the VM's compute allocation, but separately retained resources can remain billable. Managed disks, public IP resources under their applicable charging model, backup data, and other supporting services need their own lifecycle review. The administrator should inventory the lab group and any explicitly created external dependencies, then remove only the resources no longer required. Cost data can lag cleanup, so both resource state and later cost evidence are useful. References: Chapters 5 and 12.

Q19. An application stores its only database copy on a VM's temporary disk. What must change in the design?

Temporary storage should not hold the only durable copy of required data. The administrator should move persistent application state to a supported durable storage design and implement appropriate backup and recovery. The workload must also tolerate the documented lifecycle behavior of temporary storage. A VM remaining healthy today does not prove the temporary contents will survive future host or maintenance events. Verification should include a recovery test, not merely a successful file copy. Reference: Chapter 12.

Q20. Two application VMs occupy different availability zones, but both depend on one unprotected database. What availability claim would be misleading?

It would be misleading to claim that the complete application is resilient to a zone failure solely because its compute instances are distributed. The shared database can remain a single dependency whose failure prevents both instances from serving useful work. The administrator should map the full request path, data availability, traffic distribution, and recovery requirements. Zone placement contributes failure isolation, but application resilience requires compatible behavior across all critical components. Reference: Chapter 13.

Q21. A worker service has a growing queue while average CPU stays low. Why might a CPU-only autoscale rule miss the problem?

CPU may not represent the workload's limiting demand or backlog. The worker could be waiting on an external dependency, processing unevenly, or needing a queue-based scaling signal. The administrator should inspect queue depth, processing rate, dependency latency, and the configured minimum and maximum capacity. Increasing instances without understanding the constraint can add cost without improving throughput. The scaling signal should correspond to the service objective and be tested under representative demand. References: Chapters 13 and 15.

Q22. A Bicep deployment passes compilation, but what-if reports an unresolved expression for a critical resource. What should happen before deployment?

The unresolved result should remain an explicit uncertainty. Compilation proves a narrower property than the final resource change, and what-if can have analysis limitations. The administrator should inspect parameters, references, and the critical resource's proposed configuration, obtaining additional evidence where needed. An unclear preview should not be reported as proof that the resource is unchanged. The deployment record should distinguish local checks, preview findings, actual deployment, and operational verification. Reference: Chapter 14.

Q23. A container image pulls successfully, but the container repeatedly exits. Which investigation is more relevant than changing registry permissions?

The successful pull points beyond registry retrieval toward startup behavior. The administrator should inspect the exact image reference, command, environment variables, secret references, exit code, logs, and resource limits. A missing required setting or application failure can stop the process even though the image is available. Repeatedly broadening registry access does not repair that stage. The correction should target the observed startup cause and be verified with the intended configuration. Reference: Chapter 15.

Q24. A deployment-slot swap succeeds, but the new release has made an incompatible database change. Why is swapping back not a complete recovery plan?

A slot swap changes application routing and supported configuration behavior; it does not automatically reverse external database mutations. The older application version may no longer be compatible with the changed data schema. The release plan should therefore address database compatibility, migration sequencing, backups, and the supported rollback or forward-fix path. The administrator should validate the complete workload after a swap rather than treating the slot operation's success as proof of safe recovery. Reference: Chapter 16.

Virtual Networking

Q25. Two teams independently choose 10.80.0.0/16 for networks they later want to peer. Why can different VNet names not resolve the conflict?

Routing uses destination addresses and prefixes, not friendly resource names, to identify the intended network path. Identical address spaces make those destinations ambiguous for ordinary peering. The administrator should reconcile the organization-wide address plan and choose a supported redesign before connecting the networks. Renaming one VNet changes a label, not its address coverage. Future connected environments should be included in allocation review so local deployment success does not conceal a later integration problem. Reference: Chapter 6.

Q26. A default route sends traffic toward a firewall, but a destination follows a more specific route. Is that automatically an Azure fault?

No. Ordinary route selection favors the longest applicable matching prefix, so a more specific route can take precedence over a default route. Azure also has documented preferred-route cases that need service-specific review. The administrator should inspect the source interface's effective routes, matching destination prefix, route source, and selected next hop. A default route should not be described as proof that every packet traverses the firewall without examining those details. Reference: Chapter 6.

Q27. A hub is peered to two spokes, and each spoke can reach the hub. Why might the spokes still be unable to reach each other?

Ordinary peering is not automatically transitive. The two successful spoke-to-hub connections do not establish a forwarding service between spokes. The administrator should identify the intended transit mechanism, required routes, forwarding configuration, peering settings, and return path. Enabling one forwarded-traffic option alone does not turn the hub into a router. The design must provide a supported complete path, and the application flow must then satisfy its security and listener requirements. Reference: Chapter 6.

Q28. An administrator adds a deny rule, but an already open session continues working. What should be tested before declaring the rule ineffective?

NSGs maintain connection state, so an established flow can behave differently from a new connection after a rule change. The administrator should close the test session, establish a fresh flow with the exact intended source, destination, protocol, and ports, and inspect the effective rules. The test should also confirm the NSG association and priority. Reusing a long-lived session is insufficient evidence that the current rule set permits newly initiated traffic. Reference: Chapter 7.

Q29. A private VM accepts a Bastion session but cannot download an update from a public repository. What does the successful session fail to prove?

It proves only the administrative path through Bastion to the VM, including the relevant guest authentication. It does not establish a public outbound path from the VM. The administrator should inspect the subnet's actual outbound configuration and a supported explicit egress method, along with DNS and filtering. Modern private-subnet defaults make implicit internet access an unsafe assumption. Bastion is not a replacement for the workload's separate outbound connectivity design. References: Chapters 6–7.

Q30. A client using a service endpoint still resolves the storage hostname to a public address. Why can that be expected behavior?

A service endpoint does not create a private IP interface for the target service. It establishes supported subnet-aware connectivity to the service's public endpoint, with the resource's network rules controlling allowed sources. Public name resolution is therefore not evidence by itself that the service endpoint is missing. The administrator should verify the subnet endpoint, target network rule, route behavior, and data authorization. A private endpoint has a different addressing and DNS model. Reference: Chapter 7.

Q31. A storage blob private endpoint works, but an application using the file service fails. Which assumption should be challenged?

The administrator should challenge the assumption that one private endpoint covers every storage service surface. Blob and file access can require separate subresource connections and corresponding DNS relationships. The actual hostname and service used by the failing application should be identified, then compared with the endpoint target, approval state, and private DNS configuration. A working blob test proves that blob path only; it does not validate the file-service path or its authorization. Reference: Chapter 7.

Q32. One VM resolves a service hostname privately while another resolves it publicly. What evidence best guides the investigation?

The administrator should compare the clients' configured DNS servers, virtual-network links, forwarding path, relevant records, and cache state. The private endpoint's existence does not guarantee that every client uses a resolver able to see its private zone. The investigation should capture the actual answers from both sources and the hostname the application uses. Changing Azure roles cannot correct a resolver selecting the wrong destination address for the intended private connection. References: Chapters 7–8.

Monitoring and Recovery

Q33. Two dashboards show different CPU values for the same VM at the same apparent time. What should be compared before assuming one is wrong?

The administrator should compare the metric name, time range, time grain, aggregation, dimensions, and resource scope. An average over a longer interval can differ substantially from a short maximum without either result being incorrect. The source and refresh timing also matter. A meaningful comparison uses equivalent definitions and windows. The operational question then determines which view is appropriate, such as sustained load, brief peaks, or a particular instance's behavior. Reference: Chapter 17.

Q34. Diagnostic settings were enabled today, but a log query asks for an event from last week. Why might an empty result be expected?

The setting routes selected supported data from its configured source to a destination; it does not generally recreate historical events that were never exported there. The administrator should confirm when collection began, which categories were enabled, the destination workspace, table, time filter, and ingestion status. An empty query is not proof that the event never occurred. The investigation may need another retained source that actually collected the relevant period. Reference: Chapter 17.

Q35. An alert instance exists, but the expected operator received no notification. Which part of the design should be inspected separately?

The administrator should inspect the action group or supported notification route, recipient details, delivery evidence, and applicable alert processing rules. Alert detection and notification are separate stages. A correctly evaluated condition can still fail to reach the responsible person because routing is wrong or actions are suppressed. The record should distinguish the alert's creation time from delivery attempts and verify the intended recipient through an approved test rather than changing the threshold blindly. Reference: Chapter 17.

Q36. A maintenance window suppresses alert notifications. Why should the administrator avoid saying that monitoring has stopped?

Suppressing actions through an alert processing arrangement is different from disabling data collection or changing every alert condition. Metrics and logs can continue to be collected, and alert evaluation can still produce evidence according to the configuration. The administrator should state exactly which actions are suppressed, for which scope and period. After maintenance, the notification path should be verified so a temporary suppression does not become an unnoticed permanent loss of operational response. Reference: Chapter 17.

Q37. A backup job reports success. What additional evidence is needed before claiming the application is recoverable?

The administrator should verify that the intended workload and data are included, that usable recovery points exist within the required retention, and that a restore can produce an operationally valid result. A controlled restore should check files or application data, permissions, dependencies, and the service's acceptance criteria. Job success is valuable evidence of one operation, but it is narrower than a tested recovery path. The record should state the actual scope exercised. Reference: Chapter 18.

Q38. A recovery point is recent, but restoring useful service takes too long. Which recovery objective is most directly at risk?

The recovery time objective is at risk because it concerns the time required to restore usable service. A recent point can support the recovery point objective, which concerns acceptable lost recent data, without guaranteeing a fast restoration. The administrator should measure decision time, resource preparation, restore execution, dependency repair, and application verification. Those stages can dominate the total outage even when the underlying data is sufficiently recent for the workload's requirement. Reference: Chapter 18.

Q39. A production VM is healthy while Site Recovery reports increasing replication lag. Why does the issue still require attention?

Current application availability and recovery readiness are different objectives. The source can serve users while the replica becomes too old to meet the required recovery point objective. The administrator should inspect replication health, connectivity, supporting components, and recent source changes, then verify recovery progress after correction. Waiting for a production outage would remove the opportunity to repair the protection path under normal conditions. A healthy source is not proof of healthy disaster recovery. Reference: Chapter 18.

Q40. A regional failover succeeds and users can work in the recovery region. What protection task can still remain unfinished?

The workload may need reprotection to establish replication in the intended direction from its now-active location. A successful failover job does not automatically prove that ongoing recovery protection or a later return path is ready. The administrator should inspect the active data location, replication direction, target resources, health, and documented failback plan. The incident's final state should include both usable service and the agreed recovery readiness, with any temporary gap assigned an owner. Reference: Chapter 18.